Verification Continuum™

Verdi® User Guide and Tutorial

Version X-2025.06-SP1, September 2025

SYNOPSYS®

Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Feedback

Contents

Audience 21
Customer Support 21
Synopsys Statement on Inclusivity and Diversity 22

1. **Introduction to Verdi** 23
Overview 23
Technology Overview 23
Compilers, Interfaces and Interoperability 24
Databases 25
Analysis Engines 25
Graphical User Interface 25
Visualization 26

2. **User Interface Conventions** 27
Overview 27
Common User Interface Features 29
Frame Banner 30
Pull-Down Menu Commands 31
Mnemonic Keys 32
BindKeys 32
Toolbars 32
Mouse Operation 32
Right Mouse Button Menus 33
Undock/Dock 33
Online Help 38
Feedback Button 38
Feedback Email Format 41
Limitation 41
Unified Tooltip Viewer 41
Limitation 44
Next Generation Debug GUI 44
Color Themes 45
Grid and Density 46

Banners 48
Find 50
Search 51
Quick Launcher 54
Recently Used Command Buttons 55
Toolbar With Text and Icon 56
nTrace User Interface 57
nTrace Design Browser Pane 58
nTrace Source Code Pane 59
Source Code Area 59
Indicator Area 60
nTrace Message Pane 60
nTrace Toolbar Icons 61
nWave User Interface 61
nWave Signal Pane 63
Signal Name 63
Signal Cursor 63
Group Name 63
nWave Value Pane 63
nWave Waveform Pane 64
Cursor 64
Marker 64
Zoom-Scale Ruler 65
Full-Scale Ruler 65
nWave Toolbar Icons 65
Get Signals 65
nWave Mouse Operations 67
nSchema User Interface 69
nSchema Toolbar Icons 71
nSchema Mouse Operations 71
nState User Interface 73
nState Toolbar Icons 74
nState Mouse Operations 74
Flow View User Interface 75
Flow View Toolbar Icons 76
Flow View Mouse Operations 77
Transaction/Message User Interface 78
Detailed Transaction/Message View in nWave 78
Transaction/Message Properties 80

Transaction/Message Attributes 81
Analyzing Transactions/Messages 82
nCompare User Interface 83
Comparing Different Simulation Runs 84
Rule File 84
Compare Waveforms and View Errors 85
nCompare Mouse Operations 85
nECO User Interface 85
nAnalyzer User Interface 85

3. Before You Begin 86
Installation and Setup 86
Demo Details 87

4. Launching Techniques 88
Dumping Elaboration Database 88
Overview 88
Use Model 89
Compilation Stage 90
Using Multiple -kdb Switches 92
Suppressing Inline PSL Errors During VHDL Compilation 92
Optimizing Disk Space Usage in KDB Files 93
Invoking Verdi 94
Invoking Verdi in the simv Flow 95
Runtime Upward of Product License Mode 95
Use Model 97
Limitations 98
Loading Designs With Different Strategies 99
Using Verdi Executable to Import Design from UFE 101
Preload and Smart-load 101
Preload 101
Smart Load 103
Reference Source Files on the Command Line 104
Compile Source Code into a Library 104
Reference Design and FSDB on the Command Line 104

Perform Behavior Analysis on the Command Line 105
Replay a File 106
Start Verdi Without Specifying Any Source Files 106
Loading When Design and FSDB Hierarchies do not Match 108
Replacing the Absolute Paths in Verdi KDB 109
Option to Retain the Paths Specified Inside the KDB 109
Option to Specify Absolute Path to the Compile Directory 109
Limitations 110
Support of Incremental Dump in vlogan Without PSL 110
Getting Relative Path in FSDB 110
Required Conditions 111
Limitations 111
-kdb=common_elab Supports VCS Dynamic Test Loading 111
Use Model 112
Verdi Support for Dynamic Test Loading 113
Limitations 113

5. User Interface Tutorial 115
Overview 115
Start Verdi Platform 116
Using the Welcome Page 116
Saving and Restoring a Session 117
Changing the Default Frame Location 119
Maximizing the Display 119
Modifying the Menu/Toolbar 120
Searching for a Command 121
Customizing BindKeys 121
Customizing Toolbar Icons 122
Using Unified Find 124
Support for Active Window Icon With the Spotlight Icon 125
Enhancement to the Search Option in the Message Frame 126
Enhancement to the Search Option in the Transaction Table View, Transaction Relation Navigator Window, and SmartSheet 127
Enhancement to the Spotlight Icon 127
Enhancements to the Right-Click Options 130

Enhancements to the Temporal Flow View Tools Menu 131
Enhancements to the Preferences Form 132
Docking and Framework Page 132
Find/Filter Page 133
Miscellaneous Page 134
Limitations 135
Support for Active Window in Verdi 136
Scenarios of Active Window 137
Limitations 138
Next Generation Debug GUI 138
Color Themes 138
Limitations 139
Grid and Density 140
Banners 142
Find 144
Search 145
Quick Launcher 148
Recently Used Command Buttons 149
Toolbar With Text and Icon 150
Verdi Integration With Euclide IDE 151

6. nTrace Tutorial 154
Overview 154
Traverse the Design Hierarchy in nTrace 154
Access a Block's Source Code 156
Goto Line 156
Use Model 157
Find Scope 158
Variable Inside Generate Block From nTrace to nWave 159
Support for Multiple Range Selection in the Source View 160
Trace Drivers and Loads 164
Find String 164
Trace Driver 165
Add Bookmarks 167
Trace Load 168
Trace Connectivity 168
Save Trace Result and Reset History 169

Edit Source Code 170
Use Active Annotation 170
Tracing the Active Driver 172
Fault Source Code 174
Use Model 174
Limitations 175
Support for Specify Array Index 175
Basic Operation Flow 176
Load FSDB and show Active Annotation 176
Operate on Source Panel 176
Miscellaneous Usage 180
Example 1 180
Example 2 185
Example 3 185
Example 4 186
Example 5 187

7. nSchema Tutorial 188
Overview 188
Start nSchema 189
Manipulate the Schematic View 191
Change the Schematic View Among Instances 193
Enable Viewing Objects 195
Find an Instance or a Signal in a Schematic 196
Change the Color of the Selected Signal 198
Trace Signals 198
Find the Drivers of a Signal 198
Find the Load of a Signal 199
Find the Connectivity of a Signal and Generate a New Schematic From Trace Results 200
Show RTL Block Diagram in a More Meaningful Way 200
Generate Partial Schematics 202
Hierarchical 202
Flattened Window 204
Fan-in and Fan-out 205
Trace Between Two Points 207
Use Active Annotation to Show Signal Values 209

Change the Color or the Line Style for Annotations 210
Support for Annotation and Fault Coverage on nSchema Window 210
Example 1: 210
Example 2: 211
Example 3: 211
Show Tip 212
Fault Status Bar 212
New Tcl Command Options 213
schSetOptions 213
Support for GM/FM Waveform 214
Example1: 215
Example2: 215
Example3 216
Example4: 216
Example5: 217
Example6 218
Example7 218

8. nWave Tutorial 220
Add Signals 221
Add Signals From Other Windows 221
Use Get Signals to Add Signals 222
Search for Signals to Add 223
Creating a Parent Group With Sub-Groups 224
Manipulate the Waveform View 226
Set the Cursor/Marker Positions 227
Zoom Cursor With Three Clicks 228
Fast Zoom on the Full Scale Ruler 228
Pan the Waveform 228
Use BindKey Commands 229
Turn On/Off Signal Grids 229
Add Marker Labels 230
Change the Display Sequence of Signals 231
Search for Signal Value Transitions 232
Add Comments 233
Compress Time Ranges 234
Split the Waveform View 234
Change Signal/Group Attributes 235

Search for a Group 235
Change the Group Name 235
Modify the Display Format in the Value Window 236
Display Hierarchical Signal Names 236
Add Alias to Display Bus Values 237
Change the Spacing and Signal Height 238
Change Signal Color/Pattern 239
Display Jitter Injection in Waveform Pane 239
Create New Signals/Buses From Existing Signals 240
Logical Operations 241
Bus Creation 242
Expand or Collapse the Bus 243
Save and Restore Signals 244
Save the Displayed Signals 244
Restore Previously Saved Signals 244
Create a Second Waveform Window and Restore Other Signals 244
Group or Ungroup Multiple FSDB Files 245
Calculate Toggle Coverage 249
Define Events and Complex Events 252
Create a Single Event 252
Save and Reload Events 254
Create a Complex Event 254
Create a Complex Event With a Timer 255
Example 1 256
Example 2 257
Example 3 258
Create Complex Event with a Counter 259
Example 1 259
Example 2 260
Support VHDL Fixed-point Types in nWave 261
Display Fixed-point Value 261
Convert Radix 261
Leading Zero 262
Convert Notation 262
Enable Set Radix Entry for Single-bit VHDL Fixed Point Signal 263
FSDB Utility Support for VHDL Fixed-point Type 263
Limitations 264

Change the Binary Point for VHDL Fixed-point Signal and Verilog Vector Signal . . . 265
Capture and Display Frequency of a Scalar Signal as a New Signal . . . 267
Generate Dump Files From Verdi Signal RC Files . . . 270

9. nState Tutorial . . . 271
Overview . . . 271
Starting nState in nSchema Window . . . 272
Manipulate the State Diagram View . . . 273
Enable Viewing Objects . . . 274
Find the Start and End States of a Transition . . . 275
Create a Partial Finite State Machine Frame . . . 276
State Animation . . . 277
State Machine Analysis . . . 281

10. SmartLog Tutorial . . . 282
Overview . . . 282
Invoking SmartLog . . . 284
Auto Select Rules . . . 288
Checking Mapping Rule File . . . 288
Predicting Mapping Rule . . . 289
Auto Select Rule Mode . . . 289
Displaying ANSI Colors in SmartLog . . . 290
Opening a Log File Using Verdi Commands . . . 292
Examples . . . 293
Navigating SmartLog . . . 293
Shortcut Keys for SmartLog . . . 294
Wrapped Text . . . 294
Browse Views . . . 296
File View . . . 296
Structure View . . . 296
User Specified Columns . . . 299
Rearranging the Columns . . . 300
Restoring Previous Column Display Settings . . . 300
Specifying Time Unit in UVM/OVM Log File . . . 300
Specifying the Log Time Unit . . . 301

Specifying the Display Time Unit . . . 301
Using Hyperlink Rule File . . . 301
Timing Violation Debug . . . 304
Configuring a New Partitioning Rule . . . 307
Example to Demonstrate Creating a Customized Partition Rule File . . . 309
Creating Log Time Unit Rule . . . 313
Backward Compatibility for Customized Partitioning Rule File Created Using Tcl Commands . . . 316
Applying Partitioning Rule . . . 317
Opening Multiple SmartLog Windows and Synchronizing With nWave . . . 318
Locating Objects . . . 321
Locating a Specified Line Number . . . 321
Locating a Specified Time Point . . . 322
Setting the Scroll Offset . . . 322
Locating the Specified Line Number With Scroll Offset . . . 323
Locating the Specified Time Point With Scroll Offset . . . 323
Synchronizing the Cursor in the nWave Pane . . . 324
Locating the Previous/Next Point . . . 325
Searching, Filtering, and Reloading the Log File . . . 326
Setting the Message Filter . . . 326
Setting the Time Filter . . . 328
Searching . . . 329
Example Use Cases . . . 330
Displaying the Search Result Section . . . 331
Reloading the Log File . . . 332
Setting the Column Filter . . . 332
Saving or Loading a Filter Result . . . 333
Debugging in Verdi Frames . . . 333
Using SmartLog in Interactive Debug . . . 336
Invoking Interactive Debug . . . 336
Command Entry . . . 338
Scroll Bar Behavior . . . 339
Applying User-Defined Partitioning Rules . . . 339
Example . . . 340
Known Issues and Limitations . . . 340

11. **OneSearch Tutorial** . . . 342
Overview . . . 342
GUI Use Model . . . 343
Search Domains . . . 345
Search Modes . . . 347
Overview Mode . . . 347
Single Domain Mode . . . 347
Support for Multiple Line Results . . . 348
Overview Mode . . . 348
Single-Domain Mode . . . 349
Support for Synonyms . . . 349
Example Use Case . . . 350
Specifying an Additional Synonym Database . . . 350
Limitations . . . 351
Command Line Use Model . . . 351
Usage Examples . . . 353
Find Bar . . . 353

12. **Temporal Flow View Tutorial** . . . 357
Overview . . . 357
Behavior Analysis Engine . . . 359
Automatic Cause and Effect Tracing . . . 359
Trace to Root Cause Automatically . . . 361
From nTrace . . . 361
From nWave . . . 362
Open Temporal Flow View . . . 362
Setup Temporal Flow View . . . 362
Create Temporal Flow View . . . 363
Temporal Flow View Features to Trace Automatically . . . 364
Trace This Value . . . 364
Show Marker for Control With Transition . . . 365
Correlate Other Panes . . . 366
Display Source Code Automatically . . . 366
Show Traced Signals in nWave . . . 367
Show Fan-in Signals in nSchema . . . 367
Temporal Flow View Application . . . 368

Trace Unknown Values 368
Summary of Steps to Trace Unknown Values 371
Debug Memory 371
Open a Temporal Flow View 374
Manipulate the View 376
Display More Information 377
Show Active Statements 378
Display Source Code 379
Add Signals From the Temporal Flow View to nWave 381
Specify Node Color for Compact and Register Tutorial 382

13. Debug a Design With Simulation Results Tutorial 384
Find the Active Driver 384
Generate Fan-in 386
Debug Memory Content 389

14. nCompare Tutorial 392
Overview 392
Start nCompare and Compare Waveforms 392
Viewing Errors 395
Sorted by Time 395
Sorted by Hierarchy 395
In the Waveform 396
Error Report File 397
Save the Current Error File 397
Load the Previous Error File 397
Support Multi-process With CDPL 398
Support Merge Error Report and Compare Log 398
Limitation 399
Example 399
Support nCompare With CDPL 400
Integrate normal nCompare multi-process into CDPL 400
Enable Runtime Scheduling 401
Set Computing Farm of CDPL With Command Line Option 401
Auto Set all Environment Variables Associated With CDPL 401

15. Application Tutorials 403

Searching Backward for Value Causes 403
Open a Temporal Flow View 404
Show Active Statements 405
Trace This Value Automatically 406
Trace Another Path 409
Show Signals on *nWave* 410

Debug Memories 410
Debug Synthesizeable Memory Models 411
Locate the Cause of a Value on a Signal 411
Locate the Last Write of a Specific Address Location 412
Show Memory Contents 413
Search Values in the nMemory Frame 415
Synchronize the nMemory Frame With nWave 415
Change Address and Time in the nMemory Frame 416
Customize the nMemory Window 417
Display Calculated Memory Contents in nWave 418
Debug Non-synthesizeable Memory Models 419
1-port Static RAM 419
Multiple-port Static RAM 424
Debug PLI Memory Models 426
Create a PLI Memory Definition File 426
Load the PLI Memory Definition File 428
Trace the Content of the PLI Memory 428

Debug Gate vs. RTL Simulation Mismatch 430
Locate the Signal to Compare 430
Load Simulation Results and Display Waveforms 432
Compare the Simulation Results 434
Isolate the Problem 435

Behavior Trace for Root Cause of Simulation Mismatches 436
Locate the Simulation Mismatch 437
Behavior Trace for the Root Cause of Mismatch 439

Debug Unknown (X) Values 441
Locate the Root Cause of the “X” Value on ZFout 442
Visualize the Active Paths in the Temporal Flow View 446

Debug Forced Signals 448
Enable Force Debug 449
Debug in Source Code 450

Trace Signal . . . 450
Active Annotation . . . 451
Debug in the Waveform View . . . 452
Icons in the Waveform Pane . . . 452
Show Tips . . . 453
Search by Force/Deposit/Release . . . 453
Debug in the Schematic View . . . 454
Debug in the Temporal Flow View . . . 454
Practice . . . 455
Debug With SystemVerilog . . . 458
Import the Design . . . 459
Load Files Directly . . . 459
Use Compiled Library - Optional . . . 460
Visualize SystemVerilog Source Code . . . 460
Design Browser Frame . . . 461
Source Code . . . 462
Schematic . . . 463
View SystemVerilog Simulation Results . . . 465
Waveform . . . 466
Source Code . . . 467
Generate Constructs . . . 468
Debug With SystemVerilog Assertions . . . 469
Import the Design . . . 470
Load Files Directly . . . 470
Use Compiled Library - Optional . . . 471
Visualize SVA Source Code . . . 472
Design Browser and Source Code . . . 472
View SVA Simulation Results . . . 473
Statistics Frame . . . 474
Waveform . . . 476
Source Code . . . 480
Analyze SVA Assertions . . . 481
Evaluate SVA Assertions . . . 483
SVA Evaluation of Runtime Assertions . . . 484
Import Designs and Assertions . . . 484
Evaluate With Temporary Assertions . . . 484
Save Temporary Assertions . . . 488
Load Temporary Assertions . . . 489
Note the following: . . . 489
Debug With Transactors . . . 490

What is a Transaction? 490
Generating Transaction Data 491
Provided FSDB Dumpers 491
Transaction IP Partners 491
SVA Extraction 491
FSDB Writer API and the Open Transaction Interface (OTI) 492
SVTB Automatic Logging of OVM/UVM Component and Port Transactions 492
View Transactions in nWave 492
View Transactions in Transaction Table View Window 495
Add/Remove Transaction Streams 495
Merge Transaction Streams 497
Manipulate the Stream View 498
Generate Statistics 505

16. PPA Information in Verdi 508
Loading Information in Verdi 508
Instance View 510
Selecting Table Types And Scenarios 511
Tree View And Functions 511
Hierarchical Header 512
Columns 512
Toolbar 513
Search 513
Visualizations 514
Freeze First Column 519
Column Configuration 519
Units 522
Decimal Points 523
Filter Columns 523
Switching to Module/Instance View 525
Cross-Probe to Source 526
Expanding/Collapsing Tree by Level 526
Configuring Columns 527
Configuring Root 527
Configuring Color Scheme 528
Canvas View 530
User Interface 530
Tcl Command 530
nTrace Hierarchy 532

Synthesis Summary View 533
Canvas View Window 534
Color Code GUI 535
Top Grids 542
Construct Boundary 545
Clear Boundary 546

17. Simprofile 548
Loading the Simprofile Database (W-2024.09 and Earlier Releases) 549
Loading the Simprofile Database (X-2025.06 Release Onward) 549
Opening profileView 550
profileView Window Panes 551
Summary Pane 552
Filter Pane 553
Top N Usage Pane 554
Detail Pane 556
Module Tab 557
Instance Tab 557
Construct Tab 558
Caller Callee Tab 558
File Tab 559
Summary Tab 559
PLI/DPI/DirectC Tab 560
Menus and Commands of the profileView Window 561
Toolbar Commands 561
Menu Bar Commands 562
RMB Context Menus 562
Source View Indicator 563
Limitations of the Source View Indicator 565
SVA Distribution Mode 565
Enabling the SVA Distribution Mode 565
Switching Between SVA and Non-SVA Distribution Mode 565
Merged Simprofile Report 567
Merged Simprofile Report Interface in Verdi 568
DUT/TB Proportion Display 570
UI Behavior in Verdi 570
Limitations 571

A. Appendix A: Supported Waveform Formats 573

Overview 573

Fast Fourier Transformers (FFT) 573

Getting Data From Analog Signal 573

Data Manipulation in FFT Window 575

EVCD 577

Analog Waveform Example 580

View the Analog Waveform 580

Manipulate the Analog Waveform 581

Change the Signal Height 581

Display the Analog Ruler 582

View Different Simulation Results in the Same Window 582

Overlap Analog Signals From Different Simulation Results 583

B. Appendix B: Supported FSM Coding Styles 584

Overview 584

One-Process Always 585

Example 1 - Verilog (one_process.v) 585

Example 2 - VHDL (one_process.vhd) 586

Two-Process Always 588

Example 1 - Verilog (two_process.v) 589

Example 2 - VHDL (two_process.vhd) 592

One-Hot Encoding 594

Shift Arithmetic Operation 597

Case-Statement vs. If-Statement 598

Example 1 598

Example 2 600

Gate-Like FSM 602

Next_State = signal 603

Next_State = Current_State + N 605

VHDL Record Type 606

C. Appendix C: Enhanced RTL Extraction 609

Overview 609

Instance Array 611
For Loop 611
Aggregate 612
Partial Bits Assignment 616
Displaying Pure Memory Blocks 618

D. Appendix D: Additional Transaction Example 620
Extracting Transactions Using SVA 620
SVA Code 621
Recommended Coding Style 622
Unsupported Coding Style 623
Code Example 624

E. Appendix E: Integration Features 626
Native Integration of Verdi and VCS 626
UCLI Dump Command for FSDB Dumping 626
Introduction 626
Use Model 627
Enhanced and New UCLI Dump Options 627
Limitation 627
Optimized Performance of Gate-Level Designs Using Native FSDB Gate 628
Introduction 628
Prerequisites 628
Using the FSDB Gate Feature 629
Limitations 629
Unified Transaction Debug - Verdi and Protocol Analyzer Integration 630
Introduction 630
Use Model 631
Unified UVM Library 631
Use Model 631
Scope-Based Peak Analysis 631
Use Model 632
Generating Waveform With Report Entry 635
Exporting the What-if Configuration File 636

Preface

This book is designed to allow you to quickly become proficient in the Verdi platform. This manual focuses on the most commonly used commands without going into detail on everything. For detailed descriptions of individual commands, refer to the appropriate chapter of the *Verdi and Siloti Command Reference Manual*.

The manual should be read from beginning to end, although you may skip any sections with which you are already familiar.

- If you are new to the Verdi platform, begin with the User Interface, *Launching Techniques* and various *Tutorials* chapters. After you are familiar with the individual modules, review the *Application Tutorials* chapter.
- If you are familiar with the Verdi platform but want to learn new ways to apply it, review the *Application Tutorials* chapter.

Audience

The audience for this manual includes engineers who are familiar with languages and tools used in design and verification such as Verilog, VHDL, SystemVerilog, simulators, timing analyzers, and transactions. The application of these languages may be for System-on-Chip (SoC), Application Specific Integrated Circuit (ASIC), and Field Programmable Gate Array (FPGA) designs.

This document assumes that you have a basic knowledge of the platform on which your version of Verdi platform runs: Unix or Linux, and that you are knowledgeable in design and verification languages, simulation software, and digital logic design.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

https://solvnetplus.synopsys.com

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to https://solvnetplus.synopsys.com.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to Verdi

This chapter describes the following topics:

- Overview
- Technology Overview

Overview

The Verdi® Automated Debug Platform is a sophisticated tool designed to enhance the debugging of digital designs, thereby boosting productivity for complex System-on-Chip (SoC), ASIC, and FPGA projects. Unlike traditional debug tools that depend solely on structural information and the engineer's ability to deduce design behavior from its structure, the Verdi platform offers advanced technology to help users understand intricate and unfamiliar design behaviors. It automates challenging and monotonous debugging tasks, integrates various complex design environments, and deduces the dynamic behavior of a design over time.

The Verdi platform offers a comprehensive suite of features beyond the basics of a source code browser, schematics, waveforms, state machine diagrams, and waveform comparison for FSDB (Fast Signal Database) format simulation results. It includes advanced capabilities such as automatic signal activity tracing with temporal flow views, assertion-based debugging, power aware debugging, and the analysis of transaction and message data. These features are accessible through a customizable graphical user interface built on the Qt platform, which supports multi-window docking.

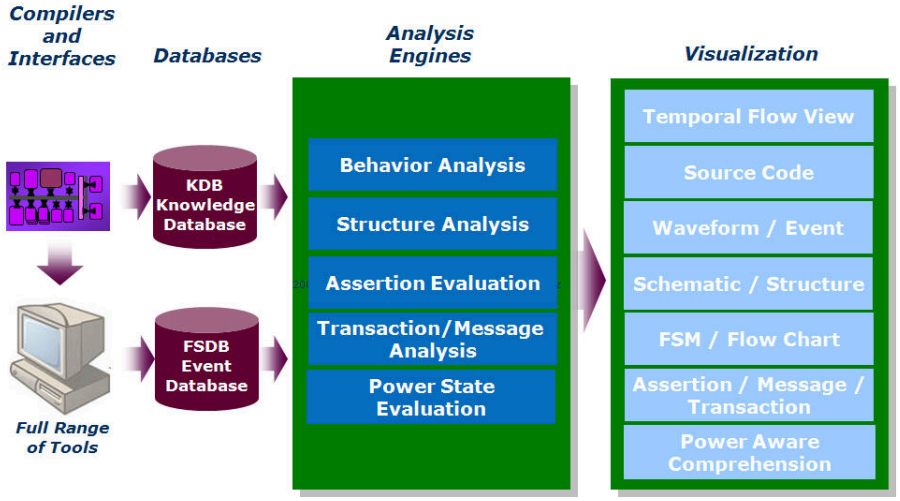
The Verdi platform helps engineers to swiftly identify, isolate, comprehend, and fix bugs, thereby boosting the efficiency and productivity of engineering resources, cutting costs, and accelerating the time-to-market for silicon products.

Technology Overview

A technology base is constructed that is optimized for design exploration, understanding, and debugging. The Verdi platform's unique architecture features powerful compilers, interfaces, databases, analysis engines and visualization tools in an integrated system for complete debugging.

Feedback

Figure 1 Verdi Technology Overview



Compilers, Interfaces and Interoperability

The Verdi platform has compilers for the most common design/verification languages and provides several interfaces for standard simulators.

- **Compilers**: The Verdi platform provides compilers for the languages used in most design and verification environments, such as Verilog, VHDL, SystemVerilog (both design and verification code), and power code (CPF or UPF). As the code is analyzed and compiled, it is checked for syntax and semantic errors.
- **Interfaces**: The Verdi platform's readers import industry-standard VCD and SDF data from all simulators and timing tools. The results are read from the detection tool and stored in the Fast Signal Database (FSDB). Direct dumping to FSDB through the object files linked to a verification tool (simulator) results in smaller waveform files and flexible access to post- simulation data.
- **Interoperability**: The Verdi platform provides extensive, well-documented, and supported interfaces that ensure interoperability with all leading logic simulators and a variety of formal verification and timing analysis tools. These interfaces also enable the integration of other verification applications through Tcl (Tool Command Language) and C language APIs. Synopsys has partnered with many design and verification companies to integrate their tools with the Verdi platform, which helps save time and reduce costs by offering a unified view across the entire verification and debug workflow.

Databases

The Verdi platform provides two databases. All analysis engines and visualization tools use these databases.

- **Knowledge Database (KDB)**: The Verdi platform employs its internal synthesis technology during the design compilation process to identify and extract detailed structural, logical, and functional information about the design. This comprehensive design data is subsequently stored in the Knowledge Database (KDB).
- **Fast Signal Database (FSDB)**: The FSDB (Fast Signal Database) is designed to efficiently store simulation results, including transaction data and logged messages from SystemVerilog Testbench (SVTB) or other supported languages, in a compact format that allows for rapid data access. Synopsys provides object files that can be linked with popular simulators to directly save simulation results in FSDB format. You can create FSDB files either from the provided routines or by converting VCD (Value Change Dump) files. Additionally, FSDB read/write API routines are available for customers and partners to use.

Analysis Engines

Using the information from the KDB and FSDB, the Verdi platform provides a set of analysis engines for different applications, including:

- **Structure Analysis**: analyze design structure to show how components are connected.
- **Behavior Analysis**: analyze design and simulation results to display design operation over time.
- **Assertion Evaluation**: answer questions and search for details about design operation from a previous simulation.
- **Transaction/Message Analysis**: analyze transaction and message (log) data in the FSDB file and visualize in *nWave* and a spreadsheet view.
- **Power State Evaluation**: evaluate the power state based on the power intent description in the CPF/UPF and the values of related signals in the FSDB file.

Graphical User Interface

The graphical user interface uses the Qt platform and provides the following functions:

- A *Welcome* page summarizing the available resources in a single location.
- History support enabling easy restoration of previous sessions.

- Typical work modes with predefined window layouts making the debug content easy to locate.
- A unique Spotlight function searches for a command without exhaustively searching through all the drop-down menus.
- Several customization options:
  - System frames and toolbar icons can be undocked, moved to a new location, and then docked again.
  - A pane can be maximized by double-clicking the pane banner so that the content is more visible. You can double-click it again to shrink it to its original size.
  - The visible toolbar icons can be selected through a menu option.
  - Bindkey values, drop-down menu names, and locations can be customized through the customization form.

Visualization

The Verdi platform delivers outstanding temporal visualization features with its *Temporal Flow View*. This cutting-edge tool extracts and displays multicycle temporal behavior from design data and simulation results. .

The Verdi platform offers sophisticated structure visualization and analysis tools such as *nTrace* for source code, *nWave* for waveforms, *nSchema* for schematic/logic diagrams, and *nState* for finite state machines (FSMs). These tools are intended to analyze the structure of the design in the form of the signal relationships in the RTL, physical connections in schematic/logic diagrams, states and transitions in FSM bubble diagrams, and value changes in waveforms.

The *Property Tools* window in the Verdi platform provides integrated support for assertions and enables quick traversal from an assertion failure to the related design activity. While the *Transaction/Message Analyzer* enables debug and analysis at higher levels of abstraction from transaction or log information saved to the FSDB file, the *Power Manager* window provides visualization of the power intent and supports cross-probing with other Verdi platform windows.

The Verdi platform offers fully integrated views, enabling you to select any section of the design source code and immediately generate corresponding hierarchical or flattened logic diagrams. This allows for efficient exploration of a design and its verification results by clicking on context-sensitive hyperlinked objects and signals within any view. This seamless integration facilitates quick and easy switching between views to find and isolate the specific information required to understand any part of the design and resolve any issues.

2

User Interface Conventions

This chapter describes the following topics:

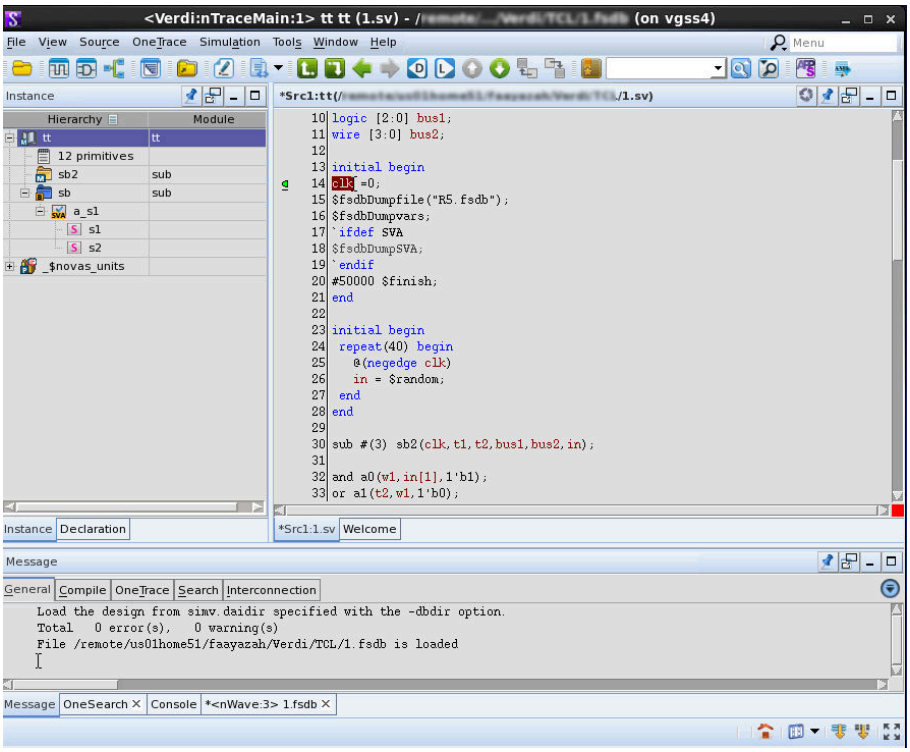
- Overview
- Common User Interface Features
- nTrace User Interface
- nWave User Interface
- nSchema User Interface
- nState User Interface
- Flow View User Interface
- Transaction/Message User Interface
- nCompare User Interface
- nECO User Interface
- nAnalyzer User Interface

Overview

The Verdi® Automated Debug Platform has a highly customizable graphical user interface with a contemporary look. The following figures illustrate the look of the Verdi platform.

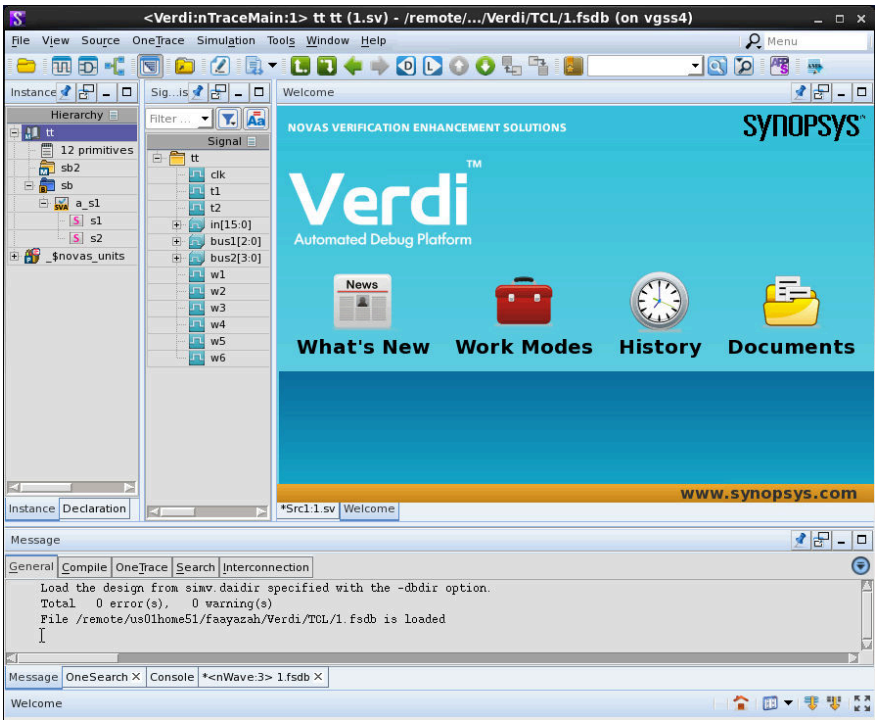
Feedback

Figure 2 Verdi Main Window



Feedback

Figure 3 Verdi Window With Welcome Page



The Verdi platform has a large number of commands, including many that are invoked through mouse clicks or drags rather than selecting from pull-down menus at the top of each window. Read this chapter to become familiar with the interface conventions of the Verdi platform before proceeding further.

Common User Interface Features

The features described are common to *nWave*, *nTrace*, *nSchema*, *nState*, and *Flow View* components. Refer to the *User Interface Overview* section of the *Introduction* chapter in the *Verdi and Siloti Command Reference Manual* for additional information.

This section consists of the following topics:

- Frame Banner
- Pull-Down Menu Commands
- Mnemonic Keys
- BindKeys
- Toolbars

Feedback

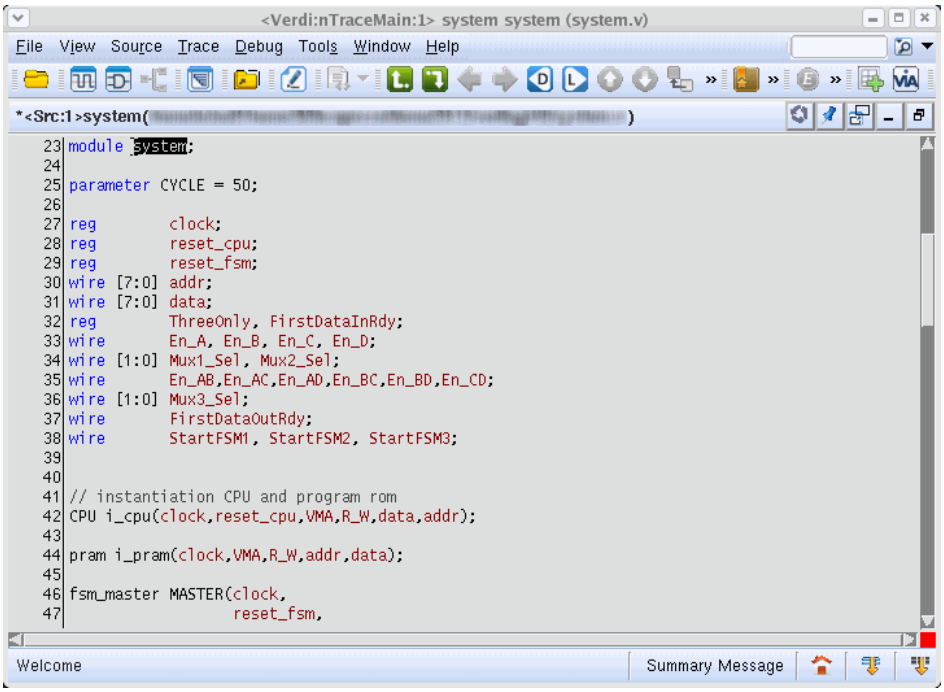
- Mouse Operation
- Right Mouse Button Menus
- Undock/Dock
- Online Help
- Feedback Button
- Unified Tooltip Viewer
- Next Generation Debug GUI

Frame Banner

The banner at the top of each pane identifies the application, frame number (such as *<nWave:2>*), and file, unit, or scope displayed in that pane. The asterisk (*) character appearing at the front of the banner indicates that the pane is the active one.

Double-click the pane banner bar to maximize a pane or to shrink the pane back to the previous size as shown in the following figure.

Figure 4 Maximize the Source Code Pane



Right-click the banner of a dockable pane to display a configuration option menu that lists all the available dockable panes and toolbar categories. Toggle the option to hide/show the entire pull-down menu, dockable pane, or toolbar category.

Figure 5 Configuration Option Menu to Hide/Show Dock Panes/Toolbar Categories

Pull-Down Menu Commands

A pull-down menu bar is located just below the banner for panes that are also windows. Each menu item contains several commands that display when the menu is selected. The pull-down menu can be hidden or displayed by selecting the right-click **Menu** option invoked from the window banner, menu bar, or toolbar.

When the **Menu** option is toggled *off*, the pull-down menu of the pane or window is hidden. You can press the **Alt** key within the pane or window to display or hide the pull-down menu again. Also, when the cursor is clicked elsewhere in the pane or window, the menu is automatically hidden again.

When the **Menu** option is toggled *on* (the value *on* means always show the pull-down menu), the pull-down menu cannot be shown/hidden by pressing the **Alt** key.

For each subwindow or window in the Verdi platform, custom commands can be added using the **Tools -> Customize Menu/Toolbar** command.

Mnemonic Keys

The pull-down menus support **Meta** key invocation using mnemonics. The mnemonic for each item is indicated by an underline. For example, to display the **File -> Open** menu (`meta -fo`), press and hold the <Meta> key on your keyboard (the diamond key/<Alt> key on Sun keyboards or the <Alt> key on Windows' keyboards) and press the "f" key, then release the <Meta> key and press the letter "o" key.

BindKeys

A command can be bound to either a keystroke or a mouse button. After the bindkeys are defined, commands can be invoked with a keystroke or mouse click. For example, the **Source -> Active Annotation** command can be invoked using the "X" key (the defined bindkey is the letter after the command in the menu). The bindkeys of the menu commands can be customized using the **Tools -> Customize Menu/Toolbar** command.

Toolbars

A row of icons appears beneath the pull-down menu bar on panes that are also windows. These icons provide access to frequently used commands for the current window.

The available toolbar icons may be modified using one of the following methods:

- Enable/disable the icon category using the main window right-click option menu.
- Left-click to select the separator bar and then drag left or right to decrease or increase the associated space. When the space is decreased such that some icons are no longer visible, a double arrow (>>) symbol is displayed to the right of the category. Clicking this symbol displays the hidden icons.
- Left-click to select the gray bar and then drag up or down to undock the category and then move to a new location on the toolbar or to left/right/below the window. Available slots are highlighted with a blue dashed line.
- Define/modify/add toolbar icons and categories using the **Tools -> Customize Menu/Toolbar** command. Refer to the *nTrace* chapter of the *Verdi and Siloti Command Reference Manual* for details.

Mouse Operation

The mouse is most often used to select objects by clicking the left mouse button. A range of objects can be selected by dragging with the left mouse button over the objects or by using the <Shift> key along with left-clicking. To add or remove individual objects to or from the selection, use the <Ctrl> key along with the left-click.

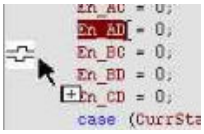
Feedback

The Verdi platform also makes use of drag-and-drop to move information from one pane/window to another. Normally drag-and-drop is performed by pressing the middle mouse button or left mouse button to select the object, holding the button as the mouse is moved to a new location, and then releasing the button to “drop” the object into a new location.

The drag-and-drop operation can be performed between different pane types, for example, dragging a signal from the *nWave* pane and dropping it to the source code pane executes tracing connectivity of the selected signal. If the pane is in the background (displayed as a tab), moving the dragged object to the tab name and dropping it changes the tab to the foreground and drops the object. The resulting behavior is the same as if the object is dropped directly in the pane.

The drag-and-drop operation depends on the center of the arrow cursor and not the tip of this cursor. For example, if a signal is dragged from *nWave* then it is dropped to the *Source Code* pane depending on the center of the arrow cursor as shown in the figure:

Figure 6 Example of Cursor Position



Right Mouse Button Menus

Right-click an object to display a menu with commands appropriate for that object type. These menus are described in detail in the *Right-Click Commands* sections of the *Verdi and Siloti Command Reference Manual*.

Undock/Dock

The main window of the Verdi platform consists of dockable panes that can be released (undocked) from the main window. The dockable panes can be docked to the main window again.

Every dockable pane has its own banner or title bar. The dockable panes can be moved from one dock area to another by dragging the pane banner. A dockable pane can attach above, below, left, right or over another dockable pane. A tab is created when you dock a pane over another dockable pane. Refer to the following figures for examples.

Feedback

Figure 7 Undock Design Browser Pane from Main Window

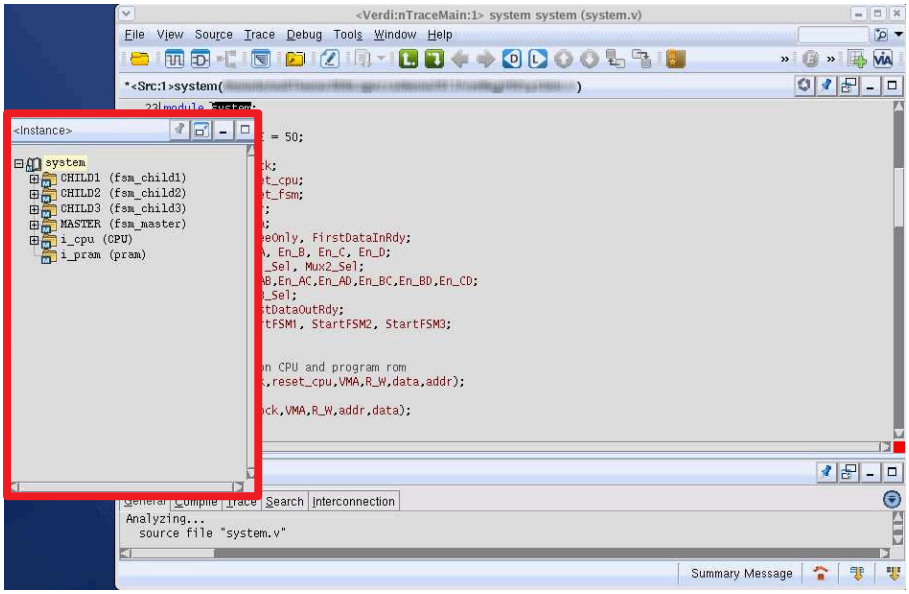
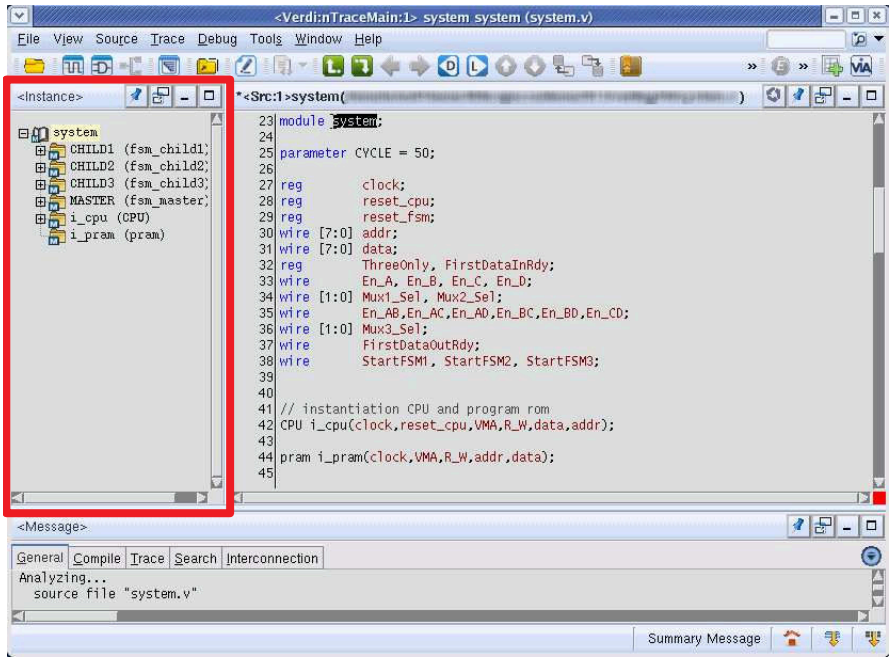


Figure 8 Redock Design Browser Pane to Main Window



Feedback

Figure 9 Dock nWave to the Right of the Design Browser Pane

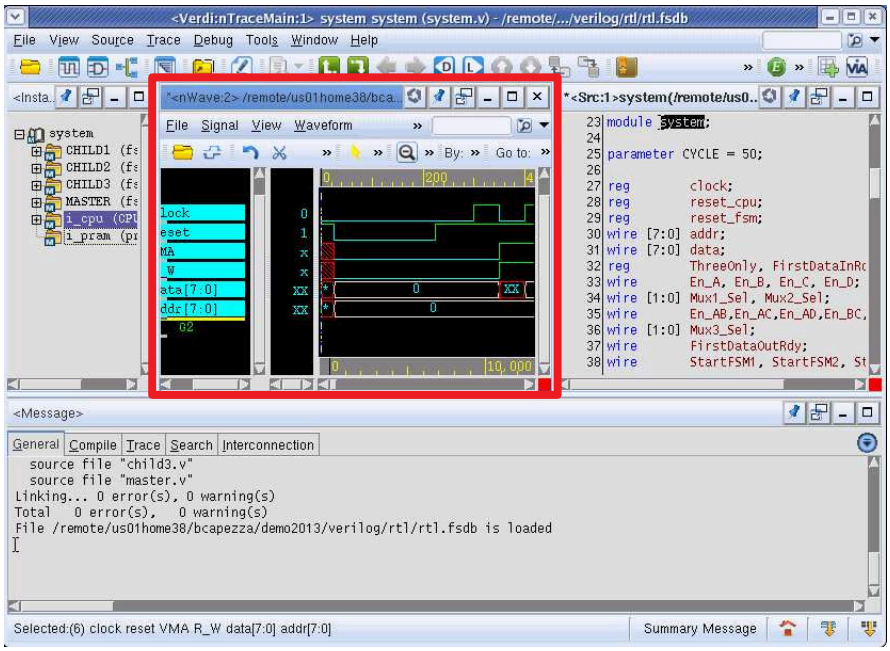
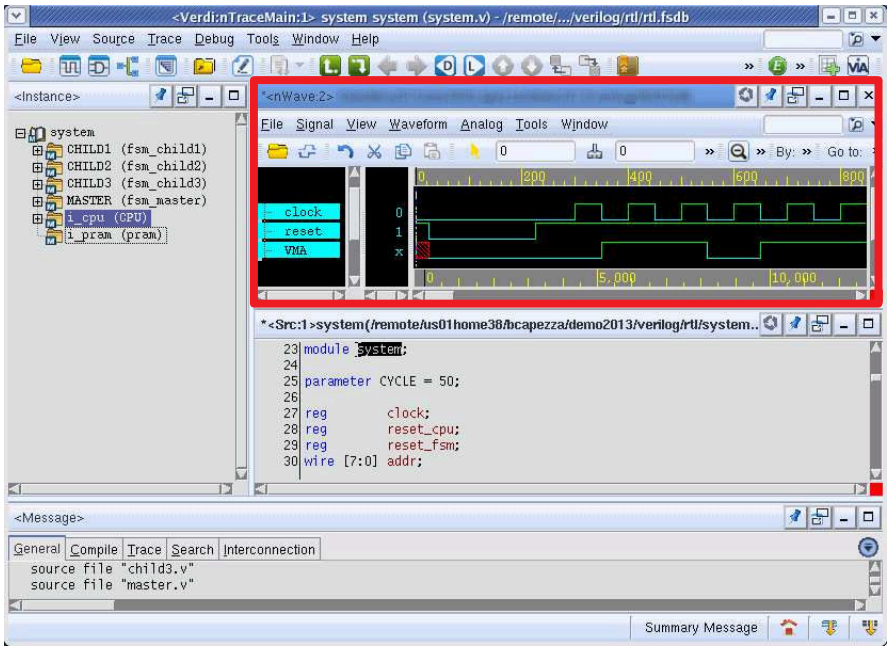
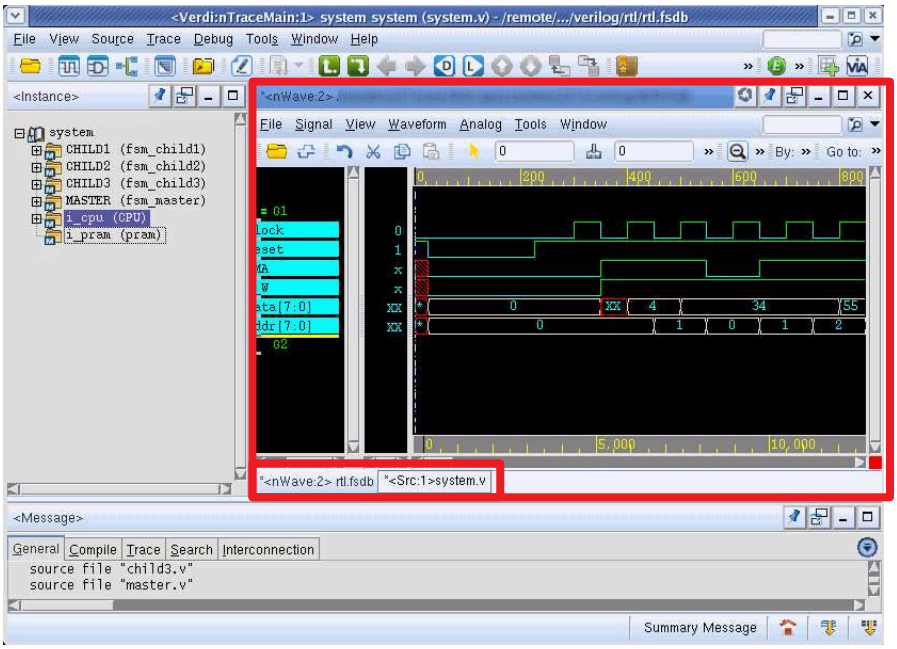


Figure 10 Dock nWave above the Design Source Code Pane



Feedback

Figure 11 Dock nWave over the Design Source Code Pane to become a Tab



A pane can also be docked/undocked by clicking the **Dock/Undock** icons on the pane banner. Some major dockable panes, like *nWave* and *nSchema*, can be released to become standalone windows. Other panes (such as, *message* and *source code* panes) that belong to the *nTrace* main window can also be released to become widgets.

Refer to the *Icons for User Interface Overview* section in the *Introduction* chapter of the *Verdi and Siloti Command Reference Manual* for more information.

Feedback

Figure 12 nSchema Docked as a Pane

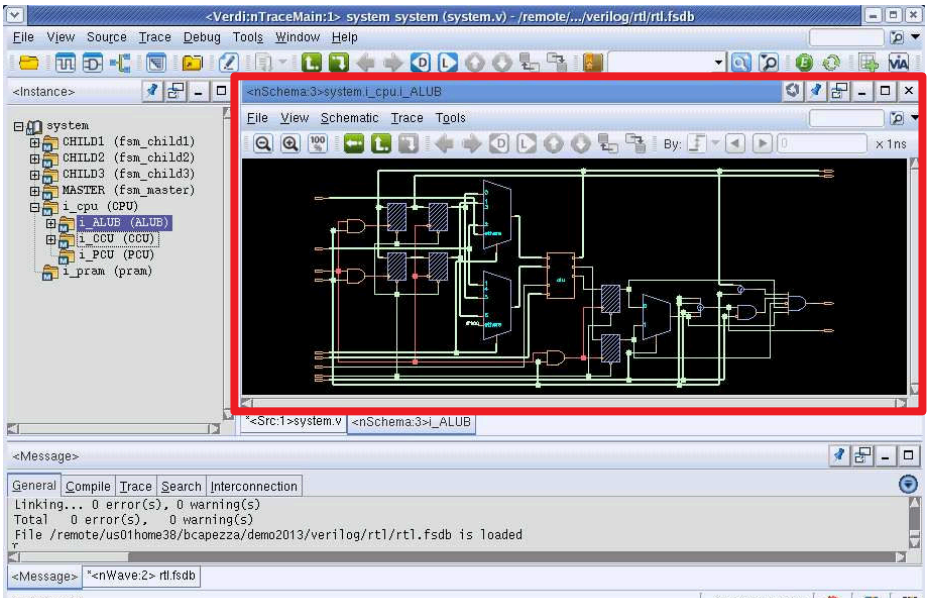
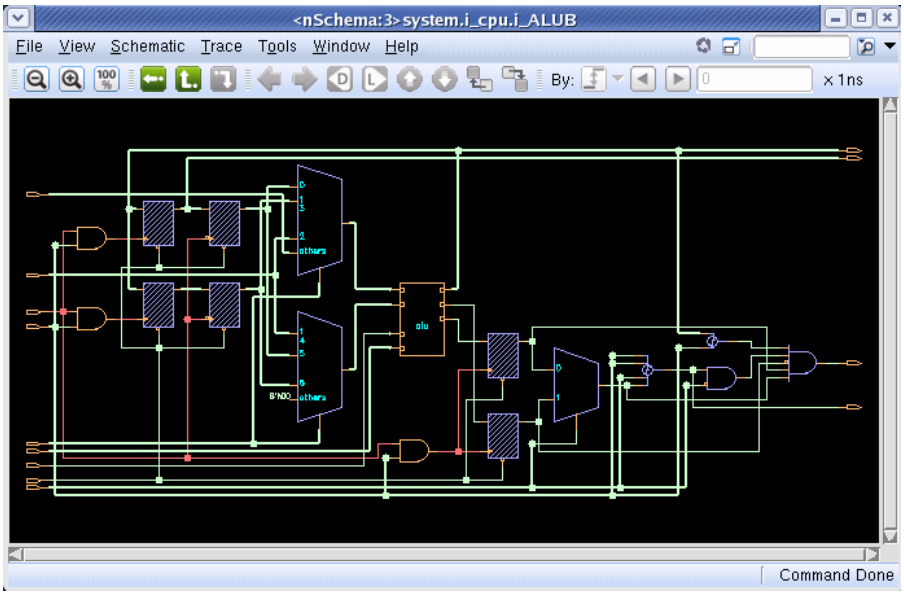


Figure 13 nSchema Undocked as a Window



Right-click on any pane banner to display a configuration option menu that lists all the available dockable panes and toolbar categories. Toggle the option to hide or show the entire pull-down menu, any dockable pane, or toolbar category.

The layout of the main framework can be saved or restored by invoking the **Window -> Save/Restore User Layout** command. To switch to the previous or next layout, invoke **Window -> Previous Layout** or **Next Layout** commands respectively.

Refer to the *Window/Frame Right-Click Options* sections of the *Verdi and Siloti Command Reference Manual* for details.

Online Help

The *nTrace* main window and the standalone *nWave*/*nSchema* windows provide online help, which can be accessed through the **Help** menu.

Feedback Button

In each window (including undocked windows), you can invoke the feedback dialog by the menu command **Help -> Feedback**. You can fill the entries and the feedback is sent to verdi_feedback@synopsys.com, by default.

To disable this feature, set the environment variable `VERDI_DISABLE_FEEDBACK` using the following command:

```
setenv VERDI_DISABLE_FEEDBACK 1
```

The following options are available on the **Feedback Dialog** window:

- **Type**: Specifies the type of feedback. The options are as follows:
  - **I Like Something**: Select this option if you have a positive feedback.
  - **I Don’t Like Something**: Select this option if you have a negative feedback.
  - **I Have a Suggestion**: Select this option if you have a suggestion to provide.
- **Feature**: Specifies the feature you want to provide the feedback for. The options are as follows:
  - Source View
  - Waveform View
  - Schematic View
  - AMS Debug
  - PowerAware Debug
  - Testbench Debug
  - HW/SW debug

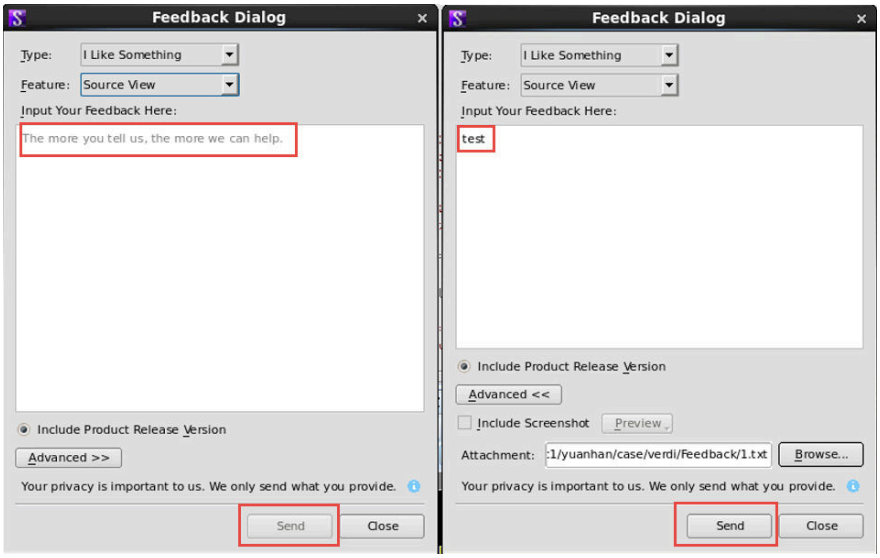
- Fault Analysis Debug
- Coverage and Planner
- Other

Note:
When you click the menu command **Help -> Feedback** from a specific window, the feature name is autofilled based on where you click the feedback menu command from. For example, *Feature* is set to *Waveform View* automatically if the user clicks feedback menu command from **nWave** window.

- **Input Your Feedback Here**: You can specify your feedback in this field. There is a placeholder “The more you tell us, the more we can help” in it. Input at least one character in this field for the **Send** button to be enabled.

  The following figure shows the behavior of this field. When nothing is specified, **Send** button is disabled and when some text is entered, the **Send** button is enabled.

Figure 14 Input Your Text Here

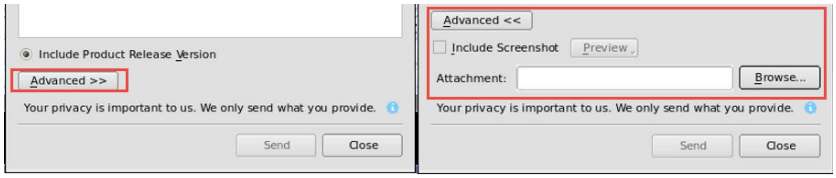


- **Include Product Release Version**: Specifies that you agree to add the Verdi version along with the feedback email content. The value of this option is *on* by default. Toggle the option to *off* if you do not want to add the Verdi version along with the feedback email content.
- **Advanced>>**: Click this option if you want to include a screen shot or an attachment to be sent along with the feedback email content.

- Include Screenshot
- Preview
- Attachment

The screen shot only captures the window under Verdi instead of the whole desktop. If the check box **Include Screenshot** is toggled off, the **Preview** button is disabled. Otherwise, it is enabled. When you click the **Preview** button, you can see the screenshot image. The following figure depicts the behavior of **Advanced>>** option.

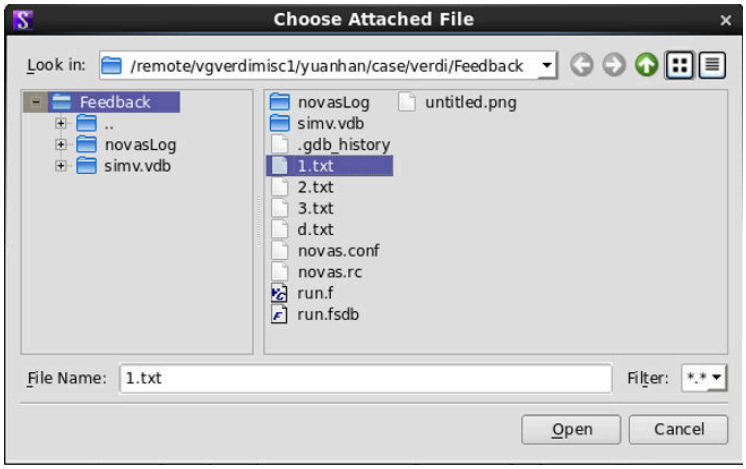
Figure 15 Advanced>> option



Note:
If the screenshot needs to exclude this **Feedback Dialog** window, the only way is to hide this dialog, make screenshot and show it again.

Clicking **Browse** pops up a file dialog **Choose Attached File** for you to choose single attachment, as shown in the following figure:

Figure 16 Choose Attached File



The selected file is shown in the Attachment label.

Figure 17 Attachment Label

Send: Click this option to send the feedback email to verdi_feedback@synopsys.com,, by default.

To configure the destination address, set the environment variable `VERDI_FEEDBACK_EMAIL` using the following command:

```
setenv VERDI_FEEDBACK_EMAIL [recipient_address]
```

- **Close**: Click this option to close the **Feedback Dialog** window.

Feedback Email Format

The type is specified in the mail body and the feature is added to the subject line. The screenshot and attached file appears as attachments in the email. The version of Verdi (if **Include Product Release Version** is *on*) and the message you enter in the **Input Your Feedback Here** field are added as the mail content.

The following figure depicts the feedback email:

Figure 18 Feedback Email

Limitation

When you toggle *on* the check box **Including screenshot**, the GUI flashes because the screenshot needs to exclude the feedback dialog.

Unified Tooltip Viewer

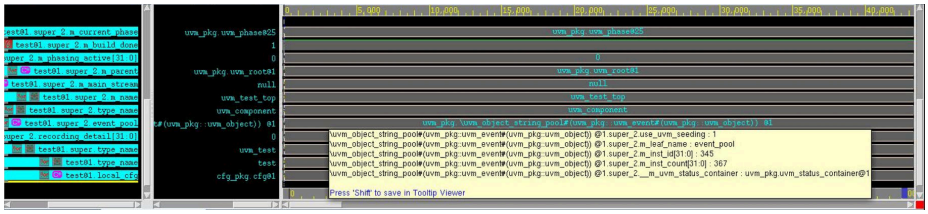
When a tooltip pops up in any window in Verdi, you can press **Shift** key on the keyboard to save the tooltip content and view it in the *Tooltip Viewer* dialog. You can also use the

Feedback

menu command **View -> Tooltip Viewer** to invoke the *Tooltip Viewer* dialog. An example scenario is as follows:

1. Add a class pointer signal and hover the mouse over the waveform to display the tooltip. There is information at the end of the tip to indicate the way to save the tip content into Tooltip Viewer dialog, as illustrated in the following figure:

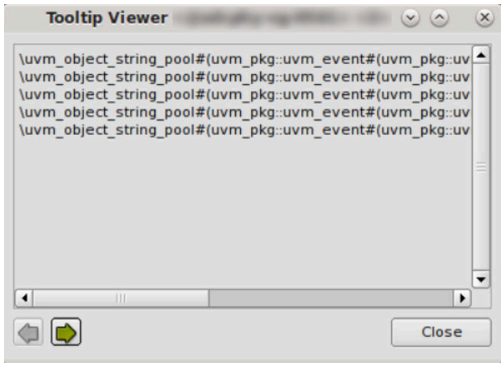
Figure 19 Tooltip Information



The tooltip information is in the form of rich text, which can display the color and the hyperlink. The hyperlink appears in blue text. Currently, only **SmartLog** has hyperlinks in its tooltip. To differentiate the hyperlinks and short description, short description color is displayed in gray color.

2. Press **Shift** or select the **View -> Tooltip Viewer** menu to invoke the *Tooltip Viewer* dialog and fill in the tip content, as illustrated in the following figure:

Figure 20 Tooltip Viewer



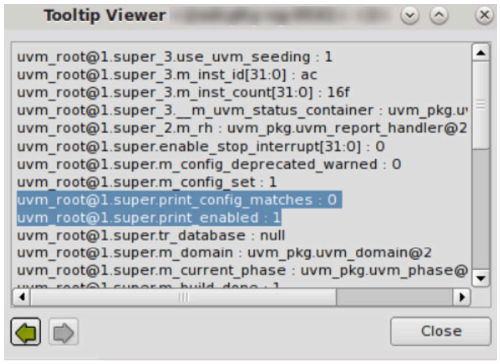
The Shift hotkey works for all tooltips in Verdi.

The menu command *Tooltip Viewer* is enabled only when at least one tooltip is saved in the unified Tooltip Viewer. When there is a tooltip popup in any window, you can press *Shift* to save the tooltip content in the *Tooltip Viewer* dialog.

Feedback

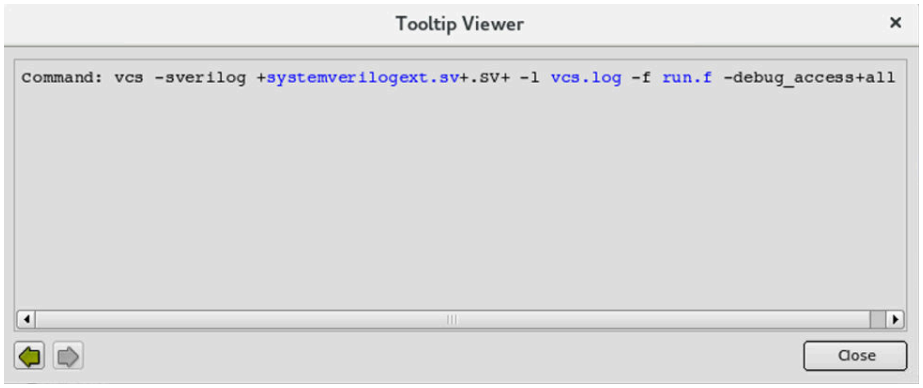
3. Select and copy the text in the dialog, or view the previous/next history tip content, as illustrated in the following figure:

Figure 21 Tooltip Viewer



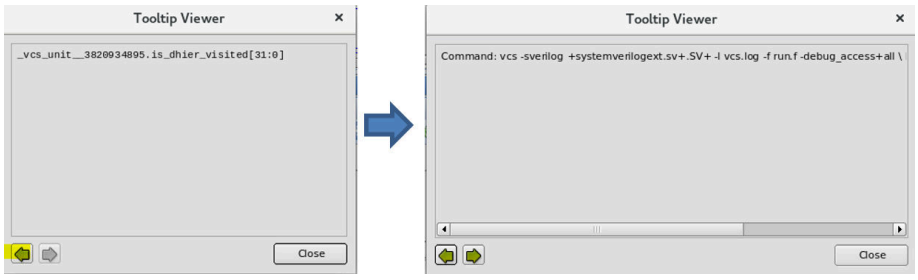
For the tooltip with hyperlinks, rich text is shown only when it is the latest tooltip in the *Tooltip Viewer*.

Figure 22 Tooltip Viewer with Hyperlinks



Add a new tooltip to the *Tooltip Viewer* and click the previous button; the original tooltip with hyperlinks becomes plain text.

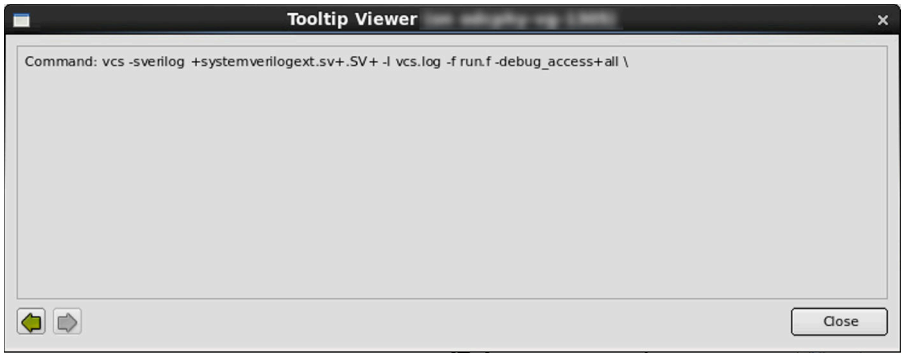
Figure 23 Plain text for the Previous History Content



Feedback

If the hyperlink in the latest tooltip comes from a window, and if you close the corresponding window, the tooltip content becomes plain text.

Figure 24 Tooltip Content Changes to Plain Text if the Window is Closed



Limitation

- Does not support the tooltips displayed in both QT/Motif dialogs.
- When the active window is a dialog, the **Shift** hot key does not work except for the unified *Tooltip Viewer* dialog.
- Unified Tooltip Viewer is not supported for Verdi Coverage.

Next Generation Debug GUI

Verdi enhances the look and feel of the GUI by providing the following customized Next-Gen GUI enhancements:

- Color Themes
- Grid and Density
- Banners
- Search
- Find
- Quick Launcher
- Recently Used Command Buttons
- Toolbar With Text and Icon

Feedback

Color Themes

You can configure the color theme through preferences for the preferred look and feel.

Use the following environment variable to enable this feature:

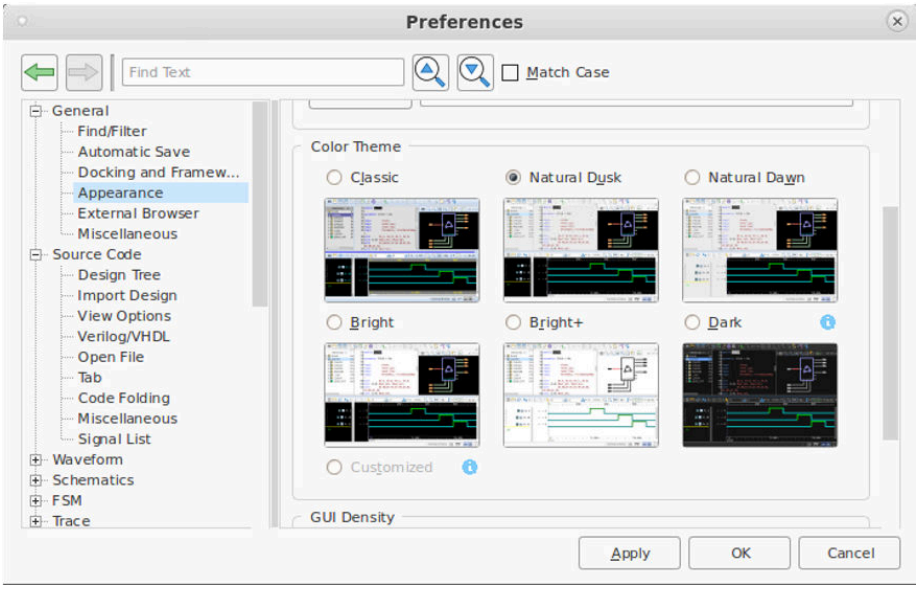
```
setenv VERDI_NEW_COLOR_THEME 1
```

The following figure illustrates the various color themes supported by the Verdi GUI.

Note:

Dark mode and other modes cannot be switched on-the-fly. The settings are saved to *novas.rc* and applied next time when Verdi GUI is invoked.

Figure 25 Themes



Limitations

Following are the limitations of the color themes:

- Runtime switch from any color theme in light mode to dark mode is not supported due to Qt4 limitation. You can only enable dark mode by setting the environment variable `VERDI_COLOR_THEME dark` or launching Verdi with the `-dark` option.
- Color settings in `novas.rc` file may conflict with the selected color theme when launching Verdi, thus overwriting the required theme color. To avoid this, remove all the `rc` settings according to the color when the user switches between light/dark color themes. If there are no `rc` settings, there is no overwrite issue.

Feedback

- Only backward compatibility is supported for old `novas.rc`, which means that, if users open Verdi with the old `novas.rc`, then the default color setting is applied for them as some `rc` keys of new GUI features do not exist in old `novas.rc`. When user launches Verdi in dark mode and closes it, the dark color setting is saved in `novas.rc`. If users open an old version of Verdi or with a disabled color theme environment variable next time, the dark color setting is still loaded which displays Verdi GUI with the incorrect background colors (some windows have black background, some have gray background). If users open an old version of Verdi with dark theme `novas.rc`, a popup message is displayed to inform the users the current application version is older than the `rc` file version, which may lead to incompatible problems.

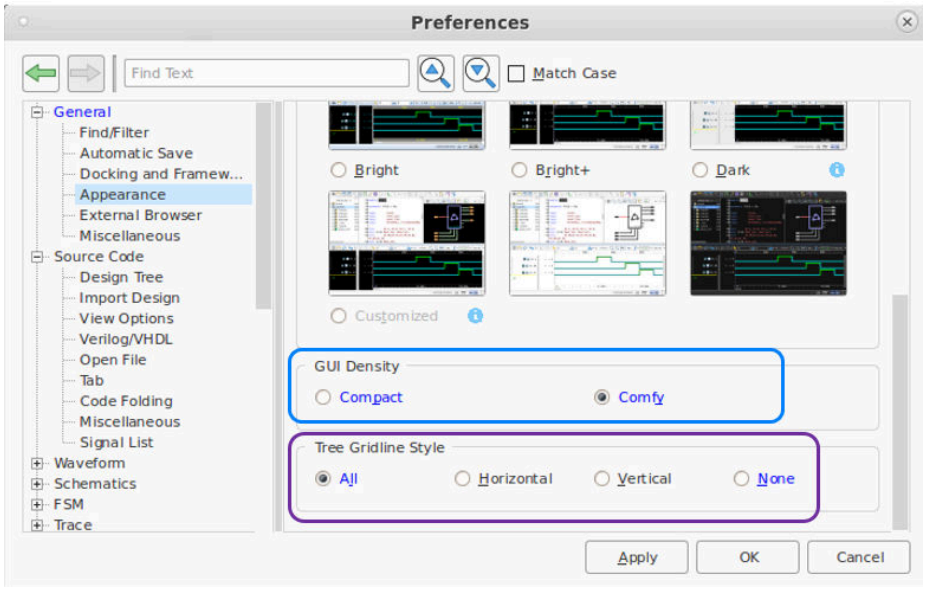
Grid and Density

Verdi supports customizing the grid and density GUI features.

Perform the following steps to customize the GUI density feature:

1. Launch Verdi GUI.
2. Select **Preferences > General > Appearance** option.
3. Scroll-down the page and observe that **GUI Density** section provides the following features, as shown in the following figure.
   - Compact
   - Comfy

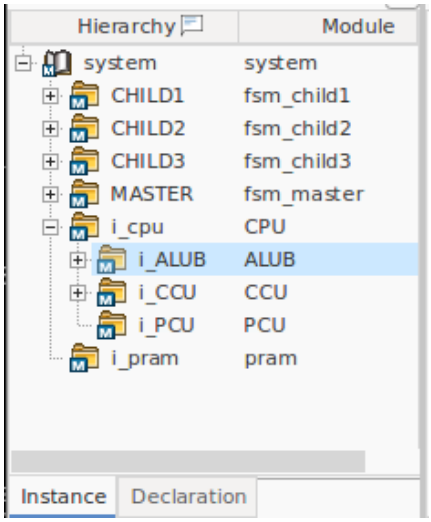
Figure 26 GUI Density



Feedback

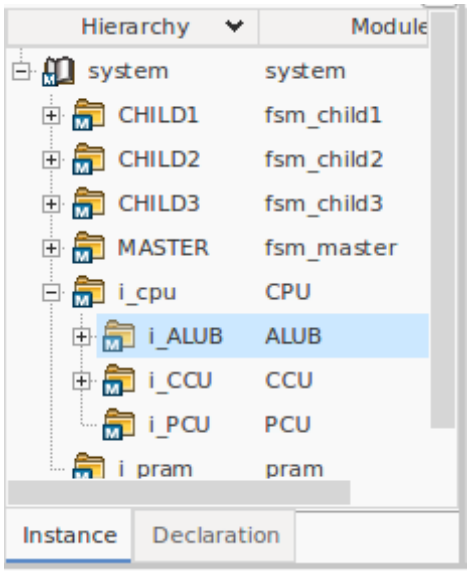
4. Select **Compact** radio button.
5. Click **Apply** and **OK** options for the compact view of the hierarchical modules, as shown in the following figure:

Figure 27 Compact View

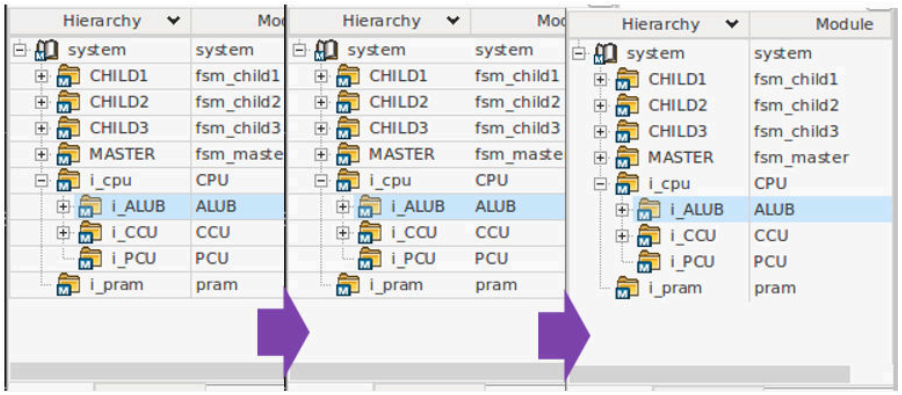


6. Select **Comfy** radio button.
7. Click **Apply** and **OK** options for the comfy view of the hierarchical modules, as shown in the following figure:

Figure 28 Comfy View



8. When you scroll down the **Appearance** window, you can observe that **Tree Gridline Style** section, you can use this feature to configure the gridline styles. It provides the following gridline style options, as shown in the following figure:
   - **All**
   - **Horizontal**
   - **Vertical**
   - **None**



9. You can select the required gridline style and click **Apply** and **OK** options.

Banners

The banners are auto-collapsed for more vertical debug spacing.

Perform the following steps to collapse or expand the banners:

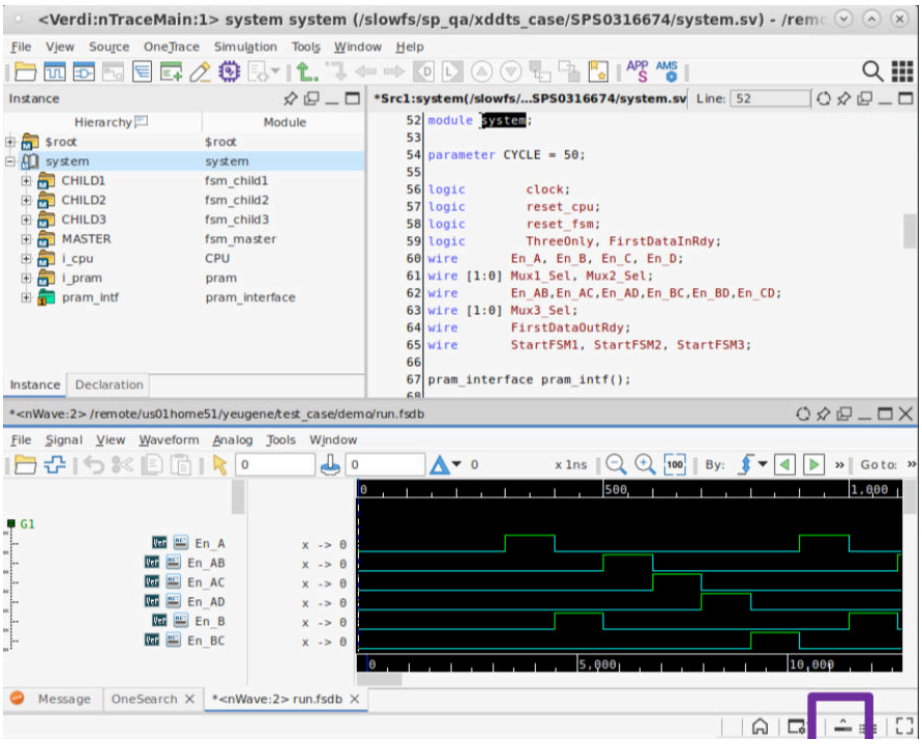
1. Launch Verdi GUI.
2. Select the icons as shown in the following figures to collapse or expand the banners.

You can use the following environment variable to auto-collapse or hide the banner feature:

```
setenv VERDI_AUTO_COLLAPSE_BNR 1
```

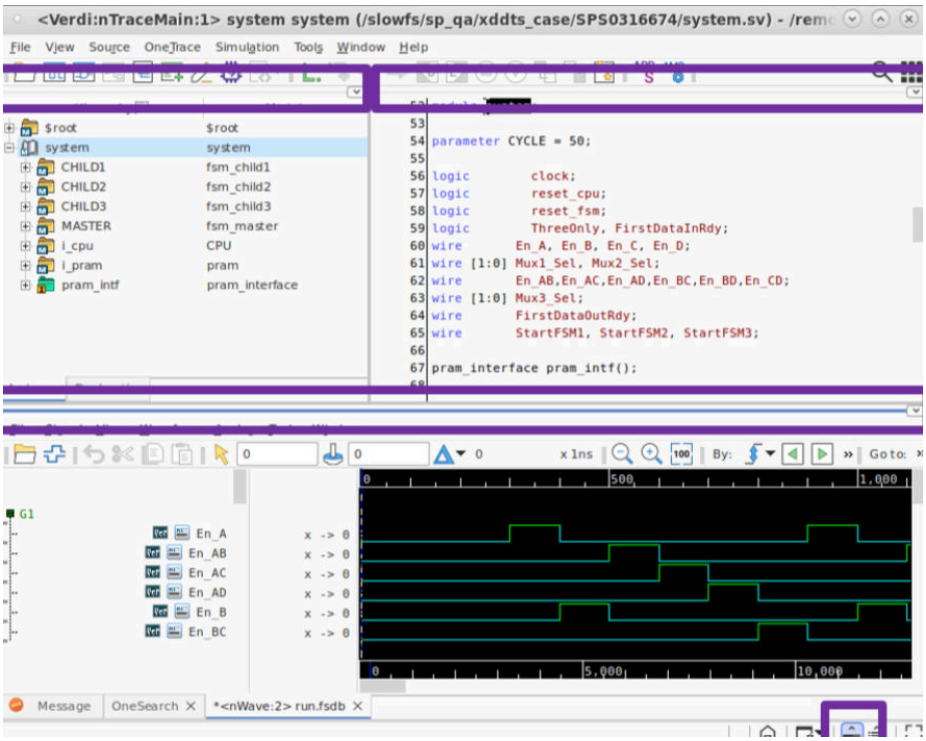
Feedback

Figure 29 Banners



Feedback

Figure 30 Auto-Collapse Docked Banners



Find

Verdi allows you to find information in a specific window by pressing **Ctrl+F** keys in a window. Each window has a separate find bar. Following are the various search constraints supported by this feature:

- Match Case
- Wildcard
- Regular Expression
- Find Whole Words Only

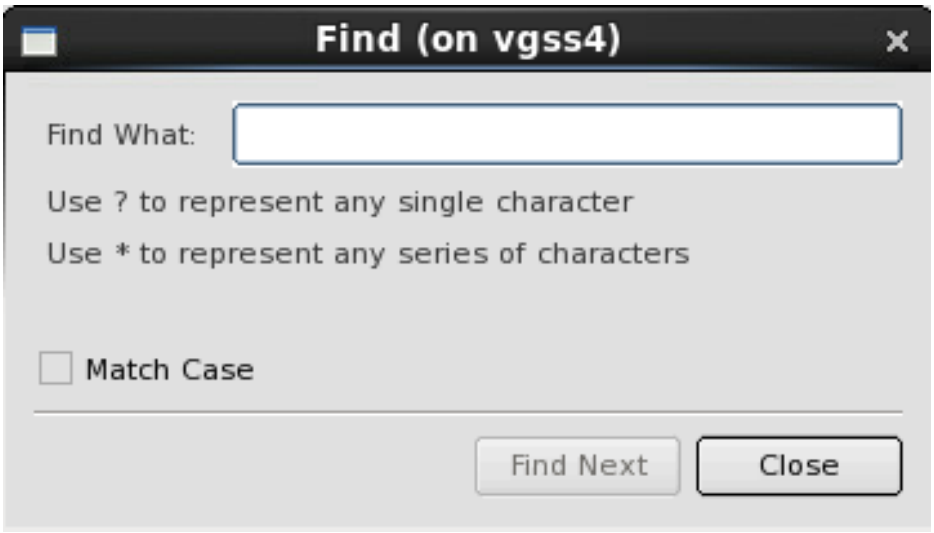
Use the following environment variable to enable this feature:

```
setenv VERDI_NEW_SEARCH_FIND 1
```

Feedback

The following figure illustrates how the find feature is used in the specific windows:

Figure 31 Find



Search

Verdi provides search pane for the broader search. Search applies to all the windows of the Verdi.

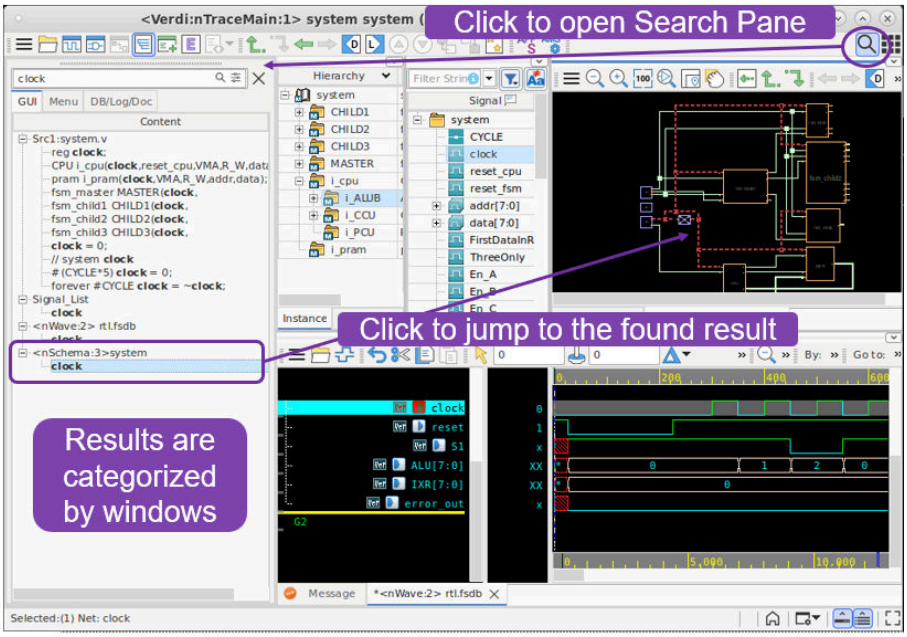
Use the following environment variable to enable this feature:

```
setenv VERDI_NEW_SEARCH_FIND 1
```

Feedback

Perform the following steps to use the search feature:

1. Click **search** (magnifying glass) icon on the upper right corner of the window to open the search pane, as shown in the following figure:

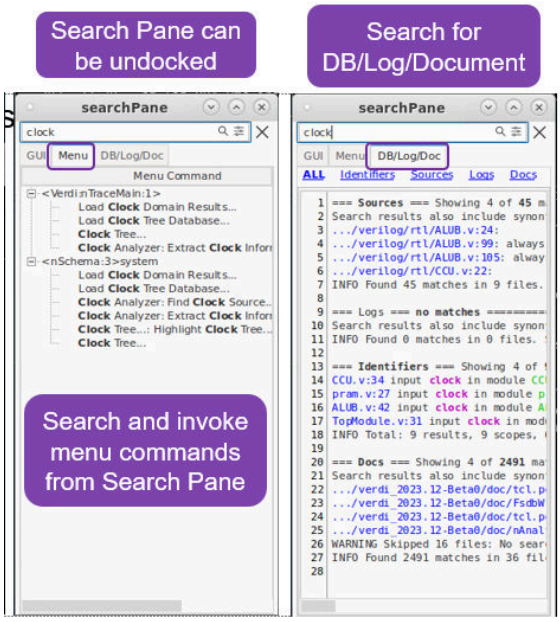


2. Enter any search key/text in the search pane.
3. Click *search* (magnifying glass) icon in the search pane.
4. Click on any searched result to go to the found result.
5. Search pane can be undocked as an individual pane, as shown in the following figure:
6. Search pane has the following tabs to search the results based on these categories:
   - **GUI**
   - **Menu**
   - **DB/Log/Doc**

Feedback

7. You can choose the required tab and search the information.

Figure 32 Undocked Search Pane and Search Tabs



Limitations

Following are the limitations of this feature:

- The correct behavior of searching when the editor is on editing and unsaved status is not guaranteed. The result content may be wrong.
  - If line content was deleted or changed, although the current content does not match with the search keyword, it may still show on the search pane when the previous document matches the keyword before modified.
  - If line content was added, although the fresh new content matches with the keyword, Verdi cannot read in the content and search pane does not show the fresh matched line content.
- The correct behavior of the jumping when the editor is edited after searching is not guaranteed. As a result items display with the line number of the matched line of the old document. If document content changes, the line number of the new document does not always reflect the match line number of the result. Any clicking on the result item may lead to unpredictable behavior.
- GUI tab in new search pane apply blocking search which may cause user to wait for the search results, therefore, a busy dialog is provided for the earlier abort.

- All window supported unified find has supported find bar, but not all the supported search pane.
- Onesearch has its own special searching rule and non-blocking search and other tab in search does not work.
- Original onesearch Tcl does not work.
- The following windows are not supported:
  - OVM/UVM Hierarchy tree view
  - Interactive Debug

Quick Launcher

Verdi provides Quick Launcher feature to allow you to access the integrated tools/ applications/sessions quickly.

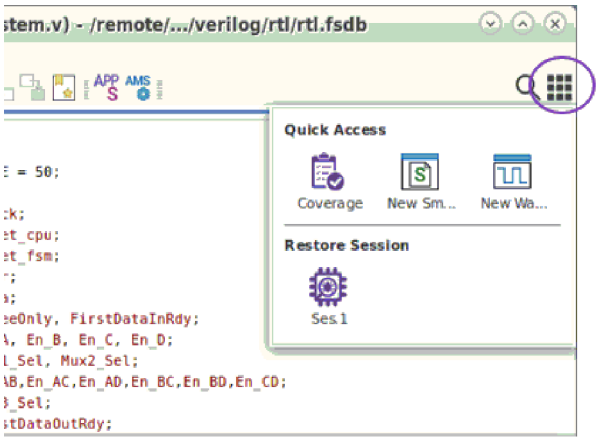
Use the following environment variable to enable this feature:

```
setenv VERDI_APP_LAUNCHER 1
```

Perform the following steps to open the quick launcher:

1. Click **Quick Launch** icon (as shown in the following figure) on the upper right corner of the window to open the quick launcher window, as shown in the following figure:

Figure 33 Quick Launcher



You can view **Quick Access** and **Restore Session** categories in the form. Integrated tools are placed in the form by default. Sessions under working directory are placed in the form.

1. Click on the required application/tool/session to open/launch quickly

Feedback

Menu and customized commands can be placed in the form using *Tcl*, as shown in the following commands:

- `verdiAppLauncher -add -act logViewer`
- `verdiAppLauncher -add -act hbInvokeWave`

Recently Used Command Buttons

Verdi provides a button in the upper-right corner that provides a drop-down list for recently used commands according to the active windows. This button is shown on demand (for example, at least one recently used command must be there).

Verdi stores 10 recently used menu commands and toolbar commands of each window type. Different windows with the same window type (for example, nWave2, nWave3, nWave4 and so on) shares the same recently used commands. Each window type has their own recently used command set.

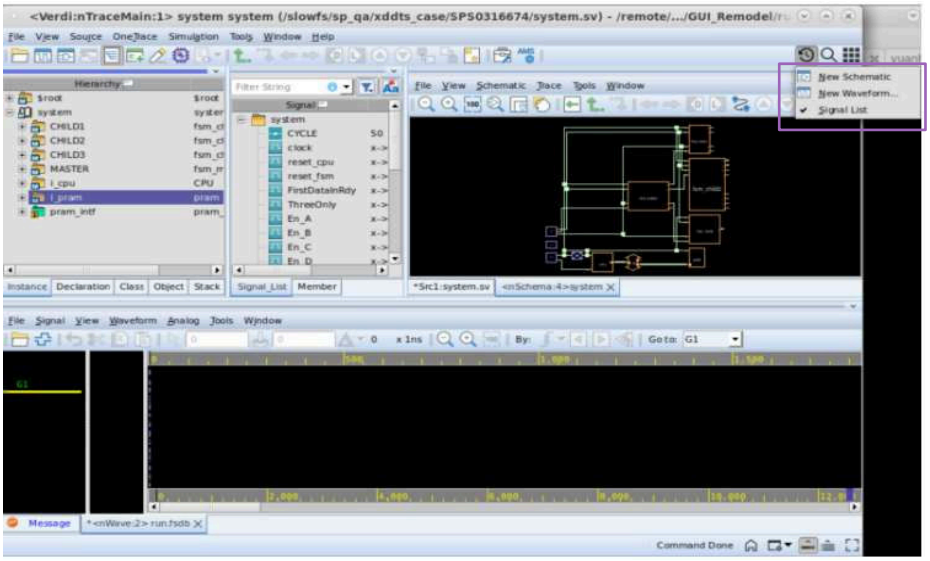
The recently used commands are stored in novas.rc so that next time when user invoke Verdi, recently command button restores the setting from novas.rc

Use the following environment variable to enable this feature:

```
setenv VERDI_RECENT_CMD 1
```

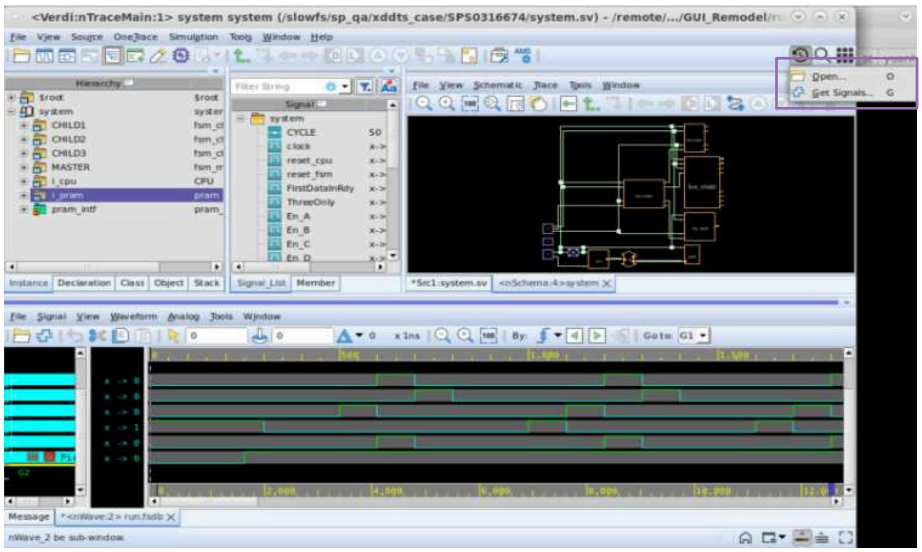
The following figures illustrate the recently used command buttons in Verdi top and nWave windows:

Figure 34 Recently Used Command Buttons in Verdi Top Window



Feedback

Figure 35 Recently Used Command Buttons in nWave Window



Toolbar With Text and Icon

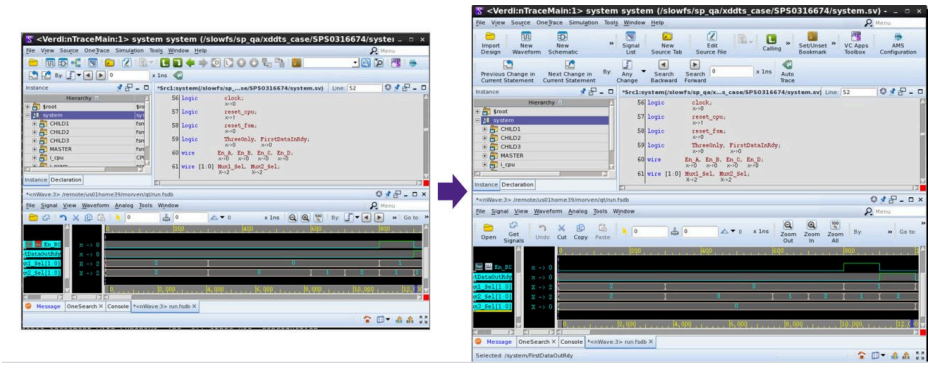
Verdi provides a toolbar with text and icon feature help Rockie and Zebu users to easily get started.

Use the following environment variable to enable this feature:

```
setenv VERDI_TOOLBAR_TEXT_UNDER_ICON 1
```

The following figure illustrates the toolbar with text and icon feature:

Figure 36 Toolbar With Text and Icon



Feedback

nTrace User Interface

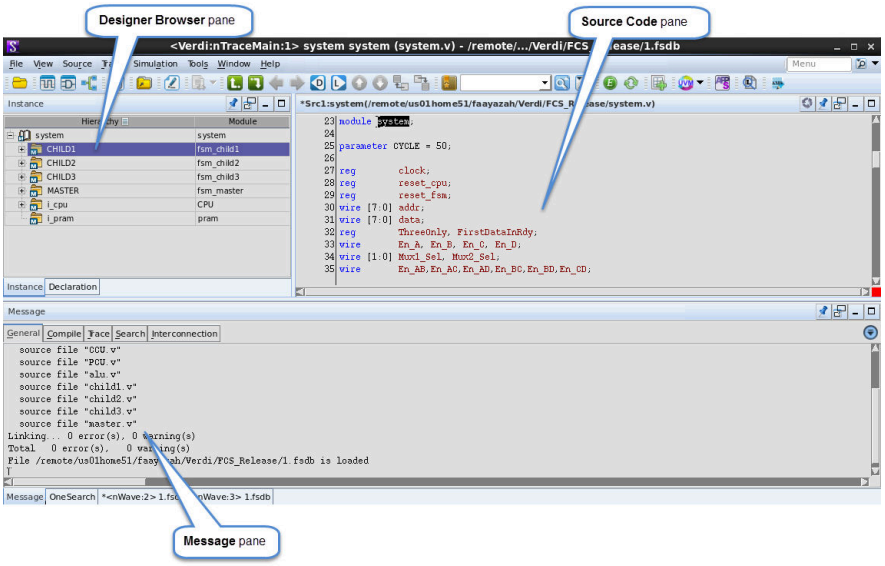
When you start the Verdi platform, the *nTrace* main window displays and serves as the main window from which other frames/windows are created. When you import a new design into the *nTrace* main window (using the **File -> Import Design** command), the Verdi platform closes existing *nWave* and *nSchema* panes/windows started from the open session.

The *nTrace* main window contains three resizable panes:

- Design browser pane
- Source code pane
- Message pane

An example *nTrace* main window is shown in the following figure:

Figure 37 nTrace Example Window



When you open a design with the Verdi platform, the HDL source code of the top-level unit is displayed in the *Source Code* pane.

The top-level unit is shown as the root of the design hierarchy in the design browser (refer to the nTrace Design Browser Pane following section).

The message pane reports errors or other information related to the Verdi platform's operation.

Feedback

nTrace Design Browser Pane

Located on the left side of the *nTrace* main window, the *Design Browser* pane displays the design hierarchy and provides a way to navigate through the hierarchy (see *nTrace Example Window* figure above).

The *Instance* tab in the *nTrace Design Browser* pane consists of the following two columns:

- Hierarchy
- Module

The **Hierarchy** column can be sorted in ascending, descending, or type order by clicking the heading of the column. By default, the columns are sorted in the type order.

The **Module** column can be sorted in ascending or descending order by clicking the heading of the column.

You can also filter and search the *Instance* tab for a specific node. See *Design Browser Frame Right-Click Options* section in the *Verdi and Siloti Command Reference Manual* for more information.

Limitation:

Verdi supports a maximum of 520 hierarchy layers.

This window contains the following icons and symbols:

Symbol	Name	Description
⊞ ⊟	PlusMinus	Click these symbols to either expand (plus) or collapse (minus) the display of the selected unit's hierarchy.
	Opened-folder	Indicates that the relevant design scope is active and the related source code is displayed in the Source Code frame. The letter on the folder is a mnemonic for the scope type, such as M for module, L for library, T for task, and F for function.
	Closed-folder	Indicates the relevant design scope is non-active. As in the **Opened-folder** symbol, the letter on the closed-folder symbol indicates the type of design scope.
	Icon with a bookmark	Indicates the relevant design scope is set with a bookmark.

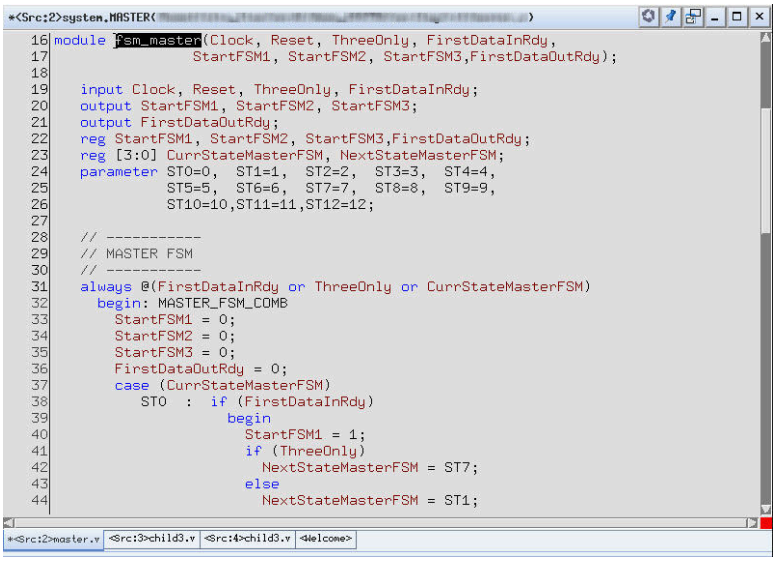
Feedback

Symbol	Name	Description
	Highlighted Design Scope	The highlighted design scope is selected and acting as the target scope for further relevant operations in the design browser.

nTrace Source Code Pane

The source code pane appears on the right side of the *nTrace* main window. Multiple source code files can be displayed as multiple tabs. If a module is described in multiple source files, multiple tabs are opened to display the complete source code when the node is set as an active scope. Each tab is undockable.

Figure 38 nTrace Multiple Source Code Tabs



The source code view displays the source code for the active unit in the design browser. This window is divided into the following two areas:

- Source Code Area
- Indicator Area

Source Code Area

The source code area contains the HDL source code. The Verdi platform color-codes the source code to differentiate syntax elements. You can set the syntax colors to your preferences. Some colors change during debugging. For example, signals that are traced are displayed in green to highlight the trace history. You can reset all the traced signals' colors to their default settings using the **Trace -> Reset Traced Signal's Color** command.

Feedback

Indicator Area

The indicator area contains line numbers and graphical indicators that result from load tracing, driver tracing, connectivity tracing, and bookmarking. The following table describes the symbols used in the indicator area:

Symbol	Definition
	Driver - result from last trace command. Multiple drivers are possible.
	Active driver - selected driver from last active trace command or current *Show* command.
	Load - result from last trace command. Multiple loads are possible.
	Possible driver - result from the possible analyzed traced result.
	Unanalyzed driver - result from the unanalyzed traced result.
	Bookmark - marks a selection for easy referral.
	Pass through load - result from the pass-through driver traced results.
	Pass through driver - result from the pass-through load traced results.
	Pass through connectivity - result from the pass-through connectivity traced results.

The indicator area also shows interactive simulation controls such as breakpoints and current active statement arrows.

nTrace Message Pane

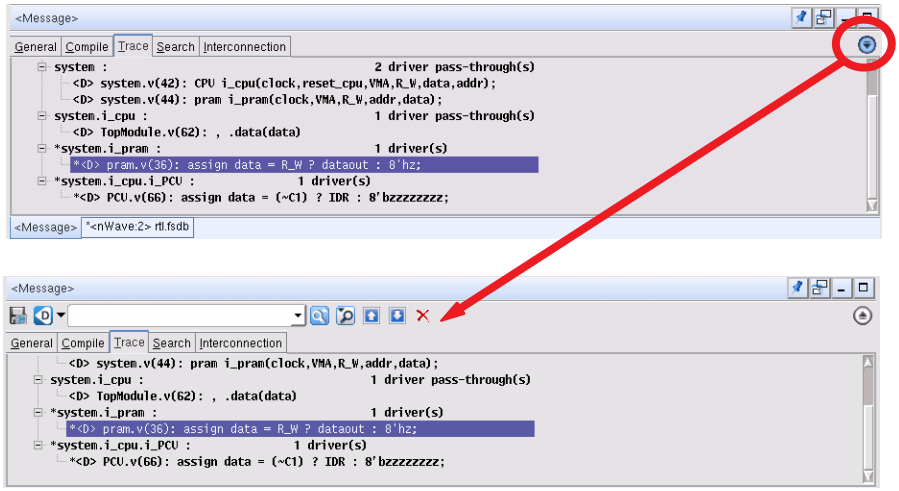
The message pane at the bottom of the *nTrace* main window contains *General*, *Compile*, *Trace*, *Search*,and*Interconnection* tabs.

You can drag-and-drop a signal from the *Source Code* pane to the *Trace* or the *Search* tab to list the results of **Trace Driver** or the search results respectively.

Feedback

A *Find* field appears above the message tabs when the **Find** toolbar icon is clicked. Refer to the *Message Frame* section in the *nTrace* chapter of the *Verdi and Siloti Command Reference Manual* for details.

Figure 39 nTrace Find Bar on the Message Frame



nTrace Toolbar Icons

Refer to the *Toolbar Icons and Fields* section in the *nTrace* chapter of the *Verdi and Siloti Command Reference Manual* for information regarding available toolbar icons.

Note:

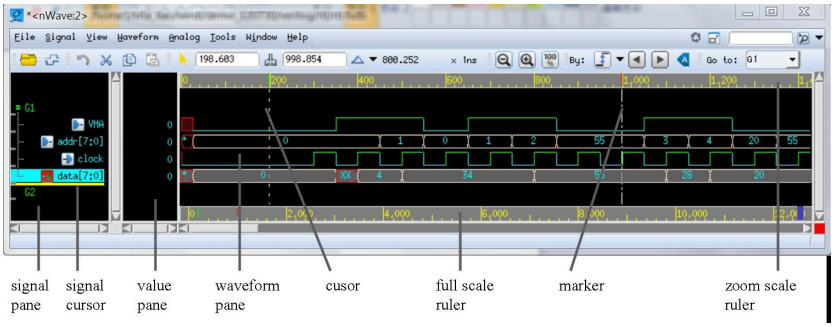
The default toolbar can be modified through the **Tools -> Customize Menu/ Toolbar** command.

nWave User Interface

You can open a new *nWave* pane from the *nTrace* main window by clicking the **New Waveform** icon or choosing the **Tools -> New Waveform** command. The *nWave* pane is released from the main window to become a standalone window by clicking the **Undock** toolbar icon. An example *nWave* standalone window is shown in the following figure.

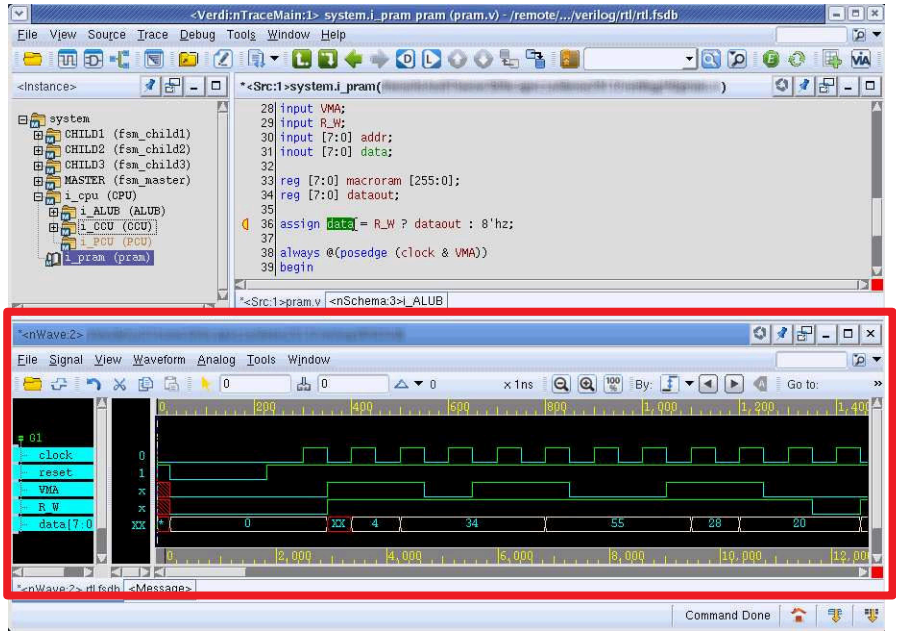
Feedback

Figure 40 Example nWave Standalone Window



An example docked *nWave* pane is shown in the following figure.

Figure 41 Example nWave Dock Pane



The *nWave* pane/standalone window consists of three resizable subwindows (also known as panes):

- Signal pane
- Value pane
- Waveform pane

Feedback

This section consists of the following topics:

- nWave Signal Pane
- nWave Value Pane
- nWave Waveform Pane
- nWave Toolbar Icons
- Get Signals
- nWave Mouse Operations

nWave Signal Pane

The *Signal* pane displays signals and group names on the left side of the *nWave* display. You can use the *Signal* pane to select and manipulate signals and groups of signals. Three types of objects appear in the *Signal* pane:

- Signal name
- Signal cursor
- Group name

Signal Name

A signal name appears to the left of its waveform. In addition to identifying the waveforms, the signal names are selectable areas; clicking on a signal name selects that signal for manipulation. The signal name can be displayed as either a full hierarchical name or a local name. By default, *nWave* right-justifies the signal name. However, you can change the justification. If a name is too long, use the horizontal scroll bar or adjust the window size to see the entire name.

Signal Cursor

The signal cursor marks the insertion point for signal commands: **Add**, **Move**, **Paste**, **Overlay Signals**, and **Create Bus**. Middle-click to set the signal cursor.

Group Name

You can place similar signals in the same group. The group name can be changed from the default of G1, G2, and so on.

nWave Value Pane

The *Value* pane is next to the *Signal* pane and displays the value of each signal at the cursor time in the *Waveform* pane. You can select the display format for signals. For

example, they can be displayed as hex, octal, binary, decimal value, or user-defined alias text.

Preferences for what is displayed (for example, leading zeros, marker value), can be set through the **Value pane** menu or the **Tools -> Preferences** command.

For any value change of a signal, *nWave* displays the old value to the new value in the *Value* pane indicating that the value is changed from 0 to 1 or 1 to 0. If the value (such as the value change of the long bus value) is not fully visible due to the width of the *Value* pane, move the cursor on top of that value in the *Value* pane, and the value is displayed in the tip window.

---

nWave Waveform Pane

The *Waveform* pane appears to the right and displays the waveforms. In addition, the *Waveform* pane contains the following objects:

- Cursor
- Marker
- Zoom scale ruler
- Full scale ruler

Cursor

The cursor is used to show the current simulation time for all windows and to provide one endpoint for delta time calculations. To set the cursor, left-click the mouse button. The toolbar displays the cursor time.

Note the following when setting the cursor:

- The setting affects the time display (and, therefore, the results) in all panes/windows that display values.
- If you click inside the *Waveform* pane and choose the **Waveform -> Snap Cursor to Transitions** command (“s” key), the cursor can only be set where there is a signal transition.
- If you de-select the **Waveform -> Snap Cursor to Transition** command, you can set the cursor to any location.

Marker

The marker is used to provide the second point of a delta calculation. To set the marker, click the middle mouse button. The toolbar displays the amount of time between the cursor and the marker (the delta time).

Note the following when setting the marker:

- If you click inside the *Waveform* pane and choose the **Waveform -> Snap Cursor to Transitions** command (“s” key), the marker can only be set where there is a signal transition.
- If you de-select the **Waveform -> Snap Cursor to Transition** command, you can set the marker to any location.
- If you choose the **Waveform -> Fix Cursor/Marker Delta Time** command (“x” key), cursor or marker is spaced at the same delta time.
- If you de-select the **Waveform -> Fix Cursor/Marker Delta Time** command, cursor or marker is not spaced at the same delta time.

Note:
After you set the cursor time and the marker time, right-click to zoom and fit the waveform display to the time range between the cursor and the marker times.

Zoom-Scale Ruler

The zoom-scale ruler appears at the top of the *Waveform* pane and displays the current displayed time range.

Full-Scale Ruler

The full-scale ruler appears at the bottom of the *Waveform* pane. This ruler displays the time range of all the results (not just the displayed portion) and indicates where the cursor and marker positions are in this range. You can change cursor and marker times by clicking on the full-scale ruler. Selecting a range (dragging with the left mouse button) zooms the display so that the selected area zooms the display in the *Waveform* pane to the selected area.

---

nWave Toolbar Icons

Refer to the *Toolbar Icons and Fields* section in the *nWave* chapter of the *Verdi and Siloti Command Reference Manual* for information regarding available toolbar icons.

Note:
The default toolbar can be modified through the **Tools -> Customize Menu/ Toolbar** command.

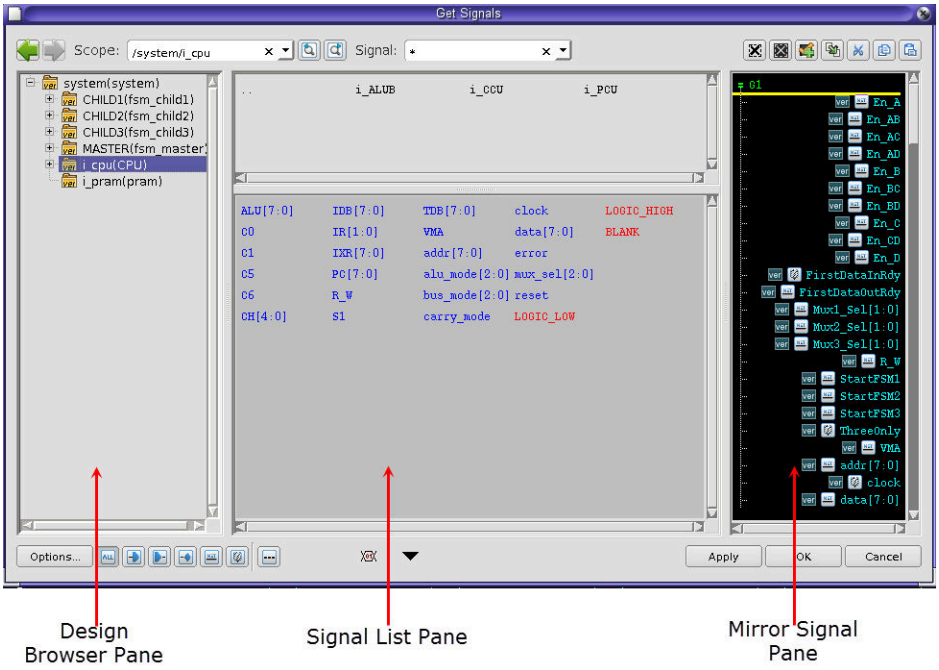
Get Signals

The *nWave* window does not display any signals by default; signals are added by dragging from other windows or by selection in the *Get Signals* form (using the **Signal -> Get**

Feedback

Signals command). Signals are displayed hierarchically based on the design unit selected in the design hierarchy box on the left side of the form.

Figure 42 nWave Get Signal Form



To select signals, navigate the design tree in the *Design Hierarchy* pane to find the required signals, then either drag them to the mirror signal pane in the right-side pane (which mirrors the signals in the *Waveform* pane) or select the signals and click **Apply**. When you have selected all of the signals of interest, click **OK**.

Note:

The design hierarchy of the simulation files may not match that of the currently opened design source. You can display waveform data independent of the design that is loaded.

The mirror signal pane allows you to manipulate the arrangement of the signals displayed in the *Get Signals* form without immediately affecting the waveform pane. After finishing the signal arrangement, click **Apply** to synchronize the waveform pane. Click **OK** to apply the arrangement and close the form.

Refer to the **Get Signals** command description in the *nWave* chapter of the *Verdi and Siloti Command Reference Manual* for details.

Feedback

nWave Mouse Operations

The following tables list the *nWave* mouse actions:

Mouse Action	nWave - Signal Pane
Left-click	De-select the current selected signals/group and select the signal under the mouse button.
Left-click drag-and-drop	Select the object first and then drag-and-drop the object to the new location using the left mouse button on the indicated item.
Left-click the plus/minus icon of a group containing signals	Unhide/hide the signals of the selected group.
Left-click a Group	De-select the current selected signals/group and select the group under the mouse button.
Middle-click	Set the Signal Cursor position for the destination of commands: Move, Paste, Add, Overlap and Create Bus.
Right-click	Open a context-sensitive menu that provides some commands, which apply to the signal/group under the mouse button.
Double-click a bus	Expand or collapse bus member.
Double-click a power domain signal	Expand to three member signals to display value changes for power state, power nominal (for CPF; power nominal is power alias when a UPF file is loaded), and power voltage.
Double-click an interface subgroup	Expand or collapse the node.
Drag & Drop	Move the selected signals to the Signal cursor position. **Note:** The Signal Cursor position is moved along with the dragged mouse pointer.
Drag & Drop a signal to a schematic frame/window	Display the schematic in which the signal is found and select it.
Drag & Drop a signal to a source frame/window	Trace signal's connectivity and highlight the result by symbols in the indicator area of the source code pane.

Mouse Action	nWave - Signal Pane
Drag & Drop a signal to a Temporal Flow View	Highlight the corresponding signal if it exists. Add the signal and driving instance as a reference if it doesn't exist. The global cursor time is used to identify the signal.
Drag-left	Area selection for multiple signals.
Drop an interface signal	Adds an expanded subgroup node with all interface signals at current position.
Shift-left-click a signal	Add to selection list for multiple signal selection.

Mouse Action	nWave - Value Pane
Right-click on bus or signal	Open a context-sensitive menu that provides some commands, which apply to a bus (such as Radix, Notation) or a signal (such as Edit Alias, Remove Alias).

Mouse Action	nWave - Waveform Pane
Left-click	Set the Cursor position.
Middle-click	Set the Marker position.
Right-click	Zoom to time range between the Cursor and Marker position.
Double-click	Find the signal's driver statements in source code frame.
Drag & Drop	Move the selected signals to the Signal cursor position. **Note:** The Signal Cursor position is moved along with the dragged mouse pointer.
Drag & Drop a signal to a schematic frame/window	Display the schematic in which the signal is found and selected.
Drag & Drop a signal to a Temporal Flow View window	Highlight the corresponding signal if it exists. Add the signal and driving instance for the current cursor time if it doesn't exist.

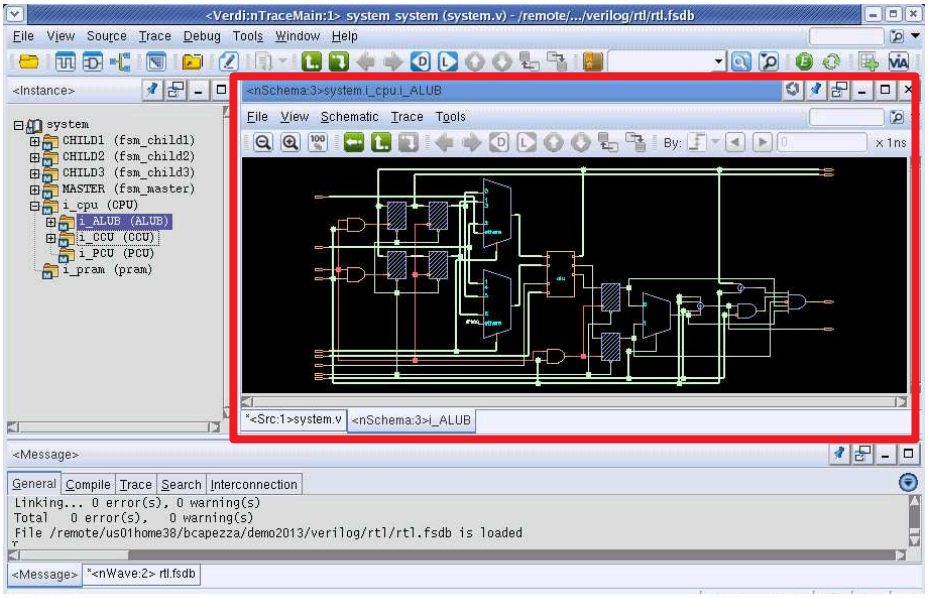
Mouse Action	nWave - Waveform Pane
Drag & Drop a signal to a source frame	Trace signal's connectivity and highlight the result by symbols in the indicator area of the source code frame.
Drag-left horizontally on a waveform window, full scale ruler and zoom scale ruler	Zoom into the time range of the dragged time interval.
Drag-left vertically on an analog signal	Zoom into the value range of the dragged value interval.
Ctrl + Mouse Wheel	Zoom-in or zoom-out the time range.
Right-click a signal waveform	Open a context-sensitive menu that shows Temporal Flow View debug commands (that is, Temporal Flow View, Trace This Value, Show Fan-in, and so on).

nSchema User Interface

You can open a new *nSchema* pane from the *nTrace* main window by clicking on the **New Schematic** icon or using the **Tools -> New Schematic from Source -> New Schematic** command. The schematic for the active unit in the design browser frame (*nTrace*) is displayed in the *nSchema* frame, as shown in the following example.

Feedback

Figure 43 Example nSchema Frame



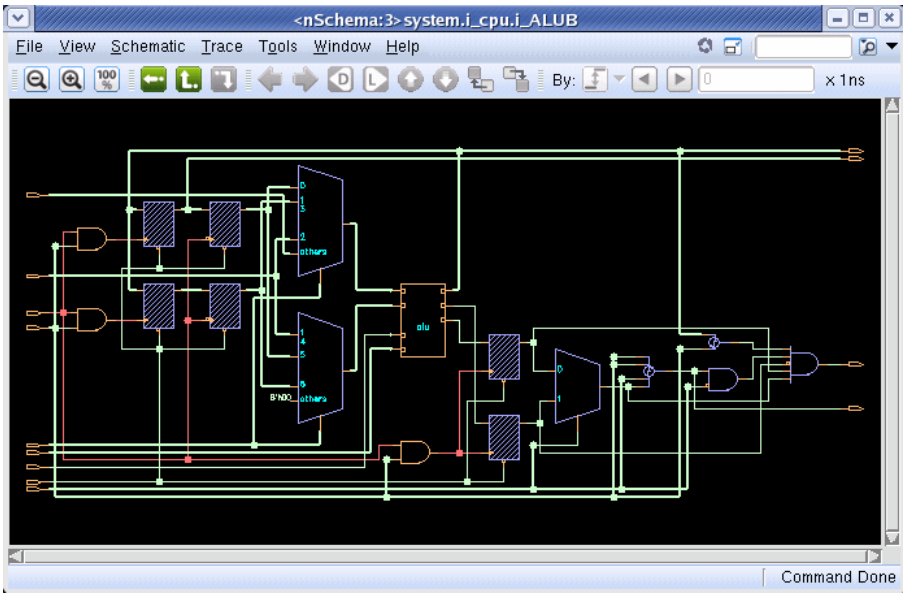
The schematic window displays the schematic generated from the corresponding HDL source code and provides another design view for debugging. You can debug the design using menu commands or mouse operations.

In *nSchema*, VDD, VCC, VEE, POWER and PWR net names are treated as supply nets and VSS, GND and GROUND net names are treated as ground nets. These nets are case insensitive. When a signal is treated as a power/ground global signal, trace actions are skipped.

The *nSchema* pane is released from the main window to become a standalone window by clicking the **Undock** toolbar icon. The *nSchema* standalone window is shown in the following figure.

Feedback

Figure 44 nSchema Standalone Window



For more information on how to dock and undock panes to customize the layout of your workspace, click the following video link:

This video takes you through the capability of docking and undocking panes. You can undock a pane by dragging the pane banner (press and hold the middle mouse button) and dropping it outside the main window. It works as a standalone window. When you click the **Dock** icon on the standalone window, the window is docked as a pane in the main window.

nSchema Toolbar Icons

Refer to the *Toolbar Icons and Fields* section in the *nSchema* chapter of the *Verdi and Siloti Command Reference Manual* for information regarding available toolbar icons.

Note:

The default toolbar can be modified through the **Tools -> Customize Menu/ Toolbar** command.

nSchema Mouse Operations

The following table lists the *nSchema* mouse actions.

Feedback

Mouse Action	Schematic Window
Left-click a signal/instance	De-selects the current selection and select the signal/instance.
Left-click drag-and-drop	Select the object first and then drag-and-drop the object to the new location using the left mouse button on the indicated item.
Shift-left-click a signal/instance	Adds the signal to a selection list for multiple signals/instances selection.
Left-click anywhere without a signal/instance	De-selects all.
Drag-left	Zooms in an area.
Right-click	Opens a context-sensitive menu.
Double-click a signal	Highlights the connection (driving instance to loading instances with the connecting net) for the selected signal.
Double-click an instance	Pushes view into the schematic for the instance.
Drag & Drop an instance to a waveform frame/window	Displays the corresponding instance's I/O signal waveform.
Drag & Drop an RTL block to a waveform frame/window	Displays the corresponding RTL block's I/O signal waveform.
Drag & Drop a signal to a waveform frame/window	Displays the corresponding signal's waveform.
Drag & Drop an instance to a source frame/window	Finds and highlights the associated instance in source code pane/window.
Drag & Drop an RTL block to a source frame/window	Finds and highlights the corresponding source code of the RTL block.
Drag & Drop an instance / RTL block to a Temporal Flow View window	Highlights the corresponding instance's output signal if it exists. Adds the instance as a reference if it doesn't exist. The global cursor time is used to identify the signal.
Drag & Drop a signal to a Temporal Flow View window	Highlights the corresponding signal if it exists. Adds the signal and driving instance as a reference if it doesn't exist. The global cursor time is used to identify the signal.

Feedback

Mouse Action	Schematic Window
Drag & Drop a signal to a source frame/window	Traces the signal's connectivity in the source code pane/window.
Drag & Drop any state from nSchema to nState	When you drag an FSM block from a schematic pane/window to a *nState* window, *nState* displays the state diagram of that FSM block.
Drag & Drop any state from nSchema to nWave	When you drag an FSM block from a schematic pane/window to a *nWave* pane/window, *nWave* adds all the I/O and state signals of that FSM block to the location of the cursor bar in the *nWave* pane/window.
Ctrl + Mouse Wheel	Zooms-in or zooms-out an area.
Ctrl + Drag-left	Pans the *nSchema* window.

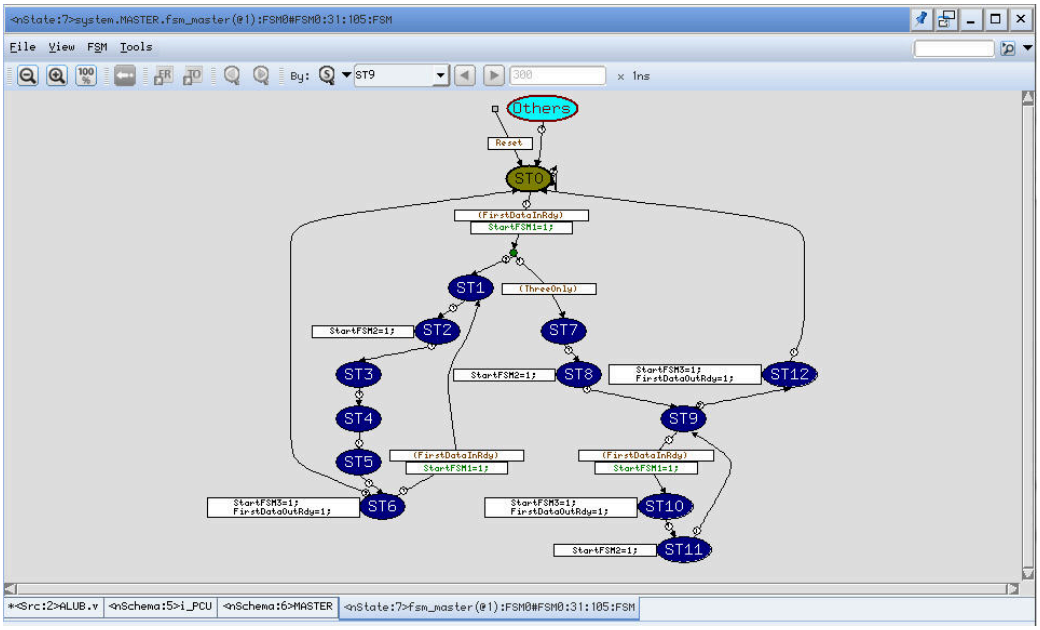
nState User Interface

To open a *nState* frame, double-click the finite state machine (FSM) symbol (see left) in the *nSchema* pane/window. The *nState* pane is released from the main window to

Feedback

become a standalone window by clicking the **Undock** toolbar icon. An example *nState* window/pane is shown in the following figure:

Figure 45 Example nState Frame



The *nState* window displays the generated bubble diagram for the corresponding state machine and provides another design view for debugging and understanding your finite state machine. You can debug your finite state machine using the menu commands or mouse actions in this window.

nState Toolbar Icons

Refer to the *Toolbar Icons and Fields* section in the *nState* chapter of the *Verdi and Siloti Command Reference* manual for information regarding available toolbar icons.

Note:

The default toolbar can be modified through the **Tools -> Customize Menu/ Toolbar** command.

nState Mouse Operations

The following table lists the *nState* mouse actions.

Feedback

Mouse Action	nState Window
Right-click a transition in a *nState* window	A transition-context-sensitive menu opens for the commands: **Jump to From State**, **Jump to To State**, **Fit Select Set**, **Transition Condition**, and **Properties**.
Left-click drag-and-drop	Select the object first and then drag-and-drop the object to the new location using the left mouse button on the indicated item.
Right-click a state in a nState window	A state-context-sensitive menu opens for the commands: **State Action**, **Fit Select Set**, and **Properties**.
Right-click the white space in a nState window	A finite-state-machine-context-sensitive menu opens for the commands: **Zoom All**, **Last View**, **Edit Search Sequence**, **Print**, and **Properties**.
Double-click a port in a nState window	If there are two ports, a properties dialog box opens to select the state. If there is only one port, go to the state properties directly.
Ctrl + Drag-left	Pans the *nState* window.
Drag & Drop any state or transition from nState to nSchema	When you drag any state or transition from inside a *nState* window to a *Schematic* window, *nSchema* displays the schematic with the corresponding FSM block whose state diagram is shown in that *nState* window.
Drag & Drop any state or transition from nState to nTrace	When you drag any state or transition from inside a *nState* window to the *Source code* pane, the corresponding source code is highlighted.
Drag & Drop any state or transition from nState to nState	When you drag a state or transition from one *nState* window to another *nState* window, the target *nState* window displays the same state diagram as in the source *nState* window; that is, the two *nState* windows are synchronized. The same state or transition is highlighted in both windows.
Drag & Drop any state from nSchema to nState	When you drag an FSM block from a *nSchema* window to a *nState* window, *nState* displays the state diagram of that FSM block.

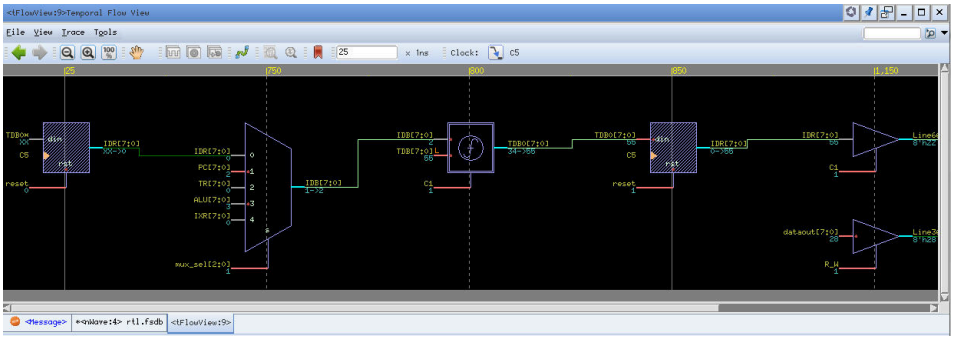
Flow View User Interface

The *Flow View* pane can be invoked from the *nTrace* main window or *nWave* pane through the **Temporal Flow View -> New Temporal Flow View** command. A *Flow View* pane

Feedback

can be released from the main window to become a standalone window by clicking the **Undock** toolbar icon. An example of a *Temporal Flow View* pane/window is shown in the following figure.

Figure 46 Example Temporal Flow View Window



The *Flow View* window displays a generated view of your design over time, starting from the selected reference signal and time. This provides another view in which to debug your design using the menu commands or mouse actions.

From a *Temporal Flow View* window, you can open the *Temporal Register Flow View* or the *Compact Temporal Flow View*. Refer to the *Verdi and Siloti Command Reference Manual* for detailed information regarding these views.

For more information on how to create a temporal flow view and how you can identify the root cause of the displayed value, click the following video link:

This video takes you through the process of creating a temporal flow view and how you can identify the root cause of the displayed value.

Flow View Toolbar Icons

Refer to the *Toolbar Icons and Fields* section in the *Flow View* chapter of the *Verdi and Siloti Command Reference Manual* for information regarding available toolbar icons.

Note:

The default toolbar can be modified through the **Tools -> Customize Menu/ Toolbar** command.

Feedback

Flow View Mouse Operations

The following table lists the *Flow View* mouse actions:

Mouse Action	Temporal Flow View
Left-click a signal or instance or instance pin	De-selects the current selection and selects the signal/instance/port.
Left-click drag-and-drop	Select the object first and then drag-and-drop the object to the new location using the left mouse button on the indicated item.
Ctrl-left-click a signal/instance	Adds the signal/instance to the selection for multiple signals/instances selection.
Left-click anywhere without a signal/instance	Deselects all.
Drag-left in main display area (pan mode)	Pans left, right, up, or down
Drag-left in main display area (pointer mode)	Zooms in area.
Drag-left on time ruler	Zooms in area.
Right-click instance or instance pin	Opens a context-sensitive menu.
Double-click an instance pin	Traces the signal's drivers.
Drag & Drop an instance to a waveform window	Displays the corresponding instance's I/O signal waveform.
Drag & Drop an instance pin to a waveform window	Displays the corresponding signal's waveform.
Drag & Drop an instance to a source window	Finds and highlights the source code associated with the instance.
Drag & Drop an instance pin to a source window	Traces the signal's connectivity in the source code frame.
Drag & Drop an instance pin to nSchema window	Changes the scope to the signal's hierarchy and highlights the corresponding signal.
Drag & Drop an instance to nSchema window	Changes to the instance's hierarchy and highlights the corresponding instance.

Feedback

Mouse Action	Temporal Flow View
Left-click an instance with nWave icon enabled	Adds the instance IO to nWave if they don't exist. Highlights the instance output if it exists.
Left-click an instance output port with Show Source Code icon enabled	Finds and highlights the source code associated with the output signal.
Left-click an instance with Show Source Code icon enabled	Finds and highlights the source code associated with the instance.
Ctrl + Mouse Wheel	Zooms-in or zooms-out an area.

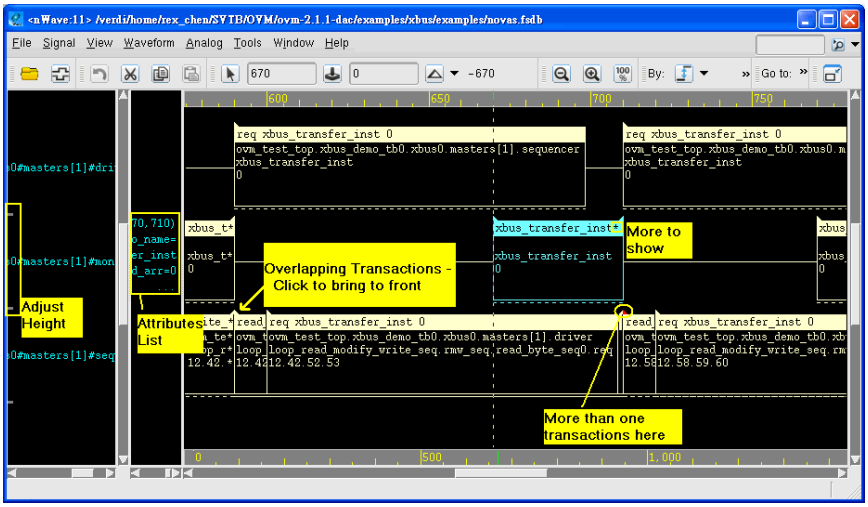
Transaction/Message User Interface

A transaction or message FSDB file is loaded into nWave using the same process as a general FSDB file.. A stream name is shown in the *Signal* pane; begin time, end time, and attributes are shown in the *Value* pane. The transaction/message is shown in the *Waveform* pane as rectangles enclosing all the attributes.

Detailed Transaction/Message View in nWave

The following figure summarizes the various aspects of transaction and message viewing in *nWave*.

Figure 47 Detailed Transaction/Message View



Feedback

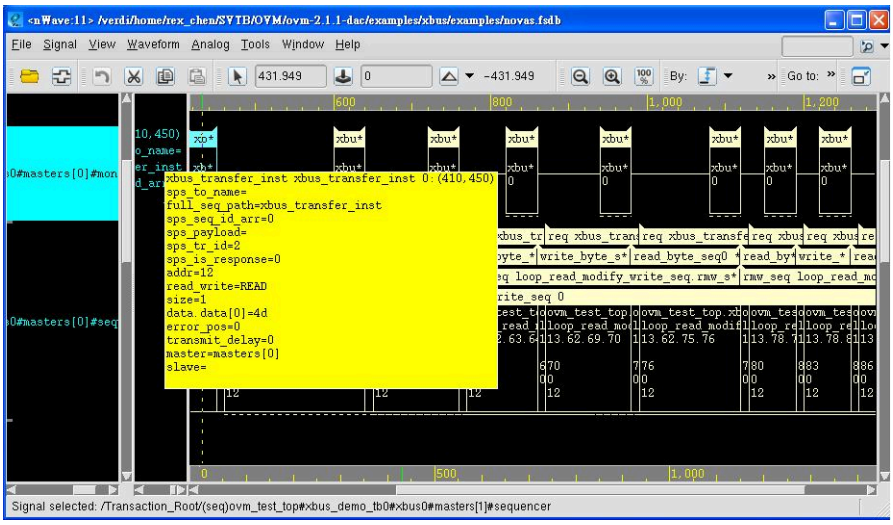
A transaction or message has a begin time and an end time. When you click a message or transaction, the cursor is positioned at the begin time. To step through the transactions and messages on the *nWave* toolbar, perform the following steps:

1. Select a stream.
2. Click the ***Search Backward*** or ***Search Forward*** icon (left or right arrows).

A dashed line under the transaction/message box indicates the availability of more attributes than are displayed. To view additional (or fewer) attributes, increase (or decrease) the height of the stream in the **Signal** pane.

Alternatively, move the cursor on top of the transaction or message attributes in the ***Value*** pane (middle column). Doing so activates a yellow tip window that shows all attributes as shown in the following figure:

Figure 48 Transaction/Message Tip



To select individual transactions or messages, click the label in the ***Waveform*** pane. Doing so changes the background color of the selected transaction or message.

To change the waveform cursor time, click the **Search Backward** and **Search Forward** toolbar icons.

Selecting individual transactions or messages in necessary to view the messages that are covered or obscured due to a time overlap with other messages.

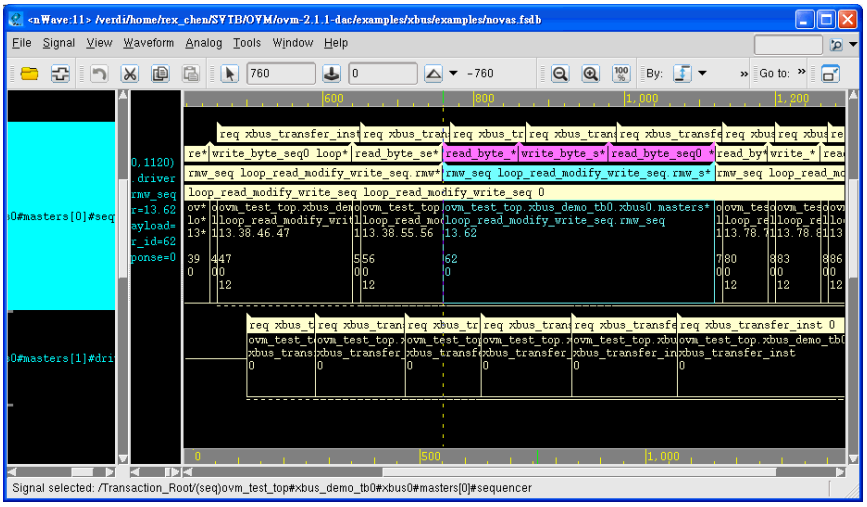
Feedback

To view the underlying transaction or messages, select the top triangle to bring the message to the front. Alternatively, to remove an overlap, select a stream and click on any of the following commands:

- **Waveform > Classic Transaction > Expand/Shrink Overlapping**
- **Waveform > Classic Message > Expand/Shrink Overlapping** .

Transactions or messages related to the selected transaction or message are highlighted with a pink background as shown in the following example:

Figure 49 Transaction/Message Relationships



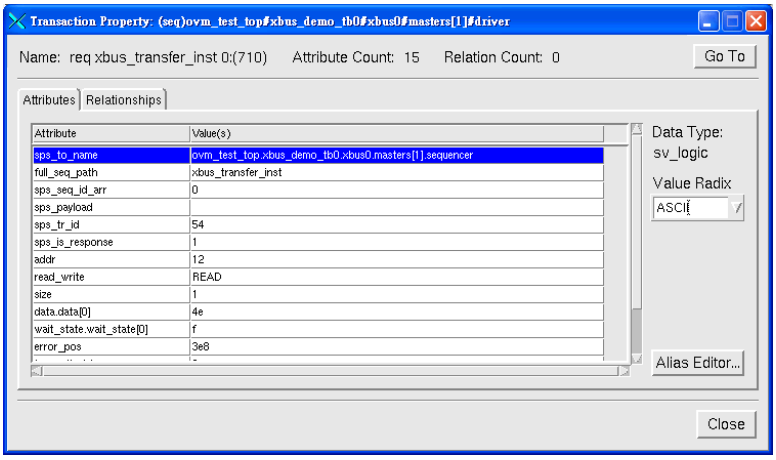
Transaction/Message Properties

Transactions/Messages contain a lot of data. You can view the attributes and relationships of a selected transaction/message in a tabular format. Perform the following steps to open the *Transaction Property* or *Message Property* form.

- Select a transaction or a message
- Right-click to open the context menu
- Choose the **Properties** command.

Feedback

Figure 50 Transaction Property Dialog Window - Attributes



You can view the relationships of the selected transaction by clicking the **Relationship** tab in the *Transaction Property* form.

Transaction/Message Attributes

You can use string matching to search for attributes.

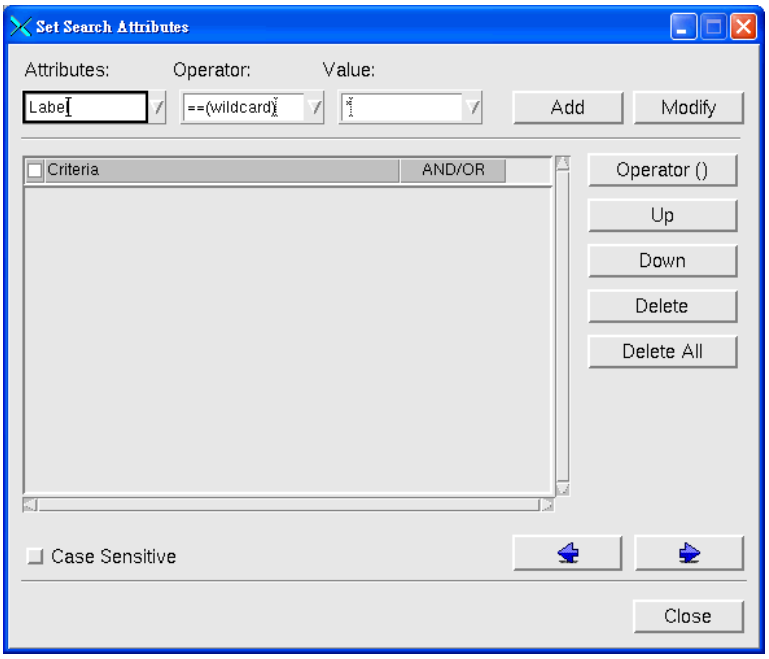
Perform the following steps In *nWave*:

1. In nWave, to open the Search Attribute Value form, click the ***Waveform -> Set Search Attributes*** command. Alternatively, left-click the **Search By**: icon on the toolbar and select the **Transaction Attribute Values** option.
2. Specify the search criteria, such as attribute name and value.
3. Click OK.

   You can use the Search Forward/Search Backward icons on the nWave toolbar to step through the transactions/messages of the selected streams.

Feedback

Figure 51 Search Attribute Value Form



You can specify the attribute name and value. After you've entered the search criteria and clicked **OK**, you can use the **Search Forward/Search Backward** icons on the *nWave* toolbar to step through the transactions/messages of the selected streams.

Analyzing Transactions/Messages

You can analyze transaction and messages by using the *Transaction Analyzer* window. To do so, click one of the following commands from nWave:

- **Tools -> Classic Transaction -> Analysis Window** command.

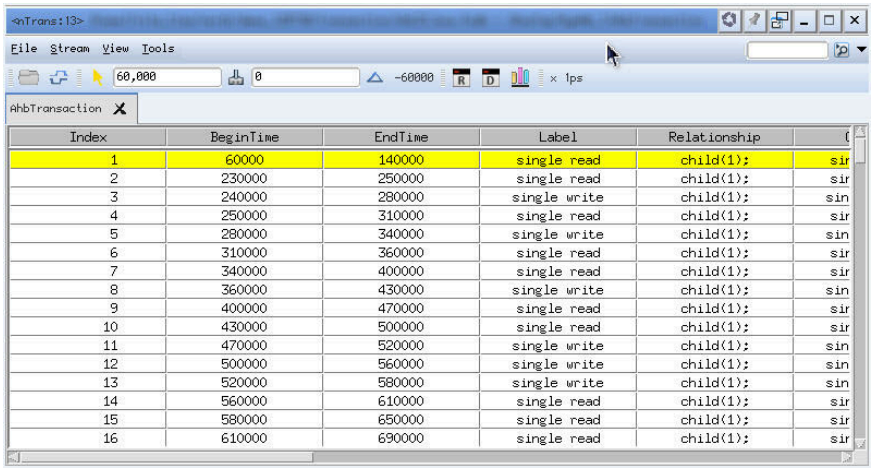
  Or,

- **Tools -> Classic Message -> Analysis Window** command) from *nWave*.

Feedback

The window is similar to the following:

Figure 52 Transaction Analyzer Window



Index	BeginTime	EndTime	Label	Relationship
1	60000	140000	single read	child(1);
2	230000	250000	single read	child(1);
3	240000	280000	single write	child(1);
4	250000	310000	single read	child(1);
5	280000	340000	single write	child(1);
6	310000	360000	single read	child(1);
7	340000	400000	single read	child(1);
8	360000	430000	single write	child(1);
9	400000	470000	single read	child(1);
10	430000	500000	single read	child(1);
11	470000	520000	single write	child(1);
12	500000	560000	single write	child(1);
13	520000	580000	single write	child(1);
14	560000	610000	single read	child(1);
15	580000	650000	single read	child(1);
16	610000	690000	single read	child(1);

For the current selected stream (or merged streams), you can use the **View -> Search** command to locate a string or pattern.

You can also use the **View -> Filter/Colorize** command to filter and display transactions/messages whose attributes match user-specified conditions.

These commands allow you to navigate the streams quickly and focus on the transactions/messages of interest.

After clicking the **Sync. Signal Selection Enabled** icon (see left) on both the *Transaction Analyzer* pane and the *nWave* pane, you can select a transaction/message in the spreadsheet view and the corresponding transaction/message gets selected in the waveform

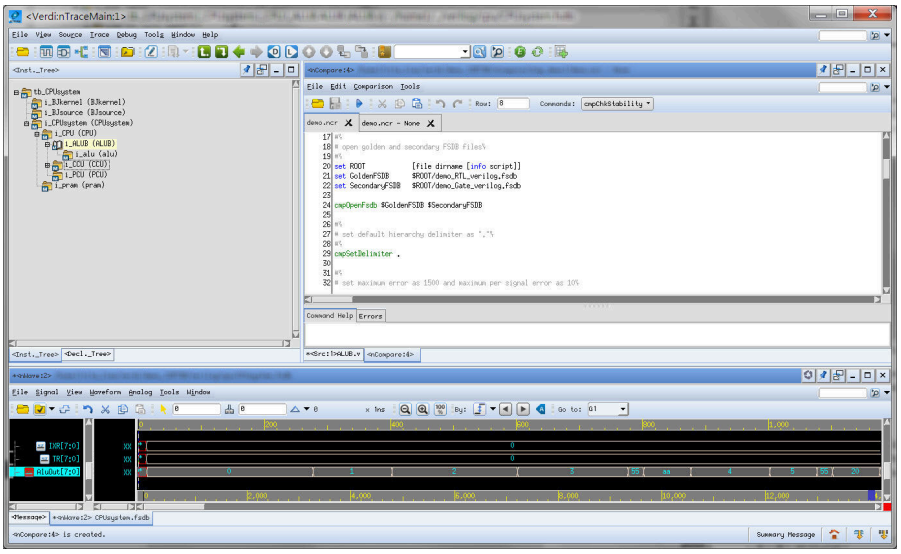
nCompare User Interface

The *nCompare* pane compares simulation results stored in FSDB dump files using flexible, user-specified comparison criteria. It is optimized for rapid comparison of large data sets and is seamlessly integrated with the Verdi platform, allowing you to easily identify and display differences between simulation runs.

Feedback

The *nCompare* pane can be accessed by selecting the **Tools -> nCompare** command from the *nWave* pane. Once the pane is open and the waveform comparison is finished, the *nCompare* pane is displayed as shown in the following figure:

Figure 53 nCompare Pane



Comparing Different Simulation Runs

The *nCompare* pane is used to identify mismatch simulation errors between various simulation runs, such as those occurring between pre-synthesis and post-synthesis stages, different clock speeds of the same design, different technologies, or simulation files produced by different simulators.

Rule File

The rule file is described using Tcl language. The *nCompare* module uses Tcl language and nCompare-defined-Tcl-extended comparison commands to describe the comparison rules and specify comparison options.

A basic rule file should have at least the following three parts:

1. Specification of golden and secondary simulation files.
2. Specification of compared signal pairs.
3. Start time-based comparison.

The following is a simple rule file that would compare all signals in *1.fsdb* and *2.fsdb*:

```
cmpOpenFsdbcmpOpenFsdb 1.fsdb 2.fsdb
cmpSetSignalPaircmpSetSignalPair top -level 0
cmpCompare
```

Compare Waveforms and View Errors

Once the rule file is created and the comparison is executed either in the GUI (Graphical User Interface) or using the *nCompare* utility, the *nCompare* pane displays the mismatch errors. These errors can be sorted by design hierarchy or time sequence, allowing for easy navigation.

nCompare Mouse Operations

The default mouse action in the *nCompare* pane is summarized in the following table:

Mouse Action	Command Operations
Double-click a mismatch error node	Double-clicking a mismatch error node in the nCompare pane launches the waveform tool, adds the mismatch signals into the waveform tool, and changes the cursor time of the waveform tool to the mismatch time.

nECO User Interface

The *nECO* module provides the ability to perform gate-level engineering change orders (ECOs) in the flexible schematic views. The *nECO* module takes full advantage of the sophisticated capabilities in the Verdi platform to propagate changes throughout the design hierarchy and automatically create any new nets and ports that are required.

Refer to the *User Interface* chapter of the *nECO User's Guide and Tutorial* for details.

nAnalyzer User Interface

The *nAnalyzer* module provides the ability to analyze clock and reset trees (including crossing paths), to qualify Clock Tree Synthesis (CTS), to annotate standard delay format (SDF) files and CTS results, to load and display timing results from standard timing analysis tools, and to perform switching analysis on the design. These functions build on top of the functional debug aspects of the Verdi platform. The *nAnalyzer* module uses the same interface as *nSchema*.

Refer to the *nAnalyzer User's Guide and Tutorial* for details.

3

Before You Begin

Before you begin the tutorial, you (or your system manager) must have installed the Verdi and Siloti platforms as described in the accompanying *Installation and System Administration Guide*.

This chapter includes the following topics:

- Installation and Setup
- Demo Details

Note:
The optional demo package (for example, Verdi-J-201412-demo.tar.gz where J corresponds to the version, 2014 corresponds to the year, and 12 corresponds to the month) must be installed.

Installation and Setup

You must also complete the following actions to set up the Verdi environment and the files required for this tutorial:

1. Add the Verdi application (binary) to the search path and specify the search path to the license file:

   Refer to the *Setting Up the Environment and Running the Software* section in the *Installation and System Administration Guide* for details.

2. Create a working directory:

   ```
   % mkdir <working_dir>
   ```

3. All of the tutorial data resides in the *$VERDI_HOME/demo* directory. Make a copy of these demo files in your working directory:

   ```
   % cp -r $VERDI_HOME/demo <working_dir>
   ```

Feedback

Demo Details

The primary demo design used in this section is a simple micro-programmed CPU design delivered with the installation. The example represents a complete design spanning the behavioral, RTL, and gate levels.

Most tutorials use the Verilog design demo. However, be sure to check the instructions for each tutorial to ensure that you are running the correct demo that was included with your installation. Use the following commands to set the tutorial data:

- For Verilog Design:

  ```
  % cd <working_dir>/demo/verilog/cpu
  ```

- For VHDL Design:

  ```
  % cd <working_dir>/demo/vhdl/rtl
  ```

- For Mixed Design:

  ```
  % cd <working_dir>/demo/mixed/rtl
  ```

- For SystemVerilog Design:

  ```
  % cd <working_dir>/demo/systemverilog
  ```

- For Transactions:

  ```
  % cd <working_dir>/demo/transaction
  ```

4

Launching Techniques

This chapter describes the following topics:

- Dumping Elaboration Database
- Invoking Verdi
- Runtime Upward of Product License Mode
- Loading Designs With Different Strategies
- Using Verdi Executable to Import Design from UFE
- Reference Source Files on the Command Line
- Compile Source Code into a Library
- Reference Design and FSDB on the Command Line
- Perform Behavior Analysis on the Command Line
- Replay a File
- Start Verdi Without Specifying Any Source Files
- Loading When Design and FSDB Hierarchies do not Match
- Replacing the Absolute Paths in Verdi KDB
- Support of Incremental Dump in vlogan Without PSL
- Getting Relative Path in FSDB
- -kdb=common_elab Supports VCS Dynamic Test Loading

Dumping Elaboration Database

Overview

In previous releases, Verdi elaborated the design at runtime. As a result, loading a complex design took a lot of time and memory. Verdi elaborates the design and saves the

Feedback

elaboration database to the disk in batch mode. The elaboration time and memory usage are reduced significantly by using this new process.

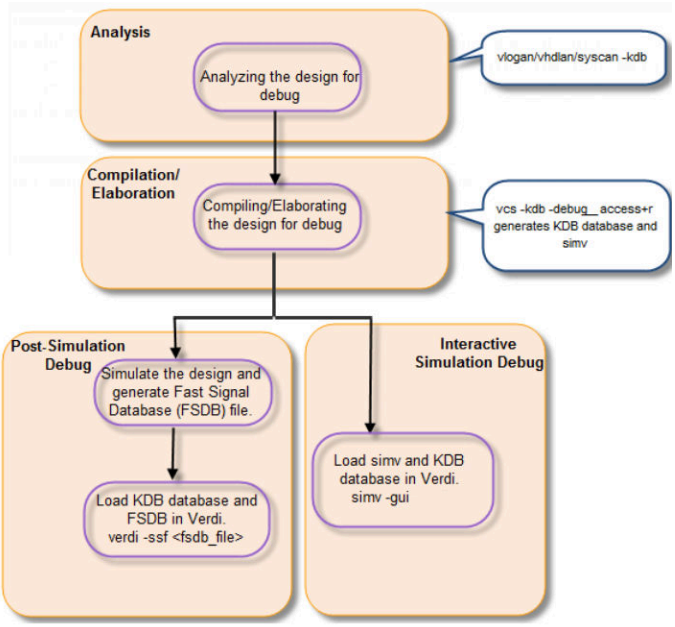
With `-kdb`, VCS invokes elabcom to do Verdi elaboration for generating resolved KDB. With `-kdb=common_elab` option, VCS generates the resolved KDB by using the VCS common elaboration engine in one single flow.

The `-kdb=common_elab` option is also needed in vlogan, because some extra files need to be dumped in the analyzing stage.

Use Model

You can generate the Verdi Knowledge Database (KDB) using the `-kdb` or `-kdb=common_elab` option either in the VCS two-step flow or three-step flow. In the VCS two-step flow, add the `-kdb` or `-kdb=common_elab` option to the vcs command line to generate the KDB. In case of VCS three-step flow, add the `-kdb` or `-kdb=common_elab` option in all the *`vlogan/vhdlan`* command lines and vcs command line. The following flowchart shows the Verdi debug flow:

Figure 54 Verdi Debug Flow



Feedback

Compilation Stage

You can compile the design using VCS and generate both VCS elaboration database and Verdi kdb. Following is the use model:

Using `-kdb` or `-kdb=common_elab` in VCS two-step flow:

```
% vcs -kdb/-kdb=common_elab <compile_options> <source files>
```

Using `-kdb` or `-kdb=common_elab` in VCS three-step flow:

```
% vlogan -kdb/-kdb=common_elab <vlogan_options> <source files>
% vhdlan -kdb/-kdb=common_elab <vhdlan_options> <source files>
% vcs -kdb/-kdb=common_elab <top_name> <compile_options>
```

-kdb=only

Generates only the kdb that is required for both interactive simulation and post-simulation debug with Verdi.

In VCS two-step flow, this option does not generate the VCS compile data/executable, and does not disturb the existing VCS compile data/executables.

Following is the use model in VCS two-step flow:

```
% vcs -kdb=only <compile_options> <source files>
```

Following is the use model in VCS three-step flow:

```
% vlogan -kdb <vlogan_options> <source files>
% vhdlan -kdb <vhdlan_options> <source files>
% vcs -kdb=only <top_name> <compile_options>
```

Limitations:

This option does not support Native Low Power (NLP) flow and PC-NLP flow in the following scenario:

```
% vcs -f run.f -upf file.upf
% ./simv
% vcs -f run.f -upf file.upf -kdb=only
```

In this case, VCS ignores `-kdb=only` and uses `-kdb` to generate Verdi kdb and VCS simulation database, and reports the `KDB-ONLY-NLP` warning message.

To skip kdb elaboration and create a post elaboration script, specify the following argument with the `-kdb/-kdb=common_elab` option:

Note:

It is not recommended to use the `-kdb=only` and `-kdb=postelab` options along with the VCS `-simcopy` option.

Feedback

-kdb=postelab

Tells VCS elaboration process to skip kdb elaboration (running elabcom process) and creates a csh script by name `kdb_postelab.csh`. This script can be used for kdb elaboration separately at later point of time. This option allows you to run elabcom separately.

Following is the use model:

```
% vcs -kdb=postelab <compile_options> <source files>
```

VCS creates a csh script named `kdb_postelab.csh`. You can run this script or use the `-kdb=exec_postelab` compile option as follows to perform kdb elaboration separately at a later point of time.

```
% vcs -kdb=exec_postelab <compile_options> <source files>
```

With `-kdb=exec_postelab`, VCS finds the `kdb_postelab.csh` file in the current working directory and runs the script directly without generating simulation database. Following is the use model:

```
% vcs -kdb=postelab <compile_options> <source files>
% vcs -kdb=exec_postelab <compile_options> <source files>
```

Note:

It is not recommended to use the `-kdb=only` and `-kdb=postelab` options along with the VCS `-simcopy` option.

-kdb=exec_postelab

VCS finds the `-kdb=exec_postelab` file in the current working directory and runs the script directly without generating simulation database.

Following is the use model:

```
% vcs -kdb=postelab <compile_options> <source files>
% vcs -kdb=exec_postelab <compile_options> <source files>
```

-kdb=gatesrc

For gate-level designs, you usually rely on schematic view or waveform to debug.

Source codes are seldom used since the gate-netlists are usually generated by tools that are not human-readable. In previous releases, during compilation, compiler used to spend huge amount of resource (time/diskspace) dealing with the textDoc.

Following are the results:

- Dumping of textDocs to disk for Gate-Level designs does not happen.
- It is expected that the disk size for KDB would be much smaller (30% ~ 50%)

Feedback

Following is the use model:

```
% vlogan -kdb=gatesrc [other options] <design files>
```

Note:
Verdi does not need any additional options while importing the design compiled with `-kdb=gatesrc`.

Limitations:

- When a design containing `vcs_vip` protected content is compiled with the `-kdb=gatesrc` option, only the encrypted content within protected blocks is visible.
- The option `-kdb=gatesrc` is used for gate level designs. If Verdi is launching with RTL designs compiled with the `-kdb=gatesrc` option, then most of the nTrace features on the source window does not work as expected. For example, the features requiring full source code visibility may be unavailable or the source code bindings may be incorrect.

Using Multiple -kdb Switches

Use Model for Adding With Multiple KDBs

```
% vlogan -kdb=common_elab -kdb=only [other options] <design files>
```

Use Model for Adding With Single KDB Separated by Comma “,”

Following is the use model:

```
% vlogan -kdb=common_elab,only,gatesrc [other options] <design files>
```

Suppressing Inline PSL Errors During VHDL Compilation

The feature to suppress inline PSL (Property Specification Language) errors during VHDL compilation is designed to address compatibility issues between the `-kdb` and `-psl` options in the Unified Front-End (UFE) flow. When both options are used together, you may encounter an error that blocks compilation. This feature introduces an environment variable to demote the error to a warning, allowing the compilation to proceed. However, debugging inline PSL parts in Verdi remains unsupported.

Benefits

- Enables compilation of VHDL designs with both `-kdb` and `-psl` options.
- Prevents job failures caused by the incompkdb=gaterc:atibility of these options.
- Maintains transparency during the VCS elaboration stage.

Feedback

To enable this feature, perform the following steps:

1. Set the `SNPS_VCS_VERDI_IGNORE_PSL` environment variable to `1` using the following command:

   ```
   setenv SNPS_VCS_VERDI_IGNORE_PSL 1
   ```

   This variable suppresses the error message and converts it into a warning.

2. Run the `vhdlan` command with both `-kdb` and `-psl` options. The environment variable ensures the error is demoted to a warning.

   Example

   ```
   vhdlan -kdb test.vhd -psl
   ```

   Output

   ```
   Warning-[KDB-OPT-IG] KDB Option Ignored
        Option '-kdb' cannot work together with option '-psl', it is
    ignored.
        Remove this option and recompile the design.
   ```

Optimizing Disk Space Usage in KDB Files

The Knowledge Database (KDB) is a critical component in simulation workflows, storing essential data for debugging and analysis. However, its size can grow significantly, consuming excessive disk space. To address this, Verdi introduces a disk space optimization feature that applies advanced compression techniques to reduce the size of KDB files under the `simv.daidir` directory.

Benefits of Disk Space Optimization

- **Reduced Disk Usage**: Compresses textDoc and data model components, significantly lowering storage requirements.
- **Customizable Compression**: Allows you to selectively enable compression for textDoc and data model using two different options.

Enabling Disk Space Optimization

The disk space optimization is enabled by default. It can be overridden using the compile-time option `-kdb=diskopt:<sub-option>`, which can be used during the compilation process. This can be applied in both the 3-step and 2-step compilation flows.

The `<sub-option>` parameter allows you to control the scope of compression. The supported sub-options are `all` and `none`.

The default behavior is a balanced solution between compile-time overhead and file space. If you need faster compilation, use `-kdb=diskopt:none` and if you need smaller file, use `-kdb=diskopt:all` options respectively.

Example: 3-Step Compilation Flow

```
vlogan -kdb=diskopt:all <vlogan_options> <source_files>
vlogan -kdb=diskopt:all <vlogan_options> <source_files>
vcs -kdb=diskopt:all <vcs_options> <top_module>
```

Example: 2-Step Compilation Flow

```
vcs -kdb=diskopt:all
```

After enabling the feature, the size of the `simv.daidir` directory should be significantly reduced. Use standard disk usage commands (for example, `du -sh simv.daidir`) to confirm the reduction.

Limitations

- **Backward Compatibility**: Older versions of Verdi cannot load KDB files compiled with compression enabled. Ensure you use the same major release of Verdi for compatibility.
- **Performance Impact**: Enabling compression may increase compile, elaboration, and import times due to the additional CPU effort required for compression and decompression.
- **Language Support**: Currently, only Verilog is supported. VHDL and other languages are supported in future stages.

Invoking Verdi

To invoke Verdi 2021.09 and onwards, use the following commands:

- For Verdi Base

  ```
  verdi -base
  or
  verdi -Base
  ```

- For Verdi Elite

  ```
  verdi -elite
  or
  verdi -Elite
  ```

- For Verdi Apex

Feedback

```
verdi -apex
or
verdi -Apex
```

Invoking Verdi in the simv Flow

For the simv GUI mode

- For Verdi Base

```
simv -gui=base
or simv -gui=Base
```

- For Verdi Elite

```
simv -gui=elite
or
simv -gui=Elite
```

- For Verdi Apex

```
simv -gui=apex
or
simv -gui=Apex
```

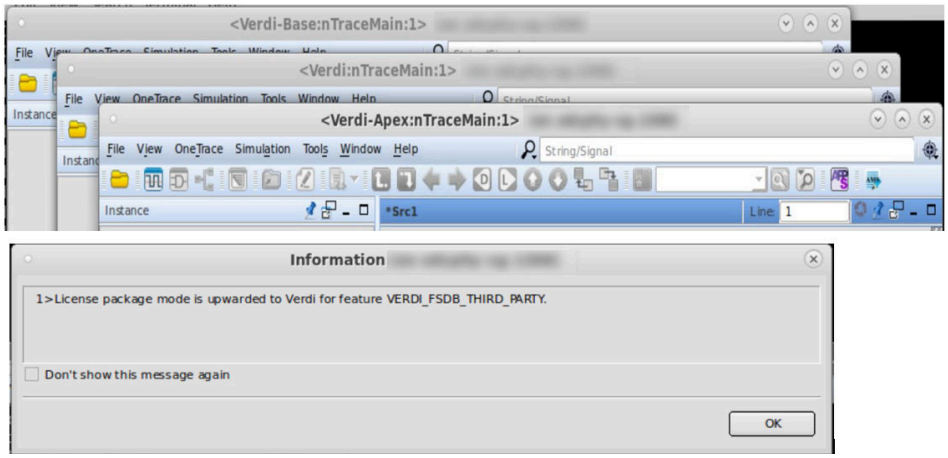
Runtime Upward of Product License Mode

Verdi provides the flexibility to upward the license mode from Base to Elite to Apex for the advanced features at runtime. To enable upward license search from Base to Elite to Apex, you need to set the `VERDI_FEAT_PKG_MODE_UPWARD` environment variable before launching Verdi, using the following command:

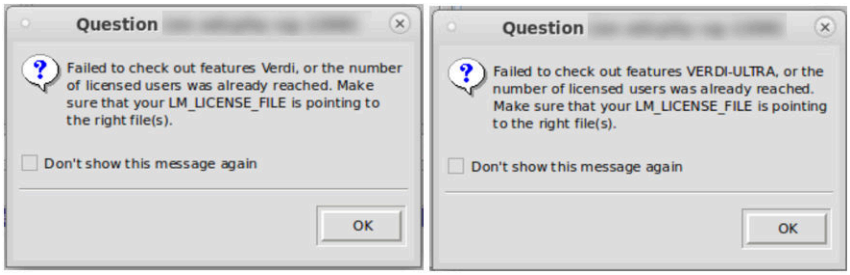
```
setenv VERDI_FEAT_PKG_MODE_UPWARD
```

Note:
Make sure to use the latest license package file while running Verdi.

When the product license mode is upwarded, the upward information would be displayed on the title bar of the top window.



When the license cannot check-out successfully, then license upward fails and pops up the question dialog box to notify you about the failure:



By default, Verdi enables upward license search from Elite to Apex, allowing you with an Elite license to access Apex features if an Apex license is available. To disable this default upward license search, set the `VERDI_DIS_UPWARD` environment variable to `1` before launching Verdi.

When you set `VERDI_DIS_UPWARD` to `1`, Verdi does not attempt to check out a higher-tier license (for example, from Elite to Apex), and only uses the license tier you initially requested.

Table 1 License Upward Search

Command	VERDI_FEAT_PKG_MODE_UPWARD	VERDI_DIS_UPWARD	Behavior
`verdi -base`	Not Set	Not Applicable	Checkout Base
`verdi -base`	Set	Not Applicable	Checkout Base -> Elite -> Apex
`verdi`	Not Applicable	Not Set	Checkout Elite -> Apex
`verdi`	Not Applicable	Set	Checkout Elite

Feedback

Table 1 License Upward Search (Continued)

`verdi -elite`	Not Applicable	Not Set	Checkout Elite -> Apex
`verdi -elite`	Not Applicable	Set	Checkout Elite
`verdi -apex`	Not Applicable	Not Applicable	Checkout Apex

Use Model

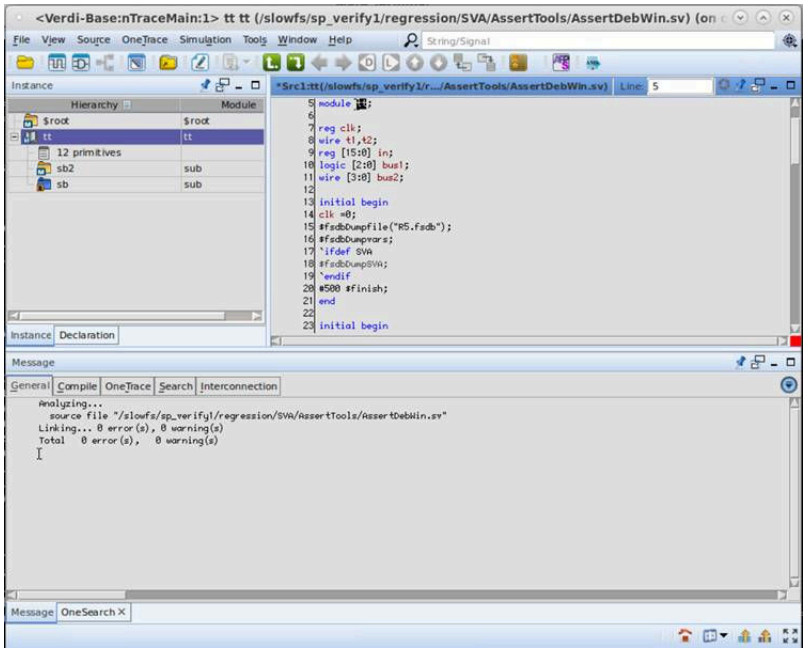
1. Set the `VERDI_FEAT_PKG_MODE_UPWARD` environment variable to `1`:

   ```
   setenv VERDI_FEAT_PKG_MODE_UPWARD 1
   ```

2. Invoke Verdi in Base mode:

   ```
   verdi -base -sv /slowfs/sp_verify1/regression/SVA/AssertTools/
   AssertDebWin.sv
   ```

   The title bar of the top window displays “Verdi-Base” as depicted in the following figure:

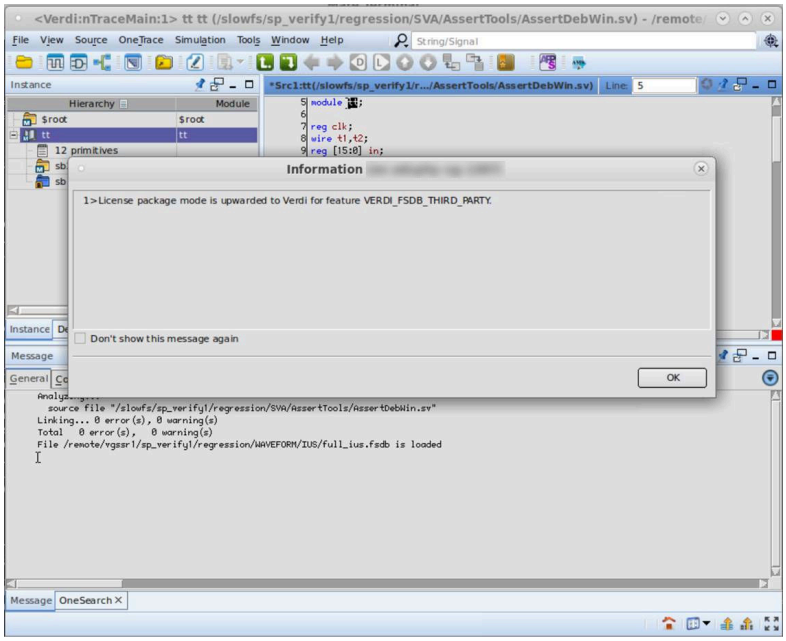


3. Choose **File -> Open Waveform File** to open the third-party FSDB file “/remote/vgssr1/sp_verify1/regression/WAVEFORM/IUS/full_ius.fsdb”.

   You can also create nWave window, and open the FSDB file in the window.

Feedback

Verdi now upward the license to “Verdi-Elite” with the following message as shown in the following figure, and displays “Verdi” on the title bar of the third-party FSDB file window. This happens because the third-party FSDB file is allowed either in Elite or Apex mode only.



Note:

Before this enhancement, that is, if the environment variable `VERDI_FEAT_PKG_MODE_UPWARD` is not turned on, then Verdi-Base does not allow the third-party FSDB to open, and you need to exit Verdi to invoke Verdi-Elite or Verdi-Apex again.

Limitations

- If you have only a base license, you must specify the `-base` command line option. Otherwise, Verdi does not launch.
- With a Base or Elite license, all GUI feature entries remain visible. However, attempting to use advanced Elite or Apex features results in a license failure message.
- License upgrades are supported, but downgrades are not. After using advanced Elite or Apex features, the license cannot revert to a lower tier. For example, closing a third-party FSDB file does not restore the previous license state.
- This functionality is not supported when Verdi is initialized with an outdated license.
- The feature is incompatible with the `VERDI_LIC_ORDER` environment variable.

- This feature does not work for standalone nWave and nCompare.
- Automatic license upgrades are not supported by this utility. For instance, auto-upgrade is not available for fsdb2saif.
- When you start Verdi with the `-licqueue` option or by setting the `NOVAS_LICENSE_QUEUE` environment variable to `1`, the default upward searching from Elite to Apex is disabled, and only the initially requested license mode is utilized.

Loading Designs With Different Strategies

When a design KDB is generated by `vcs -kdb`, here are some of the strategies about how a design is loaded into Verdi.

You can fully load your design with entire hierarchy and instances into Verdi at once, then Verdi loads all the details about the design. The following is the command used:

```
> verdi -dbdir <design.dbdir>
```

When your design is large, complex and expected to take huge memory during loading, a choice is to use Smart Load to only load first touched level hierarchy first in the beginning. When next hierarchy is going to be tracked/touched/clicked, Verdi then incrementally loads the hierarchy on demand. This saves time and resources as they are not touched, so instances won't occupy them. The usage is as follows:

```
> verdi -dbdir <design.dbdir> -smart_load_kdb
```

Refer to the `-smart_load_kdb` option description in the *Verdi and Siloti Command Reference Manual* for details.

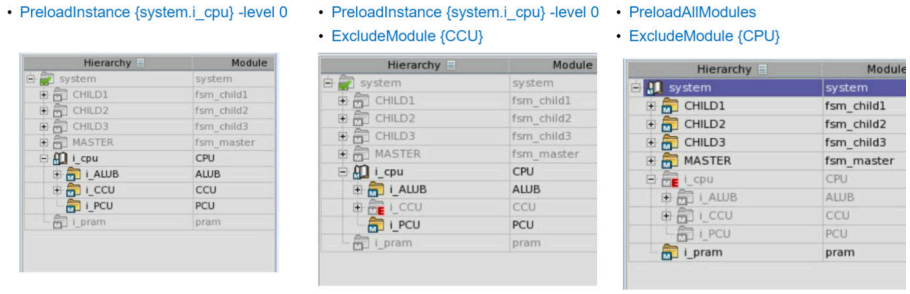
When your design is complex and with several hierarchical instanced modules, and you may also want to focus on a specific design scope, or only exclude some modules while loading the design, then Preload is the right way to do this. After loading specific scopes, you can also easily load others into Verdi again without reloading all scopes. The usage is as follows:

```
> verdi -dbdir <design.dbdir> -preload <preload_cfg_file>
```

Refer to the `-preload` option description in the *Verdi and Siloti Command Reference Manual* for details.

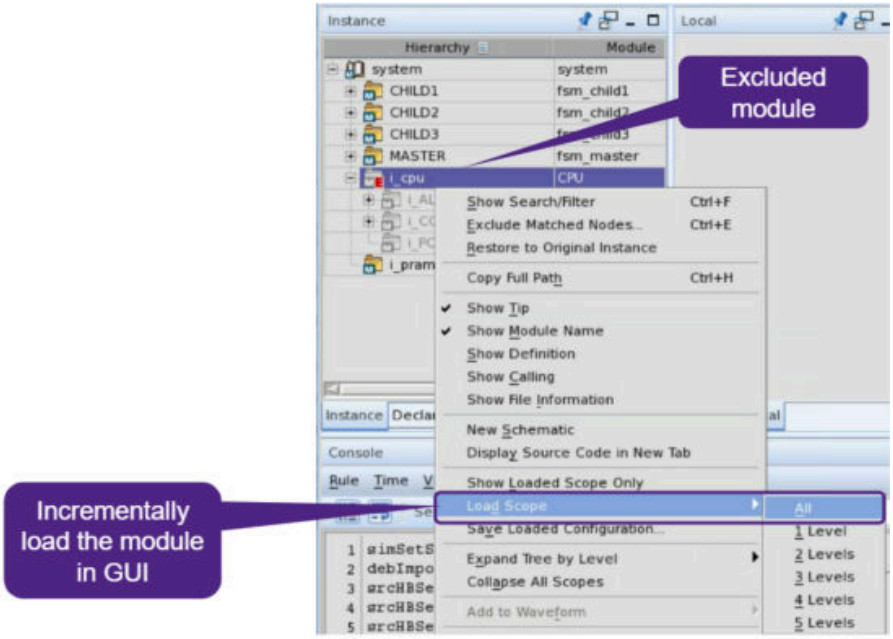
Following are some examples to show how they work with different preload configuration files:

Figure 55 Preload Examples



When you invoke a tracing command, Verdi *nTrace* stops on the boundary of the loaded design. When you want to check other unloaded scope, use **RMB -> Load Scope -> All** or select *1~ 5* level for incrementally loading the selected scope into Verdi.

Figure 56 Excluded Module



Additionally, the status of current loaded and excluded instances could be saved to a preload configure file. It can used to read your saved status while loading the design with `-preload` option next time.

Feedback

Using Verdi Executable to Import Design from UFE

The Unified Compiler Flow (UFE) uses VCS with the `-kdb` option and the generated `simv.daidar` includes the KDB information.

Verdi supports the following use models to import design from `simv.daidar` related files:

- "verdi -dbdir *simv.daidir*"
- "verdi -simBin *simv*"
- "verdi -ssf *novas.fsdb*"

The priority to import design from UFE is as following (from high to low):

1. Use the new `-dbdir` option to specify the `simv.daidar` directory

   Similar to: `dve -dbdir simv.daidir`

2. Use the `-simBin` option to specify simv

   Load `simv.daidar` from the same directory as simv and invoke Verdi if `simv.daidar` is not available.

3. Use the `-ssf` option to specify FSDB file and the design is automatically loaded.

   Load KDB automatically from FSDB. The FSDB file needs to be generated from "`vcs -kdb`" with VCS K-2015.09-SP2 or later.

Note:
The FSDB or all FSDB files in virtual FSDB should be generated by the same simv that is generated by VCS with the `-kdb` option. The import options `-top`, `-elab`, `-lib`, `-f`, and `-ridb` cannot be specified for this usage.

Preload and Smart-load

Preload

Verdi allows you to load a module/instance during Verdi import phase using the `-preload` option. Following is the syntax:

```
verdi -preload <config_file>
```

This feature allows you to improve runtime performance if you already know which module/instance to debug at runtime.

Preload Configuration file

```
PreloadInstance {instance_path} -level level_num
ExcludeModule {module_identifier}
```

Feedback

The `instance_item` rule specifies that all the selected instances in the list is loaded. When the level is `0`, Verdi loads the instances recursively to all their sub-instances. When the level is `1`, just load the specified instance. Positive level specifies how far the instance hierarchy is loaded. If the level is not specified, then it is equivalent to level 0.

Verdi takes the first instance in list as the beginning instance.

Preload Instance

Apply the specified preload mode to all the instances in the list and recursively to all the sub-instances.

```
PreloadInstance {top.bridge.cpu} -level 1
PreloadInstance {top.\##ii## .i1.i0} -level 1
```

Exclude Module

Exclude Module is applicable when debug is not required and Verdi won't load the module into the memory.

For Example:

```
ExcludeModule {CPU}
PreloadAllModules
```

In this example, Verdi loads all modules except module CPU.

Preload all Modules

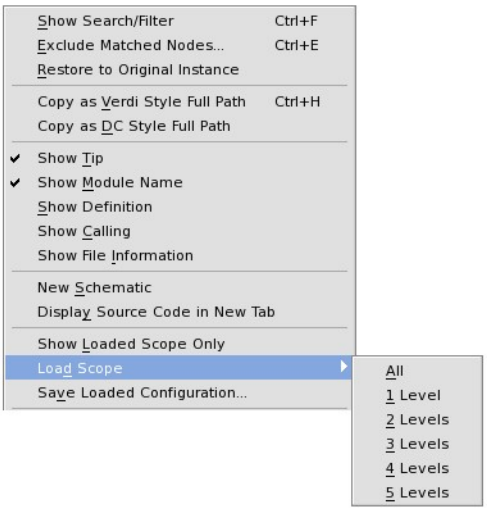
`PreloadAllModules` command is used to load all the instances and it is used mostly with `ExcludeModule` command. If you just specify the `ExcludeModule` command without loading instance by `PreloadAllModules or PreloadInstance` command, Verdi does not load anything.

Feedback

Load Exclude by GUI

Provide RMB command to load the scope by levels as shown in the following figure. In case of this command, *nTrace* provides load in the level range **1~5** and it is shown in the preload flow as shown in the following figure:

Figure 57 Load Scope



Virtual Top

For virtual top to function, you can move to the preload flow or Preload support load partial design by reading the virtual top map file, but Preload does not change the design hierarchy.

Example of `Verdi -perload vtop.file`:

```
=== vtop.file ===
cpu=system.i_cpu
pram=system.i_pram
==============

Preload load instance "system.i_cpu" and "system.i_parm"  include its
 sub-instances..
It is same with following commands
PreloadInstance {system.i_cpu} -level 0
PreloadInstance {system.i_pram} -level 0
```

Smart Load

Using Smart-load, Verdi loads the necessary design information in the beginning and then loads the remaining design information on demand. In this way, Verdi can get better performance during import but the performance is not good during runtime.

For example, `verdi -smart_load_kdb`.

The following message appears in Verdi:

Smart-load is enabled, complete KDB is loaded on demand.

Reference Source Files on the Command Line

This method loads the design directly from the source files. It is not recommended for mixed language designs.

1. Use the following command to reference the source files on the command line:

   ```
   % verdi -f <source_file_name>
   ```

   Where, *source_file_name* is a file that lists all of the HDL source files.

2. Use the **File -> Open Waveform File** command to load the FSDB.

Compile Source Code into a Library

This method must be used if you have a mixed language design.

1. Use the utility program *vhdlcom* (supplied with the Verdi and Siloti installation) to compile your VHDL source code into a library and use *vericom* for Verilog code:

   ```
   % vericom -lib <libName> block1.v block2.v ...
   % vhdlcom -lib <libName> block1.vhd block2.vhd ...
   ```

2. Use the following command to load the compiled design:

   ```
   % verdi -lib <libName> -top <TopBlock>
   ```

   Where, *libName* is the compiled library and *TopBlock* is the highest-level block you want to see and *fsdb_file_name* is the name of the FSDB file.

Reference Design and FSDB on the Command Line

1. Use the following commands to reference both the source files and the FSDB on the command line:

   ```
   % verdi -f <source_file_name> -ssf <fsdb_file_name>
   ```

   Where, *source_file_name* is the source file name and *fsdb_file_name* is the name of the FSDB file.

2. Use the following commands to reference both the compiled library and the FSDB on the command line:

   ```
   % verdi -lib <libName> -top <TopBlock> -ssf <fsdb_file_name>
   ```

   Where, *libName* is the compiled library, *TopBlock* is the highest-level block you want to see, and *fsdb_file_name* is the name of the FSDB file.

Note:
If the specified FSDB file is an Essential Signal FSDB, you are presented with a *Question* dialog related to Data Expansion. If you plan to use the Siloti system, click **Yes**; otherwise, click **No**. Data Expansion can always be started or the options changed by invoking the **Tools -> Visibility -> Data Expansion -> Setup Data Expansion** command

Perform Behavior Analysis on the Command Line

To perform Behavior Analysis on the command line, you must specify the `-ba` option.

- Referencing the source files:

  ```
  % verdi -f <source_file_name> -ba -ssf <fsdb_file_name>
  ```

  Where, *source_file_name* is the source file name and *fsdb_file_name* is the name of the FSDB file.

- Referencing the compiled library:

  ```
  % verdi -lib <libName> -top <TopBlock> -ba -ssf <fsdb_file_name>
  ```

  Where, *libName* is the compiled library, *TopBlock* is the highest-level block you want to see, and *fsdb_file_name* is the name of the FSDB file.

Note:

- The `-ba` option executes Behavior Analysis using the Behavior Analysis settings from the `novas.rc` resource file unless you specifically include them on the command line. Refer to the `verdi` utility description in the `Verdi and Siloti Command Reference` for a list of Behavior Analysis options.
- If you get a warning message that behavior analysis has failed, as shown in the following example, set one of the following environment variables:

  ```
  NOVAS_LIBS = libraryName [libraryName...]
  NOVAS_LIBPATHS = libraryPath [libraryPath...]
  ```

  This is because you must set your own library path rather than using an internal library of Verdi.

  Example message

Feedback

```
Behavior Analysis has aborted because symbol
library "/tools/synopsys/verdi3/2020.12-SP2-6/
share/symlib/64/artisan_l.lib++" was compiled with
the map2SymDB utility. Recompile library
"artisan_l" with the latest syn2SymDB utility
version.
```

Replay a File

Use the following command to replay a file containing commands that load both the design and the FSDB (and perform a variety of other tasks):

```
% verdi -play <command_file_name>
```

Where, *command_file_name* is a file with a number of Tcl commands.

Start Verdi Without Specifying Any Source Files

1. Use the following command to start the Verdi platform:

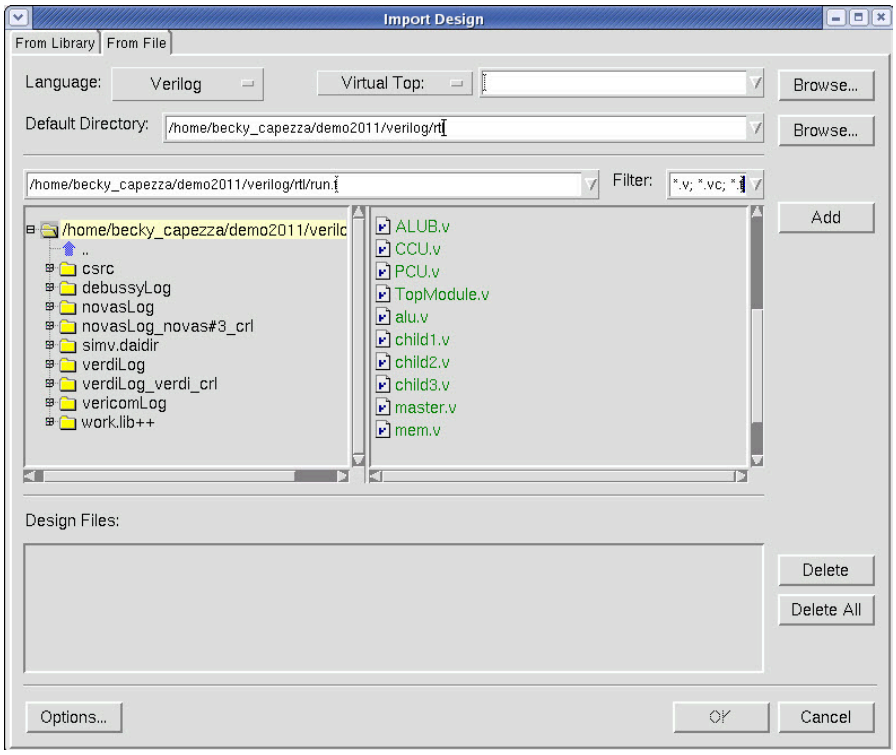
   ```
   % verdi
   ```

   A blank `nTrace` main window is displayed.

Feedback

2. From the main menu, choose the **File -> Import Design** command (or the **Import Design** icon on the toolbar) to open an `Import Design` form, similar to the example in the following figure:

Figure 58 Example Import Design Window



3. If you are loading the design from the source files directly, do the following:
   a. Click the **From File** tab at the top of the window.
   b. Select the HDL language in the **Language** selection field.
   c. To open a folder, click the folder name. A list of sub-folders and/or files appears to the right.
   d. Double-click the name of the files or click the **Add** button to the right of the window to add the file to the path name, which appears in the white space directly above the two windows in which you are working.
   e. Select the design file(s) of interest. The recommendation is to use a run file where the individual design files are listed.
   f. Click the **Add** button.
   g. Click the **OK** button.

4. If you are loading the design from a compiled library, do the following:
   a. Click the **From Library** tab at the top of the window.
   b. To select a library, click the library name. A list of design units appears to the right.
   c. Select the top design unit.
   d. Click the **Add** button.
   e. Click the **OK** button.
5. Use the **File -> Open Waveform** File command to load the FSDB.

   You should see an *nTrace* main window with the design information displayed.

Loading When Design and FSDB Hierarchies do not Match

Use the following command if you have dumped the FSDB file for the entire design, but only want to load a portion of the design for debug:

```
% verdi -f <source_file_name> -ssf <fsdb_file_name> -vtop <map_file_name>
```

Where, `source_file_name` is the source file name, `fsdb_file_name` is the name of the FSDB file, and `map_file_name` is the name of the map file.

The map file is used to match the design hierarchy to the hierarchy in the FSDB file so the simulation results are correctly annotated on the source code and schematic views. The Verdi platform automatically generates a virtual hierarchy in the design according to the definitions in the *map* file. The syntax is as follows:

```
module_name = hierarchical_instance_path
```

Note:
The map file matches the case sensitivity of the associated language. For example, Verilog is case-sensitive so if the module definition is all capitalized, the map file description needs to be as well (that is, cpu does not equal CPU). VHDL is not case-sensitive (that is, cpu equals CPU).

If you incorrectly enter the module name in the map file, you may see an error similar to the following in the compiler.log (**File -> View Import Log**):

```
*Error* view cpu is not defined for inst i_cpu
 "virtual_top_autov_15123.gen:, 7:
```

This error needs to be eliminated. Check the module definition in the source code and confirm that the map file matches it exactly.

Feedback

Replacing the Absolute Paths in Verdi KDB

When any compile-time collateral (`simv`, `simv.daidir`, `csrc`, `kdb`, `compile vdb`) is copied or moved from one location to another, the absolute paths pointing to the original directory become invalid (additionally, the original directory might no longer exist). Therefore, when the KDB is reused in the new location, it could impact the functionality.

Verdi uses the design source files that are stored in binary format within KDB; therefore, most of the Verdi debug operations that require source files refer them from the KDB itself. There are a few exceptions to the rule; where Verdi looks for the source files in the compile directory (as per the path to the compile directory recorded in the KDB). The following are the cases:

- Open Source Editor within Verdi to edit the source files
- OneSearch: Search from the source files
- Coverage: Source code viewer

For the above three use cases, (Verdi Editor, One Search, and Coverage Source Code Viewer), Verdi continues to look for the source files in the compile area (where KDB is created), which does not work when relocated.

KDB retains the specified paths during compile time. Verdi allows you to specify the absolute path to the compile directory.

Option to Retain the Paths Specified Inside the KDB

Verdi provides a compile-time command line option, which saves the source files location in the relative path that you provided. If absolute paths are provided, they remain as absolute paths in the KDB. If relative paths are used, they remain relative.

For example:

```
vcs -preserve_path
```

The option `-preserve_path` is used to ensure that the tool does not change the user-defined path that the user has provided during *vlogan/vhdlan/syscan/vcs*.

Option to Specify Absolute Path to the Compile Directory

Verdi also provides an additional option while invocation to specify the absolute path to the compile directory. You can provide a Verdi command line option to specify the reference compile directory for the relative path of the source files. This reference directory must be specified as an absolute path.

Feedback

For example:

```
Verdi -comdir <absolute path of the compile dir>
```

When you load the design with `-comdir <absolute path of the compile dir>`, Verdi uses `-comdir` directory as the reference directory of the relative path in KDB to find the location of the source files.

When you load the design without `-comdir`, Verdi uses the directory that contains `simv.daidir` as the reference directory of the relative paths in KDB, for the following operations:

- Open Source Editor within Verdi to edit the source files
- OneSearch: Search from the source files
- Coverage Source code viewer

Limitations

The KDB relocation methodology is applicable only for 2-step flow.

Support of Incremental Dump in vlogan Without PSL

Incremental dump feature reduces the KDB memory overhead with `vlogan`. However, incremental dump in KDB does not work with PSL syntax. If there is only inline PSL in the design, `-kdb=common_elab` could skip the inline PSL, which allows the incremental dump to work as expected. In other words, incremental dump occurs with `-psl` and `-kdb=exclude_inline_psl`.

If the design consists of both inline PSL and vunit syntax, incremental dump does not work. That is, `-kdb=exclude_inline_psl` fails to work with `-pslfile` or `*.psl` or `*.vu` or `*.vunit`.

Getting Relative Path in FSDB

To ensure the correct path of `simv.daidir` can be found, FSDB stores the absolute path of the working directory of simulation, as the reference directory. Verdi can transfer the absolute paths to relative paths from the reference directory, so that the correct folder path and the KDB can be found.

Required Conditions

The feature is supported under the following conditions:

- The folder `simv.daidir` is under working directory (working directory is the reference directory where the simulation is executed).

  For example, when you execute simulation in the `/a/b/c/` path, `simv.daidir` should be in the `/a/b/c/simv.daidir/` or `/a/b/c/../simv.daidir/` path.

- The relative path between working directory and `simv.daidir` is the same as the one between the directory where users open Verdi and `simv.daidir`.

When you load FSDB with the `-ssf` option, Verdi uses working directory as a reference directory and transfer the absolute path of `simv.daidir` to a relative path. Therefore, the correct folder path and KDB can be found.

For example,

- Consider you execute simulation in the following path:`/home/debug/`.
- After simulation, FSDB contains absolute path to `simv.daidir` at `/home/debug/simv.daidir/` and to working directory at `/home/debug/`.
- After downloading it to another path, Verdi operates these two paths and finds the relative path for `simv.daidir` is `./simv.daidir/`.

  Note:
  When `simv.daidir` exists at both original path and download path, the one in original path is set to use.

Limitations

- You must make sure that the solution is used under the conditions described in the Required Conditions section.
- For the solution to perform successfully, after download, the relative paths between `simv.daidir`, source files, and KDB should not be changed.

-kdb=common_elab Supports VCS Dynamic Test Loading

Dynamic Test Loading (DTL) is a mechanism to load Verilog packages at any time in the middle of a simulation. The package is dynamically loaded and can have testbench collateral. For example, it includes UVM test classes or UVM sequence classes. This mechanism can be used to run different UVM tests or sequences without having to regenerate the simulator executable (*simv*).

Feedback

The Save-Restore feature allows you to save the simulation state and then resume simulation from the saved state.

The partition compiler feature allows you to compile Verdi KDB data into partition library. The new test is compiled into partition library.

Use Model

The following options are used:

- `-enable_dynamic_tb`: This option is used to compile and create a simulator executable that can dynamically load a test.
- `-dynamic_tb`: This option is used to compile the Precompiled IP for the test package that has to be dynamically loaded.
- `-kdb=common_elab -partcomp`: This option is used to create KDB related models and enable partition compiler.
- `-debug_access+all`: This option is used for Verdi interactive debug in the save state step.

Following are the steps involved in VCS common elaboration partition compiler flow:

1. Enclose all the basic test classes in a Verilog package, for example, `base_test_pkg`. The following is the command used: `vcs -kdb=common_elab -partcomp -enable_dynamic_tb`

   These base classes may be derived from UVM classes. Compile the `base_test` package using the VCS methodology. Integrate the DUT and the `base_test package` together to create the simulator executable (`simv`).

   - Verdi interactive debug mode: `-debug_access+all`
2. Following options are used for saving simulation state after the common part of the test is executed:
   - Verdi Interactive debug: `-ucli2Proc`
   - Verdi UVM Debug: `+UVM_VERDI_TRACE=HIER` is used to write UVM tree to FSDB.
   - Example of `save.cmd` UCLI command file: `stop -in uvm_component::post_reset_phase -once -continue -command {run 0; save saveddesign; ; run}`
   - Example of `restore.cmd` UCLI command file:

     ```
     restore saveddesign call \$dtl_load call top.refresh call
      uvm_custom_install_verdi_recording.get_new_test
     ```

3. Enclose other tests and sequences derived from the base test classes in separate packages. This reuses the base classes from `base_test_pkg` option. Therefore, `-partcomp_sharedlib=[partitionlib_name]` is necessary to refer the data of base classes. For example, `test1_pkg, test2_pkg, test3_pkg.`
   - Verdi interactive debug mode: `-debug_access+all`
4. Run Verdi with other new tests by dynamically loading the test packages.
   - You need to specify the following test to be loaded and run dynamically: `+dtl_add_pkg=<Path to Test PIP>:<Name of test package>`
   - Verdi interactive debug mode: `-i [restore_cmd_file]`

Verdi Support for Dynamic Test Loading

Verdi features can be used to debug and trace new test in post-process mode and interactive mode.

- Post-process mode: New test package is added into source view, instance tree, and declaration tree.
- Interactive debug mode: Using the option `+dtl_add_pkg=[new_test] -i restore.cmds`, you can debug new test package with interactive debug features.
- UVM-aware debug:
  - OVM/UVM hierarchy tree: New test name can be shown using OVM/UVM hierarchy tree.

Note:
To know more about the usage of DTL, refer to the *Dynamic Test Loading* chapter of the *VCS LCA Features Guide*.

Limitations

Following are the limitations of this feature:

1. The following technologies do not support DTL:
   - FGP
   - Dynamic Reconfiguration
   - Debug of classes
   - VMC
2. Cross Modules References (XMR) from the Dynamic Test package to the DUT is not be supported.

3. The PIP created for the Dynamic Test Package should not have any modules. Only packages are supported.
4. If source code of the base test is changed, then the base test package needs to be recompiled and `simv` needs to be regenerated. If the new test is dependent on the base test, then the new test has to be recompiled.
5. The dynamic test package cannot be part of the simulator executable (`simv`).
6. Dynamically loaded UVM packages should configure the tests in the main phase and not in the build phase.
7. In Verdi interactive debug usage, you cannot specify other script in `simv -verdi -i restore.cmds`. Also, you cannot append the UCLI commands to `restore.cmd`.

5

User Interface Tutorial

This chapter describes the following topics:

- Overview
- Start Verdi Platform
- Using the Welcome Page
- Saving and Restoring a Session
- Changing the Default Frame Location
- Maximizing the Display
- Modifying the Menu/Toolbar
- Searching for a Command
- Customizing BindKeys
- Customizing Toolbar Icons
- Using Unified Find
- Support for Active Window in Verdi
- Next Generation Debug GUI
- Verdi Integration With Euclide IDE

Overview

The Verdi platform is a multi-window docking application with a flexible and easy-to-use graphical user interface (GUI).

The Verdi platform layout can be customized by dragging a frame away from its original position (undocking) and then dropping it in a new position (docking) to attach it to the left, right, above, below, or on top of another frame. Similarly, a toolbar category can be moved to the left, right, above, or below the relevant frame. The bind key of any command can be changed and the toolbar categories arranged fairly easily.

Feedback

Before you begin this tutorial, follow the instructions in the Before You Begin chapter.

Refer to the Launching Techniques chapter for more information on starting the Verdi platform, and refer to the User Interface chapter for more details regarding the interface.

Start Verdi Platform

1. Change the directory to `<working_dir>/demo/verilog/rtl`.

   ```
   % cd <working_dir>/demo/verilog/rtl
   ```

2. Start the design using the following command:

   ```
   % verdi -f run.f
   ```

Using the Welcome Page

1. Choose the **Help -> Welcome** command if the *Welcome* page is not displayed.

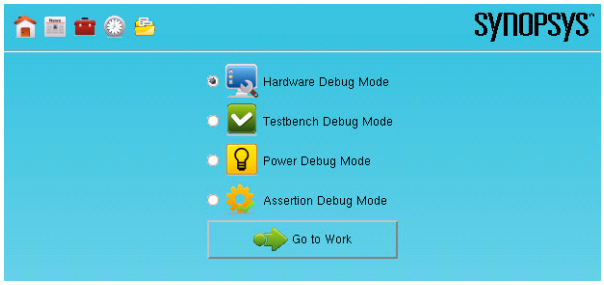
Figure 59 Welcome Page

2. On the *Welcome* page, click the **What's New** icon and then click the **Application Notes** icon. You can see the application note files and the FAQ file here.

3. Click the **Home Page** icon to go back to the *Welcome* page and then click the **Work Modes** icon.

Feedback

Figure 60 Work Modes Page



4. Select the **Testbench Debug Mode** option and then click the **Go to Work** button.

   The window layout of the Verdi platform adds the *Constraint, Inheritance, FSDB_Msg* and the *Static* frames for testbench code browsing and message debugging purposes.

5. Click the **Welcome** icon in the lower right corner of the main window to display the *Welcome* page.

6. Click the **Work Modes** icon to show to the *Work Modes* page again.

7. Select the **Hardware Debug Mode** option and then click the **Go to Work** button.

   The window layout of the Verdi platform is now optimized for **Hardware Debug Mode**.

Note:
The work mode may also be specified on the Verdi command line with the `-workMode` option.

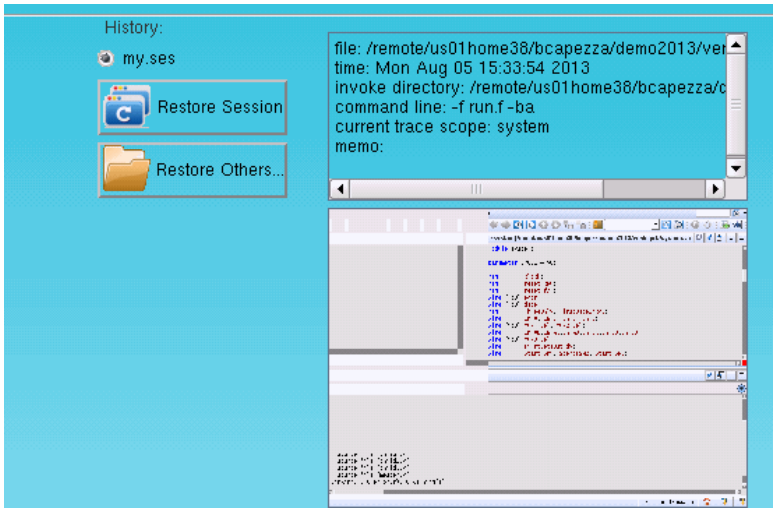
Saving and Restoring a Session

1. Choose the **File -> Save Session** command and save the current session to “`my.ses`”.
2. Go to the *Welcome* page and click the **History** icon to show the *History* page.

Feedback

3. Select the **my.ses** option on the *History* page. A screen shot of the session is displayed to the right as shown in the following figure:

Figure 61 Session Preview



4. Select the `novas_autosave.ses` option.

   The image shows the time the session was saved.

   Note:
   If you do not see this file, exit the Verdi session and start it again. The file is automatically created on exit.

5. Click the **Restore Session** button to restore the **novas_autosave.ses** session.

6. Go back to the *History* page and restore the **my.ses** session.

For more information on how to save and restore a debug session in Verdi, click the following video link:

This video explains how to save and restore a debug session in Verdi. In the Verdi GUI, you save a session by selecting **File > Save Session**, enabling the **Save Simulation State** and **Enable Reverse Debug on Restore Session** options, specifying a file name, and clicking **Save**. To restore a session, select **File > Restore Session**, choose the saved session file, and click **OK**. These methods ensure that your simulation state, checkpoints, and GUI layout are preserved for efficient debugging.

Feedback

Changing the Default Frame Location

1. Click the **New Waveform** icon to open an *nWave* window.

   The *nWave* frame is added as a new tab on top of the *Message* frame.

2. Click the frame banner of the *nWave* frame and drag it to the left.

   The *nWave* frame is now floating (undocked).

3. Drag the *nWave* frame around and see that the dockable area (outlined with a dashed line) is changing along with the cursor position.

4. Drop the *nWave* frame to dock it.

5. Undock and dock the *nWave* frame (or another frame) to different positions several times to become familiar with the usage.

For more information on how to change the layout arrangement on a Verdi platform, click the following video link:

This video explains how to change the layout arrangement on a Verdi platform. The layout in nTrace can be restored for various actions, including docking or undocking a frame using the toolbar icon, changing a docked frame to a top-level window or vice versa, moving a docked frame to another dock location, resizing the top window or any inner frames, hiding a frame, changing layout modes, and maximizing or restoring a frame.

Maximizing the Display

1. Click the **Undock** icon on the *nWave* toolbar to make it a standalone window.

2. Click the **Dock** icon on the *nWave* standalone window.

   The *nWave* frame is docked to the main window again.

3. Double-click the *nWave* frame banner. This maximizes the *nWave* frame size and makes it easy to see the content of the frame more clearly.

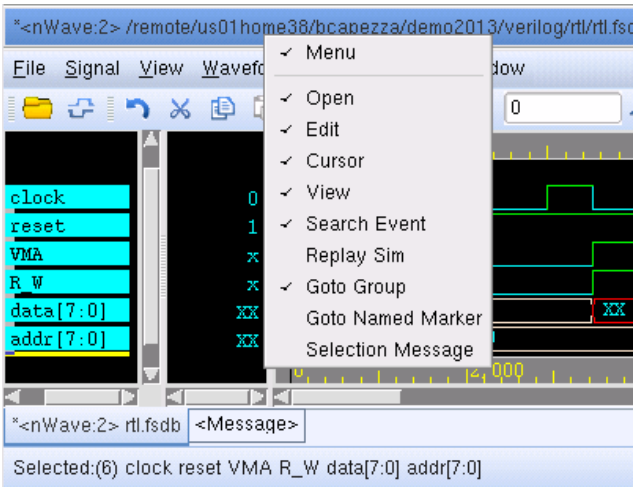
4. Double-click the *nWave* frame banner again and the *nWave* frame goes back to its original size.

Feedback

Modifying the Menu/Toolbar

1. Right-click the *nWave* frame banner to display the right-click command menu and select the **Menu** option to hide the menu bar.

Figure 62 nWave Configuration Menu

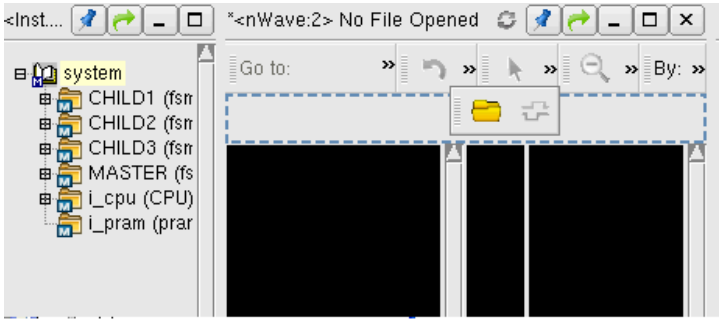


2. After the **Menu** option is turned *off*, press the **Alt** key within the *nWave* frame's central area to show the menu bar. Pressing the **Alt** key again hides the menu.
3. Turn *on* the **Menu** option in the right-click command menu.
4. On the right-click command menu, select the **Open** option to hide the icons associated with the **Open** toolbar category.

   The **Open** category disappears from the toolbar of the *nWave* frame.
5. On the right-click command menu, select the **Open** option again to restore the **Open** toolbar category.
6. Click the left handle (vertical bar) of the **Open** toolbar category and drag a little. The **Open** toolbar category is floating.
7. Drag it around and observe the dockable area (outlined with dashed line) is changing along with the cursor position.

Feedback

Figure 63 Relocating Toolbar Icons

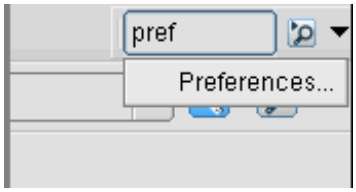


8. Drop the **Open** toolbar category to dock it.
9. Move the **Open** toolbar category several times to become familiar with the usage.

Searching for a Command

1. On the top right corner of the main window, select the **Menu** option, type "*pref*" in the **Spotlight** text field and press the **Enter** key. This displays a list of commands matching the pattern.
2. Select one of the commands from the list, for example, **Preferences**, and then the **Preferences** command is invoked.

Figure 64 Spotlight Search Results

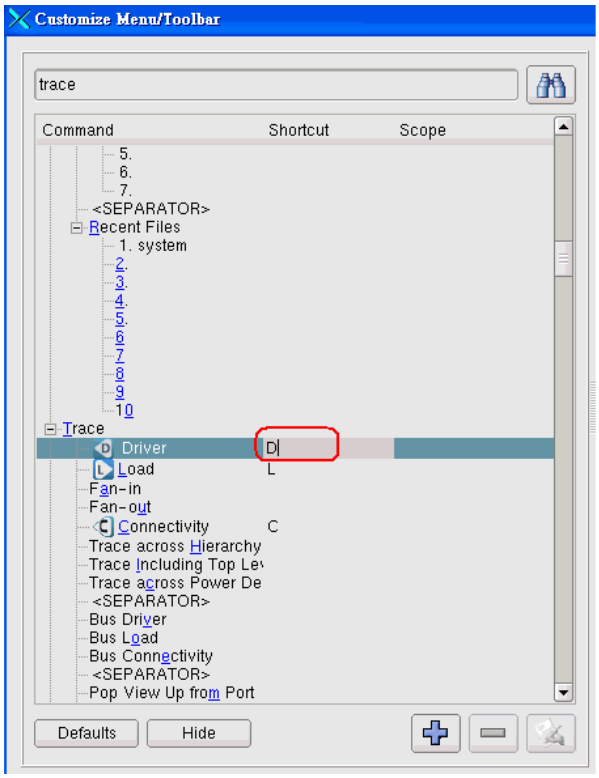


Customizing BindKeys

1. Choose the **Tools -> Customize Menu/Toolbar** command from the main window.
2. Type "`trace`" in the text field and click the **Search** icon to locate the **Trace** menu.

Feedback

Figure 65 Customize BindKey



3. Double-click the **Shortcut** cell of the **Driver** command. Press the **D** key on the keyboard to change the command's bindkey to **D**.
4. Similarly, double-click the **Shortcut** cell of the **Load** command. Press the **L** key on the keyboard to change the command's bindkey to **L**.
5. Click the **OK** button to complete the setting.
6. In the Source Code frame, select any signal and press the **D** key to execute the **Driver** command.
7. Press the **L** key and the **Load** command is executed. This is useful when you want to execute frequently used commands with bindkeys you favor.

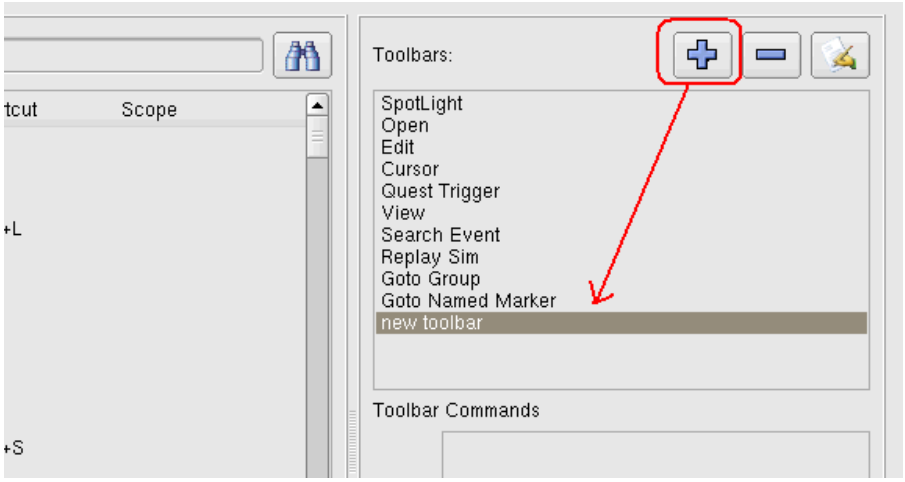
Customizing Toolbar Icons

1. Click the **Undock** icon on the *nWave* frame to make it become a standalone window.
2. Choose the **Tools -> Customize Menu/Toolbar** command from the *nWave* window.

Feedback

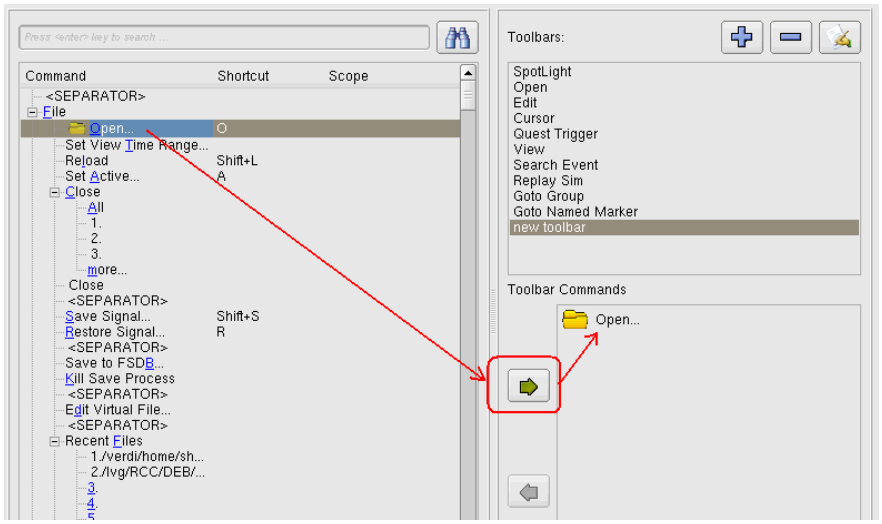
3. Click the **Add Custom Toolbar** icon in the upper right section of the *Customize Menu/ Toolbar* form to add a new toolbar category named "`new toolbar`".

Figure 66 Add New Toolbar Category



4. Select the **Open** command under the **File** menu section in the left pane.
5. Click the **Add Selected Command to Toolbar** button to add the **Open** command to the newly created "`new toolbar`" toolbar category.

Figure 67 Add Commands to New Toolbar Icon Category

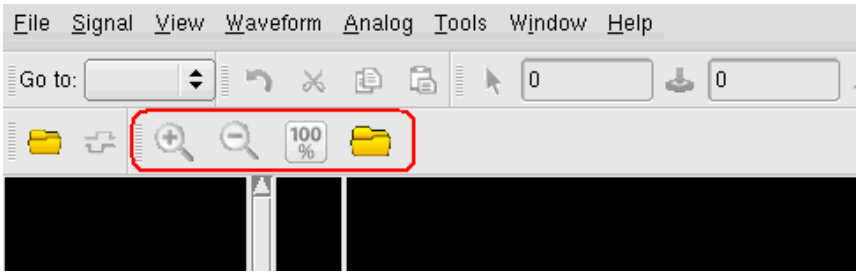


6. Similarly, add the **Zoom In**, **Zoom Out** and **Zoom All** commands under the **Zoom** menu section to the newly created "`new toolbar`" category.

Feedback

7. Click the **OK** button to complete the setting. The new toolbar is added to the *nWave* toolbar area.

Figure 68 New Toolbar Icon Category



Using Unified Find

For each docked Verdi component, the search toolbar and spotlight search is consolidated in the top window's spotlight search **String/Signal** option as the **Unified Find** option which is applied in the active window.

You can invoke the **Unified Find** option with the **Ctrl+F** shortcut key, which auto switches the spotlight search as the **String/Signal** option and auto-focuses in the key-in text box. If the text box is empty then it is auto filled with the active window's selection text.

The search option is also enhanced in Verdi for the **Unified Find** option.

This section consists of the following subsections:

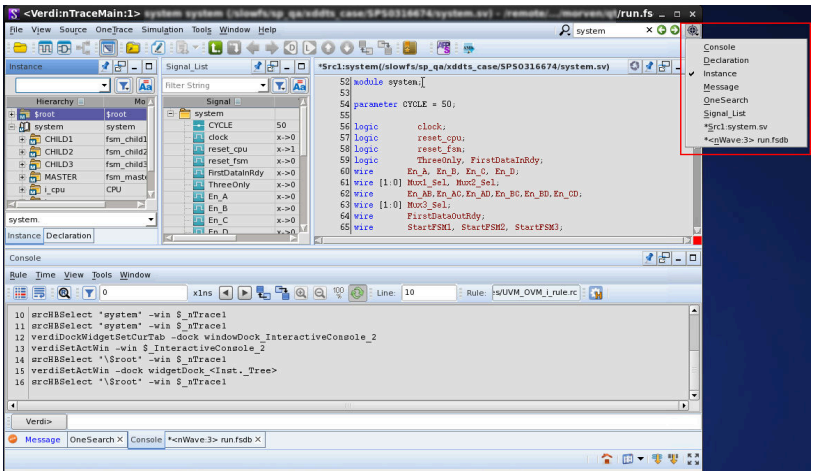
- Support for Active Window Icon With the Spotlight Icon
- Enhancement to the Search Option in the Message Frame
- Enhancement to the Search Option in the Transaction Table View, Transaction Relation Navigator Window, and SmartSheet
- Enhancement to the Spotlight Icon
- Enhancements to the Right-Click Options
- Enhancements to the Temporal Flow View Tools Menu
- Enhancements to the Preferences Form
- Limitations

Feedback

Support for Active Window Icon With the Spotlight Icon

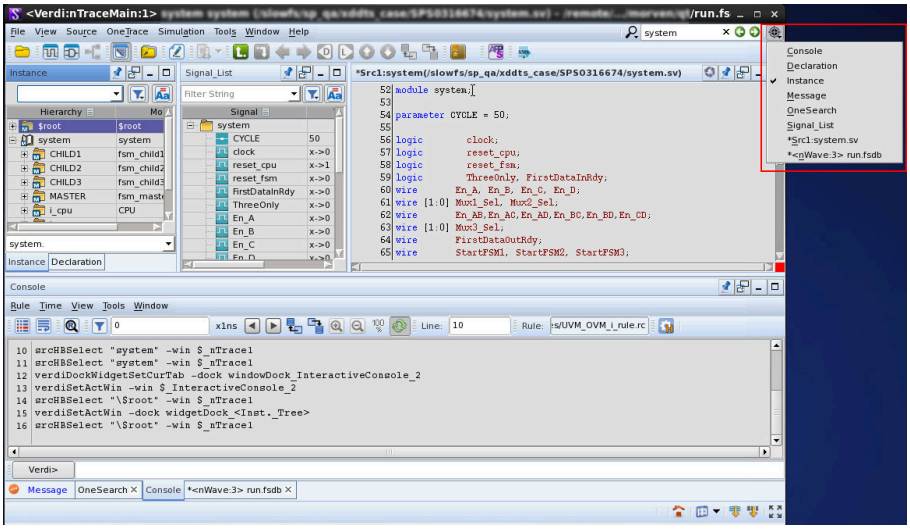
Verdi supports **Active Window** icon on the **Spotlight** function area as shown in the following figure:

Figure 69 Active Window Icon



Click the triangle of the **Active Window** icon to view the active docked windows available as shown in the following figure:

Figure 70 Active Window Icon Options

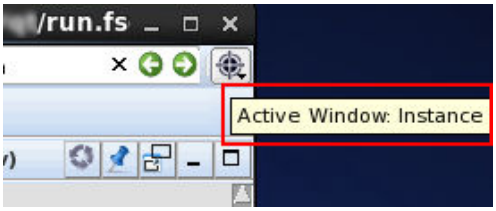


A check mark is displayed for the current active window. You can select any other docked active window available in the **Active Window** icon to change the current active window.

Feedback

A tooltip is displayed which provides the active window information, as shown in the following figure:

Figure 71 Tooltip Information



Note:

In the *Source Code* pane for the active window, the **Unified Find** option searches for the string from the cursor point. For example, if the cursor point is at the 99th line of a 100 line source code and line 3 and line 100 has the searched string, then since the cursor is at line 99, the first search match found is at 100th line and then the 3rd line. Also, for the **SmartLog**, **Console**, and **OneSearch**, the **Unified Find** option searches for the string from the viewable content.

Enhancement to the Search Option in the Message Frame

The search and find option is enhanced in the *Message* frame as follows:

1. The **Find String** text box is available only in the **OneTrace** tab and **Search** tab.
2. The toolbar icons to find previous string and next string are removed.
3. Only support to auto-fill the selected string is available in **OneTrace** tab. When you select **Ctrl+F** and find a text, the text field is empty.
4. The **Find Whole Words Only** option in the **String/Signal** option is disabled when the current active window is the *Message* frame.

Feedback

Enhancement to the Search Option in the Transaction Table View, Transaction Relation Navigator Window, and SmartSheet

When the **Find Whole Words Only** option in the **String/Signal** option is turned *on*, the identifiers matching the complete word you enter in the **Spotlight** text field are searched in the table cell as shown in the following figure:

Figure 72 Example of Transaction Table View and Transaction Relation Navigator

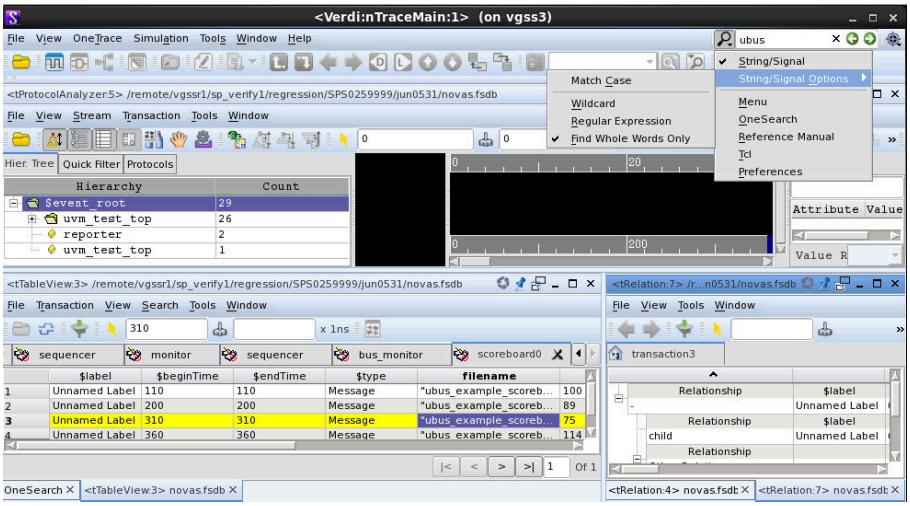
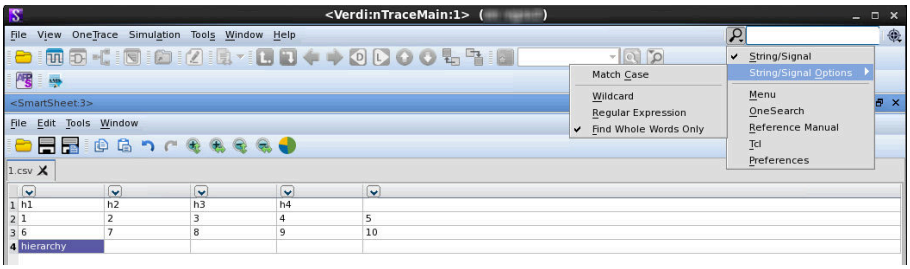


Figure 73 Example of SmartSheet

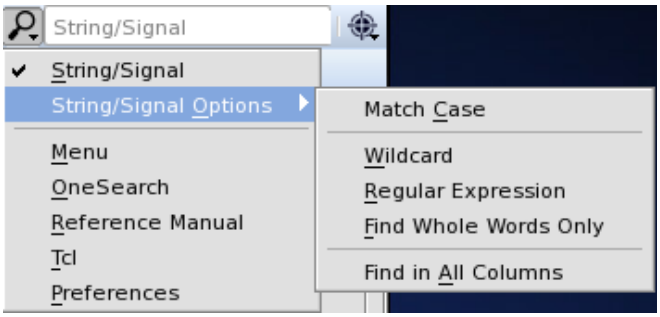


Enhancement to the Spotlight Icon

The **String/Signal** option is added in the **Spotlight** icon, as shown in the following figure.

Feedback

Figure 74 Spotlight Functions



Click the triangle as shown in the figure to select **String/Signal** option and then specify a string to search in the text field to dynamically search for the first object that matches in the current pane/window.

You can click the **Find Previous** or **Find Next** icons to find the previous or next string specified in the **Spotlight** text field.

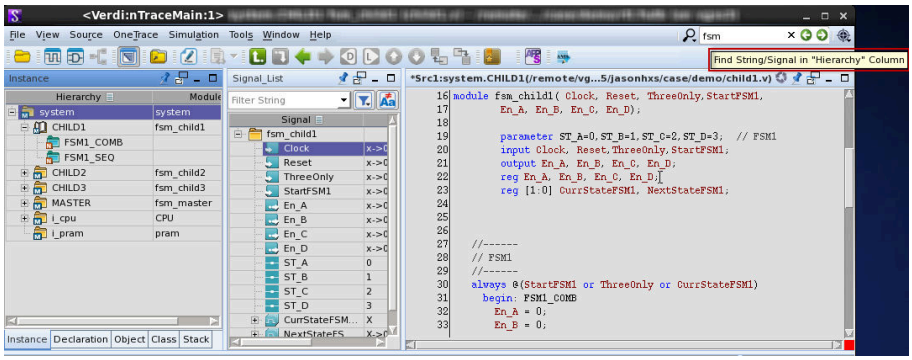
Following options are available with the **String/Signal Options**:

- **Match Case**: When this option is turned *on*, strings or signals matching the capitalization of the text entered in the **Spotlight** text field is found.
- **Wildcard**: When this option is turned *on*, the wildcard characters (* or ?) are supported to specify the pattern-matching string. For example, if EN_A* is entered, EN_AB, EN_AC, and En_ AD are all found.
- **Regular Expression**: When this option is turned *on*, additional regular expressions are supported. Special keywords (*, ?, +, -, |) are treated as regular expressions.
- **Find Whole Words Only**: When this option is turned *on*, the identifiers matching the complete word you enter in the **Spotlight** text field are searched. Note that this option is disabled when the current active window is switched to the *Source Code* window to avoid conflicts when the **Match Whole Word Only** option in selected in the *nTrace* **Find String** form.
- **Find in All Columns**: When this option is turned *on*, the signal is searched in all the columns available in the active tab. For example, specify a string "fsm" to search in the text field. When the **Find in All Columns** option is turned *on*, and the **Instance** tab is the active window, then "fsm" is searched in all the columns available in the **Instance**

Feedback

tab. When the **Find in All Columns** option is turned *off*, then the tooltip is displayed which provides information about where the search string is available, as shown in the following figure:

Figure 75 Tooltip Information



By default, the **Find in All Columns** option is turned *off*.

Note:

The **Find in All Columns** option is enabled if the current active window is one of following:

1. Constraint Debug tab
2. Class tab
3. Object tab
4. Member pane
5. Stack pane
6. Local pane
7. Watch pane
8. Assertion Debug
9. Instance Tree (enable this option **Source Code->Design Tree->Search and Filter in Expanded Tree** in the *Preferences* form to search for the string in the *Instance* tab, instead of Declaration tab)
10. Signal List
11. Message Pane (enable this option when current tab is *OneTrace* and has content available in it)

Feedback

When the **Wildcard**, **Regular Expression**, **Find Whole Words Only** are all turned *off*, it is in default mode. That is, you try to find an item that partially matches the pattern even if wildcard characters are added.

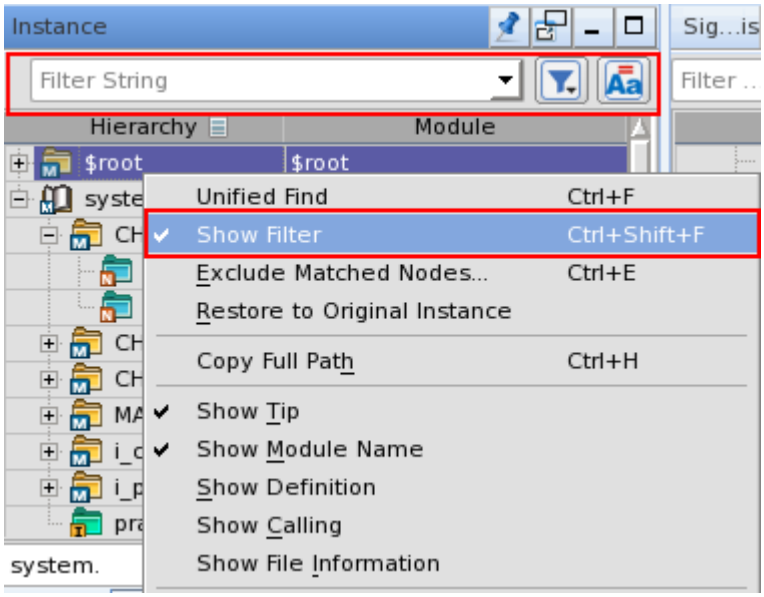
You can also set the default option in the **Find/filter** page (**General -> Find/Filter -> Find Options**) of the *Preferences* form.

Enhancements to the Right-Click Options

Following are the enhancements to the right-click options:

- The **Show Search/Filter** right-click option is renamed to **Show Filter** and the shortcut key is updated from **Ctrl+F** to **Ctrl+Shift+F** in Verdi. Also, the **Search String** text field is removed as shown in the following figure.

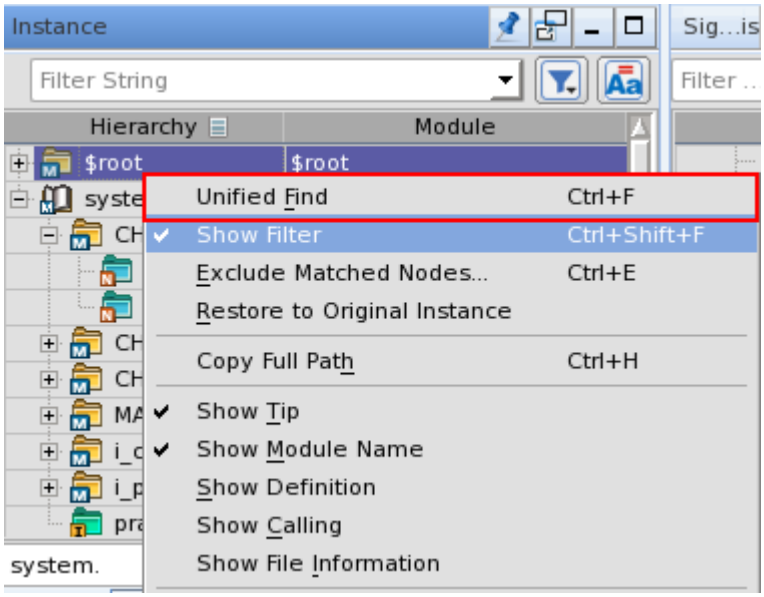
Figure 76 Example of Show Filter Option



Feedback

- The **Unified Find** right-click option with the shortcut key **Ctrl+F** is added in the *Verdi* windows as shown in the following figure:

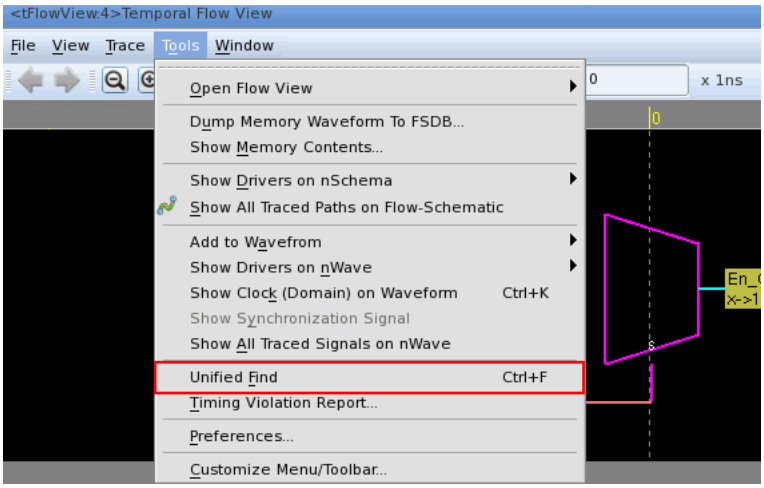
Figure 77 Example of Unified Find Option



Enhancements to the Temporal Flow View Tools Menu

In the **Tools** menu, the **Unified Find** menu option with the shortcut key **Ctrl+F** is added as shown in the following figure:

Figure 78 Unified Find Menu Option



Feedback

Enhancements to the Preferences Form

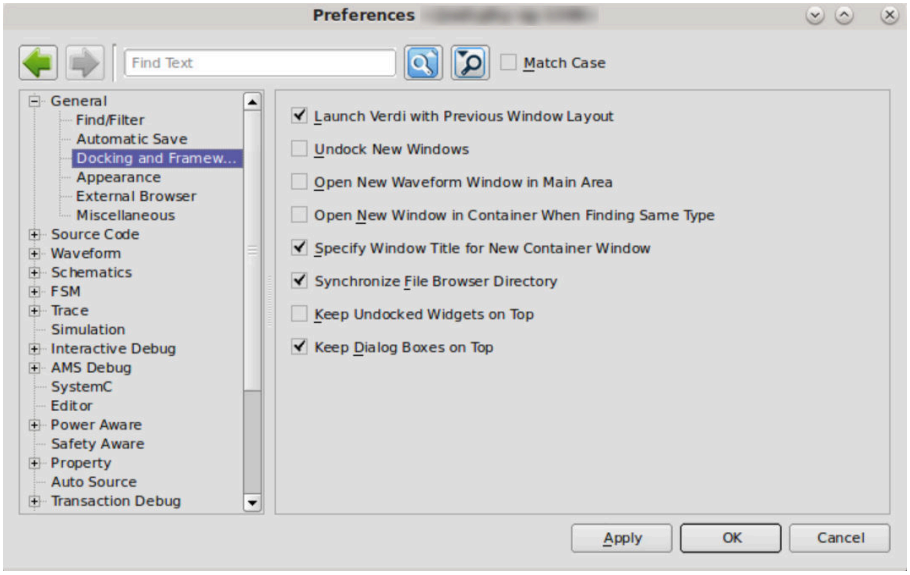
The following pages are enhanced to simplify search and find options:

- Docking and Framework Page
- Find/Filter Page
- Miscellaneous Page

Docking and Framework Page

The **Spotlight Search** option is moved from the **Docking and Framework** page to the **Find/Filter** page as shown in the following figure:

Figure 79 Docking and Framework Page

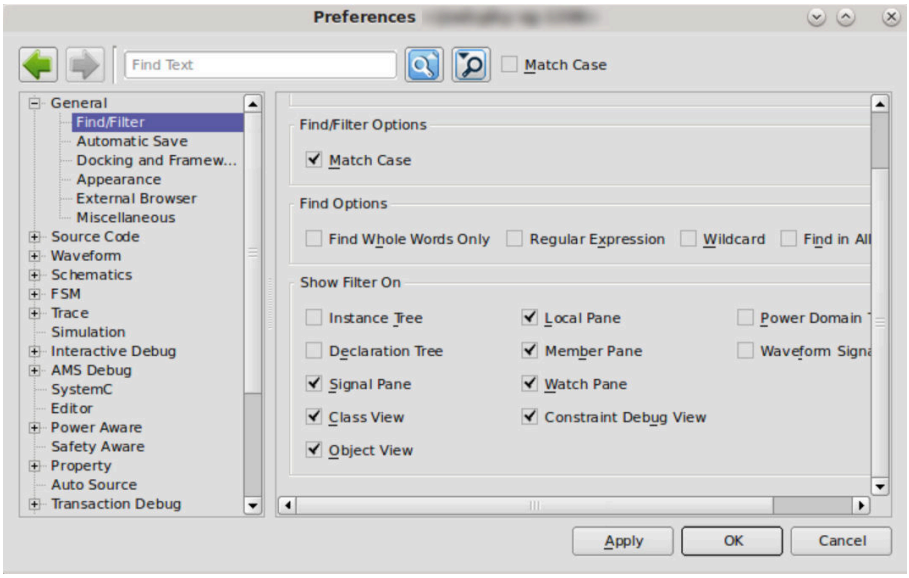


Feedback

Find/Filter Page

In the **General** folder -> select the **Find/Filter** page (invoked using **Tools -> Preferences**) as shown in the following figure:

Figure 80 Find/Filter Page



The following option is available in the **Spotlight Search** section:

- **Default Target**: Select the preferred default option (**String/Signal, Menu**, **OneSearch**, **Reference Manual**, **Tcl**, or **Preferences**) to search for a list of objects matching the search string in the **Spotlight Search** icon. The default value of this option is *String/Signal*.

The following option is available in the **Find/Filter Options** section:

- **Match Case**: When this option is turned *on*, the find and filter is case-sensitive. When this option is turned *off*, case is not considered during a search. The default value of this option is *off*. This option affects the **Unified Find** option, also the default value of other **Search/Find** features and the filter option in each component.

The following options are available in the **Find Options** section:

- **Find Whole Words Only**: When this option is turned *on*, the **Find Whole Words Only** option is enabled by default in the **String/Signal** option.
- **Regular Expression**: When this option is turned *on*, the **Regular Expression** option is enabled by default in the **String/Signal** option.

Feedback

- **Wildcard**: When this option is turned *on*, the **Wildcard** option is enabled by default in the **String/Signal** option.
- **Find in All Columns**: When this option is turned *on*, the **Find in All Columns** option is enabled by default in the **String/Signal** option. By default, the **Find in All Columns** option is turned *off*.

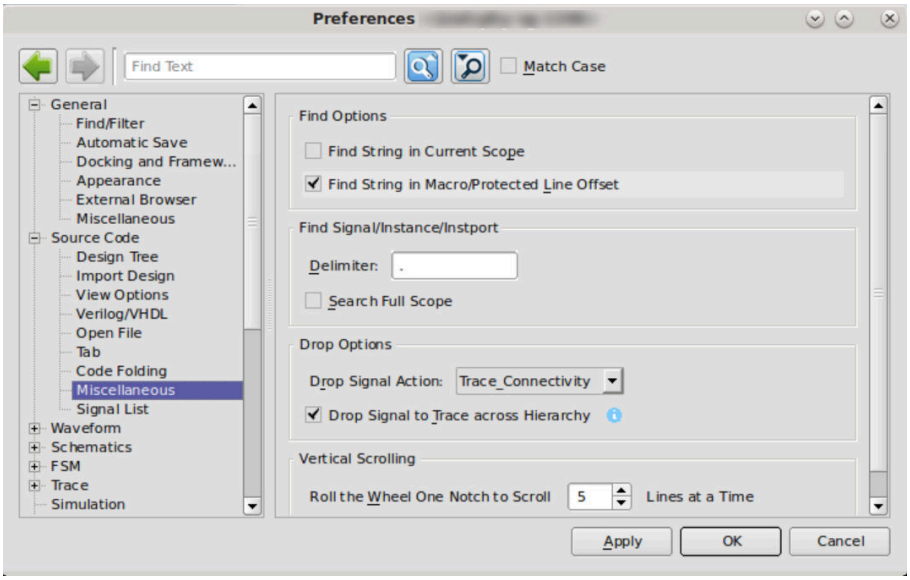
The **Enable Search/Filter on** section is renamed to **Show Filter on** section.

In the **Show Filter on** section the **Stack Pane** and **UVM Register View** options are removed.

Miscellaneous Page

In the **Source Code** folder -> **Miscellaneous** page (invoked using **Tools -> Preferences**), the **Case Matching** option is removed as shown in the following figure:

Figure 81 Miscellaneous Page



Feedback

Limitations

This feature has the following limitations:

- The following components highlight the matched items searched using spotlight function:
  - SmartLog
  - Console
  - OneSearch Search within Result
- The Spotlight search is fixed in the menu and cannot be moved.
- Spotlight search and other search types do not support the same options and syntax as **String/Signal** option.
- The **Unified Find** option for *nSchema* supports searching of only signals and not instances even if instances are available.
- In **SmartLog** folder, under ***General*** page, if **Enable Loop Searching** is turned *off* then the Loop Searching is disabled, that is, the search stops after it hits the end of the log even if **Unified Find** option is turned `on`.
- The *Source Code* pane and the *Instance Tree* does not support multiple keywords that are separated by the OR operator ("||") in the default match mode.
- The **Find in All Columns** option is disabled if the current active window is one of following:
  1. nClock Tree Browser or Clock Domains window
  2. nClock Extract
  3. Console window
  4. SmartLog window
  5. nWave Event Sequence window
  6. Clock Browser
  7. Emulation Clock Window
  8. Design Manipulation

- The **Find in All Columns** option is not available if the current active window/pane is one of following as the window/pane does not have multiple columns:

  1. Source Code
  2. nSchema window
  3. Waveform Signal pane
  4. Declaration Tree pane
  5. VC AutoTestbench
  6. Protocol Analyzer window
  7. Performance Analyzer window
  8. Transaction Table View window
  9. Transaction Relationship Navigator window
  10. OVM/UVM Hierarchy Tree pane
  11. UVM Register View
  12. Temporal Flow View window
  13. Power State Table pane
  14. SmartSheet
  15. nCompare Error Report Tree
  16. zTime Path View
  17. VCST SDCFiles
  18. Design Check View

Support for Active Window in Verdi

Verdi has many docked windows/panes in the top window and each docked pane can have different selections. Commands in the top window can be applied on these selections. You cannot identify which window is active currently. Therefore, this causes confusion on which commands in the top window apply for which docked pane's selection.

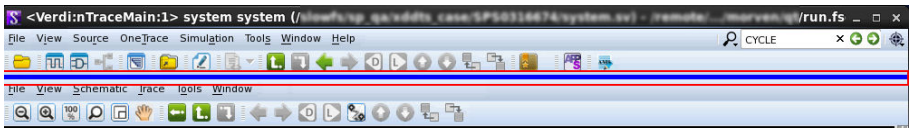
You can click on the docked pane to trigger as the active window with a dark-blue banner, and the top window commands (menu commands and toolbar commands) can be applied to the active window's selection. Verdi disables the commands if it cannot be applied in the current active window, or if the commands are specific to the source view/signal list. These commands are activated only if source view/signal list is the current active window.

Feedback

To make a window active, perform either of the following options:

1. Click the mouse button (left, right, or center) on the window.
2. Click the banner on the window. The banner color changes to blue. When the docked banner is hidden using the **Hide Banners** option, then a blue line appears as shown in the following figure:

Figure 82 Banner Color

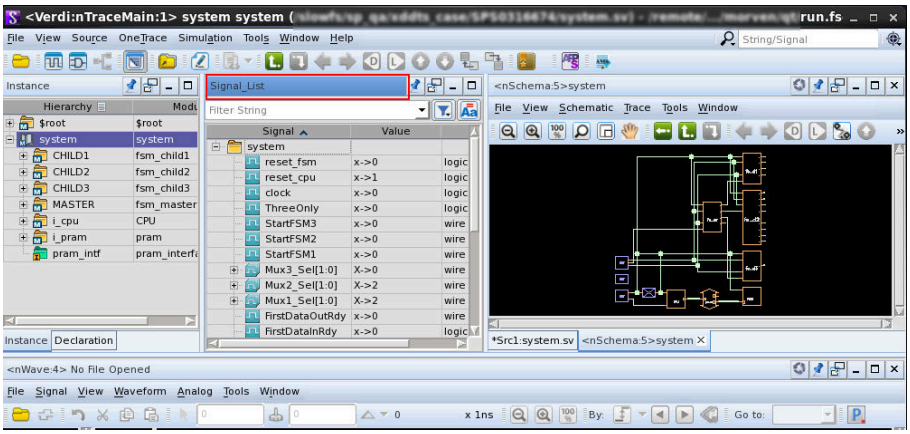


3. Drag an item/items from a window to make it active.

Note:
Drop operation does not make a window active.

You can also use the shortcut key **Ctrl+Shift+Tab** to switch active windows for visible docked views.

Figure 83 Example of Active Window



Scenarios of Active Window

- When the current active window is closed or hidden, then the last visible docked window becomes the active window.

  If the docked view is in the middle, then the next docked view becomes visible and active. For example, if the docked view is in the middle (for example, *nSchema6*), then next dock view (*nSchema7*) become visible and active.

- If the docked view is on the right, then the previous window becomes visible and active.

Limitations

This feature has the following limitations:

- When the window is undocked, then you cannot run the **trace/radix/notation** command in the top window.
- When the window is docked in container window, then you cannot run the shortcut commands in top window.
- When *Signal List* has different selection with source view, then some trace command and radix/notation command are disabled.
- A shortcut is applied on an active window only if the same commands that are available on the top window are also available in the active window.

  For example, TFV does not have **OneTrace** commands, therefore **OneTrace** menu commands and shortcuts (**OneTrace -> Driver**) do not apply even if TFV is the active window and **OneTrace** commands are available in the top window.

Next Generation Debug GUI

Verdi enhances the look and feel of the GUI by providing the following customized Next-Gen GUI enhancements:

- Color Themes
- Grid and Density
- Banners
- Search
- Find
- Quick Launcher
- Recently Used Command Buttons
- Toolbar With Text and Icon

Color Themes

You can configure the color theme through preferences for the preferred look and feel.

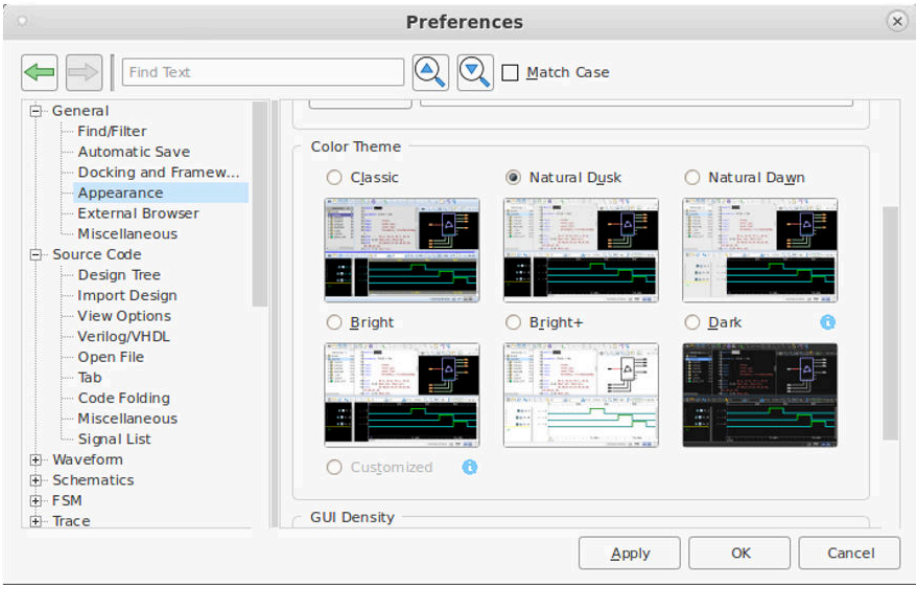
The following figure illustrates the various color themes supported by the Verdi GUI.

Feedback

Note:

Dark mode and other modes cannot be switched on-the-fly. The settings are saved to*novas.rc* and applied next time when Verdi GUI is invoked.

Figure 84 Themes



Limitations

Following are the limitations of the color themes:

- Runtime switch from any color theme in light mode to dark mode is not supported due to Qt4 limitation. You can only enable dark mode by setting the environment variable `VERDI_COLOR_THEME dark` or launching Verdi with the `-dark` option.
- Color settings in `novas.rc` file may conflict with the selected color theme when launching Verdi, thus overwriting the required theme color. To avoid this, remove all the `rc` settings according to the color when the user switches between light/dark color themes. If there are no `rc` settings, there is no overwrite issue.
- Only backward compatibility is supported for old `novas.rc`, which means that, if users open Verdi with the old `novas.rc`, then the default color setting is applied for them as some `rc` keys of new GUI features do not exist in old `novas.rc`. When user launches Verdi in dark mode and closes it, the dark color setting is saved in `novas.rc`. If users open an old version of Verdi or with a disabled color theme environment variable next time, the dark color setting is still loaded which displays Verdi GUI with the incorrect background colors (some windows have black background, some have

Feedback

gray background). If users open an old version of Verdi with dark theme `novas.rc`, a popup message is displayed to inform the users the current application version is older than the `rc` file version, which may lead to incompatible problems.

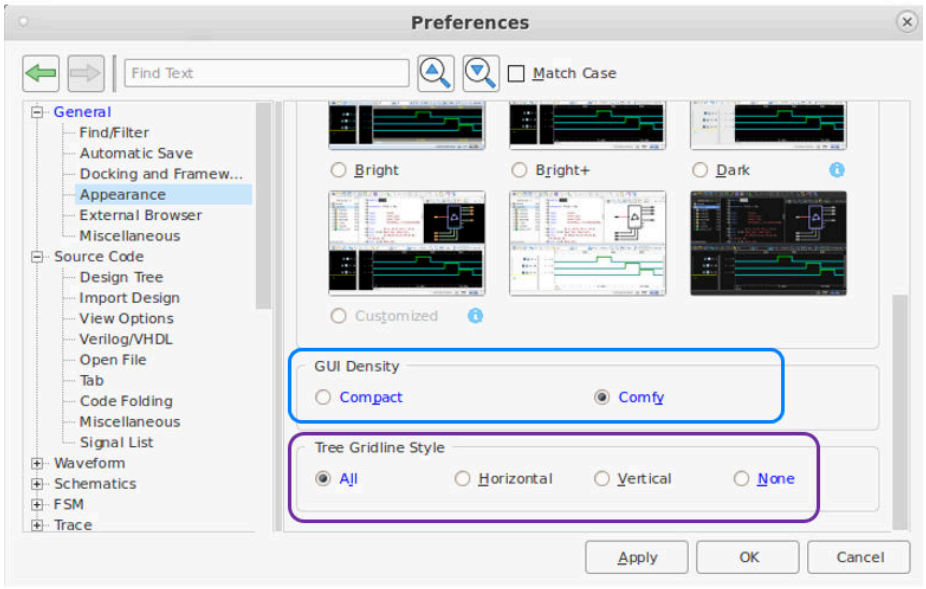
Grid and Density

Verdi supports customizing the grid and density GUI features.

Perform the following steps to customize the GUI density feature:

1. Launch Verdi GUI.
2. Select **Preferences > General > Appearance** option.
3. Scroll-down the page and observe that **GUI Density** section provides the following features, as shown in the following figure.
   - Compact
   - Comfy

Figure 85 GUI Density

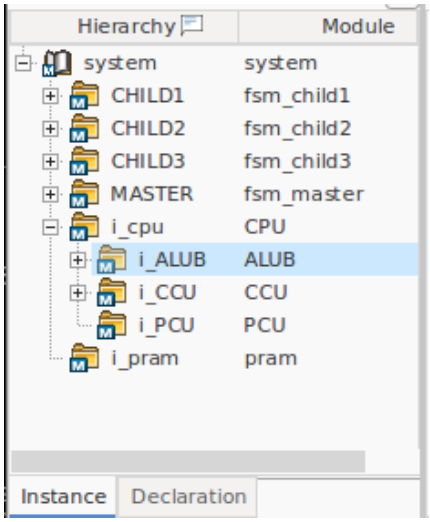


4. Select **Compact** radio button.

Feedback

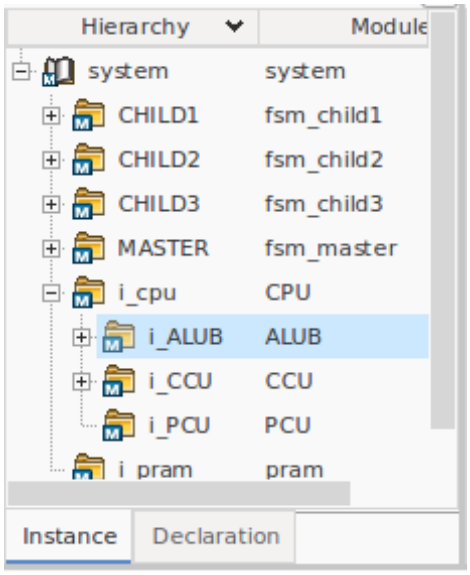
5. Click **Apply** and **OK** options for the compact view of the hierarchical modules, as shown in the following figure:

Figure 86 Compact View



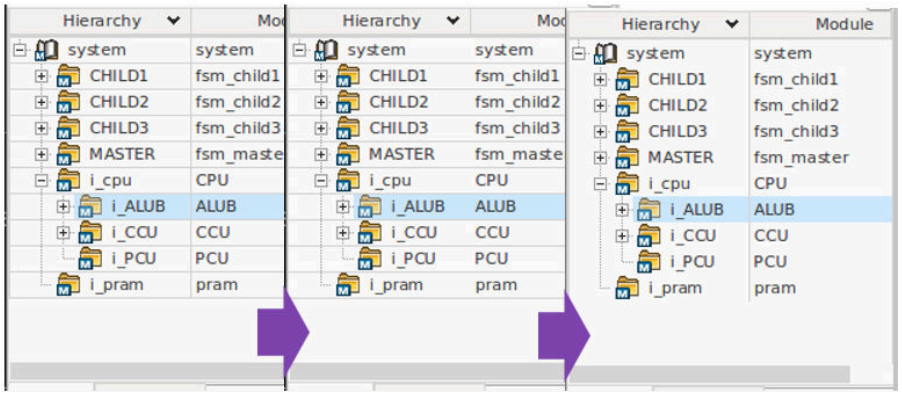
6. Select **Comfy** radio button.
7. Click **Apply** and **OK** options for the comfy view of the hierarchical modules, as shown in the following figure:

Figure 87 Comfy View



Feedback

8. When you scroll down the **Appearance** window, you can observe that **Tree Gridline Style** section, you can use this feature to configure the gridline styles. It provides the following gridline style options, as shown in the following figure:
   - **All**
   - **Horizontal**
   - **Vertical**
   - **None**



9. You can select the required gridline style and click **Apply** and **OK** options.

Banners

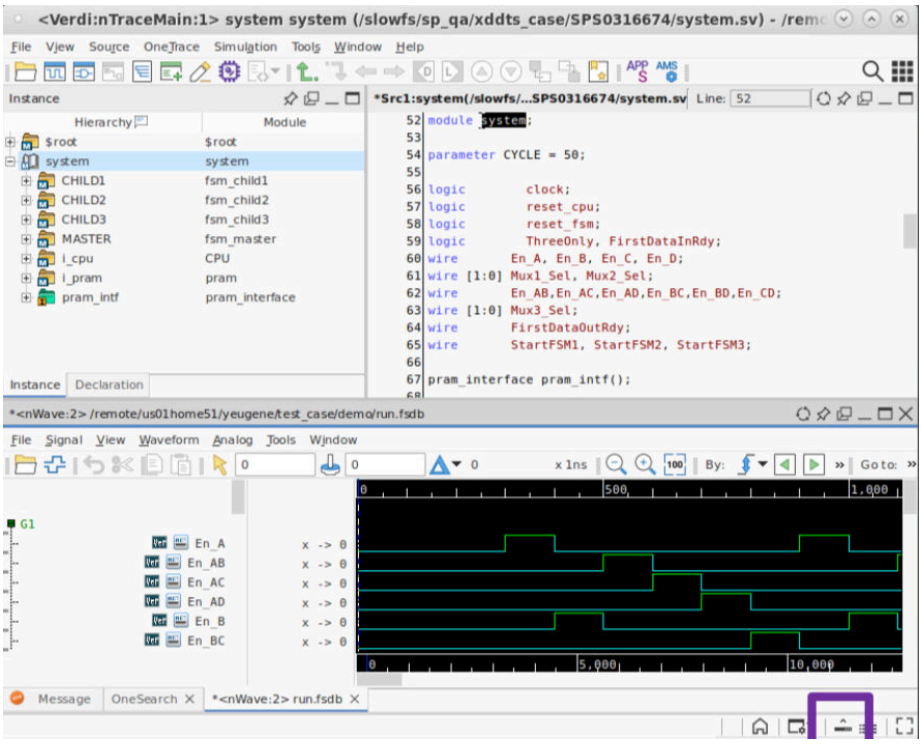
The banners are auto-collapsed for more vertical debug spacing.

Perform the following steps to collapse or expand the banners:

1. Launch Verdi GUI.
2. Select the icons as shown in the following figures to collapse or expand the banners.

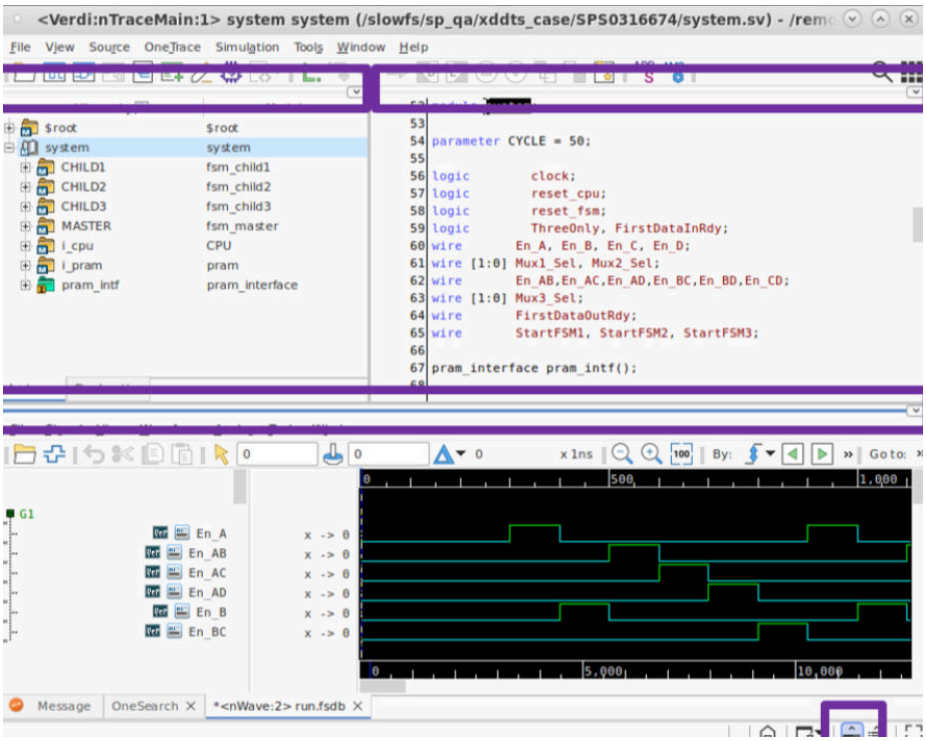
Feedback

Figure 88 Banners



Feedback

Figure 89 Auto-Collapse Docked Banners



Find

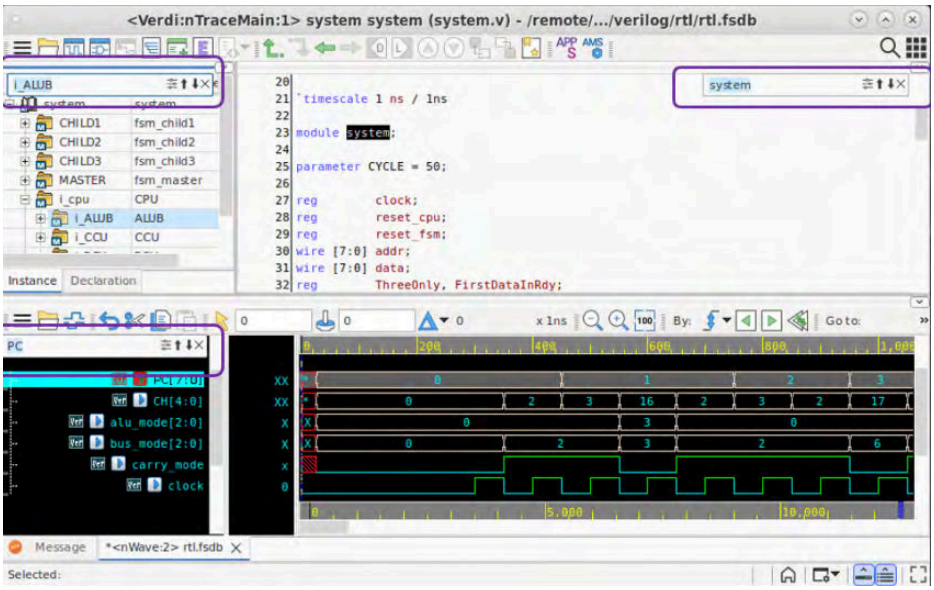
Verdi allows you to find information in a specific window by pressing **Ctrl+F** keys in a window. Each window has a separate find bar. Following are the various search constraints supported by this feature:

- Match Case
- Wildcard
- Regular Expression
- Find Whole Words Only

Feedback

The following figure illustrates how the find feature is used in the specific windows:

Figure 90 Find



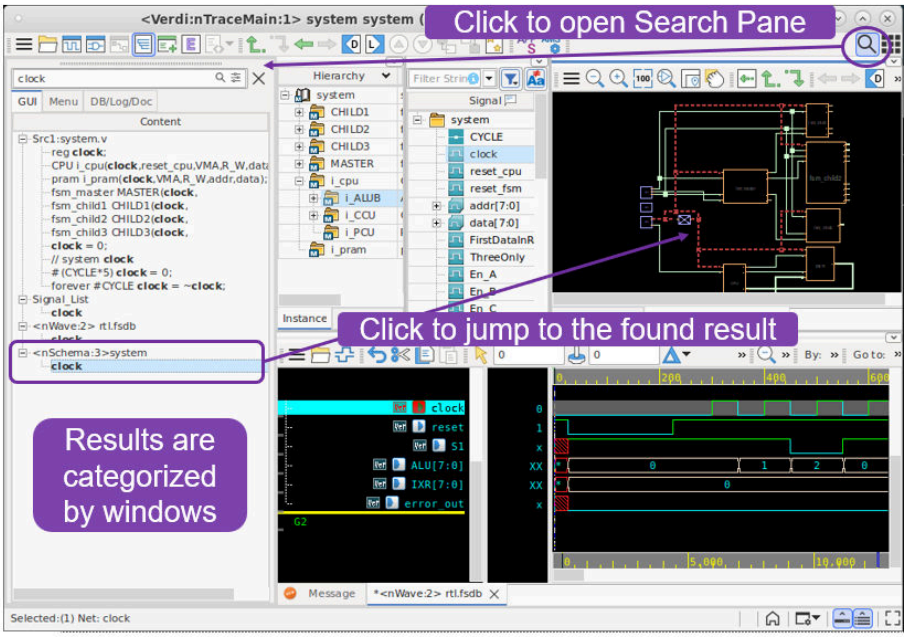
Search

Verdi provides search pane for the broader search. Search applies to all the windows of the Verdi.

Feedback

Perform the following steps to use the search feature:

1. Click *search* (magnifying glass) icon on the upper right corner of the window to open the search pane, as shown in the following figure:

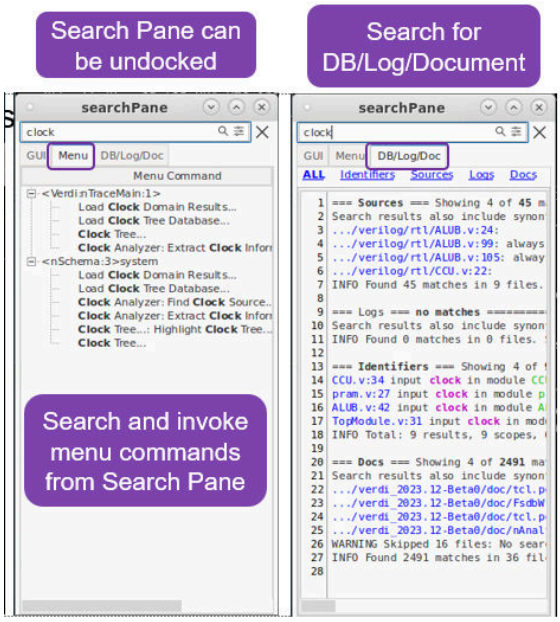


2. Enter any search key/text in the search pane.
3. Click *search* (magnifying glass) icon in the search pane.
4. Click on any searched result to go to the found result.
5. Search pane can be undocked as an individual pane and has the following tabs to search the results based on these categories:
   - **GUI**
   - **Menu**
   - **DB/Log/Doc**

Feedback

6. You can choose the required tab and search the information.

Figure 91 Undocked Search Pane and Search Tabs



Limitations

Following are the limitations of this feature:

- The correct behavior of searching when the editor is on editing and unsaved status is not guaranteed. The result content may be wrong.
  - If line content was deleted or changed, although the current content does not match with the search keyword, it may still show on the search pane when the previous document matches the keyword before modified.
  - If line content was added, although the fresh new content matches with the keyword, Verdi cannot read in the content and search pane does not show the fresh matched line content.
- The correct behavior of the jumping when the editor is edited after searching is not guaranteed. As a result items display with the line number of the matched line of the old document. If document content changes, the line number of the new document does not always reflect the match line number of the result. Any clicking on the result item may lead to unpredictable behavior.
- GUI tab in new search pane apply blocking search which may cause user to wait for the search results, therefore, a busy dialog is provided for the earlier abort.

Feedback

- All window supported unified find has supported find bar, but not all the supported search pane.
- Onesearch has its own special searching rule and non-blocking search and other tab in search does not work.
- Original onesearch Tcl does not work.
- The following windows are not supported:
  - OVM/UVM Hierarchy tree view
  - Interactive Debug

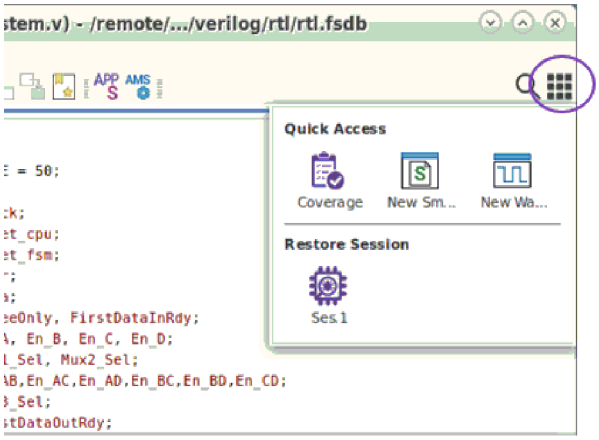
Quick Launcher

Verdi provides Quick Launcher feature to allow you to access the integrated tools/ applications/sessions quickly.

Perform the following steps to open the quick launcher:

1. Click **Quick Launch** icon (as shown in the following figure) on the upper right corner of the window to open the quick launcher window, as shown in the following figure:

Figure 92 Quick Launcher



You can view **Quick Access** and **Restore Session** categories in the form. Integrated tools are placed in the form by default. Sessions under working directory are placed in the form.

1. Click on the required application/tool/session to open/launch quickly

Feedback

Menu and customized commands can be placed in the form using *Tcl*, as shown in the following commands:

- `verdiAppLauncher -add -act logViewer`
- `verdiAppLauncher -add -act hbInvokeWave`

Recently Used Command Buttons

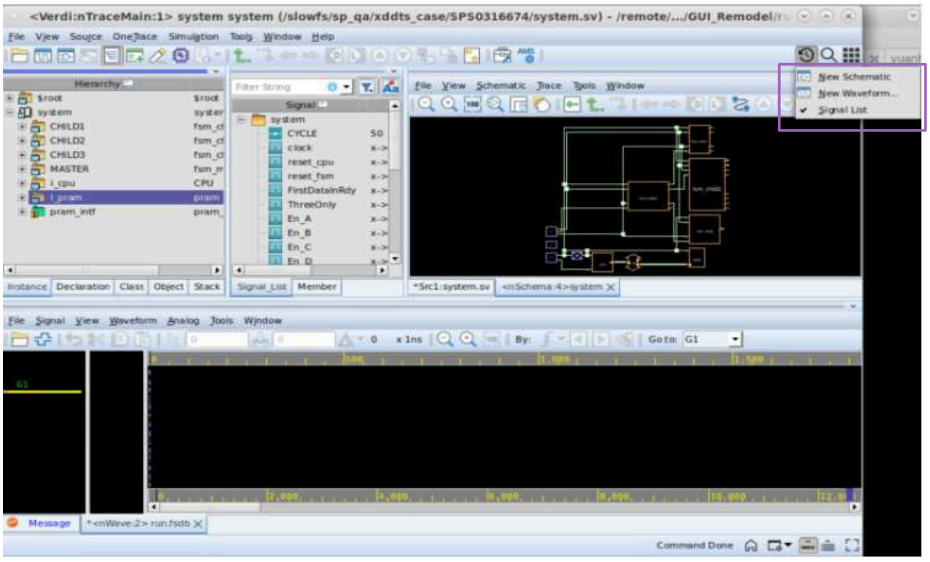
Verdi provides a button in the upper-right corner that provides a drop-down list for recently used commands according to the active windows. This button is shown on demand (for example, at least one recently used command must be there).

Verdi stores 10 recently used menu commands and toolbar commands of each window type. Different windows with the same window type (for example, nWave2, nWave3, nWave4 and so on) shares the same recently used commands. Each window type has their own recently used command set.

The recently used commands are stored in novas.rc so that next time when user invokes Verdi, recently used command button restores the setting from novas.rc.

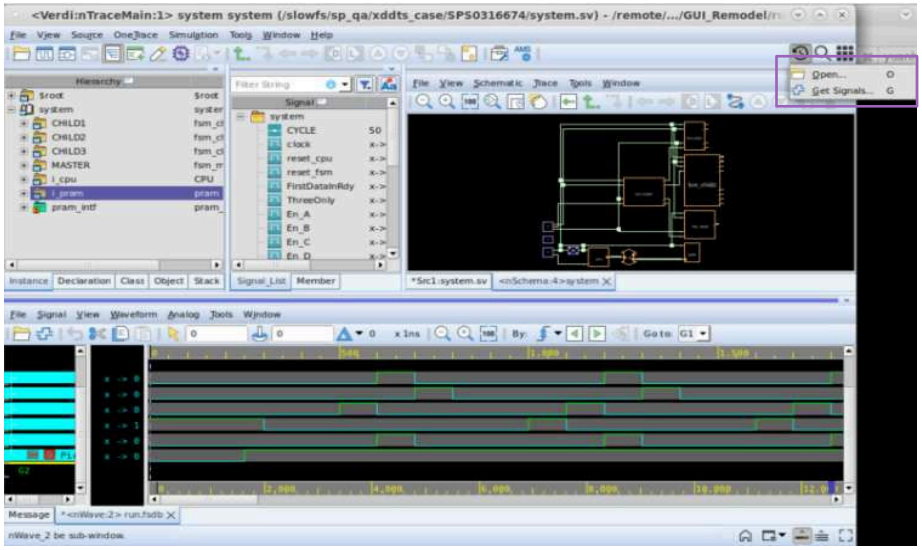
The following figures illustrate the recently used command buttons in Verdi top and nWave windows:

Figure 93 Recently Used Command Buttons in Verdi Top Window



Feedback

Figure 94 Recently Used Command Buttons in nWave Window



Toolbar With Text and Icon

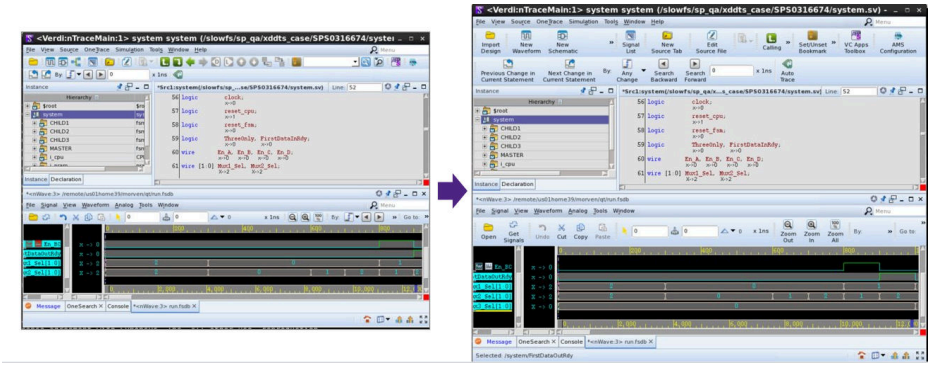
Verdi provides a toolbar with text and icon feature help Rockie and Zebu users to easily get started.

Use the following environment variable to enable this feature:

```
setenv VERDI_TOOLBAR_TEXT_UNDER_ICON 1
```

The following figure illustrates the toolbar with text and icon feature:

Figure 95 Toolbar With Text and Icon



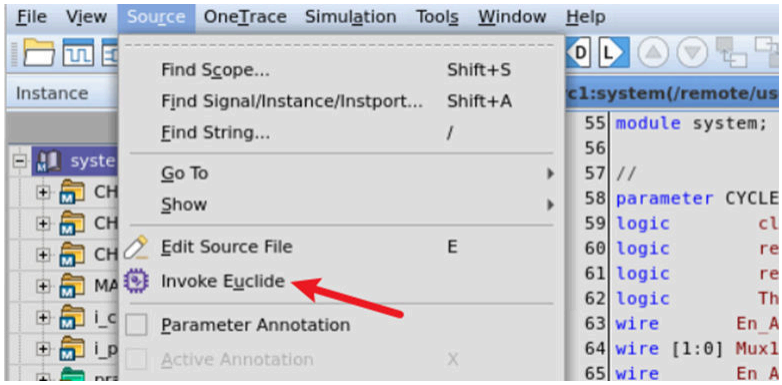
Feedback

Verdi Integration With Euclide IDE

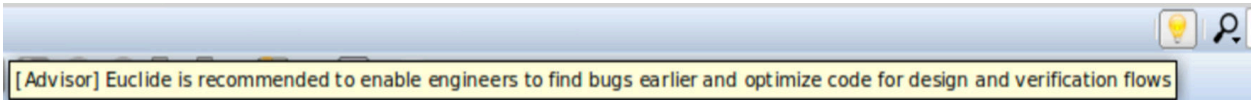
Euclide is an Eclipse-based integrated development environment (IDE) for SystemVerilog featuring on-the-fly incremental compilation, elaboration, pseudo-synthesis and rule checking, all of which are integrated into the editor and provides feedback in seconds. Euclide helps to reduce implementation bugs in RTL and testbench, improving project convergence rate, and eliminating patchy code

You may invoke the Euclide GUI from the Verdi GUI by:

- clicking the icon on the toolbar
- navigating to **Source** → **Invoke Euclide** menu command as shown in the following figure:



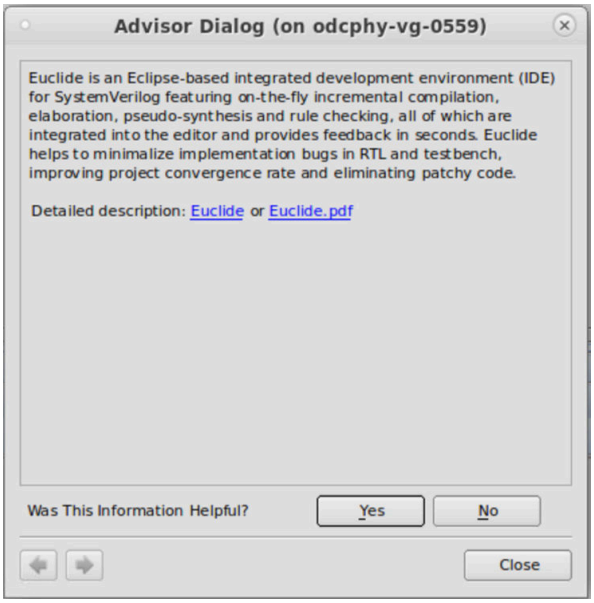
When you click the Invoke Euclide button for the first time, the advisor bulb blinks. When you hover over the advisor bulb, the following recommendation is displayed:



The same information is displayed in the Status bar as well. You may also access the Advisor Dialog using the **Help** → **Advisor** menu command.

Feedback

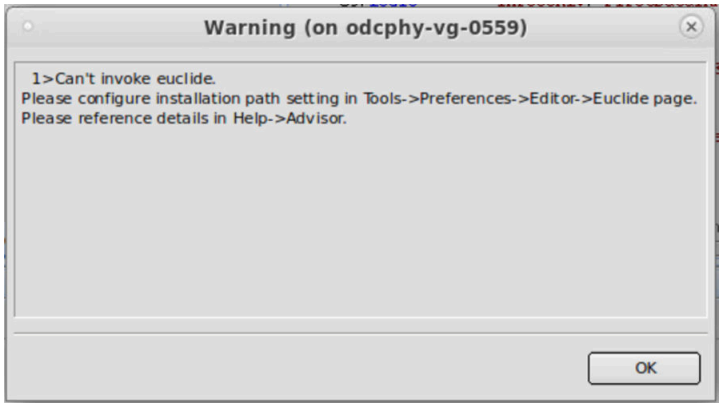
The Advisor Dialog is as follows:



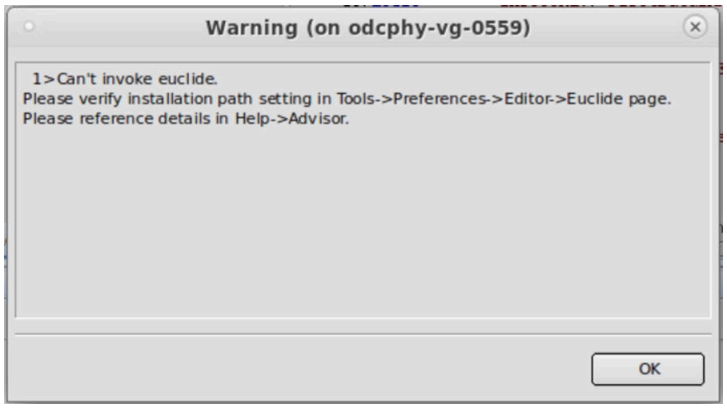
The following warning dialog boxes pops up when you click the Euclide toolbar icon and

Feedback

- if Euclide installation path is not configured:



- if Euclide installation path is configured, however, failed to find:



6

nTrace Tutorial

This chapter describes the following topics:

- Overview
- Traverse the Design Hierarchy in nTrace
- Access a Block's Source Code
- Trace Drivers and Loads
- Edit Source Code
- Use Active Annotation
- Tracing the Active Driver
- Fault Source Code
- Support for Specify Array Index

Overview

The *nTrace* main window is a source code viewer and analyzer that operates on the KDB to display the design hierarchy and source code (Verilog, VHDL, SystemVerilog, mixed) for selected design blocks. The Verdi platform quickly identifies signal connectivity information (drivers and loads) without any simulation overhead. With the FSDB, the simulation results can be back-annotated in the source code and then the Verdi platform can analyze and determine a signal's active driver at a particular simulation time.

Before you begin this tutorial, follow the instructions in the Before You Begin chapter.

Refer to the Launching Techniques chapter for more information on starting the Verdi platform and opening the *nTrace* main window, which is the default window. Also refer to the User Interface chapter for more details regarding the *nTrace* interface.

Traverse the Design Hierarchy in nTrace

You can traverse the design hierarchy to understand the design structure.

Feedback

1. Change to the demo directory.

   ```
   % cd <working_dir>/demo/verilog/cpu/src
   ```

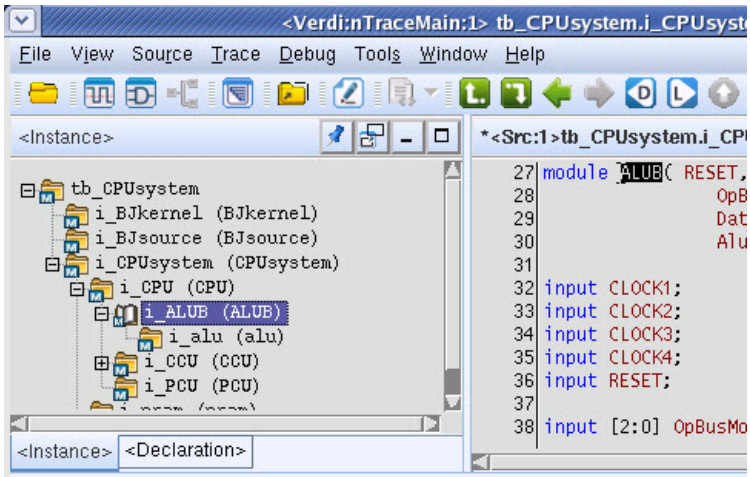
2. Start the design using the following command:

   ```
   % verdi -f run.f -workMode hardwareDebug &
   ```

   Note:
   This tutorial uses a Verilog design example. The same capability is available for VHDL or mixed designs.

3. To expand a block hierarchy on the **Instance** tab in the design browser frame, click the plus symbol to the left of the *i_CPUsystem* block instance name to reveal its *i_CPU* and *i_pram* subblocks.

Figure 96 Expand the Hierarchy in nTrace



4. Click the plus symbol to the left of the *i_CPU* block instance name to reveal its *i_ALUB*, *i_CCU*, and *i_PCU* subblocks.

5. To collapse the hierarchy, click the minus symbol to the left of the name.

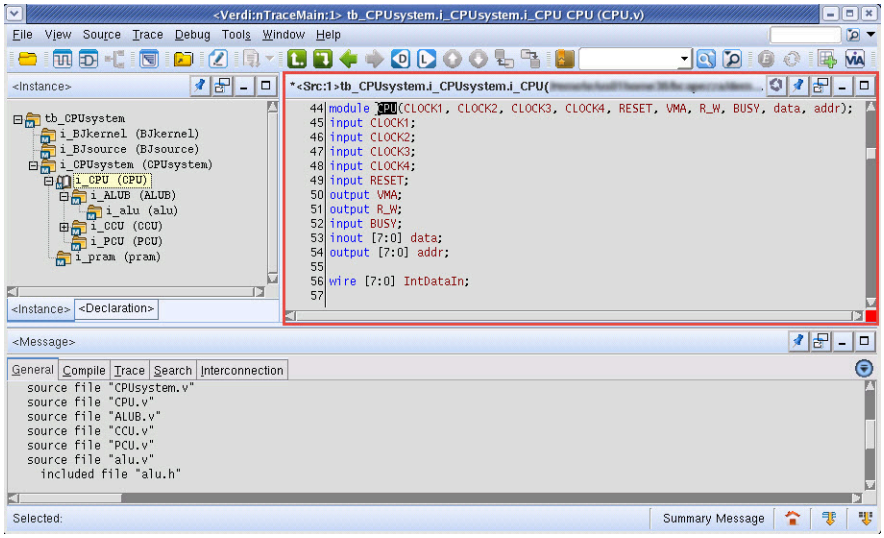
The plus/minus symbols to the left of the block instance names in the design browser frame are used to expand/collapse the display of the selected block's hierarchy.

Feedback

Access a Block's Source Code

1. To access source code, double-click the *i_CPU*unit instance name in the design browser frame. The source code is displayed in the Source Code frame, as shown in the following figure:

Figure 97 Source Code for the CPU Block



   By default, the name of the block (*CPU*) is highlighted in the source code.

2. Double-click *CPU* to change the source code context to the calling block, which is *CPUsystem* (the corresponding *i_CPUsystem* block instance name is automatically highlighted in the design browser frame).

   By default, the instantiation of the previous block (*i_CPU*) is highlighted in the source code.

3. Double-click *i_CPU* to return the source code context to the *CPU* block.

4. You can also click the right mouse button to access a menu with the **Show Calling** and **Show Definition** commands to display the calling or definition of the block.

Goto Line

In the *nTrace* Source toolbar, **Goto Line**: text box is available in which you can enter the line number, which directly points to the line number in the *Source code* frame. The cursor automatically moves to the line and the line comes at the top of the *Source code* frame.

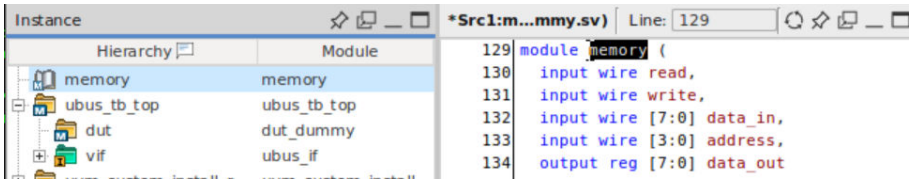
Feedback

Use Model

This is about Verdi automatically showing the line information in the source code view. The following figures depict the default behavior of Verdi based on the user operations.

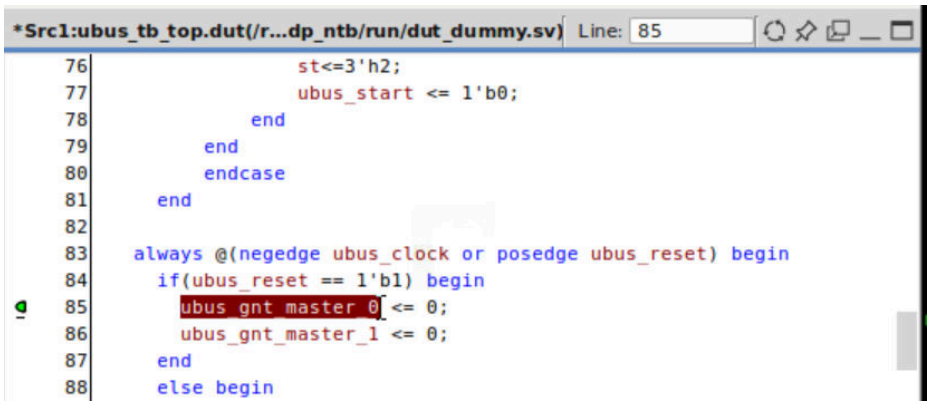
- The following figure shows the field that displays the current line in the source code that is linked to or connected to the target object by other windows:

Figure 98 Current Line Position



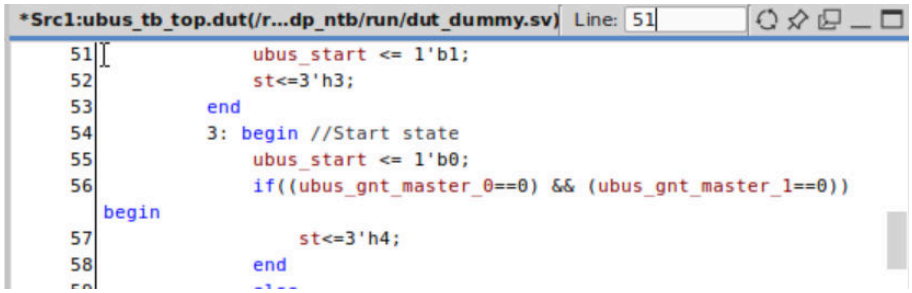
- The following figure shows the field where the tracing stops:

Figure 99 Tracing Stop Point



- You can enter the line number where you want to jump to in the source code view as shown in the following figure:

Figure 100 Target Line Position



Feedback

Note:

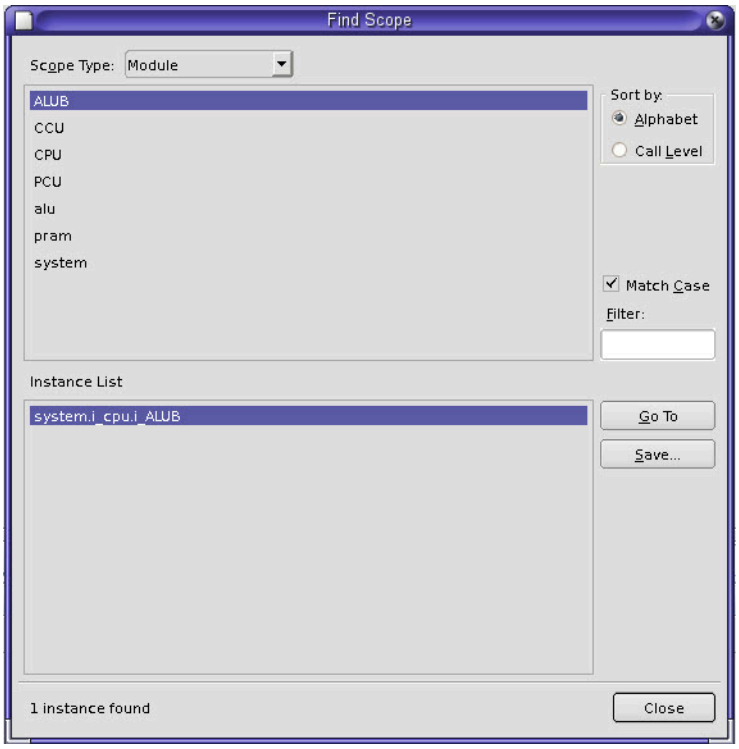
- Enter only positive integers in the **Line** text box.
- If the entered line number is greater than the total line count of the current *Source code* frame, cursor navigates to the last line of the *Source code* frame.
- If you change the cursor position with an action in the *Source code* frame, the line number in the **Line** text box is updated.
- The Goto line text box supports multiple *Source code* frames and Power Views.
- When multiple *Source code* frames are available, a new **Line** text box is created for every new tab.

Find Scope

1. To locate a scope, choose the **Source -> Find Scope** command (or bindkey “S”).

   A *Find Scope* form displays, similar to the following example:

Figure 101 Find Scope Form



Feedback

2. Enter **CU** in the **Filter** box and press **Enter** on the keyboard.

   The top frame updates to display the modules matching the search string.

3. Left-click to select *PCU*.

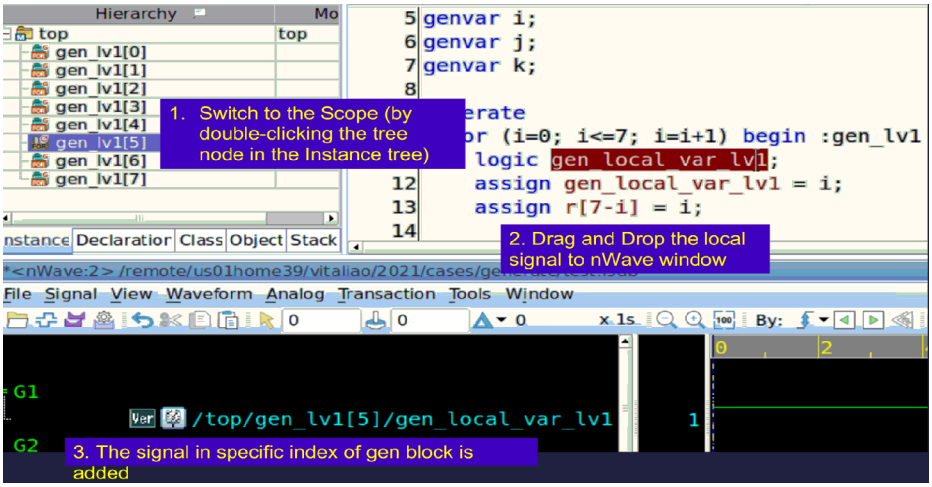
   The bottom frame lists all hierarchical paths for the module. In this case there is one path.

4. Click **Go To** to locate the associated module in the *Source Code*frame.

Variable Inside Generate Block From nTrace to nWave

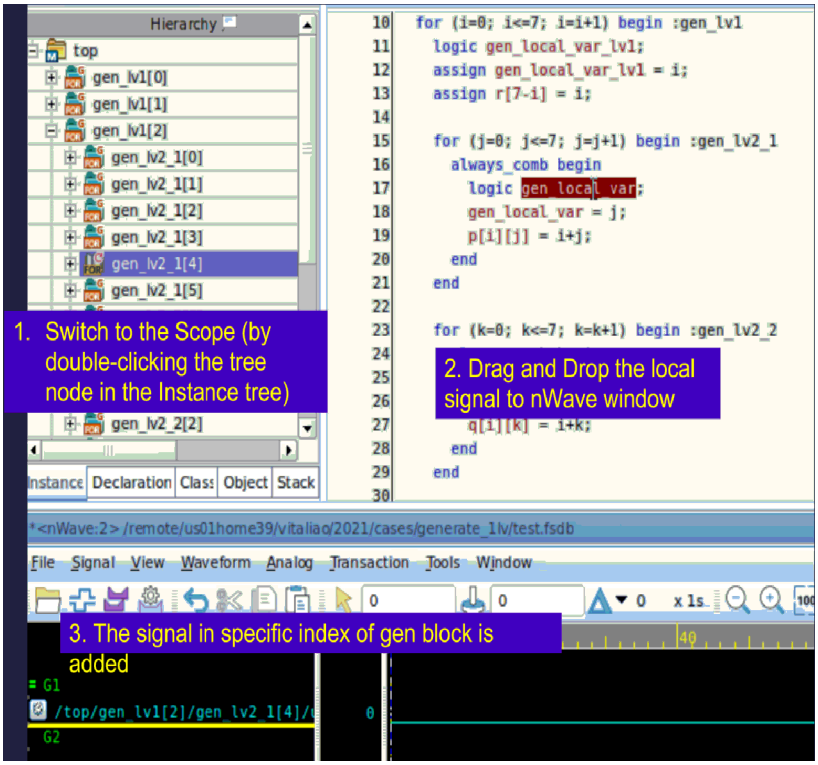
To bring signal which is defined inside generate block from nTrace to nWave, signal is selectable and all instances of the selected signal can be added from nTrace to nWave when drag and drop action or "ctrl" + "w" is performed.

It needs to switch to the scope and the local signal should be then selectable as shown in the following figure:



Also, this feature supports only one-level generate. To use this feature for local variable in the second and further levels, you need to switch/select to the active scope.

Feedback



Support for Multiple Range Selection in the Source View

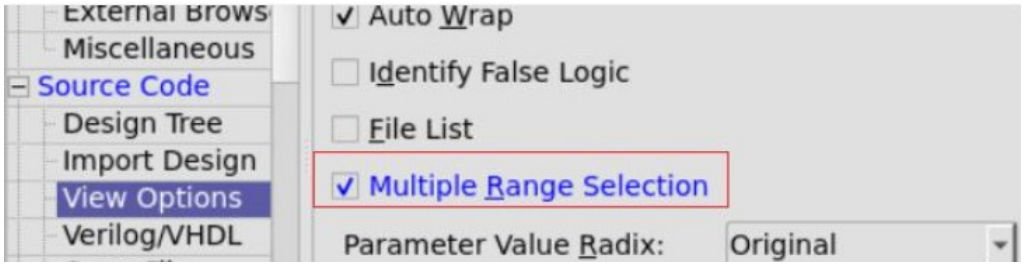
Verdi supports multiple range selection in the *Source Code* pane. To enable multiple range selection, perform the following steps:

1. Choose **Tools > Preferences** menu command.
2. Choose **Source Code > View Options**.
3. Select the **Multiple Range Selection** option.

Feedback

Refer to the following figure:

Figure 102 Multiple Range Selection

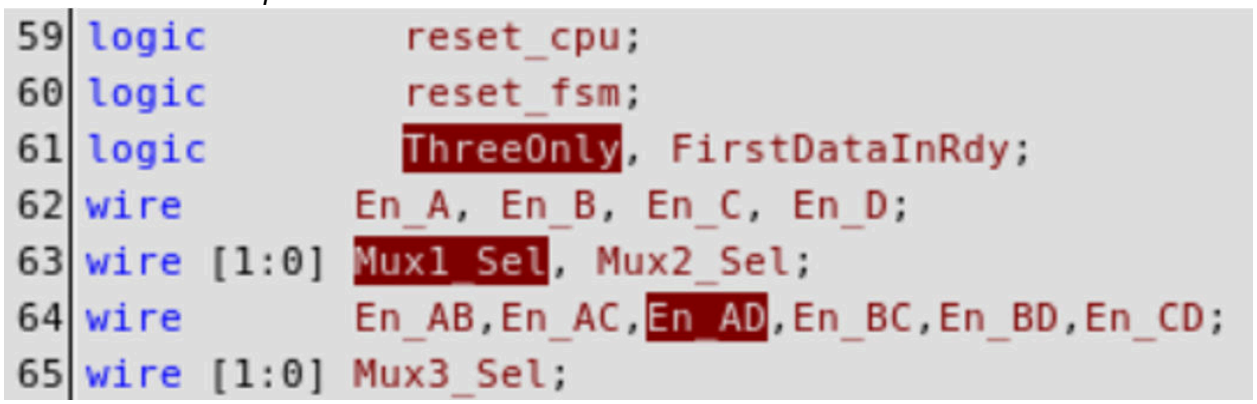


By default, this option is not selected.

The following are the ways to select and manage multiple range selections in the *nTrace Source Code* Pane:

- Press Ctrl and left-click on a word to select the word. You can select multiple words or areas as shown in the following figure:

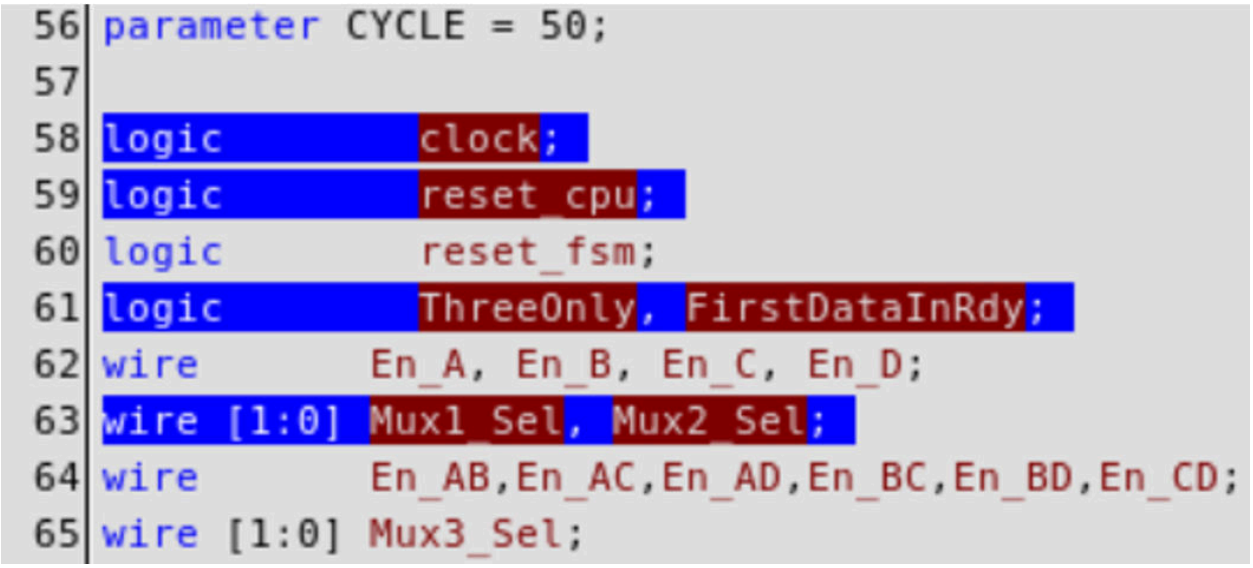
Figure 103 Select Multiple Words



- Press Ctrl, left-click, and drag to create a new selection area while keeping the original selection intact. If the new area overlaps with the original selection, the two areas merge into one larger selection.

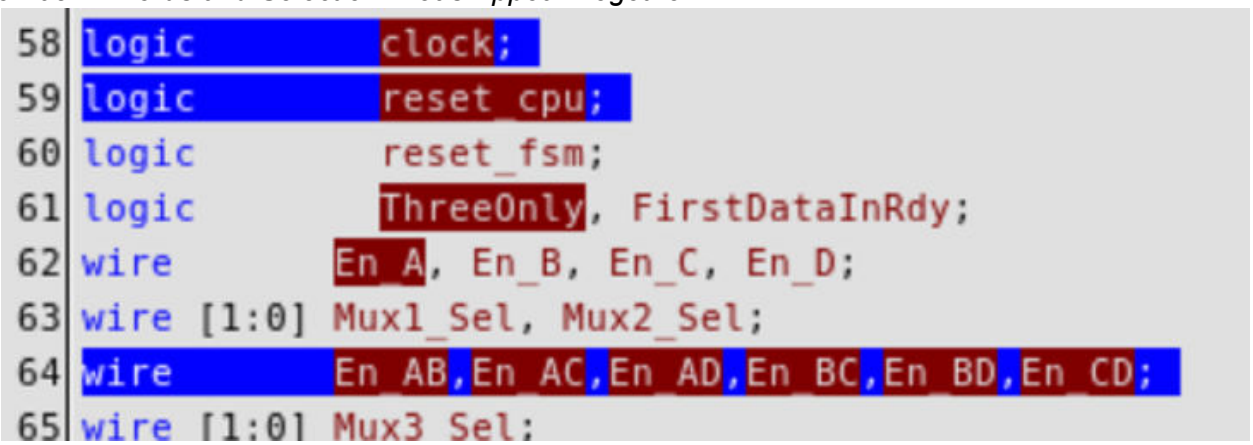
Feedback

Figure 104 Create a New Selection



The selected word and selection area appear together as shown in the following figure:

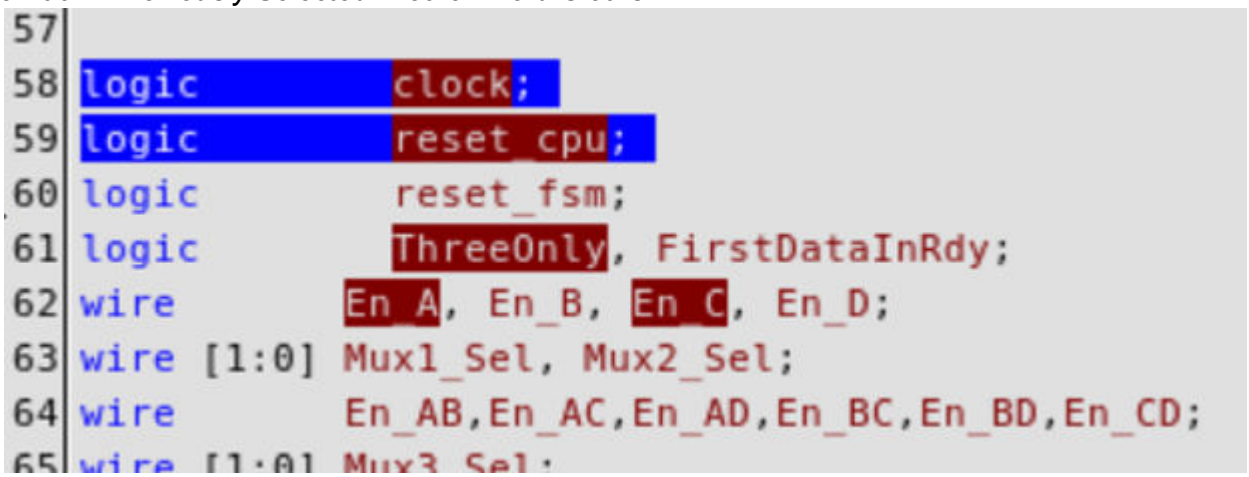
Figure 105 Words and Selection Areas Appear Together



Feedback

- Press Ctrl and left-click in a previously selected area or on a word to clear that specific selection as shown in the following figure:

Figure 106 Previously Selected Area or Word Clears



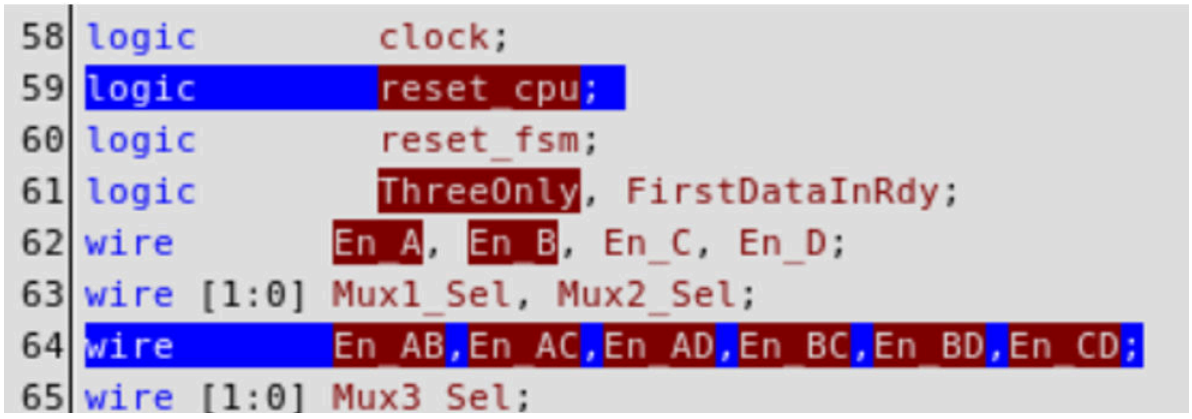
Click without pressing Ctrl to clear all selections.

- If the mouse points to a selectable object, the object is selected.

  All the selected regions are sorted in the first group. All individually selected words are sorted in the second group. That is, the selected regions are sorted first, and then the individually selected words. When copying the selected areas, the selections are sorted by row number (small to large). If multiple selections are in the same row, they are sorted from left to right. Each selection range is separated by a newline character.

  Example: Consider the following figure:

Figure 107 Copy and Paste Results



Feedback

Copy and Paste Results: On copying and pasting, the selected content is formatted as follows:

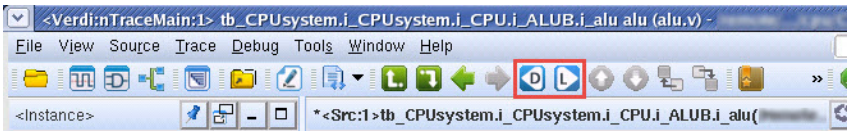
```
logic reset_cpu;

ThreeOnly

En_A

En_B

wire En_AB,En_AC,En_AD,En_BC,En_BD,En_CD;
```

- Multi-scope selection information is available to be saved and restored by using the **Save/Restore Session** feature. Choose **File > Save Session** or **Restore Session**.

Trace Drivers and Loads

The **OneTrace -> Driver** and **OneTrace -> Load** commands (or their equivalent toolbar icons) trace all of the drivers and loads, respectively, that are associated with a selected signal. The **OneTrace -> Connectivity** command (only found in the menus) traces drivers and loads simultaneously. These commands can also be accessed by right-clicking a signal in the *Source Code* frame.

Figure 108 Trace Driver/Load Icons in nTrace



Find String

1. In the *nTrace* main window, double-click *i_CPU* in the design browser frame to display the associated source code.
2. To find a certain string to trace, choose the **Source -> Find String** command (or bindkey “/”).

Feedback

A *Find String* form displays, similar to the following example:

Figure 109 Find String Form

3. Enter *data* in the **Pattern** box.
4. Click **Find All** and then click **Close**.

   The results are displayed in the **Search** tab of the message frame.
5. Double-click the result `cpu.v(53):inout[7:0]data;` to highlight the associated line in the source code.

   Note:
   The **Source -> Find Signal/Instance/Instport** command can also be used to locate a scope or a signal name anywhere in the hierarchy.

Trace Driver

As a result of Find String, you can view an 8-bit group of inout signals called data:

```
:
inout [7:0] data;
:
```

1. To begin the trace, double-click *data* or click the **Trace Driver** toolbar icon or either of the following:
   - From the main menu, choose the **OneTrace -> Driver** command.
   - Or right-click the signal, and choose the **Trace Driver** command from that menu.

     The source code displays immediately changes to the pram block and highlights the signal data in the driving statement, as follows:

     ```
     :
     assign data = R_W ? dataout :8'hz;
     :
     ```

Feedback

Drivers are indicated in the *Source Code* frame with left-handed semicircle next to the line number. The message frame also displays all of the drivers of the selected signal including any pass-throughs (the term *"pass-throughs"* refers to any intermediate nets on the path between the driver and the load as the path passes through different hierarchical levels in the design).

The **Show Previous in Hierarchy** toolbar icon is now enabled.

If other drivers exist in the same hierarchy, the **Show Next** or **Show Previous** toolbar icons may also become enabled.

Note:
The icons that are enabled are all dependent on the results of a trace.

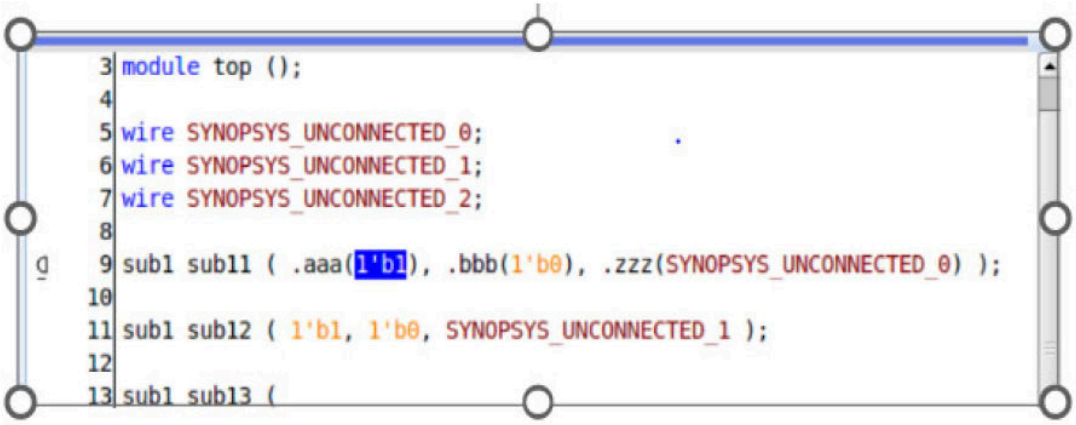
2. Click the **Show Previous in Hierarchy** icon to go to the *ExtData* driver in the *PCU* block.

   The **Show Previous in Hierarchy** toolbar icon is now disabled, and the **Show Next in Hierarchy** toolbar icon is enabled.

3. Click the **Show Next in Hierarchy** icon to return to the *data* driver in the *pram* block.

   When you execute the **Trace Driver** command on a signal and the signal connects to the instant port that has been connected to a constant `1` or `0`, Verdi highlights the signal for connecting the module instance ports by name. For example, in the following figure, the specific instant port with constant `1'b1` is highlighted after tracing.

Figure 110 Highlight 1/0 Clip port



Feedback

For more information on how to trace driver in nTrace on a Verdi platform, click the following video link:

This video explains how to trace driver in nTrace on a Verdi platform . A "trace driver" is a feature used in debugging and analyzing digital designs, and helps in pinpointing the exact locations in the code where a signal gets its values, which is crucial for debugging and verifying the correctness of the design. When you perform a trace driver operation, the tool highlights the driving statements in the source code and displays results in the message frame. This includes all possible drivers of the selected signal, even across different hierarchical levels of the design. The results can be navigated using various commands and toolbar icons, to move through the driver results within the current scope or across different scopes.

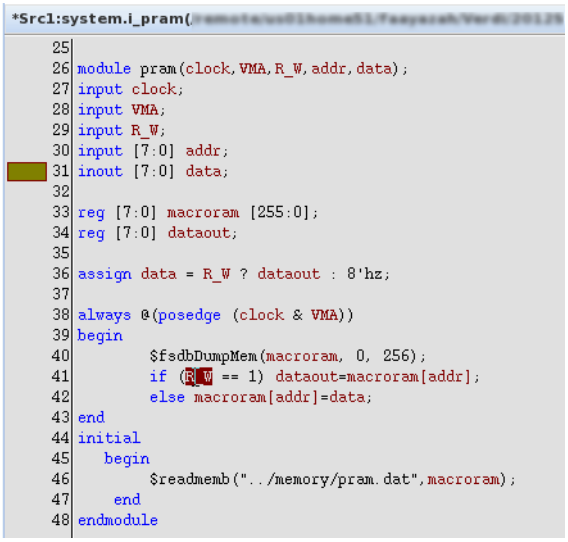
Add Bookmarks

You can add a bookmark to any line number to mark it for future reference. To add a bookmark from **Trace Driver** results:

1. Click line 31 in *i_pram* where *data* is assigned.
2. Click the **Set/Unset Bookmark** icon on the toolbar.

   The bookmark symbol appears, as shown in the following figure:

Figure 111 Example of a Bookmarked Source Code Line



Feedback

Trace Load

1. To locate the first load on this net, highlight *data*, and use the **Trace Load** toolbar icon .
   - You can also choose the **OneTrace -> Load** command from the main menu.
   - Or right-click the signal, and choose the **Trace Load** command from that menu.

Loads are indicated in the *Source Code* frame with right-handed semicircle next to the line number. The message frame also displays all of the loads of the selected signal including any pass-throughs (the term *"pass-throughs"* refers to any intermediate nets on the path between the driver and the load as the path passes through different hierarchical levels in the design).

The source code display changes to the pram block and highlights the signal data in the loading statement as follows:

```
: macroram[addr]=data; :
```

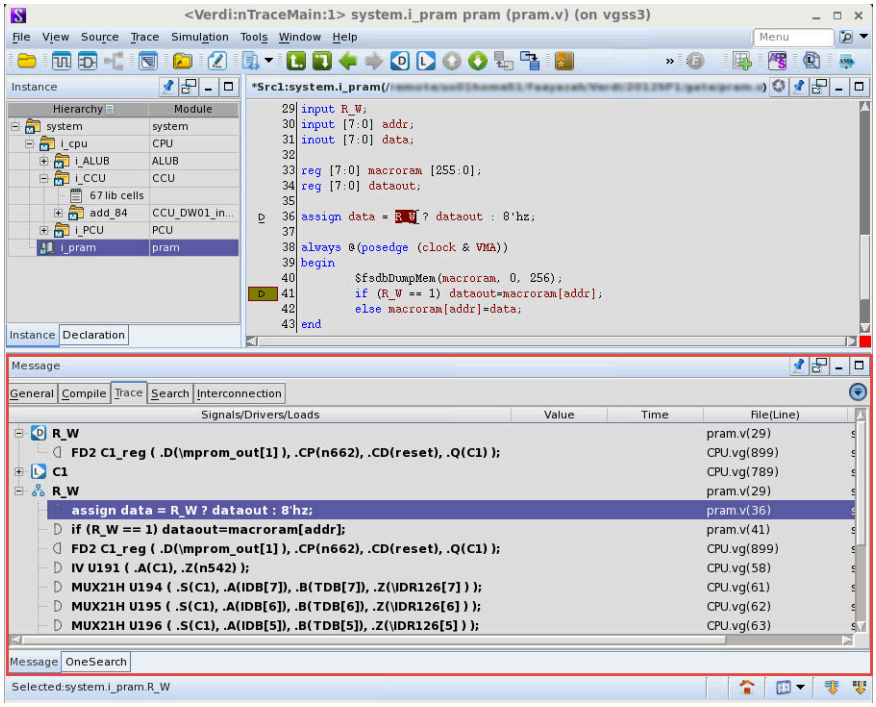
Trace Connectivity

1. To find a bookmarked line, choose the **Source -> Manage Bookmarks** command, then select the line from the list that displays (**1. pram.v(41)**).
2. To trace the connectivity of a signal, highlight a signal, and choose the ***One*Trace -> Connectivity** command from the main menu or the right mouse button menu.

Feedback

The results of the trace are displayed in **OneTrace** tab of the message frame in the *nTrace* main window, similar to the following example:

Figure 112 Example of nTrace Window With Trace Connectivity Results



3. The **Show Next** toolbar icon is now enabled. Click this icon to locate the next load (this is equivalent to using the **Trace -> Show Next** command).

4. Note that the **Show Previous** toolbar icon is now enabled. Click this icon to locate the previous load/driver (same as using the **Trace -> Show Previous** command).

Save Trace Result and Reset History

There are two methods for saving the trace results.

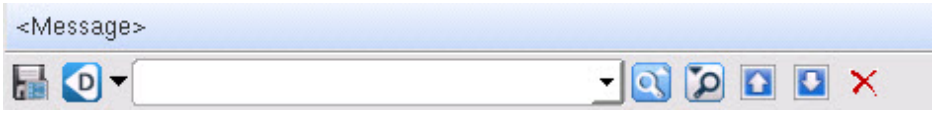
1. Use the **Save** command:

   a. On the **OneTrace** tab of the *Message* frame, click the right mouse button and choose the **Save** command.

   b. Enter the file name and click **OK**.

2. Use the **Save** icon:

   a. On the tab bar of the *Message* frame, click the **Show Toolbar** icon . A *Message* frame toolbar is displayed as shown in the following figure.



   b. Click the **Save** icon.

3. To reset the history of all signals, choose the **Trace -> Reset History** command from the main menu.

Edit Source Code

After you have located the source code which needs to be edited, you can modify the source code from the *nTrace* main window. After the code is modified you need to recompile and load the design and also regenerate the simulation results file.

1. In the *nTrace* main window, choose the **Tools -> Preferences** command to open the *Preferences* form.
2. Expand the **Editor** folder.
3. Select the page for the editor, for example, **nEditor**, and turn on the **Set as Default Editor** option.

   Note:
   If the editor you require is not listed, you can specify your own edit command on the **Other** page.

4. Click **OK**.
5. In the *nTrace* main window, double-click *i_ALUB* in the design browser frame to display the associated source code.
6. Click the **Edit Source File** icon to open a second source viewer. Alternatively, choose the **Source -> Edit Source File** command in the *nTrace* main window.

Use Active Annotation

Active Annotation™ allows you to view your verification results in the context of the source code. Active Annotation allows you to view - in one place - the value resulting from a logic expression coupled with the values of the arguments feeding that expression.

Feedback

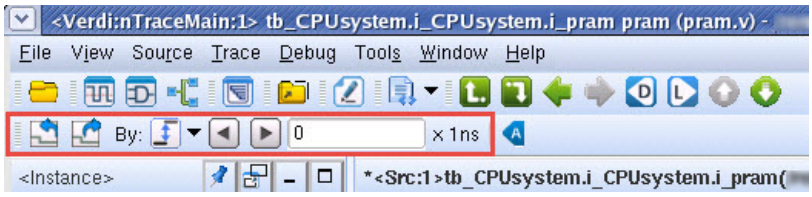
Note:
Active Annotation can also be used to display verification results in other views.

Before using Active Annotation, you must first load a set of simulation results in the form of a FSDB.

Note:
Other formats can be loaded and are automatically converted.

1. Load the simulation results using the **File -> Open Waveform File** command in the *nTrace* main window which opens the *Load Simulation Results* form.
2. In the *Load Simulation Results* form, move up one directory from the current directory.
3. Select *CPUsystem.fsdb*.
4. Click **OK** to load the file.
5. In the design browser frame, double-click *i_ALUB*.
6. Choose the **Source -> GoTo -> Line** command, and enter `85`.
7. Click **OK**.
8. Choose the **Source -> Active Annotation** command (or “x” key after putting the cursor in the *Source Code* frame) to activate Active Annotation. The values associated with each signal (as the time 0) are displayed under the signals and a new sub-toolbar appears, as shown in the following figure:

Figure 113 Sub-toolbar on nTrace Window Main Toolbar



Note:
The *XX* Active Annotation symbol shown under the signals reflect the un-initialized condition of these signals at time 0.

9. Select the *RESET* signal, and click the **Search Forward** and **Search Backward** icons on the new Active Annotation sub-toolbar. Note that the display updates to reflect the transitions from value to value at the time in which it occurs.
10. Search for rising edge changes on the *RESET* signal by changing the search **By** selection to **Rising Edge**

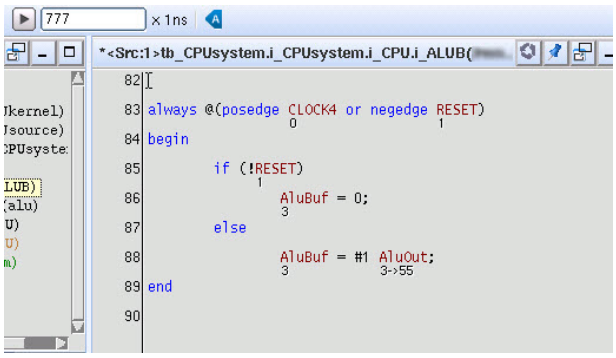
and continue to click the **Search Forward** and **Search Backward** icons.

Feedback

11. On the toolbar, choose the **Source -> Go To -> Line** command.
12. In the *Go To Line* form, enter *82* and click **OK**.
13. On the toolbar, enter 777 in the **Cursor Time** box.
14. Press the **<Enter>** key on the keyboard.

    The *nTrace* main window updates the display similar to the following figure:

Figure 114 Active Annotation



All the signals in the design are assigned to non-*X* values.

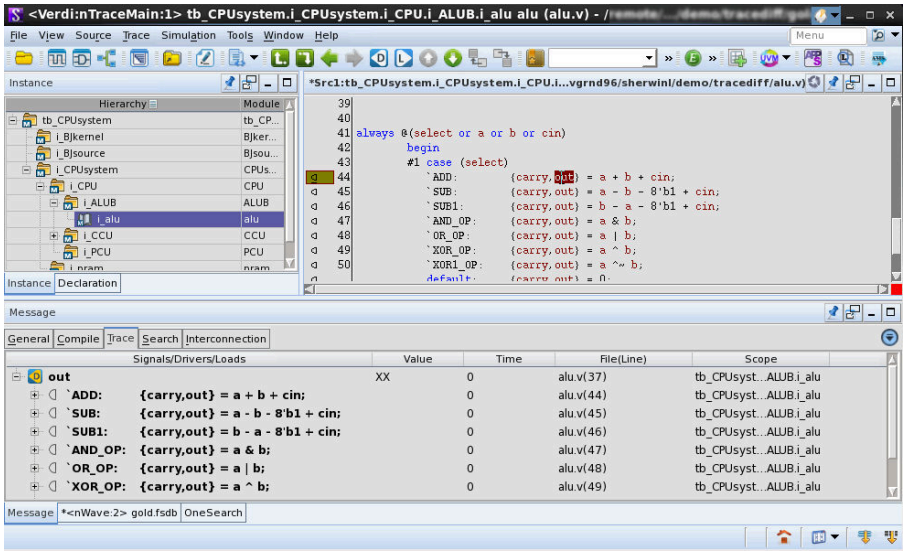
Tracing the Active Driver

1. Using the results from the previous section, go to line *44*, and select *AluOut*.
2. Invoke the **Active Trace** command by one of the following ways:
   - Right-click and select **Active Annotation** to enable active annotation. Click the **Driver** icon in the toolbar.
   - Use the bindkey, Ctrl+T.

Feedback

The source code displays changes to the active driving unit, and the signal in the driving statement is highlighted, as shown in the following figure:

Figure 115 Active Trace



`AluOut` changed names to *out* as it crossed hierarchy boundary. See the message frame in the figure above for example.

The time field in the toolbar may also change to reflect when this assignment to the signal is made. This information is also presented in an *Information* dialog window.

3. Check the equation, and note that *55* appears from signal *a*.
4. Select *a*, and choose **Active Trace** again.

   You can continue to Active Trace until you locate the source of a value.

   For more information on how to Trace Active in nTrace on a Verdi platform, click the following video link

This video explains how to trace Active in nTrace on a Verdi platform. Active Trace is a feature in Verdi that allows you to trace the actual driver of a signal at a specific time. It helps in identifying the source of a value change in a signal by highlighting the driving statement in the source code. This feature is particularly useful for debugging and understanding the behavior of signals in a design. When you perform an active trace, it locates the active driver in the source code frame for the selected signal and displays the results in the Trace tab of the Message pane.

Feedback

Fault Source Code

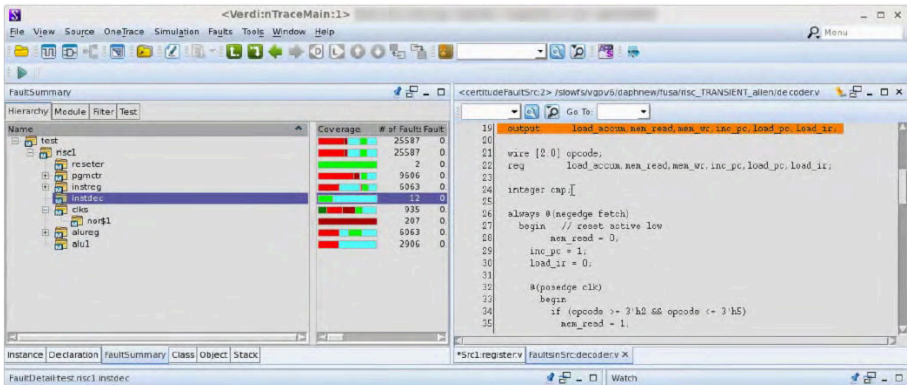
Verdi supports annotating fault information in the source code. The Fault data is loaded from the Fault server through the Fault API.

Use Model

A new top-level pull-down menu is added and all fault related sub-menus are under the main menu.

When you double-click an instance/module, the Source Code view displays the selected instance/module, and the fault information in the Source Code view is color-annotated as shown in the following figure. If there are more than one faults in the same location, the color for mixed fault status is a special color (default to orange).

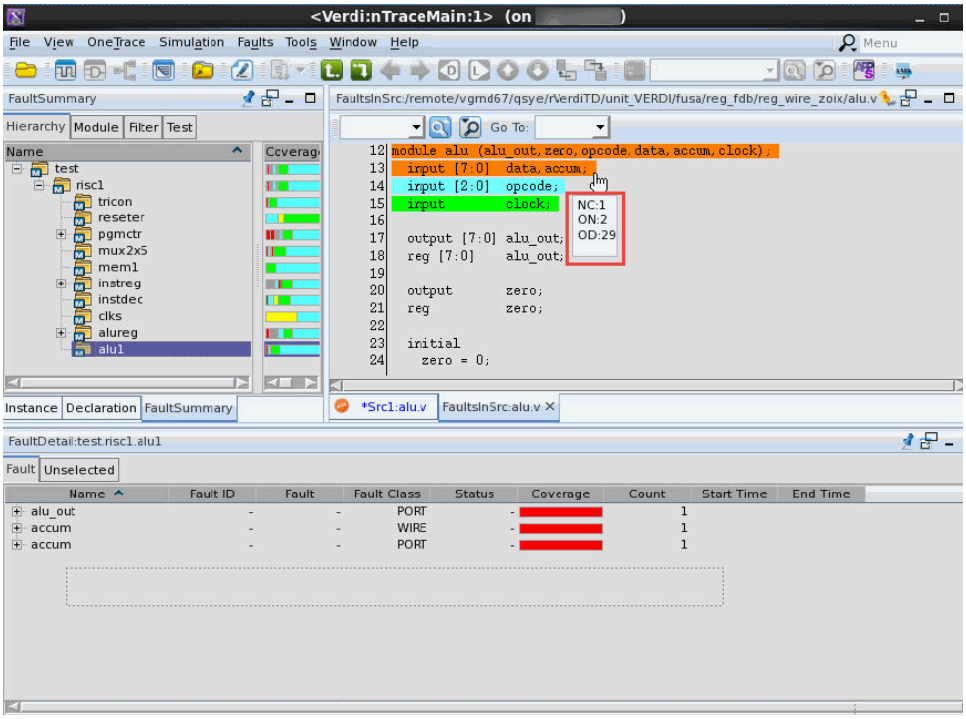
Figure 116 Color-Annotated Source Code



The following figure shows color-annotated Source Code with tooltip enabled.

Feedback

Figure 117 Color-Annotated Source Code With Tooltip Enabled



Limitations

You can load FDB without loading the KDB, but due to missing complete source code information from KDB, the fault annotation in the Source Code pane has the following limitations:

- Unable to open source for instances without faults.
- Dragging and Dropping is not supported.

For a full functioning Source Code pane, you should load the KDB with Fault database.

Support for Specify Array Index

The feature, **specify array index** was supported for loops or generate-for. Now the feature **specify array index** is also supported for the keyword *foreach*. This feature behaves like specify array index for loops or generate-for.

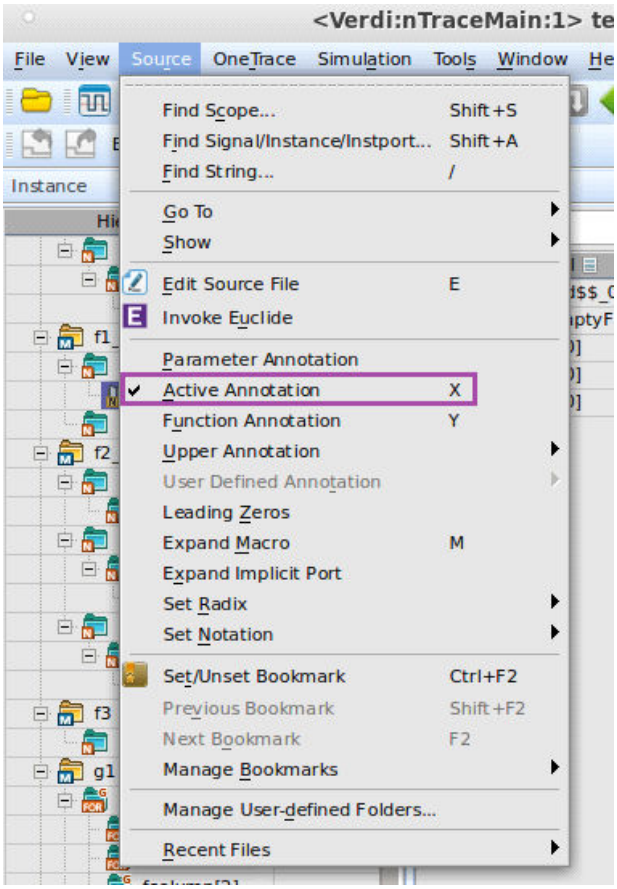
Feedback

Basic Operation Flow

Load FSDB and show Active Annotation

Once the FSDB file has been loaded to Verdi; enable the **Source -> Active Annotation** option in the *nTrace* menu bar to turn on the Active Annotation as shown in the following figure.

Figure 118 Enabling Active Annotation

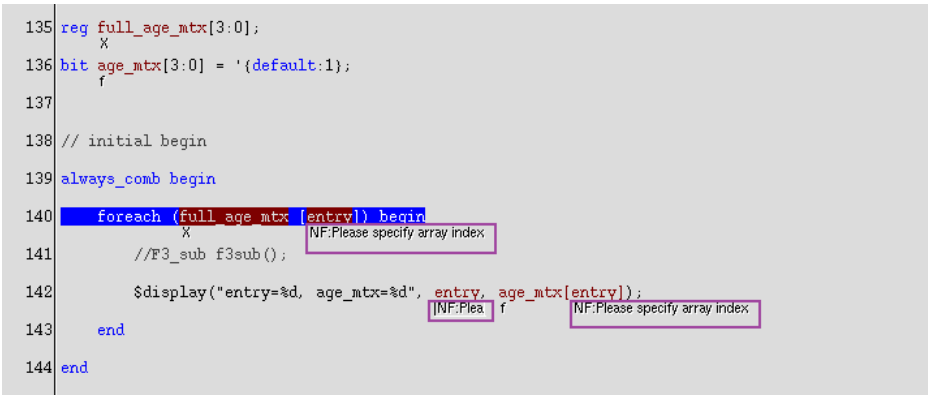


Operate on Source Panel

If the array index could be specified, show hint below the index as shown in the following figure.

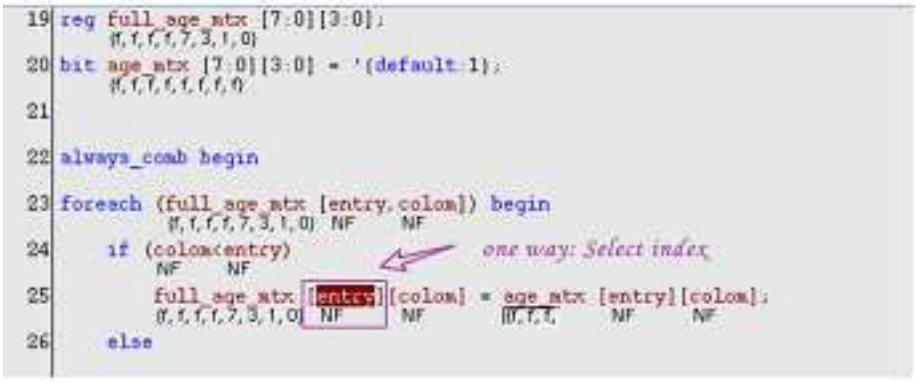
Feedback

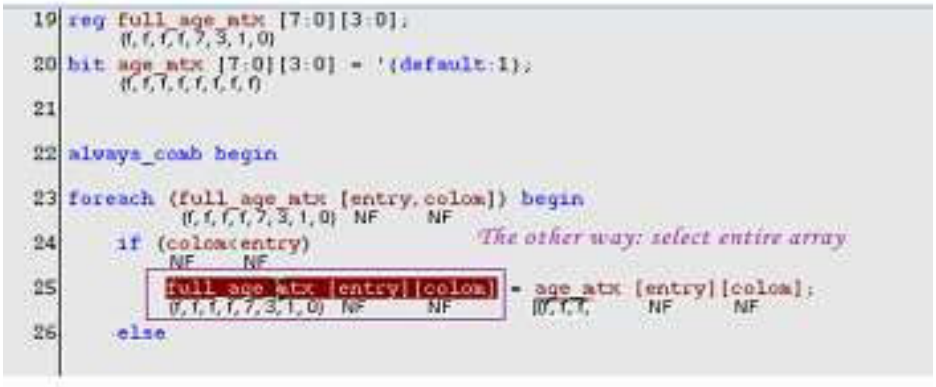
Figure 119 Showing Hint Below the Index



There are two ways to select. One is to click the index, and the other is to click the entire array as shown in the following figure.

Figure 120 Two Ways to Select Index

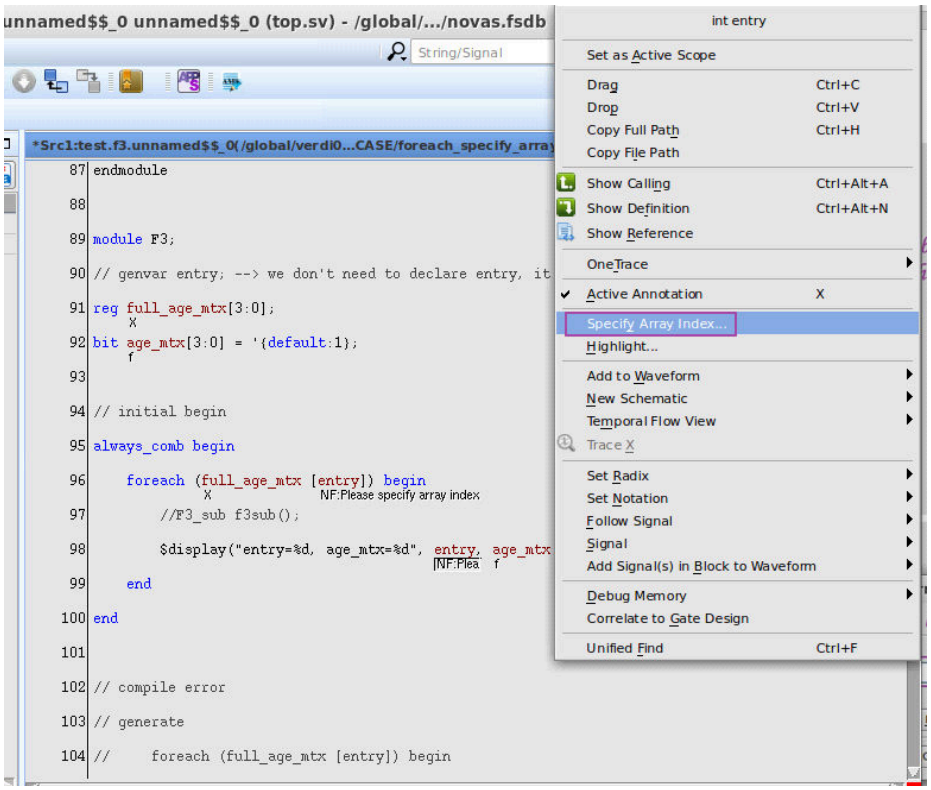




Feedback

- **Go to RMB -> Specify Array Index...** as shown in the following figure:

Figure 121 Specific Array Index



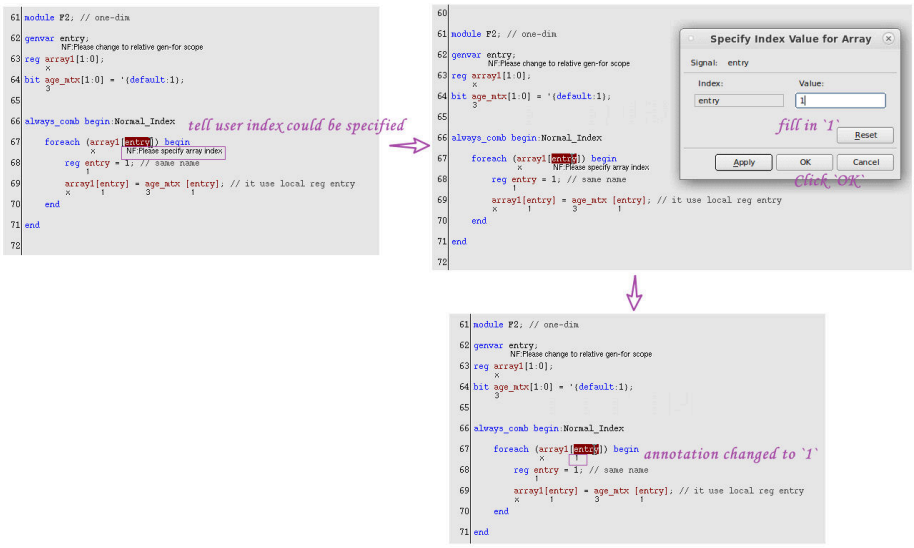
- After selecting **Specify Array Index...**, a select dialog pops up on the front. The dialog shows signal name, legal value range and so on. As the following figure shows, the index entry could be specified to some value within legal range *(0...3)*, You need to fill in legal value on the blank and then click **Apply** or **OK** to submit. Otherwise, leave it unchanged by clicking **Cancel**.

Feedback

Figure 122 Specify Index Value for Array

The complete flow is shown in the following figure:

Figure 123 Complete Flow

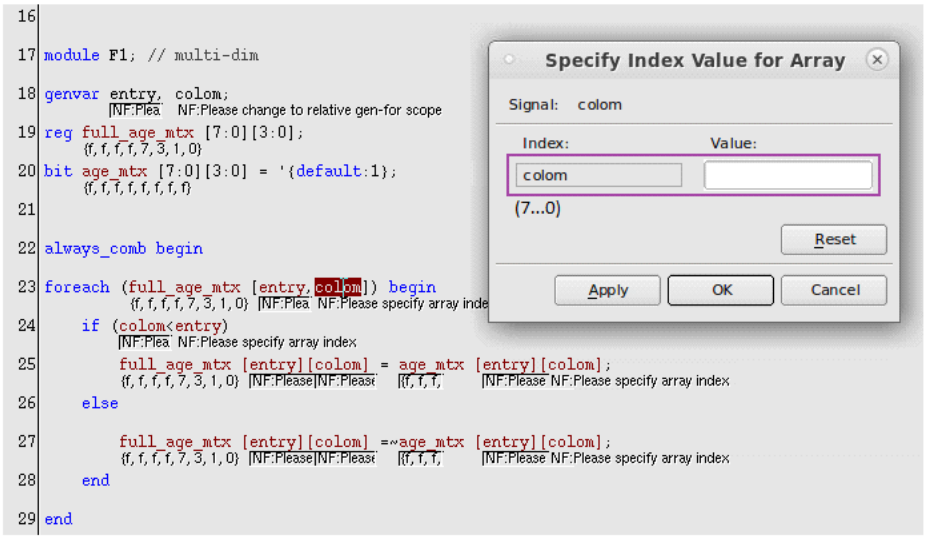


Feedback

Miscellaneous Usage

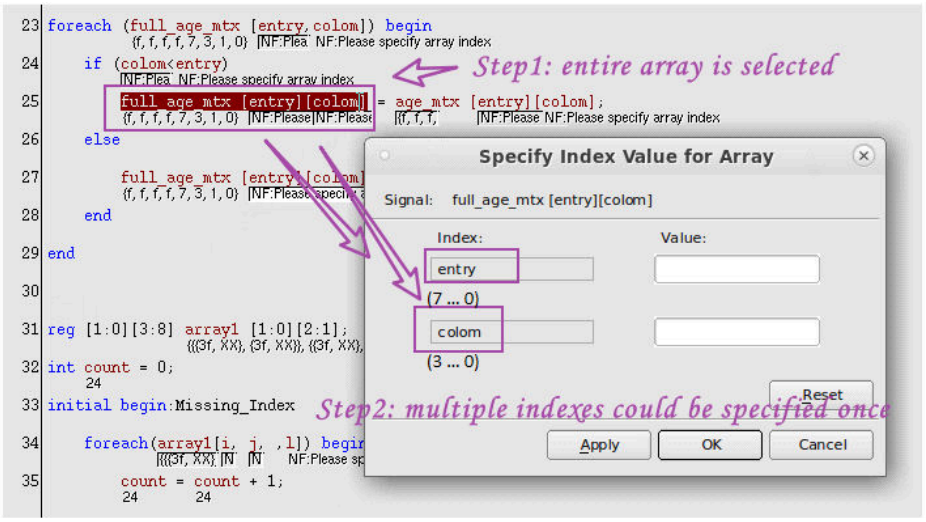
Example 1

- Multiple indexes can be specified separately as shown in the following figure:



- You can specify multiple indexes on one combined window by selecting the entire array on line *25* inside the *foreach* loop.

  Note:
  Selecting array on line *23* does not work.



Feedback

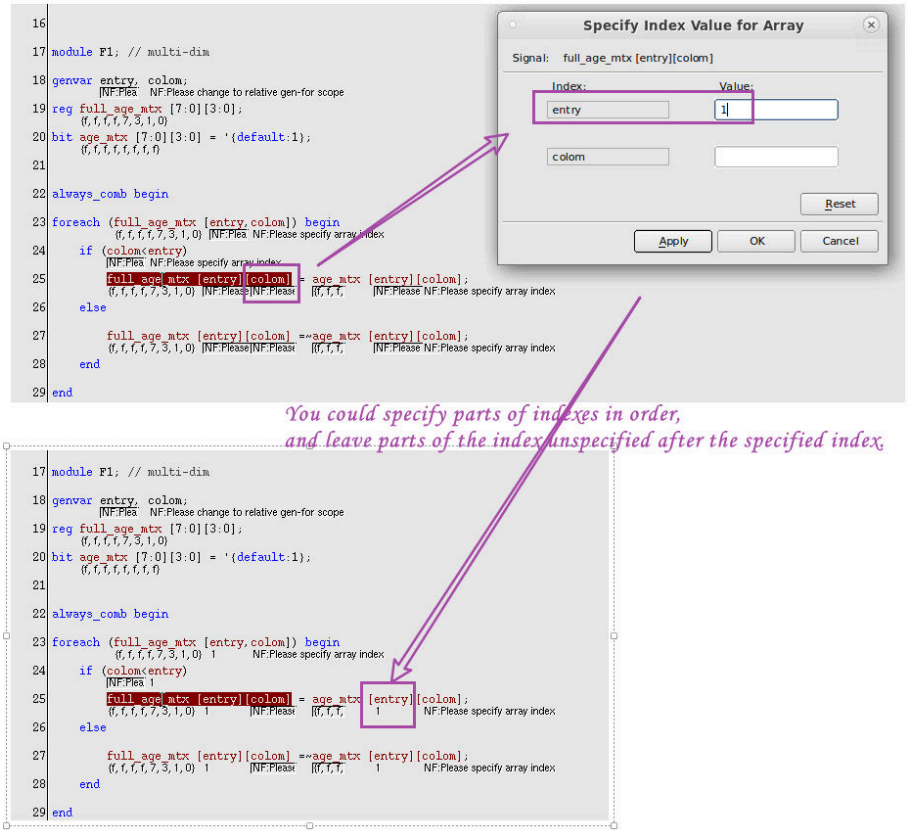
The following are the rules about selecting part of the indexes:

- Rule 1: You should specify parts of indexes in order, that is, leave the unspecified indexes unspecified after the specified index.
- An exception when you put cursor onto array inside for each brace is not restricted by Rule 1

Feedback

- The following figure shows the scenario when you obey Rule 1:

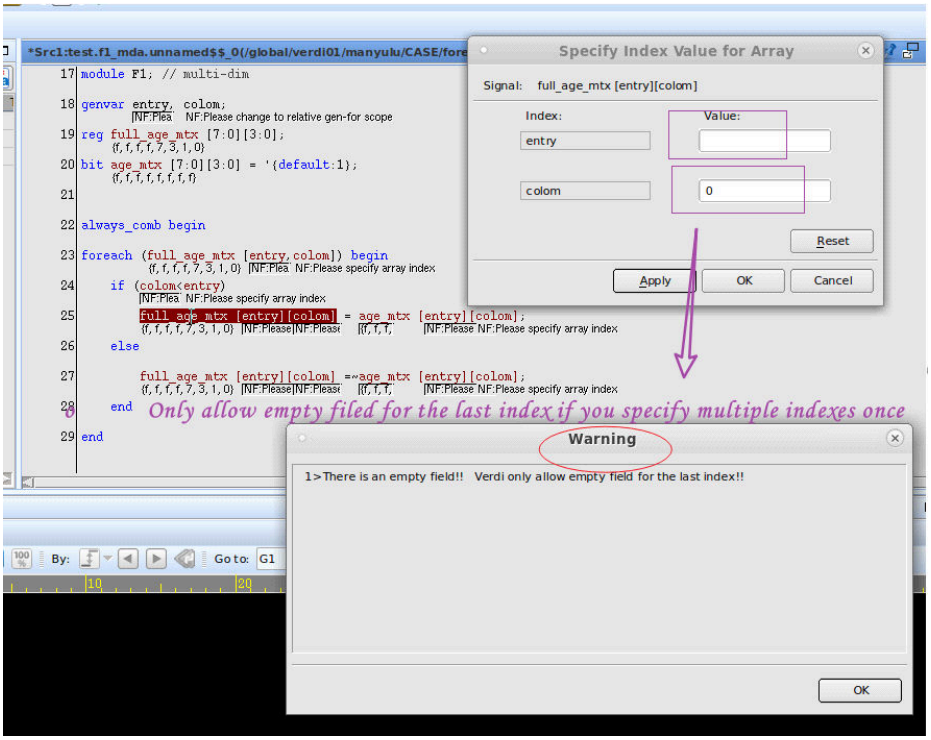
Figure 124 Obey Rule1



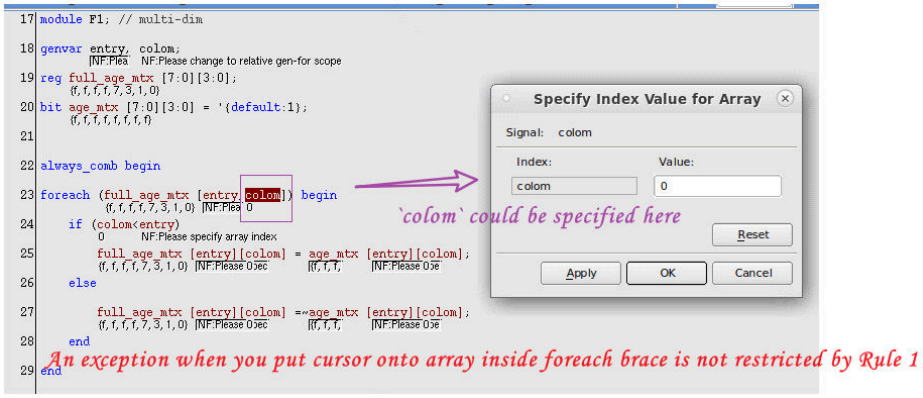
Feedback

- The following figure shows the scenario when you disobey Rule 1:

Figure 125 Disobey Rule1



- An exception to Rule1 is shown in the following figure:

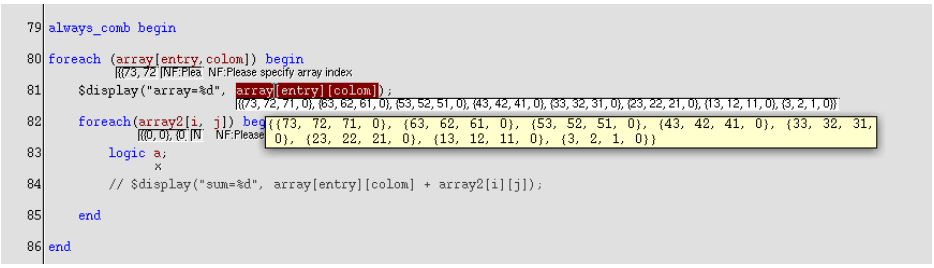


Feedback

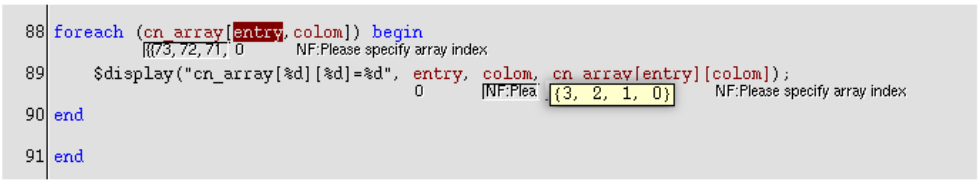
The annotation is dependent on which index you specify. For example,

In a given 2-D Array, presume that you have four possible usages. The usages are as follows:

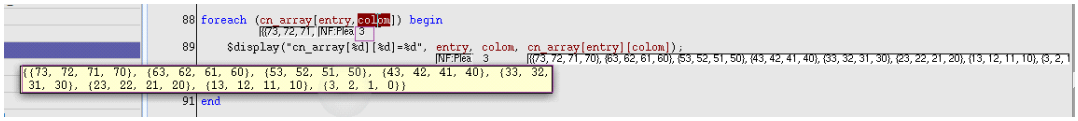
1. User does not specify any index; annotation should show the complete array.



2. User specifies the part of indexes, with the specified index before the unspecified index. This example shows 1-D array named `cn_array[0]`.



3. User specifies part of indexes with the specified index after the unspecified index, as it is illegal operation, so the annotation would be the same as before.

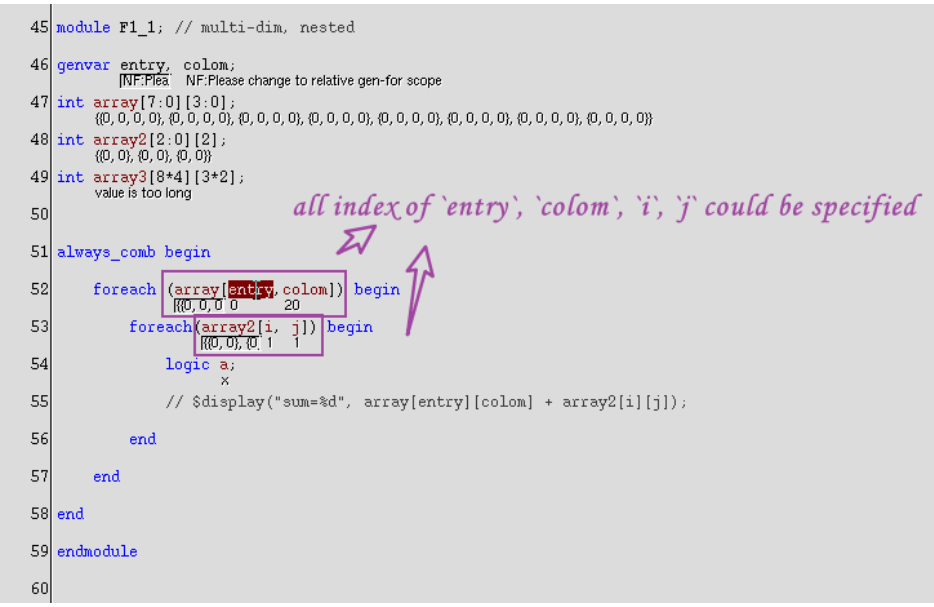


4. User specifies all indexes. This example shows the variable named `cn_array[3][1]`.

Feedback

Example 2

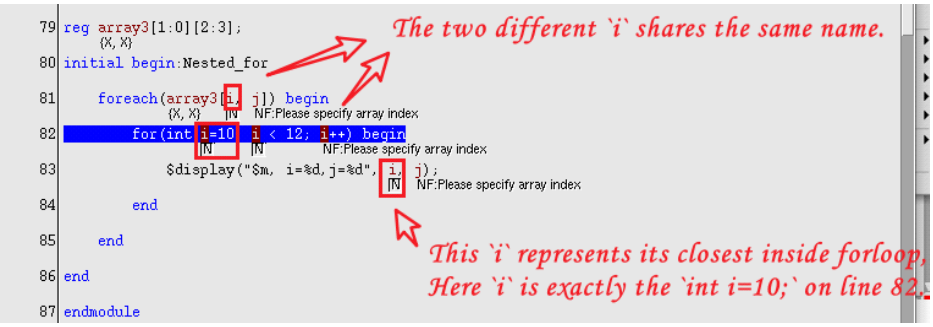
It can be nested and used. For *foreach* string, multilevel works. You can specify all indexes as shown in the following figure:



Example 3

The string *foreach* is compatible with the keyword *for*.

The variable *i* in line *81* is used with *foreach*, which is specified. Meanwhile, the var *i* in line *82* is specified too. They are not in conflict as shown in the following figure:



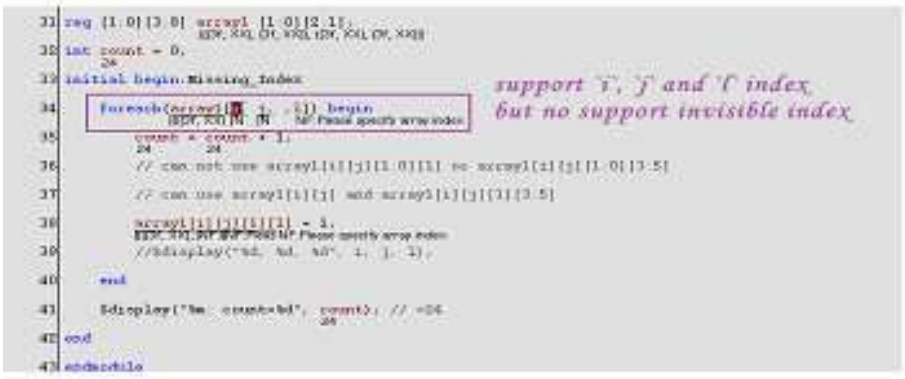
Feedback

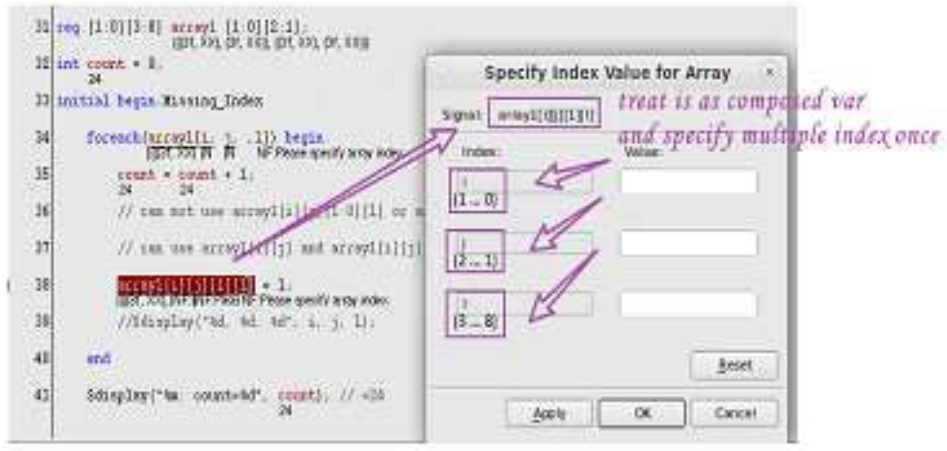
The string *foreach* cannot be used for undefined index. As there are no defined indexes, no indexes appear here as shown in the following figure:

```
73 initial begin:Empty_Index                    Not Support
74     foreach (array1[,]) begin // foreach(array1[]) compile error
                 x
75         $display("%m Empty_Index"); // never execute this line
76     end
77 end
```

Example 4

In the case of missing index, specifying on visible indexes work as usual and specifying multiple indexes also works as shown in the following figure:

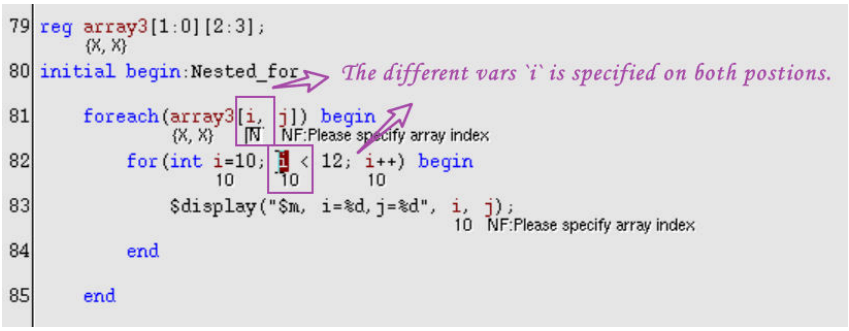




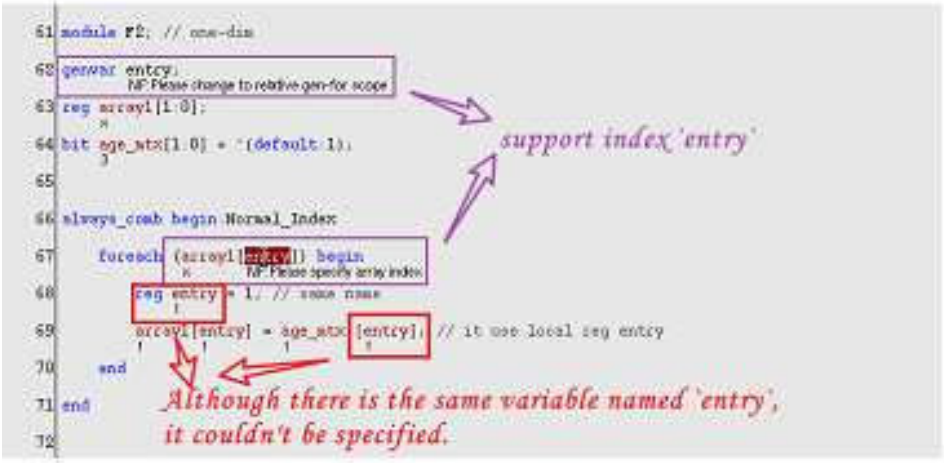
Feedback

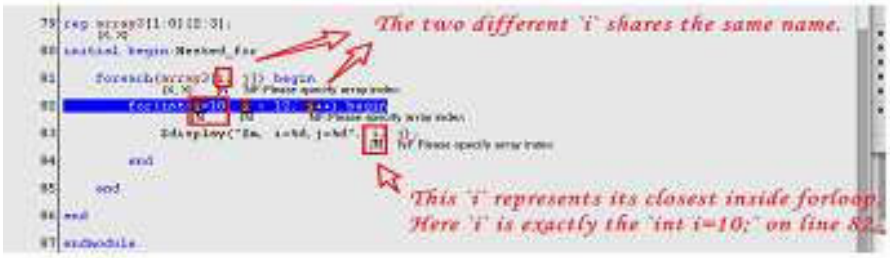
Example 5

In this example, the variable *i*, is specified at two positions as shown in the following figure:



The variable *entry* is used across many positions but they represent different variables. The representation depends on its closest definition as shown in the following figure:





7

nSchema Tutorial

This chapter describes the following topics:

- Overview
- Start nSchema
- Manipulate the Schematic View
- Trace Signals
- Show RTL Block Diagram in a More Meaningful Way
- Generate Partial Schematics
- Use Active Annotation to Show Signal Values
- Support for Annotation and Fault Coverage on nSchema Window

Overview

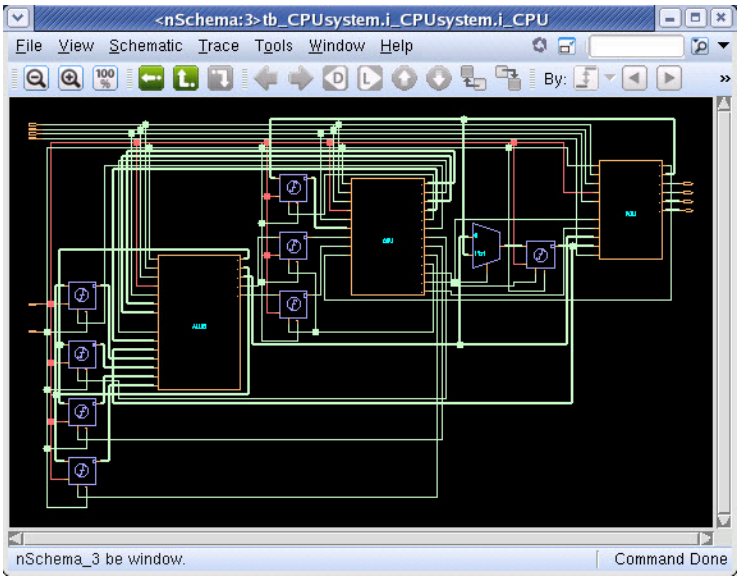
nSchema is a schematic viewer and analyzer that generates interactive debug-specific logic diagrams displaying the structure of the selected portions of a design. RTL diagrams show the interconnection of finite state machines, storage elements, and multiplexers. Gate-level diagrams show the interconnection of semiconductor vendor cells and special flattened diagrams cut through the design hierarchy to isolate connected design elements. *nSchema* dynamically generate partial schematics to focus on the circuits of interest in a large design.

Before you begin this tutorial, follow the instructions in the Before You Begin chapter. Refer to the User Interface chapter for general information on the *nSchema* window.

Feedback

The *nSchema* window is used to display auto-generated schematics and logical diagrams, the frame can be undocked to be a standalone *nSchema* window, as shown in the following example:

Figure 126 Example of nSchema Window



nSchema generates the schematic for both RTL and gate-level designs. For RTL designs, the Verdi platform extracts certain types of synthesizeable function blocks from the HDL code, such as registers, latches, multiplexers, pure combinatorial or sequential circuits, and so on. With this capability, you can get a clear picture of the design intent, especially for a design with which you are unfamiliar. For gate-level designs, the Verdi platform uses standard symbols, such as nand, nor, inverter, and so on, to make the schematic more readable and understandable. To perform certain functions, such as signal tracing or intuitive searching, you can drag-and-drop items between windows to cross-link the tools.

Start nSchema

1. Change the directory to <*working_dir*>*/demo/verilog/cpu/src*, and execute the following command to import the sample CPU design:

   ```
   % verdi -f run.f -workMode hardwareDebug &
   ```

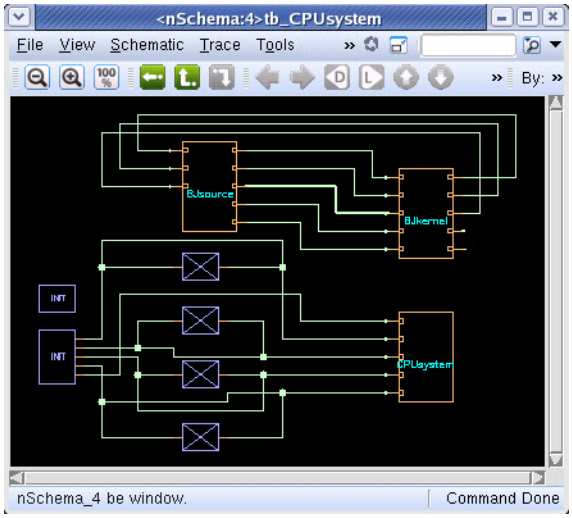
   You can continue from the previous Verdi session if the window is still open.

2. In the *nTrace* main window, highlight the folder *tb_CPUsystem* in the **Instance** tab of the design browser frame.

Feedback

3. Click the **Hierarchy Flattened Schematic** icon on the toolbar in the *nTrace* main window (or choose the **Tools -> New Schematic from Source -> Hierarchical Flattened,Modular or the Blank Schematic** command), the *nSchema* window is displayed. Click the **Undock** icon in the upper right of the *nSchema* window, a separate window showing the schematic of the current module (*tb_CPUsystem)* displays, as follows:

Figure 127 nSchema Window Displaying tb_CPUsystem Schematic



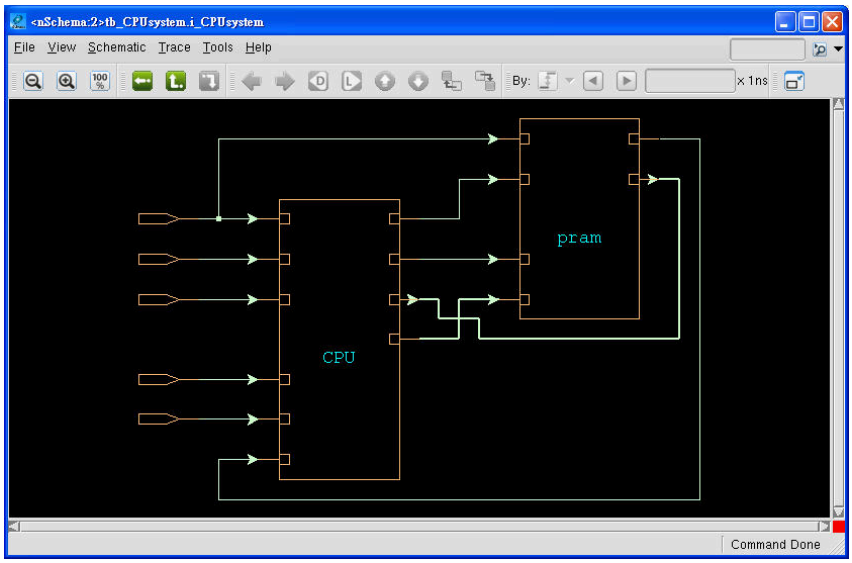
Click the **Modular** icon to create a new schematic frame in full hierarchical view. Each new schematic frame initially shows the schematic view of the HDL source module currently displayed in the source code frame.

Feedback

4. Drag-and-drop the instance, *i_CPUsystem,* in the design browser frame to the separate schematic window to display the instance's schematic.

   The results are displayed similar to the schematics displayed in the following figure:

Figure 128 Displaying CPU Block as Schematic



For more information on how to create hierarchical schematic view in Verdi, click the following video link:

This video explains how to create hierarchical schematic view in Verdi. The nSchema window is displayed when the **Tools > New Schematic from Source > Current Scope** command is invoked from the nTrace menu bar or the New Schematic icon. The Full Hierarchical view displays the complete design in a specific scope. The design can be traversed hierarchically by pushing the view into a lower level for the hierarchical instance, or popping the view up to an upper level. This video takes you through the process of creating hierarchical schematic view. You need to select the scope in the design browser frame and click **Tools > New Schematic from Source > Current Scope** command in the nTrace window to open the Full Hierarchy window.

Manipulate the Schematic View

Double-click the *i_CPUsystem, i_CPU* and *i_CCU* module names in the design browser frame to access the source code of the module *CCU*.

Feedback

You can change the view of the schematic using the following zoom commands:

- **Zoom In** - View more details of the schematic by moving the view 50% from the center point in both the horizontal and vertical directions. Invoke this command in one of following three ways:
  - Toolbar icon
  - Bind key "Z"
  - Menu **View -> Zoom -> Zoom In** command
- **Zoom Out** - View more contents of the schematic by expanding the view 2X from the center point, both horizontally and vertically. Invoke this command in one of the following three ways:
  - Toolbar icon
  - Bind key "z"
  - Menu **View -> Zoom -> Zoom Out** command
- **Zoom All** - View the entire contents of the schematic. Invoke this command in one of the following three ways:
  - Toolbar icon
  - Bind key "f"
  - Menu **View -> Zoom -> Zoom All** command
- **Zoom Area** - View more details in a specific area of the schematic by dragging-left to form a rectangle over the area.

  Note:
  You can change if the right mouse button or the left mouse button performs the zoom on the **General** page under the **General** folder of the *Preferences* form (invoked with the **Tools -> Preferences** command). This example considers that the left mouse button is set to zoom.

You can move the viewing area of the schematic in different directions:

- **Scrolling** - Click or drag the scroll bar of the schematic window horizontally or vertically.
- **Panning** - Move the viewing area up, down, left, or right using the arrow keys on your keyboard or the menu commands: **View -> Pan -> Pan Up**, **View -> Pan -> Pan Down**, **View -> Pan -> Pan Left**, and **View -> Pan -> Pan Right**.

  In addition, you can use the **View -> Last View** command or the bindkey "l".

Feedback

- In the *nSchema* window, choose the **Tools -> New Schematic -> Current Scope** command from the main menu to create a new *nSchema* window with the same schematic view.
- To close schematic windows that are no longer required use the window manager's control button (the ‘X’ icon in the upper right) or the menu **File -> Close Window** command.

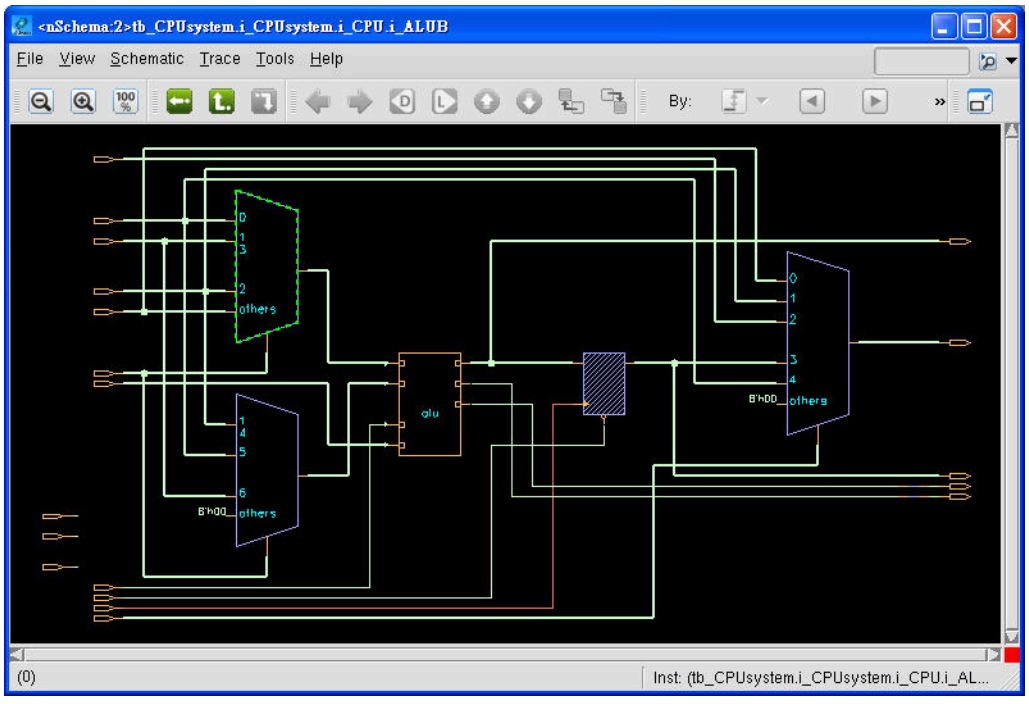
For more information on drag and drop objects among panes, click the following video link:

This video focuses on the drag-and-drop functionality in nSchema, a powerful schematic viewer in Verdi.

Change the Schematic View Among Instances

1. Using the middle mouse button, drag the instance *i_ALUB* from the design browser frame and drop it into the *nSchema* window to show the corresponding schematic.

Figure 129 Displaying i_ALUB Schematic



2. Move your cursor over various symbols or nets in the schematic view, and notice the name is identified in *nSchema*’s lower bar. The name information is shown in the lower

Feedback

bar of the main window if the *nSchema* windows is not undocked as a standalone window.

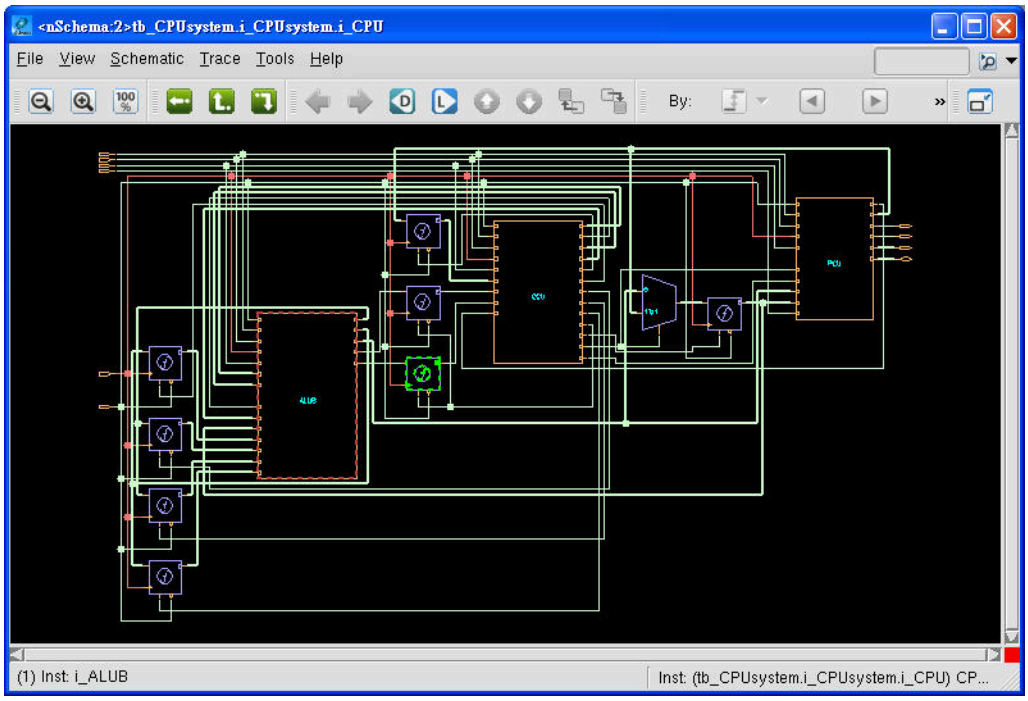
For more information on how to dock and undock panes to customize the layout of your workspace, click the following video link:

This video takes you through the capability of docking and undocking panes. You can undock a pane by dragging the pane banner (press and hold the middle mouse button) and dropping it outside the main window. It works as a standalone window. When you click the **Dock** icon on the standalone window, the window is docked as a pane in the main window.

3. Right-click the schematic window, select **Pop View Up** in the menu and the schematic view changes from the child module *ALUB* to its parent module *CPU* with *ALUB* block highlighted as the selected object.
4. Choose the **View -> Push View In** command or its corresponding toolbar icon to update the schematic view to the selected module *ALUB*.
5. Choose the **View -> Pop View Up** command or its corresponding toolbar icon to change the schematic view to module *CPU*.

Feedback

Figure 130 Displaying CPU Module With ALUB Block Highlighted



6. Choose the **View -> Last View** command or the bindkey **L** or its corresponding toolbar icon to roll back the schematic view to module *ALUB*.

Enable Viewing Objects

You can enable or disable viewing for different objects (for example, nets, instances, ports, and so on) in the schematic window.

1. In *nSchema*, choose the **Tools -> Preferences** command to open the *Preferences* form.
2. Select the **Color/Font** page under the **Schematics** folder.
   - Change the **Type** field from the default **Background** selection to **Selected Set**.
   - Change the **Color** to red and the **Line Style** to dashed.

   Notice the changes that affect the open schematic.
3. Select the **View** page under the **Display Options** folder and turn on the **Local Net Name**, **Instance Name**, and **IO Port Name** options.

Feedback

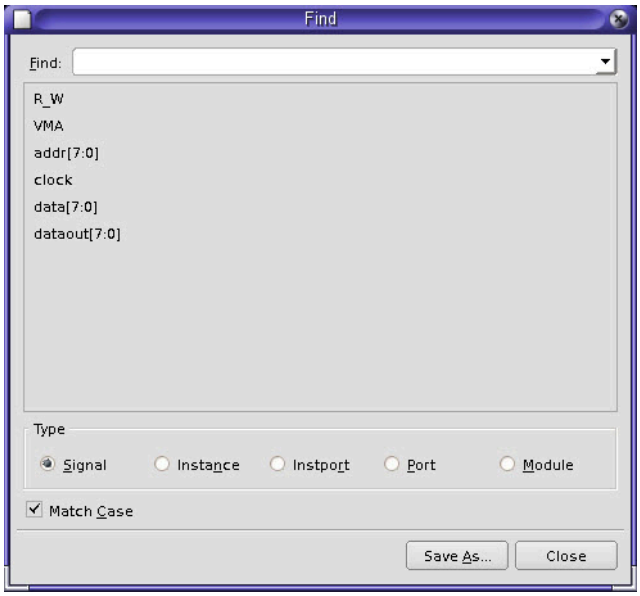
4. Click **OK** to close this form and apply the changes.
5. In the *nSchema* window, open the **View** menu and select the **Net Name**, **Instance Name**, and **IO Port Name** commands individually. The net, instance and port names are removed from this schematic.

The options on the *Preferences* form affects all schematics - a global setting. The **View** menu only affects the current schematic - a local setting.

Find an Instance or a Signal in a Schematic

1. Display the *i_ALUB* module in the *nSchema* window and then drag-left around the multiplexor in the upper left corner to zoom in.
2. Choose the **Schematic -> Find in Current Scope** command (or “a” bindkey) to display the *Find* form as shown in the following figure:

Figure 131 Find Form



Select the **Instance** option or the **Signal** option to list all the instances or signals under the current module in the alphabetical order.

Note:
If the **Schematic -> Auto Fit Found Object(s)** toggle command is enabled or the **Auto Fit Selection** option is enabled on the **Schematics -> Select** page of the *Preferences* form (invoked with the **Tools -> Preferences** command) and a target instance/signal is selected, its corresponding object

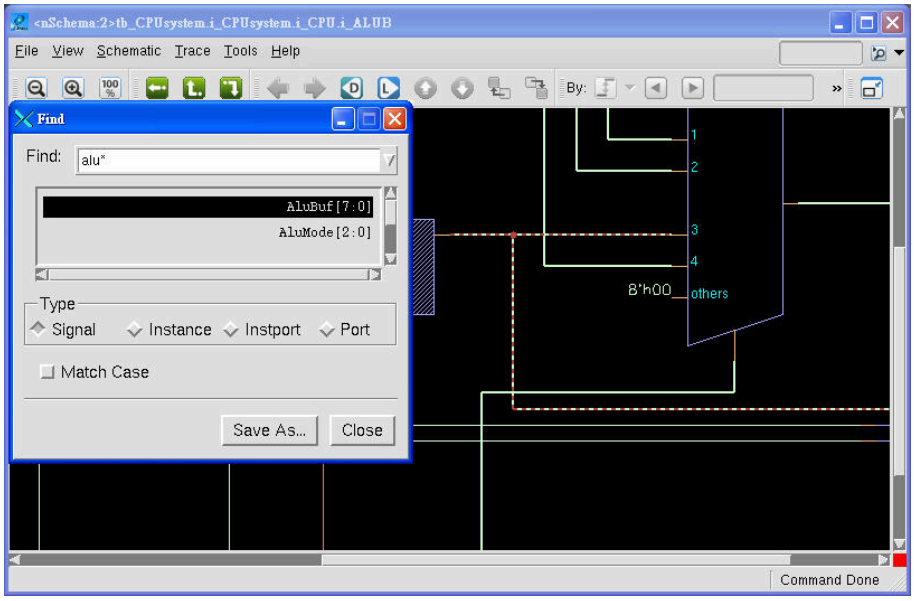
Feedback

is immediately selected in the schematic window and the schematic view scales properly to make the target object viewable.

3. Select the **Signal** option, and uncheck the **Match Case** option.
4. In the **Find** text field, enter *alu** and press **<Enter>**.
5. Select *AluBuf[7:0].*

   The signal is highlighted in the schematic view, as shown in the following figure:

Figure 132 Find Form With Highlighted Signal



6. Click **Close** to close the form.
7. In *nSchema*, double-click the multiplex or instance attached to the *AluBuf* signal. A *View Source Code* window opens displaying the associated source code lines.
8. With the same multiplex or instance selected, use the middle mouse button to drag the instance to the *Source Code* frame to display the source code in context.
9. In the **Instance** tab of the design browser frame, double-click *i_CCU* to locate the source code.
10. Locate the signal declaration for *IXR_load* in the *Source Code* frame.
11. From the *Source Code* frame, drag-middle and drop the signal in any open *nSchema* window.

    The schematic is updated and the signal is highlighted.

Feedback

Change the Color of the Selected Signal

1. To identify certain signals in a schematic clearly, choose the **Schematic -> Change Color** command.

   A *Change Selection Color* form displays the selected signal name and its color, as shown in the following figure:

Figure 133 Change Selection Color Form

2. Click a color in the color map to change the *FirstDataInRdy* signal's color instantly. Click **Default** if you want to reset a signal color.
3. Click **Close** to close the form.
4. To return everything to the default colors, in the *nSchema* window, choose the **Schematic -> All Objects to Default Color** command.
5. Close all open schematic windows.

Trace Signals

The following sections are based on the schematic of *i_CPUsystem*. Before you begin, double-click *i_CPUsystem* in the design browser frame, and click the **New Schematic** icon to display the schematic. Then click the **Undock** icon in upper right to open a standalone *nSchema* window.

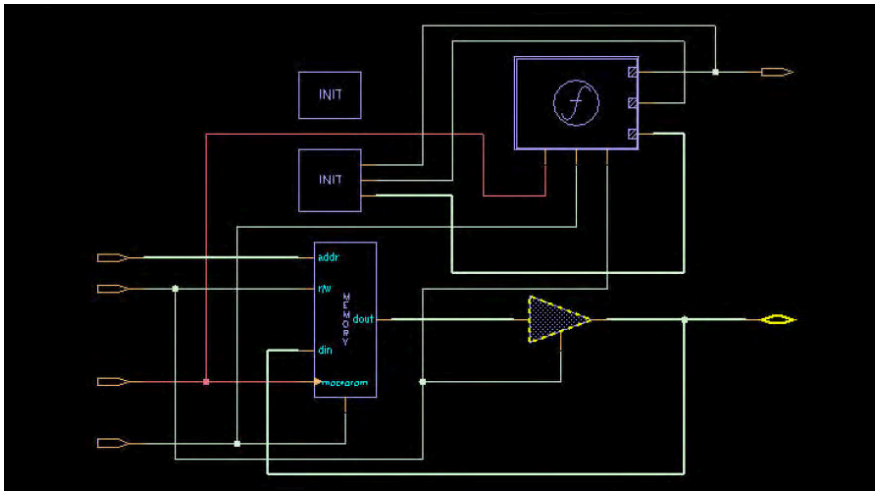
Find the Drivers of a Signal

The **Trace Driver** function shows all drivers of a selected signal on the schematic.

Feedback

1. For example, choose the **Schematic -> Find in Current Scope** command to locate and select *data[7:0]* in the schematic window.
2. Choose the **Trace -> Driver** command (or click the icon) on the selected bus, *data[7:0]*. The result is shown in the following figure:

Figure 134 Example Results of Trace Driver on data[7:0]



The figure above shows a driver found in module *pram*. Since no other drivers exist in that module, the **Show Previous** and **Show Next** icons are disabled. You can access the **Show Next in Hierarchy** icon because drivers exist in other modules.

3. To show the schematic of *PCU* and the traced drivers in that module, click **Show Next in Hierarchy**. The **Show Previous in Hierarchy** becomes enabled.
4. To return to the schematic view of module *pram*, click **Show Previous in Hierarchy**.

Find the Load of a Signal

The **Trace Load** function shows all loads of a selected signal or the schematic.

1. Use the pop view up icon to go back to the *CPUsystem* schematic.
2. Highlight *data[7:0]*.
3. Choose the **Trace -> Load** command or the toolbar icon .

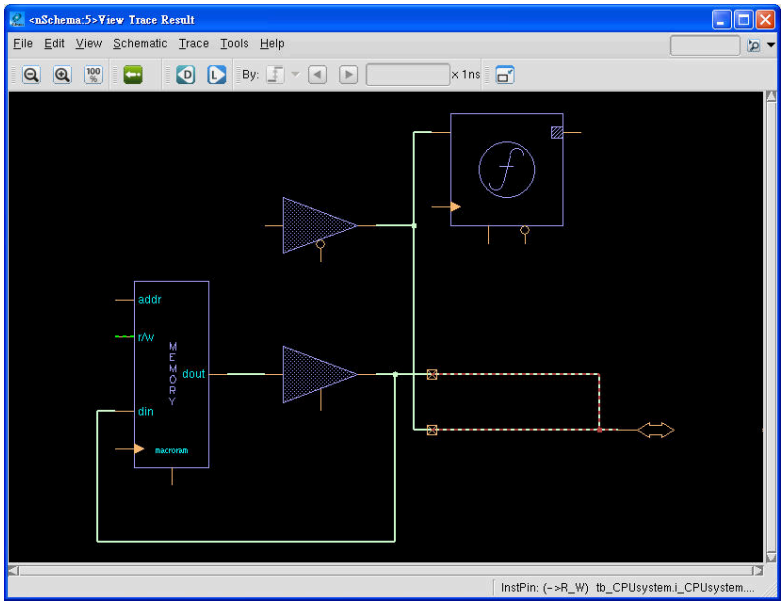
Feedback

Find the Connectivity of a Signal and Generate a New Schematic From Trace Results

To narrow the debugging scope, you can generate a partial schematic containing only the trace results.

1. Use the pop view up icon to go back to the *i_CPUsystem* schematic.
2. In the *i_CPUsystem* schematic, choose the **Trace -> Connectivity** command on the selected bus, *data[7:0]*. The schematic updates with the highlights of the trace results.
3. Choose the **Tools -> New Schematic -> From Trace Result** command to create a new schematic frame with only the trace results. Click the **Undock** icon in upper right to make the frame a standalone *nSchema* window.

Figure 135 Displaying Trace Results for data[7:0] as a Schematic



4. Close all open schematic windows.

Show RTL Block Diagram in a More Meaningful Way

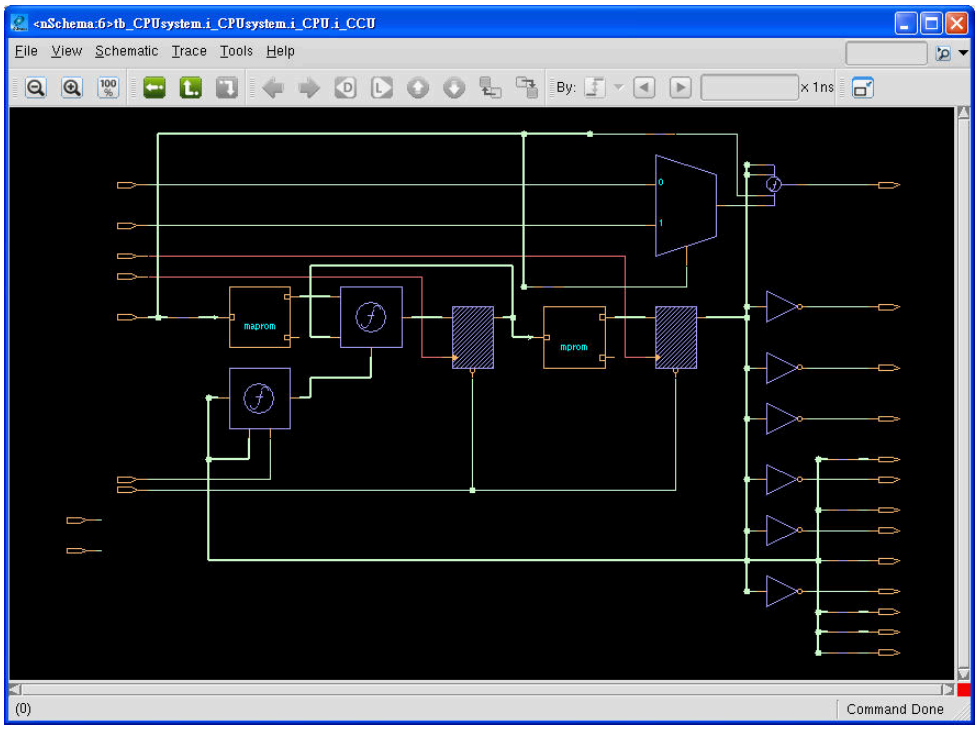
The Verdi platform can recognize some specific hardware elements and display them using meaningful RTL block-diagram symbols. See Appendix C: Enhanced RTL Extraction

Feedback

for a complete list of symbols. When you want to see the Boolean equivalent views, perform the following steps:

1. Open the *i_CCU* block in *nSchema*:

Figure 136 Displaying RTL Block as a Schematic



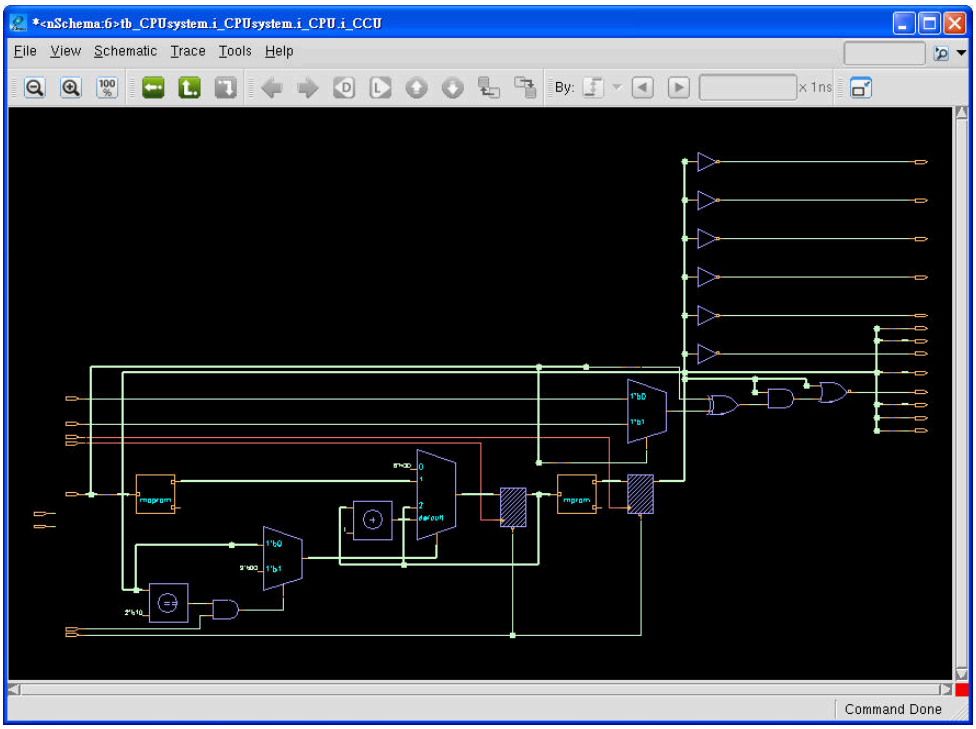
Note that the function symbol is in the upper right side.

2. Double-click the function symbol to see the associated source code (or drag to the source code frame and drop).
3. To enable the detailed RTL view, choose the **Tools -> Preferences** command to open the *Preferences* form.
4. On the *Preferences* form, select the **RTL** page under the **Schematics** folder and then turn *on* the **Enable Detailed RTL** option.
5. Click **OK**.

Feedback

The results are displayed similar to the schematics displayed in the following figure:

Figure 137 Displaying RTL Block in Detail



The function symbol is gone and has been replaced with an *xor*, *and*, and *nor* gate.

Note:
The preferences affect **all** windows. Use the **View -> Detail RTL** command to only change **one** window.

Generate Partial Schematics

Hierarchical

Often, the top-level block diagram is too cluttered and you want to be able to focus on a couple of blocks or signals.

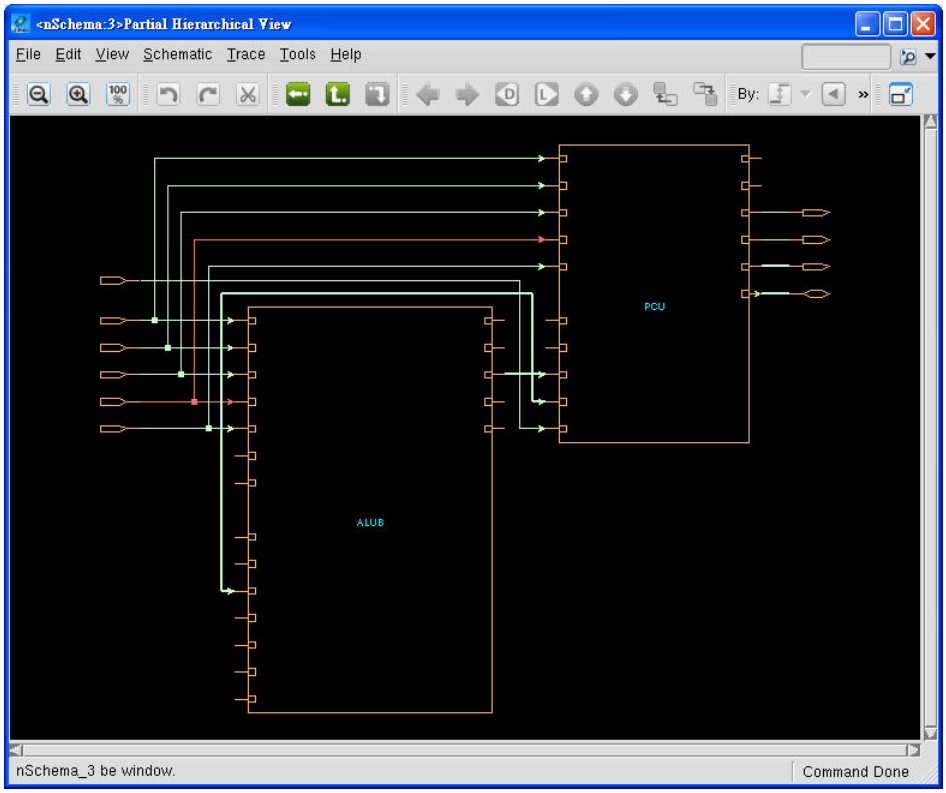
1. In the design browser frame, select *i_CPU*, and open a new schematic frame.
2. In the *nSchema* frame, select the *ALUB* block.
3. Press the <Shift> key and, select the *PCU* block (the same thing can be accomplished with multiple nets).

Feedback

4. Choose the **Tools -> New Schematic -> Partial Modular** command.

   A new schematic frame with the selected block and connections is displayed. Click the **Undock** icon in upper right to create a standalone *nSchema* window, as shown in the following figure:

Figure 138 nSchema Partial Hierarchical View



   The *CCU* block is not included in the above window.

5. Double-click the *ALUB* block and notice that there is a reduced logic display and therefore a partial hierarchical view.
6. Go back to the previous hierarchy using the **Pop View Up** icon.
7. Choose the **View -> InstPort Name** command to annotate the instance port names on the schematic.
8. Double-click the *AluOut[7:0]* port on the *ALUB* block (upper right) to expand the connecting logic. (Any port can be expanded).

Feedback

9. Click the **Undo** toolbar icon to get back to the previous view.
10. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

Flattened Window

The following topics work equally well for RTL designs, a gate-level design is more interesting.

1. Before you start, close the current Verdi session, and change the directory to *<working_dir>/demo/verilog/gate.*
2. Set the environment variables:

```
% setenv NOVAS_LIBPATHS $VERDI_HOME/share/symlib/32
% setenv NOVAS_LIBS lsi10k_u
```

3. Invoke the Verdi platform:

```
% verdi -f run.f -workMode hardwareDebug
```

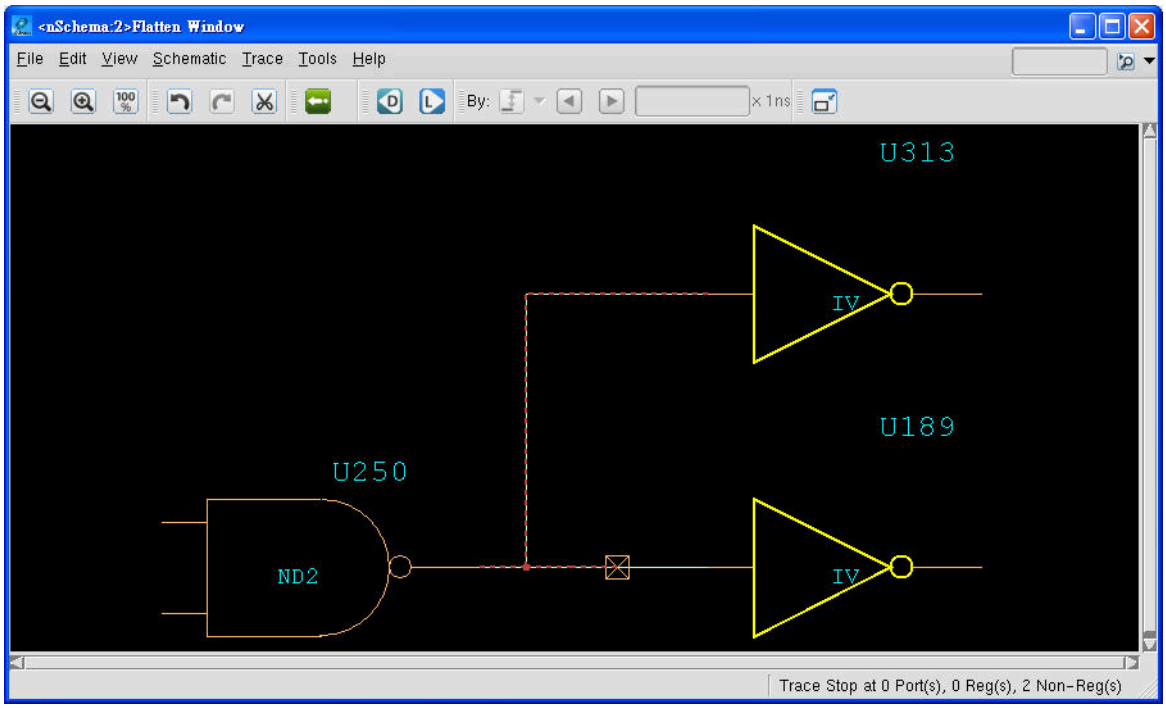
4. In the **Instance** tab of the design hierarchy frame, expand *system*, *i_cpu*, and then double-click *i_ALUB* to display the source code.
5. In the **Find String** box on the toolbar, enter *U250.*
6. Click the **Find Next** icon to find the instance on line 639.
7. Choose the **Tools -> New Schematic from Source -> Flattened** command.

   The associated NAND gate is displayed in *nSchema*. Click the **Undock** icon is the upper right to change the *nSchema* window to a standalone window.
8. Double-click the output port to expand the loading logic.

   The symbol of a box with a cross in it indicates a crossed hierarchy

Feedback

Figure 139 nSchema Expanded Logic



9. Choose the **View-> Instance Name** command.
10. Select the instance *U313*, and click the **Remove** icon to delete the gate from the view.
11. Click the **Undo** icon to add it back to the schematic.

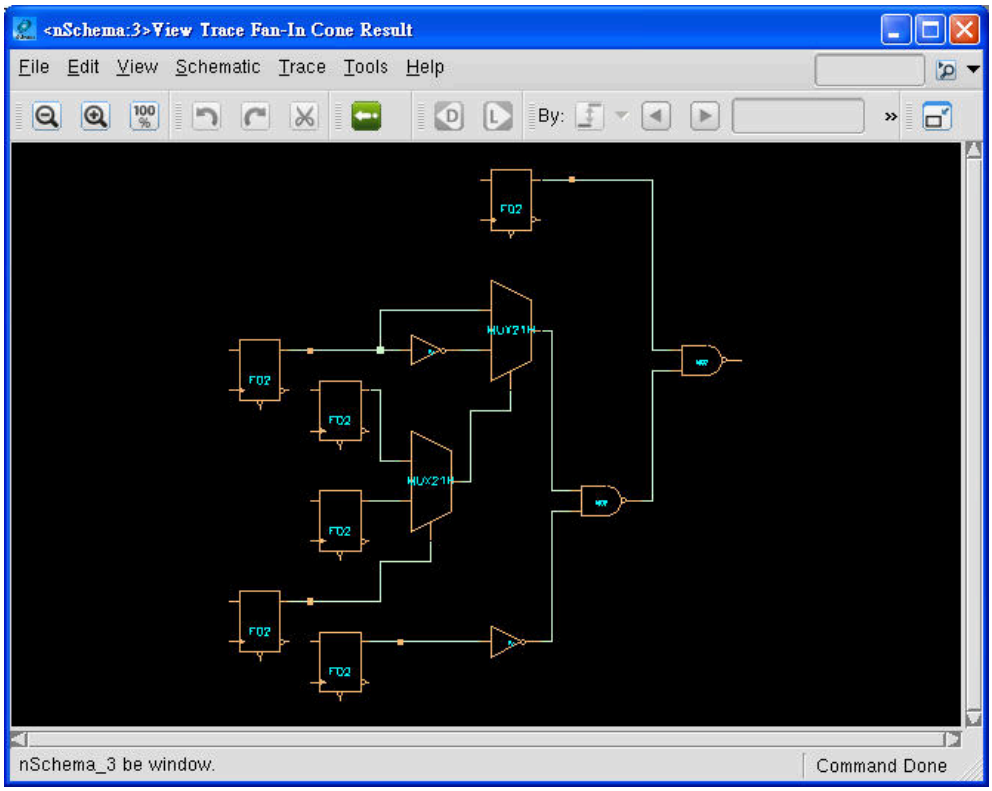
Fan-in and Fan-out

The Fan-in and Fan-out functions automatically generate the fan-in or fan-out cones for the selected instance or net.

1. Continue from the gate design.
2. Select *U250* in the *Flattened Window* schematic window.
3. In the *nSchema* window, choose the **Tools -> New Schematic -> Fan-in** command. Click the **Undock** icon in upper right to create the *View Trace Fan-In Cone Result* window from the *nSchema* window.

Feedback

Figure 140 nSchema Displaying Fan-in Cone



Several hierarchies are represented and tracing automatically stops on storage elements.

4. In the *Source Code* frame, select *U251*, line 640, of *i_ALUB*.
5. Use the middle mouse button to drag *U251* to the *View Trace Fan-In Cone Result* schematic view, and drop it.

   Note that *U251* is not connected to any of the existing logic.
6. Change the design browser frame to *i_CCU*, and open the source code.
7. Find *U248* on line 832.
8. Use the middle mouse button to drag *U248* to the *View Trace Fan-In Cone Result* schematic view, and drop it.

   Note that it automatically connects to an existing storage element.

   In partial flattened schematics, you can easily add logic by double-clicking to expand ports or dragging and dropping instances or nets from other windows.

Feedback

Trace Between Two Points

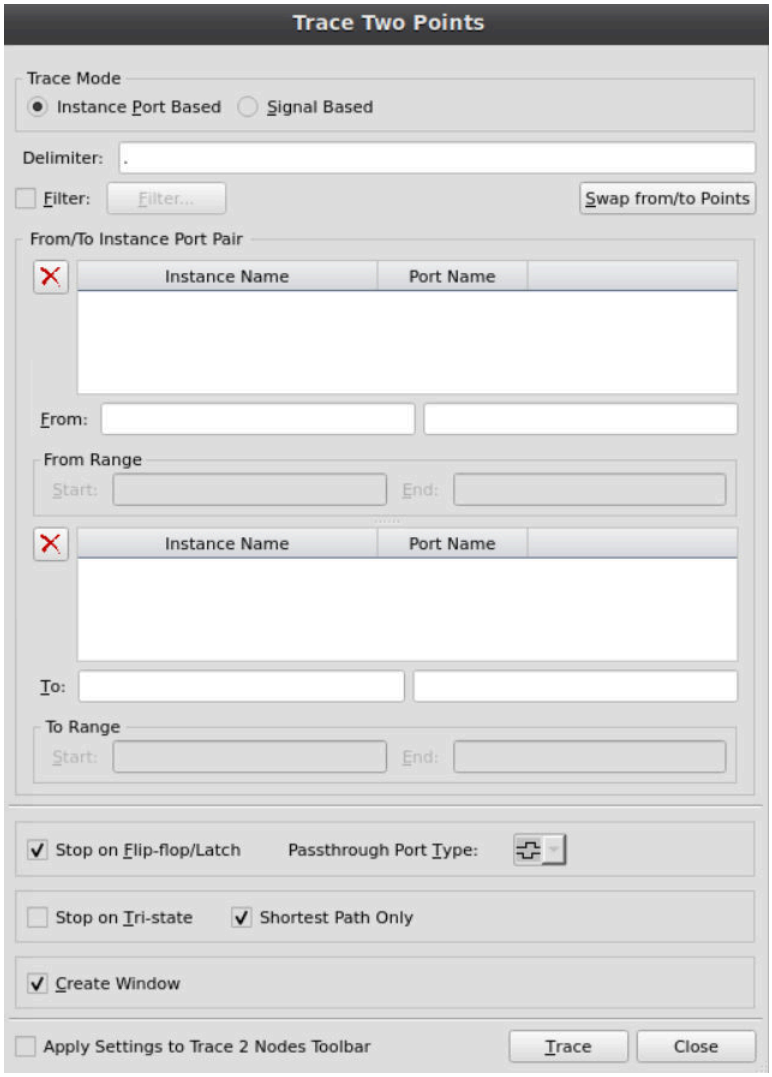
Tracing between two points isolates connecting logic between two storage elements and display the results.

1. Close the existing *nSchema* windows.
2. In the *nTrace* main window, choose the **Source -> Find String** command.
3. In the *Find String* form, enter *IDR_reg* in the pattern box.
4. Turn on the **In All Files** option.
5. Click the **Find All** button. The results are listed in the **Search** tab of the message frame.
6. Double-click the text line associated with *IDR_reg[7]*.
7. In the *nTrace* main window, click the **New Schematic** icon to open the *nSchema* frame. Click the **Undock** icon on upper right to change the *nSchema* window to a standalone *nSchema* window.
8. Drag the instance name *\IDR_reg[7]* from the *Source Code* frame and drop to the *nSchema* window. The instance is highlighted.
9. Zoom in around the highlighted instance by dragging the left mouse button over the area.

Feedback

10. Choose the **Trace -> Two Points** command. The *Trace Two Points* form is displayed as follows:

Figure 141 Trace Two Points Form



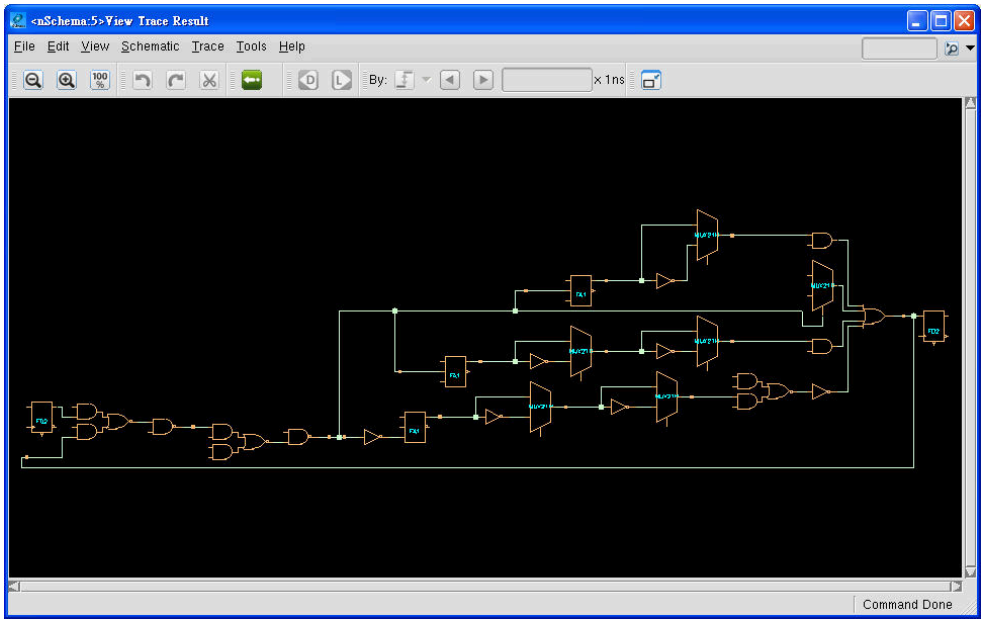
11. Use the middle mouse button to select the *Q* output port of *\IDR_reg[7]*, and then drag and drop it in the **From** box of the form.
12. Move the form out of the way while you find the next point.
13. In the *nTrace* main window, search for *IXR_reg* using **Find String**.
14. Double-click the text associated with *\IXR_reg[7]* in the **Search** tab of the message frame.

Feedback

15. In the *Source Code* frame, drag and drop *\IXR_reg[7]* to the **To** box of the *Trace Two Points* form, and select *D* in the *Port name* column.
16. In the *Trace Two Points* form, confirm the **Create Window** option is enabled (depressed), and click **Trace**.

    The results are displayed in a new *nSchema* window, click **Undock** icon to create the *View Trace Result* schematic window:

Figure 142 Results of Trace Two Points



Use Active Annotation to Show Signal Values

The **Schematic -> Active Annotation** command allows you to display signal values from simulation directly on the schematic. The display style can appear in either text value or line coloring (for values 0, 1, z, and x only).

1. In the *nTrace* main window, choose the **File -> Load Simulation Results** command to load the *gate.fsdb* results file.
2. In the *nTrace* main window, select the *i_PCU* unit in the **Instance** tab of the design browser frame, and open a new schematic.
3. Zoom in around the upper right corner.
4. In *nSchema*, choose the **Schematic -> Active Annotation** command or use the ‘x’ hot key to display the waveform values on the schematic.

5. Select the *ALU[7:0]* signal and use the **Search Forward** icon on the toolbar to step through value changes. You can also **Search Backward**.
6. Add line coloring annotation by turning on the **Schematic -> Annotate in Color** toggle command.

   The definitions of the annotation text and colors are as follows:

   - 1: logic high (green)
   - 0: logic low (yellow)
   - x: unknown (red)

Change the Color or the Line Style for Annotations

1. To change the color or the line style for annotation, choose the **Tools -> Preferences** command in the *nTrace* main window to display the *Preferences* form.
2. Select the **Color/Font** page under the **Schematics** folder.
3. Click the **Type** option menu.
4. Set the color and line style (including the annotation line coloring for value 1, value 0, value x, and value z) for the list of objects displayed.
5. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

Support for Annotation and Fault Coverage on nSchema Window

Annotation is supported in *nSchema* Full window, Flatten window, and Hierarchy Flatten window. The color and size of the fault coverage bars is based on the fault database and settings. When the fault data such as test enabled or disabled, fault color, campaign loaded or unloaded, is changed, the color and size of the fault coverage bars is recalculated and redrawn automatically in the *nSchema* window.

Following are examples of annotation supported on Full window, Flatten window, and Hierarchy Flatten window:

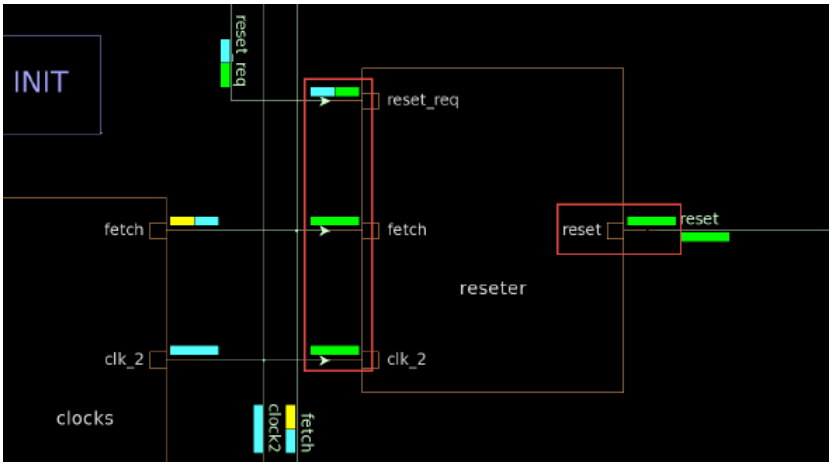
Example 1:

In the following figure, coverage bar is shown on the pins of instance port. The direction of the coverage bar can be **Input/Output/InOut**.

Feedback

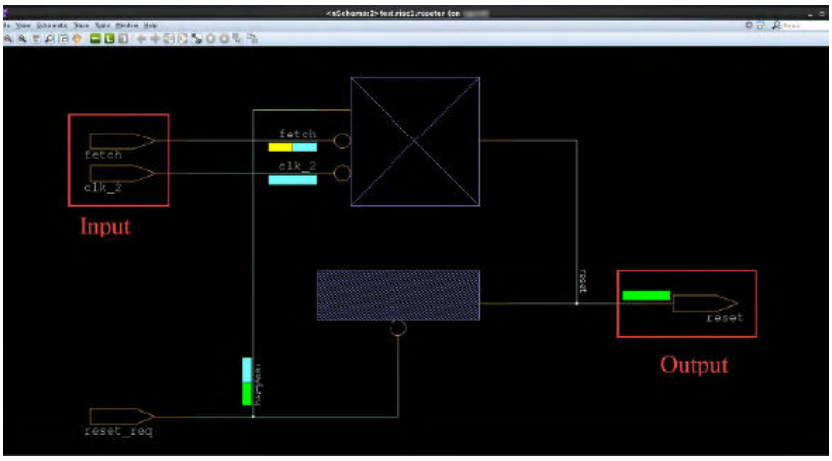
If FSDB annotation is on the input instance port, the coverage bar shifts before annotation, which means from the relative position, coverage bar is left and annotation is right.

If FSDB annotation is on the output or InOut instance port, the coverage bar shifts after annotation, which means that the coverage bar is on the right side of annotation.



Example 2:

In the following figure, coverage bar is shown on the pin of port. The direction of the coverage bar can be **Output/InOut**. If the port direction is Input, *nSchema* skips it.



Example 3:

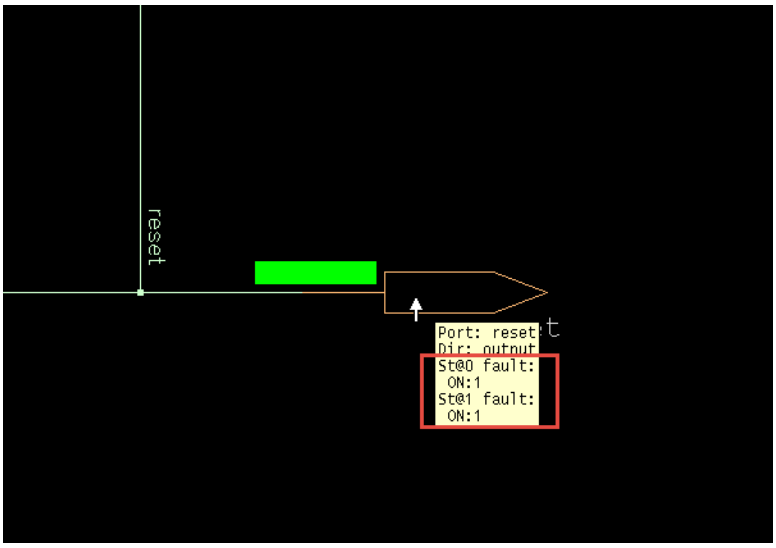
Coverage bar can also be shown horizontally or vertically for the signals. If the signal name is on the horizontal line, coverage bar position is under the signal. If the signal name is on the vertical line, coverage bar position is on the left side of the vertical line.

Feedback

Show Tip

When this option is turned on and a cursor moves over signals or blocks (for example, instances, ports, and instance pins), the signals or blocks are highlighted and a tip is displayed in the *nSchema* window. To enable this option, select **Schematic page -> Select -> Show Tip**.

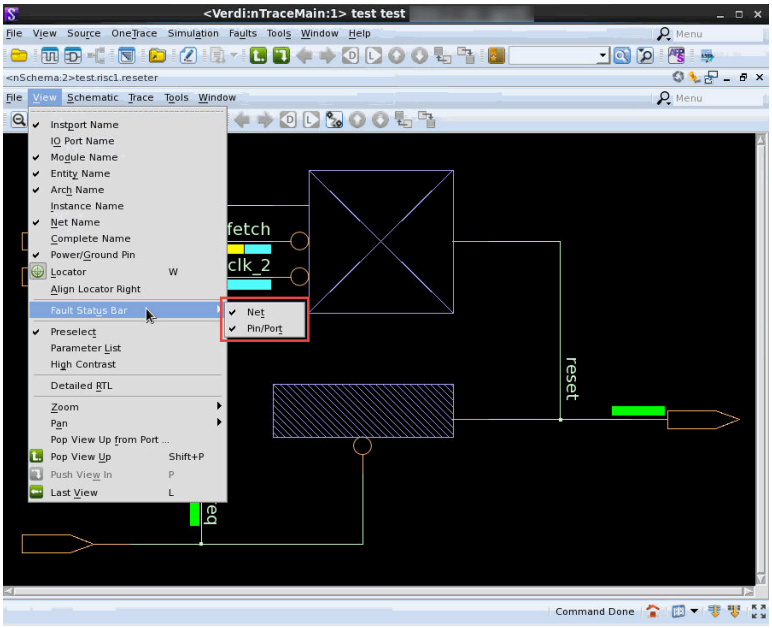
The following figure shows tooltip on the port that has fault data and the detailed fault information can be seen in the tooltip.



Fault Status Bar

This option enables coverage bar on net, instance port, or port.

Menu Bar: View -> Fault Status Bar



When you toggle **Fault Status Bar -> Net**, *nSchema* window shows coverage bar on the net view. When you toggle **Fault Status Bar -> Pin/Port**, *nSchema* window shows coverage bar on the instance port and port.

When the fault database loads successfully, the menus are enabled and toggled automatically. When fault database is closed or import fails, the menus are hidden.

New Tcl Command Options

schSetOptions

Description

Controls view options.

Syntax

```
schSetOptions -win windId -showFaultStatusBarOnNet
on|off -showFaultStatusBarOnPinPort on/off
```

Arguments

`-showFaultStatusBarOnNet`

Show or hide fault coverage bar on signals.

`-showFaultStatusBarOnPinPort`

Show or hide fault coverage bar on instance port and port.

Example

- The **Fault Status Bar -> Net** command dumps the Tcl command `schSetOptions -win winId -showFaultStatusBarOnNet off`.
- With the input Tcl command `schSetOptions -win winId -showFaultStatusBarOnPinPort off`, the option **Fault Status Bar -> Pin/Port** is unchecked and the fault coverage bar on the instance port is hidden.

Limitations

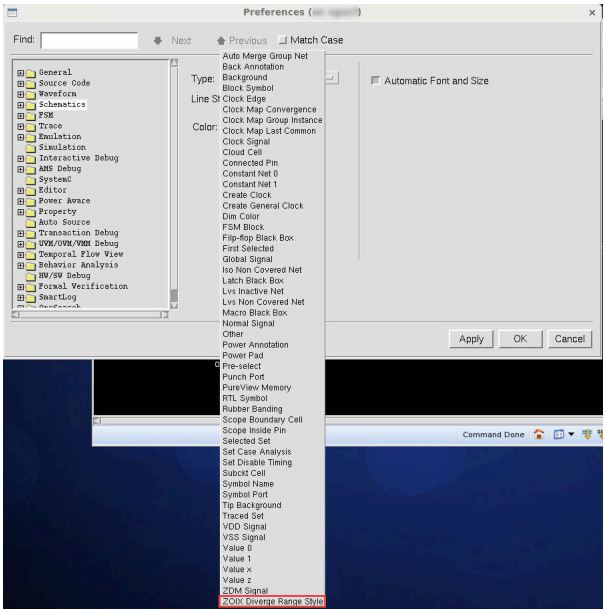
- In motif *nSchema* window, the color can be different from color set in fault database.
- This feature only draws coverage bars on the instance port, port, and signal.

  Note:
  If Golden Model (GM) and Fault Model (FM) has mismatch due to glitch, Verdi does not show mismatch value in the schematic.

Support for GM/FM Waveform

When you import GM/FM FSDB file in Verdi, *nSchema* window displays two values to indicate the mismatch in values. Different colors are used to highlight the mismatch part in the *nSchema* window. You can see the color change after toggling schematic menu option **Active Annotation**.

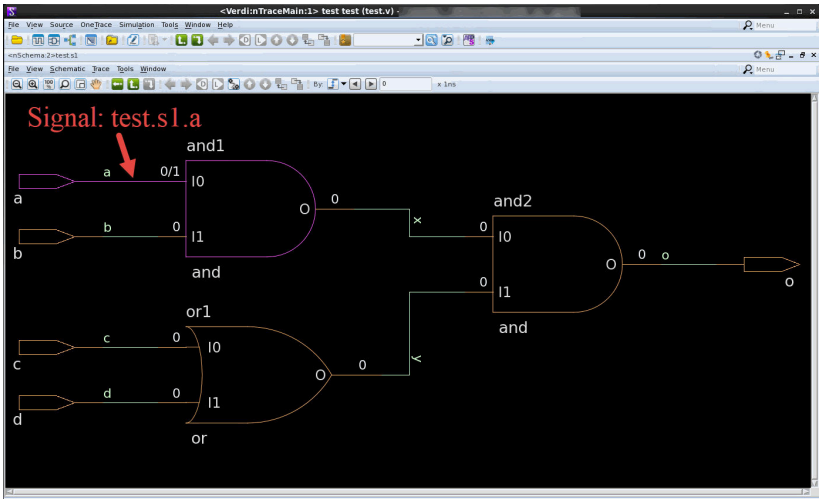
By default, the color setting is `ID_PURPLE6`. You can change the color setting in the *Preferences* form (go to **Preference -> Schematics -> Color/Font**) by clicking the **Type** list and select **ZOIX Diverge Range Style** option as shown in the following figure:



You can change the color patterns in different ways by applying certain rules. Following are the different examples of color pattern change:

Example1:

If the FSDB value on the signal is a mismatch, then the signal color changes. In the following figure, the FSDB value of the signal `test.s1.a` is 0/1, which is a mismatch, therefore, the color changes to `ID_PURPLE6`.

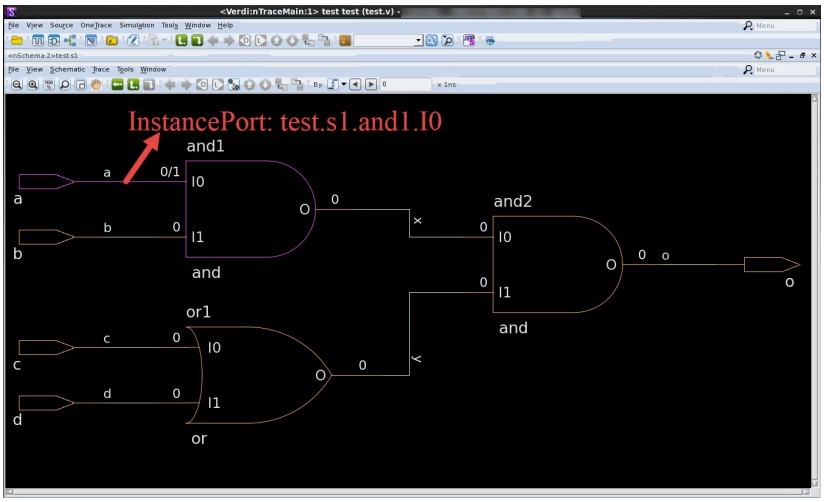


Example2:

If the FSDB value on the connected signal is a mismatch, then the instance port color changes.

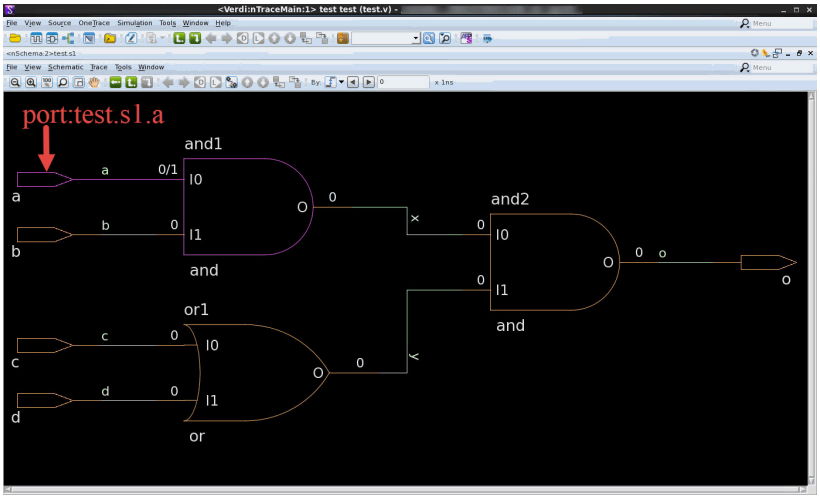
Feedback

When the instance port `test.s1.and1.I0` is connected to the signal `test.s1.a`, the FSDB value of the signal `test.s1.a` is 0/1, which is a mismatch. Therefore, the instance port color changes to `ID_PURPLE6` as shown in the following figure:



Example3

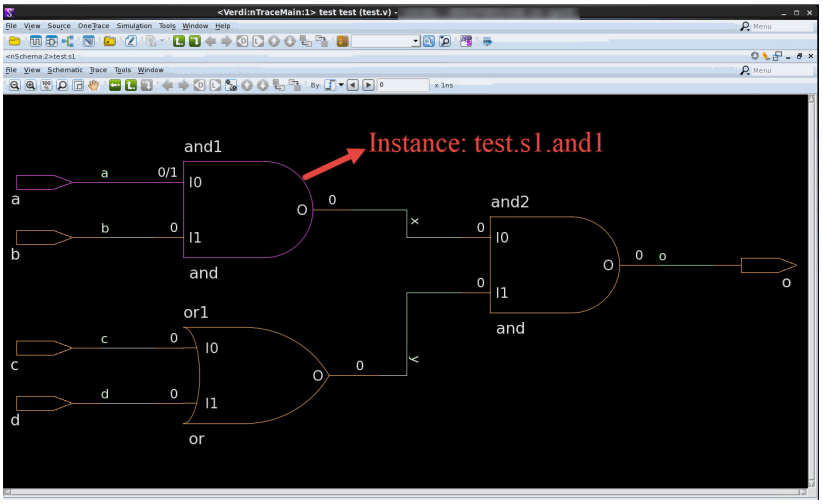
When the signal port `test.s1.a` is connected to the signal `test.s1.a`, the FSDB value on the signal `test.s1.a` is 0/1, which is a mismatch. Therefore, the instance port color changes to `ID_PURPLE6` as shown in the following figure:



Example4:

The instance color change is based on all the instance ports. If one of instance ports meet the change rule, the instance color changes to `ID_PURPLE6`.

If one of the instance ports is highlighted with `ID_PRUPLE6`, the color of `test.s1.and1` instance port automatically changes to `ID_PURPLE6` as shown in the following figure:

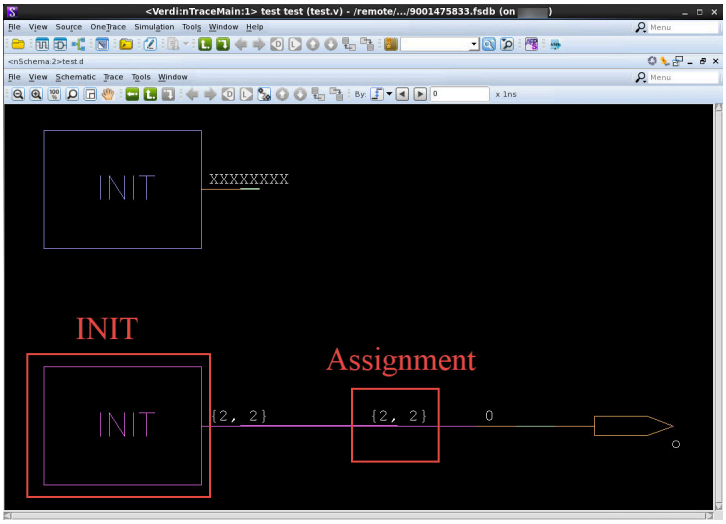


Example5:

For composite signal, which includes structure signal and bus signal, if the FSDB value on one of its sub signals is a mismatch, the color of composite signal changes to `ID_PURPLE6`.

For example, if the sub signal value of the composite signal array`1[1:0][1:0]` is a mismatch, the color of the composite signal changes to `ID_PURPLE6` in the *nSchema* window.

In the following figure, the signal is connected to two instance ports of **INIT** and **Assignment** blocks. The color of both the blocks changes to `ID_PURPLE6`.

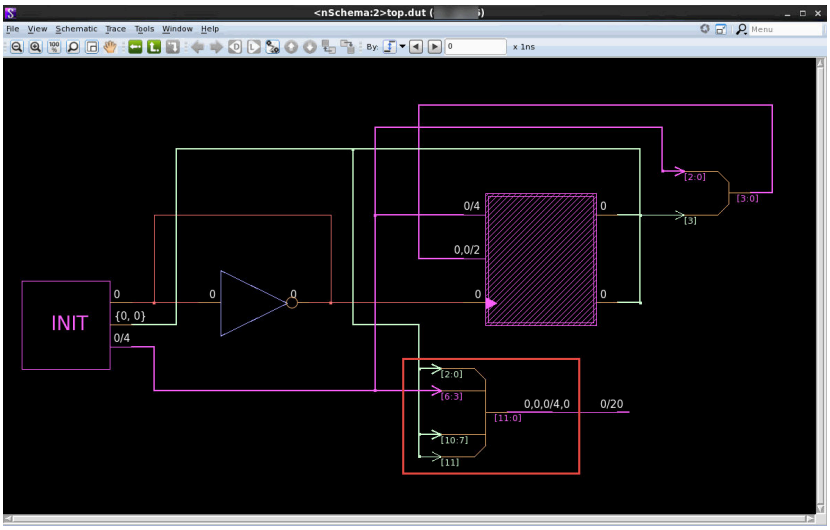


Feedback

Example6

For concat signal, if the FSDB value on one of its sub signals is a mismatch, the color of concat signal changes to `ID_PURPLE6`.

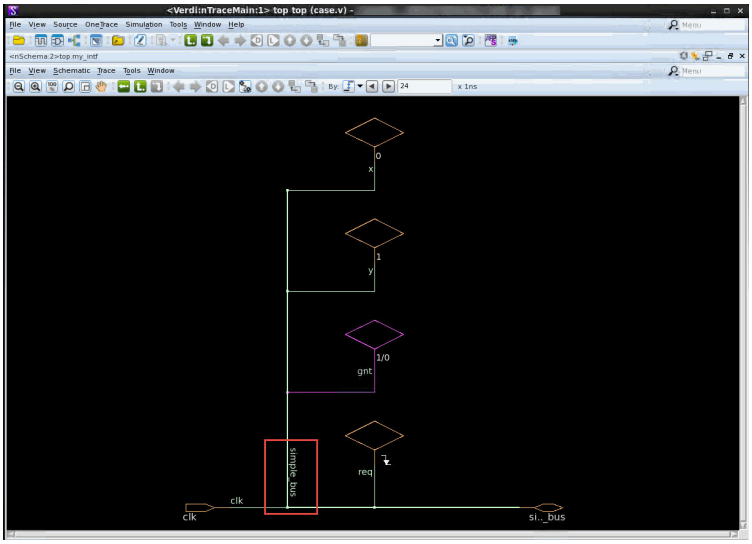
In the following figure, color of the concat signals, `top.dut.s_rp2[4][0]`, `s_rp2[5]`, `s_b4b[3:0]`, `s_rp2[4][3:1]`, changes to `ID_PURPLE6` as the signals have diverge range FSDB values (0,0,0/4,0). For the sub signal of the concat signals, the color is based on its own FSDB value.



Example7

For the interface signal, there is no color change as the signal has no diverge range check.

In the following figure, there is no color change for the interface signal `simple_bus` in the *nSchema* window.


0
x
1
y
1/0
gnt
req
simple_bus
clk
clk
si..._bus

8

nWave Tutorial

nWave is a state-of-the-art graphical waveform viewer and analyzer that is fully integrated with the source code, schematic, and flow views of the Verdi platform. A waveform search engine combined with backward and forward navigation allows you to search for signal transitions, bus values, discrepancies, and user-defined events easily.

nWave offers the following:

- Flexible signal group management
- Customizable glitch detection
- A built-in logic analyzer
- Logical operations and events
- Display of delays back-annotated from SDF files
- Mixed analog/digital (A/D) display capabilities, including overlap, vertical zoom, delta x and y, arithmetic operations, analog-to-digital signal conversion, and others
- Display of transactions and messages

Before you begin this tutorial, follow the instructions in the Before You Begin chapter. Refer to the User Interface chapter for more details regarding the *nWave* interface.

This chapter describes the following topics:

- Add Signals
- Manipulate the Waveform View
- Change Signal/Group Attributes
- Create New Signals/Buses From Existing Signals
- Save and Restore Signals
- Group or Ungroup Multiple FSDB Files
- Calculate Toggle Coverage
- Define Events and Complex Events

Feedback

- Support VHDL Fixed-point Types in nWave
- Capture and Display Frequency of a Scalar Signal as a New Signal
- Generate Dump Files From Verdi Signal RC Files

Add Signals

There are two primary methods to add signals as follows:

- Drag and drop signals from other Verdi frames or windows.
- Use the *Get Signals* form to search and add signals.

Add Signals From Other Windows

Perform the following steps to add signals to the *nWave* window:

1. In the design browser frame, expand *i_cpusystem* to display *i_cpu*.
2. Use the middle mouse button to drag and drop the *i_cpu* to the signal pane of *nWave* and display the I/O signals.

   Note:
   The horizontal yellow line in the signal pane moves so that it is under the signal added to the display. This line is the *signal cursor*, which marks the insertion point for signal commands for example, **Add**, **Move**, **Overlay**, and **Create Bus**. Right-click in the signal pane to access related commands.
3. In the design browser frame, expand *i_cpu* and double-click *i_ALUB* to display the source code.
4. In the *Source Code* frame, scroll to line 70 and select *AluBuf* from the signal declaration and drag to G2 to drop.

   Note:

   To select individual, non-contiguous signals, press and hold the <Ctrl> key and click the signals you want. To select a large range of signals, drag the mouse over the selection or select the first line in the range and hold the <Shift> key while selecting the last line in the range.

   You can also drag and drop signals or instances from other Verdi windows such as *nSchema* or the *Temporal Flow View*.

Feedback

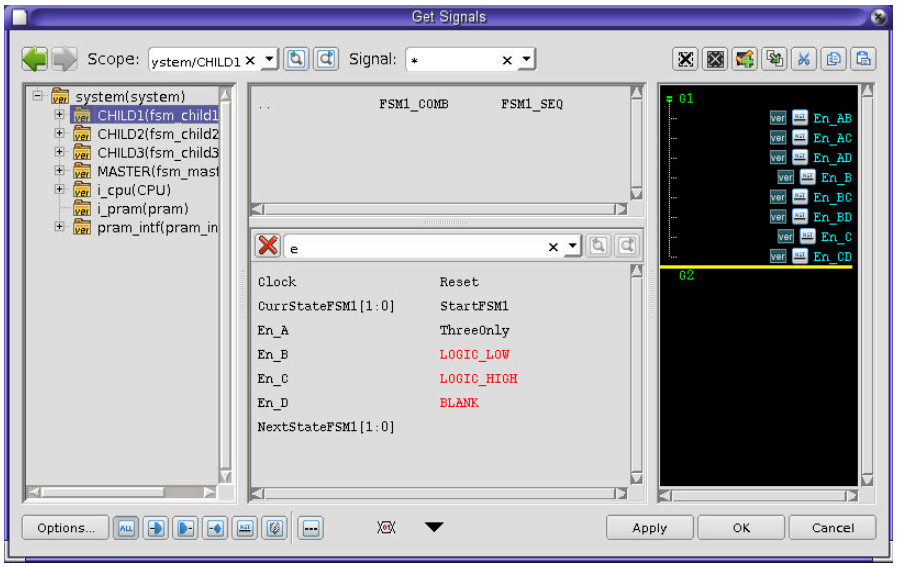
Use Get Signals to Add Signals

Perform the following steps to add signals using the *Get Signals* form. There are four panes in this form.

1. To display signals of interest, choose the **Signal -> Get Signals** command or click the toolbar icon .
2. In the *Get Signals* form, click *CHILD1* in the design hierarchy box.

   The signal list displays all the signals in *CHILD1*, as shown in the following figure:

Figure 143 Get Signals Window



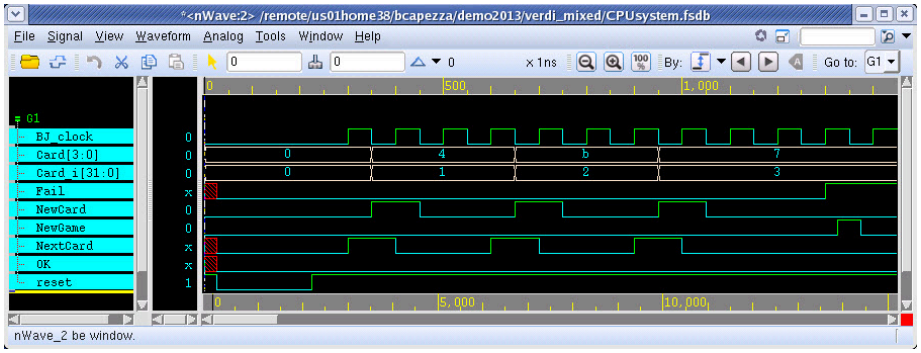
3. Click the middle button on group G3 in the mirror signal pane. This selection moves the signal cursor bar from group G2 to group G3, indicating that the insertion point for adding new selected signals is changed to group G3.
4. Click the **Select/Deselect All Signals** icon to select all signals.
5. Click the **Add Selected Signals** icon to add the signals to the mirror signal pane.

Feedback

Note:
You can rearrange the signal sequence in the mirror signal pane using the middle button to drag the signals to a new location.

6. Click **OK** to display the waveforms of the selected signals. The results are similar to the following:

Figure 144 Signal Display in nWave



Search for Signals to Add

You can also search for signals to add in the *Get Signals* form.

1. Type 'g' in the waveform pane to invoke the **Signal -> Get Signals** command using the hot key. The *Get Signals* form appears.
2. Expand *i_cpusystem* and *i_cpu* and then click *i_CCU*.
3. Set the signal cursor position under group G4 in the mirror signal pane.
4. In the *Get Signals* form, click **Options**.
5. On the **Search** tab of the *Options* form, turn on the **Search Signals with Case Matching** option.
6. Click **Close**.
7. Type *clo** in the **Find Signal** field.
8. Press <**Enter**>.

   Note that no signals match.
9. Now type *CLO** in the **Find Signal** field.
10. Press <**Enter**>.

    Signals with names starting with *CLO* are listed.

Feedback

11. Drag-left in the middle pane to select *CLOCK1, CLOCK2, CLOCK3,* and *CLOCK4.*
12. Click **Apply**.
13. In the *Get Signals* form, click **Options**.
14. On the **Search** tab of the *Options* form, turn on the **Search Signals in Sub-Scopes** option and click the **Close** button.
15. Click *i_cpusystem*.
16. Type *sel** in the **Find Signal** field.
17. Press <**Enter**>.

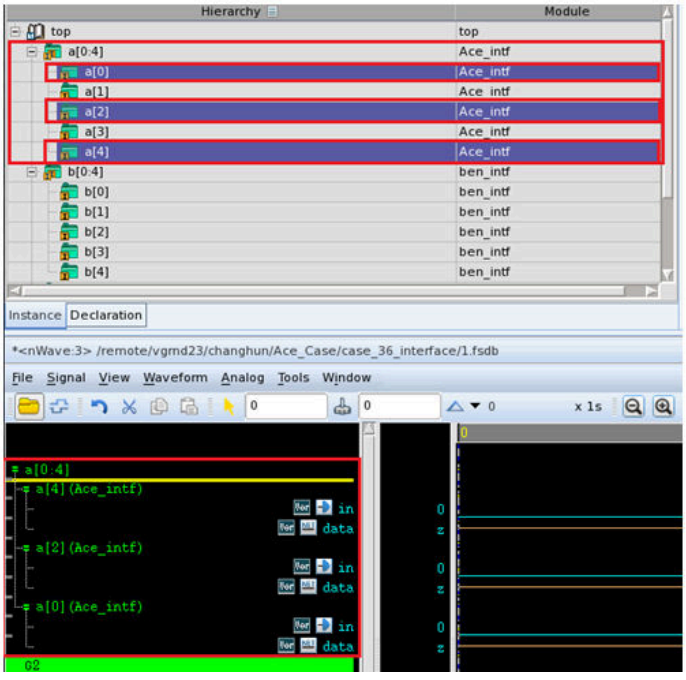
    Signals with names starting with *sel* in all scopes are listed.
18. Double-click *sel[2:0]* in the middle frame to add it to the right-hand window.
19. Click **OK**.

    The selected signals are displayed in *nWave* window.

Creating a Parent Group With Sub-Groups

When adding an array of interface with the same parent, it creates a parent group to contain these sub-groups, as illustrated in the following figure:

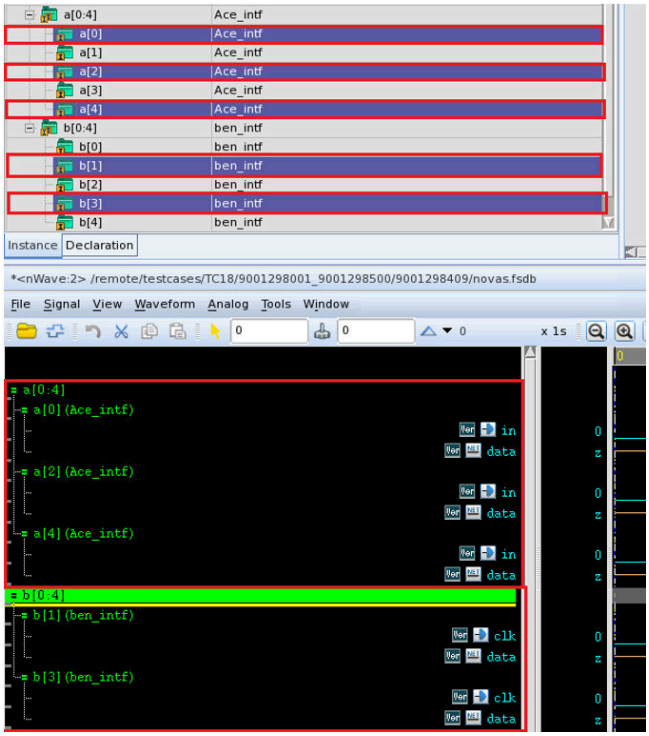
Figure 145 Array of interface added With the Same Parent Name



Feedback

If the array of interfaces is added without the same parent name, then it creates a parent group first to contain these sub-groups. The location for adding it depends on the cursor position of the horizontal yellow line in the signal pane as illustrated in the following figure:

Figure 146 Array of interface added Without the Same Parent Name

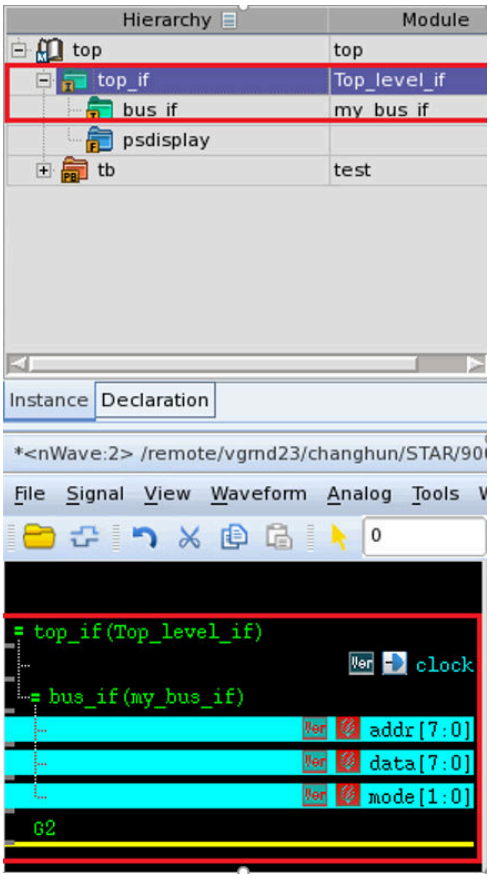


If you right-click to add array of interfaces to the waveform and the signals are not available in the group, then it creates a parent group to contain these sub-groups. However, when a group is not empty, and **Add to New Group** option is turned *on*, it creates a parent group to contain these sub-groups. If the **Add to New Group** option is turned off and you right-click to add array of interface, it creates parent group to contain these sub-groups.

Feedback

When adding an interface or a scope to signal pane, it adds the complete interface including the nested interface to the waves and organizes them hierarchically as illustrated in the following figure:

Figure 147 Added Nested Interface



Manipulate the Waveform View

nWave displays a cursor and a marker in the waveform pane. Use the cursor/marker to measure time differences, perform a fast zoom, or examine signal values. The cursor appears as a dashed yellow line, and the marker is a dashed white line. You can also add grid lines to make it easier to line up multiple signal transitions.

This section consists of the following topics:

- Set the Cursor/Marker Positions
- Zoom Cursor With Three Clicks
- Fast Zoom on the Full Scale Ruler

Feedback

- Pan the Waveform
- Turn On/Off Signal Grids
- Add Marker Labels
- Change the Display Sequence of Signals
- Search for Signal Value Transitions
- Add Comments
- Compress Time Ranges
- Split the Waveform View

Set the Cursor/Marker Positions

1. Left-click the signal *R_W* under group G1 where it transitions from 0 to 1 at time 276 in the *nWave* window and note the following:
   - A vertical *cursor* line appears in the waveform pane.
   - The simulation time (276 ns) associated with the cursor's current location is displayed in the toolbar.
   - The value pane is automatically updated to reflect the values on the signals at the current cursor time.
   - The Active Annotation values (if enabled) in the source code frame are automatically updated to reflect the values on the signals at the current cursor time.

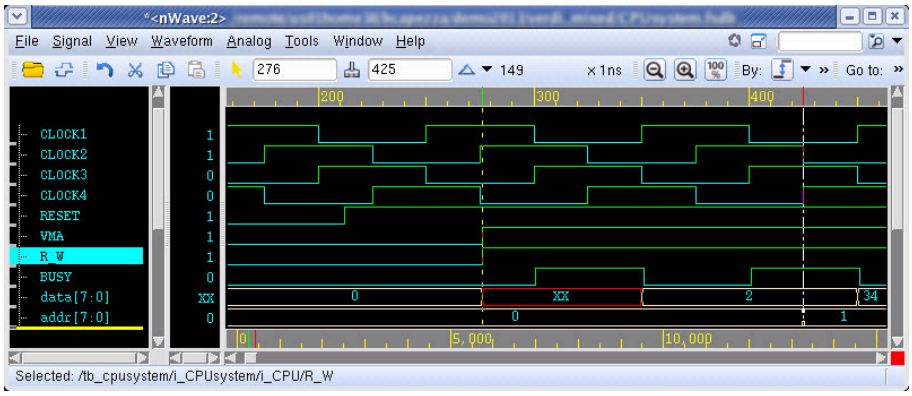
   Note:
   You can also use the **Waveform -> Go To -> Time** command to set the cursor position to the specified time.
2. Click the middle button on the next transition (0 to 1) of the signal *addr[7:0]* under group G1 and note the following:
   - A vertical *marker* appears in the waveform pane.
   - The simulation time (425 ns) associated with the marker's current location is displayed in the toolbar.
   - The delta (time difference) between the cursor and marker is displayed in the toolbar.

   Note:
   By default, the cursor and marker snaps to the closest transition on the selected signal. (You can turn on the **Waveform -> Snap Cursor to**

Feedback

Transitions toggle command to allow you to set the cursor/marker to any location).

Figure 148 Example of Time Delta Between Cursor and Marker in nWave Window



3. In the *nWave* window, choose the **View -> Zoom -> Zoom All** command (or the toolbar icon or the “f” key) to display the entire simulation results for the signals currently on display.

Zoom Cursor With Three Clicks

You can set the cursor and marker positions using left-click and middle-click in a signal's waveform, then click the **Delta Time** icon on the toolbar to zoom in and view the waveform between the cursor and the marker.

Fast Zoom on the Full Scale Ruler

A full-scale ruler appears at the bottom of the waveform pane. The ruler shows the full simulation time span of the opened dump file. To view a time range quickly, left-click and middle-click the specified time along this ruler, then click the **Delta Time** icon on the toolbar to view the waveform between the two points. You can also drag-left along the full-scale ruler for **Fast Zoom**.

Pan the Waveform

To pan different time ranges or signals, use the waveform pane's horizontal or vertical scroll bars and the up, down, left, and right arrow keys.

Feedback

Use BindKey Commands

The following table lists the bindkeys that you can use to view the waveforms quickly and easily:

BindKey	**Action**
Up Arrow Key	Pan Up
Down Arrow Key	Pan Down
Left Arrow Key	Pan Left
Right Arrow Key	Pan Right
z	Zoom Out
Z	Zoom In
f	Zoom All
l	Last View

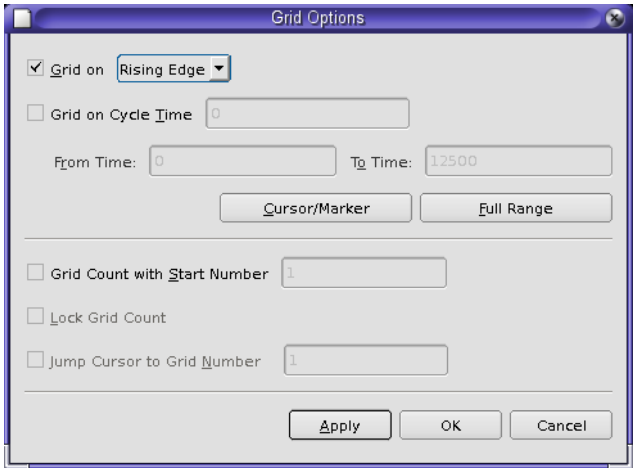
Turn On/Off Signal Grids

1. In the *nWave* window, zoom in around the 0 to 1000 time range.
2. Select *CLOCK1*.
3. Choose the **View -> Grid Options** command to open the *Grid Options* form. Turn on the **Grid on** option and select the **Rising Edge** in the option field.

Feedback

4. Turn on the **Grid Count with Start Number** option and input **1** in the associated text field. The *Grid Options* form is similar to the following:

Figure 149 Grid Option form



5. Click **Apply** to show the numbering along with each grid line starting from the current cursor time.
6. Disable the **Grid on** option and click **OK** to remove grids.

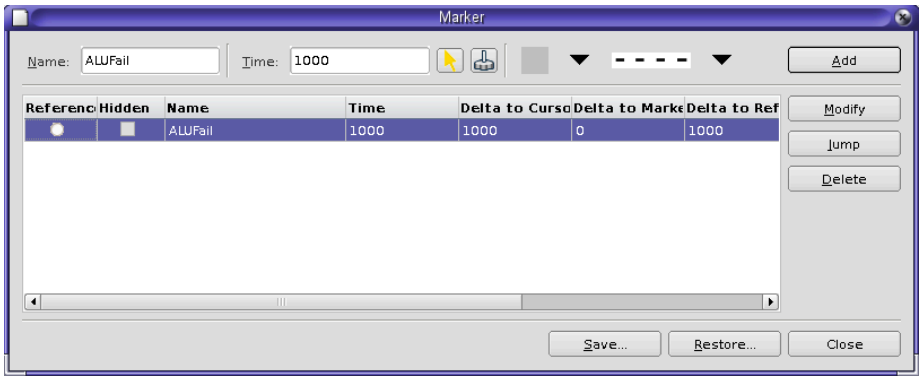
Add Marker Labels

1. In the waveform pane, place the cursor on the 3 to a transition of signal *alubuf[7:0]* at time 825.
2. Choose the **Waveform -> Marker** command (or use the **shift-m** bindkey) to open the *Marker* form.
3. In the *Marker* form, specify *ALUFail* in the **Name** field.
4. Click **Get Cursor Time** .
5. Click **Add**.

Feedback

The *Marker* form is similar to the following:

Figure 150 Marker Form



6. Click **Close**.

   The marker label is added to the top of the waveform view at the appropriate time and the **Goto Marker** field is added to the toolbar.

Figure 151 Marker Label Additions

Change the Display Sequence of Signals

To rearrange signals, you can drag-left to select and drag-middle to move or remove signals from the waveform pane with the **Cut** icon. The signals are copied to the clipboard. You can then use the **Paste** command to copy signals from the clipboard to the signal cursor position.

1. To rearrange the display sequence of signals, drag-left in the signal pane to select four signals under group G1. For example, *RESET, VMA, R_W,* and *BUSY*.
2. Use the middle mouse button to drag them to right after the signal *addr[7:0]*.
3. Click the **Cut** icon to remove the *RESET, VMA, R_W,* and *BUSY* signals. Then middle-click to set the signal cursor position under group G3.
4. Click the **Paste** icon to insert the four signals at the current signal cursor position.

Feedback

5. Click the **Undo** icon . The four newly inserted signals are deleted.
6. Click **Undo** again and the signals are inserted back under group G3. You can undo for one level only.

Search for Signal Value Transitions

You can search a signal by **Any Changes, Rising Edge, Falling Edge, Analog Values, Bus Values, Mismatch, Taint Change**, or **Invoke Search Constraint Window**.

Note:
Taint Change is shown in the search menu only if there is a file which has T-propagation information and Taint Change supports setting search constraint.

1. Select the signal *CLOCK1*.
2. Click the **Search Forward** icon on the toolbar. The cursor jumps to the next signal transition.
3. Click the **Search Backward** icon to move the cursor to the previous transition.
4. Click the **Search By Any Changes** icon to change the search criteria to **Rising Edge**.
5. Search again and notice the difference.
6. Select the signal *data[7:0]*.
7. Click the **Search By Any Changes** icon to change the criteria to **Bus Values**.
8. In the *Search Value* form, enter *20* in the **Signal Value** field.

   Note:
   You can also search for value to value transitions (including mnemonics) by entering ‘value1 -> value2’ in the **Signal Value** field.
9. Click **OK**.

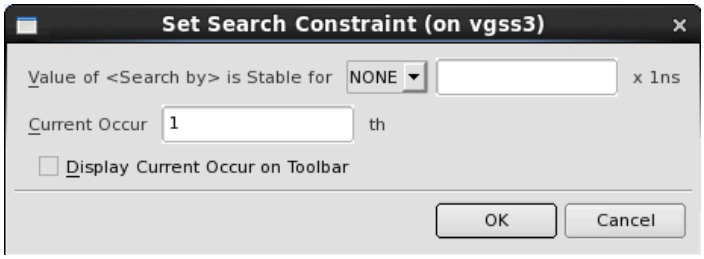
   To change the bus value, choose the **Waveform -> Set Search Value** command or change the value on the toolbar to define the search value.
10. Click the **Search Forward** or **Search Backward** icons to find the value of *20* at time 4650 ns.
11. With *data* still selected and a bus value of *20*, choose the **Waveform -> Set Search Constraint** command.

    The *Set Search Constraint* form displays.

Feedback

12. Change the **Value of <Search By> is stable for** menu to <= and enter 100 in the **x 1ns** box, as shown in the following figure:

Figure 152 Set Search Constraint Box Filled



13. Click **OK**.
14. Use **Search Forward** and **Search Backward** icons to find any occurrences of the value *20* on *data*, stable for less than 100 ns.

    There is one at time 1576 ns.
15. Change **Search Constraint** back to **NONE**, and **Search By** to any transition.

Add Comments

You can insert comments to indicate items of interest.

1. Continue from the previous example and place the cursor under *data[7:0]*.
2. Choose the **Signal -> Comment -> Insert** command to add a comment field to the waveform.

   The cursor should still be at time *1576* where data value *20* is less than *100*. You want to identify this location with a comment.
3. Choose the **Signal -> Comment -> Add Attached Square Box** command.
4. Left-click near the value of 20 between the red comment lines to add the comment box.

   After the comment box is added, you can reposition it with the left mouse button.
5. Place the cursor over the **Comment Box** text and select the text by double-clicking.
6. Press **Delete** on the keyboard to remove the text.
7. Type '`This is where the width is too small.`' to enter the new comment text.
8. Resize the comment box as needed.

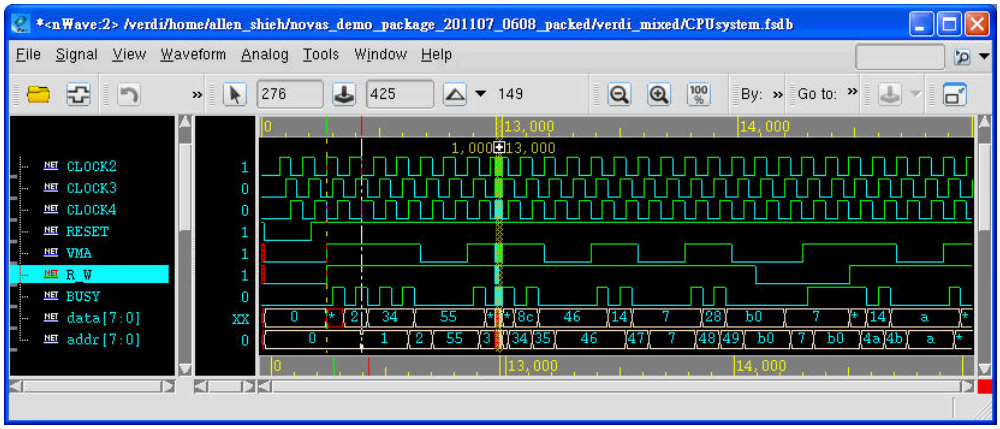
Feedback

Compress Time Ranges

You can compress time ranges to make viewing different times easier.

1. In *nWave*, choose the **View -> Compress Time Range** command to open the *Compress Time Range* form.
2. In the *Compress Time Range* form, enter 1000 in the **From Time** field and 13000 in the **To Time** Field.
3. Click **Insert**.
4. In the waveform, click the **Zoom All** toolbar icon. The *nWave* window is similar to the following figure:

Figure 153 Compress Time Range



The yellow vertical bar indicates where time is compressed. Only the waveform view is affected.

5. In the *Compress Time Range* form, click **Delete All** and **Close** to remove the compressed time range.

Split the Waveform View

You can split the waveform window to keep a standard set of signals in the top part of the window while you scroll through the remaining signals.

1. In *nWave*, choose the **Window -> Horizontal Split** command to split the window.

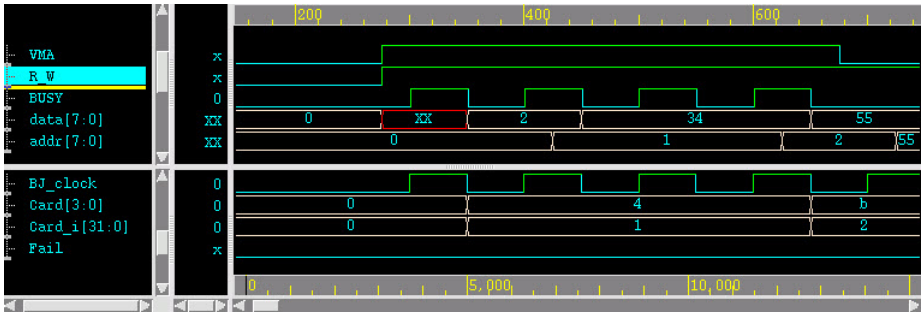
   You can select the separating bar and drag to change the split size.

2. In the upper split, scroll to display *addr[7:0]* and *data[7:0]*.

Feedback

3. In the lower split, scroll to display the signals in group *G4*. The *nWave* window is similar to the following figure:

Figure 154 nWave Split Window



4. Choose the **Window -> Stop Split** command to return to a single window.

Change Signal/Group Attributes

Search for a Group

1. Scroll to the bottom of the waveform list.
2. Right-click the signal pane. A **Signal** right mouse button command menu appears.
3. Select the group on the **Go To** sub-menu to jump among groups.
4. Go to group G1.

Change the Group Name

1. Right-click the group G1. A **Signal** right mouse button command menu displays.
2. Choose the **Rename** command. G1 turns orange.
3. Drag-left to highlight G1.
4. Type *CPU* and press <**Enter**>. The group name G1 changes to CPU.

   Note:
   You may need to use delete or backspace to remove the existing group name.

5. Change group G2 to *ALUB*, and change group G3 to *MISC*.

Feedback

6. Double-click the *ALUB* group to collapse.

   Note:
   You can also create hierarchical groups by choosing the **View -> Group Manager** command.

Modify the Display Format in the Value Window

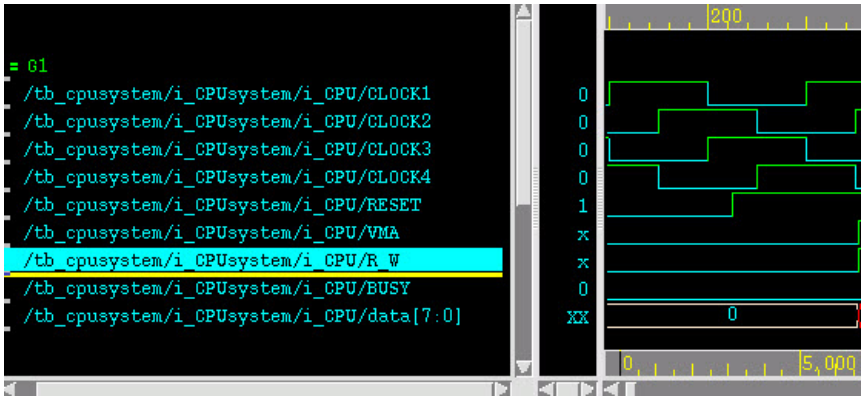
1. Zoom the waveform to the time range between 6500 and 7500.
2. In the signal pane, search for the bus *addr[7:0]*. The bus values displayed on the waveform are in hexadecimal format.
3. In the value pane, right-click the value of *addr[7:0].* A data format menu appears.
4. Choose the **Radix -> Binary** command. The value displayed for *addr[7:0]* changes to binary format.
5. Change back to hexadecimal format.

Display Hierarchical Signal Names

1. Choose the **View -> Hierarchical Name** command to display the signal names with their full hierarchical paths.
2. To make the signal pane bigger, drag the boundary between the signal pane and the value pane to widen the signal name pane. This change allows you to view the full hierarchical names.

   The *nWave* window is similar to the following figure:

Figure 155 Full Hierarchical Names



3. Choose the **View -> Hierarchical Name** command again to restore the signals to their base names.

Feedback

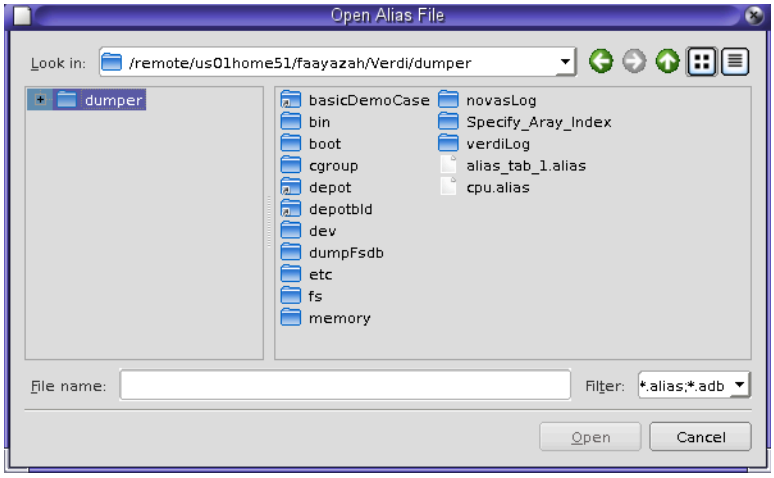
Add Alias to Display Bus Values

You can display logic states in a more meaningful way than by only viewing the plain logic values using the alias mechanism in *nWave*. You can associate mnemonics with logic states using the **Waveform -> Signal Value Radix -> Add Alias from File** command. Using a symbolic alias can make your debugging process easier.

1. In the signal pane click the signal *data[7:0].*
2. Choose the **Waveform -> Signal Value Radix -> Add Alias from File** command.

   The *Open Alias File* form displays, as shown in the following figure:

Figure 156 Example Open Alias File Form



3. Select the *CPU.alias* file under the *<working_dir>/demo/verilog_mixed* directory and click **OK**.

   The values displayed on the waveform for signal *data[7:0]* change to alias strings, which are more readable and meaningful. Use a text editor to create your own alias file by following the format in the *CPU.alias* file.

   Note:
   You can add color to the alias values by choosing the **Waveform -> Signal Value Radix -> Edit Alias** command while the aliased signal is selected. This opens the *Alias Editor* form where you can specify a background color for the alias values.

4. Create a copy of the signal *data[7:0]*.

a. In the signal pane, place the cursor above *data[7:0]*.

b. Select *data[7:0]*.

c. Click the **Copy** icon on the toolbar and then click the **Paste** icon.

5. Place the cursor over the value in the value column for the *data[7:0]* signal copy and right-click to view the right mouse button menu.
6. Choose the **Remove Local Alias** command.

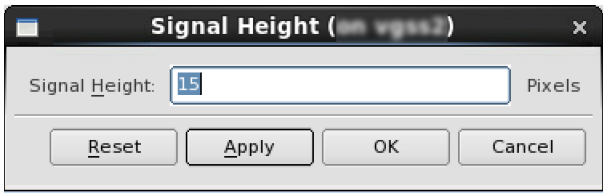
   The values displayed on the waveform for *data[7:0]* signal copy change back to hexadecimal values. Now you can see the numeric value and the mnemonic value simultaneously.

Change the Spacing and Signal Height

You can change the spacing equally among all displayed signals. You can also change the signal height for each signal individually.

1. Disable the **Signal -> Select Group Mode** toggle command (Group/Signal). This allows you to easily select all signals in a group instead of the group name.
2. Click group *MISC* to select all the signals under that group.
3. Choose the **Waveform -> Height** command and type *20* in the **Signal Height** field in the *Signal Height* form, shown in the following figure:

Figure 157 Signal Height Form



4. Click **Apply** to change the signal height to 20 pixels.
5. Click **Reset** to go back to default height.
6. Choose the **Waveform -> Spacing** command to change the signal spacing.

   Note:
   The height and spacing can be changed globally through the **Waveform -> Default Value** folder -> **Display Signal** page on the *Preferences* form (invoked with the **Tools -> Preferences** command).

Feedback

7. Turn on the **Signal -> Select Group Mode** toggle command (Group/Signal).

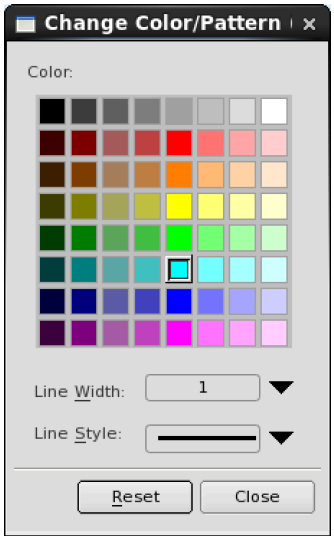
   This allows you to select group names.

Change Signal Color/Pattern

To change a signal's color or line width/style, perform the steps following steps:

1. Choose the **Waveform -> Color/Pattern** command. The color palette is displayed, as shown in the following figure:

Figure 158 Example Color/Pattern Palette



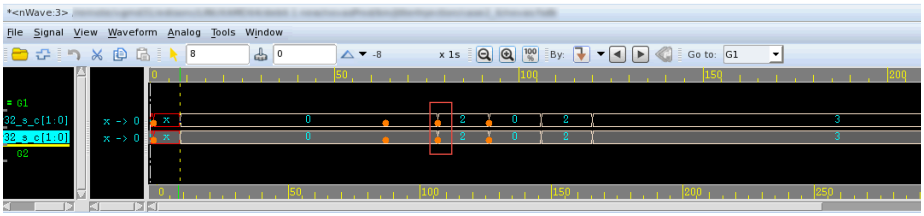
2. Select the signals you want to change and the preferred color from the color palette. The color for the selected signals changes accordingly.
3. You can also change the signal's line width and line style by setting the corresponding option menu.
4. Click **Reset** on the color/pattern palette to change the selected signals to their default color/pattern.

Display Jitter Injection in Waveform Pane

Bus, struct, and union signals show an orange dot in waveform pane if there is any one member signal with jitter injection as shown in the following figure. The bus signals can have jitter event on individual bits. The jitter injection can coexist with force.

Feedback

Figure 159 Jitter Injection in Waveform Pane



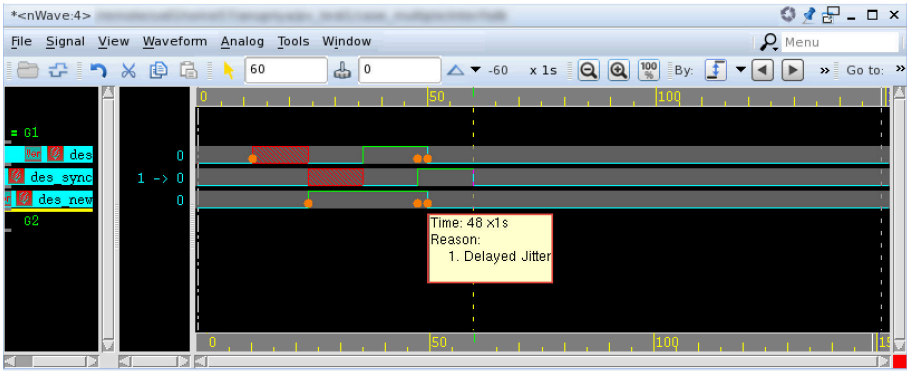
If Bus/D2A signals are generated using signals having jitter information, the new created runtime signals does not include jitter information.

If multiple signals overlap, waveform pane does not show jitter indicator on signals.

The cursor tip on the orange dot shows details about the jitter as shown in the following figure:

To enable Cursor tip on the orange dot, go to **Tools** → **Preferences** → **Waveform** → **View Options** → **Waveform Pane** → **Enable Tip**.

Figure 160 Jitter Injection with Cursor Tip Enabled



Create New Signals/Buses From Existing Signals

Sometimes it may be necessary to manipulate signals for better understanding or to test a theory.

There are two methods to perform this function:

- Logical Operations
- Bus Creation

Anything created with these commands can be saved to the signal file.

Feedback

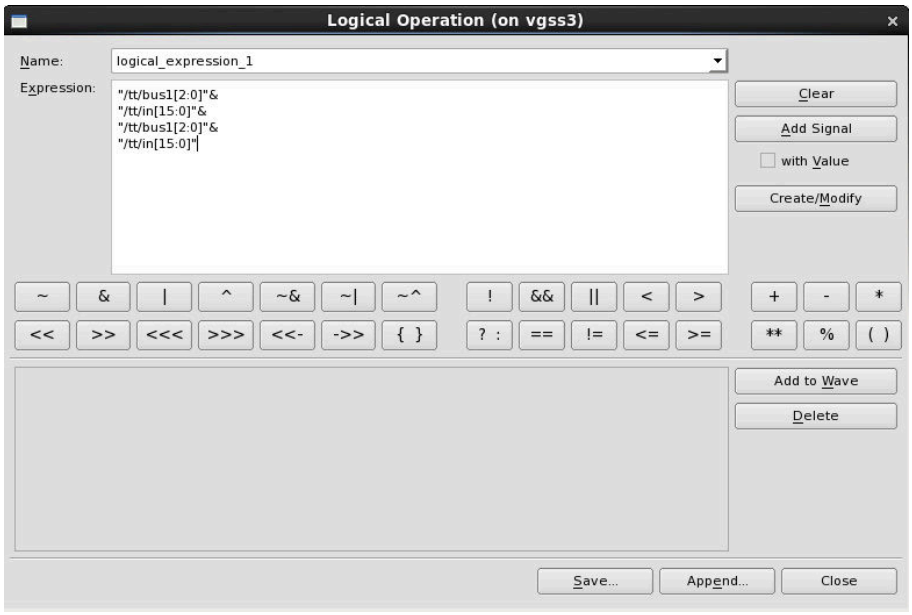
Logical Operations

If you want to view what the waveform looks like if *BUSY* was combined with inverted *VMA* signal.

1. In the CPU group, select *BUSY* and *VMA* (press and hold the <Ctrl> key to select multiple signals).
2. Choose the **Signal -> Logical Operation** command.

   The *Logical Operation* form displays, as shown in the following figure:

Figure 161 Logical Operation Form

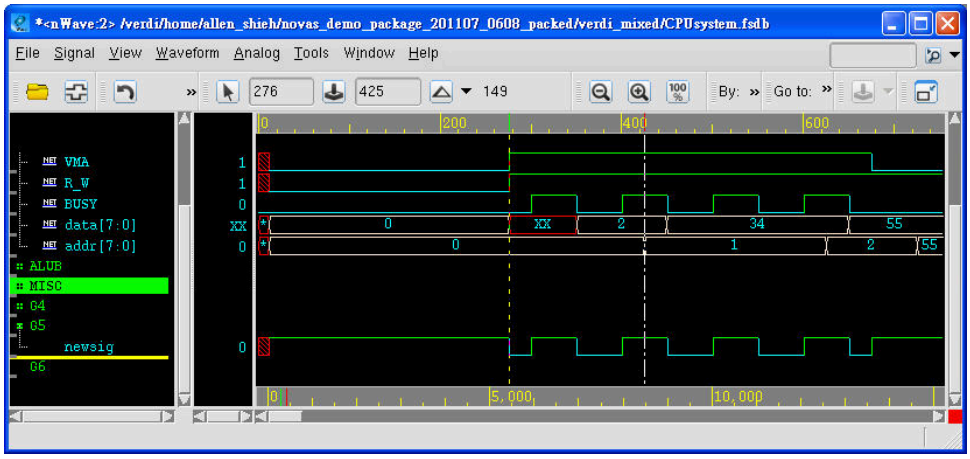


3. Enter *newsig* in the **Name** field.
4. In the **Expression** field, highlight the “&” symbol and delete using the *Delete* key on the keyboard.
5. With the cursor between the signals, left-click the *|:B-or*, *R-or* button under the **Expression** field to insert the new operator.
6. Put the cursor in front of the string for VMA and insert the *~:B-negation* operator.

Feedback

7. Click **Create/Modify** to create the new signal. The *nWave* window is similar to the following figure:

Figure 162 Logical Expression



The new signal is added to the waveform at the current cursor location.

8. Click **Close**.

Bus Creation

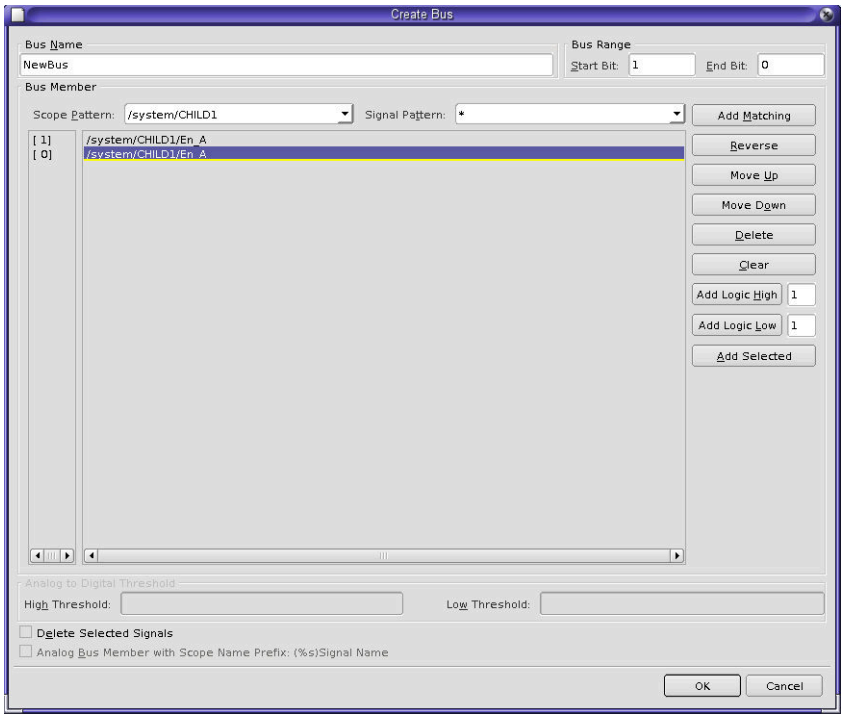
This exercise describes how to create a bus from the displayed signals.

1. Put the cursor in group G4.
2. Select signals *CLOCK1, CLOCK2, CLOCK3,* and *CLOCK4* in group G4.
3. Choose the **Signal -> Bus Operations -> Create Bus** command, or use the right mouse button and choose the **Bus Operations -> Create Bus** command.

Feedback

The *Create Bus* form displays, as shown in the following figure:

Figure 163 Create Bus Form



4. Enter *CBUS* in the **Bus Name** field.

   CBUS is the new name for the created bus.

5. Click the **Reverse** button.

   CLOCK 4 becomes the MSB.

6. Click **OK**.

   The created bus *CBUS[3:0]* is added to the waveform at the signal cursor position.

Expand or Collapse the Bus

To expand a bus to its individual members, double-click the bus in the signal pane. You can also choose the **Signal -> Expand Bus** command.

1. Double-click *CBUS*. Signals *CLOCK1, CLOCK2, CLOCK3,* and *CLOCK4* are displayed after *CBUS[3:0]*.

2. Double-click *CBUS* again to collapse the bus.

Save and Restore Signals

Save the Displayed Signals

nWave saves all the displayed signals and their signal attributes such as color, height, and other information to a save signal file. You can easily restore the same waveforms later by restoring this file.

1. Choose the **File -> Save Signal** command.

   The *Save Signal* form displays.

2. Click **Options**, and decide which attributes to save.

3. Enter *demo.rc* as the file name and click **OK**. By default, the file name is saved as *signal.rc*.

Restore Previously Saved Signals

1. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

2. Start the Verdi platform and *nWave* again.

3. Choose the **File -> Restore Signal** command and select *demo.rc* in the file name list.

4. Click **OK**.

   nWave displays the signals you saved in the last session.

Create a Second Waveform Window and Restore Other Signals

1. Choose the **Tools -> New Waveform** command to create a name *nWave* window. Click the **Undock** icon to make the new *nWave* window as a standalone *nWave* window.

2. Open the <*working_dir*>*/demo/verdi_mixed/CPUsystem.fsdb* file that contains the results from a different RTL simulation.

3. Restore the file *demo.rc*.

   Note that the signal arrangement used for the first *nWave* window is now shown in the second *nWave* window.

   If no file is currently open, the **File -> Restore Signal** command opens the simulation result file specified in the *demo.rc* file and restores the signals.

Feedback

If a file is open, the **File -> Restore Signal** command ignores the file specified in *demo.rc* and restores the signals in the open file.

4. After you have two waveforms open, you can choose the **Window -> Change to Primary** command to change the primary waveform window. This command selects which FSDB file is used for active annotation.

   The primary *nWave* window is identified with a red square in the lower-right corner.

   Note:
   If you load multiple FSDB files in the same *nWave* window, you can specify the active file by choosing the **File -> Set Active** command. This opens the *Active File* form, where you can select the required file and turn on the **Apply to Active Annotation** option.

To understand the steps required to save and restore the signals and their configurations, click the following video link:

In nWave, you can save and restore the same set of signals and their configurations in nWave, making it easy to reuse them across different instances or sessions. This video takes you through the steps required to save and restore the signals and their configurations.

Group or Ungroup Multiple FSDB Files

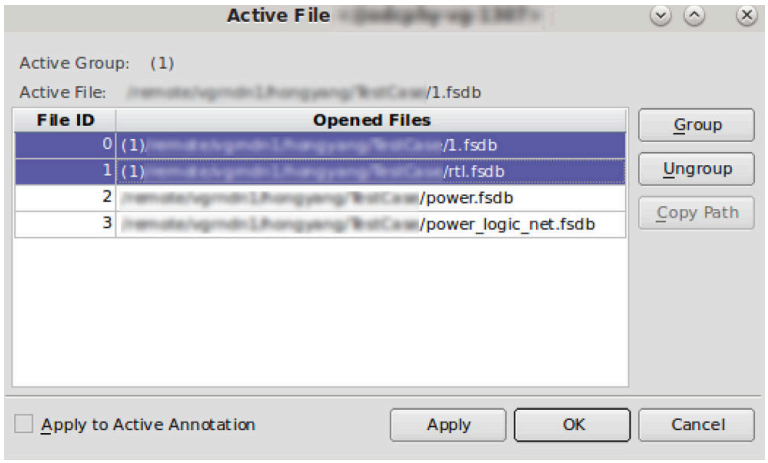
Verdi provides the functionality to group multiple FSDB files and get the signals from the group that is active.

1. Load multiple FSDB files to the Waveform.
2. Choose **File > Set Active**.

   This opens the **Active File** window.

Feedback

Figure 164 Active File Window

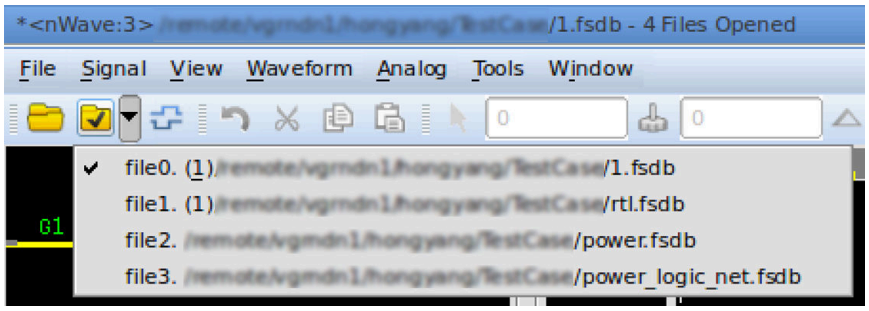


3. Select multiple FSDB files and click **Group**.

   The selected files are grouped and an ID is prepended to the file name under **Opened Files**.

   If a file is grouped to a group, its group ID is prepended to the file name in **Set Active** menu.

Figure 165 Set Active Menu



If selected files belong to different groups, Verdi pops up a dialog box, which says "Ungroup grouped files first".

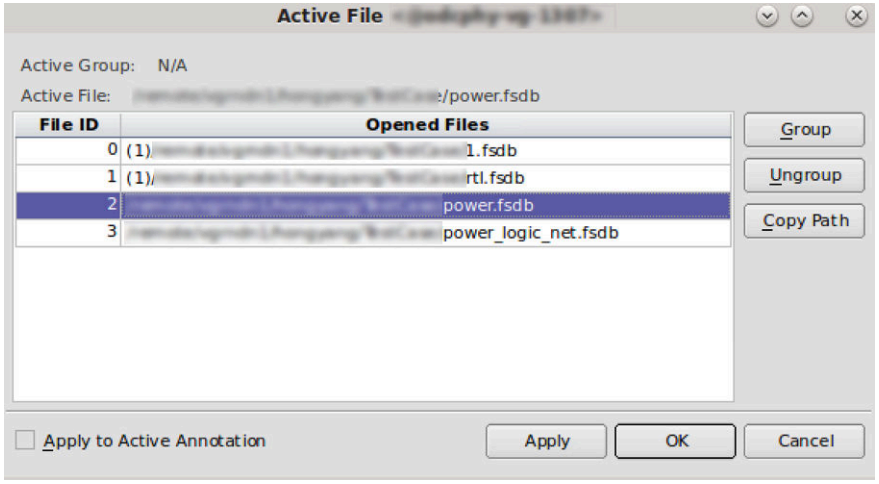
4. Click **Ungroup** to ungroup all the selected files.

   Note that you cannot use the Copy Path option if multiple files are selected.

   If the active file is not grouped, there is no active group and Verdi add only the signals from the active file. The **Active Group** field displays *N/A*.

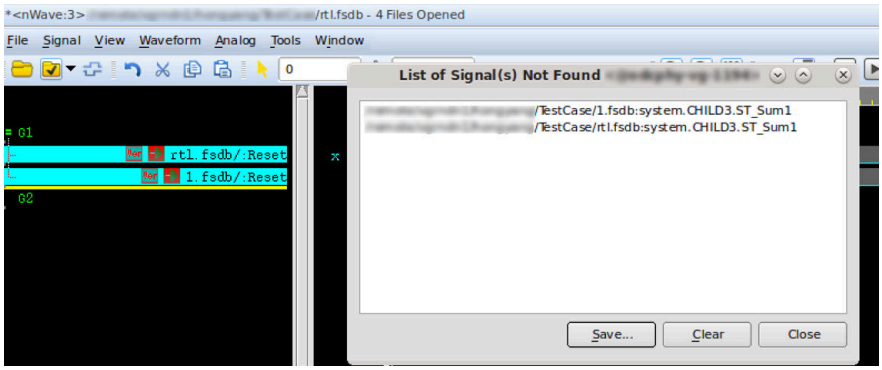
Feedback

Figure 166 No Active Files Selected



In the sample figure above, if the `rtl.fsdb` file is set to active, the active group would be group (1). The signals in `1.fsdb` and `rtl.fsdb` gets added. You can view the list of files that cannot find the signal in the **List of Signal(s) Not Found** dialog box.

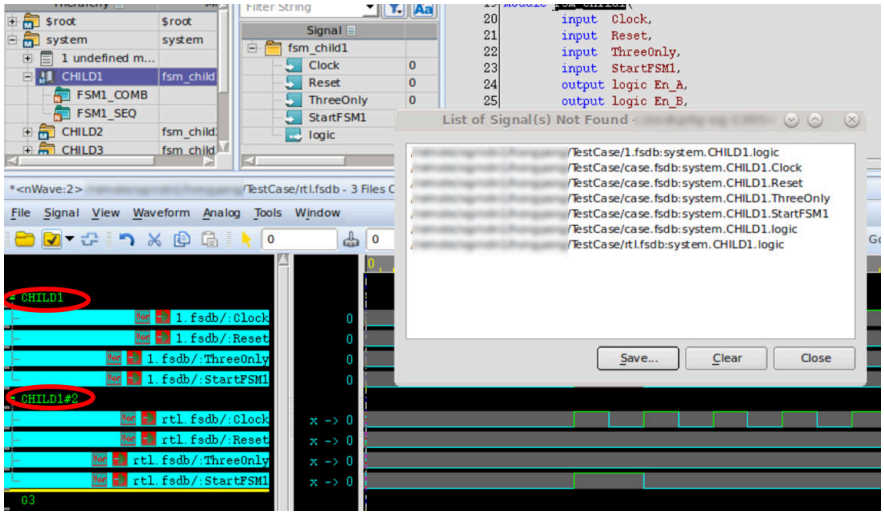
Figure 167 List of Signal(s) Not Found Window



Feedback

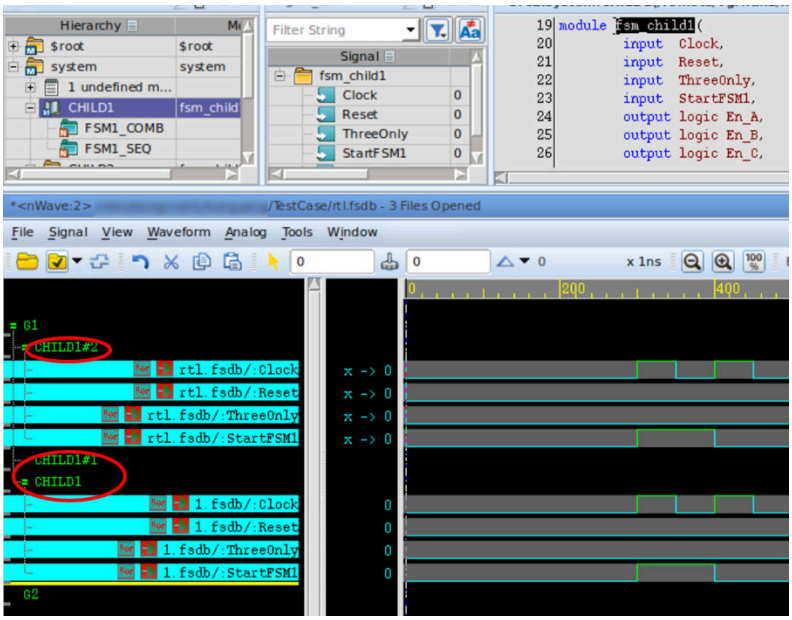
If you drop the scope to the tail empty group, the corresponding scope in files of active group is added according to order of opened files. For example, Files' order in active group: `1.fsdb > case.fsdb > rtl.fsdb`.

Figure 168 Files in the Order of Opened Files



If you drop the scope to other positions, the corresponding scope in files of active group is added according to reverse order of the opened files.

Figure 169 Files in the Reverse Order of Opened Files



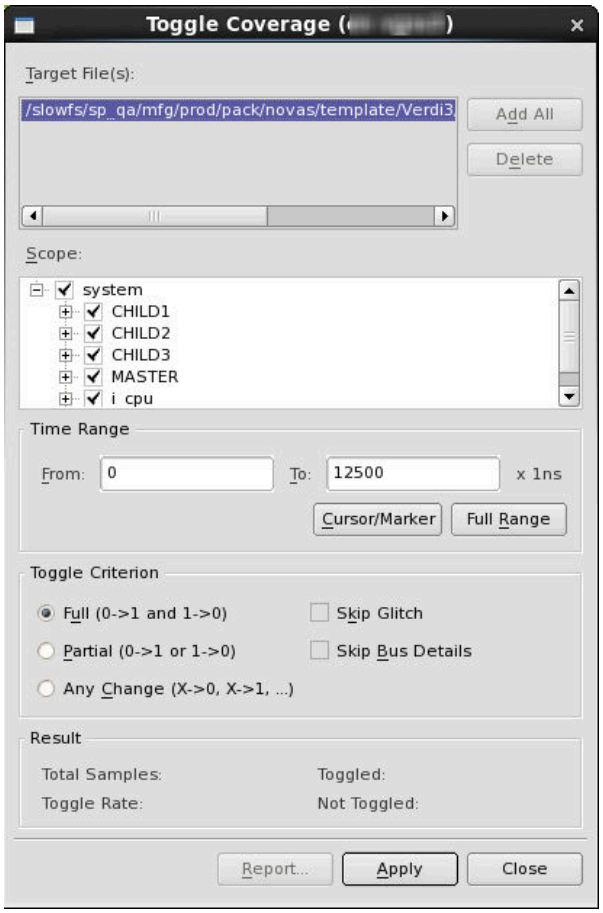
Feedback

Calculate Toggle Coverage

Sometimes, it is not easy to determine if the test patterns toggle all the signals in the design. The **Toggle Coverage Report** command analyzes all signals in the design for transitions using a post-simulation FSDB file.

1. After the FSDB file is loaded, choose the **Tools -> Toggle Coverage Report** command in the *nWave* window to open the *Toggle Coverage* form as shown in the following figure.

Figure 170 Toggle Coverage Form



The *Toggle Coverage* form provides several options to direct the coverage analysis.

2. In the **Scope** section, uncheck the CHILD1, CHILD2, CHILD3, and MASTER scopes.
3. Click **Full Range** to add the entire FSDB time range.

Feedback

4. Confirm **Full** is selected in the **Toggle Criterion** section. That is, the signals that go from 0 to 1 and then 1 to 0 or signals that go from 1 to 0 and then 0 to 1 are identified as one toggle.
5. Turn *on* the **Skip Glitch** option so glitches are not included in the toggle counts.
6. Click **Apply** to start the toggle coverage analysis. During the analysis process an *Information* dialog window is opened.
7. Click **OK** on the *Information* dialog window.

   When the toggle coverage analysis is complete, the results are displayed in the **Result** section of *Toggle Coverage* form.

Figure 171 Result Section of Toggle Coverage Form

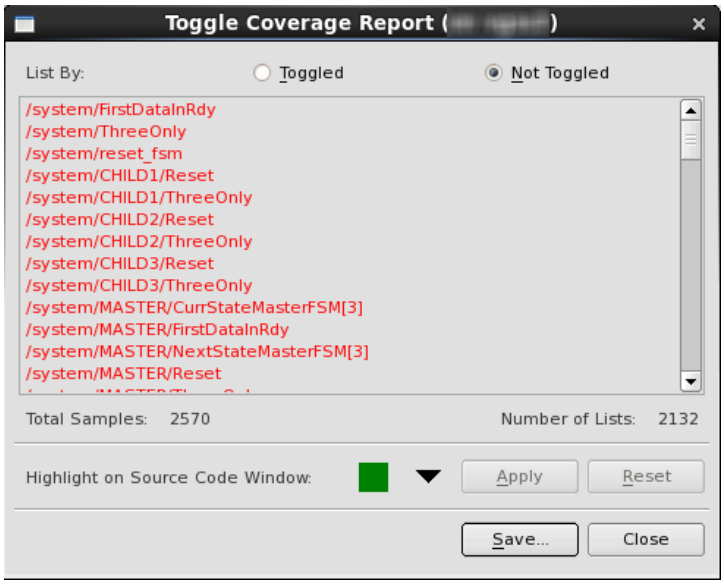
The description of each item in the **Result** section is listed here:

- **Total Samples**: Total number of analyzed signals.
- **Toggled**: Signals that are toggled.
- **Not Toggled**: Signals that are not toggled.
- **Toggle Rate**: The percentage of toggled signals divided by the analyzed signals (that is, **Toggled** / **Total Samples**).

8. In the *Toggle Coverage* form, click **Report** to open the *Toggle Coverage Report* form. The default displays the **Not Toggled** signal list.

Feedback

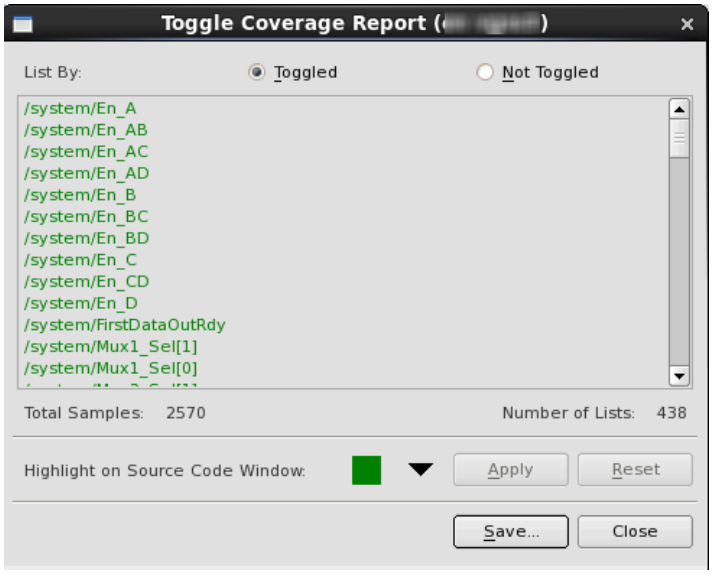
Figure 172 Not Toggled Results



The untoggled signals are listed in red. The total number of signals analyzed and the current list are summarized in the bottom of the form.

9. Select the **Toggled** option to display the toggled signal list. The toggled signals are listed in green.

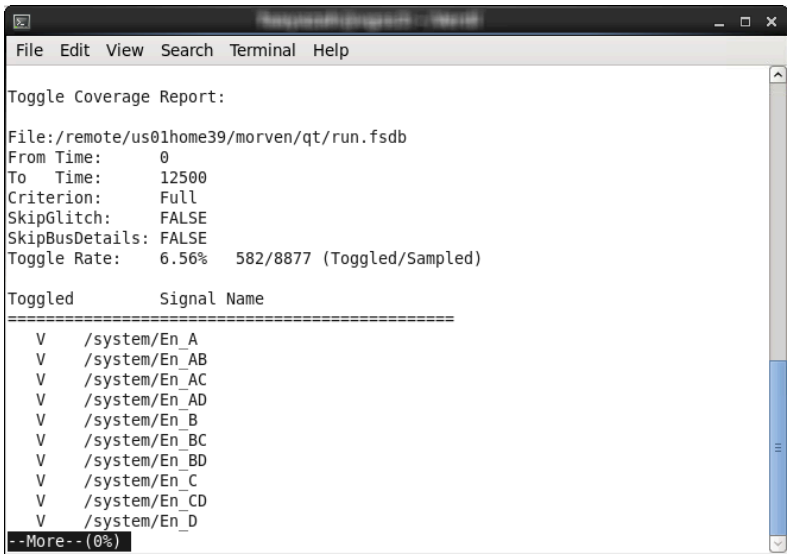
Figure 173 Toggled Results



Feedback

10. In the *Toggle Coverage Report* form, click **Save** to open the *Save Report To* form and specify a file name for the saved report. The information in the saved text file includes all the settings and information related to the current toggle coverage analysis.

Figure 174 Toggle Coverage Text Summary



Define Events and Complex Events

nWave allows you to create events to help figure out complex conditions. Through previous events, you can then capture the designated conditions clearly.

This section consists of the following topics:

- Create a Single Event
- Save and Reload Events
- Create a Complex Event
- Create a Complex Event With a Timer
- Create Complex Event with a Counter

Create a Single Event

Simple events can be created through a fixed combination of signals. For example, to capture a synchronous read and write cycle, you can specify the correct signals and values. Simple events can then be used to form a complex event (for example, limit the duration, the occurred sequence, or the number of times the event must be issued).

Feedback

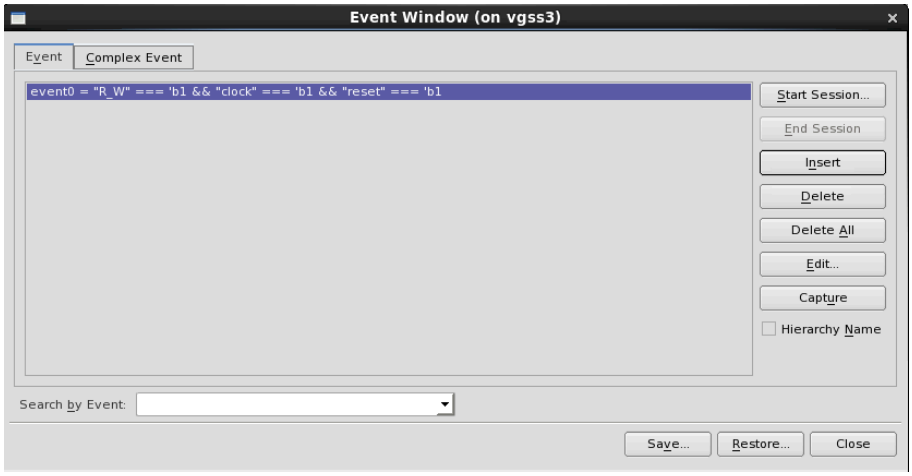
1. Close the previous Verdi session.
2. Change directories to the *<working_dir>/demo/verilog/rtl* directory and then invoke the Verdi platform as follows.

```
% cd <working_dir>/demo/verilog/rtl
% verdi -f run.f -ssf rtl.fsdb -workMode hardwareDebug &
```

3. In the *nWave* window, choose the **Signal -> Get Signals** command to open the *Get Signals* form.
4. In the *Get Signals* form, select the R_W, clock, and reset signals under the system/i_cpu scope and click the **OK** button.
5. Choose the **Waveform -> Go To -> Time** command to open the *Search Time* form.
6. Enter *400* in the **Time Value** field and click the **OK** button.

   The cursor changes to time 400 in the waveform pane.
7. With the three signals selected, choose the **Signal -> Event** command to open the *Event Window*.
8. In the *Event Window*, click **Insert** to create *event 0*.

Figure 175 nWave Event Window



An event is inserted using the selected signals current values as the event conditions.

9. In the *Event Window*, click **Edit**.
10. In the *Edit Event* form, change the name to *read_cycle* in the **Event Name** field. Do not change the expression which should be:

```
"clock"=== `b1 && "reset" === `b1 && "R_W" === `b1
```

Feedback

11. Click **OK** and the *read_cycle* event is added into *Event Window*.
12. Click **Capture** to view the related waveform. Leave the *Event Window* open.

Save and Reload Events

1. In the *Event Window*, click **Save** and save the event to a file named read_cycle.rc
2. Exit the Verdi session.
3. Start the Verdi platform again with:

```
% verdi -f run.f -ssf rtl.fsdb -workMode hardwareDebug &
```

4. In the *nWave* window, choose the **Signal -> Event** command.
5. In the *Event Window*, click the **Restore** button and load the read_cycle.rc file. The *read_cycle* event is listed in **Event** tab.
6. Select the *read_cycle* event and click **Capture**, you find that the *read_cycle* event is added to the *nWave* window.

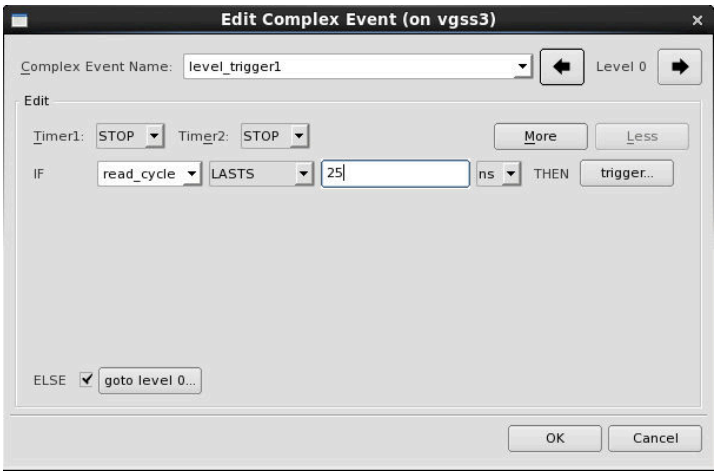
Create a Complex Event

1. In the *Event Window*, click the **Complex Event** tab to add complex events.
2. On the **Complex Event** tab, click **Edit**. The *Edit Complex Event* form opens.
3. In the *Edit Complex Event* form stay at level 0 and perform the following:
   a. In the **Complex Event Name** field, enter *level_trigger1*.
   b. Change the condition to **IF read_cycle LASTS** (click **OCCURS** to change the action) **25ns THEN trigger**.

Feedback

After the changes, the form is similar the following figure:

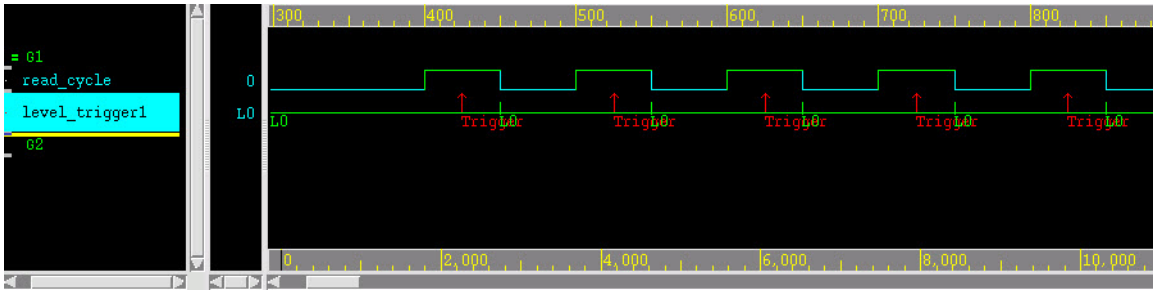
Figure 176 nWave Edit Complex Event Form



c. Click **OK** to create this complex event and close the *Edit Complex Event* form.

4. In the *Event Window*, click **Capture** to capture the event. The event is added to the *nWave* window with signal name *level_trigger1* as shown in the following figure:

Figure 177 level_trigger1 Captured in nWave



Create a Complex Event With a Timer

Two built-in timers are provided that can be used to compose the complex event. Timers are used to trace the period of a complex event. They can be controlled with different options: START, STOP, PAUSE, and CONT. The timer is globally set. If no value is set for the timer in the sub-conditions, then the value is inherited from the global setting. The sub-condition can have its own setting with a priority higher than the global ones.

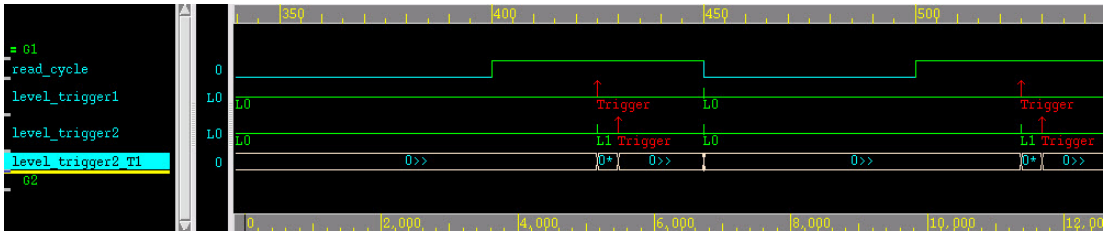
Feedback

Example 1

1. In the *Event Window*, **Complex Event** tab, click **Edit** to open the *Edit Complex Event* form.
2. In the *Edit Complex Event* form, stay at level 0 and perform the following:
   a. In the **Complex Event Name** field, enter *level_trigger2*.
   b. Change the condition to **IF read_cycle LASTS 25 ns THEN goto level 1** (click **trigger** to change the action).
   c. Leave the remaining fields unchanged.
   d. Increase the level number from **Level 0** to **Level 1** by clicking the right-arrow.
3. In the *Edit Complex Event* window, stay at level 1 and perform the following:
   a. Change **Timer1** to **START**
   b. Change the condition to **IF timer1 == 5 ns THEN trigger**
   c. Leave the remaining fields unchanged.
4. Click **OK** to create this complex event and close the *Edit Complex Event* form.
5. In the *Event Window*, turn *on* the **Also Capture Timer and Counter** option and click the **Capture** button to capture the event.

   The event is added to the *nWave* window with signal name *level_trigger2*. When the state jumps from level0 to level1, timer1 is started, and then after 5 ns, the trigger occurs as shown as in the following figure:

Figure 178 level_trigger2 Captured in nWave



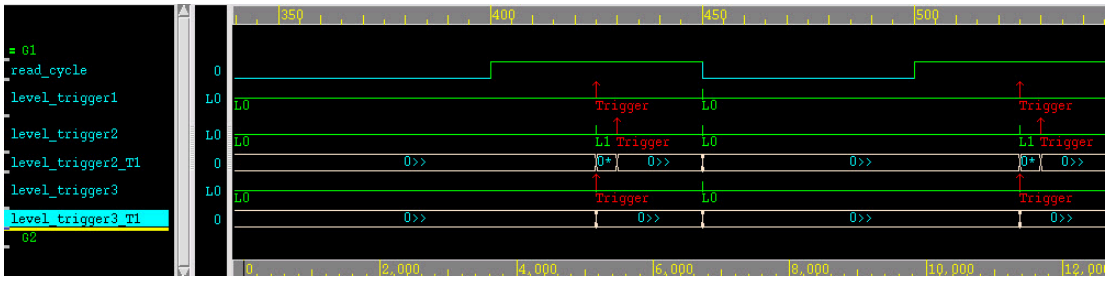
Feedback

Example 2

1. In the *Event Window*, **Complex Event** tab, click **Edit** to open the *Edit Complex Event* form.
2. In the *Edit Complex Event* form, stay at level 0 and perform the following:
   a. In the **Complex Event Name** field, enter *level_trigger3.*
   b. Change **Timer1** to **START**.
   c. Change the condition to **IF read_cycle LASTS 25 ns THEN goto level 1** (click **trigger** to change the action).
   d. Leave the remaining fields unchanged.
   e. Increase the level number from **Level 0** to **Level 1** by clicking the right arrow.
3. In the *Edit Complex Event* window, stay at level 1 and perform the following:
   a. Change **Timer1** to **STOP**.
   b. Change the condition to **IF timer1 > 5 ns THEN trigger**.
   c. Leave the remaining fields unchanged.
4. Click **OK** to create this complex event and close the *Edit Complex Event* form.
5. In the *Event Window*, turn *on* the **Also Capture Timer and Counter** option and click **Capture** to capture the event.

   The event is added to the *nWave* window with signal name *level_trigger3*. When the state jumped to level1, timer1 is stopped, but the “IF” condition is checked at the same time. At this time point, timer1 is still larger than 5, so the trigger occurs as shown in the following figure:

Figure 179 level_trigger3 Captured in nWave



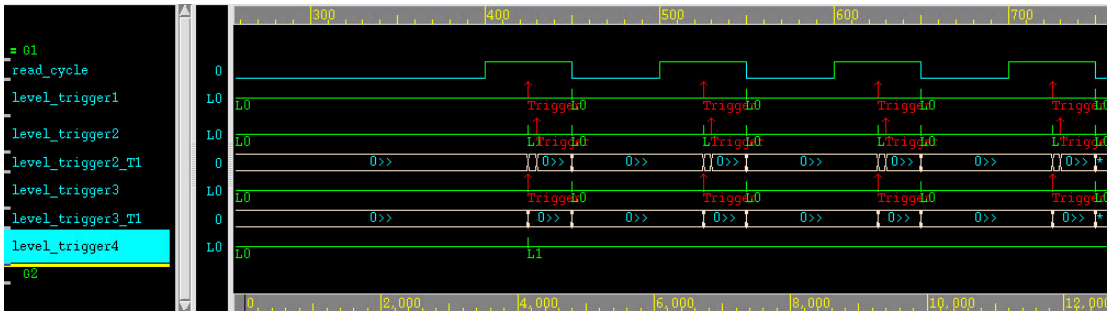
Feedback

Example 3

1. In the *Event Window*, **Complex Event** tab, click **Edit** to open the *Edit Complex Event* form.
2. In the *Edit Complex Event* form, stay at level 0 and perform the following:
   a. In the **Complex Event Name** field, enter *level_trigger4*.
   b. Change **Timer1** to **STOP**.
   c. Change the condition to **IF read_cycle LASTS 25 ns THEN goto level 1** (click the trigger button to change the action).
   d. Leave the remaining fields unchanged.
   e. Increase the level number from **Level 0** to **Level 1** by clicking the right arrow.
3. In the *Edit Complex Event* window, stay at level 1 and perform the following:
   a. Change **Timer1** to **STOP**.
   b. Change the condition to **IF timer1 > 5 ns THEN trigger**.
   c. Leave the remaining fields unchanged.
4. Click **OK** to create this complex event and close the *Edit Complex Event* form.
5. In the *Event Window*, turn *on* the **Also Capture Timer and Counter** option and click **Capture** to capture the event.

   The event is added to the *nWave* window with signal name *level_trigger4*. The timer is not started. When the state jumped to level1, it checks the “ELSE” condition and stays in level1 as shown in the following figure:

Figure 180 level_trigger4 Captured in nWave



6. Exit the Verdi session.

Feedback

Create Complex Event with a Counter

Two built-in counters are provided that are used for counting if some condition occurs or not. Counters can be controlled with different options: INC, DEC, or RESET. The counters are also set globally and they do not stop counting when the level changes unless you reset them.

1. Start the Verdi platform again with:

```
% verdi -f run.f -ssf rtl.fsdb -workMode hardwareDebug
```

2. In the *nWave* window, choose the **Signal -> Event** command.
3. In the *Event Window*, click **Restore** and load the read_cycle.rc file. The *read_cycle* event is listed in **Event** tab.
4. Select the *read_cycle* event and click **Capture**, you find the *read_cycle* event is added to the *nWave* window.

Example 1

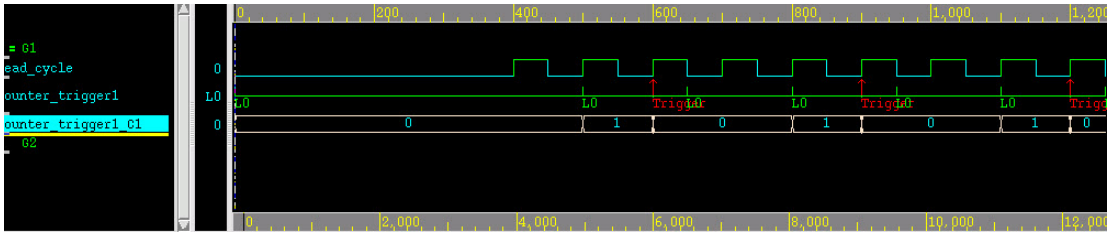
1. In the *Event Window*, **Complex Event** tab, click **Edit** to open the *Edit Complex Event* form.
2. In the *Edit Complex Event* window stay at level 0 and perform the following:
   a. In the **Complex Event Name** field, enter *counter_trigger1*.
   b. Change the condition to **IF read_cycle OCCURS 2 Times THEN goto level 0**. When you click the **trigger** button to change the value to “goto level 0”, the *Actions* form opens.
   c. In the *Actions* form, turn on **Counter1** and choose **INC**, then click **Apply**.
   d. Click **More**.
   e. Change the ELSE IF condition to **ELSE IF counter1 == 2 Times THEN trigger**.
   f. Click **trigger** to open the *Actions* form.
   g. In the *Actions* form, turn on **Counter1** and choose **RESET**, then click **Apply**.
   h. Leave the remaining fields unchanged.
3. Click **OK** to create this complex event and close the *Edit Complex Event* form.

Feedback

4. In the *Event Window*, turn on the **Also Capture Timer and Counter** option, and then click **Capture** to capture the event.

   The event is added to the *nWave* window with signal name *counter_trigger1*. The system is operating in real time, not sequential, so the transition of *read_cycle* is checked twice when the state jumped to level0 as shown in the following figure:

Figure 181 counter_trigger1 Captured in nWave

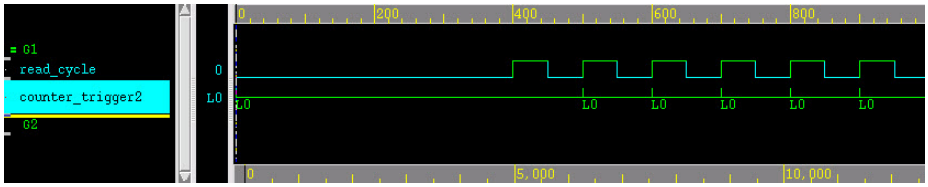


Example 2

1. In the *Event Window*, **Complex Event** tab, click **Edit** to open the *Edit Complex Event* form.
2. In the *Edit Complex Event* window stay at level 0 and perform the following:
   a. Click **New** to clear the previous event description.
   b. In the **Complex Event Name** field, enter *counter_trigger2.*
   c. Change the condition to **IF read_cycle OCCURS 2 Times THEN goto level 0**.
   d. Click **More**.
   e. Change the **ELSE IF condition to ELSE IF counter1 == 2 Times THEN trigger**.
   f. Leave the remaining fields unchanged.
3. Click **OK** to create this complex event and close the *Edit Complex Event* form.
4. In the *Event Window*, click **Capture** to capture the event.

   The event is added to the *nWave* window with signal name *counter_trigger2*. As counter1 is not increased in the condition; it does not reach 2, so no trigger occurs as shown in the following figure:

Figure 182 counter_trigger2 Captured in nWave



Feedback

Support VHDL Fixed-point Types in nWave

Verdi supports to display a bit-vector in signed/unsigned fixed-point representation in nWave. The following features are explained in this section:

- Shows a fixed-point value automatically if the signal follows VHDL fixed-point standard or Verilog vector signal.
- Allows to set radix (Binary/Octal/Decimal/Hexadecimal) for single-bit VHDL fixed-point signal.
- Supports new FSDB utility for VHDL fixed-point type.
- Supports fixed-point for a Verilog vector signal.

Display Fixed-point Value

Verdi shows a fixed-point value automatically if the signal follow VHDL fixed-point standard.

The fixed point signal can be classified by its range into the following two types that have different display specifications:

- Signal range that contain binary point; that is, MSB ≥ -1 and LSB ≤ -1

  For example,

  ```
  sig1[4:-3] -> 11111.111
  sig2[-1:-3] -> .111
  ```

- Signal range that does not contain binary point; that is, MSB < *-1* or LSB > *-1*

  For example,

  ```
  sig3[-2:-4] -> 111 represent for 0.0111
  sig4[3:1] -> 111 represent for 1110
  ```

Convert Radix

The following table shows the results of the raw values converted to different radix:

	raw value	Binary	Octal	Hexadecimal	Decimal
sig1 ufixed(6 downto -7)	11_0000_1000_1110	1100001.0001110	141.070	61.1C	97.109375

Feedback

sig2 ufixed(-1 downto -5)	1_1111	.11111	.76	.F8	0.96875
sig3 ufixed(-2 downto -6)	1_1111	11111	37	1F	0.484375
sig4 ufixed(3 downto 1)	111	111	7	7	14

The binary is displayed at the left of index `-1`. For changing the radix, take `sig1`, change to hex as in the following example:

For the integer part, group the digits from right to left: `110_0001 -> 61`

For the fractional part, group the digits from left to right and add zeros for the remaining part: `0001_1100 -> 1C`

The display binary is not displayed; instead, raw value is directly displayed. For changing radix, take `sig3` change to hex, as in the following example:

Group the digits from right to left: `1_1111 -> 1F`

Leading Zero

When you turn off the leading zero, the integer part leading zero gets removed and the fractional part retains all its digits.

For example,

- `0011.0101 -> 11.0101`: Removes first 2 zeros' in the integer part.
- `0000.0110 -> 0.0110`: The integer part removes its first 3 zeros and preserves 1 zero to represents its integer value.
- `ufixed` (`-3 downto -10`): `00000110` (value equal to `0.0000000110`) -> `00000110`, since its value represents the fractional part value, all digits are preserved.
- `ufixed` (`10 downto 3`): `00000110` (value equal to `0000011000`) -> `110`, since its value represents the integer part value, the first 5 zeros are removed.

Convert Notation

Follow the current Verilog bit-vector behavior:

If the value is positive, all notations are same as unsigned notation. If it is negative, you may use a signed symbol "-" and an unsigned value to represent 1's complement or 2's complement or signed magnitude.

Feedback

	Raw value	Unsigned (binary)	2's (decimal)	2's (binary)	1's (decimal)	1's (binary)	signed (decimal)	signed (binary)
reg sig[3:0] (in Verilog)	1110	1110	-2	-0010	-1	-0001	-6	-0110
sfixed (5 downto 2)	1110	1110	-8	-0010	-4	-0001	-24	-0110
sfixed (1 downto -2)	1110	11.10	-0.5	-00.10	-0.25	-00.01	-1.5	-01.10
sfixed (0 downto -3)	1110	1.110	-0.25	-0.010	-0.125	-0.001	-0.75	-0.110
sfixed (-1 downto -4)	1110	1110	-0.125	-.0010	-0.0625	-.0001	-0.375	-.0110
sfixed (-2 downto -5)	1110	1110	-0.0625	-0010	-0.03125	-0001	-0.1875	-0110

Enable Set Radix Entry for Single-bit VHDL Fixed Point Signal

For VHDL fixed point support, it is valid to declare a single signal as `sfixed`. In such a case, you may want to check the decimal value for this fixed point signal.

nWave enables "Set Radix" entry and "Set Notation" for such single-bit signal, the available radix is Binary/Decimal.

Example: `signal ft_var1: sfixed (3 downto 3);`

It is meaningful to convert the format to decimal.

For example, raw value = `1` -> represent `8` (decimal), after applying "Decimal" radix:

FSDB Utility Support for VHDL Fixed-point Type

The following tables shows the behavior on how the FSDB utilities handle VHDL fixed-point type signals.

FSDB Utility	Behavior
`fsdbextract`	The fixed-point type information is lost in the output FSDB.

Feedback

FSDB Utility	Behavior
`waveutils --tool=extract`	The fixed-point type information is lost in the output FSDB.
`fsdbedit`	The fixed-point type information is lost in the output FSDB.
`fsdbmerge`	The fixed-point type information is lost in the output FSDB.
`parallelWaveExpand`	The fixed-point type information is retained in the output FSDB.
`fsdbeport`	The `fsdbreport` utility only shows the raw values of the fixed-point type signals.
`waveutils --tool=report`	The `waveutils --tool=report` utility only shows the raw values of the fixed-point type signals.
`fsdb2vcd`	The `fsdb2vcd` utility does not consider the fixed-point type information.
`fsdbsaif`	The `fsdbsaif` utility does not consider the fixed-point type information.

Note:
All types of virtual files are supported for the VHDL fixed-point type.

Limitations

The following FSDB utilities do not support fixed-point type. The fixed-point type FSDB can be used; however, the fixed-point type information is lost:

- `fsdbextract`
- `waveutils --tool=extract`
- `fsdbedit`
- `fsdbmerge`

The following FSDB utilities do not support fixed-point type. The fixed-point type FSDB can be used; however, only the raw values of fixed-point signals are shown:

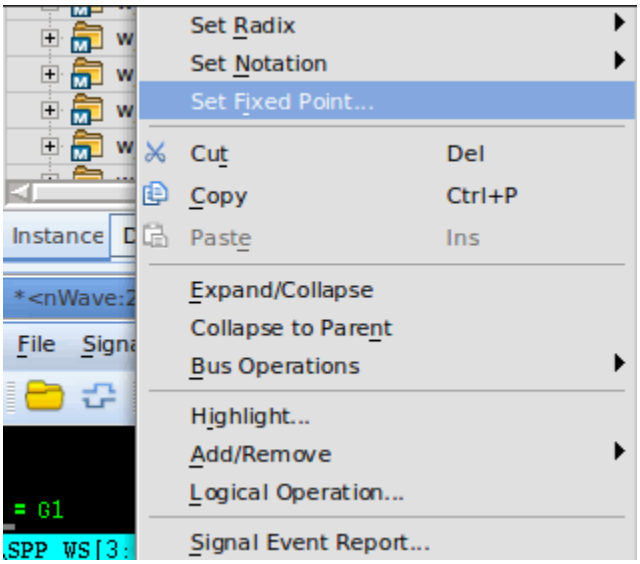
- `fsdbeport`
- `waveutils --tool=report`

Feedback

Change the Binary Point for VHDL Fixed-point Signal and Verilog Vector Signal

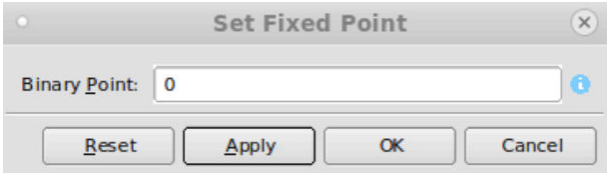
You can set the binary point for VHDL fixed point type signals and Verilog vector signals using the **Waveform -> Set Fixed Point** menu command.

Figure 183 Set Fixed Point Menu



The **Set Fixed Point** dialog box opens as follows:

Figure 184 Set Fixed Point



The **Set Fixed Point** field is available to select on the signal pane if the radix of the selected node includes Binary/Octal/Decimal/Hexadecimal.

The **Set Fixed Point** field is displayed on the value pane only if the selected node includes digital signal and is available to select if the radix of the selected node includes Binary/ Octal/Decimal/Hexadecimal.

The Binary Point field accepts only integers ranging from -512 to 512.

Feedback

If **-512 <= input < 0** or **signal size < input <= 512** (that is, the input is out of range of the bit numbers), the following applies:

- Turn padding zeros preference on: Show zero padding
- Turn padding zeros preference off: Show original value string

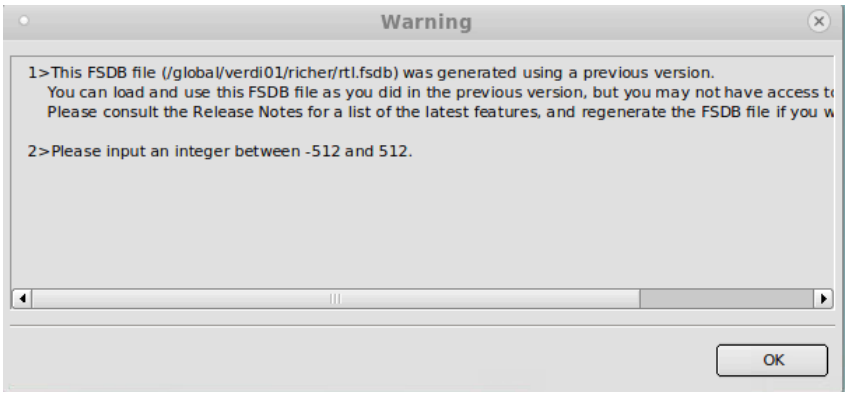
You can enable/disable the **Display Padding Zeros When Binary Point is Out Of Range** under **Waveform -> View Options -> Value Pane**

Consider the following example,

bus[3:0] = 111, radix is Binary

bus[3:0]=111	Set binary point to 4	Set binary point to -1
Padding zero preference on	.**0**111	111**0**
Padding zero preference off	111	111

A warning message is displayed if the input value is greater than 512.



Examples:

VHDL fix point signal:

```
sig[3:0] = 1110 => apply binary point = 2 (11.10) reset => 1110
sig[2:-1] = 111.0 => apply binary point = 2 (11.10) reset => 111.0
```

Verilog signal:

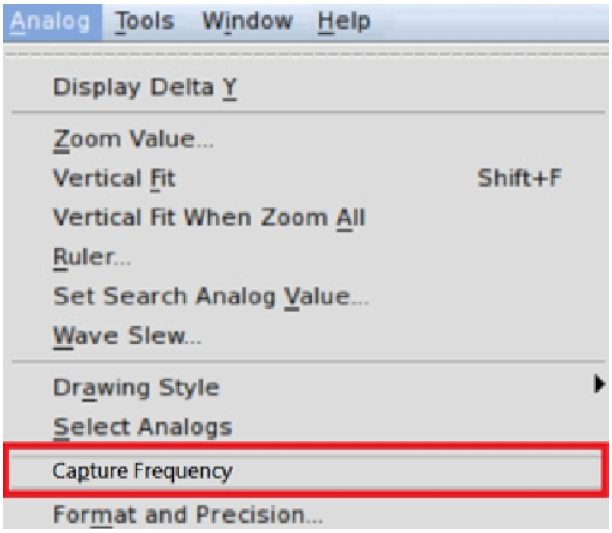
```
sig[3:0] = 1110 => apply binary point = 2 (11.10) reset => 1110
sig[2:-1] = 1110 => apply binary point = 2 (11.10) reset => 1110
```

Feedback

Capture and Display Frequency of a Scalar Signal as a New Signal

Verdi allows you to capture and view the frequency of a scalar signal as a new signal in the Wave view. You must select the **Analog -> Capture Frequency** option as shown in the following figure to enable this feature. This feature allows you to understand exactly where the frequency of the signal change happens.

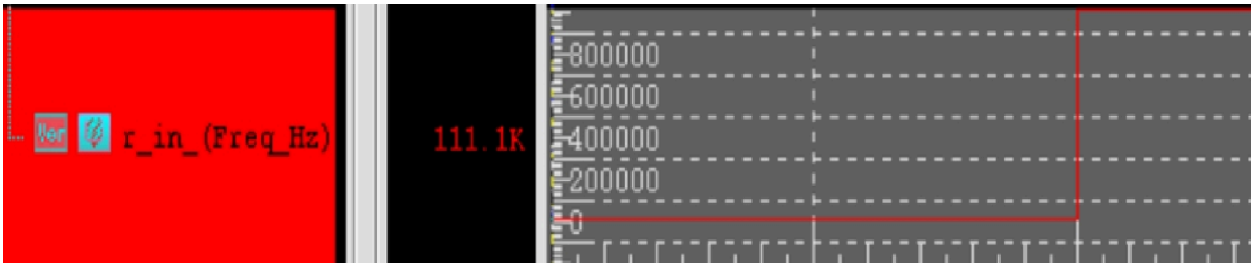
Figure 185 Capture Frequency



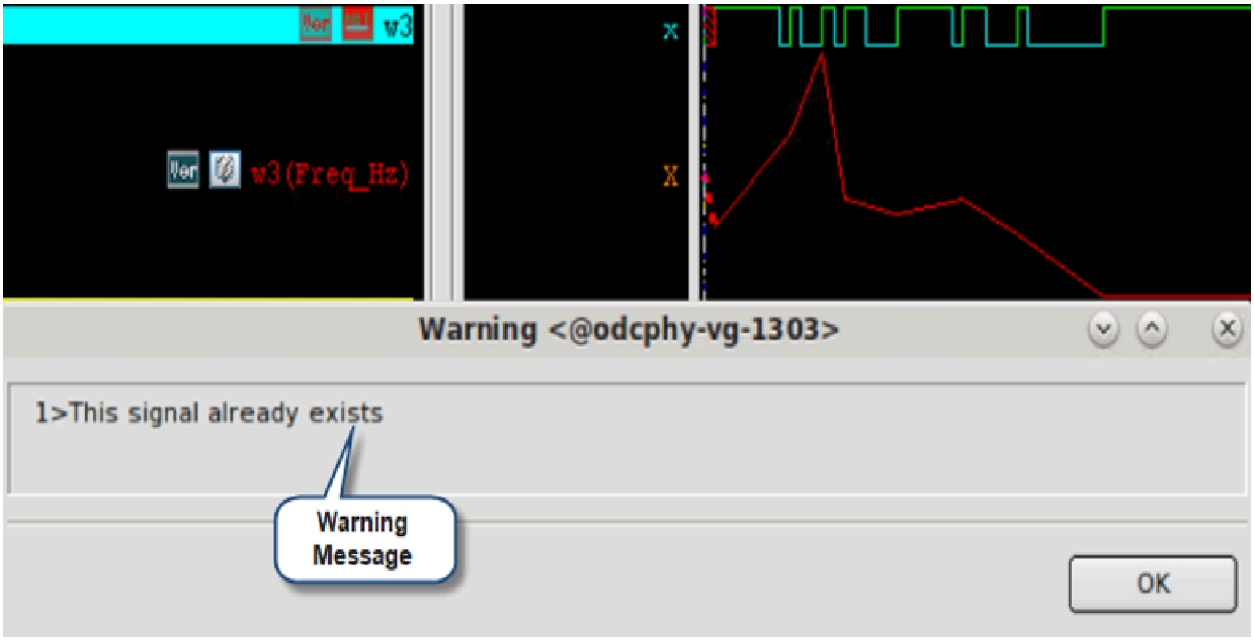
The new signal name is followed by (Freq_Hz) as the suffix, as shown in the following figure:

Feedback

After capturing the frequency of the signal, you can select **Analog -> Ruler** option to view the frequency, as shown in the following figure:



Verdi issues a warning message as shown in the following figure if you capture the same frequency signal.



Verdi uses the posedge trigger of a signal for calculating the frequency. For example, consider a signal with the following clock:

0ns 10ns 20ns 40ns 60ns 80ns …

1 0 1 0 1 0 …

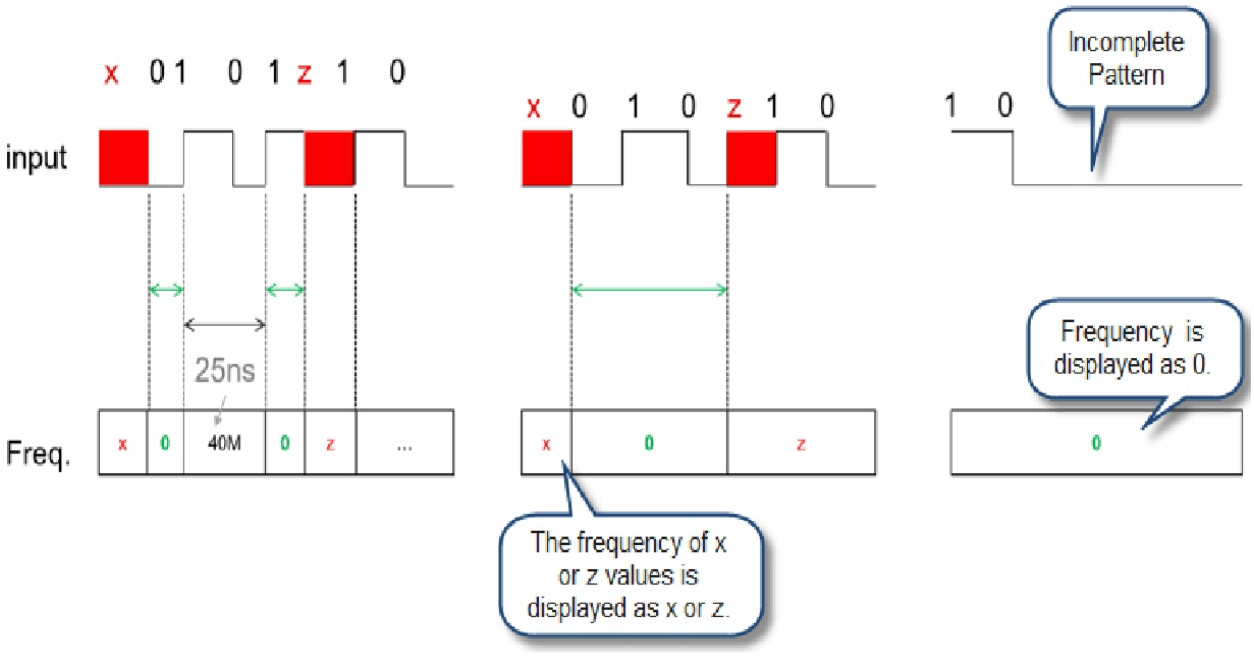
First cycle (1->0->1) is: 0~20ns, frequency is 50MHz (1/20ns)

Second cycle is: 20ns~60ns, frequency is 25MHz (1/40ns)

Therefore the value of the created analog signal is 50MHz (at time 0ns), 25MHz (at time 20ns), and so on.

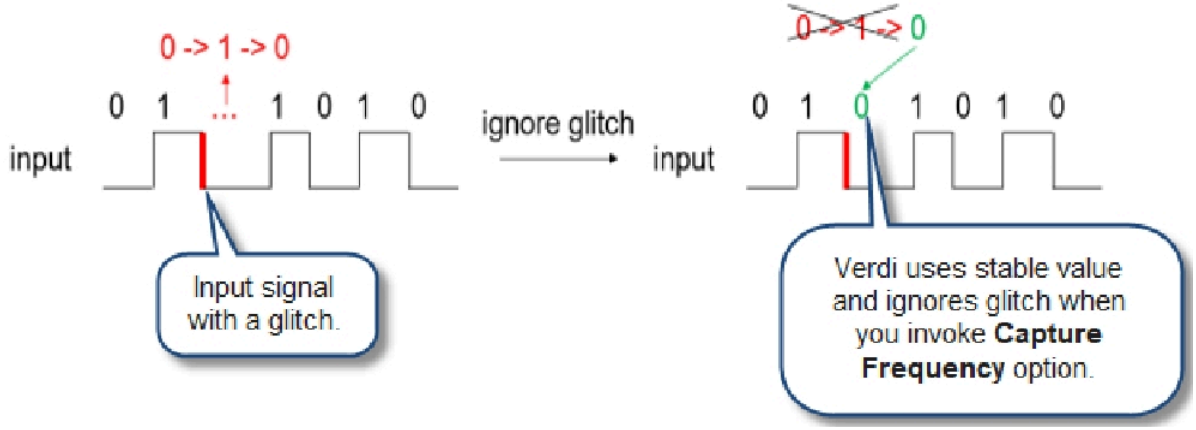
Frequency for Unknown Values in Input Signal

Verdi displays the frequency of unknown (x) or high impedance (z) values of an input signal as x or z. It displays the frequency for incomplete patterns as 0.



Using Stable Values for Glitches in the Input Signal

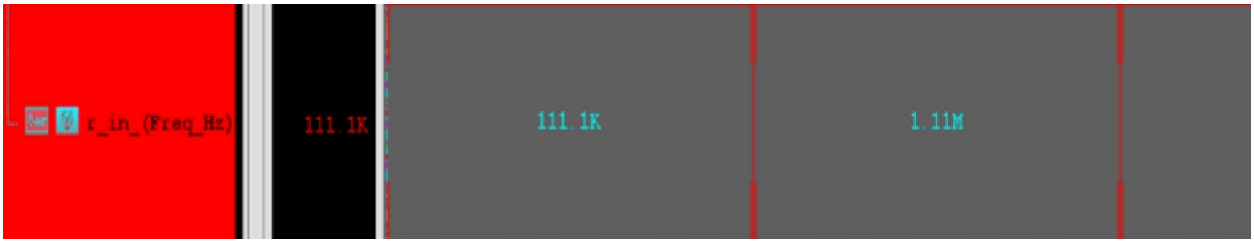
If an input signal contains a glitch, Verdi ignores it and uses a stable value when you invoke the Capture Frequency option.



Feedback

Viewing Analog Signal as Digital Vector in Waveform

You can use the Digital Waveform right-click option in the waveform view to view an analog signal as a digital vector as shown in the following figure:



Generate Dump Files From Verdi Signal RC Files

Verdi enables you to automatically convert simulation dump files (such as `.do` files) from Verdi signal RC files using a provided Perl script. By converting the signal selections saved in a Verdi .rc file into a format suitable for batch-mode simulation dumping, you can efficiently prepare signal dumps for subsequent debugging and analysis.

To enabling automatic dump file generation, perform the following steps:

1. Ensure you have a Verdi signal RC file (for example, `signal.rc`) containing the signals you want to dump.
2. Obtain the Perl script `genDumpFileBySignalRC.pl`. You get access to this script when `VERDI_HOME` is set by typing the script name.
3. Run the Perl script from the command line to create the `.do` file.

   ```
   genDumpFileBySignalRC.pl signal.rc test.do
   ```

   This command reads `signal.rc` and generates `test.do` containing the appropriate dump commands.
4. You can execute the Perl script within a simulation `.do` file to generate the dump file dynamically.

Example

`my_dumping.do`

```
Exec genDumpFileBySignalRC.pl test.rc test.do
fsdbDumpVarsByFile "$env(WORKSPACE/test.do)" test.fsdb
```

This approach ensures the dump file is always up to date with the latest signal selections.

9

nState Tutorial

This chapter describes the following topics:

- Overview
- Starting nState in nSchema Window
- Manipulate the State Diagram View
- State Animation
- State Machine Analysis

Overview

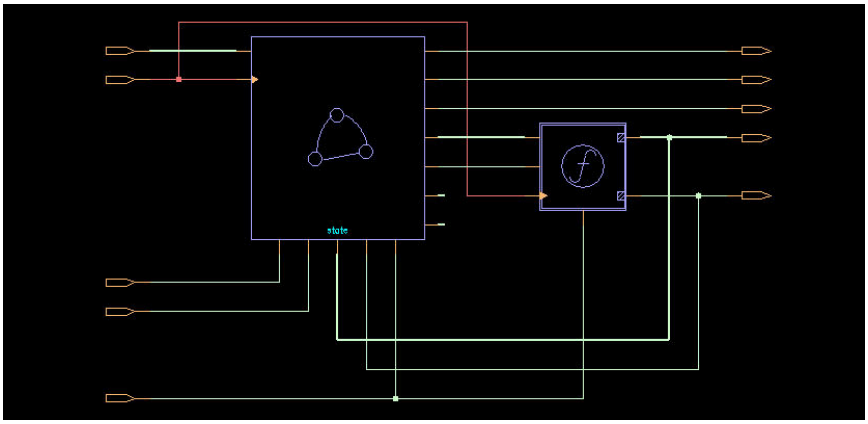
nState is a Finite State Machine (FSM) viewer and analyzer. It generates bubble diagrams for visualization of state machines. These diagrams are automatically recognized by the Verdi platform when compiling the Verilog or VHDL source code modules. States and transitions are annotated with logic conditions and animated with simulation results. *nState* analyzes the simulation results to determine the state and transition coverage.

It is recommended that you follow the instructions given in the Before You Begin chapter before starting this tutorial.

Feedback

The Verdi platform automatically recognizes FSMs and displays them in *nSchema* with a symbol having three linked circles, as shown in the following figure:

Figure 186 Example of FSM Symbol in the nSchema Window



Starting nState in nSchema Window

1. Change the directory to *<working_dir>/demo/verilog/cpu/src*, and execute the following command to import the FSM design:

   ```
   % verdi -f run.f -workMode hardwareDebug &
   ```

2. Open a schematic window by double-clicking to select *i_BJkernel* in the **Instance** tab of the design browser frame.

3. Click the **New Schematic** icon on the toolbar. The *nSchema* window is displayed, as shown in the Figure 186 figure above.

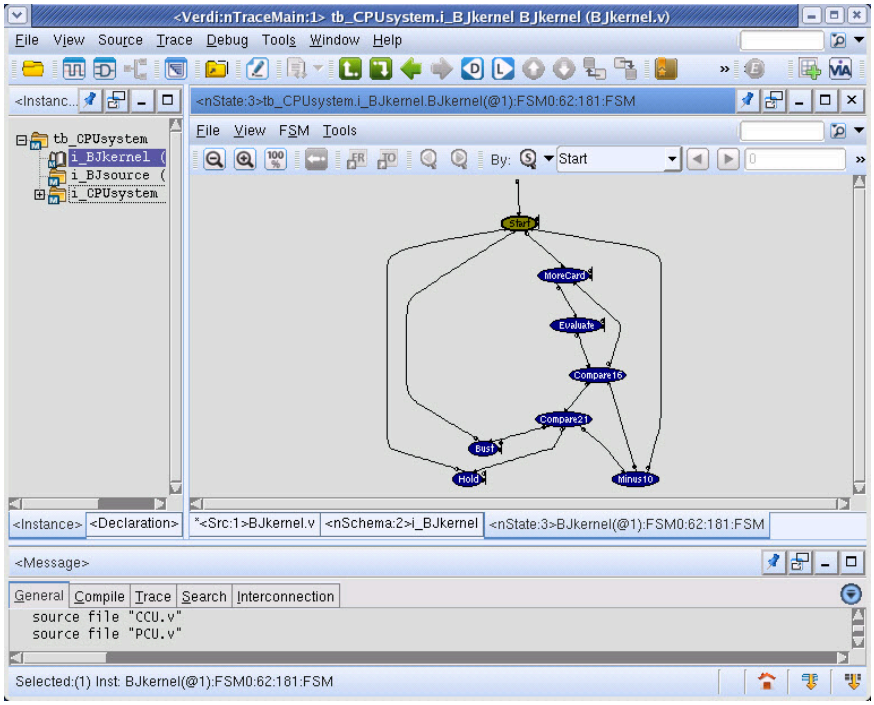
   Note:
   If the *nSchema* window does not look like the above figure, confirm the **Enable Detail RTL** option is turned on. This option is available on the **RTL**

Feedback

page under the **Schematics** folder of the *Preferences* form (invoked with the **Tools -> Preferences** command).

4. Double-click the FSM block in the *nSchema* window to view the state diagram in an *nState* frame. An *nState* frame opens as a new tab in the same area as the *nSchema* window:

Figure 187 nState Frame Displaying FSM



Manipulate the State Diagram View

You can change the view of the state diagram using the following zoom commands:

- **Zoom In** - You can view more details of the state diagram by moving the view 50% from the center point in both the horizontal and vertical directions. Invoke this command using one of the following methods:
  - Toolbar icon
  - Bindkey **Z**
  - **View -> Zoom -> Zoom In** command

- **Zoom Out** - You can view more contents of the state diagram by expanding the view 2X from the center point, both horizontally and vertically. Invoke this command using one of the following methods:
  - Toolbar icon
  - Bindkey **z**
  - **View -> Zoom -> Zoom Out** command
- **Zoom All** - You can view the entire contents of the state diagram. Invoke this command using one of the following methods:
  - Toolbar icon
  - Bindkey **f**
  - **View -> Zoom -> Zoom All** command
- **Zoom Area** - You can view a specific area of the state diagram by dragging-left to form a rectangle over the zoomed area.

  You can move the viewing area of the state diagram in different directions using the following methods:
- **Scrolling** - Click or drag the scroll bar of the *nState* window either horizontally or vertically.
- **Panning** - Move the view up, down, left, or right using the arrow keys or menus: **View -> Pan -> Pan Up**, **View -> Pan -> Pan Down**, **View -> Pan - > Pan Left**, and **View -> Pan -> Pan Right**.

  In addition, you can use the **View -> Last View** command or the bindkey "l" (lowercase L) to return to the last view.

Enable Viewing Objects

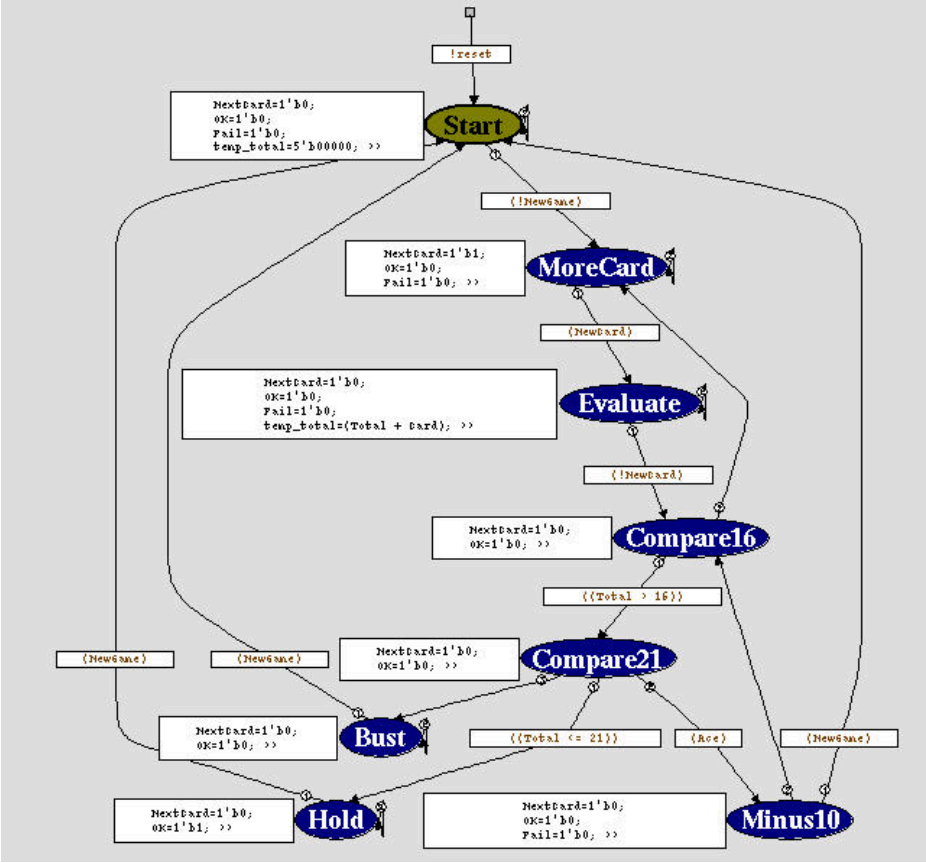
You can enable or disable viewing for different objects (for example, state actions, transition conditions, and so on) in the *nState* frame.

1. In *nState* frame, choose **Tools -> Preferences** command to open the *Preferences* form.
2. Select the **View Options** page under the **FSM** folder and turn on the **State Action**, **Transition Action**, and **Transition Condition** options.
3. Click **OK** to close this form and apply the changes.

Feedback

4. Choose **Tools -> Duplicate Window** command to open another *nState* frame and see the additional information. The *nState* frame is displayed as follows:

Figure 188 nState Frame With Viewing Objects



5. In the original *nState* frame, open the **View** menu and select the **State Action**, **Transition Action**, and **Transition Condition** commands individually. The state action and transition details are added to this view.

6. Choose **Tools -> Preferences** command to open the *Preferences* form again, select the **View Options** page under the **FSM** folder. Turn off the **State Action**, **Transition Action**, and **Transition Condition** options, and click **OK** to close the form.

   The options on the *Preferences* form affect all new *nState* frames - a global setting. The **View** menu only affects the current *nState* frame - a local setting.

Find the Start and End States of a Transition

You can view the state that a transition is coming from and going to by clicking the toolbar commands.

Feedback

1. Click any transition in the *nState* frame.

   For example, the transition leaves from the *Evaluate* state with a transition condition of (*!NewCard*). Notice that the starting point of a transition arrow represents the starting point of the transition.

   After you have selected a transition, the **Jump to From State** and **Jump to To State** icons on the toolbar are enabled.

2. Click the **Jump to From State** icon to see where the selected transition is from.

   The *Evaluate* state is highlighted with red color, that is, the selected transition is starting from the *Evaluate* state.

3. Click the **Jump to To State** icon .

   The *Compare16* state is highlighted with red color, which indicates that the selected transition ends in the *Compare16* state.

4. Click any of the transitions to view what state the transition is coming from and going to.

5. Select any transition and use the middle mouse button to drag the state and drop it in the source code frame. The transition is automatically located and loaded into the source code frame. All the statements associated with the transition are highlighted.

6. Repeat the drag and drop action for any state for example, the *Compare16* state. The statements associated with the state are automatically highlighted.

Create a Partial Finite State Machine Frame

When your state machine is very large with many states, you may focus on a portion only.

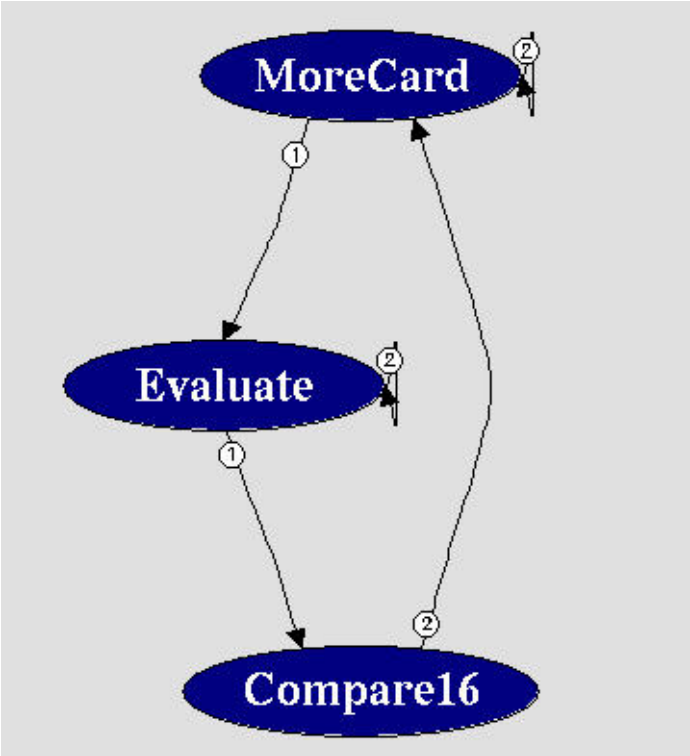
1. Select the *MoreCard* state.

2. Press and hold the <Shift> key, and select states *Evaluate* and *Compare16*.

3. Choose **Tools -> Partial FSM** command to open a new *nState* frame showing only the selected states.

Feedback

Figure 189 nState Frame Displaying Partial FSM



4. Click the **Undock** icon in upper right to make the partial FSM frame a standalone window. From the full *nState* frame, select the *Compare21* state and drag it to the partial frame.

   The state connection is added.

5. Close the Partial FSM frame.

State Animation

One of the challenges of state machine debug is seeing how the state machine changes based on the current stimulus. You can use *nState*, *nWave,* and **State Animation** to understand the flow.

Continue the following steps with an opened *nState* frame.

1. In the *nTrace* main window, choose **File -> Load Simulation Results** command to open the *Load Simulation Results* form and load the simulation results.
2. Select the <*working_dir*>*/demo/verilog/cpu/CPUsystem.fsdb,* and click **OK** to load the simulation result file.

Feedback

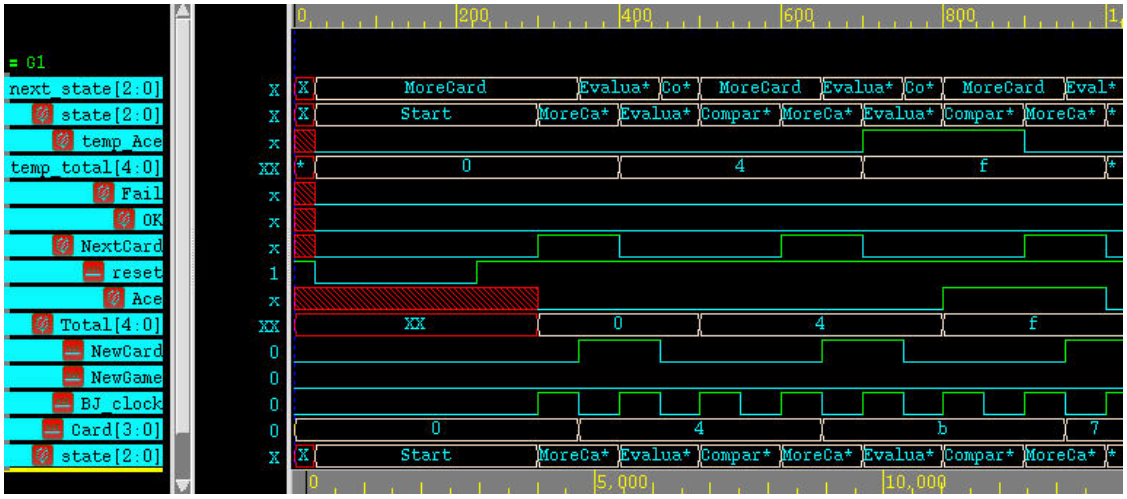
3. Click the **New Waveform** icon in the *nTrace* main window, the *nWave* frame displays in the bottom.

   The simulation file loaded in Step 1 is used as the loaded simulation result file in *nWave*.

   Note:
   For information on getting signals into the *nWave* window, refer to the nWave Tutorial chapter.

4. In *nSchema*, select the FSM symbol and use the middle mouse button to drag and drop to *nWave*. The FSM signals are added, as shown in the following figure:

Figure 190 nWave Frame Displaying FSM Signals



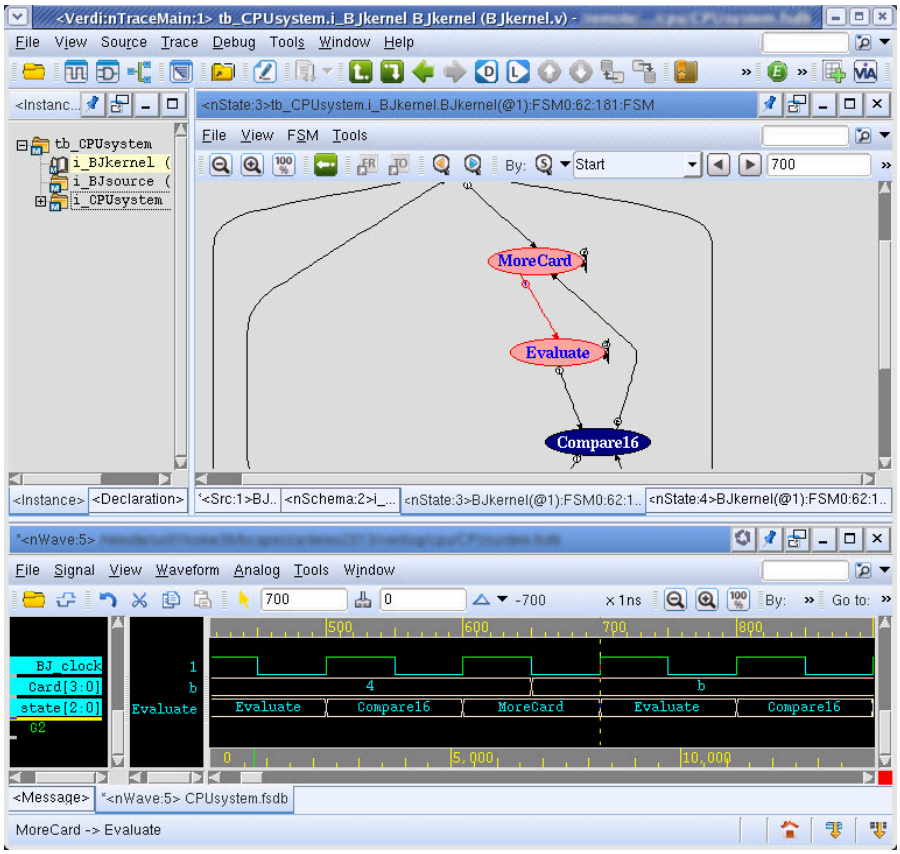
   Note:
   The state variable signal displays state aliases.

5. After the simulation results are loaded, turn on the **FSM -> State Animation** toggle command in the *nState* frame to enable state animation.

   The *nState* frame shows the value changes by highlighting the state(s) and transition, while the *nWave* window shows the value changes in the waveform format. Compare the state signal *State[2:0]* in the waveform pane of the *nWave* window with the state signal in the *nState* window. The following figure shows *nState* and *nWave* at cursor time 700.

Feedback

Figure 191 nState and nWave During State Animation



6. In *nState*, change the cursor time to *300ns* by typing *300* in the **Cursor Time** text box on the toolbar and press **Enter** on the keyboard.

   nState highlights in pink the state(s) and the transition at the specified cursor time in the *nState* frame. The *Start* state, *MoreCard* state are highlighted. The cursor time in the *nWave* window synchronizes with the **Cursor Time** text box in the *nState* window.

7. Click the **Next State** icon to move to the next state event.

   The *MoreCard* state, the *Evaluate* state, and the transition in between are highlighted. Notice that the cursor time changes to *400*. This time change indicates that the next value change occurs at the cursor time of *400*.

8. Click the **Previous State** icon .

   The *Start* state, the *MoreCard* state, and the transition in between are highlighted, and the cursor time is *300*.

Feedback

Note:
Sometimes when the next/previous state is the same as the current state, the highlight on the *nState* frame remains the same but the cursor time changes.

In addition to **Next State** and **Previous State** icons, the **State List** text box can be used to go to a state. Following are the functions of the **State List** box:

- Finding the state in the *nState* window.
- Defining the state or state sequence for the **Search Forward** and **Search Backward** icons.

9. Set the **Search By** criteria to *State* in the toolbar of the *nState* frame.
10. Use the **State List** box to find the state in the *nState* window. For example, select *Evaluate* in the **State List** box.

    nState finds and highlights the *Evaluate* state in the *nState* window (the state changes from a blue background to a red background).

11. Click the **Search Backward** icon and **Search Forward** icon .

    The **Cursor Time** text box shows the time when the state signal value changes for the *Evaluate* state.

12. Choose **FSM -> Edit Search Sequences** command to open the *Search* form.
13. In the *Search* form, click **New** to open the *New Search Sequence* form.
14. Type *test* in the **Name** text field.
15. Double-click *Morecard*, *Evaluate*, and *Compare16* (or single-click and then add by clicking on the **Add State** icon).
16. Click **OK** on the *New Search Sequence* form and **Close** on the *Search* form.

    The **Search By** automatically changes to **Search by Sequences** and the state list box displays the sequence, *test*.

17. Click the **Search Backward** and **Search Forward** icons.

    The **Cursor Time** text box shows the time when the state sequence occurs. This sequence occurs several times during this simulation.

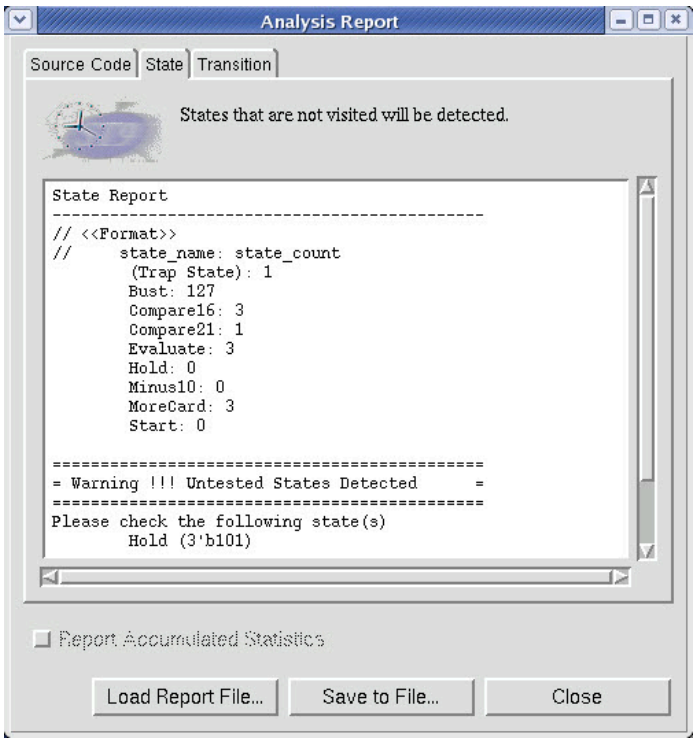
Feedback

State Machine Analysis

After the simulation results are loaded, you can analyze the state machine further based on the stimulus used.

1. Choose **FSM -> Analysis Report** command to open the *Analysis Report* form as shown in the following figure. Notice that there are three tabs - with the **Source Code** tab as the default tab.
2. Click the **State** tab.

Figure 192 Analysis Report Window



The number of times a *state* is accessed during the simulation is summarized. Also, any *states* that are not covered are listed.

3. Click the **Transition** tab.

   The number of times a transition is accessed during the simulation is summarized. Also, any transitions that are not covered are listed.
4. In the *nState* frame, choose **File -> Close Window** command to close the FSM window.

10

SmartLog Tutorial

This chapter describes the following topics:

- Overview
- Invoking SmartLog
- Navigating SmartLog
- Browse Views
- Specifying Time Unit in UVM/OVM Log File
- Using Hyperlink Rule File
- Timing Violation Debug
- Configuring a New Partitioning Rule
- Applying Partitioning Rule
- Opening Multiple SmartLog Windows and Synchronizing With nWave
- Locating Objects
- Searching, Filtering, and Reloading the Log File
- Setting the Column Filter
- Saving or Loading a Filter Result
- Debugging in Verdi Frames
- Using SmartLog in Interactive Debug
- Known Issues and Limitations

Overview

The SmartLog engine can be used to read and traverse log files in a readable view. SmartLog loads a log file with a specific format that shows a structured presentation of the log contents, thus providing a flexible and convenient use model for debugging. The

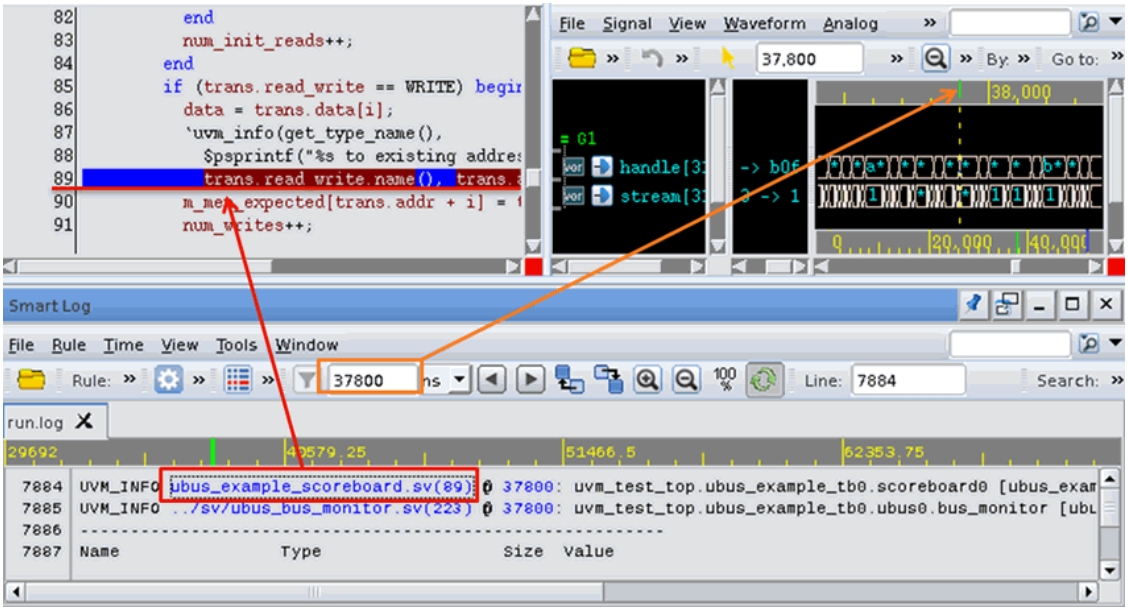
Feedback

connection between the log file and the Verdi platform is built according to the hyperlink rule files and the partitioning rule files.

Click the hyperlink to the log content in the *SmartLog* window to link to the Verdi platform, for example, link to the source code location. The hyperlink rule can be further customized with Tcl commands to execute a specific action in Verdi. A partitioning rule can also be used to group the messages in SmartLog for systematic and convenient log navigation.

Figure 193 shows an example where you can jump to the specified time and click the hyperlink in the *SmartLog pane* to directly link to the *Source Code pane* in Verdi.

Figure 193 SmartLog



SmartLog offers the following features:

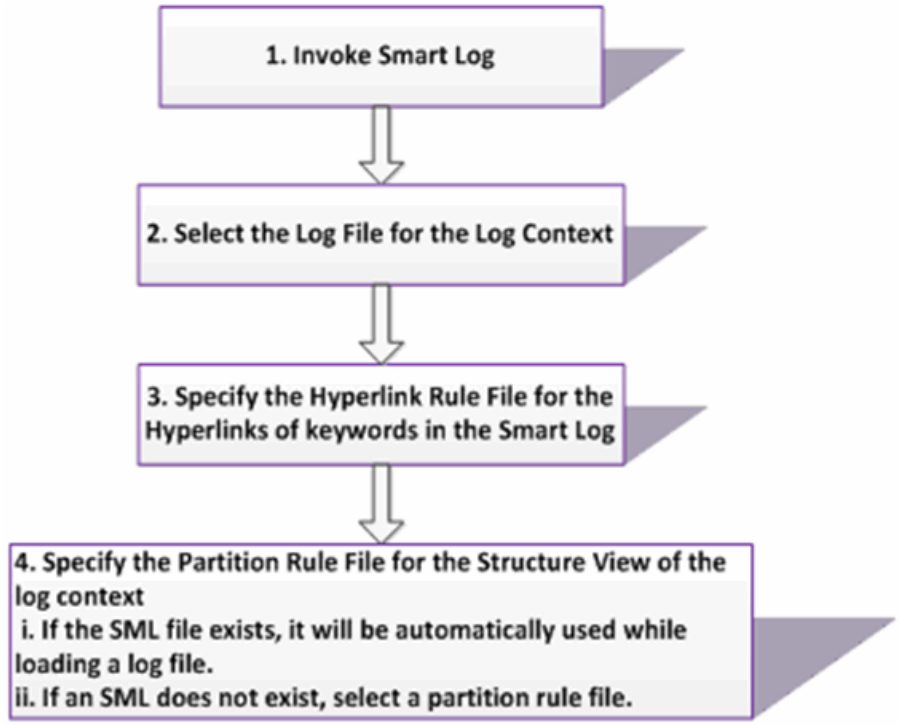
- Two browsing views to display log in the following modes:
  - *File View* for sequential information
  - *Structure View* for grouped information
- Identify and filter out the required log messages
- Synchronize with the *cursor* in the *nWave* frame
- Unified usage for interactive simulation console
- Customization of hyperlink rules and partitioning rules

Feedback

Invoking SmartLog

Figure 194 illustrates the stepwise flow for invoking SmartLog and loading a log file to SmartLog.

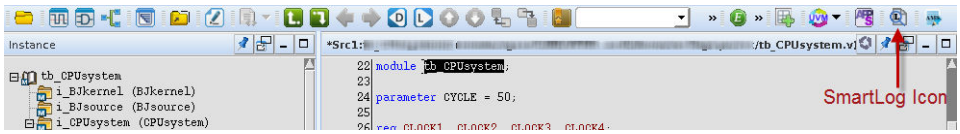
Figure 194 SmartLog Flow



The following steps describe how to invoke SmartLog and load a log file:

1. Click the **SmartLog** icon to open the *SmartLog* pane in the Verdi main window, as illustrated in Figure 195.

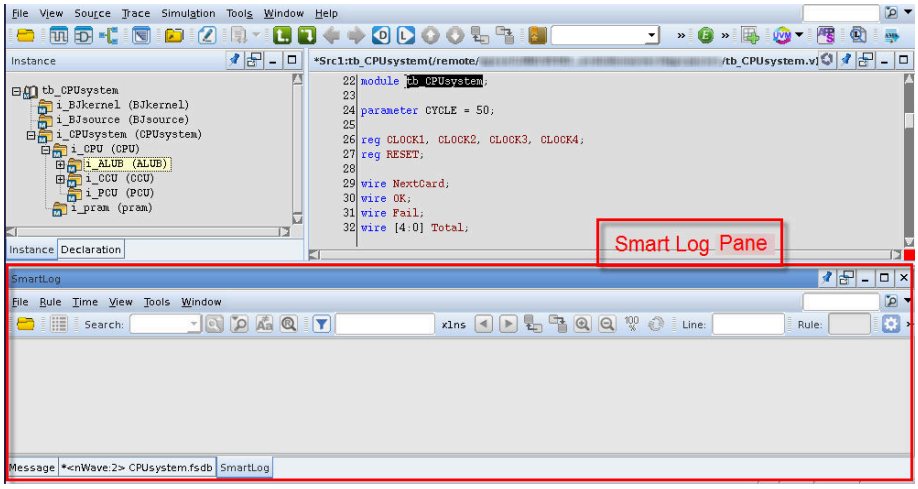
Figure 195 Invoke SmartLog Using SmartLog Icon



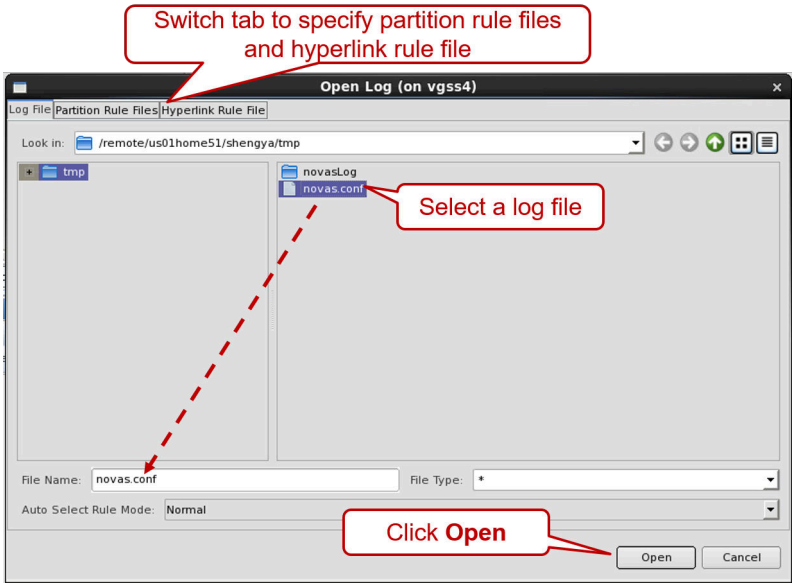
The SmartLog pane opens as illustrated in Figure 196.

Feedback

Figure 196 SmartLog Pane



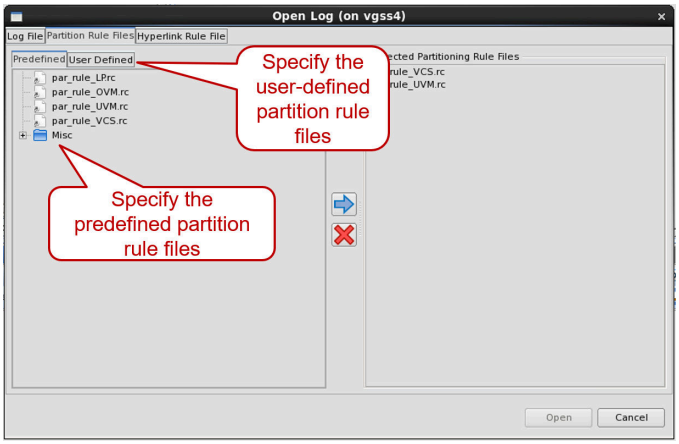
2. Click the icon in the *SmartLog* pane to open a log file. The *Open Log* form is invoked where you can select a log file in the *Log File* tab, as illustrated in the following figure:



To specify partition rule files, click the *Partition Rule Files* tab.

Feedback

- Specify a predefined partition rule file in the *Predefined* tab; click the file you want to open and click on the arrow to include it to the *Selected Partitioning Rule Files* section.
- Specify a user-defined partition rule file in the *User-Defined* tab; click the file you want to open, click on the arrow to include it to the *Selected Partitioning Rule Files* section.



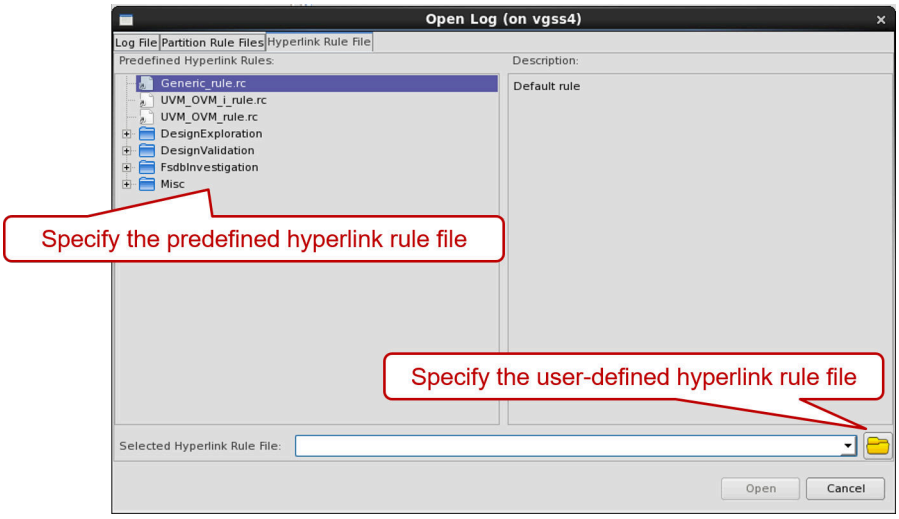
The *User-Defined* tab in the *Partitioning Rule Files* section lists all directories and files with the *.rc* extension under the root directory. You can configure the root directory and the change is saved in the *novas.rc* file.

SmartLog records the previously selected partitioning rule files in the *novas.rc* file and displays these rule files in the *Selected Partitioning Rule Files* section by default.

Feedback

To specify a hyperlink rule file, click the *Hyperlink Rule File* tab,

- Select a predefined hyperlink rule file in the *Predefined Hyperlink Rules* section.
- Select a user-defined or customized hyperlink rule file using the icon.



3. Click the **Open** button in the *Open Log* form.

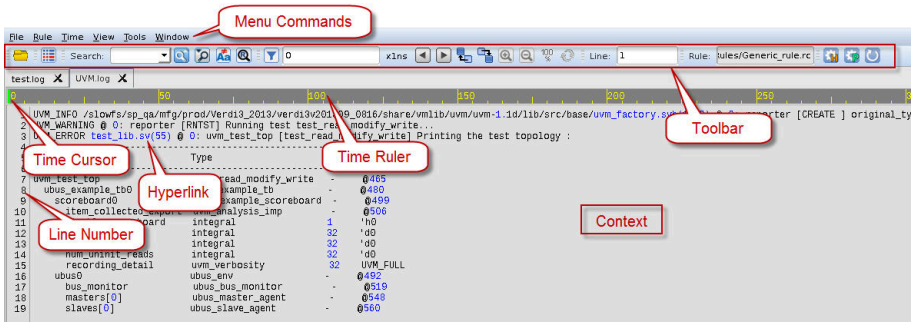
   If there is an SML file for the specified log file, the SML file is automatically used for partitioning the log file in Structure View.

   Note:
   If the log file is generated by VCS with the `-sml -l <log_file_name>` simulation option, *<log_file_name>.sml* is also generated at the same time.

   If the selected log file has a corresponding SML file, the *Partitioning Rule Files* section is hidden.

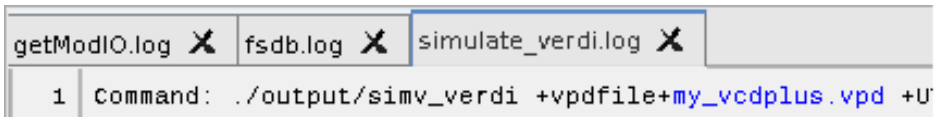
   Figure 197 illustrates an example of SmartLog displaying a UVM log file with the rule file.

Figure 197 Displaying a Log File in SmartLog



Feedback

Opening multiple log files is also supported. Each log file is loaded in the corresponding tab. The hyperlink rule file and the partitioning rule file specified in different tabs are independent.



Auto Select Rules

This section describes the auto select rules in the following sub-sections:

- Checking Mapping Rule File
- Predicting Mapping Rule
- Auto Select Rule Mode

Checking Mapping Rule File

A mapping rule file contains multiple mapping rule sets and a mapping rule set contains the following three items:

1. *patternInFileName* (regular expression) or *patternInLogHeader* (regular expression)
   a. *patternInFileName*: is used to match the log's file name
   b. *patternInLogHeader*: is used to match the log's first 30 lines
2. *hyperlinkRule*: is applied if the pattern matches
3. *partitionRules*: is applied if the pattern matches

The absolute path to the mapping rule file is specified in *novas.rc*, and *SmartLog* parses the specified file.

```
novas.rc

...
[VIA.smartLog.preference.mappingRuleFile]
vgifMappingRuleFile = absolute/path/to/mapping/rule/file
...
```

Feedback

Example

- If the file name of the selected log starts with "sim" and ends with ".log", for example, "simulation.log", the following mapping rule set gets selected:

```
patternInFileName =     ^sim.*\.log$

hyperlinkRule =         absolute/path/to/hyperlinkRule_1.rc

partitionRules =        absolute /path/to/partitionRule_1.rc \
                        absolute /path/to/partitionRule_2.rc
```

- If any line of the first 30 lines of the log file contains "listModFile", the following mapping rule set gets selected:

```
patternInLogHeader =    listModFile

hyperlinkRule =         absolute/path/to/hyperlinkRule_1.rc

partitionRules =        absolute /path/to/partitionRule_1.rc \
                        absolute /path/to/partitionRule_2.rc
```

Predicting Mapping Rule

SmartLog predicts suitable partition rule files and hyperlink rule file for the selected log, if the selected log does not match any mapping rule set in the mapping rule file.

The user-defined partition rule files are included in predicting a rule, if the directory which contains the user-defined partition rule files is specified in the *novas.rc* file.

```
novas.rc

...
[VIA.smartLog.preference.mappingRuleFile]
vgifUserDefinedParRuleDir = absolute/path/to/directory
...
```

Auto Select Rule Mode

You can change the auto select rule mode using the *Auto Select Rule Mode* field in the *Open File* form.

The following options are available in the *Auto Select Rule Mode* field:

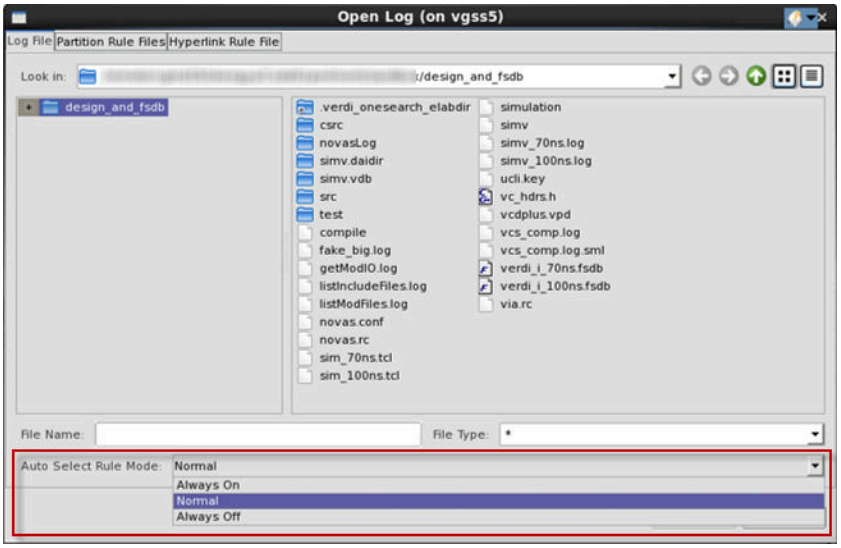
- **Always On**: Enables auto-select rule.
- **Normal**: Enables auto-select rule.

Feedback

The **Normal** mode changes to the **Always Off** mode, if any partition rule files or hyperlink rule file is added or removed in the *Open Log* form and if the **Open** button is clicked in the *Open Log* form.

- **Always Off**: Disables auto select rule.

The *Auto Select Rule Mode* field is illustrated in the following figure:

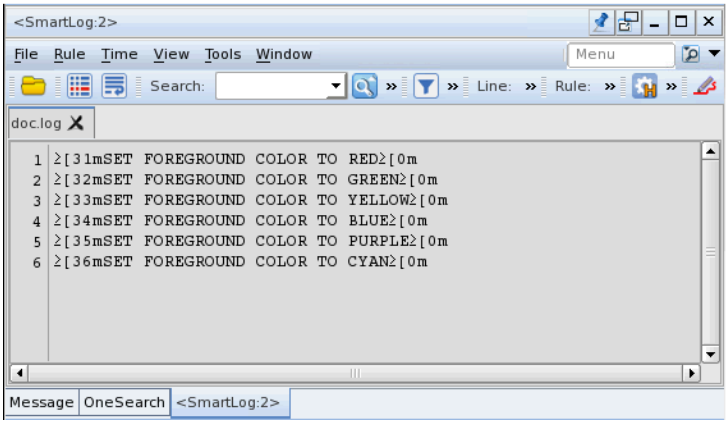


Displaying ANSI Colors in SmartLog

If a log file has an ANSI flag, by default, *SmartLog* shows the ANSI flag directly, as illustrated in the following figure:

Feedback

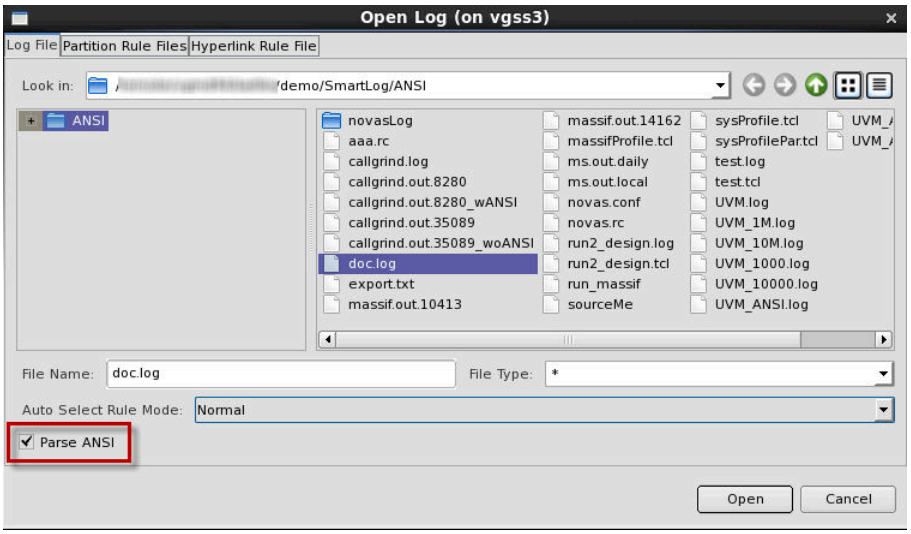
```
1 ^[[31mSET FOREGROUND COLOR TO RED^[[0m
2 ^[[32mSET FOREGROUND COLOR TO GREEN^[[0m
3 ^[[33mSET FOREGROUND COLOR TO YELLOW^[[0m
4 ^[[34mSET FOREGROUND COLOR TO BLUE^[[0m
5 ^[[35mSET FOREGROUND COLOR TO PURPLE^[[0m
6 ^[[36mSET FOREGROUND COLOR TO CYAN^[[0m
```



To enable *SmartLog* to parse the ANSI flags present in a log file and display ANSI colors in the opened log file, perform the following steps:

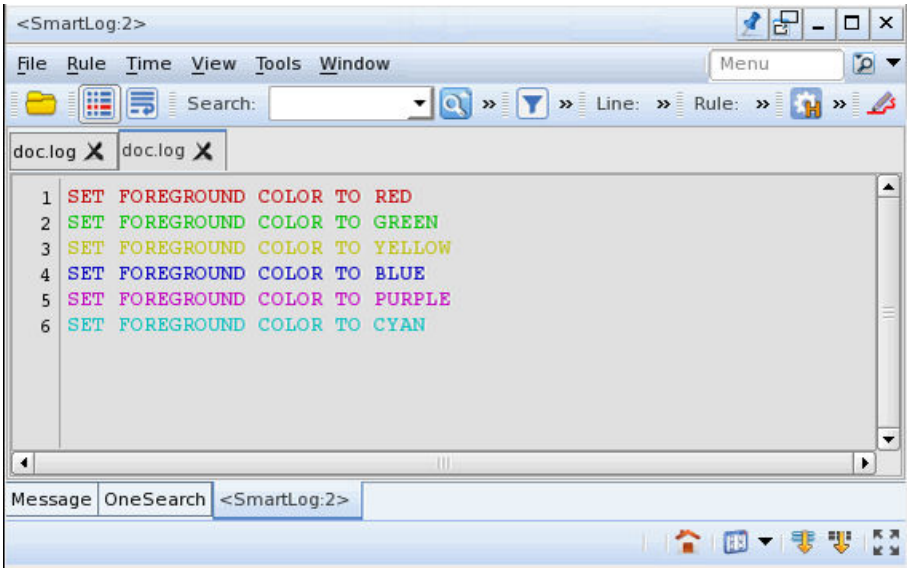
1. In the *Open Log* form, enable/check the **Parse ANSI** check box.
2. Select a log file and click **Open**.

The *Open Log* form is illustrated in the following figure:



On selecting the **Parse ANSI** check box, *SmartLog* displays the log file with ANSI colors as follows:

Feedback



Note:
Enabling the **Parse ANSI** option while opening a log file may cause performance issue, therefore, you must enable it only if required.

Opening a Log File Using Verdi Commands

You can use the following commands to open a log file in SmartLog:

- `-smlog logfile`

  Loads the specified log file with SmartLog.

- `-smlog_hyper | -smlog_h ruleFile`

  Specifies the hyperlink rule file for the specified log.

  Note:
  This option must be used with the `-smlog` option.

- `-smlog_partition | -smlog_p "ruleFile1 ruleFile2...ruleFileN"`

  Specifies the partitioning rule file(s) for the specified log. A pair of double quotes (") must be used to enclose the partitioning rule file(s).

  Note:
  This option must be used with the `-smlog` option.

Feedback

Examples

The following are some examples to demonstrate use cases to open log file(s) using Verdi commands:

Example 1 - To open a specified log (*test.log*) with specified partitioning rule file (*parRule1.rc*) and hyperlink rule file (*hyperRule.rc*), use the following command:

```
%> verdi -smlog test.log -smlog_hyperlink hyperRule.rc -smlog_partition
"parRule1.rc"
```

Example 2 - To open a specified log (*test.log*) with specified partitioning rule files (*parRule1.rc*, *parRule2.rc*) and hyperlink rule file (*hyperRule.rc*), use the following command:

```
%> verdi -smlog test.log -smlog_hyperlink hyperRule.rc -smlog_partition
"parRule1.rc parRule2.rc"
```

Example 3 - To open specified log files (*test1.log*, *test2.log*) with the same specified partitioning rule files (*parRule1.rc*, *parRule2.rc*) and hyperlink rule file (*hyperRule.rc*), use the following command:

```
%> verdi -smlog test1.log -smlog test2.log -smlog_hyperlink hyperRule.rc
-smlog_partition "parRule1.rc parRule2.rc"
```

Example 4 - To open multiple log files (*test1.log*, *test2.log*) with different partitioning rule files and hyperlink rule files, use the following command:

```
%> verdi -smlog test1.log -smlog_hyperlink hyperRule1.rc -smlog_partition
"parRule1.rc" -smlog test2.log -smlog_hyperlink hyperRule2.rc
-smlog_partition "parRule2.rc"
```

Note:

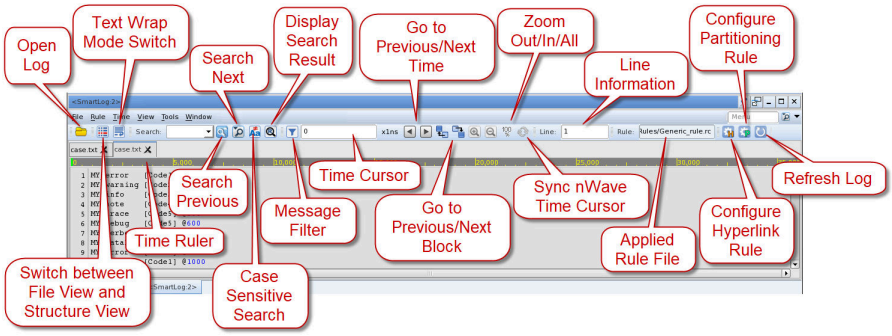
You can also use the `viaLogViewerOpenLog` Tcl command to open a log file. For details on SmartLog Tcl commands, see the *SmartLog* chapter in *Verdi and Siloti Tcl Reference Manual*.

Navigating SmartLog

SmartLog provides a toolbar with frequently used commands to navigate the log files, as illustrated in Figure 198.

Feedback

Figure 198 SmartLog Layout



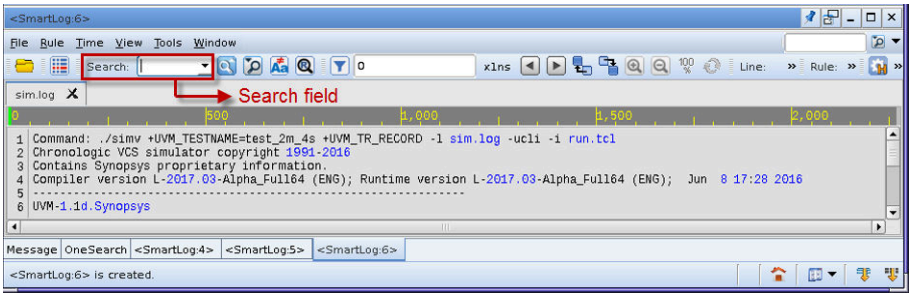
The icons in the toolbar can be used to navigate the displayed content in SmartLog for the currently displayed log file.

Shortcut Keys for SmartLog

Following is the list of shortcut keyboard keys that can be used to navigate through the SmartLog tabs:

- **Ctrl** + **PageDown** - Use this shortcut to move to the next SmartLog tab.
- **Ctrl** + **PageUp** - Use this shortcut to move to the previous SmartLog tab.
- **Ctrl** + **w** - Use this shortcut to close the SmartLog tab.
- **Ctrl** + **f** - Use this shortcut to select the search field in the SmartLog pane.

Figure 199 Search Field in SmartLog Window



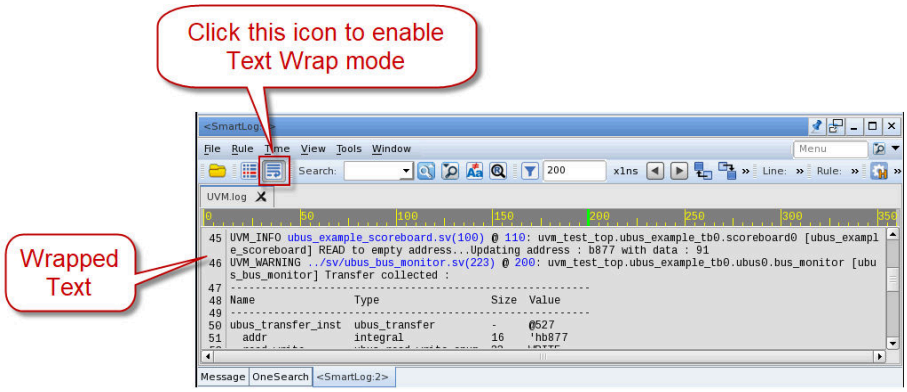
Wrapped Text

In the *SmartLog* window, if the text is a long string which is too long to be displayed in the window, you can enable the text wrap functionality to fit the text in the available width of

Feedback

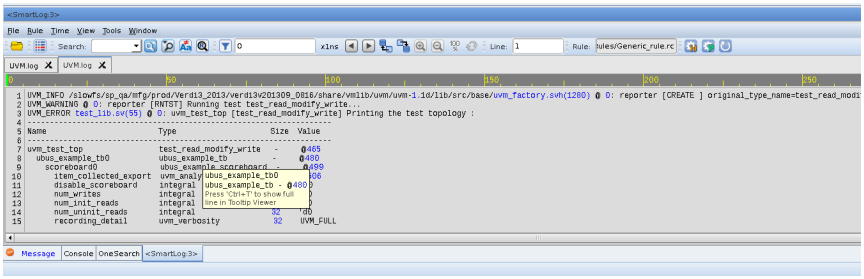
the window. To enable the text wrap functionality, click the **Text Wrap Mode Switch** icon, as illustrated in the following figure:

Figure 200 Text Wrap Mode in SmartLog Window



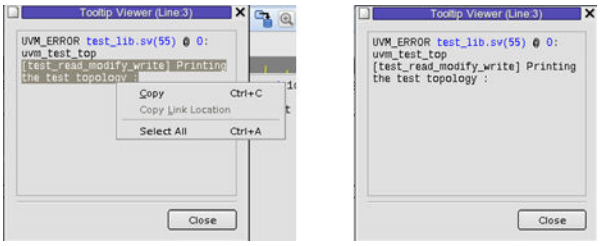
The wrapped text is also displayed in the Tooltip that appears next to it, as illustrated in the following figure:

Figure 201 Tooltip in SmartLog Window



You can use the **Ctrl +T** shortcut key to open the *Tooltip Viewer* and read or copy the complete string. If the long string contains a hyperlink, you can view and click the hyperlink in the *Tooltip viewer*. The following figure shows the *Tooltip viewer*:

Figure 202 Tooltip Viewer



Feedback

Browse Views

SmartLog has two browse views: *File View* and *Structure View*, which display the context of the log files. After loading a log file into the *SmartLog* pane, click the icon to switch *File View* and *Structure View* to display the context. These views are described in the following sections:

- File View
- Structure View

File View

As illustrated in Figure 197, *File View* displays the log text row by row, accompanied with some keywords as hyperlinks.

When a hyperlink is clicked upon, the corresponding action described in the hyperlink rule file is triggered. SmartLog uses the hyperlink rule written in a **.rc* file to define where to set the hyperlink, and what to do after clicking the hyperlink keyword. Click the icon to configure an existing hyperlink rule or customize your own hyperlink rule (for details about the usage of the hyperlink rule file, see the Using Hyperlink Rule File section).

Structure View

The *Structure View* displays the content of the log file in a spreadsheet. As illustrated in Structure View, the content is grouped into several blocks based on attributes (such as time, severity, file name and so on) specified in the partitioning rule file. Each block delivers one message and contains time, severity, and entire content for the message. For details about the partitioning rule file, see the Applying Partitioning Rule section.

Feedback

Figure 203 Structure View

The following columns are present in the *Structure View*:

- **Time**

  The simulation time of a block in the log file is displayed in the **Time** column, when the specified partitioning rule file contains the *Time* attribute. If the partitioning rule file does not contain the *Time* attribute, the value 0 is displayed in the field for each block.

  Note:
  The value of the *Time* attribute is assumed as an unsigned integral type. The values should monotonically increase with the rows in the log file.

- **Severity**

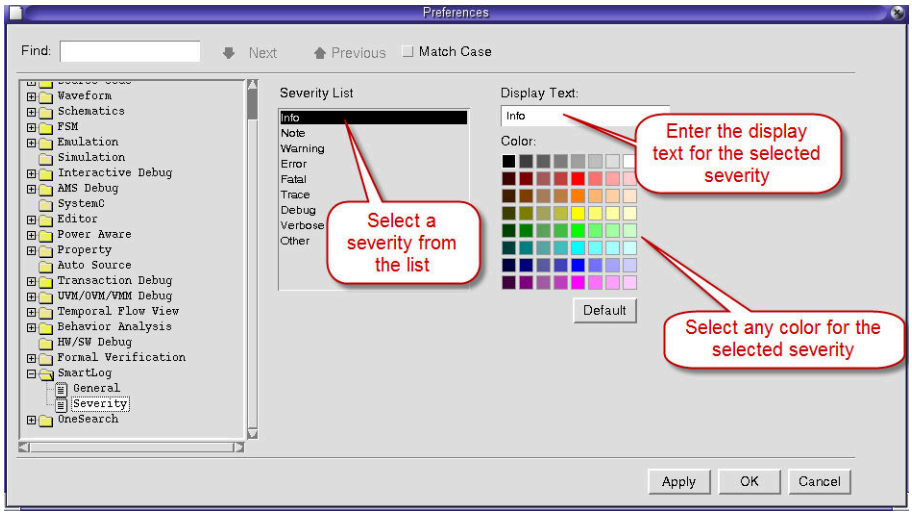
  The severity of a block is displayed in the **Severity** column.

  - The font color of the block is determined by the severity type, as follows:
  - If the severity type is *Fatal* or *Error*, the block font color is red.
  - If the severity type is *Warning*, the block font color is orange.
  - In other cases, the block font color is black.

  You can also customize the displayed text and the text color for the severity list using the **Severity** page in the *SmartLog* folder of the *Preferences* form in Verdi.

Feedback

Figure 204 SmartLog Page in Preferences Form

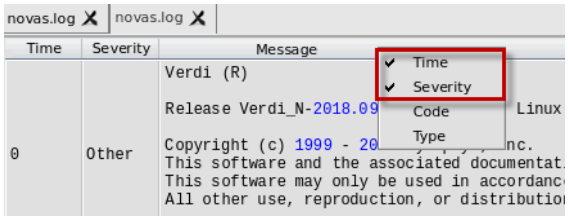


Note:
Empty spaces cannot be entered in the **Display Text** field.

- **Message**

  The entire content of a block is displayed in the **Message** column. The hyperlink rule is also applied to the text. The behavior in the **Message** column is the same as that in *File View*.

  Use **Time** and **Severity** right-click options on the heading of the columns to hide or display the corresponding column.



- **Code**

  The code of a block in the log file is displayed in the **Code** column. This column is hidden by default. To display this column, right-click on the heading of the columns and enable the **Code** option.

- **Type**

  The type of a block in the log file is displayed in the **Type** column. This column is hidden by default. To display this column, right-click on the heading of the columns and enable the **Type** option.

Feedback

Note:

You can rearrange the sequence of the columns in the *Structure* view by clicking and dragging a column. You can restore the last display settings for columns including column width and column sequence from the *novas.rc* file.

User Specified Columns

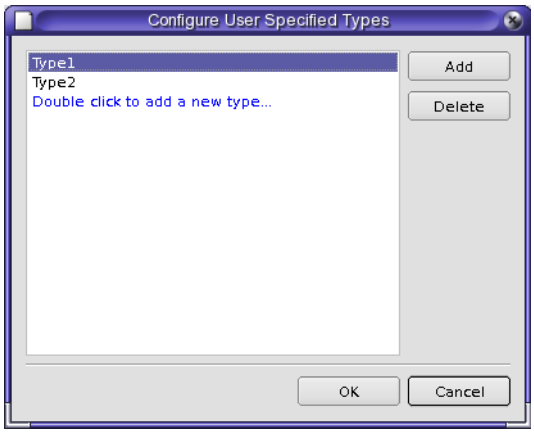
SmartLog allows users to extract valuable information from a specific log and create a new column in the *Structure* view to store that information.

To add the user specified column,

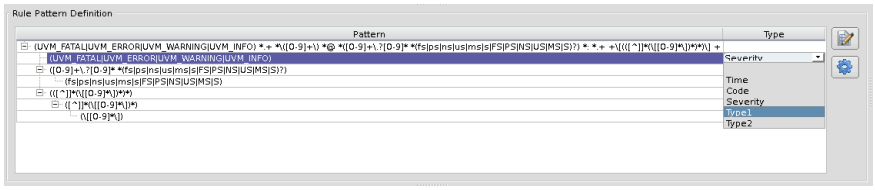
1. Click the **Configure Partitioning Rule** icon present on the SmartLog toolbar.
2. In the *Rule Pattern Definition* section in the *Configure Partitioning Rule* form, click the icon.

3. In the *Configure User Specified Types* form, edit the user specified types by clicking **Add** and **Delete** buttons.

Figure 205 Configure User Specified Types Form



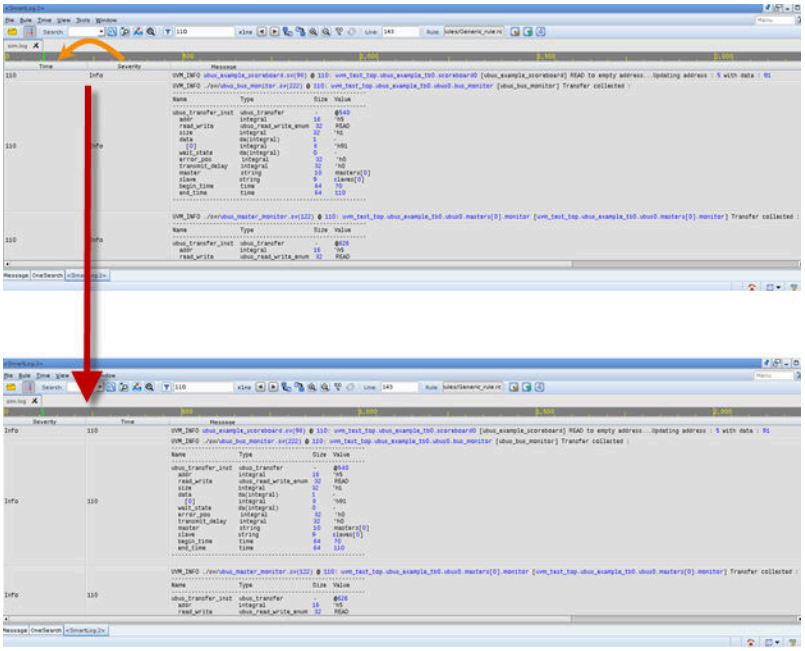
Once the configuration is done, the new types are added in the drop-down list, as illustrated in the following figure:



The user-specified columns (if any) are displayed by default in SmartLog.

Rearranging the Columns

You can also rearrange the column sequence as required, by clicking and dragging the columns towards left or right. The following figures show rearrangement of **Time** and **Severity** columns.



Restoring Previous Column Display Settings

You can restore the previous settings for display of columns using the *novas.rc* settings file. The width of the columns, sequence of the columns, and the show/hide settings for columns are restored on using the *novas.rc* file.

Specifying Time Unit in UVM/OVM Log File

SmartLog acquires the timing information from the following time units;

Feedback

- **Log Time Unit**

  The log time unit is applied on time without a unit and affects the time value.

  For example, if the time *10* without unit is recognized by *SmartLog* and log time unit is set as *ns*, *SmartLog* considers it as *10ns*.

  If the log time unit is not specified correctly, it breaks down the synchronization between *SmartLog*, *nWave*, and *Transaction and Protocol Analyzer*.

- **Display Time Unit**

  The display time unit does not affect the time value, it only impacts how the time is displayed.

  For example, if the time is *1000ns*, *SmartLog* shows *1000* with display time unit *ns*, and shows *1* with display time unit *ms*.

Specifying the Log Time Unit

You can specify the log time unit using any of the following methods:

- Click the **Time** -> **Setup Log Time Unit** menu command.
- Specify through the partition rule. For details, see Applying Partitioning Rule.

If the log time unit is not fixed, it is recommended to always print time with unit in the log file. For example, in SV HDL code, use `$timeformat` in the testbench to display time with unit in the log file.

Specifying the Display Time Unit

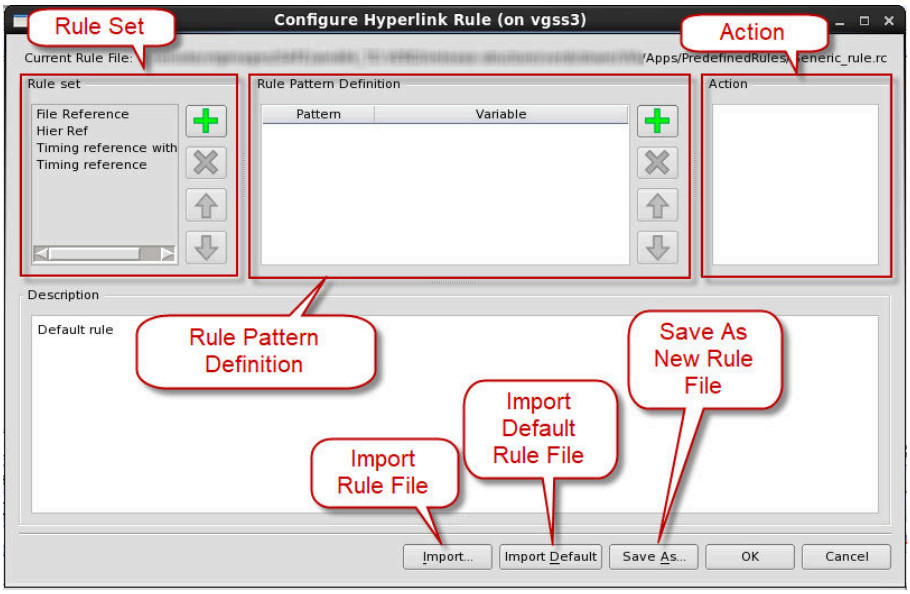
You can specify the display time unit using the **Time** -> **Setup Display Time Unit** menu command.

Using Hyperlink Rule File

For a loaded log file in SmartLog, the specified hyperlink rule file can be manually configured and updated as a new hyperlink rule file in the GUI interface. The configurations of currently loaded hyperlink rule file gets displayed in the *Configure Rule Set* form, when you click on the icon or use the **Rule** -> **Configure Hyperlink Rule** command in the *SmartLog* pane. This is also illustrated in Figure 206.

Feedback

Figure 206 Configure Hyperlink Rule Form



Click the **Import** or the **Import Default** button to get the configurations in another hyperlink rule file or the default file. After importing the rule file, the configurations are displayed in **Rule Set**, **Rule Pattern Definition**, and **Action** sections. Edit the sections to customize the rules.

The following are rule sets, rule pattern definitions, and actions in the **Configure Hyperlink Rule** form:

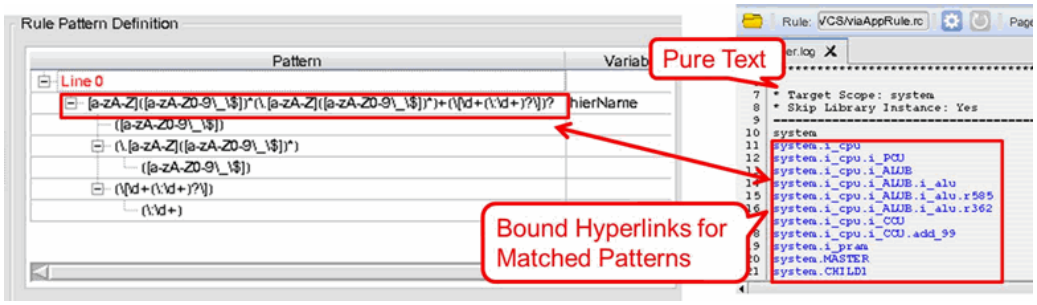
- **Rule Set**: This section displays all rule sets in the imported hyperlink rule file. When a rule set is selected in the **Rule Set** section, the corresponding rule pattern is displayed in the **Rule Pattern Definition** section. The rule set in the higher row has a higher priority in this hyperlink rule file. Use or icons to increase or decrease the priority.

  Use or icons to add or delete a rule set in the **Rule Set** section. Double-click a rule set to change the name of the rule set.

- **Rule Pattern Definition**: This section shows how to bind a hyper link in the log file using regular expressions. If the content in the log file matches the pattern specified in the **Pattern** field, a hyper link binds to the content and is displayed in the *SmartLog* pane, as illustrated in Figure 207.

Feedback

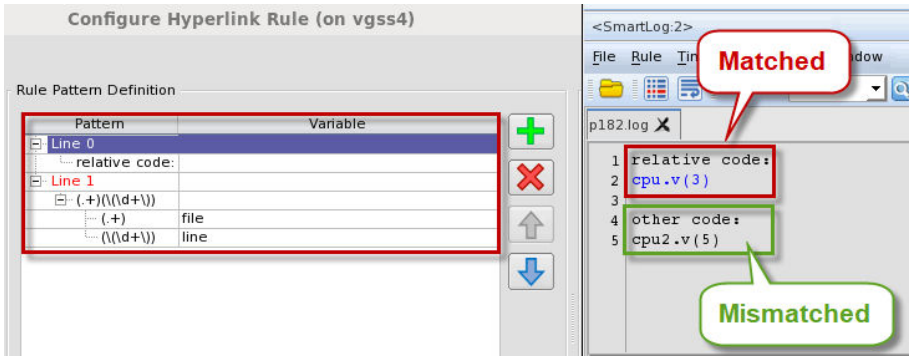
Figure 207 Hyperlink Binding



When a pattern is selected in the **Rule Pattern Definition** section, the corresponding action is displayed in the **Action** section. The variable name is specified in the **Variable** field for a pattern or sub-pattern as the variable used in the corresponding action. Use or icons to add or delete a rule pattern.

If a rule includes multiple patterns, the content in the log file matching pattern is bound to the hyper link line by line. As illustrated in Figure 208, the matched text in the log file matches the pattern specified in Line 0 and then matches the pattern specified in Line 1 in-order. Use or icons to move up or down the orders of the patterns.

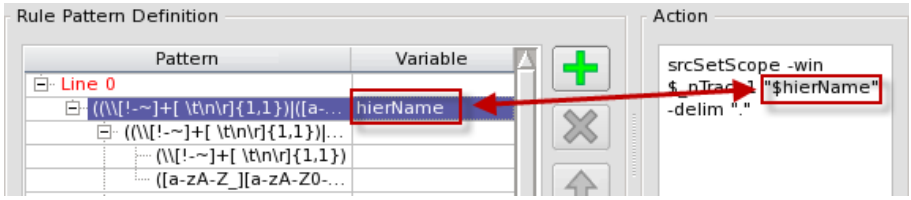
Figure 208 Line By Line Pattern



- **Action**: This section shows what action (written in Tcl) is executed in Verdi after clicking the hyperlink shown in the *SmartLog* pane. The variable specified in the **Rule Pattern Definition** section can be used in the Tcl commands with the dollar sign ($) character prefix, as illustrated in Figure 209.

Feedback

Figure 209 Variable Usage



After changing the configurations, click the **OK** button to save the changes to the hyperlink rule file and apply the rules to the loaded log file. The current log file and the other log files that use the same hyperlink rule file are automatically refreshed based on the updated hyperlink rule file.

Additionally, use the **Save As** button to save as another hyperlink rule file with *.rc* file name extension.

Note:
If the *Configure Rule Set* form is closed by clicking the **Cancel** button, *save* and *save as* actions are not reverted.

Timing Violation Debug

SmartLog provides two predefined hyperlink rules in the Verdi release package as follows that display the timing violation messages:

- **VCS rule file:** `share/VIA/Apps/PredefinedRules/timing_violation_rule.rc`
- **Third-party rule file:** `share/VIA/Apps/PredefinedRules/timing_violation_rule_3rd_party.rc`

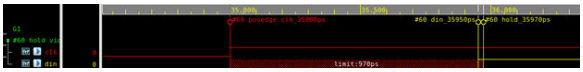
The following is the support list of timing violations in SmartLog:

- `$hold`

```
60 "flop.v", 25: Timing violation in tb.flop_inst
61     $hold( posedge clk:35000, din:35950, limit: 970 );
```

Feedback

- Click **“flop.v”, 25**:
  - If the design is imported, the source code window shows line 25 of the `flop.v` file.
- Click **tb.flop_inst**:
  - If the design is imported, the source code window shows the `tb.flop_inst` scope.
- Click **$hold(xxx)**:
  - If the `fsdb` is opened, and the signal is dumped, the nWave window shows related signals in a well-defined form as depicted in the following figure:



- `$setup`

```
47 "flop.v", 24: Timing violation in tb.flop_inst
48     $setup( din:14100, posedge clk:15000, limit: 910 );
```

- Click **“flop.v”, 24**:
  - If the design is imported, the source code window shows line 24 of the `flop.v` file.
- Click **tb.flop_inst**:
  - If the design is imported, the source code window shows the `tb.flop_inst` scope.
- Click **$setup(xxx)**:
  - If the `fsdb` is opened, and the signal is dumped, the nWave window shows related signals in a well-defined form as shown in the following figure:

- `$setuphold`

```
49 "./Source/hdbsTimChk/setuphold3.v", 30: Timing violation in HoldNegTchk.check
50     $setuphold( posedge clock:60, data:67, limits: (-5,10) );
```

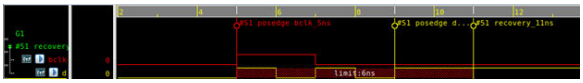
Feedback

- Click **“./Source/hdbsTimChk/setuphold3.v”, 30“./Source/hdbsTimChk/setuphold3.v”, 30**:
  - If the design is imported, the source code window shows line 30 of the `./Source/hdbsTimChk/setuphold3.v` file.
- Click **HoldNegTchk.check**:
  - If the design is imported, the source code window shows the `HoldNegTchk.check` scope.
- Click **$setuphold(xxx)**:
  - If the `fsdb` is opened, and the signal is dumped, the nWave window shows related signals in a well-defined form as shown in the following figure:

- `$recovery`

```
51 "./Source/hdbsTimChk/recovery2.v", 33: Timing violation in top.b
52     $recovery( posedge bclk:5, posedge d:9, limit: 6 );
```

  - Click **“./Source/hdbsTimChk/recovery2.v”, 33**:
    - If the design is imported, the source code window shows line 33 of the `./Source/hdbsTimChk/recovery2.v` file.
  - Click **top.b**:
    - If the design is imported, the source code window shows the `top.b` scope.
  - Click **$recovery(xxx)**:
    - If the `fsdb` is opened, and the signal is dumped, the nWave window shows related signals in a well-defined form as shown in the following figure:



- `$removal`

```
43 "./Source/hdbsTimChk/removal.v", 29: Timing violation in top.b
44     $removal( posedge bclk:5, posedge d:3, limit: 3 );
```

Feedback

- Click **./Source/hdbsTimChk/removal.v", 29**:
  - If the design is imported, the source code window shows line 29 of the `./Source/hdbsTimChk/removal.v` file.
- Click **top.b**:
  - If the design is imported, the source code window shows the `top.b` scope.
- Click **$removal(xxx)**:
  - If the `fsdb` is opened, and the signal is dumped, the nWave window shows related signals in a well-defined form as shown in the following figure:

- `$recrem`

```
49 "./Source/hdbsTimChk/recrem3.v", 30: Timing violation in RemovalNegTchk.check
50     $recrem( posedge clear:67, clock:60, limits: (-5,10) );
```

  - Click **./Source/hdbsTimChk/recrem3.v", 30**:
    - If the design is imported, the source code window shows line 30 of the `./Source/hdbsTimChk/recrem3.v` file.
  - Click **RemovalNegTchk.check**:
    - If the design is imported, the source code window shows the `RemovalNegTchk.check` scope.
  - Click **$recrem(xxx)**:
    - If the `fsdb` is opened, and the signal is dumped, the nWave window shows related signals in a well-defined form as shown in the following figure:

Configuring a New Partitioning Rule

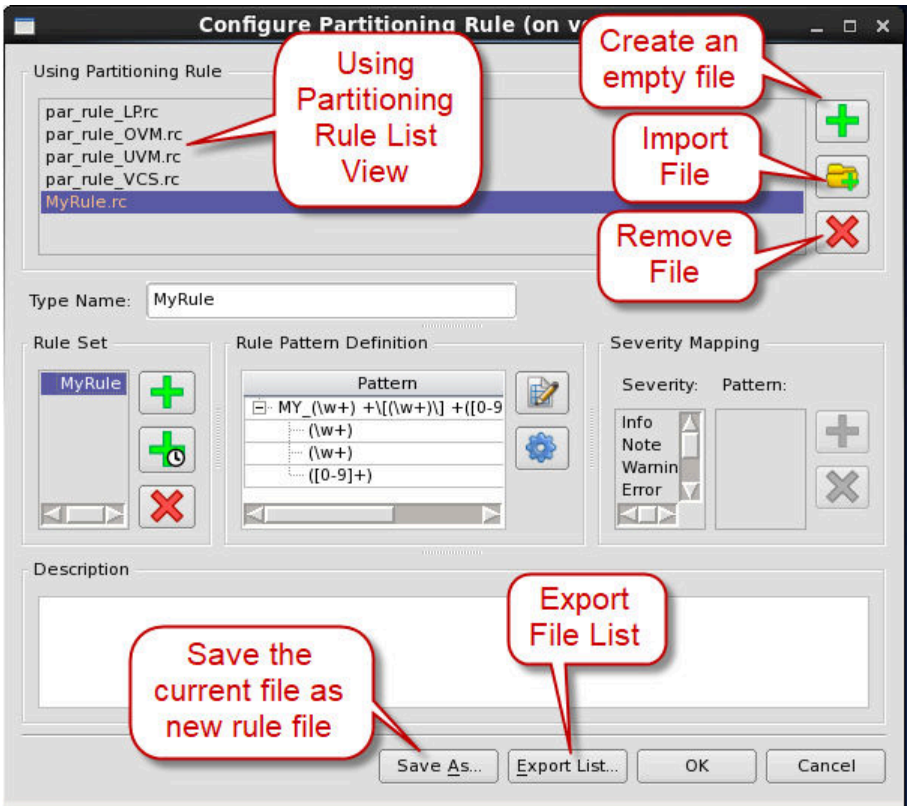
You can also create or edit multiple partitioning rule files to partition a log into several blocks and extract the properties (Time, Code, Severity) into the blocks. You can save these partitioning rule files so that these can be used based on the requirement later.

Feedback

Perform the following steps to configure or write a new partitioning rule:

1. Click the **Configure Partitioning Rule** icon present on the SmartLog toolbar. The *Configure Partitioning Rule* form opens up as follows:

Figure 210 Configure Partitioning Rule Form



Note:

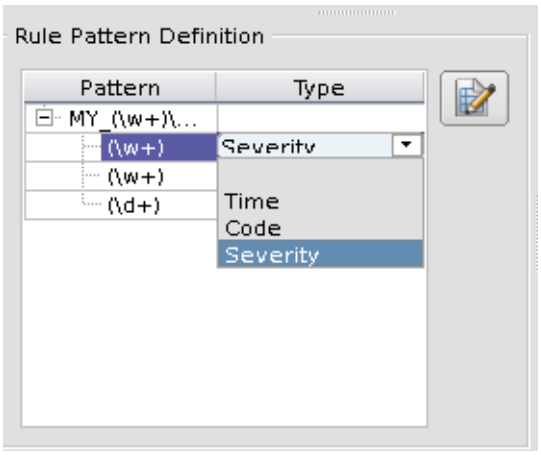
- Each log file can imply multiple partitioning rule files
- Each partitioning rule file has multiple rules and must have exclusive type name.
- Each rule has one pattern.

2. In the *Rule Pattern Definition* field, select the type of sub-pattern using the predefined combo box (as illustrated in the following figure). You can select one of the following types:
   - Severity
   - Time

Feedback

- Code

Note:
Severity/Time/Code can be selected once in one pattern.

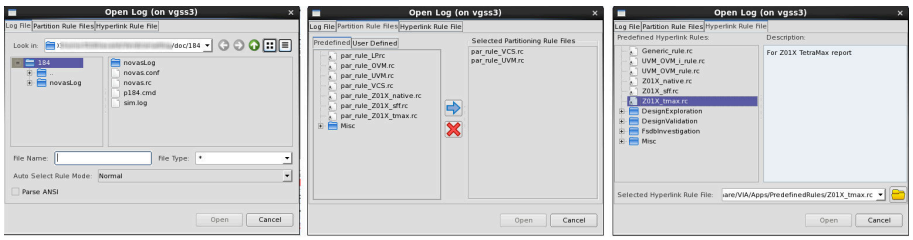


Example to Demonstrate Creating a Customized Partition Rule File

If you want to partition your log file, perform the following steps:

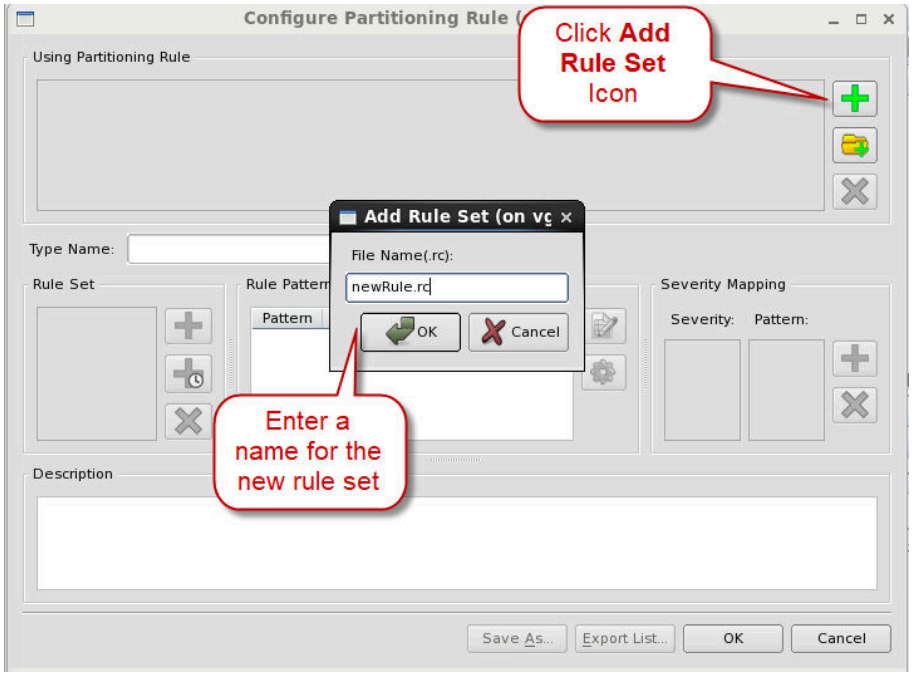
1. Click the icon in the *SmartLog* pane to open the log file.
2. In the *Open Log* form, select the log file and click **Open**.

Figure 211 Open Log Form



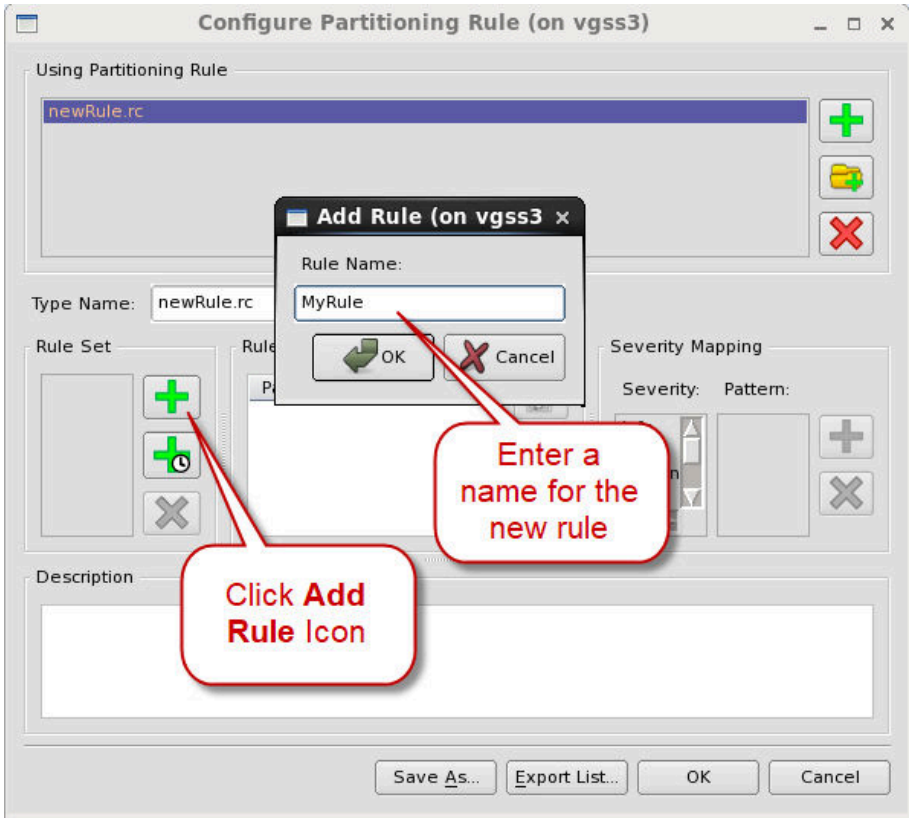
3. Click the **Configure Partitioning Rule** icon present in the SmartLog toolbar.
4. In the *Configure Partitioning Rule* form, click the **Add Rule Set** icon, enter a name for the new rule set in the **File Name(.rc)** field, and click **OK**, as illustrated in the following figure:

Feedback



5. In the *Configure Partitioning Rule* form, click the **Add Rule** icon in the *Rule Set* section, enter a name for the new rule in the **Rule Name** field, and click **OK**, as illustrated in the following figure:

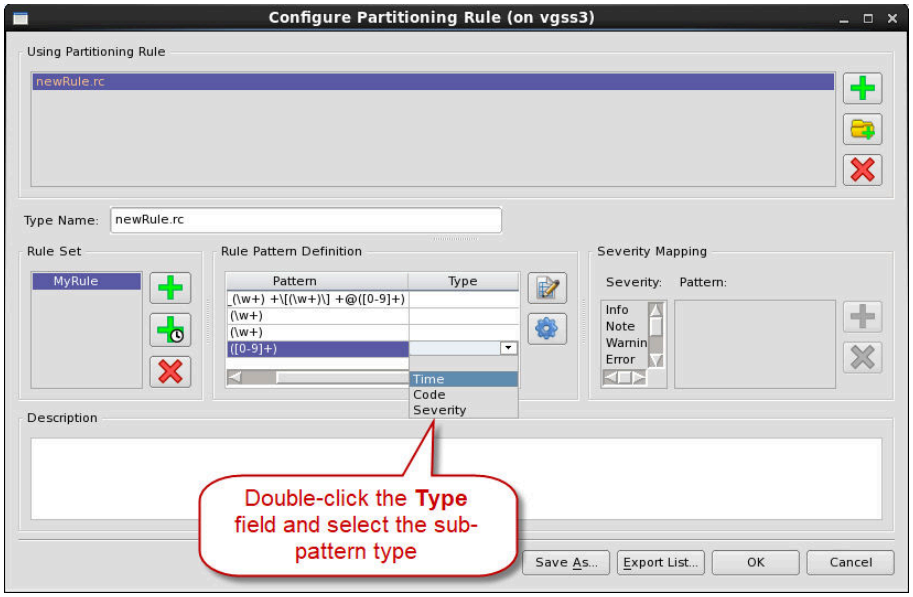
Feedback



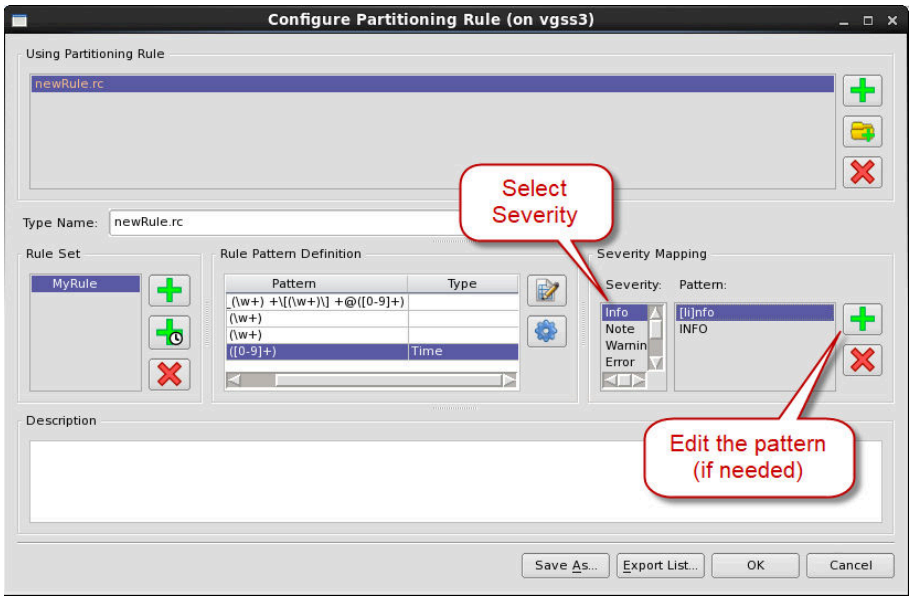
6. To edit the partitioning pattern based on the syntax of the opened log file, click the **Edit Rule Pattern** icon in the *Rule Pattern Definition* section, enter the partitioning pattern, and click **OK**, as illustrated in the following figure:

   In this example, the `MY_(\w+) +\[(\w+)\] +@([0-9]+)` pattern is used.

   Double-click the **Type** field in the *Rule Pattern Definition* section and select the sub-pattern type, as illustrated in the following figure:

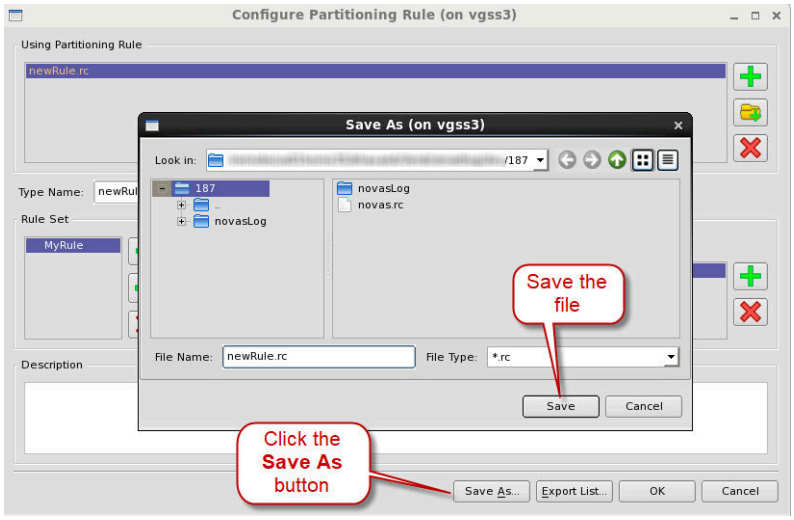


7. Select the severity and edit the pattern (if needed) in the *Severity Mapping* section, as illustrated in the following figure:



8. Click the **Save As** button in the *Configure Partition Rule* form. The *Save As* form opens up, enter the file name and click the **Save** button in the *Save As* form. Then, click on the **OK** button in the *Configure Partition Rule* form.

Feedback



The new partitioning rule file gets saved with the specified customized settings mentioned in the preceding steps.

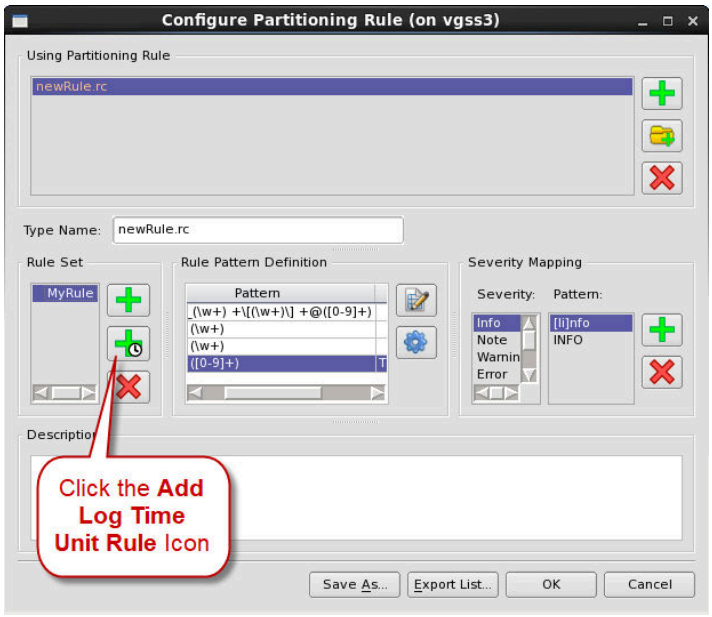
Creating Log Time Unit Rule

SmartLog allows you to create a log time unit rule and if the specified pattern in the rule matches a line in the opened log file, the log time unit is set automatically.

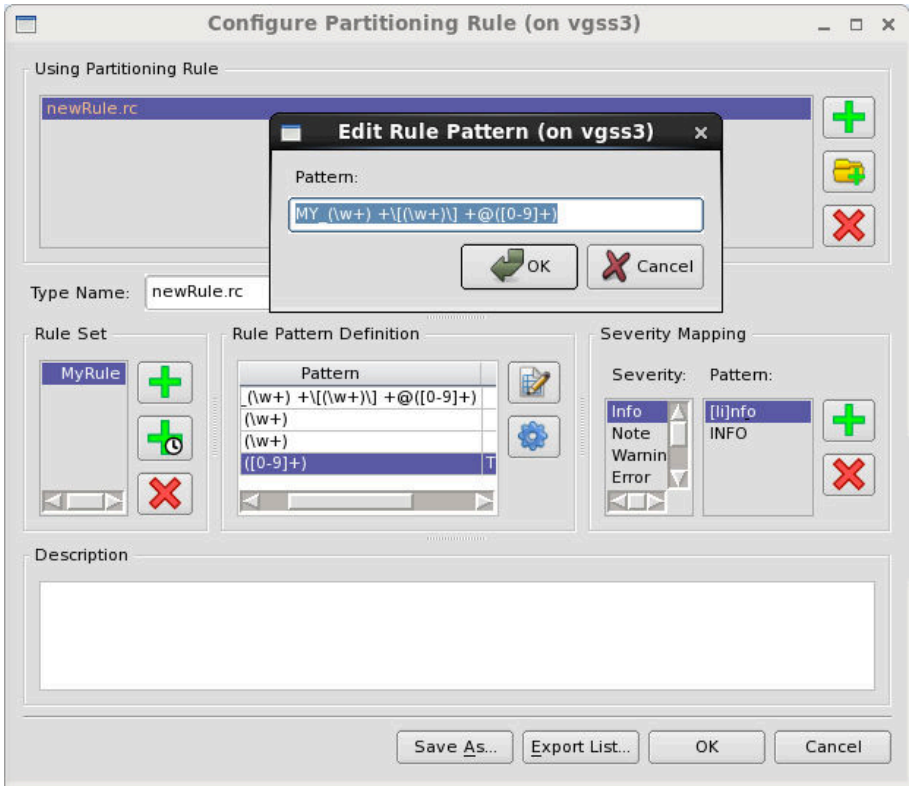
Feedback

To create a log time unit rule, perform the following steps:

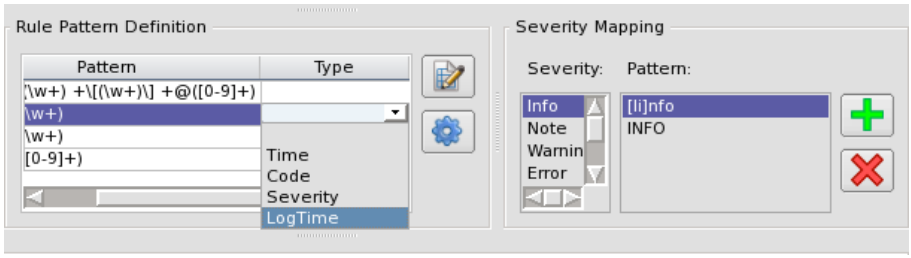
1. Click the **Configure Partitioning Rule** icon present in the SmartLog toolbar.
2. In the *Configure Partitioning Rule* form, click the partitioning rule for which you want to create a log time unit rule in the *Using Partitioning Rule* section and then click the **Add Log Time Unit Rule** icon, as illustrated in the following figure:



3. Click the **Edit Pattern** icon in the *Rule Pattern Definition* section and enter the pattern in the *Edit Rule Pattern* form, as illustrated in the following figure:



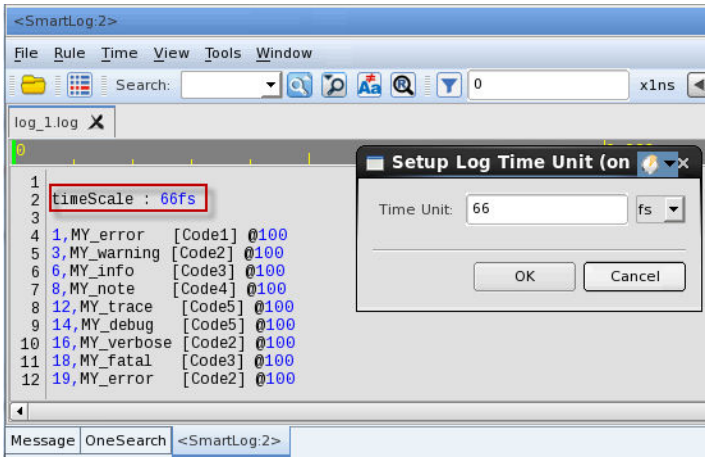
4. Double-click the *Type* column in the *Rule Pattern Definition* section and select *LogTime* for the defined pattern, as illustrated in the following figure:



5. Click the **Save As** button to save the log time unit rule.

   SmartLog searches the log file for the pattern specified in the newly created log time unit rule; if any line in the log file matches the pattern, the log time unit would be set automatically.

As an example (illustrated in the previous steps), the pattern of log time unit rule is: `timeScale : (\d+(fs|ps|ns|us|ms|s|FS|PS|NS|US|MS|S)` and the Type of *LogTime* is set to *`(\d+(fs|ps|ns|us|ms|s|FS|PS|NS|US|MS|S))`*. As a result, the timescale in the log file gets set to 66fs automatically, as illustrated in the following figure. You can view the log time unit for the log file using the **Time** -> **Setup Log Time Unit** menu option.

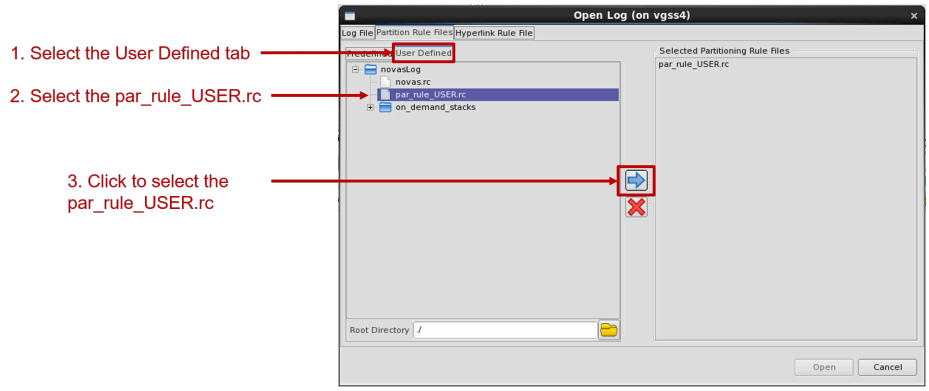


Backward Compatibility for Customized Partitioning Rule File Created Using Tcl Commands

If a customized/user-defined partitioning rule file is created using Tcl command entry, the `par_rule_USER.rc` file gets generated and saved in the `novasLog` directory.

You can open the customized/user-defined log file and apply the partitioning rule using the `par_rule_USER.rc` file by performing the following steps:

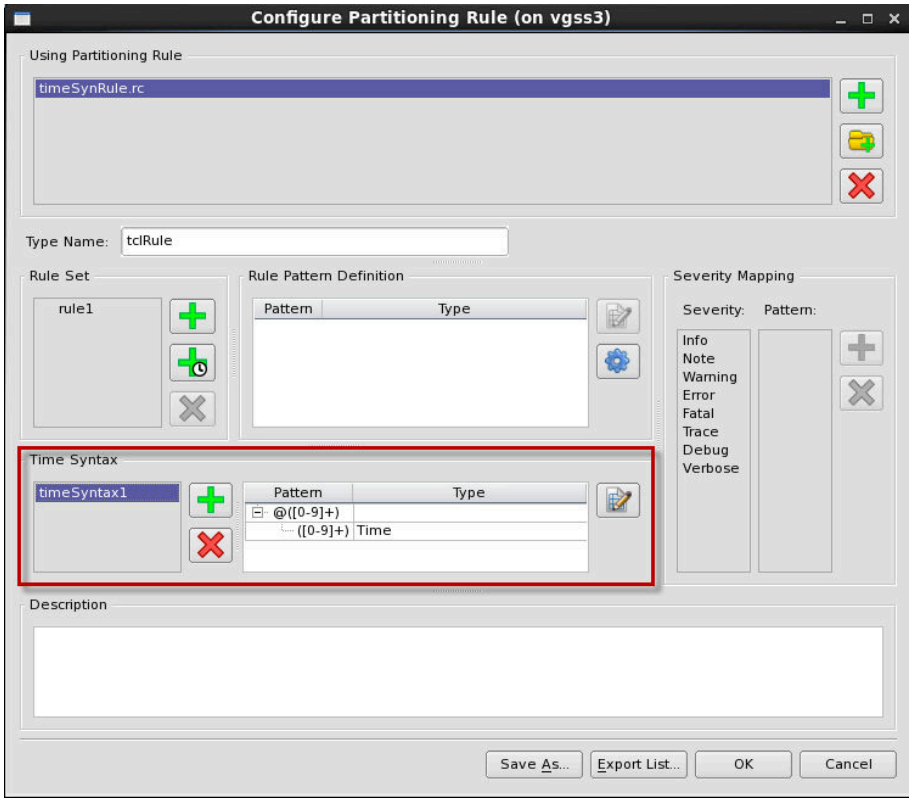
1. Open the `par_rule_USER.rc` file using the icon in the *SmartLog* pane.
2. In the *Open Log* form, select the *User-Defined* tab. In the *User-Defined* tab, click the `par_rule_USER.rc` file and click the icon to select the `par_rule_USER.rc` file.



3. Click the **Configure Partitioning Rule** icon present in the SmartLog toolbar. The *Configure Partitioning Rule* form opens that displays the `par_rule_USER.rc` partitioning rule and it's rule set.

Feedback

If time syntax is applied in the customized partitioning rule, the *Configure Partitioning Rule* form displays the **Time Syntax** field. This is illustrated in the following figure:



Applying Partitioning Rule

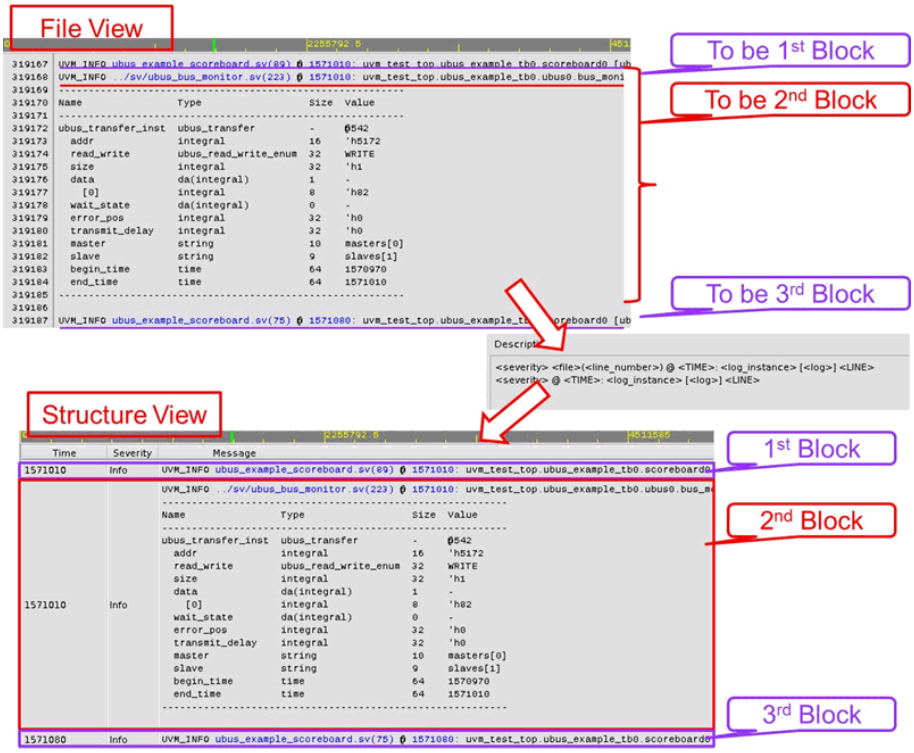
The *Structure View* in SmartLog groups and displays the log file based on the partitioning rule. If the line text in the log file matches the specified format in the partitioning rule, a block is created and the text in the following line that does not match the specified format belongs to the block.

As described in the beginning of this chapter, select one partitioning rule in the invoked *Select Partitioning Rule* form while loading a log file.

Figure 212 illustrates an example for a UVM log file applying the UVM partitioning rule.

Feedback

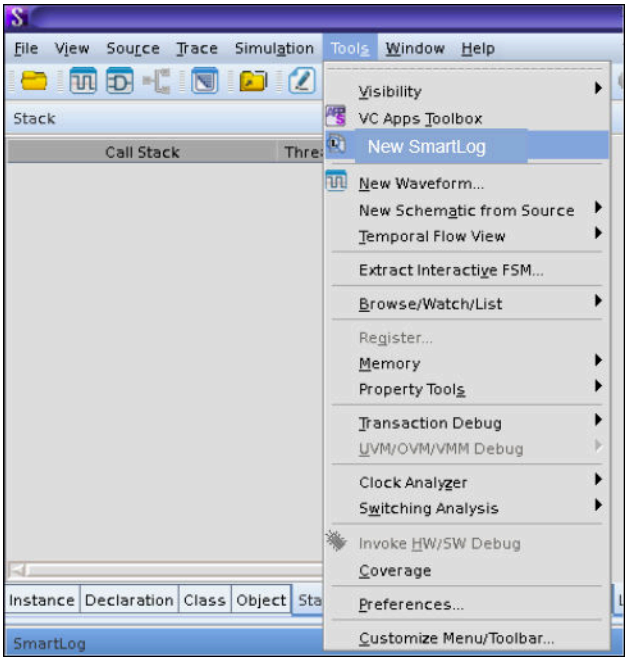
Figure 212 Applied Partitioning Rule



Opening Multiple SmartLog Windows and Synchronizing With nWave

You can also debug multiple logs and their corresponding waveforms at the same time by opening multiple SmartLog windows and synchronizing each log to a certain waveform.

To open a new SmartLog window, click the ***Tools*** -> **New SmartLog** menu option, as illustrated in the following figure:



Alternatively, you can open a new SmartLog window using the **New SmartLog** icon present on the Verdi toolbar.

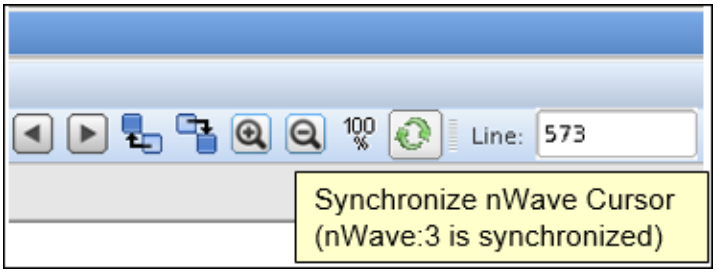
To enable or disable synchronization between multiple *SmartLog* panes and *nWave* panes, click the synchronization icon in the *SmartLog* pane.

If multiple nWave windows are opened, the *Sync to Waveform* form opens up (as illustrated in the following figure) and you can select the nWave window to be synchronized.

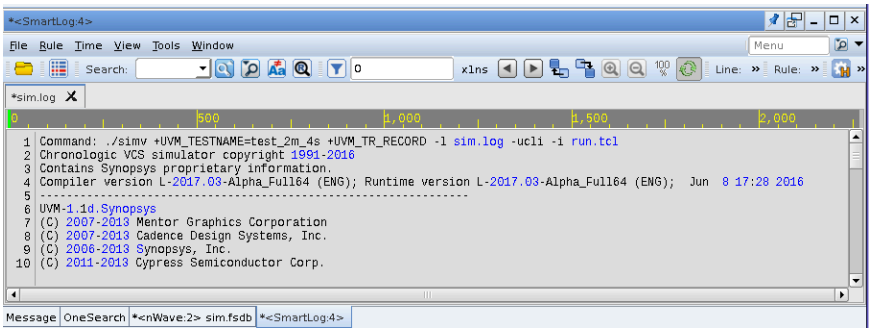
When enabling synchronization with nWave for the first time, by default, none of the synchronized waveforms is selected in the *Sync to Waveform* form; otherwise the previously specified nWave window is selected in this form.

Figure 213 Sync to Waveform Form

The tooltip of the synchronization icon shows the synchronization status of an already synchronized nWave window as illustrated in the following figure:

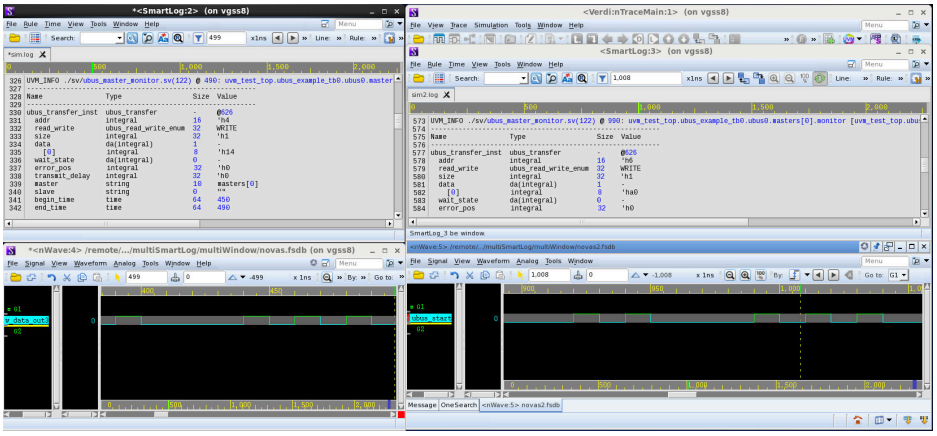


Additionally, the *SmartLog* window and the log file tab that are synced to the primary *nWave* window display the * symbol in the title, for example, `*<SmartLog:4>` and `*sim.log`.



The following figure illustrates an example of multiple SmartLog windows synchronized with multiple nWave windows:

Feedback



Locating Objects

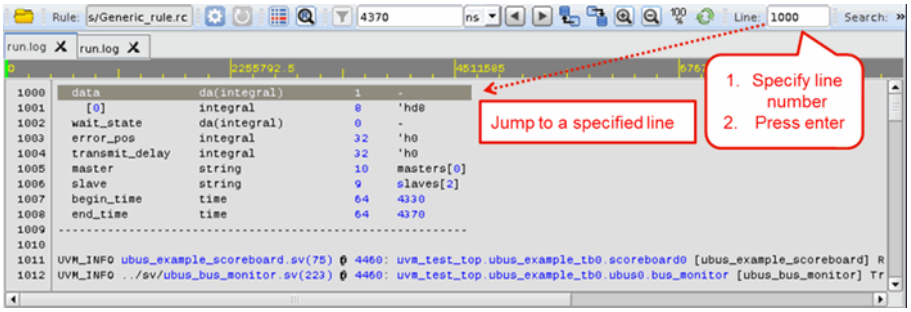
The following sections describe how to locate a specified point in the log file using SmartLog:

- Locating a Specified Line Number
- Locating a Specified Time Point
- Setting the Scroll Offset
- Synchronizing the Cursor in the nWave Pane
- Locating the Previous/Next Point

Locating a Specified Line Number

The line numbers are displayed in the *File View* of SmartLog. Specify a line number in the **Line** field and press **Enter** to jump the cursor to the specified line and highlight the same, as illustrated in Jumping to the Specified Line.

Figure 214 Jumping to the Specified Line



Feedback

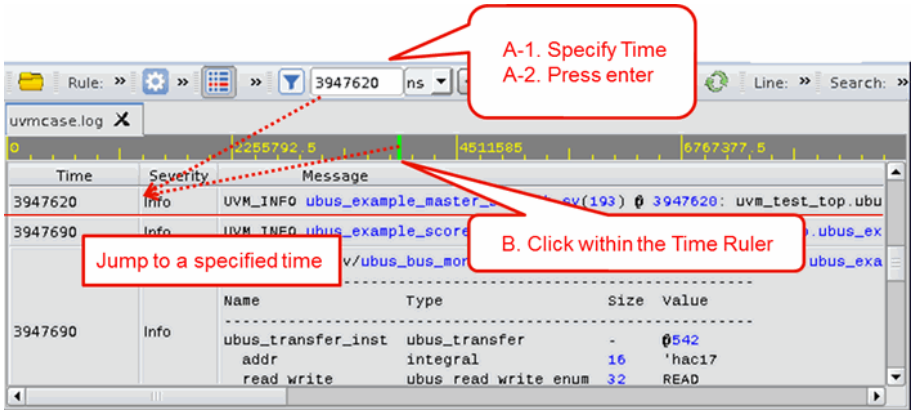
In *Structure View*, use this feature to jump to the corresponding line, however, the line number is not displayed. Specifying a line that is filtered out has no effect and no action is performed.

Locating a Specified Time Point

The **Time Cursor** field indicates the value of the time cursor in the **Time Ruler** section. You can jump the cursor to a block for which time is the closest and does not exceed the specified time using any of the following methods:

- Specify a time in the **Time Cursor** field and press **Enter**.
- Click a point within the **Time Ruler** section to move the time cursor.

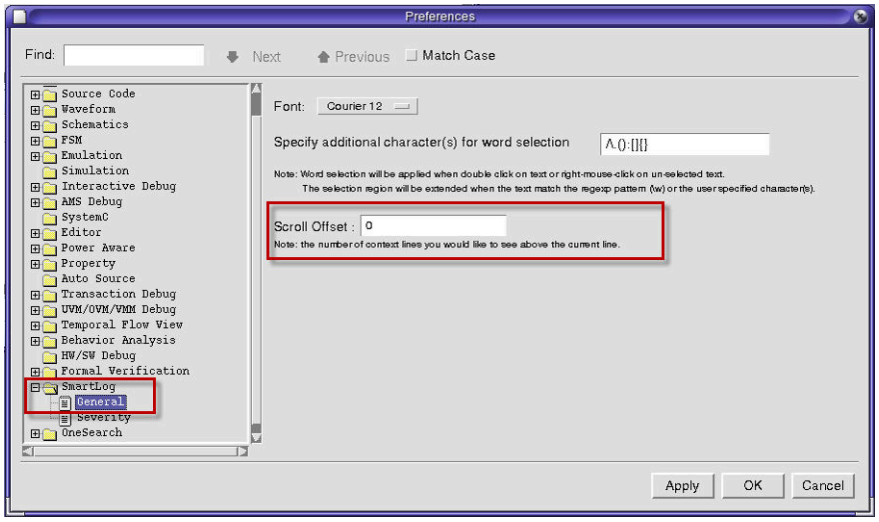
Figure 215 Jumping to the Specified Time



Setting the Scroll Offset

You can define or set the scroll offset as follows:

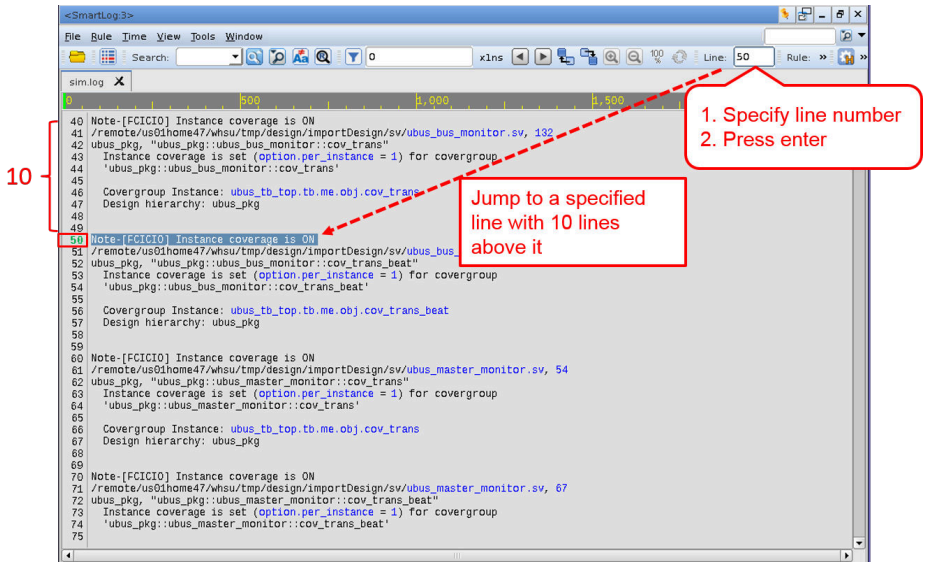
1. Click the **Tools** -> **Preferences** command to open the *Preferences* form.
2. In the **General** page in the **SmartLog** folder, specify the scroll offset in the *Scroll Offset* field, as illustrated in the following figure:



3. Click **Apply** and **OK** to save the settings.

Locating the Specified Line Number With Scroll Offset

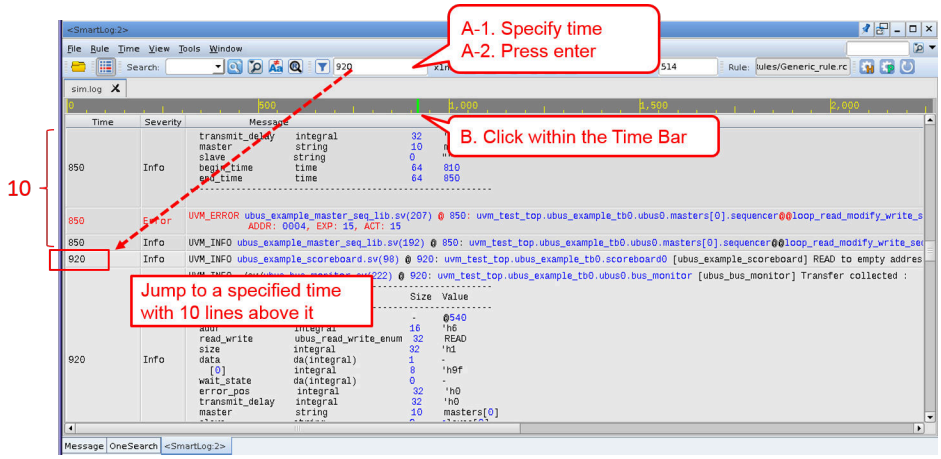
After the scroll offset is set, you can locate the line number specified in the *Line* field easily. For example, if the scroll offset is set to 10, the cursor jumps to the specified line with 10 lines above it. This is also illustrated in the following figure:



Locating the Specified Time Point With Scroll Offset

After the scroll offset is set, you can locate the time point specified in the *Time Cursor* field easily. For example, if the scroll offset is set to 10, the cursor jumps to the specified time with 10 lines above it. This is also illustrated in the following figure:

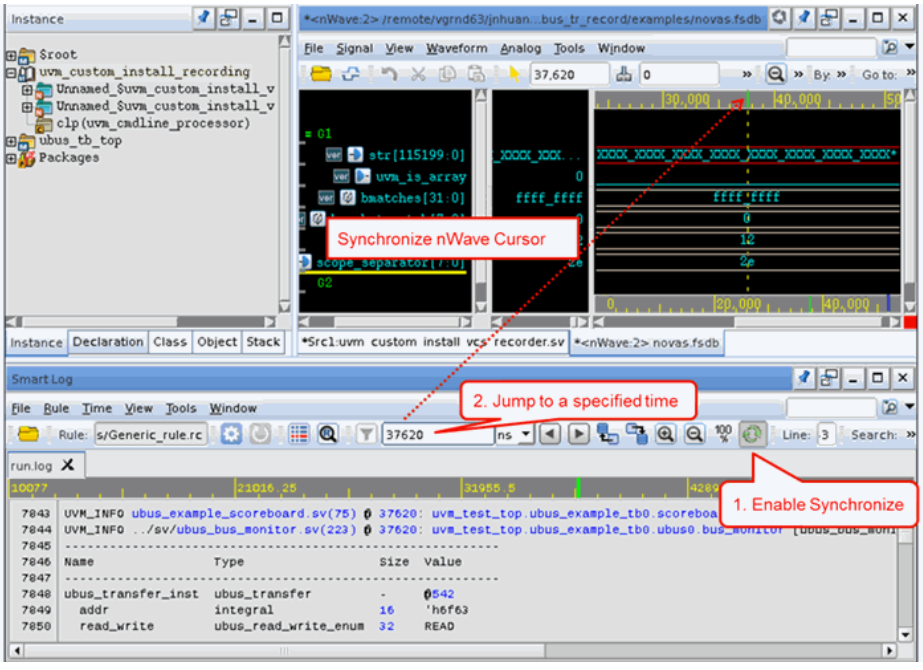
Feedback



Synchronizing the Cursor in the nWave Pane

Click the synchronization icon in the *SmartLog* pane to enable or disable synchronization between the time cursor in the *SmartLog* pane and in the *nWave* pane. When it is enabled, the *nWave* cursor is synchronized with the time specified in the **Time Cursor** field or with the time that is specified by clicking a point in the **Time Ruler** section and vice versa. This is also illustrated in Time Cursor Synchronization.

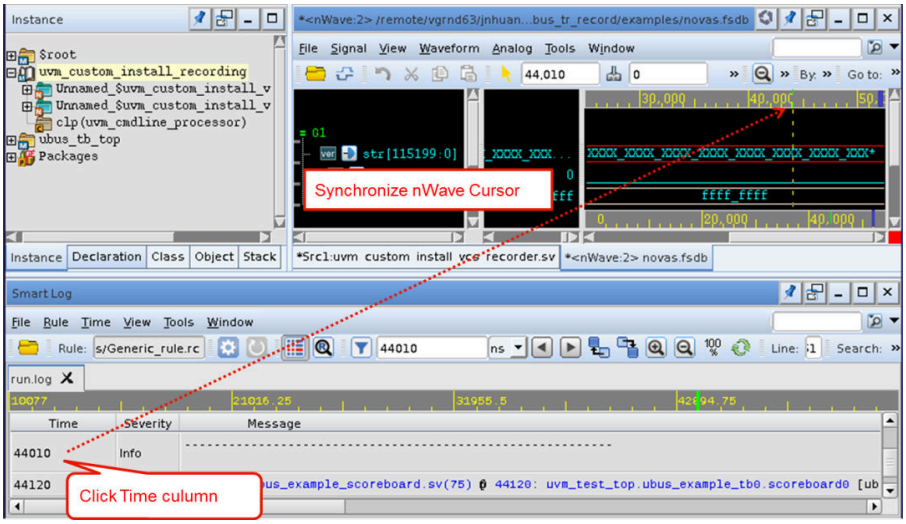
Figure 216 Time Cursor Synchronization



Feedback

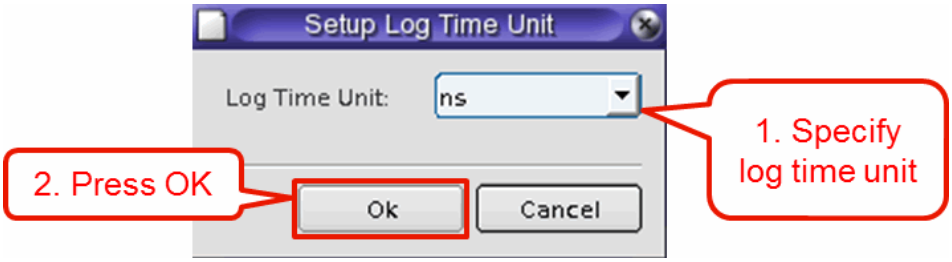
In *Structure View*, click the **Time** column to synchronize the time cursor in the *nWave* pane, as illustrated in Time Cursor Synchronization by Clicking Time Column.

Figure 217 Time Cursor Synchronization by Clicking Time Column



The default time unit of the log file is *ns*. Time unit mismatches between *nWave* and *SmartLog* panes causes synchronization mismatches. Configure the log time unit by using the **Time** -> **Setup Log Time Unit** command in SmartLog to invoke the *Setup Log Time Unit* form, as illustrated in Log Time Unit Configuration.

Figure 218 Log Time Unit Configuration



Locating the Previous/Next Point

Use the following features to jump the cursor to the previous/next time or block:

- Jump Cursor to Previous/Next time

  Different lines in the log file may have the same time information. The current time is the time of the first message (line) in the displayed content. Use or icons

Feedback

to jump to the previous or the next time for the first message in previous or next messages group with the same time.

- Jump Cursor to Previous/Next block

  Use or icons to jump cursor to the previous or the next blocks.

Searching, Filtering, and Reloading the Log File

The following sections describe how to search and filter out the content of the log file in SmartLog:

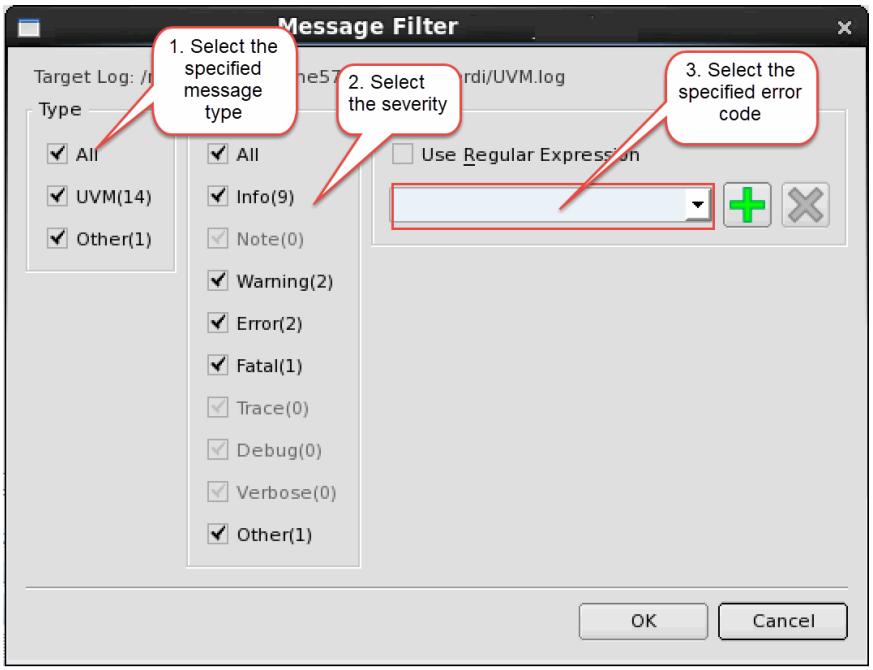
- Setting the Message Filter
- Setting the Time Filter
- Searching
- Displaying the Search Result Section
- Reloading the Log File

Setting the Message Filter

In *Structure View*, click the icon to filter out the log file with the specified message type, message severity, and error code. These three filter categories interact with AND logic and have level order. The higher level filter impacts the number of count in the lower level filters. The level sequence is: *Message type* > *Message severity* > *Error code*. As illustrated in Message Filter Setting, select the specified type, severity, and error in the invoked *Message Filter* form and click the **OK** button. SmartLog displays the blocks that match the filtering criteria in *Structure View*.

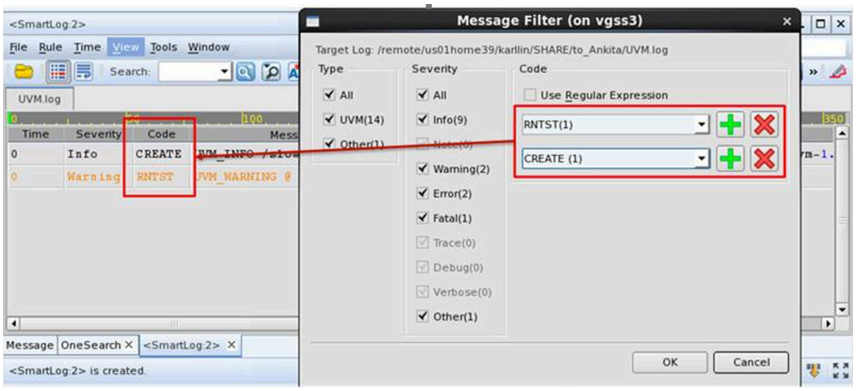
Feedback

Figure 219 Message Filter Setting



Select the *Use Regular Expression* check box to filter code by regular expression patterns. For example, you can enter TE to match **TE**ST and CREA**TE**. You can also click to create a new code filter and you can filter multiple codes simultaneously. Click to remove codes for filtering.

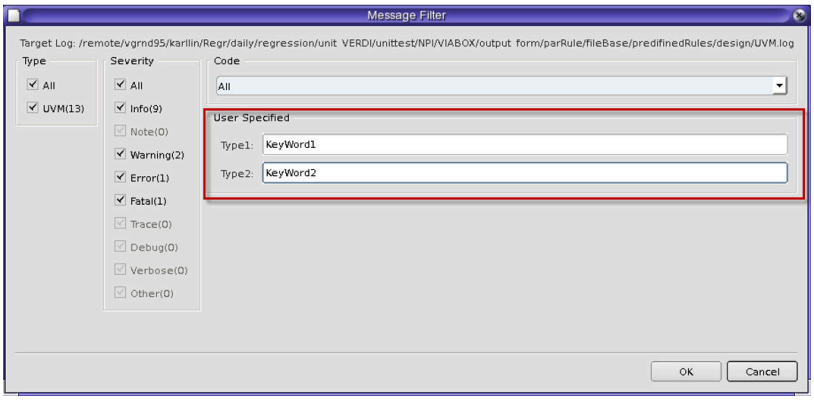
Figure 220 Message Filter-Multiple Codes Filter



If user specified columns are added in Smartlog, the *Message Filter* form shows the *User Specified* section, as illustrated in the following figure. You can filter the user specified columns as per the specified keywords. The number of filter objects within the User

Specified section is decided by the number of user-specified columns specified by the user.

Figure 221 Message Filter for User Specified Columns



Setting the Time Filter

Click , , or icons to zoom in/out/all the range of the time ruler.

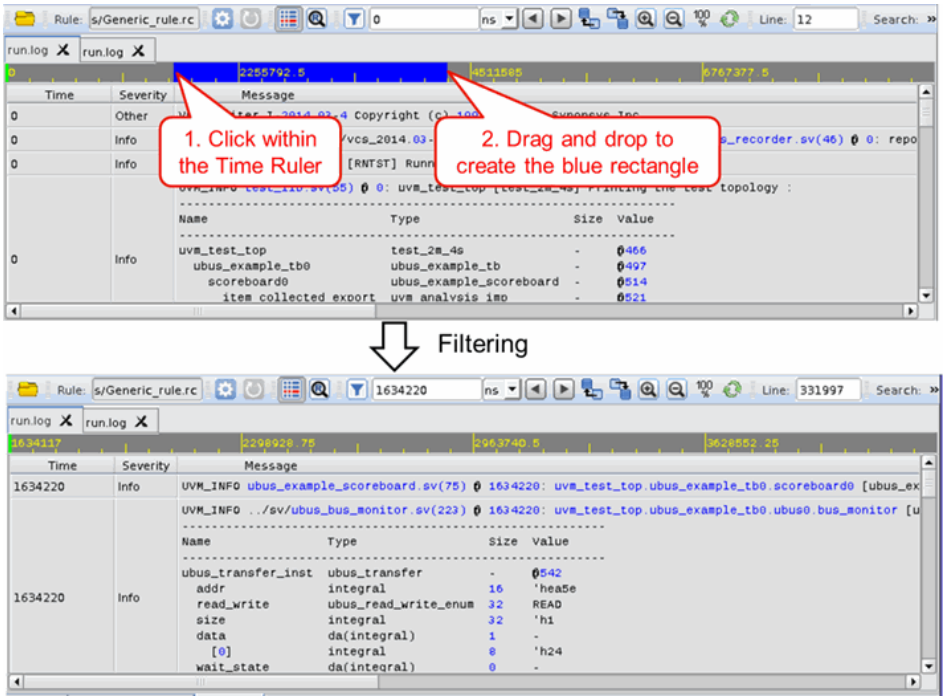
In *Structure View*, use the range of the time ruler to filter out a block if the time of the block is out of the range of the time ruler using the following method:

- Click within the *Time Ruler* section.
- Drag and drop the region to create a blue rectangle. The time ruler duration occupied by the blue rectangle is the new range of the time ruler.

This is also illustrated in Time Filter Setting.

Feedback

Figure 222 Time Filter Setting



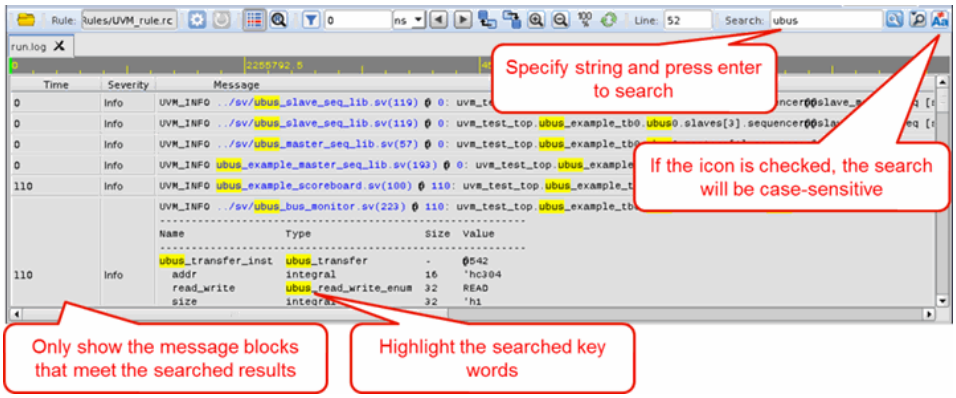
Searching

In *Structure View*, you can search the log file to highlight a string and filter out the blocks that do not contain the matched string. Specify a string (with or without the plus character (+) prefix) in the *Search* field and click the icon or press **Enter**. Only the blocks with messages that meet the searched criteria are displayed, as illustrated in Searching the Log File. Continue clicking the icon or press **Enter** to search the result in another line. Click the icon to go back to the previously searched result.

In *File View*, you can search a string without filtering the other content.

Feedback

Figure 223 Searching the Log File



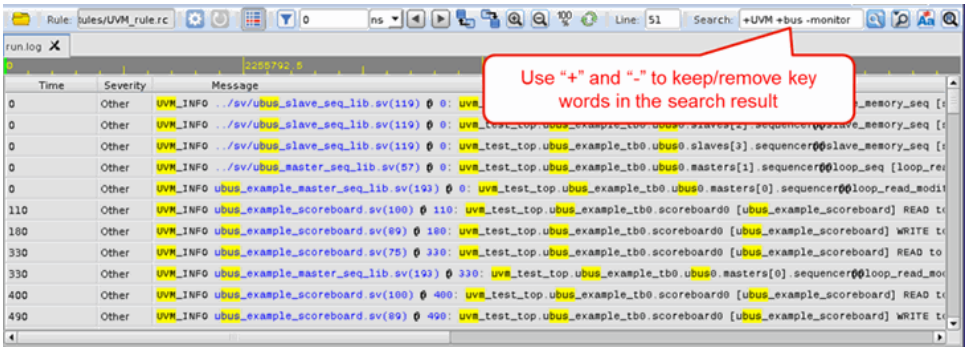
The option of searching with multiple strings is also available. The multiple strings are separated by space. To restore the original content, search an empty string.

Furthermore, to search the exception string, add the minus character (-) before the exception string, as illustrated in Searching With an Exception String. The string following the minus character (-) is considered as a must-not-have pattern for search.

Note:

If the plus character (+) is the prefix of a string, the string following the + character with the whole pattern is recognized as keyword of the inclusion string. The string is considered as a must have pattern for search. To search for the exact match of a string, enclose it in double quotes (such as, "test").

Figure 224 Searching With an Exception String



Example Use Cases

The following are some examples to demonstrate the search functionality:

- **Search string: +clk +module +top**

  SmartLog searches for the line that contains "clk", "module", and "top". It highlights "clk", "module", and "top" for the matched line.

Feedback

- **Search string: clk module top**

  SmartLog searches for the line that contains "clk" or "module" or "top". It highlights "clk", "module" and "top" for the matched line.

- **Search string: -clk -module -top**

  SmartLog searches for the line that does not contain "clk", "module", and "top". It highlights nothing.

- **Search string: +clk +module top**

  SmartLog searches for the line that contains "clk" and "module". It highlights "clk", "module" and "top" for the matched line.

- **Search string: +clk +module -top**

  SmartLog searches for the line that contains "clk" and "module", but not "module". It highlights "clk" and "module" for the matched line.

- **Search string: clk module -top**

  SmartLog searches for the line that contains "clk" or "module", but not contain "top". It highlights "clk" and "module" for the matched line.

- **Search string: +clk module -top**

  SmartLog searches for the line that contains "clk" but not "top". It highlights "clk" and "module" for the matched line.

- **Search string: "A = 3"**

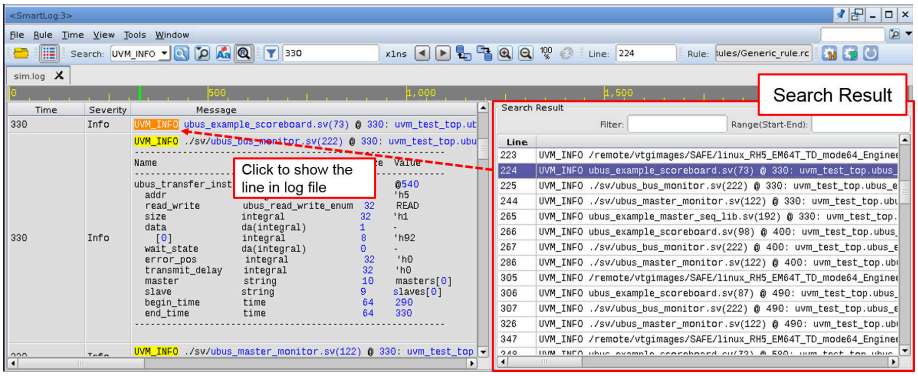
  SmartLog searches for the line that contains "A = 3". It highlights "A = 3" for the matched line.

Displaying the Search Result Section

Click the icon to display the *Search Result* section that shows the lines containing the matched string, as illustrated in Search Result Section.

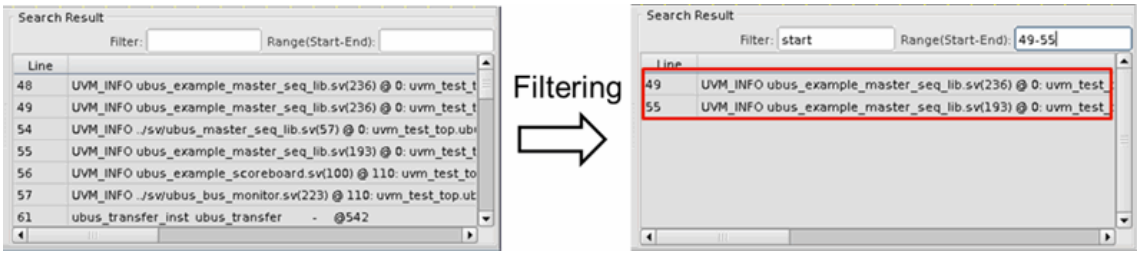
Feedback

Figure 225 Search Result Section



In the *Search Result* section, click a row to highlight it in the log file. The search result can be further filtered by specifying a string in the **Filter** field. Set the numbers of the start and end lines in the **Range (Start-End)** field to view the search result with the specified range, as illustrated in Filtering Search Result.

Figure 226 Filtering Search Result



Reloading the Log File

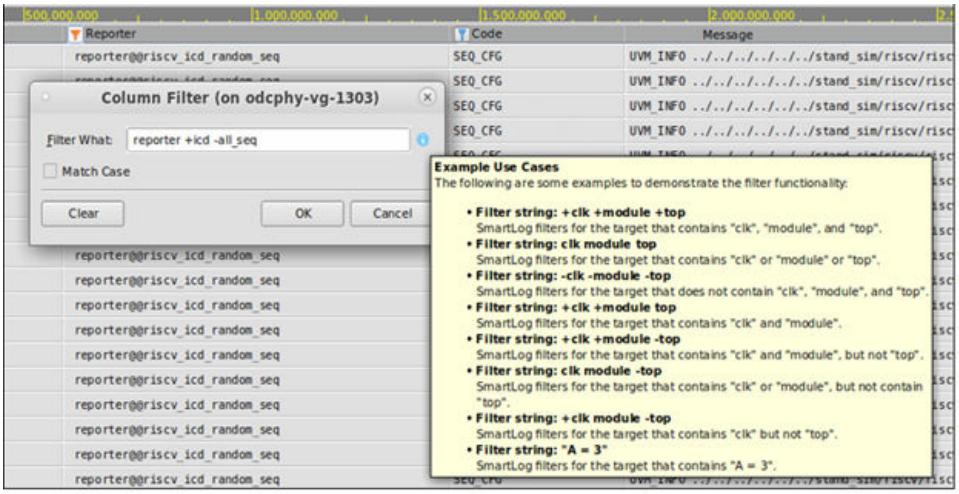
If a log file is changed or appended, you can reload the log file using the **Refresh Log** icon present in the SmartLog toolbar or using the **Rule -> Refresh Log** command. The log file gets refreshed with the original hyperlink/partitioning rule settings. SmartLog automatically refreshes the log when the hyperlink/partitioning rule changes.

Setting the Column Filter

The column filter supports filtering on certain columns, thereby makes filtering more flexible. Refer to following screenshot to get its entry and tip notes.

Feedback

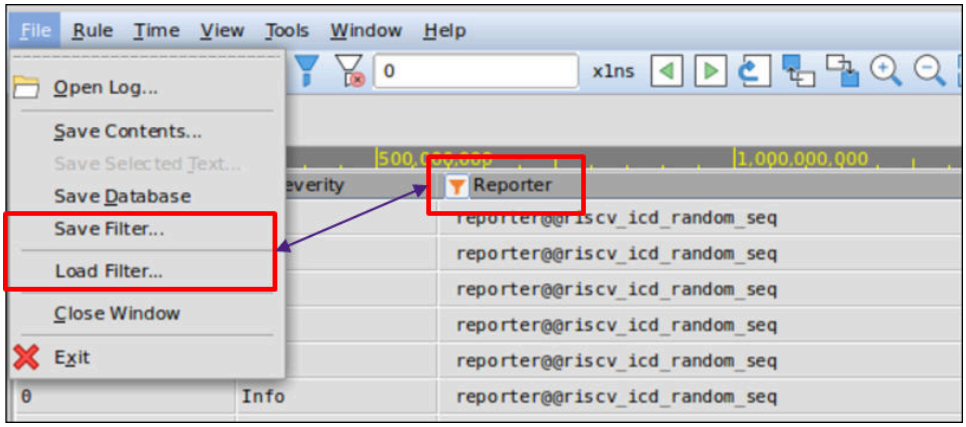
Figure 227 Column Filter



Saving or Loading a Filter Result

You can save your filter condition by using the **File**-> **Save Filter**. You can also reuse the filter condition by using **File** -> **Load Filter** during debugging the next time.

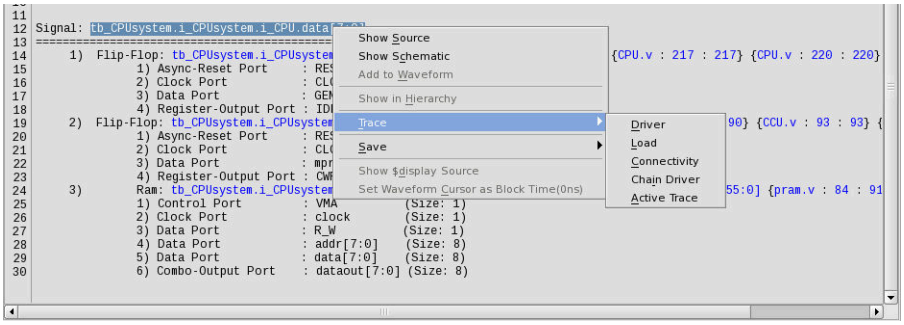
Figure 228 Save or Load Filter



Debugging in Verdi Frames

Use the *right-click command menu* to display the object in the corresponding panes for a selected signal or an instance.

Feedback



Use the following steps to debug a selected item:

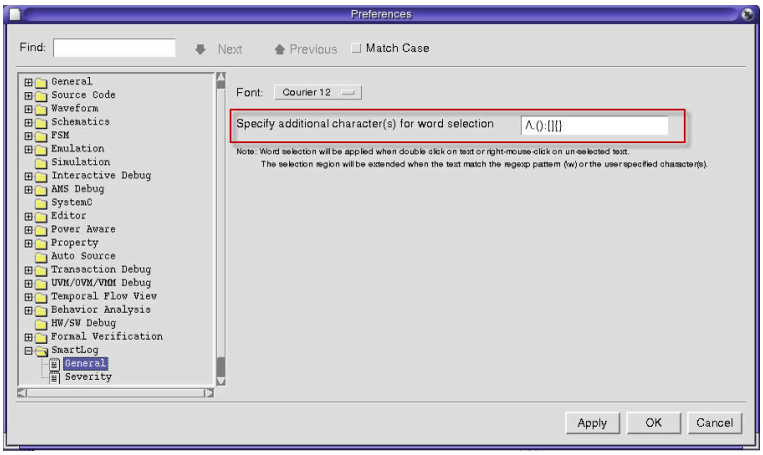
1. Select the full hierarchy path of a signal or an instance string.

   Alternatively, right-click on a full hierarchy path of a signal or an instance string and the string is selected automatically.

   You can change the auto-selection rule for a string as follows:

   a. Click the **Tools -> Preferences** command to open the *Preferences* form.

   b. In the **General** page in the **SmartLog** folder, specify the additional characters in the **Specify additional character(s) for word selection** field, as illustrated in the following figure:



Note:

Auto-selection rule on a word is applied when you double-click on a word/text or right-click on a word/text. The auto-selection region is extended when the text matches the regular expression pattern (\w) or the character(s) specified by users in the *Preferences* form.

Feedback

For example, if you right-click on the `g` character of the `Signal[0:9]` string,

- The word `Signal` gets selected, if additional characters are not specified in the *Preferences* form.
- The word `Signal[0:9]` gets selected, if additional characters specified in the *Preferences* form are*:[ ]*

2. Use the **Show Source** right-click menu option to show the source code in the *source code* pane.
3. Use the **Show Schematic** right-click menu option to show and highlight selected signal or instance in the *nSchema* pane.
4. Use the **Add to Waveform** right-click menu option to add selected signal or scope to the *nWave* pane.
5. Use the **Show in Hierarchy** right-click menu option to show the selected instance in the hierarchy tree of the *Verdi Instance* tab.
6. Use the following **Trace** right-click menu option to perform the tracing features for the selected signal in the Verdi platform:
   - **Driver**: Traces all possible drivers for the selected signal.
   - **Load**: Traces all possible loads for the selected signal.
   - **Connectivity**: Traces all possible drivers and loads for the selected signal.
   - **Chain Driver**: Traces through buffer/inverter chains and stops at the first non-single input driver.
   - **Active Trace**: Locates the active driver in the *source code* pane for the selected signal.
7. Use the following **Save** right-click menu options to perform the saving features:
   - **Contents**: Saves the entire content of the log file into a text file.
   - **Selected Text**: Saves the selected text of the log file into a text file.
8. When the *SML* file exists, use the **Show $display Source** right-click command to show the source code in the *source code* pane which prints the selected message.
9. Use the **Set Waveform Cursor as Block Time** right-click command in a block to synchronize the time of the block to the cursor time of the *nWave* pane.

Feedback

The drag and drop feature is also available. Drag one of the following texts from SmartLog and drop it to other panes in the Verdi platform:

- A signal with full hierarchy path
- An instance with full hierarchy path
- The Top module

Using SmartLog in Interactive Debug

SmartLog is also available in *Interactive Console* in the Verdi platform and the new Interactive console is enhanced with SmartLog engine. If the VCS simulator is used to generate an SML file, you can also use it with the SML file.

Note:
This is available starting with the Verdi J-2014.12-SP1 version.

The following sections describe how to invoke SmartLog in interactive debug in Verdi and how to use its features:

- Invoking Interactive Debug
- Command Entry
- Scroll Bar Behavior
- Applying User-Defined Partitioning Rules

Invoking Interactive Debug

You can invoke Verdi interactive debug with `-i` and `-simType` options in the Verdi command line. For example:

```
%> verdi -sv test.sv -i -simType VCS
```

To invoke interactive debug with the VCS-SML flow, add the `-sml=verdi` simulation option. For example:

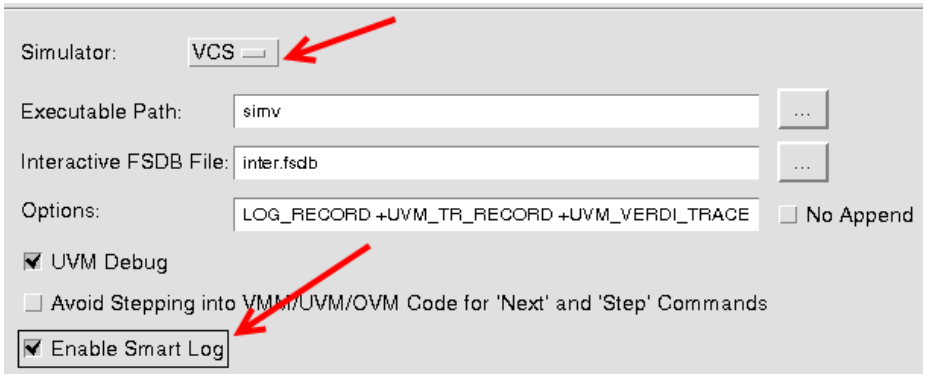
```
%> verdi -sv test.sv -i -simType VCS -simOpt "-sml=verdi"
```

Alternatively, configure interactive simulation settings in the **Simulation** page of the *Preferences* form as follows:

1. Use the **Tools -> Preferences** menu command to invoke the *Preferences* form.
2. Select the **Simulation** page and enable the **Enable SmartLog** option.
3. To invoke the VCS-SML flow, select VCS as the simulator in the **Simulator** field.

Feedback

Figure 229 Enable SmartLog from Preferences



4. Use the **Simulation -> Invoke Simulator** command to enable Interactive Simulation mode.

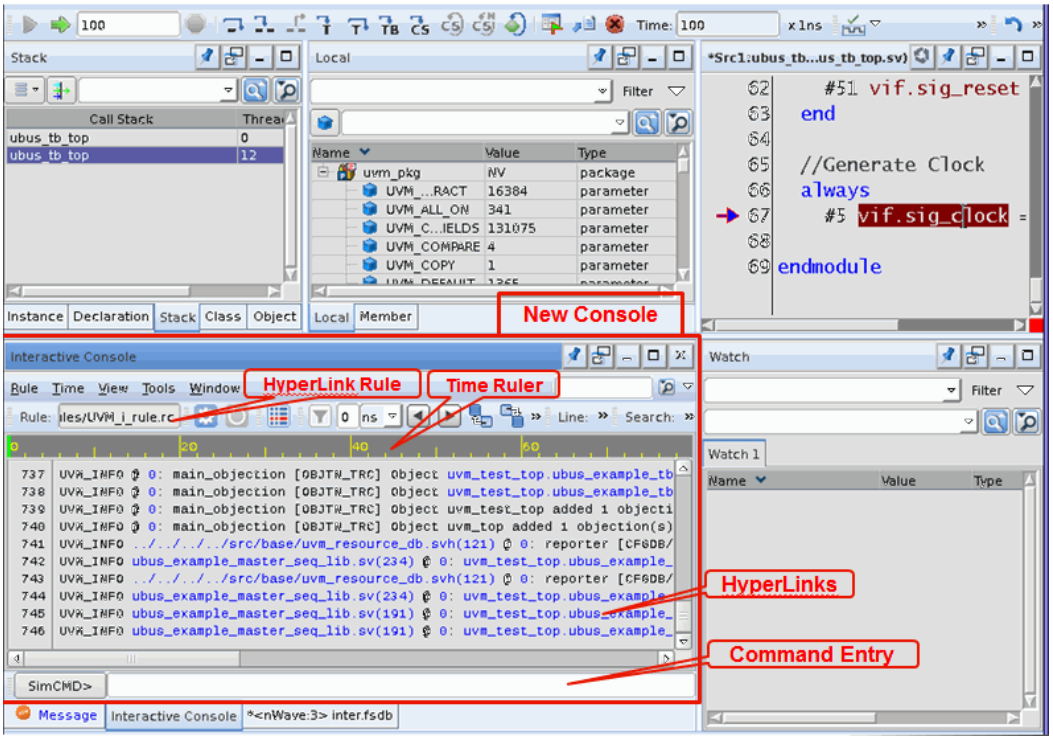
SmartLog is invoked and the heading of the pane is *Interactive Console*. The predefined UVM hyperlink rule stored in the *UVM_i_rule.rc* file is automatically applied to create the hyperlinks.

If VCS-SML flow is enabled, the partition rule in the *Interactive Console* frame follows the rules recorded in the VCS generated SML file.

If the `-sml=verdi` option has not been specified, all the predefined partition rules are considered as partition rules.

Feedback

Figure 230 Interactive Console of SmartLog



The synchronization icon is turned *on* by default, that is, the cursor time of *nWave* and *Interactive Console* panes are synchronized by default.

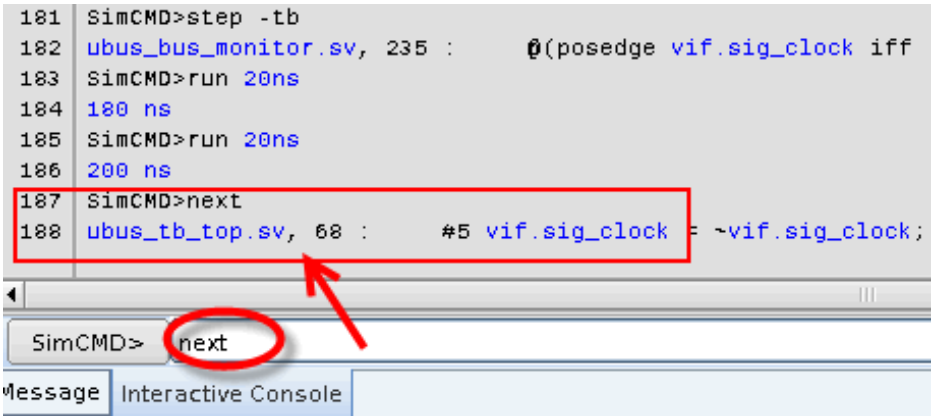
You can start to debug your design and monitor the log with SmartLog in the Interactive Simulation mode.

Command Entry

At the bottom of the *Interactive Console* pane, as illustrated in Interactive Console of SmartLog, enter the simulation commands in the **SimCMD>** field and press **Enter** or click the **SimCMD>** button to execute the commands. The command string and returned result are displayed in the *Interactive Console* pane.

Feedback

Figure 231 Command Entry



To get the historical typed-in commands, use up "↑" and down "↓" keys in the command entry.

Scroll Bar Behavior

When Interactive Mode is enabled, behavior of the *scroll bar* in the *File View* is as follows:

- While the simulation is running, scroll up the *scroll bar* to read previous lines.
- When simulation output is received in the *Interactive Console* pane and the *scroll bar* is not in the bottom, the output message is printed without scrolling down to the bottom.
- If a command is entered in the command entry, the command output is appended and the *scroll bar* is scrolled to the bottom.

Applying User-Defined Partitioning Rules

In Interactive Mode, you can apply the user-defined partitioning rules by performing the following steps:

1. Specify a directory to save the user-defined partitioning rules under the `[VIA.parRule]` section with the `parRulePathInterForm` key. The default directory is ""; Interactive console does not use user-defined partitioning rule file if the default value is "".
2. Change the directory using the *novas.rc* configuration file.
3. Interactive console uses the predefined partitioning rule files and all the rule files under the user-defined directory to perform partitioning.

Feedback

Example

Predefined Rules ($Verdi_path/share/VIA/Apps/PreDefinedParRules/):

```
par_rule_UVM.rc
par_rule_OVM.rc
par_rule_VCS.rc
par_rule_LP.rc
```

User-Defined Rules (/CAD/IAParRule):

```
myRule_1.rc
myRule_2.rc
```

If you do not specify *novas.rc*, Interactive console uses the predefined rules (mentioned above) and *novas.rc* (mentioned here).

novas.rc

```
...
[VIA.parRule]
parRulePathInterForm = ""
...
```

If you specify novas.rc, Interactive console uses both predefined and user-defined rules as follows:

```
myRule_1.rc
myRule_2.rc
par_rule_UVM.rc
par_rule_OVM.rc
par_rule_VCS.rc
par_rule_LP.rc
```

Additionally, Interactive console uses *novas.rc* as follows:

novas.rc

```
...
[VIA.parRule]
parRulePathInterForm = "CAD/IAParRule/"
...
```

Known Issues and Limitations

The following known issues and limitations exist in the *SmartLog* feature:

- When multiple keywords are entered in the search field, if the second or later keywords are numbers, the line number of the log file is considered as a part of context and is searched. For example, if "UVM_INFO 200" keywords are entered in the search field,

Feedback

the lines of the blocks meeting the following situations are displayed in the search result:

- The line with the 'UMV_INFO' string
- The line number of the line with the '200' string

- To solve performance issues caused by regular expressions, a maximum number of characters in each hyperlink line of the *SmartLog* pane should be specified. If a character number exceeds the specified value, the hyperlink in the line is not available (that is, only text is shown). The default is 4096. The following environment variable is set to modify the maximum number:

```
%> setenv VGIF_MAX_LENGTH_PER_LINE 4096
```

11

OneSearch Tutorial

This chapter describes the following topics:

- Overview
- GUI Use Model
- Search Domains
- Search Modes
- Support for Multiple Line Results
- Support for Synonyms
- Command Line Use Model
- Usage Examples
- Find Bar

Overview

The OneSearch capability in Verdi works like a web search engine and helps you to search through various aspects or domains within your design. The OneSearch interface is simple, intuitive, and easy to use; wherein you can enter a search query and get ranked and sorted results, view the results, modify your query, and refine your search.

OneSearch supports search in **Sources**, **Logs**, **Identifiers**, and **Docs** domains with domain-specific search engines, generates ranked matches, and stores results in domain-specific results file.

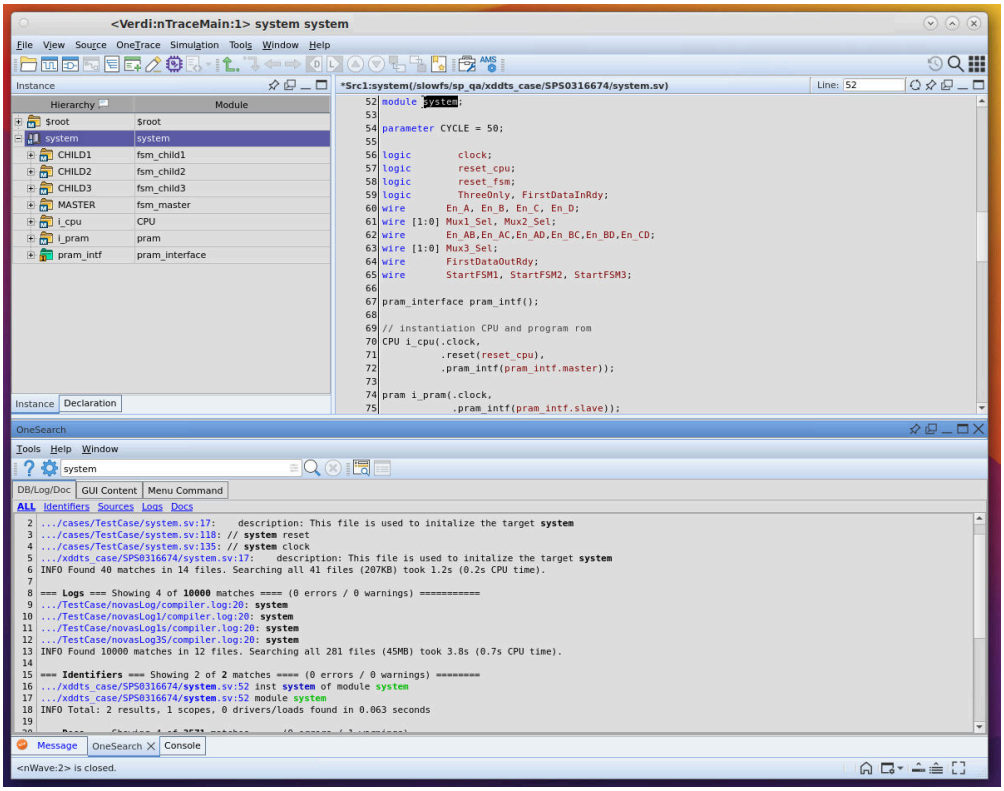
OneSearch also supports search in **DB/LOg/Doc**, **UI Content**, and **Menu Command**.

OneSearch is also aware of the current Verdi *Work Mode* (coverage/power debug and so on) and generates results based on the work mode. For example, the coverage search engine is available only in the Coverage mode.

Feedback

The following figure illustrates the default view of the OneSearch pane that appears in the Verdi GUI:

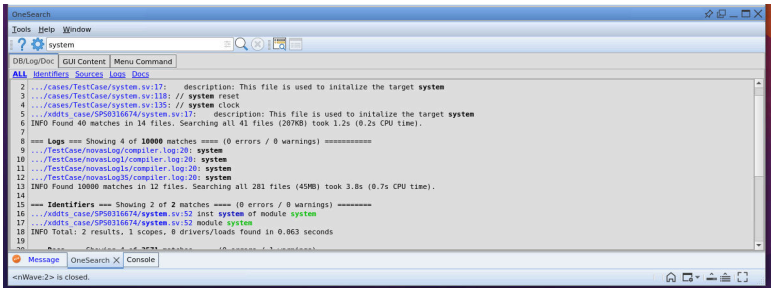
Figure 232 OneSearch Default View



GUI Use Model

The **OneSearch** tab appears by default when the Verdi GUI is invoked.

To open the OneSearch pane, click the **OneSearch** tab as illustrated in the following figure:



To search for a keyword using OneSearch capability,

- Enter the required keyword or search query (such as, "clk", "clk module Top" and "clk files:*.sv") in the left text field in the *OneSearch* pane.
- Click the icon or press the **Enter** key.

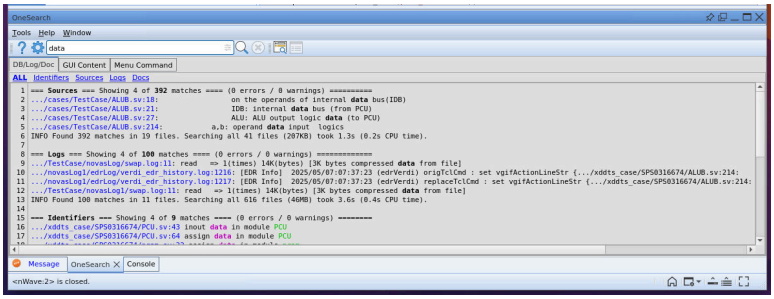
The top ranking matches for each domain are displayed in the **ALL** domain.

To view more matches and more information for each match in a specific domain, click the required domain from **Sources**, **Logs**, **Identifiers**, or **Docs link**.

Alternatively, you ca also invoke OneSearch using **Tools –> OneSearch** menu command.

The following figure illustrates an example of the results appearing in the OneSearch pane:

Figure 233 Example of OneSearch Results



Note:

To view the help information for OneSearch, click the **Show Help** icon present in the OneSearch toolbar. Clicking on this icon in the *Overview Mode* displays the help information in the **All** domain OneSearch results pane. Clicking on this icon in the *Single-domain Mode* displays the help information in the selected domain OneSearch results pane.

You can apply preference settings for OneSearch using the **Show Preferences** icon. Clicking on this icon opens the *Preferences* form (*OneSearch* page). For details, see the *OneSearch* chapter in *Verdi and Siloti Command Reference Guide*.

Feedback

Search Domains

The search domains for OneSearch capability are described as follows:

- **Sources**

  The **Sources** domain covers all (System)Verilog, VHDL, or other input files used in the design. The file list is extracted from the KDB (if available) or the VCS elaboration (if available) files. If none of these files is available, all files matching \.*[ch]*|\.*[ch]pp*|\.*[ch]xx*|\.*(cc|hh)*|\.*svh*|\.*pli*|\.*v*|\.*vrh*|\.*sv*|\.*sva*|\.*vhdl*|\.*vhd* and *Makefile*|*synopsys_sim*\.*setup*|\.*tcl*|\.*[a-z]?sh*|\.*upf* are searched.

  To specify an additional file list, use the `VERDI_SEARCH_SOURCES_FILE` environment variable.

  file: and *files:* are the supported keywords to filter the list of files matching the specified pattern only (such as, *files:a*.sv* matches only files starting with *a* and ending with *.sv*).

- **Logs**

  The **Logs** domain covers log files generated from compilation and simulation matching ucli\.key|.*\.rpt|.*\.log(\.gz).

  To specify an additional file list, use the `VERDI_SEARCH_LOGS_FILE` environment variable.

  file: and *files:* are supported keywords to filter the list of files matching the specified pattern only (such as, *files:sim** matches only files starting with *sim*).

- **Identifiers**

  The **Identifiers** domain covers HDL design and testbench elements of VCS simulation. Any identifier (that is, anything with a name) in the design is searched.

- **Docs**

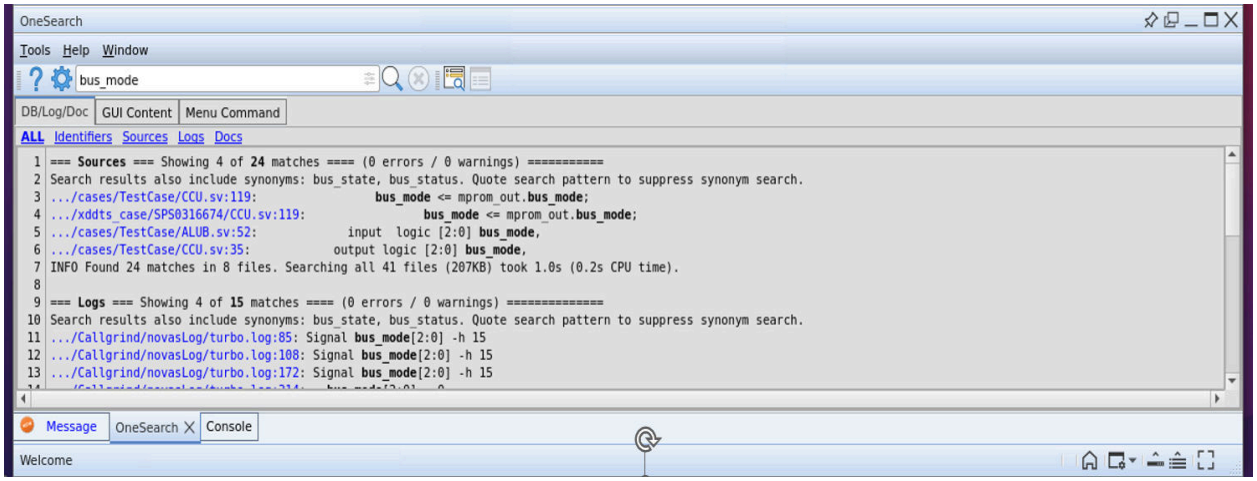
  The **Docs** domain covers all Verdi documentation (PDF) files.

  To specify an additional file list, use the `VERDI_SEARCH_DOCS_FILE` environment variable.

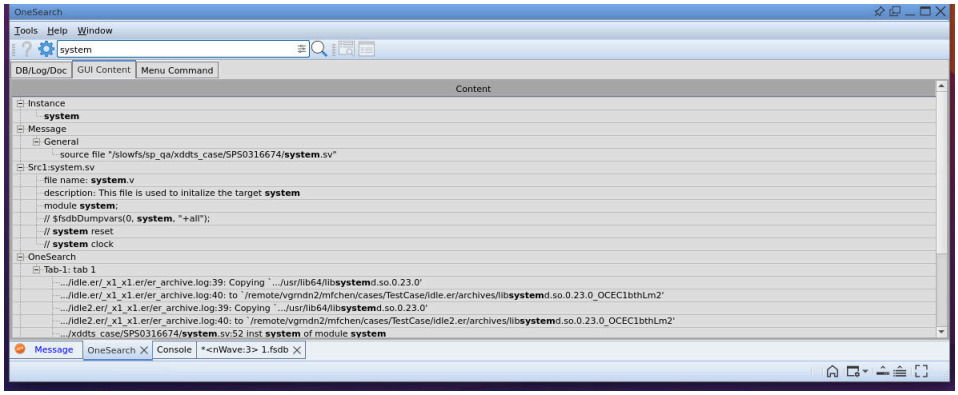
  file: and *files:* are supported keywords to filter the list of files matching the specified pattern only (such as, *files:*plan** matches only files containing *plan*).

Feedback

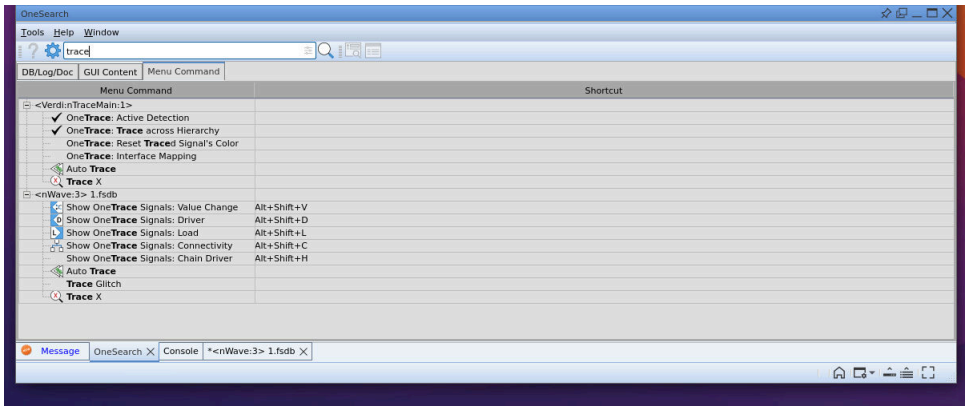
- **DB/Log/Doc**



- **GUI Content**



- **Menu Command**



Feedback

Search Modes

OneSearch supports the following search modes that decide where to search and how to display the search results:

- Overview Mode
- Single Domain Mode

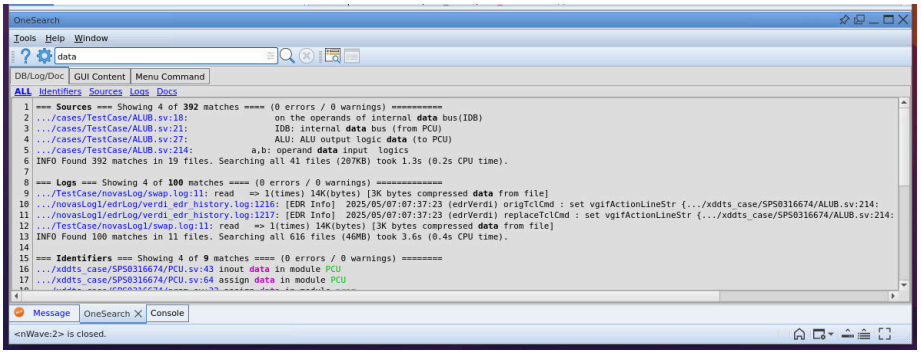
Overview Mode

This is the default mode for OneSearch.

This mode is intended as the first step in a search and provides a quick overview of the results in the entire design. In this mode, scrolling in the OneSearch results window moves entire domain groups but does not move matches within a group. When you refine (modify) the search query, such as, by adding another pattern to the query, and click the search button again, then a new search over all the domains is started.

Every group ends with a summary line that indicates the total number of results (not just the visible count). It also contains an indicator for ongoing searches.

Figure 234 Overview Mode



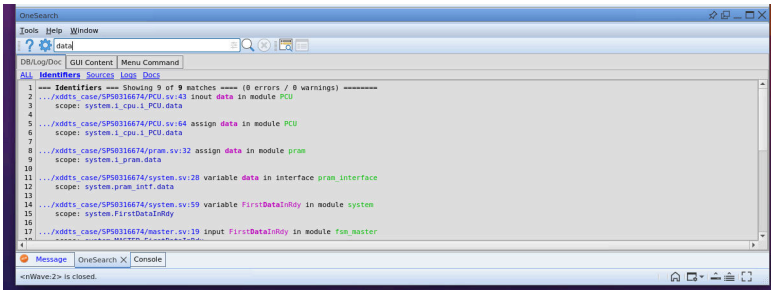
Single Domain Mode

This mode is intended as the second step to view more matched results from a single domain. To switch from Overview mode to Single-domain mode, click on any of the domain links: **Sources**, **Logs**, **Identifiers**, or **Docs**.

On clicking on a domain name, OneSearch does not trigger a new search but displays the matches from the previous search for the selected single domain. All matches from the selected domain are displayed.

Feedback

Figure 235 Single Domain Mode



Support for Multiple Line Results

OneSearch ranks the matches for documentation not only within a single line, but also combines the matches for multiple lines based on the distance between the matching words; therefore, multiple matches within a paragraph are assigned a higher ranking. A configurable upper limit ensures that these words are not too far from each other.

Overview Mode

In Overview mode, Verdi only displays the first line of the search result, which contains a brief summary of all matches.

The following code snippet illustrates an example output in overview mode for the search term: `loading`.

```
=== Docs === Showing 3 of 278 matches ==== (0 errors / 0 warnings) =====
/verdi/doc/CoverageTut.pdf:p4 (text): when loading succeeds …the loading
 times…when loading fails …

/verdi/doc/CoverageTut.pdf:p1 (text): After loading an exclusion…
 after loading related...

/verdi/doc/FsdbReader.pdf:p118 (text): and loading fails. It is therefore
 highly recommended
INFO Found 278 matches in 24 files. Searching all 24 files (12MB) took
 0.7s.
```

A four line, a two line, and a single line result is illustrated in the preceding snippet.

- Single-line results are displayed in the same manner as in previous releases (see result 3).
- Multi-line results include a brief summary of the matches to fit a maximum of 3 highlighted matches into the line. This limit is imposed by onegrep due to space limitations (see result 1).

Feedback

The following code snippet illustrates an example output in overview mode for multiple search terms:

```
=== Docs === Showing 4 of 518 matches ==== (0 errors / 0 warnings) ======
/verdi/doc/CoverageTut.pdf:p36 (text): reviewed after the reset… property
 of this module…
/verdi/doc/CoverageTut.pdf:p96 (text): After resetting an exclusion file…
 The property of the system to…
/verdi/doc/FsdbReader.pdf:p18 (text): when the reset property of…
 when reset succeeds… another property is…
/verdi/doc/FsdbReaderSummary.pdf:p1 (text): A property of an FSDB…
 incomplete resetting of …
INFO Found 518 matches in 24 files. Searching all 24 files (12MB) took
 0.7s.
```

Single-Domain Mode

In Single-domain mode, Verdi displays the complete details of the search result.

The following code snippet illustrates an example output in single-domain mode for the search term: loading.

```
=== Docs === Showing 278 of 278 matches ==== (0 errors / 0 warnings) ====
/verdi/doc/CoverageTut.pdf:p4 (text):

when loading succeeds …the loading  times…when loading fails … handled
 otherwise?
The main idea is that when loading succeeds, the system has to take to
 the loading times into account. Otherwise,
loading is deferred until the system has decided based on this property.
 In general, loading of these data structures is based on

/verdi/doc/CoverageTut.pdf:p1 (text):

After loading an exclusion… after loading related...by the system, which
 is done automatically.
After loading an exclusion file and after loading related data (see
 section 5.3.1), it

/verdi/doc/FsdbReader.pdf:p118 (text):

and loading fails. It is therefore highly recommended
[the other 275 matches are truncated in this figure]
```

Support for Synonyms

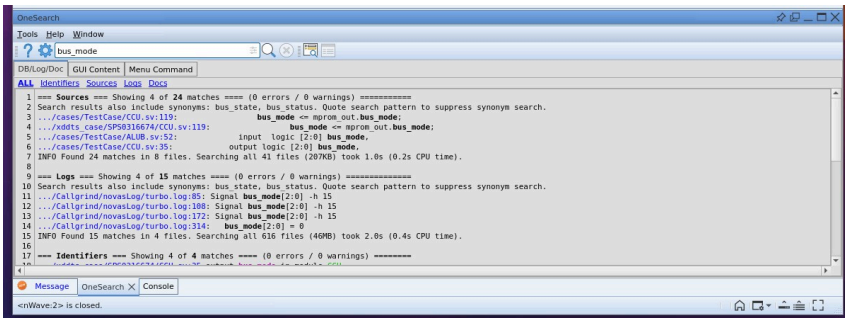
OneSearch automatically searches synonyms for terms in search queries based on a predefined synonym database. This functionality provides you the capability to search results for words with the same (or very similar) meaning.

Feedback

Note:
This feature is applicable to **Sources**, **Logs**, and **Docs** domains.

If the search query entered by you contains terms for which synonym(s) are defined in *OneSearch*, an additional message appears in the *Results* pane indicating that the search results also include synonym(s)' search results. If you do not want to view the search results for the indicated synonym(s), you need to enclose the search term in double quotation marks (""). An example of the message is illustrated in the following figure:

Figure 236 Example for OneSearch Query



Note:
Quoted search terms as well as keywords and their arguments are not subject to synonym search. OneSearch ranks exact matches/results to the search query higher than the search query's synonyms' results.

Example Use Case

If the search query is *start_transaction*, *OneSearch* looks for the synonyms of *start* and *transaction*. If *start* yields the synonym *begin* and there is no synonym defined for *transaction*, then *OneSearch* shows results for two query terms: *start_transaction* and *begin_transaction*.

If additional synonyms for *transaction* are also found, these are searched in the same manner.

Specifying an Additional Synonym Database

You can specify an additional synonym search database file (text file) using the `VERDI_SEARCH_SYNONYM_FILE` environment variable. The set of synonyms listed in this text file are added to the original pre-defined synonyms database, however, the predefined synonyms cannot be removed.

The text file must contain a set of synonyms (comma-separated list of words) in every line. All words in a line can be replaced by each other.

For example, assume that the synonym database contains the following three lines, which signifies that *A*, *B*, and *C* are synonyms to each other; *B* and *D* can be used interchangeably; and *D*, *E*, and *F* can also be used interchangeably.

```
A,B,C
B,D
D,E,F
```

If the search query contains the search term *B*, the synonym search checks the database line by line for the occurrence of the search term *B* and includes all synonyms into the search query. Thus, the search for *B* includes the synonyms: *A*, *C*, and *D*, but not *E* and *F*.

Limitations

The following limitations apply to this feature:

- Synonym search only applies to alphanumeric words. Special characters, such as -, _, *!*, @, #, *$* are not supported.
- Stemming of search terms is not supported, that is, inflection of nouns, pronouns, adjectives, and articles is not recognized. Derived forms of verbs are also not recognized.
- Words and sub-words are only subject to synonym search, if they are completely capitalized, or uncapitalized, or only the first letter is capitalized. For example, the following words (and their sub-words) are not subject to synonym search: *BeGIn*, *AMbA_tRANSMISSION*, *pOpULate*, mY_OwN_USBRegister.
- Using this feature may impact OneSearch performance.

Command Line Use Model

Besides the integration of OneSearch into the Verdi GUI, the `onesearch` command line option is also available, which can be executed in the design root directory.

The `onesearch` command line option supports the following options:

The options listed are optional.

Option	Description
--help	Prints brief help messages explaining all the search domains.
--help <*domain*>:	Prints the detailed help messages for the specified search domain.

Feedback

--maxresults [<*matches*>] [:<*lines*>]	Specifies the maximum number of matches and maximum number of lines per match (in case of multi-line matches). 0 means no limit. Settings apply to both overview (four matches, only one line per match by default) and an individual domain (3000 matches, no line limit per match).
--plain	Creates pure ASCII output without color and/or bold format.
--regexp	Specifies the regular expression style for wildcards. The default is glob style (such as, "*.sv").
--rootdir <*dir*>	Specifies the root directory to search. The default is the current working directory.

The option listed is mandatory

Option	**Explanation**
[--] <*query*>	Specifies the words and keywords to query. Use double quotation marks to enclose multiple words (such as, "as is").

Feedback

Usage Examples

The following are some of the examples for search capability:

- To search for words: *xbus_transfer* and *addr* in all **Sources**, **Logs**, **Identifiers**, and **Docs** domains and report top ranking matches for each domain, enter the following command:

  ```
  onesearch xbus_transfer addr
  ```

- To search for words: *xbus_transfer* and *addr* in all files matching **.svh* by using all **Sources**, **Logs**, **Identifiers**, and **Docs** domains and report top ranking matches for each domain, enter the following command:

  ```
  onesearch xbus_transfer addr "files:*.svh"
  ```

- To search for words: *xbus_transfer* and *addr* only in the **Identifier** search domain and report top ranking matches, enter the following command:

  ```
  onesearch xbus_transfer addr Identifiers:
  ```

Find Bar

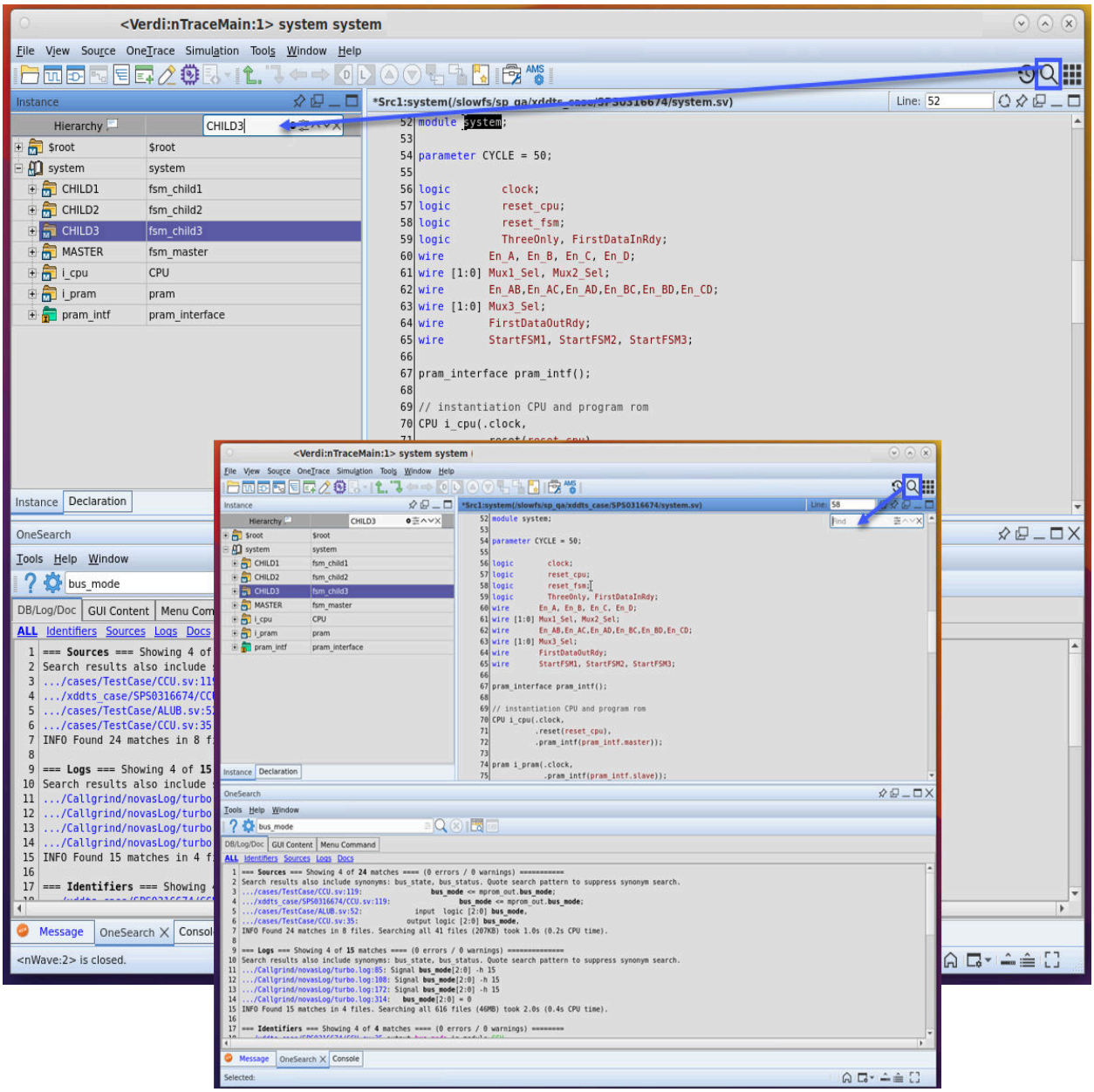
Verdi allows you to find information in a specific window by pressing **Ctrl+F** keys in a window. Each window has a separate find bar. Following are the various search constraints supported by this feature:

- Match Case
- Wildcard
- Regular Expression
- Find Whole Words Only

Feedback

The following figure illustrates how the find feature is used in the specific windows:

Figure 237 Find



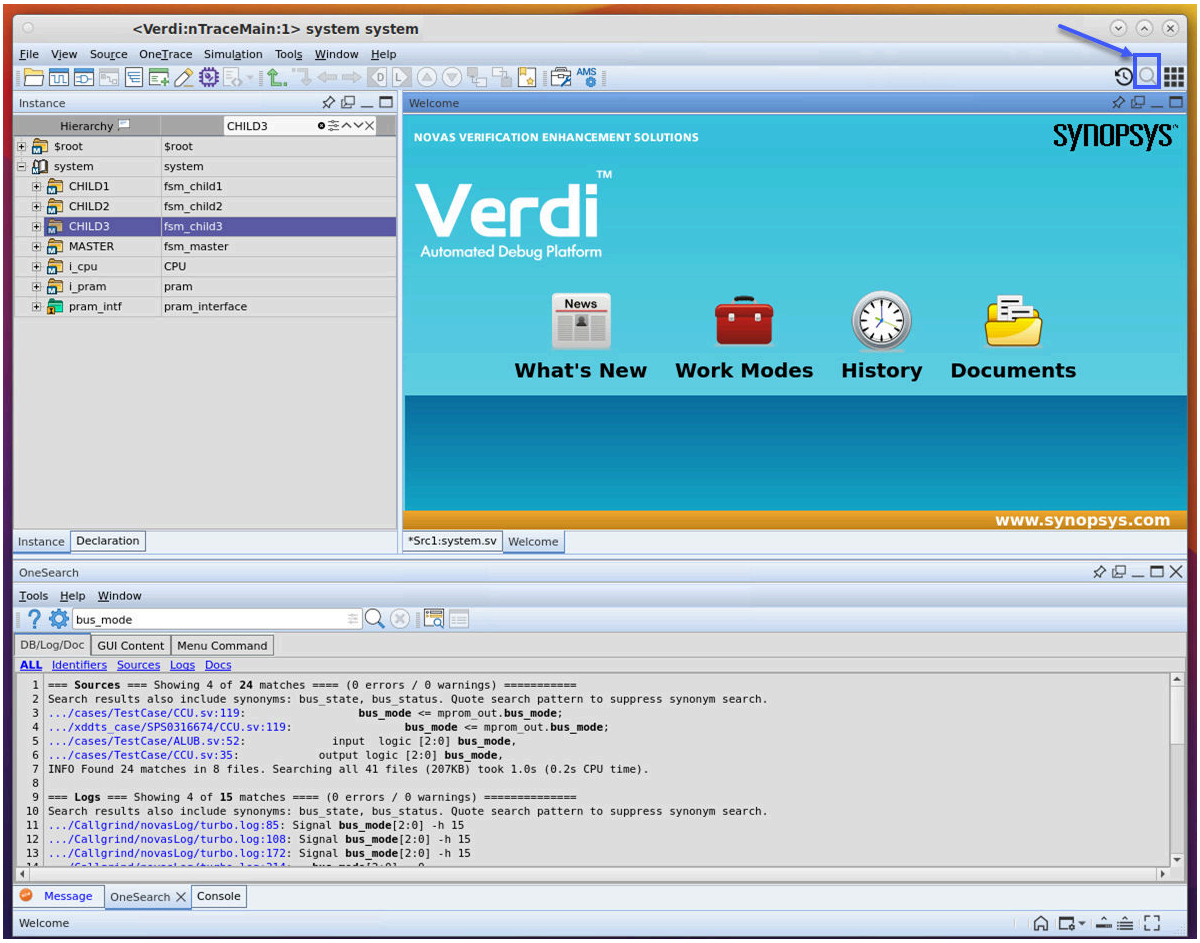
When you click the magnifier icon on the top window, Verdi pops up the active window find bar, fills in the selection content for the line edit, and sets focus to it in the find bar.

If the active window already opens the find bar, after clicking the button, fill in the selection content for the line edit and set focus to it in the find bar.

Feedback

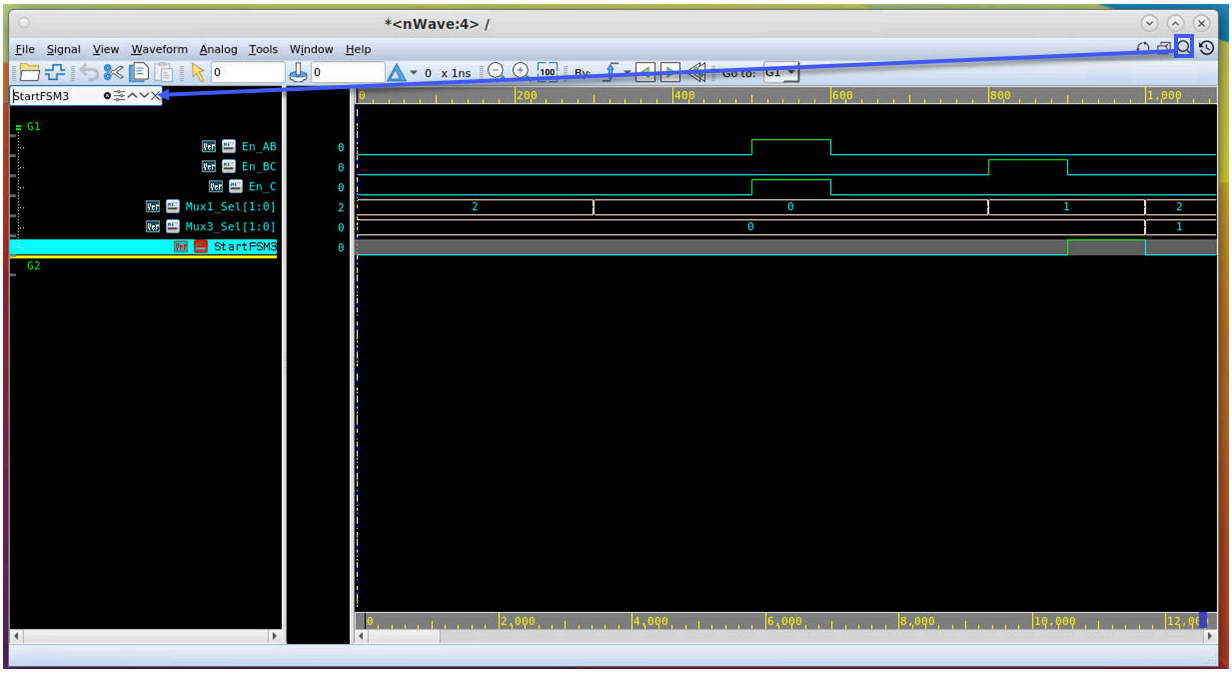
If the active window does not support find bar, this icon would be disabled as shown in the following figure:.

For example,



Feedback

If the window is undocked, the icon is shown in right-upper side as shown in the following figure:



Saving the visible status to the novas.rc file

When you open or close the find bar, the visible setting is saved to the novas.rc file. The timing to save to novas.rc file is when the you manually create or close the find bar.

This setting is window-type-based, which means if you create nWave1 and opens the find bar, then after you open nWave2, nWave3,..., the new nWave window also pops-up the find bar automatically unless you manually close a find bar in any nWave window.

At least, when you leave Verdi, the windows are closed with an opened find bar. Next time, when you invoke Verdi and open related windows, the find bar automatically pops-up.

Adjusting the icon size according to the font size

When you change the font size, the find bar also applies for the proper size due to the font size change. You can set the find bar width fixedly using the `setenv VERDI_FIND_BAR_WIDTH $WIDTH_PIXEL` environment variable.

Support Hot Keys for Find Next and Find Previous Functions

Use the hot key **Shift+Enter** for find previous and use **Enter** for find next functions.

Limitations

If a command is generated during runtime (means the command is generated by a user operation), it is not recorded to the recently used command drop- down list.

12

Temporal Flow View Tutorial

This chapter describes the following topics:

- Overview
- Open a Temporal Flow View
- Manipulate the View
- Show Active Statements
- Display Source Code
- Add Signals From the Temporal Flow View to nWave
- Specify Node Color for Compact and Register Tutorial

Overview

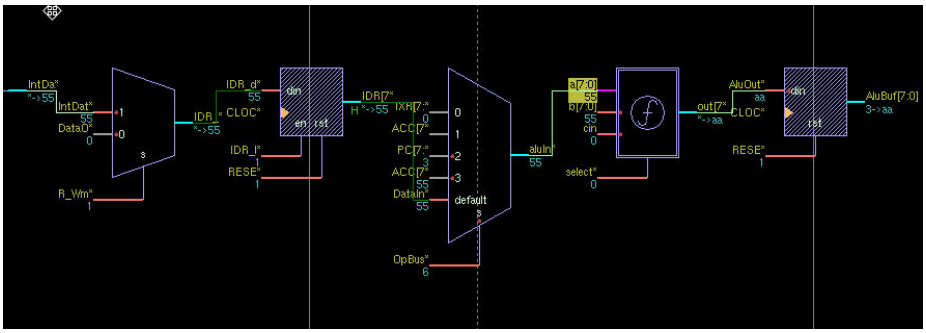
To debug a design, you need to understand the structure and the behavior of the design. Understanding the structure allows you to visualize the connection between blocks or signals. Also, understanding the behavior allows you to know the relationship between driver-signals, loader-signals, driver triggers, and the value transition. Visualizing the structure and behavior of the design is very useful and convenient to become familiar with the design and then debug it.

The flow views are unique temporal views and analysis tools that allow the visualization and analysis of the design's behavior through time. The *Temporal Flow View* identifies and displays causal control and data paths - the registers and signals that actually caused the erroneous value to occur - through multi-level combinational logic within one or more register-to-register transfer stages. The mechanism helps to quickly find the bug without repeatedly looking at the driver or fan-in signals of the signal of interest and the intelligently filtered temporal representation allows you to locate and identify problems in the most efficient manner possible without going through the different source code, schematic and waveform windows. However, the flow views also can be used to drive the source code, *nWave* (waveforms), *nState* (state machines) and *nSchema* (schematics) frames as needed.

Feedback

The display in the *Temporal Flow View* frame is similar to the following example:

Figure 238 Example Temporal Flow View Frame With Expanded Statements



In the *Temporal Flow View* frame notice the following items:

- The input data signals are on the left and output data signals are on the right of the symbol.
- On top of the gray vertical line, the number shown indicates that the clock active edge for that signal is at that time in *ns*. If you click the symbol associated with the register, the number appears in the box of the toolbar, That is, the value of the register changed at that time.
- The input control signals are on the bottom of the symbol. (An example of a control path is a multiplexer's select input, and an example of a data path is a multiplexer's data signals).
- Some input ports are pink (signal is actively contributing to the output value) or gray (signal is not actively contributing). (Turn on the **View -> Signal -> Active Nodes Only** toggle command to see all nodes or just active ones).
- The red dot associated with some ports indicates that these fan-in registers have value transition in their previous clock edges.
- The clock that drives the located register can be shown without tracing through combinational logic cones and correlating the clocks manually.
- The annotated simulations values (blue numbers).
- The signal names.
- The bus contention and active fan-in information can be clearly viewed in a single view.
- The cause of a specific data pattern on a partial bus can also be isolated.

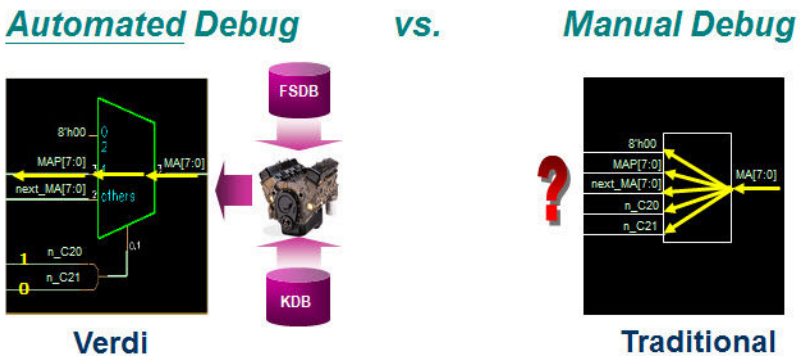
The *Temporal Flow View* traces to the root cause automatically with two major mechanisms. That is, behavior analysis engine and automatic cause and effect tracing.

Feedback

Behavior Analysis Engine

From the KDB and FSDB, Verdi builds an internal model of actual design behaviors using the synthesis and formal technology. Traditionally, the tracing stops at the problematic signals when debugging. If further tracing is required, then you must invoke and configure the tracing manually. Now, with the behavior analysis engine, the *Temporal Flow View* traces based on the behavior of the design automatically.

Figure 239 Temporal Flow View Traces Automatically to Debug



The following tracing features are available:

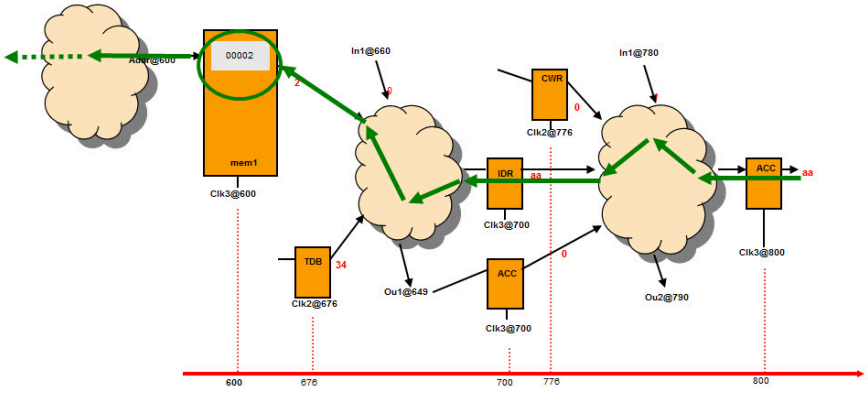
- Unrolls the function over time.
- Differentiates the data from control signals.
- Determines the clocking.
- Prunes inactive elements.

Automatic Cause and Effect Tracing

The *Temporal Flow View* traces not only to the problematic signal but to the root cause of the error. The *Temporal Flow View* provides a complete environment that helps in visualizing the behaviors with time and structure easily. The root cause and the trace path can be viewed holistically.

Feedback

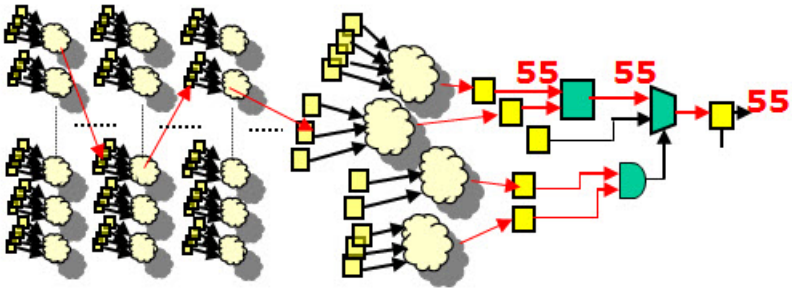
Figure 240 Temporal Flow View Traces to Root Cause of Error



The following scenarios show some of the best situations to benefit from advantages of root cause tracing of the *Temporal Flow View*:

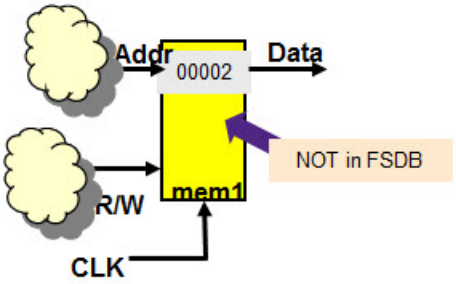
- Understand the design behaviors and locate the root cause of a wrong value.

Figure 241 Locate Root Cause of Wrong Value



- Trace the memory content that is not in FSDB and locate the memory write.

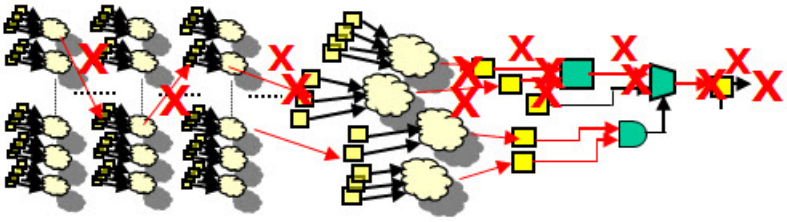
Figure 242 Trace Memory Content That is not in FSDB



- Locate the root cause of unknown (X) values.

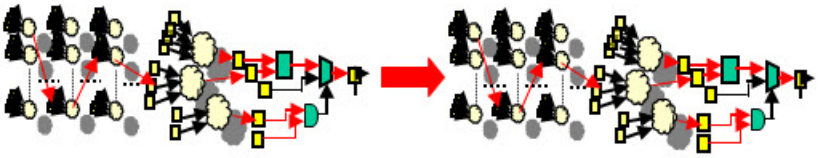
Feedback

Figure 243 Locate Root Cause of Unknown Values



- Locate the cause of different results between two simulation runs.

Figure 244 Locate Root Cause of Different Results Between Two Simulation Runs



Trace to Root Cause Automatically

From nTrace

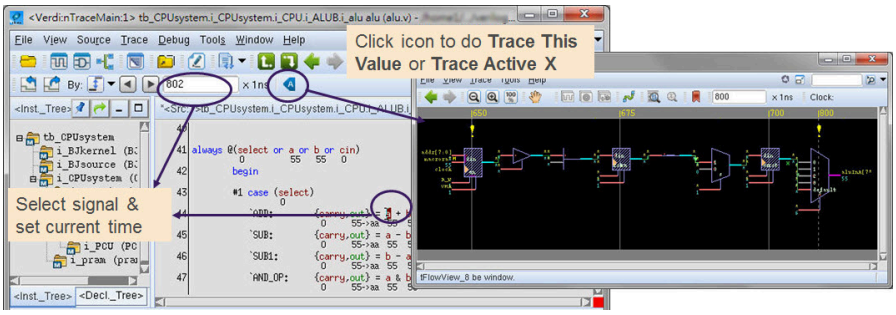
In *nTrace*, the root cause of an issue can be traced automatically with one of these commands: **Trace This Value** or **Trace X**. Select a signal in the *nTrace* source code pane, and click the **Auto Trace** toolbar icon. The *Temporal Flow View* opens and automatically performs the trace as described in the following commands:

- **Trace This Value**: For the current time if the current value is known.
- **Trace X**: For the current time if the current value is unknown.

  Note:
  The icon is only available when the **Active Annotation** option is turned *on*.

Feedback

Figure 245 Temporal Flow View Traces Root Cause With One Command in nTrace

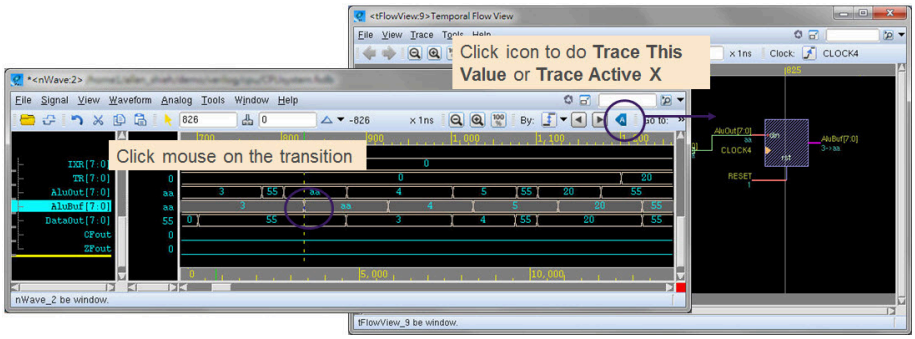


From nWave

In *nWave*, the root cause of issue can be traced automatically with one of these commands: **Trace This Value** or **Trace X**. Select a transition in the waveform pane, and click the **Auto Trace** toolbar icon. The *Temporal Flow View* opens and automatically performs the trace as described in the following commands:

- **Trace This Value**: For the current time if the current value is known.
- **Trace X**: For the current time if the current value is unknown.

Figure 246 Temporal Flow View Traces Root Cause With One Command in nWave



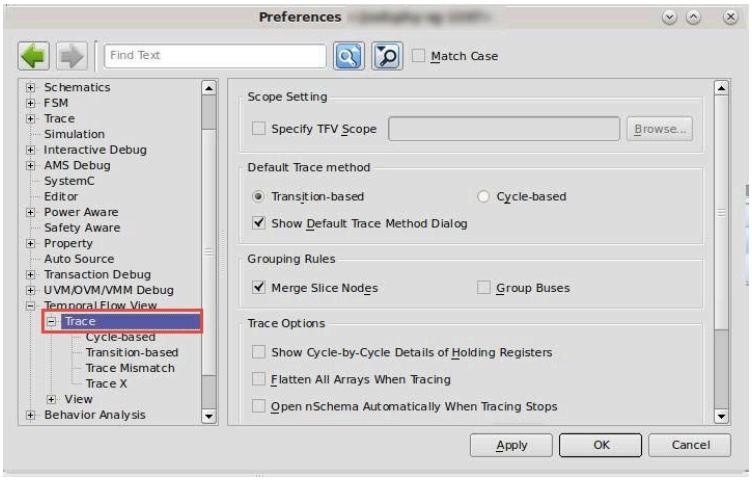
Open Temporal Flow View

Setup Temporal Flow View

1. Invoke the *Preferences* form with the **Tools** -> **Preferences** command.
2. In the *Preferences* -> *Trace* page, select trace by cycle or transition by specifying the **Cycle-based** or **Transition-based** option in the **Default Trace Method** section. The default is *Transition-based*.

Feedback

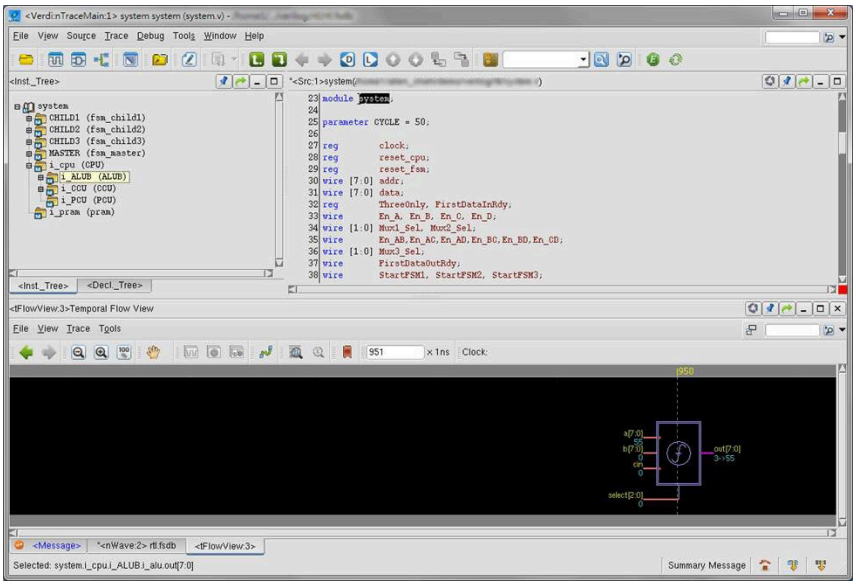
Figure 247 Specify Trace Method in Preferences Form



Create Temporal Flow View

1. In *nTrace* or *nWave*, select a signal transition and invoke the **Temporal Flow View -> New Temporal Flow View** command in the right-click menu command.
2. The *Temporal Flow View* frame appears in the bottom of the main window of *nTrace* and *nWave*.

Figure 248 Temporal Flow View Created in nTrace

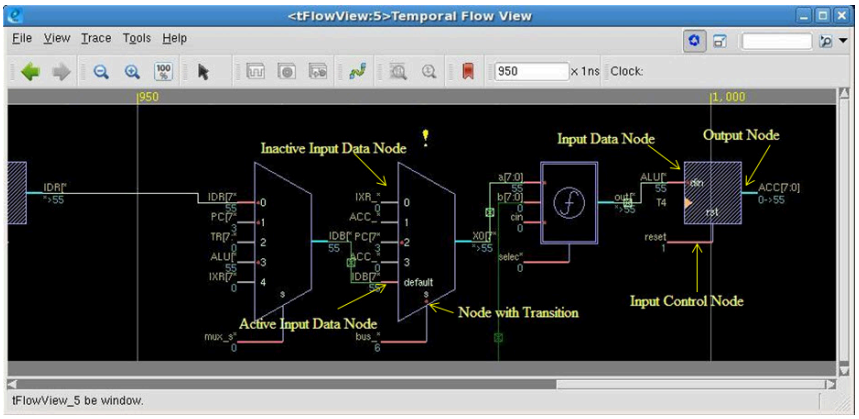


Feedback

Trace With Cycle-Based Method

Tracing with the cycle-based method, the design behavior is unrolled based on clock cycles. Signals are displayed and traced according to the final stable value within a single clock cycle. The *Temporal Flow View* collects the clock domain information for each signal.

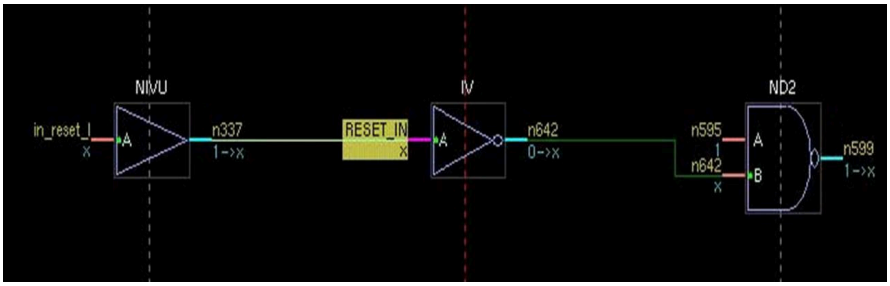
Figure 249 Trace With Cycle-Based Method



Trace With Transition-Based Method

Tracing with the transition-based method, input signals that trigger the output transition are automatically traced. The transition-based method are used to visualize the propagation of signal transitions throughout the design over time and is very useful for gate-level debug. The note distinguishes the transition-based tracing from the cycle-based tracing in the Temporal Flow View.

Figure 250 Trace With Transition-Based Method



Temporal Flow View Features to Trace Automatically

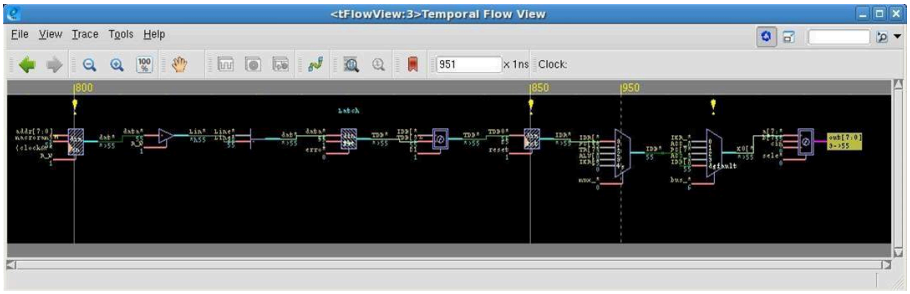
Trace This Value

Use the **Trace** -> **Trace This Value** command to trace back the data path automatically.

Feedback

- Unroll the path on time axis, and find the root cause.
- Use the **View** -> **Signal** -> **Active Data Nodes Only** command or the **View Signal** -> **Active Nodes Only** command to simplify the view and display active nodes only.

Figure 251 Trace This Value in Temporal Flow View

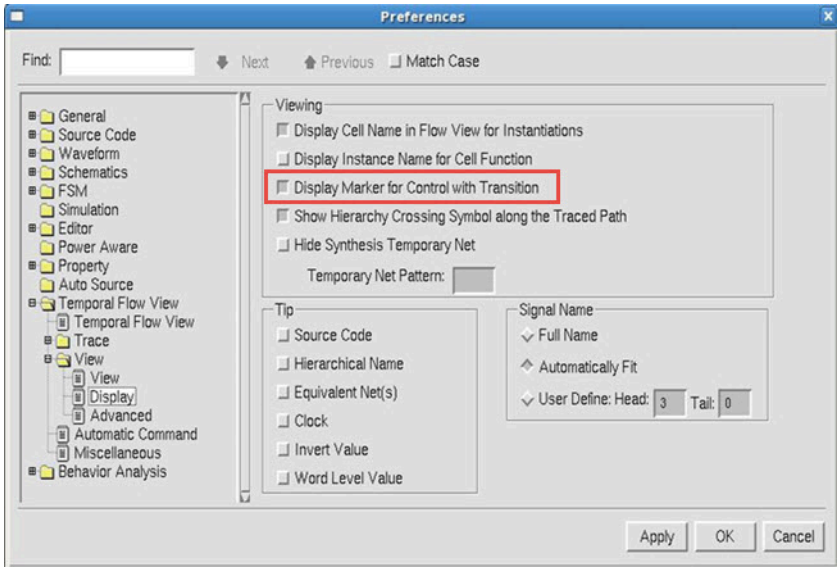


Show Marker for Control With Transition

Perform the following steps to show the output nodes with transition on the control signal in the tracing path:

1. Invoke the *Preferences* form with the **Tools** -> **Preferences** command.

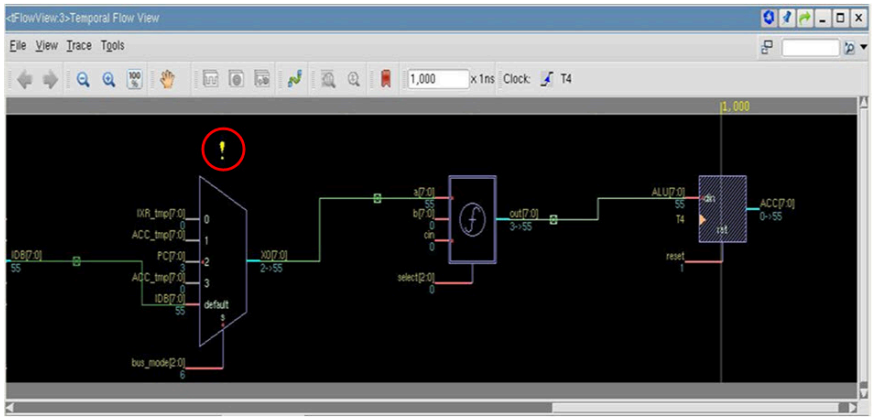
Figure 252 Enable Display Marker for Control with Transition in Preferences Form



Feedback

2. In the **Preferences -> Temporal Flow View -> View -> Display page**, enable the **Display Marker for Control with Transition** option. A yellow exclamation point are placed over output nodes in the *Temporal Flow View* pane.

Figure 253 Yellow Exclamation Point Placed over Output Notes

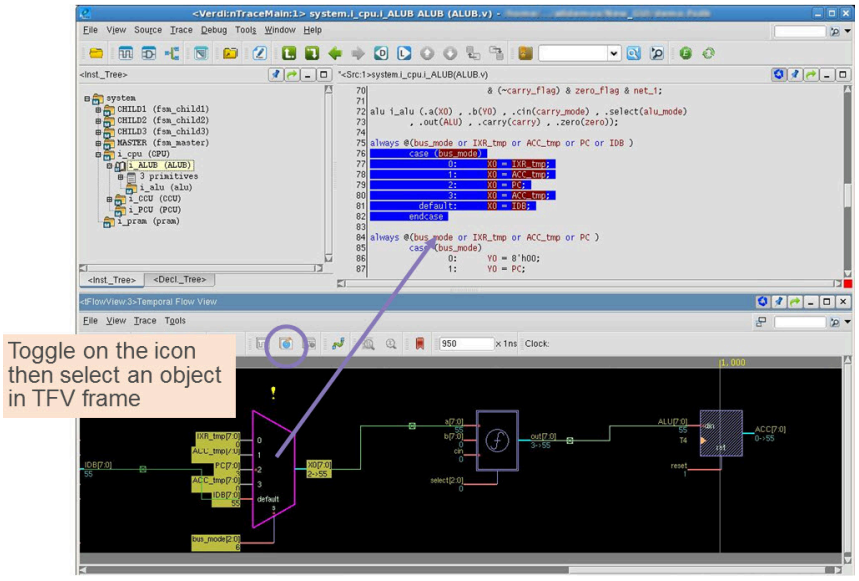


Correlate Other Panes

Display Source Code Automatically

Toggle on the **Show Source Code Automatically** icon or the **View** -> **Show Source Code Automatically** option to view the corresponding source code of a selected node.

Figure 254 Corresponding Source Code of the Selected Node

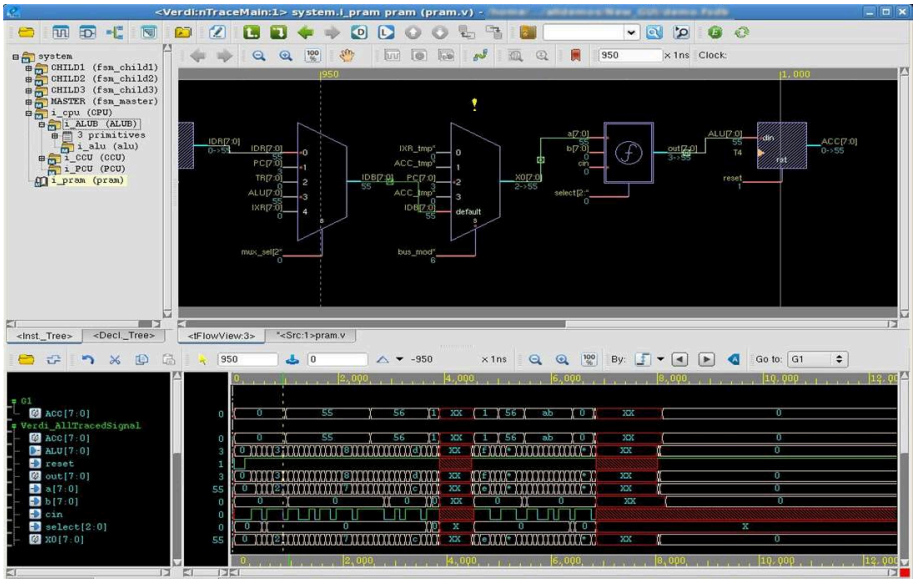


Feedback

Show Traced Signals in nWave

In the Temporal Flow View, invoke the **Tools** -> **Show All Traced Signals on nWave** command to add traced results to *nWave* to show traced signals in the waveform.

Figure 255 All Traced Signals Shown on nWave

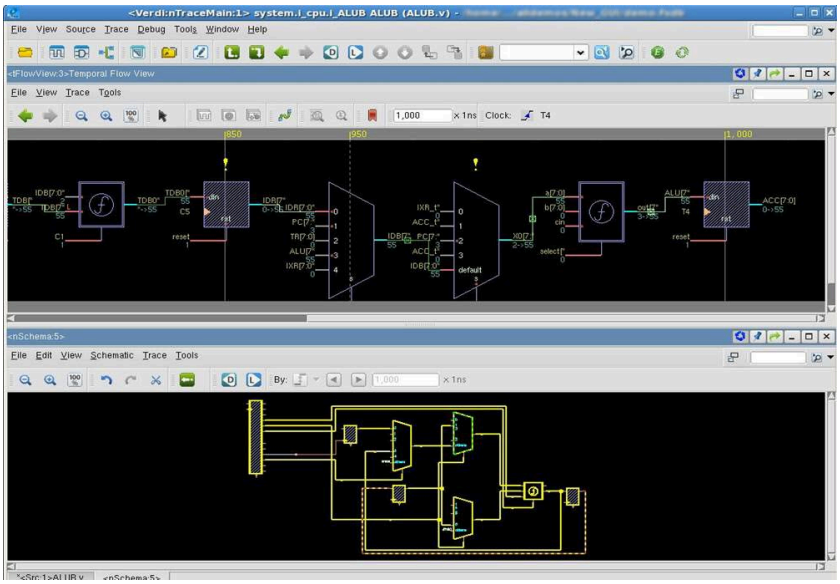


Show Fan-in Signals in nSchema

Invoke the **Tools** -> **Show Fan-ins on nSchema** -> **Active Only** command in *nSchema* after selecting a signal to view the fan-in of a signal.

Feedback

Figure 256 Show Fan-ins of Signal on nSchema

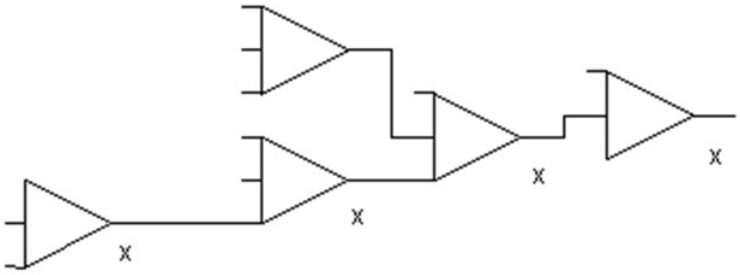


Temporal Flow View Application

Trace Unknown Values

Traditionally, unknown values (Xs) typically propagate through the design over multiple combinational statements.

Figure 257 Traditional Unknown Value Trace



However, the following issues occur to debug unknowns using traditional methods:

- Display the original signal in the waveform.
- Search the source code to find all the driving signals.
- Display the driving signals in the waveform to identify the signals that are unknown.
- Repeat this process until the root cause of the unknown is found.

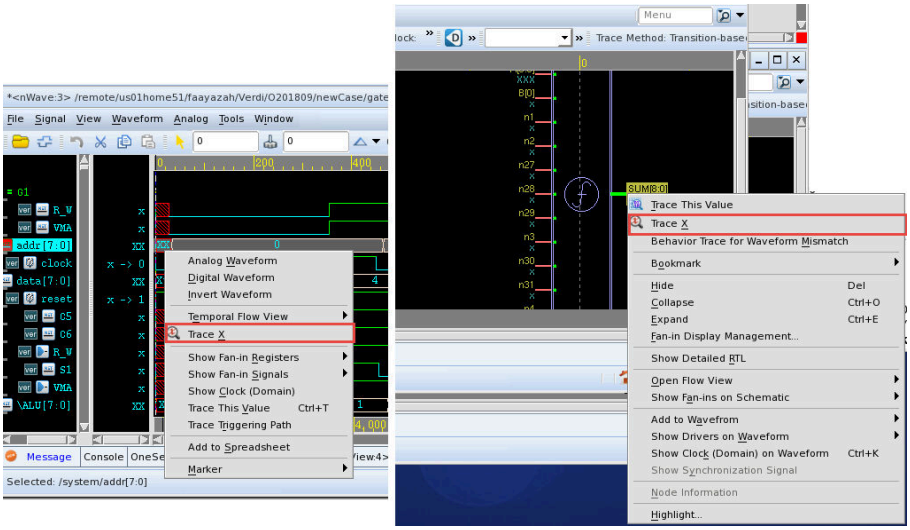
Feedback

Tracing features of the Temporal Flow View can solve the above mentioned issues. The following methods are available:

Trace X in Verdi

1. Select the unknown signal in the Temporal Flow View and then invoke the **Trace X** command in the right-click menu.
2. Select the unknown transition in the waveform pane in *nWave* and invoke the **Trace X** command in the right-click menu.

Figure 258 Trace X in Verdi



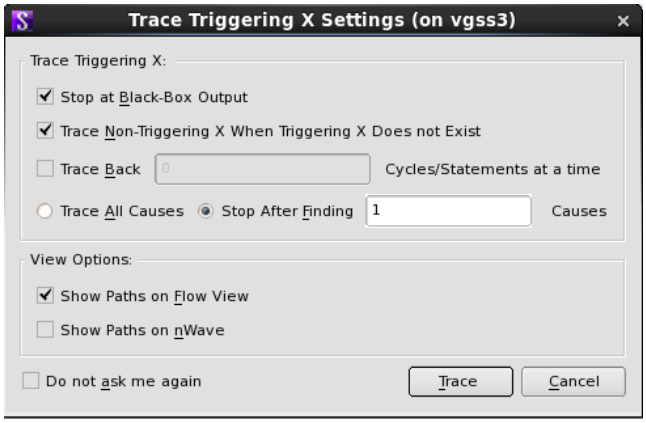
Feedback

Trace X Setting

After invoking the **Trace X** command in the right-click menu to open the *Trace Triggering X Setting* form, the following settings can be configured:

- Follow default settings for tracing X.
- Disable the **Show Paths on Flow View** option to only view the final result in the *Trace Triggering Results* frame.

Figure 259 Trace Triggering X Settings Form



Trace Triggering X Results

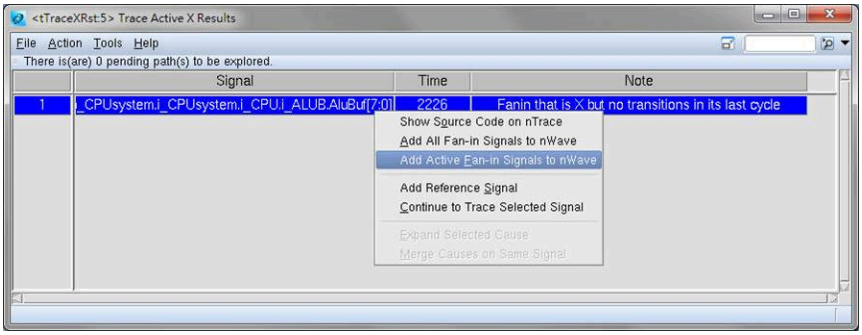
The *Trace Triggering X Results* pane opens and display the cause(s) of the X (unknown) in a tabular format. The reason of root cause is explained in the *Note* field.

The following right-click commands are available to continue tracing in Verdi:

- Show Source Code on nTrace
- Add All Fan-in Signals to nWave
- Add Active Fan-in Signals to nWave
- Add Reference Signal
- Continue to Trace Selected Signal
- Expand Selected Cause / Merge Causes on the Same Signal

Feedback

Figure 260 Trace Triggering X Results Form



Summary of Steps to Trace Unknown Values

1. Select the unknown signal in Temporal Flow View, invoke the **Trace** X right-click command to trace the root cause of the unknown.
   - Set tracing constraints in the resulting *Trace X Triggering* Settings form.
2. The cause is listed in a tabular form.
   - The reason is explained in the *Note* field.

Debug Memory

Locate Last Write on Specific Address Location

When a memory output value is incorrect, the possible reason includes:

- The wrong data is written into a location.
- The data is read from a wrong location.

There are two ways to locate the last write to a memory location:

- Double-click the memory signal in the Temporal Flow View.
- Select the memory signal in *nTrace* and invoke the **Debug Memory** -> **Trace Memory Write** right-click command.

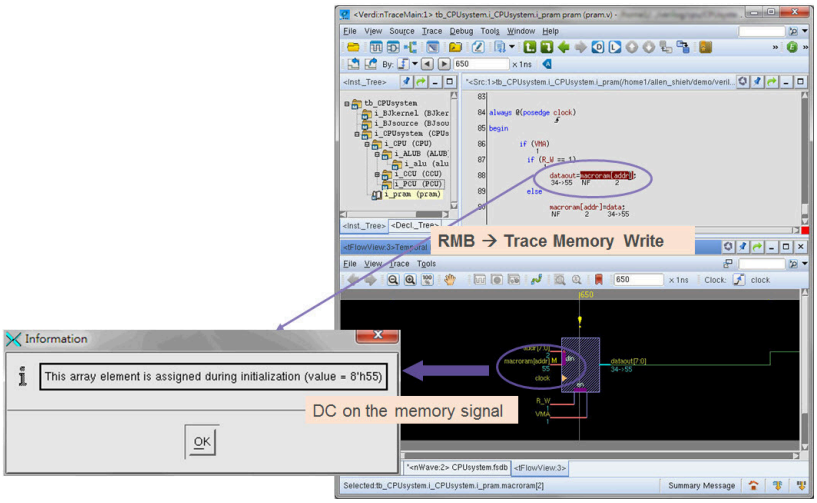
Verdi summarizes the results of the **Trace Memory Write** command in the Information form.

Verdi performs the following steps to capture the last write:

- Analyzes the control logic for the memory.
- Locates the values of these signals in the FSDB file.
- Determines when the last write to this location occurred.

Feedback

Figure 261 Trace Memory Write in nTrace



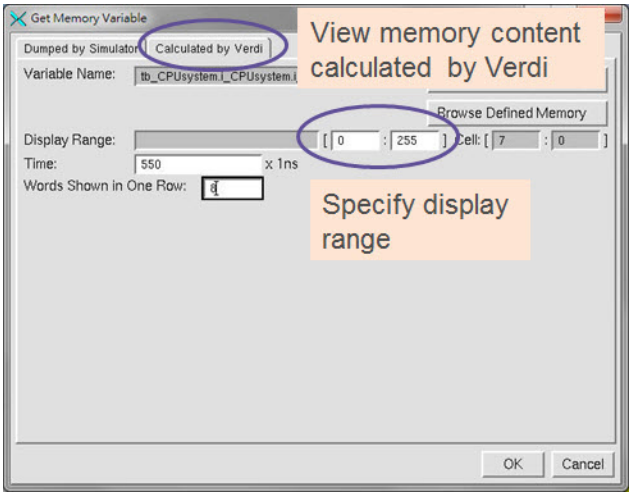
Display Calculated Memory Content

The following methods are available to display the calculated memory content:

- In *Temporal Flow View*, invoke the **Tools** -> **Show Memory Contents** command.
- In *nTrace*, select the memory signal and invoke the **Debug Memory** -> **Show Memory Content** right-click command.

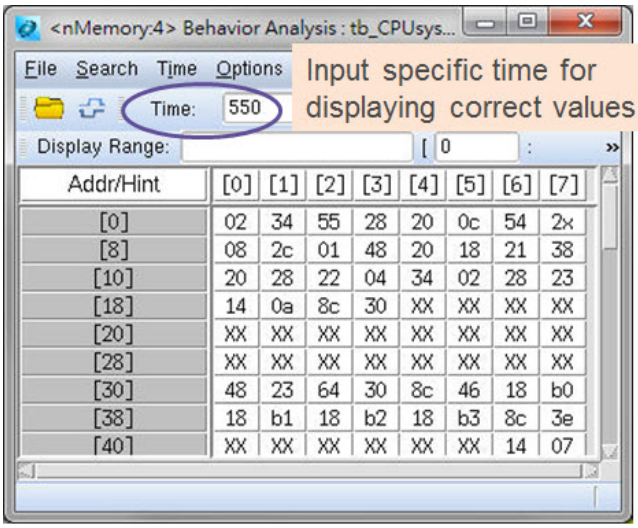
Select the **Calculated by Verdi** tab in the *Get Memory Variable* form. The memory content is shown in the tabular format. Specify the time to display correct values in the *Time* field for displaying the correct memory content. The display mode can be changed by commands under the **Options** menu option.

Figure 262 Configure Settings in Calculated by Verdi Tab



Feedback

Figure 263 Specify Time to Display Correct Values in Time Field



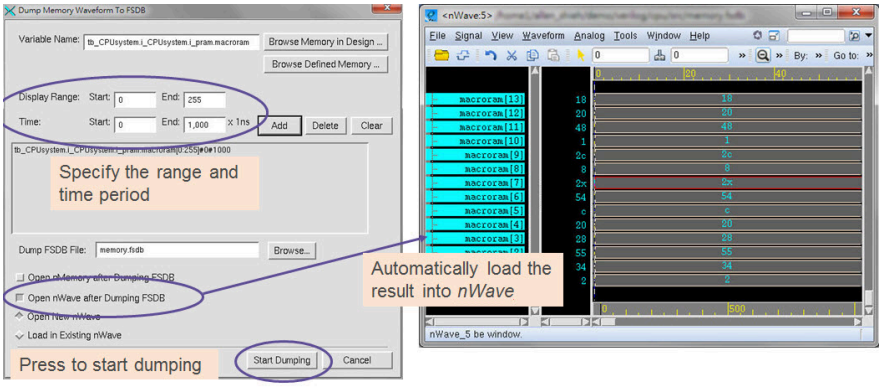
Dump Memory Content to FSDB

The following methods are available to dump the calculated memory content to a new FSDB file:

- In *Temporal Flow View*, invoke the **Tools** -> **Dump memory Waveform To FSDB** command.
- In *nTrace*, select the memory signal and invoke the **Debug Memory** -> **Dump memory Waveform To FSDB** right-click command.

Enter the memory range and time period for the calculated memory content. FSDB file can be loaded and shown on *nWave* or *nMemory* automatically.

Figure 264 Dump Memory Content to FSDB



This tutorial focuses on the *Temporal Flow View*. After reviewing the tutorial, refer to the Behavior Trace for Root Cause of Simulation Mismatches, Debug Memories,

Feedback

Debug Unknown (X) Values and Searching Backward for Value Causes sections in the Application Tutorials chapter for examples of how the *Temporal Flow View* can be applied in different debug scenarios.

Before you begin this tutorial, follow the instructions in the Before You Begin chapter.

Open a Temporal Flow View

1. Change your context to the *verdi_mixed* sub-directory, in which the demo source code files are located:

   ```
   % cd <working_dir>/demo/verdi_mixed
   ```

2. Compile the mixed design to create a work.lib++:

   ```
   % ./compile.verdi
   ```

3. Start the Verdi platform by referencing the compiled design and the FSDB file *CPUsystem.fsdb*(contains a set of simulation results) on the command line:

   ```
   % verdi -lib work -top tb_CPUsystem -ssf CPUsystem.fsdb -workMode
    hardwareDebug &
   ```

4. Resize the *nWave* window to a comfortable viewing size and locate it under the source code frame on your screen such that you can view both the windows.
5. In the **Instance** tab of the design browser frame, click the plus symbol to the left of the *i_cpusystem* block instance name to reveal its *i_cpu* and *i_pram* sub-blocks.
6. Click the plus symbol to the left of the *i_cpu* block instance name to reveal its *i_ALUB*, *i_CCU*, and *i_PCU* sub-blocks.
7. Double-click *i_ALUB* to display the associated source code.
8. In the **Find String** box on the toolbar, enter *AluBuf.*
9. Click the **Find Next** icon (see left) to find the signal.
10. Click middle mouse button to drag and drop *AluBuf* from the *Source Code* frame to the *nWave* window.
11. Click the *AluBuf* signal in *nWave* window close to the transition from the *3* value to the *aa* value and observe the following:
    - A vertical *cursor* appears in the waveform pane.

      Note:
      By default, the cursor snaps to the closest transition on the selected signal - the transition from *3* to *aa* in this case. (You can turn off the

Feedback

Waveform -> Snap Cursor to Transitions toggle command to allow you to set the cursor to any location).

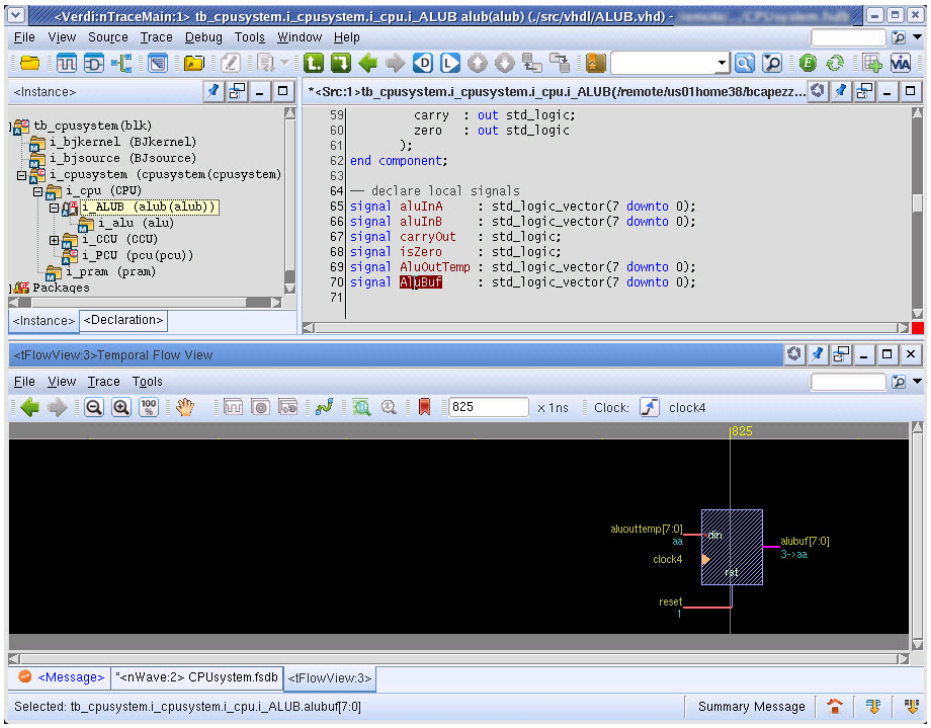
- The simulation time of 825 associated with the cursor's current location is displayed in *nWave*'s toolbar.

12. Right-click *alubuf* in the waveform pane on the transition from *3* to *aa* at time 825 and choose the **Temporal Flow View -> New Temporal Flow View** command from the right mouse button menu.

Note:
You can also access the *Create Temporal Flow View* form from the *nTrace* main window through the **Tools -> Temporal Flow View -> New Temporal Flow View** command.

A *Temporal Flow View* window opens as a new tab in the same area as the *nWave* window.

Figure 265 Temporal Flow View Window



Feedback

Manipulate the View

You can change the view of the *Temporal Flow View* using the following zoom commands:

- **Zoom In** - View more details of the *Flow View* by moving the view 50% from the center point in both the horizontal and vertical directions. Invoke this command in one of following three ways:
  - Toolbar icon
  - Bindkey "Z"
  - Menu **View -> Zoom -> Zoom In** command
- **Zoom Out** - View more contents of the *Flow View* by expanding the view 2X from the center point, both horizontally and vertically. Invoke this command in one of following three ways:
  - Toolbar icon
  - Bindkey "z"
  - Menu **View -> Zoom -> Zoom Out** command
- **Zoom All** - View the entire contents of the *Flow View*. Invoke this command in one of following three ways:
  - Toolbar's icon
  - Bindkey "f"
  - Menu command **View -> Zoom -> Zoom All** command
- **Zoom Area** - View more details in a specific area of the *Flow View* by dragging-left to form a rectangle over the area in pointer mode.

You can move the viewing area of the *Flow View* in different directions as follows:

- **Scrolling** - Click or drag the scroll bar of the *Flow View* window horizontally or vertically.
- **Panning** - Move the viewing area up, down, left, or right using the arrow keys on your keyboard or dragging left on the *Flow View* in Pan mode.

  Note:
  You can switch between pan/pointer mode by toggling the **View -> Switch to Pointer Mode/Pan Mode** command or using the *Flow View* toolbar icon.

Feedback

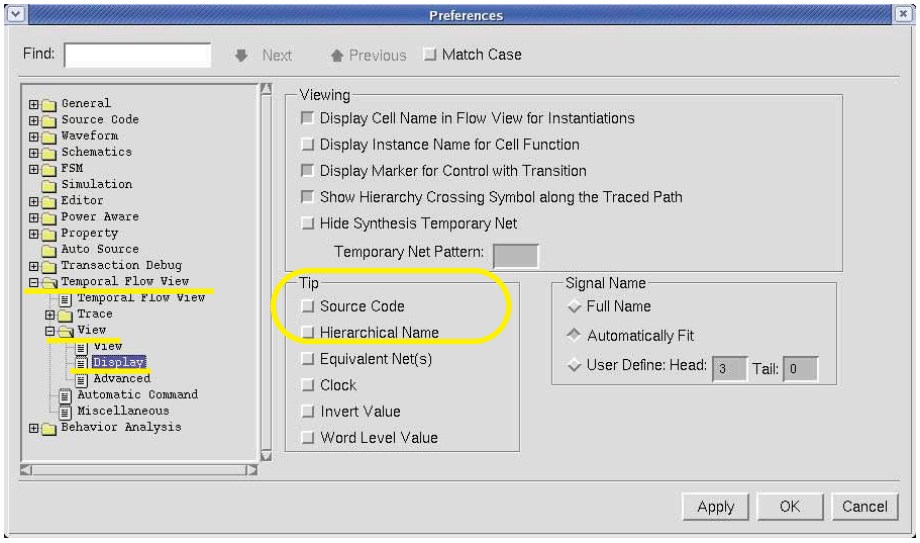
Display More Information

Extra information about your design can be shown using the TIP feature. You can enable the TIP feature by turning on the **View -> Turn on Tip** toggle command from the *Temporal Flow View* window. When the option is enabled, a yellow tip window is displayed when you put the mouse on top of a node.

1. In the *Temporal Flow View* frame, choose the **Tools -> Preferences** command.
2. In the *Preferences* form, click the **Display** page under the **Temporal Flow View -> View** folder.
3. Check the **Source Code** option in the **Tip** section. The corresponding line of source code is shown on the tip.
4. Check the **Hierarchical Name** option to see the full path to a signal.

   The *Preferences* form is similar to the following:

Figure 266 Temporal Flow View Preferences Form

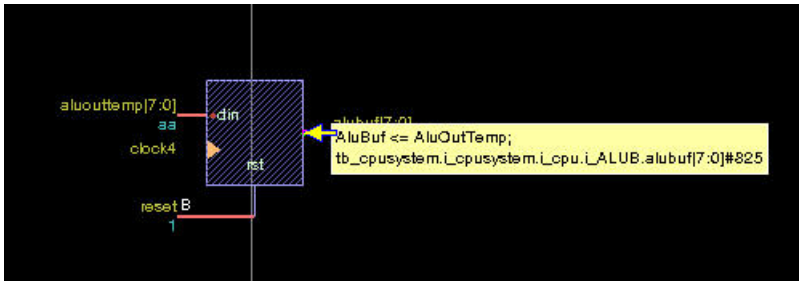


5. Click **OK**.
6. In the *Temporal Flow View* frame, click the **View -> Turn on Tip** command to turn on the TIP feature.

Feedback

7. Move your cursor over the *alubuf* node to view the associated line of code and hierarchical path.

Figure 267 Tip for alubuf

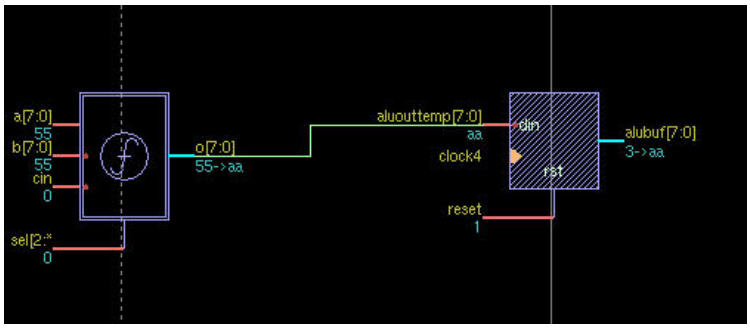


Show Active Statements

An active statement is the logic statement that generates the value for the driver signals of the current statement.

1. Double-click the *aluouttemp[7:0]* signal on the *Temporal Flow View*. You can also right-click the *aluouttemp* node in the *Temporal Flow View* and choose the **Show Active Statement** command.

Figure 268 Temporal Flow View Displaying Active Statements

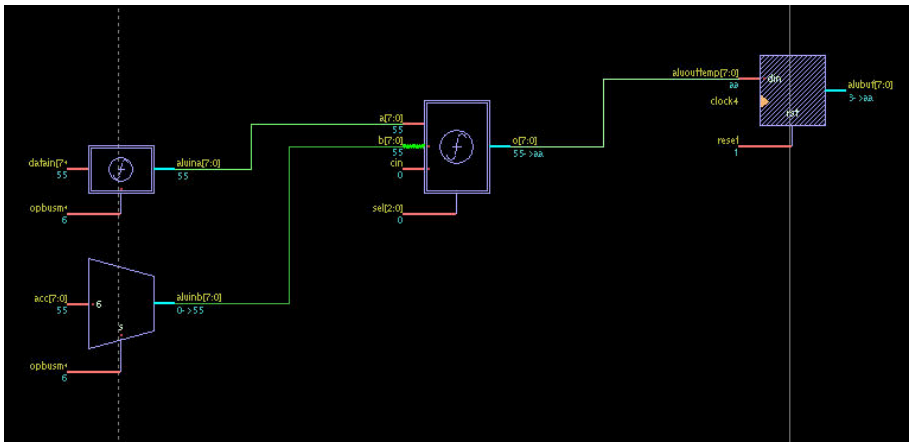


Note that another symbol (a function block) is added to the *Temporal Flow View*. You can continue tracing active statements on any input node.

2. Double-click *a[7:0]* to expand and display another function symbol.
3. Double-click *b[7:0]* to expand and display a *mux*.
4. You can use the **View -> Signal -> Active Nodes Only** command to toggle between viewing all nodes or only the active nodes. (Active nodes have a pink color to the port stub on the symbol).

Feedback

Figure 269 Temporal Flow View Displaying Active Nodes Only (in pink)



5. Use the zoom icons to manage the display. Click the **Fit Time** icon to perform a full zoom of the symbols.
6. Turn off the **View -> Signal -> Active Nodes Only** toggle command again.

Display Source Code

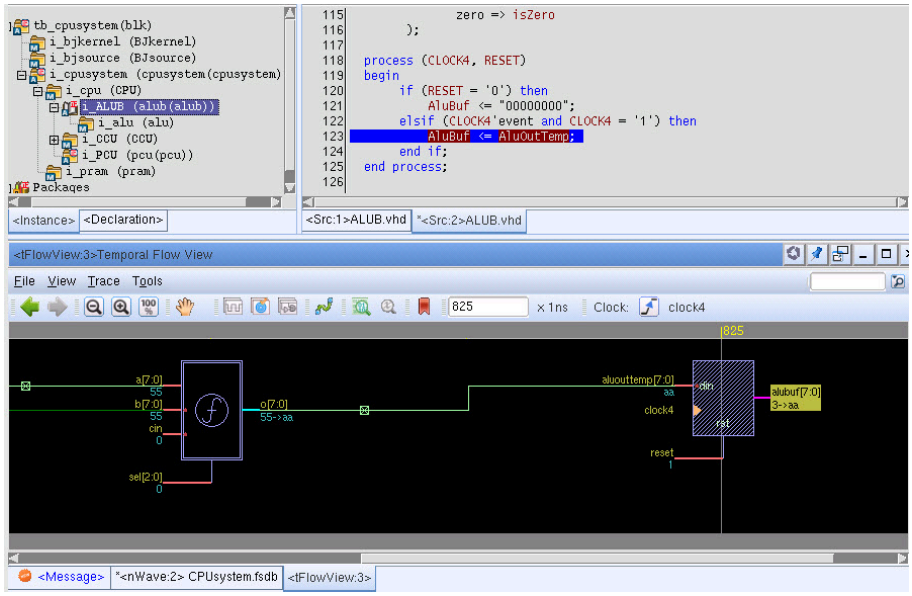
Select the display area of the source code from the *Temporal Flow View*. You can choose to display the source code in an existing *nTrace* window or a new window.

1. Choose the **Tools -> Preferences** command.
2. Select the **Automatic Command** page under the **Temporal Flow View** folder, and choose the preferred method for **Show Source Code Automatically on**.
3. In this tutorial, select the **New nTrace Window** option.
4. Click **OK**.
5. In the *Temporal Flow View*, click the **Enable Show Source Automatically** icon on the toolbar.
6. Click the *alubuf* node at time 825.

Feedback

The corresponding active (driving) statement is located and highlighted in a new tab of the source code frame:

Figure 270 nTrace Window Displaying Source Code from Temporal Flow View



The source code statement shows how the *AluOutTemp* and *RESET* signals functionally contribute to the contents of the *AluBuf* register. In particular, observe that *AluBuf* is assigned the value from *AluOutTemp* in a VHDL process.

Note that this is a mixed language design so the signal is *alubuf* in the waveform and flow views and *AluBuf* in the source code.

7. In the *Temporal Flow View,* click the output node *o[7:0]* of the function symbol.

   Note that a single line in a Verilog case statement is highlighted.

8. Click the *aluina[7:0]* output node of the second (left-most) function symbol.

   Note that a single line in a VHDL process is highlighted.

9. Click the **Disable Show Source Code Automatically** icon to disable the showing source code.

Feedback

Add Signals From the Temporal Flow View to nWave

There are several methods for adding signals to *nWave* from the flow view.

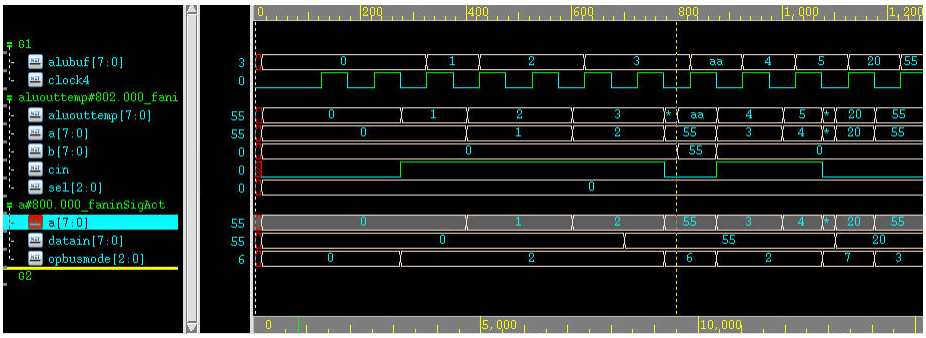
1. To simplify things, drag-left over all of the signal names in the signal pane to select the signals and then select the **Signal -> Cut** command to remove the signals from the display.
2. Click *alubuf* in the *Temporal Flow View* to select it, and then select **Ctrl+W** to add the signal's waveform to the *nWave* window.
3. Select **Ctrl+K** to add the clock signal of the register to the *nWave* window.
4. Select the output node *o[7:0]* of the function symbol.
5. Select **Ctrl+A** to add all the fan-in signals to the *nWave* window.
6. Select the output node *aluina[7:0]*.
7. Select **Ctrl+F** to add only the active fan-in signals to *nWave*.

Note that only the selected signal and the input ports with red colors (*datain*, *opbusmode*) are added.

Note:
You can change where and how the signals are added through the options in the **nWave** section of the **Automatic Command** page under the **Temporal Flow View** folder on the *Preferences* form.

The *nWave* results are similar to the following figure:

Figure 271 nWave Window Displaying Results of Adding Signals from Temporal Flow View



Feedback

Specify Node Color for Compact and Register Tutorial

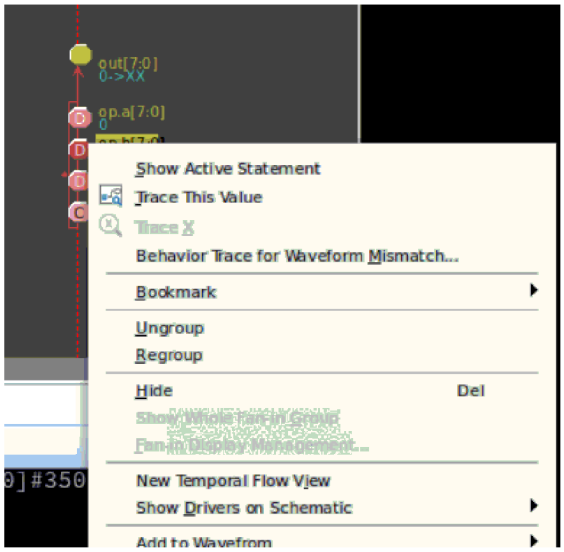
The node color options for Compact and Register Temporal Flow View are disabled by default. If you want to enable the node color options, set the following environment variable:

```
setenv TFV_COLOR_NODE 1
```

After you set this environment variable, the **Set Color** and **Reset Color** options are added to the right-click menu when you select a signal transition in *nTrace* or *nWave*.

Default Right-Click Menu

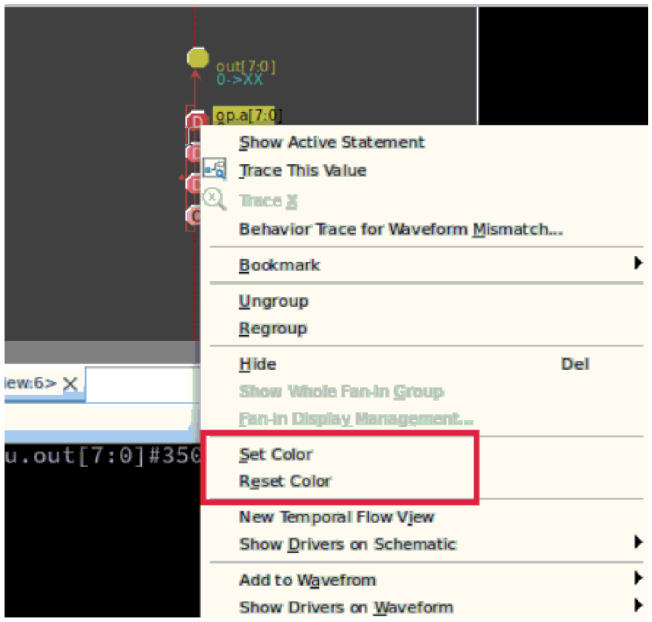
Figure 272 Default Right-Click Menu Without the Node Color Options



Right-Click Menu After Setting the Node Color Options

Feedback

Figure 273 Right-Click Menu With the Node Color Options



13

Debug a Design With Simulation Results Tutorial

Before you begin this tutorial, follow the instructions in the Before You Begin chapter.

This tutorial shows you how to use the Verdi platform to debug a simulation failure in a RTL design. A Verilog example is used; however, the same debug techniques apply to VHDL or mixed designs.

This chapter includes the following topics:

- Find the Active Driver
- Generate Fan-in
- Debug Memory Content

Find the Active Driver

Assume the transition from 3 to 55 of *ALU[7:0]* at time 951 is wrong, and you have to discover the real cause(s).

1. Change the directory to <*working_dir*>*/demo/verilog/rtl*. Execute the following command to import the design and load the simulation results:

   ```
   % verdi -f run.f -ssf rtl.fsdb -workMode hardwareDebug &
   ```

2. In the **Instance** tab of the *Design Browser* frame, click the "+" of *i_cpu(CPU)* folder to expand the hierarchy.

3. Drag and drop *i_ALUB(ALUB)* into *nWave* frame to display the signals.

   Note:
   You can also use **Get Signals** in *nWave* to select the signals of interest.

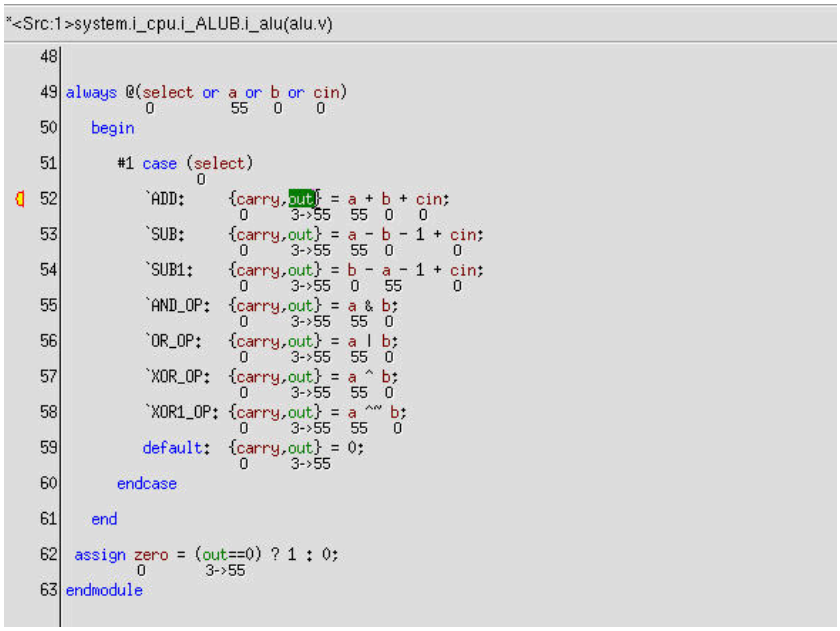
4. Choose the **Source -> Active Annotation** command to annotate the simulation results into the *Source Code* frame.

5. In *nWave*, double-click the transition from 3 to 55 of *ALU[7:0]* at time 951 ns to locate the source of the transition.

Feedback

This action displays the active driver of the signal transition in the *Source Code* frame. For this example, it is signal *out* on line 52 of *i_alu(alu)*, as shown in the following figure:

Figure 274 Active Driver of Signal Transition



Note:

ALU has crossed a hierarchical boundary, and changed names to *out*. Look at the equation for *out*. It is an OR of *a*, *b*. and *cin*. Which signal matches the incorrect value of 55?

With active annotation, it is clear the source of the 55 value is signal *a*.

6. Locate all drivers of *a* by double-clicking *a* on line 52.

   There are five drivers reported in the message frame. It would take time to trace back the logic of all drivers.

   Let's find the real active driver *a* to dramatically reduce the effort.

   Note that the time is 951 ns.

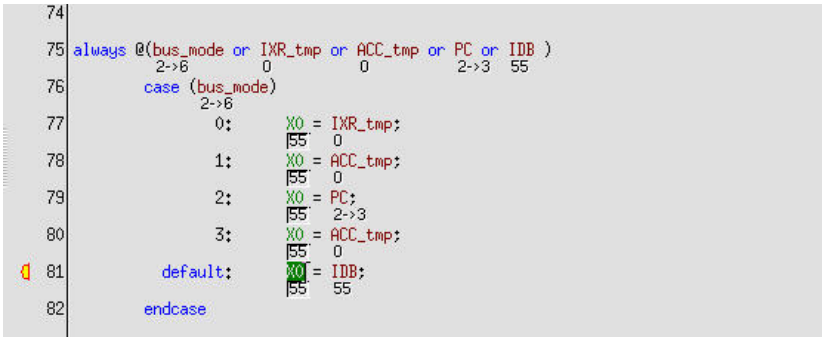
7. Click the **Backward History** icon (see left) on the *nTrace* main window toolbar to return to previous step.

8. With signal *a* selected, click the right mouse button menu, and choose **Active Trace** to find the real driver (or **Ctrl+T**).

Feedback

Now, *X0* in line 81 is selected, which means that the real driver is coming from this line.

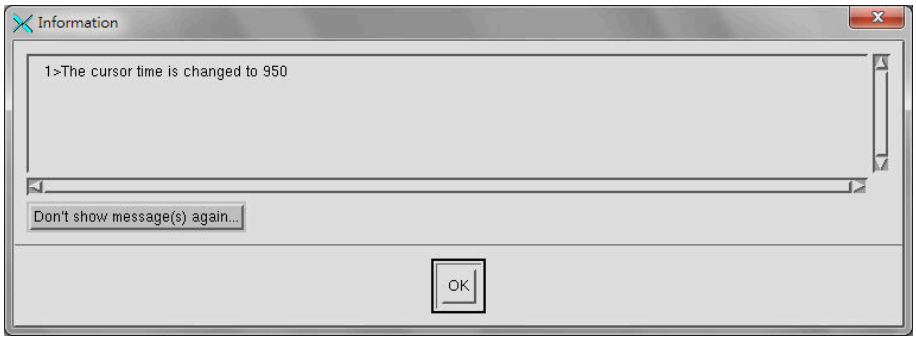
Figure 275 Partial nTrace Window Displaying Active Trace for Real Driver



Note:
Signal *a* changes the name to *X0* in *ALUB.v* because of its connectivity.

An *Information* dialog window displays (shown in the following figure) to denote a 1ns change in time.

Figure 276 Information Window Displaying Change in Time



9. Click **OK** on the *Information* dialog.

Generate Fan-in

Active trace can be performed multiple times until the cause is located. However, you may need to display a schematic that shows only the logic driving *IDB* (the active driver of *X0*), independent of hierarchy.

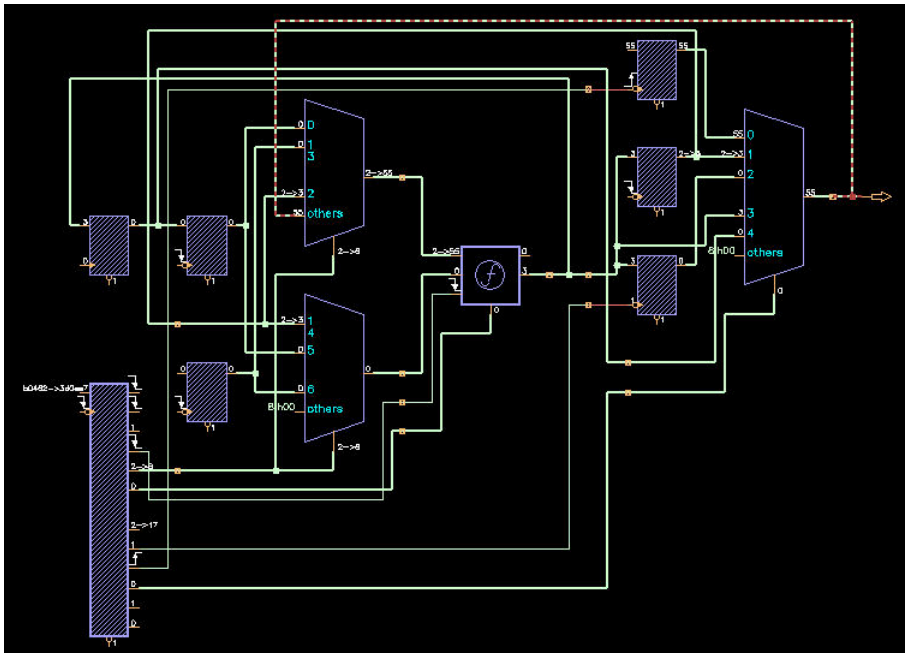
1. To generate the **Fan-In** for *IDB*, select *IDB* in line 81 on source code frame.
2. Choose the **Tools -> New Schematic from Source -> Fan-in** command.

Feedback

A flattened *nSchema* frame opens as a new tab in the same area as the source code frame, displaying the logic driving *IDB*. You can select specific blocks, and find they are from different hierarchies.

3. In the *nSchema* frame, choose the **Schematic -> Active Annotation** command to annotate the simulation results in the **Fan-in** window.

Figure 277 nSchema Frame Displaying Annotated Fan-in Schematic



4. Use the zoom function (drag-left in the upper right around the three registers and mux) to view the values on the nets in detail.

   Let's analyze the generated **Fan-in** to locate the real cause of the result.

5. Choose the **View -> Net Name** command to display signal names on the schematic.

   IDB is driven by a mux, so it is important to know the value of the select line to determine which input is active. In this scenario, the current value on the select line is 0, making the topmost input your starting point. The top input of the mux is coming from a storage element.

6. Select *IDR[7:0]* (the output pin of the storage element driving the mux input) and click the **Search Backward** icon on the toolbar to locate its transition to '0->55'.

   The time should now be 850ns.

7. Double-click the input pin of the storage element to trace the logic back. It is a mux.

Feedback

Note:
Fan-In stops at storage elements, functional blocks, FSMs, and primary IOs.

8. You want to know when the register input transitions to 55. Select the net (*TDB0[7:0]*) between the register and mux and click the **Search Backward** icon on the toolbar.

   Note the time changes to 800 and the value on the control signal of the mux is 1.

   Continue tracing the 55 value back through the signal by completing the following steps:

9. Double-click the second data input pin of the mux to expand the driving logic. It is driven by a latch.

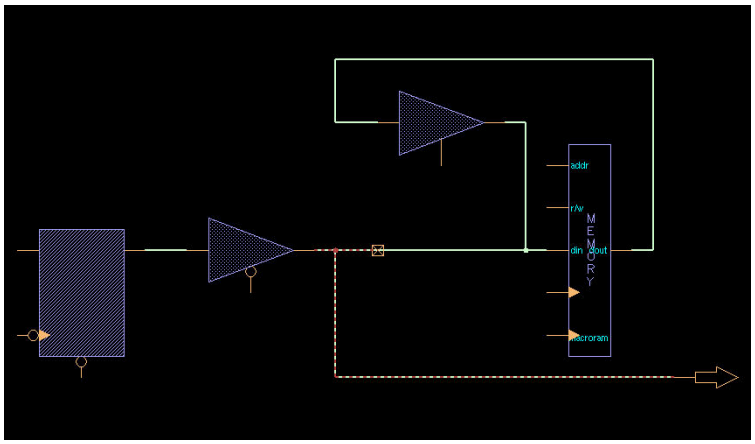
10. Double-click the data input of the latch, which is driven by a pair of tri-state devices.

    At this point, the schematic is getting a little cluttered, so let's continue on a fresh schematic.

11. Select the output of the tri-state and then choose the **Tools -> New Schematic -> Fan-in** command to generate another **Fan-in**.

    In the new *nSchema* frame, a schematic with its output driven by two tri-states and memory component is displayed, as shown in the following figure:

Figure 278 Example Tri-State and Memory Component



12. Annotate simulation results using the **Schematic -> Active Annotation** command (or use the 'x' bindkey).

    The enable pin of the tri-state is active low, and its current value is 1, which indicates that the output is driven by the memory component.

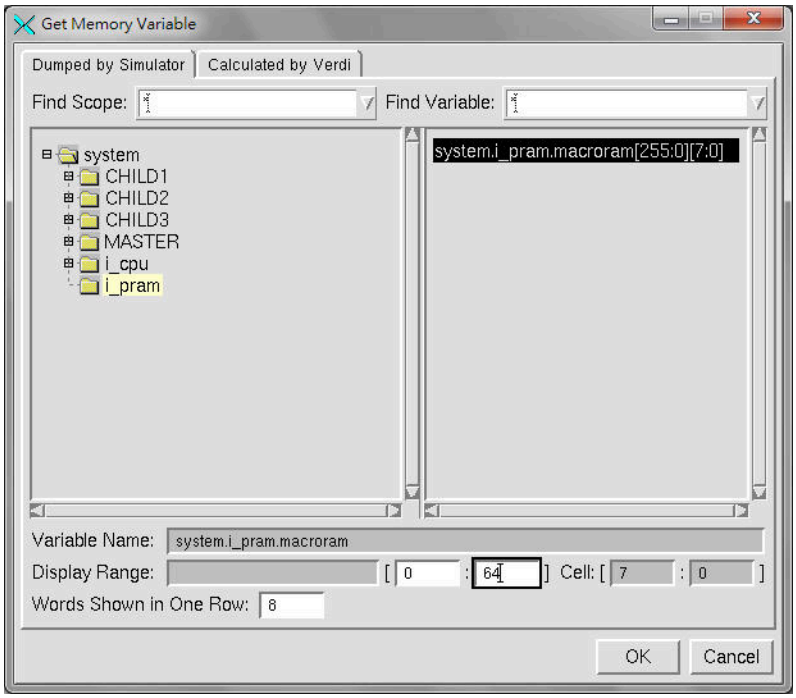
Debug Memory Content

You know the bad value is related to the memory. Now you want to look at the memory. Use the following steps to load memory content.

1. In the *nTrace* main window or the *nWave* frame, choose the **Tools -> Memory -> Memory/MDA** command, which opens an *nMemory* frame in the lower right.
2. In the *nMemory* frame, choose the **File -> Get Memory Variable** command.
3. On the **Dumped by Simulator** tab of the *Get Memory Variable* form, select *system.i_pram.macroram*.
4. In the **Display Range** text fields, specify *0* for start address and *64* for the end address.

   You know this is the only part of the memory that is used during this simulation. The form is similar to the following:

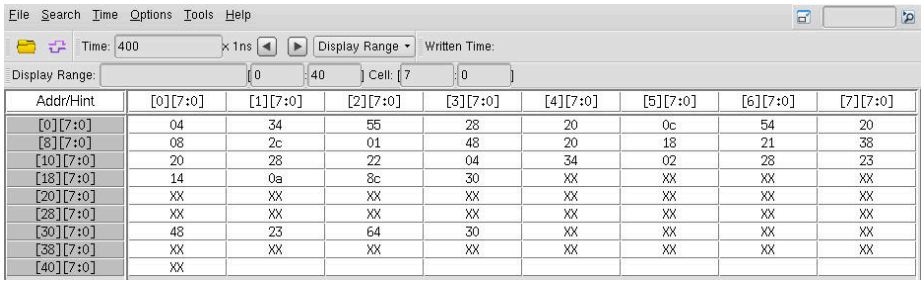
Figure 279 Get Memory Variable



Feedback

5. Click the **OK** button. The specified memory and address range is loaded into *nMemory*, similar to the following:

Figure 280 nMemory Window



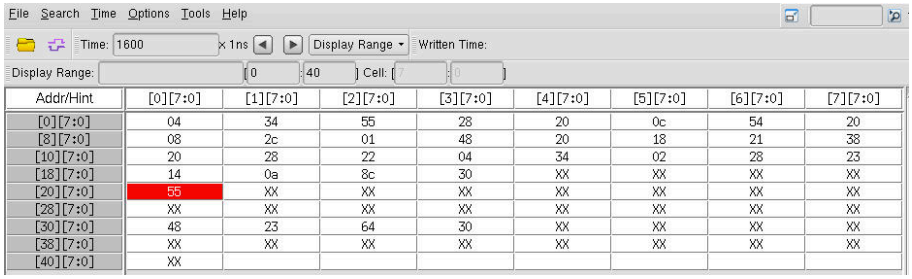
Addr/Hint	[0][7:0]	[1][7:0]	[2][7:0]	[3][7:0]	[4][7:0]	[5][7:0]	[6][7:0]	[7][7:0]
[0][7:0]	04	34	55	28	20	0c	54	20
[8][7:0]	08	2c	01	48	20	18	21	38
[10][7:0]	20	28	22	04	34	02	28	23
[18][7:0]	14	0a	8c	30	XX	XX	XX	XX
[20][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[28][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[30][7:0]	48	23	64	30	XX	XX	XX	XX
[38][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[40][7:0]	XX							

Use the following steps to look for the new data.

6. In the *nMemory* frame, turn on the **Time -> Sync Cursor Time** toggle command.

7. Click the **Next Dump** icon (see left) on the memory toolbar until you see one of the addresses highlighted in red.

Figure 281 nMemory frame Displaying a Address Highlighted in Red



Addr/Hint	[0][7:0]	[1][7:0]	[2][7:0]	[3][7:0]	[4][7:0]	[5][7:0]	[6][7:0]	[7][7:0]
[0][7:0]	04	34	55	28	20	0c	54	20
[8][7:0]	08	2c	01	48	20	18	21	38
[10][7:0]	20	28	22	04	34	02	28	23
[18][7:0]	14	0a	8c	30	XX	XX	XX	XX
[20][7:0]	55	XX	XX	XX	XX	XX	XX	XX
[28][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[30][7:0]	48	23	64	30	XX	XX	XX	XX
[38][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[40][7:0]	XX							

In this case, the data value at *address[20]* is *55* at a simulation time of *1600* ns.

Use the following steps to trace memory content in *nWave*.

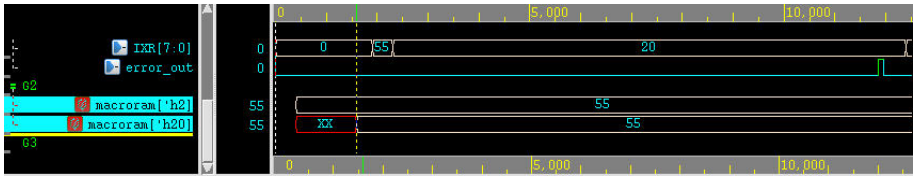
8. In the *nMemory* window, choose the **Search -> Find** command.

9. In the *Find* form, enter *55,* and use the binocular icons (**Find Next** or **Find Previous**) to search for the other value of 55, which is located at *address[2]*. Another location is *address[20]*.

Feedback

10. Select the address locations with value *55* by holding the Ctrl key to select multiple locations.
11. Use the middle mouse button to drag and drop the data to *nWave* frame to see the waveform, as shown in the following figure:

Figure 282 Memory Waveform



14

nCompare Tutorial

This chapter describes the following topics:

- Overview
- Start nCompare and Compare Waveforms
- Viewing Errors
- Error Report File
- Support Multi-process With CDPL

Overview

The *nCompare* frame compares simulation results stored in FSDB dump files by using flexible, user-specified comparison criteria. The frame is optimized for extremely fast comparisons of large data sets. It is fully integrated with the Verdi platform to intuitively display differences between runs.

The *nCompare* module uses the Tcl language. Further, it uses the *nCompare*-defined-Tcl-extended comparison commands for comparison rules and specific comparison options. A rule file written using the Tcl language is used to describe what needs to be compared. Refer to *Appendix D* in the *Verdi and Siloti Command Reference Manual* for details.

Include the following in a basic rule file:

1. Specify golden and secondary simulation files.
2. Specify compared signal pairs.
3. Start time-based comparison.

Follow this tutorial to learn how to use the basic functionality of *nCompare*.

Start nCompare and Compare Waveforms

After the *nCompare* frame is opened in the Verdi platform, the frame can be used to import a rule file in one tab and to compare and view the mismatches in another tab.

Feedback

Perform the following steps to open an *nCompare* frame and import a rule file:

1. Change to the *nCmp_demo1* directory.

   ```
   % cd <working_dir>/demo/nCompare/nCmp_demo1
   ```

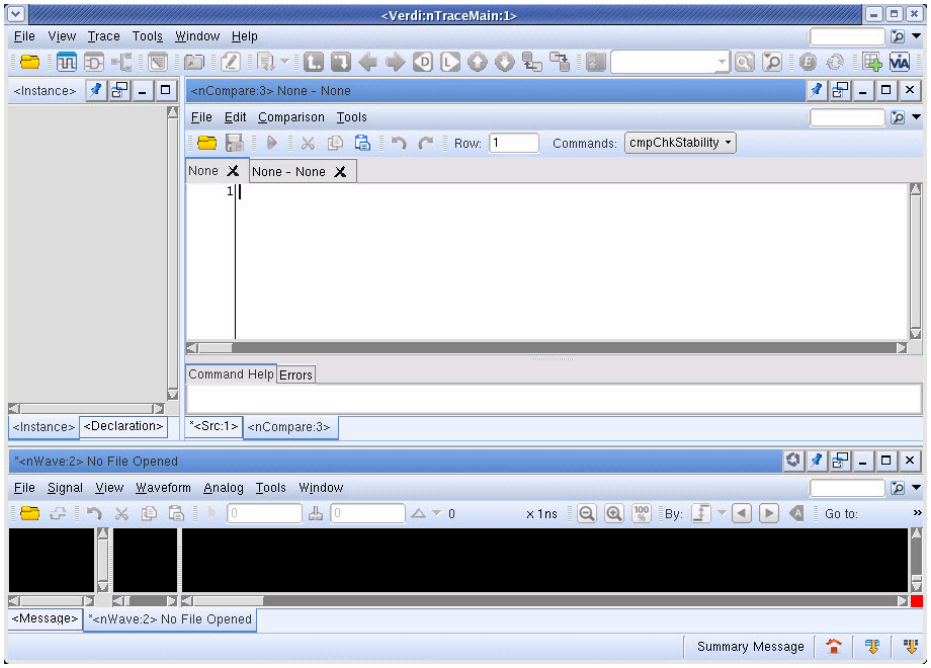
2. Invoke the Verdi platform.

   ```
   % verdi -workMode hardwareDebug &
   ```

3. In the main window, choose **Tools -> New Waveform** command to open the *nWave* window.

4. In the *nWave* window, choose **Tools -> nCompare** command to open the *nCompare* frame. The *nCompare* frame opens as a new tab in the same location as the source code frame, as illustrated in the following figure:

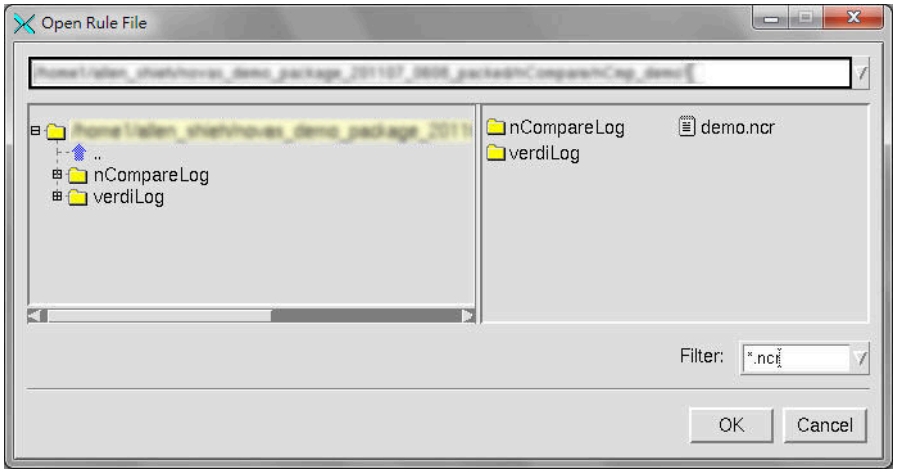
Figure 283 nCompare Frame



Feedback

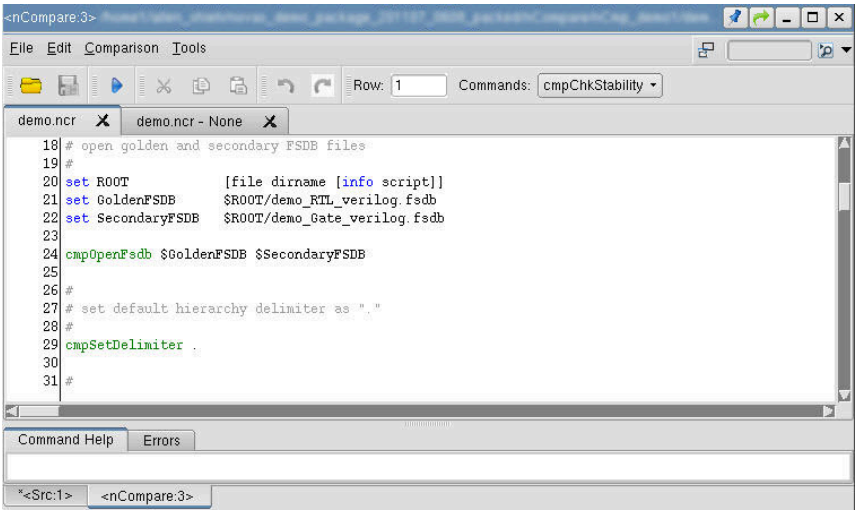
5. In the *nCompare* frame, choose **File -> Open Rule File** command. The *Open Rule File* form opens, as illustrated in the following figure:

Figure 284 Open Rule File Form



6. Select the *demo.ncr* file in the *Open Rule File* form. The opened rule file is displayed as another tab in the top half of the *nCompare* frame, as illustrated in the following figure:

Figure 285 Current Opened Rule File



7. To start the waveform comparison, choose **Comparison -> Run** command or click the **Run** icon on the toolbar of the *nCompare* frame to start the waveform comparison.
8. To pause or stop the comparison process during the comparing process, choose **Comparison -> Pause** or **Comparison -> Stop** commands. Alternatively, click the **Pause** or **Stop** icons on the toolbar.

Feedback

Viewing Errors

Following are three main methods for viewing the errors: time view, hierarchy view, or in the waveform.

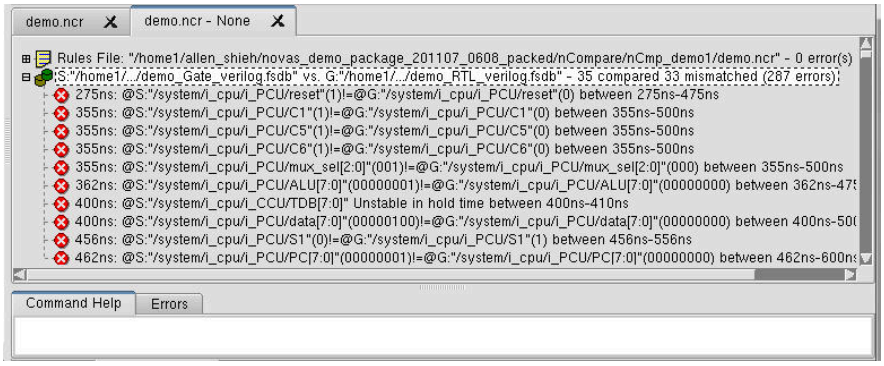
Sorted by Time

After the waveform comparison is completed, the *nCompare* frame shows the mismatches on the result window and sorts them by design hierarchy or mismatch time sequence depending on the view setting.

To view the mismatches by time sequence, choose **View -> View By Time** command.

The following figure shows the *nCompare* frame with the results sorted by time sequence.

Figure 286 nCompare Results Sorted by Time Sequence

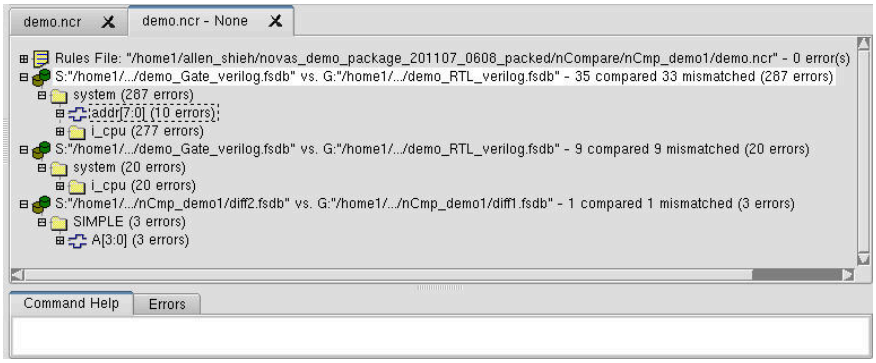


Sorted by Hierarchy

Alternatively, to view the mismatches by design hierarchy, choose **View -> View By Hierarchy** command. The following figure shows the *nCompare* frame with the results sorted by design hierarchy.

Feedback

Figure 287 nCompare Results Sorted by Design Hierarchy

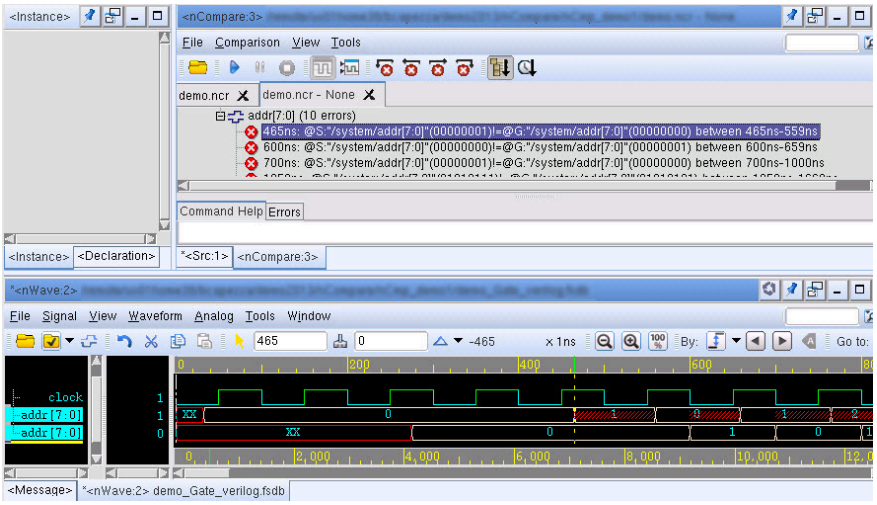


In the Waveform

The *nCompare* frame is tightly integrated with the *nWave* window for viewing mismatches. Perform the following steps to view the errors in *nWave*:

1. Confirm the **View -> View By Hierarchy** command is enabled and then click the + in the hierarchy view of the *nCompare* frame to view the mismatches.
2. Double-click a mismatch error node to add the compared signals into the *nWave* window (the *nWave* window is opened automatically if it is not opened). The cursor time in the *nWave* window is changed to the mismatch time. The result is similar to the following figure.

Figure 288 View Errors With the nWave Window



Feedback

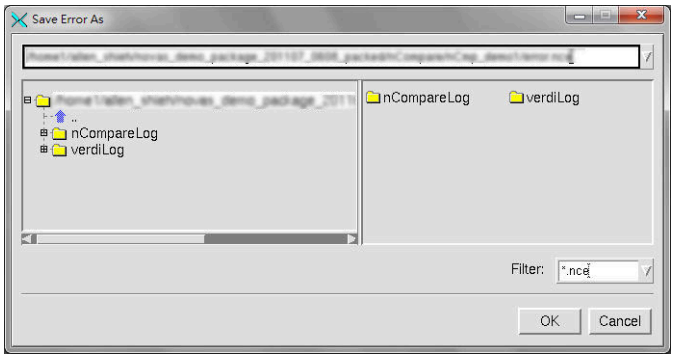
Error Report File

Save the Current Error File

The error report can be saved to a file for debug at a later time. To save the error file, perform the following steps:

1. In the *nCompare* frame, choose **File -> Save Error** command.
2. Type *error.nce* in the file name text field in the *Save Error As* form as shown in the following figure:

Figure 289 Saving the Error Report File



3. Click **OK**.

Note:
The recommendation is to save the error file with the *.nce extension.

Load the Previous Error File

An error report file that is previously saved can be loaded for additional debug. To load the previously saved error report file, perform the following steps:

1. In the *nCompare* frame, choose **File -> Open Error File** command.
2. In the *Open Error File* form, select the *error.nce* error file.
3. Click **OK**.

   The mismatch error report is displayed in the *nCompare* frame by time sequence (the default view). Errors can be viewed in the *nCompare* frame by following the steps in the Viewing Errors section.

Support Multi-process With CDPL

From 2018.09, nCompare supports command-line option `-proc` to fork multiple process for partition comparison. Each process produces the error report file and compare log with `.process%d` postfix. There was performance issue because each process spent time on preprocessing stage to collect all signals and make partition, and these sub-processes are still run on local same machine, which can result in machine resource issue.

This section consists of the following topics:

- Support Merge Error Report and Compare Log
- Support nCompare With CDPL

Support Merge Error Report and Compare Log

Merge the report files using the following command:

```
setenv CMP_MERGE_REPORT_FILE
```

Check the merged `*.nce` and `compare.log`

The error report file and the compare log of each process becomes hidden.

For regression purpose, provide the environment variables to get stable result and check the correction of the merged report:

```
setenv CMP_MULTI_PROC_STOP_EARLIER 0
setenv CMP_NCE_SORT_BY_HIERARCHY
```

- **CMP_MULTI_PROC_STOP_EARLIER**
  - Make the result of each multiple-process run stable
  - By default, this environment variable is set to 1. If you specify the maximum error number N, nCompare stops comparison when total number of errors in all processes exceeds the maximum error number. This performance improvement in the process may lead to unstable result.
  - Obtain the stable report by setting the environment variable `CMP_MULTI_PROC_STOP_EARLIER` to `0`, so each process tries to detect upto N mismatches and the merged result becomes more stable.
- **CMP_NCE_SORT_BY_HIERARCHY**

Feedback

    - Compare the results between multiple-process flow and single-process flow.
    - By default, the reports are sorted by time.
    - For regression purpose, to make it easier to check the merged result, set the environment variable `CMP_NCE_SORT_BY_HIERARCHY` to sort the report by hierarchy.

- If the maximum error N is small, the merged report may be changed with a different process number and the results become correct.

  For example,

```
cmpSetCmpOption -MaxError 2, -MaxErrorPerSignal 1
-single proc
    Sig1 0ns, Sig2 10ns => Sig1 0ns, Sig2 10ns
-proc 2
    Sig1 0ns, Sig2 10ns
    Sig3 3ns, Sig4 5ns => Sig1 0ns, Sig3 3ns
-proc 4
    Sig1 0ns, Sig2 10ns
    Sig3 3ns, Sig4 5ns
    Sig5 0ns, Sig6 6ns
    Sig7 0ns, Sig8 2ns => Sig1 0ns, Sig5 0ns
```

Limitation

Merging the report files is not supported when there are multiple `cmpOpenFsdb` in the rule file, or different signal pairs have different compare option. Separate NCE and `compare.log` for each process can be still checked.

Example

Case 1:

```
% nCompare -fsdb golden.fsdb -fsdb second.fsdb -proc 8 => support merge
 report file
```

Case 2:

```
% nCompare -rule rule1.nce => support merge#rule1.nce
#rule1.nce cmpOpenFsdb golden.fsdb second.fsdb
cmpSetSignalPair -level 0
cmpCompare
```

Case 3:

```
% nCompare -rule rule2.nce => multiple "cmpOpenFsdb" in rule file, not
 support merge report file
#rule2.nce
cmpOpenFsdb golden.fsdb second.fsdb
cmpSetSignalPair -level 0
cmpCompare
```

Feedback

```
cmpOpenFsdb golden2.fsdb second2.fsdb
cmpSetSignalPair -level 0
cmpCompare
```

Case 4:

```
% nCompare -rule rule3.nce => different signal pairs have different
 options, not support merge report file
#rule3.nce
cmpOpenFsdb golden.fsdb second.fsdb
cmpSetCmpOption -MaxError 10 -MaxErrorPerSignal 1
cmpSetSignalPair top.a -level 0
cmpSetCmpOption -MaxError 20 -MaxErrorPerSignal 2
cmpSetSignalPair top.b -level 0
cmpCompare
```

Support nCompare With CDPL

This section consists of the following topics:

- Integrate normal nCompare multi-process into CDPL
- Enable Runtime Scheduling
- Set Computing Farm of CDPL With Command Line Option
- Auto Set all Environment Variables Associated With CDPL

Integrate normal nCompare multi-process into CDPL

You may integrate the existing nCompare into CDPL without modifying any algorithm. The only difference is that the existing one can only invoke multi-process on the localhost. CDPL can extend the capability to several machines by which it can achieve higher scalability.

Set the following environment variable:

```
setenv VERDI_CDPL_ENABLE 1
setenv VERDI_CDPL_HOST_FILE [CDPL HOST FILE]
```

Extra Settings for Testing

```
setenv VERDI_CDPL_CONSOLE_OUT 1
```

Redirect the `stdout&stderr` of each worker at the log directory: `worker.W$WORKERID.$PID.stdout`

```
setenv VERDI_CDPL_WORKER_PREFIX_CMD [tool binary]
```

Run some profiling tool for each worker. For example, to view each worker's runtime and memory consumption, use `reschk`.

Feedback

Usage: `nCompare [-fsdb golden_waveform -fsdb secondary_waveform | -rule rule_file] -proc N`

Options:

`-proc N`: Specify the number of CDPL worker when enabling CDPL flow in nCompare

Note:
It needs Verdi license for each extra process, N process need N license.

Enable Runtime Scheduling

To enable runtime scheduling, set the environment variable `NCOMPARE_RUNTIME_DISPATCH_RATIO` using the following command:

```
setenv NCOMPARE_RUNTIME_DISPATCH_RATIO [ratio_number]
```

This environment variable is only valid when `VERDI_CDPL_ENABLE` is valid.

Some waveform database is too huge such that the worker needs to spend a lot of memory on it. To make sure each worker can have balanced loading and less memory consumption, you may use the environment variable to control this flow. You may divide the whole waveform database into different parts by scope and ask each worker load scope on demand instead of the whole one. This would be like producer-consumer scheme where in each worker consume partition one by one until all of them are finished.

For example, if you launch nCompare with `-proc 10` and set the above `ratio_number` to `20`, then you can divide the whole waveform database into 200 sections by scopes. 10 workers follow up producer-consumer scheme to consume the task until all of them are finished.

Set Computing Farm of CDPL With Command Line Option

Verdi nCompare supports the command-line option to set the computing farm of the CDPL conveniently instead of via the host file.

The syntax is as follows:

```
nCompare … -grid_type SGE|LSF|RTDA|PBS|NB -remote_command "$grid_command"
```

Example: `nCompare … -grid_type SGE -remote_command "qsub -P zebubench_berry_ctbm_zrs"`

Auto Set all Environment Variables Associated With CDPL

Verdi nCompare auto sets all the environment variables associated with CDPL when you specify `-remote_command` and `-grid_type` options at nCompare command line

or

when you set environment variable `setenv CMP_CDPL_ENABLE = 1`.

Feedback

The automatically set environment variables contain the following items:

- `setenv VERDI_CDPL_CONSOLE_OUT 1`
- `setenv VERDI_CDPL_WORKER_PREFIX_CMD reschk`
- `setenv NCOMPARE_RUNTIME_DISPATCH_RATIO 20`
- `setenv CMP_RESET_MEMORY_SIGNAL_NAME 0`
- `setenv CMP_MERGE_REPORT_FILE 1`

15

Application Tutorials

This chapter describes the following topics:

- Searching Backward for Value Causes
- Debug Memories
- Debug Gate vs. RTL Simulation Mismatch
- Behavior Trace for Root Cause of Simulation Mismatches
- Debug Unknown (X) Values
- Debug Forced Signals
- Debug With SystemVerilog
- Debug With SystemVerilog Assertions
- SVA Evaluation of Runtime Assertions
- Debug With Transactors

Searching Backward for Value Causes

When you trace the root cause of the signal value by the flow views, the values on the active fan-in signals are compared with the fan-out signal. If there are matching values, then they are taken as new root nodes and the operation continues. This allows you to track the first occurrence of a value of interest using a single command.

Assume that from your testbench, the *alubuf_out* signal in instance *i_ALUB* follows the sequence *0*, *1*, *2*, *3*, *4*, *5* commencing at time 0. However, it actually follows the sequence *0, 1, 2, 3, aa, 4, 5*. You need to find out the reason and the location from where the *aa* value is generated.

Before you begin this application, follow the instructions in the Before You Begin chapter.

Feedback

Open a Temporal Flow View

1. Change your context to the *verdi_mixed* sub-directory, where all the demo source code files are located:

   ```
   % cd <working_dir>/demo/verdi_mixed
   ```

2. Compile the mixed design to create a work.lib++:

   ```
   % ./compile.verdi
   ```

3. Start the Verdi platform by referencing the compiled design and the FSDB file *CPUsystem.fsdb* (contains a set of simulation results) on the command line:

   ```
   % verdi -lib work -top tb_cpusystem -ssf CPUsystem.fsdb -workMode
    -hardwareDebug &
   ```

4. Resize the *nWave* frame to a viewable size and locate it under the source code frame on your screen such that you can see both frames.

   The advanced drag-and-drop technology in Verdi platform allows you to quickly locate and display information related to selected objects.

5. In the **Instance** tab of the design browser frame, click the plus symbol to the left of the *i_cpusystem* block instance name to reveal its *i_cpu* and *i_pram* sub-blocks.

6. Double-click the *i_cpu* block instance name in the design browser frame to access the source code of this module, which is displayed in the source code frame and reveals its *i_ALUB, i_CCU*, and *i_PCU* sub-blocks.

7. Double-click *i_ALUB* to access the source code of this module.

8. Enter *AluBuf* in the **Find String** text box on the toolbar, and press <Enter> to find the signal.

   Note:
   Names are case-sensitive.

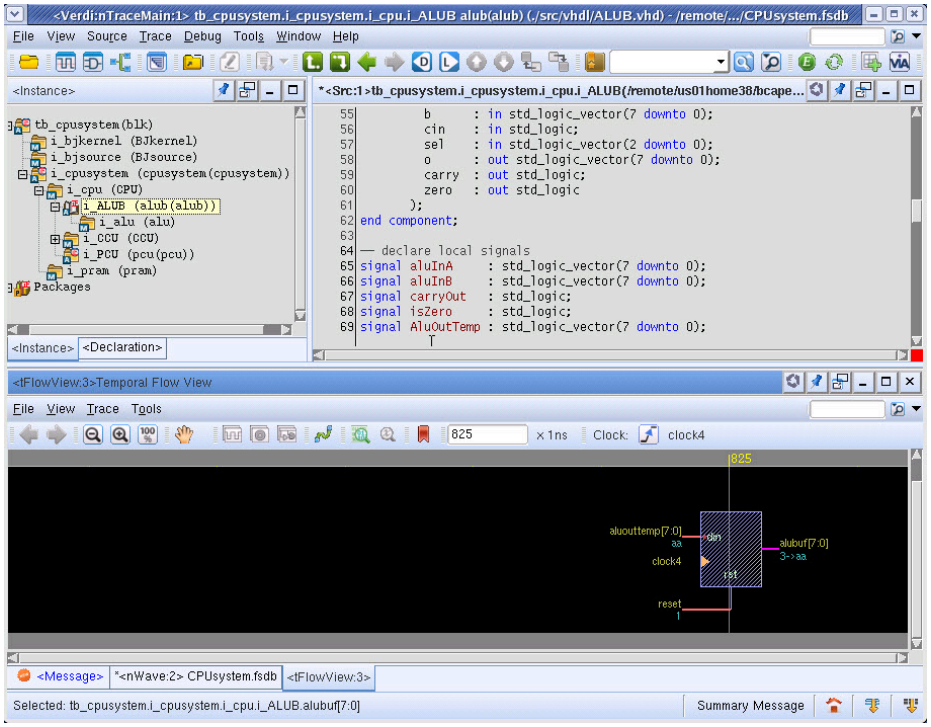
9. Use the middle mouse button to drag *AluBuf* from the source code frame, and drop in the signal pane on the left-hand side of the main *nWave* frame.

10. Right-click *alubuf* in *nWave*'s waveform pane on the transition from *3* to *aa* at time 825, and choose the **Temporal Flow View -> New Temporal Flow View** command.

Feedback

11. The *Temporal Flow View* frame is created as a new tab in the same location as the *nWave* frame.

Figure 290 Temporal Flow View Frame



Show Active Statements

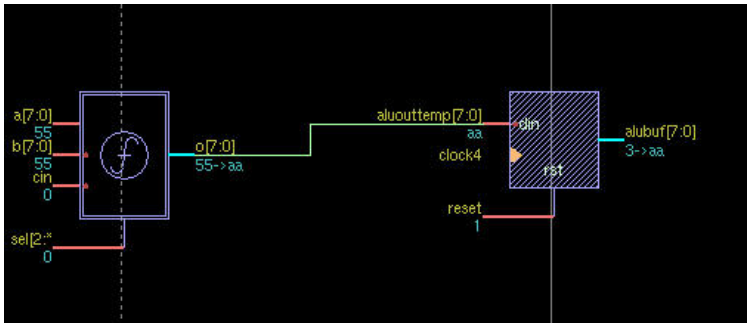
An active statement is the logic statement that generates the value for the driver signals of the current statement.

Feedback

1. Double-click the *aluouttemp[7:0]* signal in the *Temporal Flow View* frame. You can also right-click the *aluouttemp* node in the *Temporal Flow View* frame, and choose **Show Active Statement**.

   The *Temporal Flow View* frame updates, as shown in the following figure:

Figure 291 Temporal Flow View Frame Displaying Active Statements



Another symbol (a function block) is added to the *Temporal Flow View* window. You can continue tracing active statements on any input node.

Trace This Value Automatically

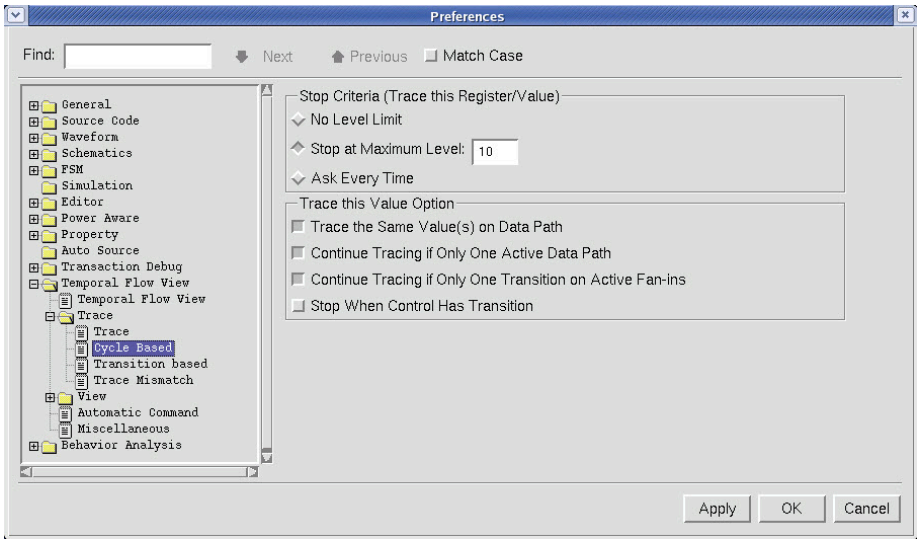
After you have traced back one statement and seen the function symbol, assume that you want to know where the *55* value on signal *a[7:0]* at time 800 comes from. You can use the **Trace This Value** command, which compares values on the active fan-ins with that of the fan-out signal. If matched values are found, they are taken as new root nodes and the operation continues. This allows the first occurrence of a value of interest to be tracked down using a single command.

1. Choose the **Tools -> Preferences** command to open the *Preferences* form and set all the options for the **Trace This Value** command.
2. In the *Preferences* form, select the **Cycle Based** page under the **Temporal Flow View -> Trace** folder.

Feedback

The form is similar to the following:

Figure 292 Preferences Form - Trace This Value Options



You have four options to customize the **Trace This Value** command:

- **Trace the Same Value(s) on Data Path**: If matched values are found among the active data path, they are taken as new root nodes and the trace continues.
- **Continue Tracing if Only One Active Data Path**: If there is only single active data fan-in, it is taken as a new root node and the operation continues.
- **Continue Tracing if Only One Transition on Active Fan-ins:** If there is only one active fan-in with transition, it is taken as a new root node and the operation continues.
- **Stop When Control Has Transition:** If the control signal has a transition, tracing stops.

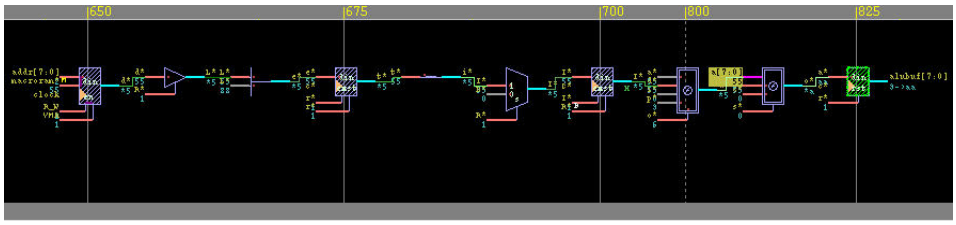
Confirm that the first three options are enabled and the last option is disabled.

3. Click the **OK** button on the *Preferences* form.
4. Right-click *a[7:0]* in the *Temporal Flow View* window, and choose **Trace This Value** from the menu.
5. Click the **Fit Time** icon (see left) on the toolbar to see the entire results.

Feedback

The *Temporal Flow View* frame updates similar to the following figure:

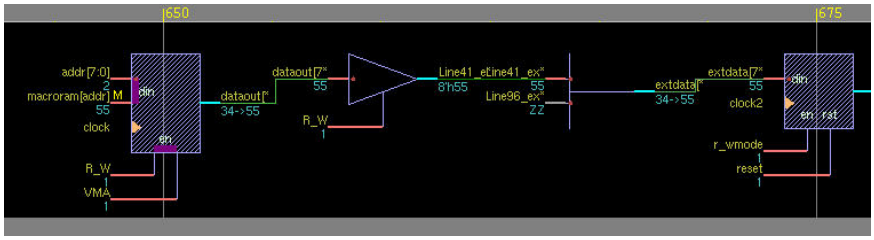
Figure 293 Trace This Value Results



The **Trace This Value** command traces back several fan-in cones and stops at a memory symbol at time 650.

6. Zoom in around the memory symbol, as shown in the following figure:

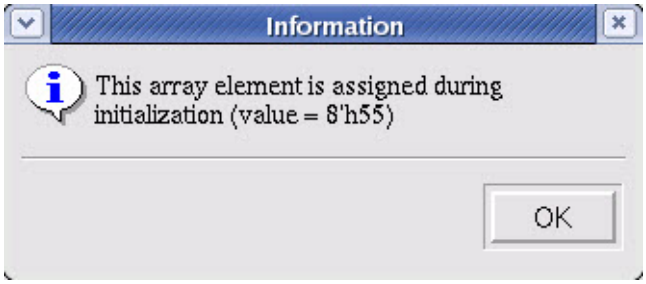
Figure 294 Memory Symbol



7. Double-click the *macroram[addr]* input node of the memory symbol to find the driving statement.

   The following message displays:

Figure 295 Information Dialog Window



This message means that the memory content is initialized in the initial block and its value is 55. No further tracing on the *Temporal Flow View* frame is possible. Refer to the Debug Synthesizeable Memory Models section for more details.

8. Click the **OK** button on the *Information* dialog window.

Feedback

You can also complete any of the previous steps using either the *Compact Temporal Flow View* or *Temporal Register View* windows. These views can be opened from the **Tools -> Open Flow View** command, or use the right mouse button and choose **Open Flow View**. The same cause is located. However, the view looks different.

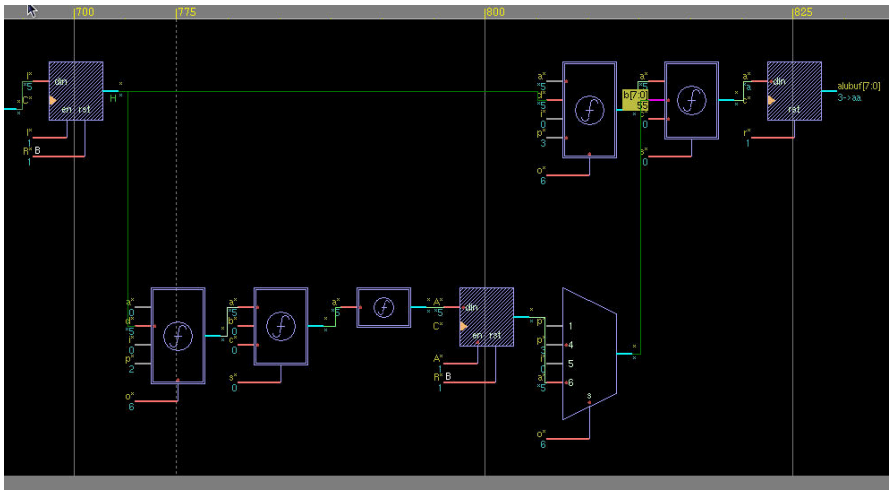
Trace Another Path

Now that you have traced one path on the *Temporal Flow View* frame, assume you also want to trace the 55 value on *b[7:0]* (where *b[7:0]* is another input node on the function symbol).

1. Right-click *b* in the *Temporal Flow View* window, and select **Trace This Value**.
2. Click the **Fit Time** icon.

   Note that another path displays.
3. Zoom in from time 700-825 to more easily see the details. The frame is similar to the following:

Figure 296 Trace Multiple Paths



Although *a* and *b* follow different paths initially, at time 700 they come from the same register.

4. Turn on the **View -> Signal -> Active Nodes Only** toggle command to display only the active signals and reduce the clutter.
5. Turn off the **View -> Signal -> Active Nodes Only** toggle command.

Feedback

Show Signals on *nWave*

At any time, you can select a node and add its waveform to *nWave*. You can also add all fan-in signals for a node.

1. Select the output node *o[7:0]* on the right most function symbol.
2. Use **Ctrl+W** to add its waveform to *nWave*.
3. Use **Ctrl+A** to add all its fan-in signals to *nWave*.

   You can also select a symbol to drag and drop to *nWave*; for example, the register symbol for *AluBuf*.
4. When you are done tracing paths in the *Temporal Flow View* frame and adding signals to *nWave*, choose the **File -> Exit** command in the *nTrace* main window to close the Verdi session.

Debug Memories

The Verdi platform provides a set of features that allow you to debug memories without rewriting your design to note memory contents or adding special tasks to dump memory contents to a file during simulation.

For memory models that can be synthesized, the Verdi platform can calculate memory values dynamically, display memory contents in the waveform display and identify the most recent write time for a specific memory location. All these features are enabled by the Behavior Analysis engine and does not require you to dump the memory contents to a file during simulation.

For memory models that cannot be synthesized, the Verdi platform provides the Memory Definition Table (MDT) template that can be used to define the control signals like write, enable, address and data for a memory. After the memory model is defined through the template, you can view the memory contents, display them in the waveform window, and identify the most recent write time of any address location.

This tutorial illustrates these memory debug features on a simple design.

This section covers the following topics:

- Debug Synthesizeable Memory Models
- Debug Non-synthesizeable Memory Models
- Debug PLI Memory Models

Before you begin this application, follow the instructions in the Before You Begin chapter.

Feedback

Debug Synthesizeable Memory Models

In this example, you traces the value in an incorrect transition on signal *ACC* back to its source - an instance of a memory model. You can then use the memory debug features to inspect the memory and determine when the value was stored.

Locate the Cause of a Value on a Signal

Enter the following commands to start the tutorial:

1. Change to the demo directory.

   ```
   % cd <working_dir>/demo/verilog/cpu/src
   ```

2. Start the Verdi platform and reference the file *run.f* in the current directory and the FSDB file *CPUsystem.fsdb* in the parent directory:

   ```
   % verdi -f run.f -ssf ../CPUsystem.fsdb -workMode hardwareDebug &
   ```

3. Display the waveform for signal *ACC[7:0]* in module instance *i_ALUB* in *nWave*. You can do this by dragging the signal from the source code to the waveform pane or by using the *Get Signals* form in *nWave*.

4. Zoom out in *nWave* until you see *55* at time 801 ns.

   In this example, you can locate the cause of a value in the *nWave* frame instead of a *Temporal Flow View* frame. The Behavior Analysis engine is still used -- the results are displayed in a more familiar format. See the Open a Temporal Flow View section for details on using the *Temporal Flow View* window.

5. Click the *ACC* signal in the *nWave* frame somewhere close to the transition from the *00* value to the *55* value and note the following:

   - A vertical cursor appears in the waveform pane.
   - The simulation time of 801 associated with the cursor's current location is displayed in the *nWave* toolbar.

     Note:
     By default, the cursor snaps to the closest transition on the selected signal, the transition from *00* to *55* in this case.

6. Choose the **Tools -> Preferences** command to set up the preferences for tracing on *nWave*. Then, in the *Preferences* form:

   a. Select the **Automatic Command** page under the **Temporal Flow View** folder.

   b. Turn on the **Highlight Signal Value When Trace on nWave** option.

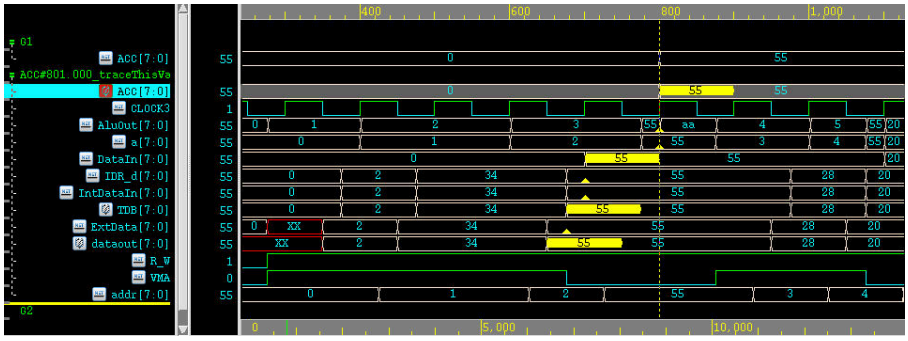
   c. Click the **OK** button.

Feedback

7. Right-click *ACC* in *nWave* on the transition from 00 to 55 and choose the **Trace This Value** command to trace the propagation of 55 across multiple cycles.
8. In the *Question* dialog window for Behavior Analysis, click **Yes**.
9. In the *Behavior Analysis* form, click the **OK** button.

   Several signals display in *nWave* in a new group, as shown in the following figure:

Figure 297 Display of New Group of Signals in nWave



The Verdi platform traces several cycles back and locates the origin of the *55*, which is signal *dataout*. This signal is the output of a memory. You can confirm this by double-clicking the transition from 34-55 on *dataout* to find the driving statement. In the source code frame, you can see *dataout* assigned by *macroram[addr].*

In the waveform, you can see that the control signals for the memory were automatically included.

The command stops at the statement that reads memory element *macroram[addr]* into dataout signal. At the time the *addr* signal has a value of 2 and the memory content that is being read out is 55.

The memory content did not get dumped out during simulation. The Verdi Behavior Analysis engine infers the value from the circuit description and simulation dump file.

Locate the Last Write of a Specific Address Location

When a memory output value is wrong, it is usually caused by one of two problems:

- The wrong data was written into the location you are reading.
- You are reading from the wrong address.

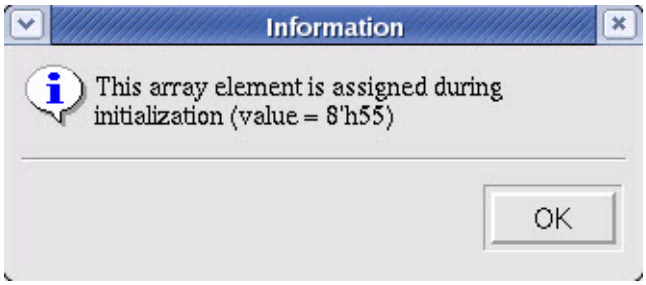
Feedback

First, let's find out when the *55* was written into address *2*:

1. In the *nWave* frame, double-click the transition from 34-55 on *dataout* to find the driving statement.
2. In the source code frame, right-click *macroram[addr*] on line 88 (confirm the cursor is over the *macroram*) and choose the **Debug Memory -> Trace Memory Write** command.

   The Verdi platform locates the last write, which was during initialization in this case.

Figure 298 Question Dialog Window



   The Verdi platform analyzes the control logic for the memory, locates the values of these signals in the FSDB file, and determines when the last write to this location occurred.

   Note:
   If you had traced the 55 value of *ACC* in the *Temporal Flow View* window, you only need to double-click the memory node to locate the last write.

3. Click the **OK** button on the *Information* dialog window.

   The initialization statement for the memory is displayed in the source code frame.

   Now, assume that the initialization file is correct, and you want to know if any other memory locations contain the correct data.

Show Memory Contents

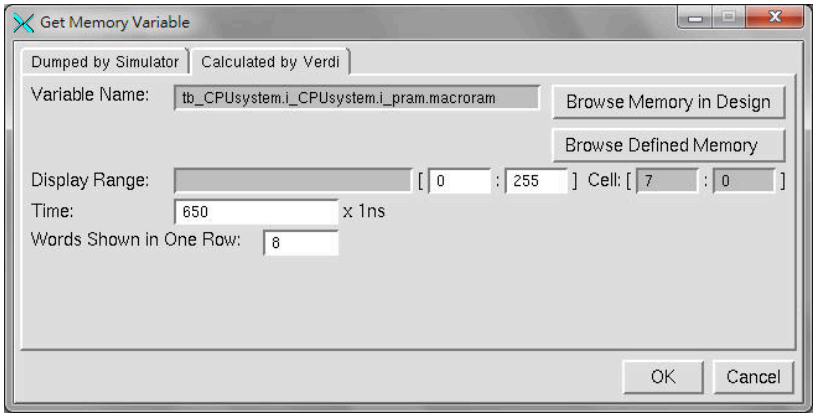
The Behavior Analysis engine can calculate the memory contents at any time you specify:

1. In *nWave*, double-click *dataout* at the 34-55 transition to go to the memory statement again.
2. With *macroram[addr]* selected, right-click and choose the **Debug Memory -> Show Memory Contents** command.

Feedback

The *Get Memory Variable* form with the **Calculated by Verdi** tab selected displays, as shown in the following figure:

Figure 299 Get Memory Variable - Calculated by Verdi

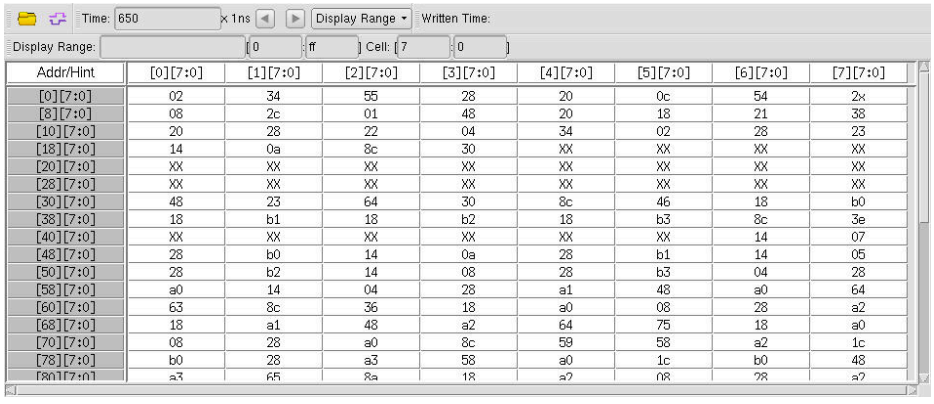


The default values of the start and end addresses are the same as the declaration of the memory in the RTL code. The time value is set to the time selected in the waveform (or flow view) by default. In this example, you have specified that you want the Verdi platform to calculate the memory contents from address 0 to address 255 at time 650.

3. Confirm 650 is entered in the **Time** text field and 0:255 is entered in the **Display Range** text fields.
4. Click the **OK** button.

   The memory values are calculated and displayed in the *nMemory* frame (in the lower-right of the main window), as shown in the following figure:

Figure 300 Show Memory Contents in nMemory



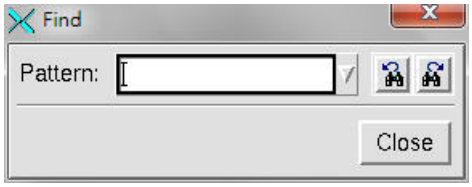
Addr/Hint	[0][7:0]	[1][7:0]	[2][7:0]	[3][7:0]	[4][7:0]	[5][7:0]	[6][7:0]	[7][7:0]
[0][7:0]	02	34	55	28	20	0c	54	2x
[8][7:0]	08	2c	01	48	20	18	21	38
[10][7:0]	20	28	22	04	34	02	28	23
[18][7:0]	14	0a	8c	30	XX	XX	XX	XX
[20][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[28][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[30][7:0]	48	23	64	30	8c	46	18	b0
[38][7:0]	18	b1	18	b2	18	b3	8c	3e
[40][7:0]	XX	XX	XX	XX	XX	XX	14	07
[48][7:0]	28	b0	14	0a	28	b1	14	05
[50][7:0]	28	b2	14	08	28	b3	04	28
[58][7:0]	a0	14	04	28	a1	48	a0	64
[60][7:0]	63	8c	36	18	a0	08	28	a2
[68][7:0]	18	a1	48	a2	64	75	18	a0
[70][7:0]	08	28	a0	8c	59	58	a2	1c
[78][7:0]	b0	28	a3	58	a0	1c	b0	48
[80][7:0]	a3	65	8a	18	a2	08	28	a2

Feedback

Search Values in the nMemory Frame

1. Choose the **Search -> Find** command to open the *Find* form, as shown in the following figure:

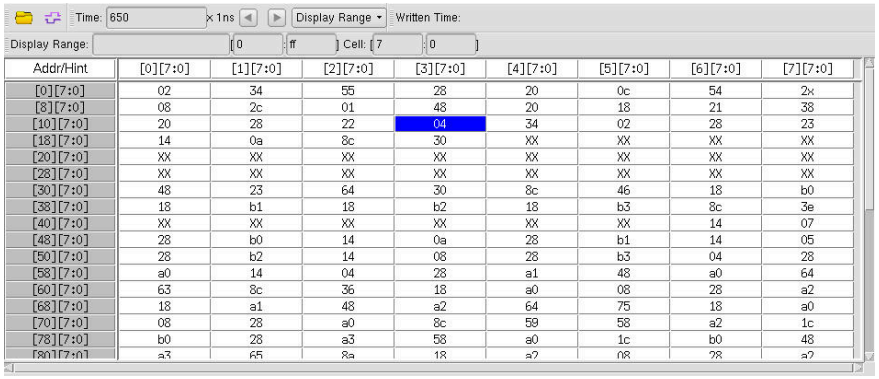
Figure 301 Find Form



2. Enter a value to search for. For example, 04.
3. Click the **Find Next/Previous** icons to search for the value.

   The required value is highlighted in the *nMemory* window, as shown in the following figure:

Figure 302 nMemory Window with Highlighted Value



Addr/Hint	[0][7:0]	[1][7:0]	[2][7:0]	[3][7:0]	[4][7:0]	[5][7:0]	[6][7:0]	[7][7:0]
[0][7:0]	02	34	55	28	20	0c	54	2x
[8][7:0]	08	2c	01	48	20	18	21	38
[10][7:0]	20	28	22	04	34	02	28	23
[18][7:0]	14	0a	8c	30	XX	XX	XX	XX
[20][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[28][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[30][7:0]	48	23	64	30	8c	46	18	b0
[38][7:0]	18	b1	18	b2	18	b3	8c	3e
[40][7:0]	XX	XX	XX	XX	XX	XX	14	07
[48][7:0]	28	b0	14	0a	28	b1	14	05
[50][7:0]	28	b2	14	08	28	b3	04	28
[58][7:0]	a0	14	04	28	a1	48	a0	64
[60][7:0]	63	8c	36	18	a0	08	28	a2
[68][7:0]	18	a1	48	a2	64	75	18	a0
[70][7:0]	08	28	a0	8c	59	58	a2	1c
[78][7:0]	b0	28	a3	58	a0	1c	b0	48
[80][7:0]	a3	65	8a	18	a2	08	28	a2

4. Click the **Close** button on the *Search Pattern* form.

Synchronize the nMemory Frame With nWave

1. Turn on the **Time -> Sync Cursor Time** toggle command to synchronize the time in the *nMemory* frame with *nWave*.

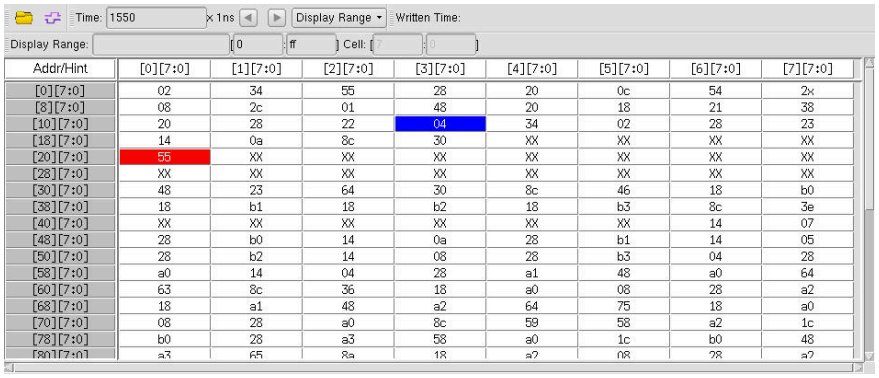
   When you move the cursor in *nWave*, the contents in the *nMemory* frame are updated.

Feedback

2. Move the cursor in *nWave* to time 1550.

   Note that the value at address *20* changed from *X* to *55*, as shown in the following figure:

Figure 303 nMemory Frame with Synchronized Time



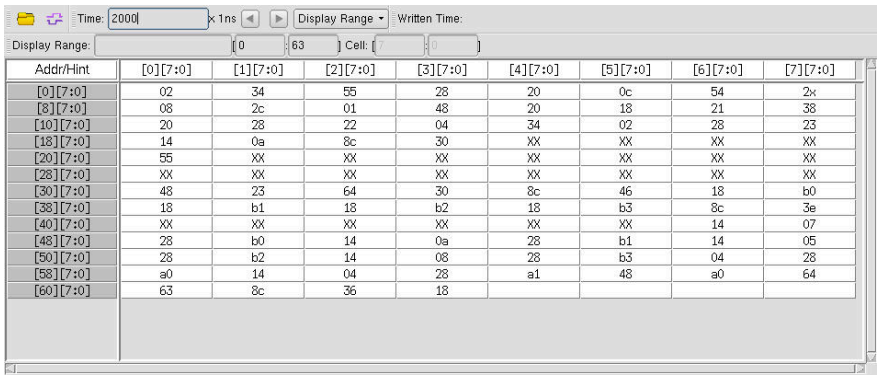
Addr/Hint	[0][7:0]	[1][7:0]	[2][7:0]	[3][7:0]	[4][7:0]	[5][7:0]	[6][7:0]	[7][7:0]
[0][7:0]	02	34	55	28	20	0c	54	2x
[8][7:0]	08	2c	01	48	20	18	21	38
[10][7:0]	20	28	22	04	34	02	28	23
[18][7:0]	14	0a	8c	30	XX	XX	XX	XX
[20][7:0]	55	XX	XX	XX	XX	XX	XX	XX
[28][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[30][7:0]	48	23	64	30	8c	46	18	b0
[38][7:0]	18	b1	18	b2	18	b3	8c	3e
[40][7:0]	XX	XX	XX	XX	XX	XX	14	07
[48][7:0]	28	b0	14	0a	28	b1	14	05
[50][7:0]	28	b2	14	08	28	b3	04	28
[58][7:0]	a0	14	04	28	a1	48	a0	64
[60][7:0]	63	8c	36	18	a0	08	28	a2
[68][7:0]	18	a1	48	a2	64	75	18	a0
[70][7:0]	08	28	a0	8c	59	58	a2	1c
[78][7:0]	b0	28	a3	58	a0	1c	b0	48
[80][7:0]	a3	65	8a	18	a2	08	28	a2

Change Address and Time in the nMemory Frame

1. On the *nMemory* toolbar, change the end address in the **Display Range** field (second text field) to *63* and press Enter on the keyboard.
2. Change **Time** to *2000* and press Enter on the keyboard.

   The *nMemory* frame updates, as shown in the following figure:

Figure 304 Reduced Address Range in nMemory



Addr/Hint	[0][7:0]	[1][7:0]	[2][7:0]	[3][7:0]	[4][7:0]	[5][7:0]	[6][7:0]	[7][7:0]
[0][7:0]	02	34	55	28	20	0c	54	2x
[8][7:0]	08	2c	01	48	20	18	21	38
[10][7:0]	20	28	22	04	34	02	28	23
[18][7:0]	14	0a	8c	30	XX	XX	XX	XX
[20][7:0]	55	XX	XX	XX	XX	XX	XX	XX
[28][7:0]	XX	XX	XX	XX	XX	XX	XX	XX
[30][7:0]	48	23	64	30	8c	46	18	b0
[38][7:0]	18	b1	18	b2	18	b3	8c	3e
[40][7:0]	XX	XX	XX	XX	XX	XX	14	07
[48][7:0]	28	b0	14	0a	28	b1	14	05
[50][7:0]	28	b2	14	08	28	b3	04	28
[58][7:0]	a0	14	04	28	a1	48	a0	64
[60][7:0]	63	8c	36	18				

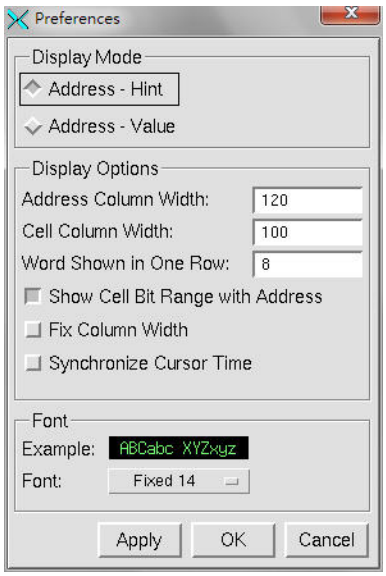
Feedback

Customize the nMemory Window

1. Choose the **Options -> Preferences** command to customize the *nMemory* display.

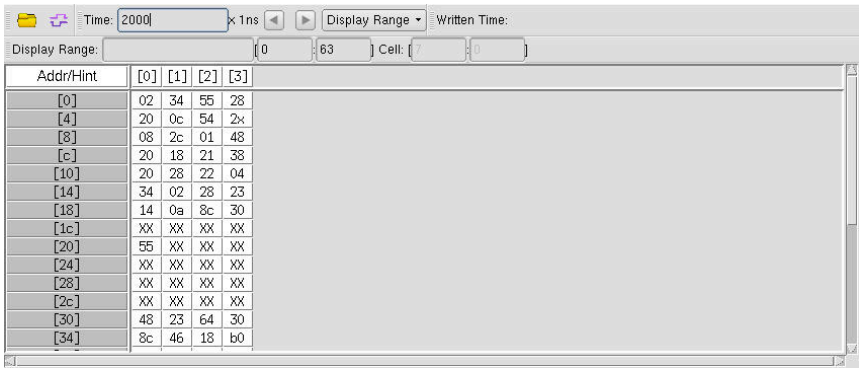
   The *Preferences* form displays, as shown in the following figure:

Figure 305 nMemory Preferences Form



2. Enter *4* in the **Words Shown in One Row** text field and *30* in the **Cell Column Width** text box and disable the **Show Cell Bit Range with Address** option.
3. Click the **OK** button. The *nMemory* frame redisplays with the new options:

Figure 306 nMemory Frame with New Display Options

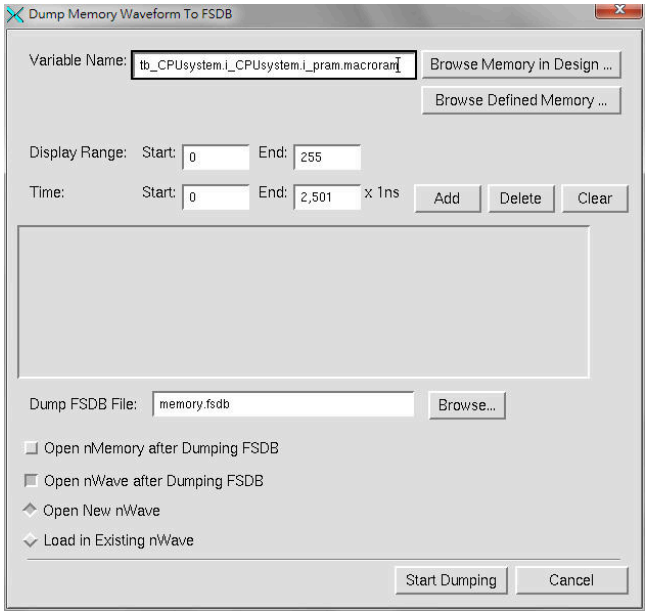


Feedback

Display Calculated Memory Contents in nWave

1. In the source code frame, right-click the *macroram[addr]* memory variable, and choose the **Debug Memory -> Dump Memory Waveform to FSDB** command to open the *Dump Memory Waveform To FSDB* form.
2. In the *Dump Memory Waveform To FSDB* form, specify the start (0) and end (255) address, the start (0) and end (2501) time, and the name of the FSDB file that you want to write to. Use the values shown in the following figure:

Figure 307 Dump Memory Waveform to FSDB Form



3. Click the **Add** button.
4. Click the **Start Dumping** button to create the FSDB file and display a new *nWave* frame with the FSDB file loaded

   [Optional] You can also load the new FSDB file into the original *nWave* frame. Choose the **File -> Open** command and select the appropriate FSDB file.
5. In *nWave*, choose **Get Signals** to select the memory to display in the waveform.

   Note:
   In the **Find Signal** box in the *Get Signals* form, you can specify which memory range to add. For example, entering *macroram[255:250]* only lists those address elements for that selection.
6. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

Feedback

Debug Non-synthesizeable Memory Models

Non-synthesizeable memory models are those HDL models described using non-synthesizeable constructs. Since the Verdi platform cannot infer the read/write operation from these HDL models, the memory content cannot be extracted automatically. The Verdi platform provides a way for you to enter the read/write operation manually to extract memory content.

In this tutorial, two static RAM models are used to illustrate the usage:

- 1-port Static RAM
- Multiple-port Static RAM

These examples shows you how to describe the Verdi templates for memory model for non-synthesizeable one or multiple-port memories. After you have described the templates, you can use the Verdi platform to trace memory contents and dump memory content to FSDB.

1-port Static RAM

Enter the following commands to start the tutorial:

1. Change to the demo directory:

   ```
   % cd <working_dir>/demo/verilog/memory_demo/1port
   ```

2. Invoke the Verdi platform using a command file:

   ```
   % verdi -play demo.cmd
   ```

   This loads the design and FSDB, and opens the GUI.

Create a Memory Model Definition for the 1-port Static RAM

1. In the source code frame, select *mem[addr]* in hierarchy *tb.s0.*

   Note:
   This should automatically be highlighted by the command file.

   The following figure shows the concurrent block that describes the write operation to the 2-D array.

Feedback

Figure 308 Source Code Frame

```
 98 always @( posedge clk )
 99   if ( strb ) begin
100     if (~rw_) begin
101        temp_data = mem[addr];
102        if ( !be_[3] )  temp_data[ 31: 24] = din[ 31: 24];
103        if ( !be_[2] )  temp_data[ 23: 16] = din[ 23: 16];
104        if ( !be_[1] )  temp_data[ 15:  8] = din[ 15:  8];
105        if ( !be_[0] )  temp_data[  7:  0] = din[  7:  0];
106        mem[addr] = temp_data;
107     end
108     else begin
109        dout <= mem[addr];
110        @( posedge clk )
111        dout <= 32'hz;
112     end
113   end
```

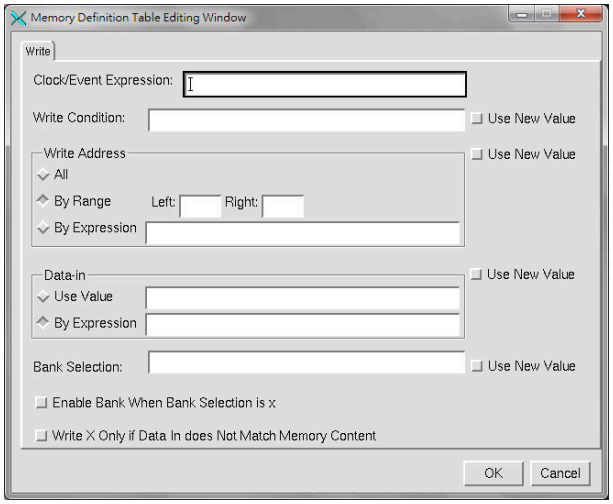
2. Right-click *mem[addr]*, and choose the **Debug Memory -> Memory Definition Table** command.

   The *Memory Definition Table* form displays. Confirm the default **Module** name is *sram* and the default **Array** is *mem*.

3. In the *Memory Definition Table* form, click the **Add** button to add the memory module name to the list.

4. In the **Operation List** section, toggle to select **Write**, and click the **Add** button. The *Memory Definition Table Editing Window* displays as per the following figure.

Figure 309 Memory Definition Table Editing Window - Write Operation



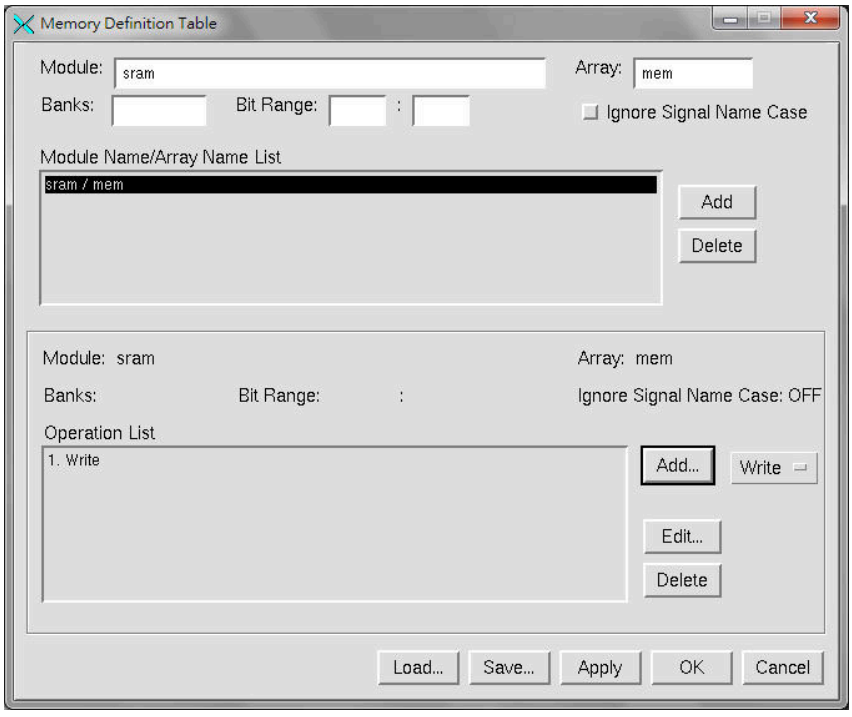
Feedback

5. Enter the following, or drag and drop from *nTrace* to this table:
   - Clock/Event Expression: *@(posedge clk)*
   - Write Condition: *strb && ~rw_*
   - Write Address: select the **By Expression** option and enter *addr*
   - Data In: select the **By Expression** option and enter *temp_data*

     The **Use New Value** option specifies that the new value should be used for the calculation if there is a value change on *temp_data* at the clock expression; for example, the positive edge of *clk*. Because *temp_data* is assigned and used within the same "*always*" statement, you need to make sure the Verdi platform uses the new values if there are any value changes on *temp_data* at the clock positive edge.
6. Turn on the **Use New Value** option for the **Data In** field.
7. Click the **OK** button in the *Memory Definition Table Editing Window* form.

   This definition is saved, and the list is shown in the **Memory Definition Table - Operation List**.

Figure 310 Memory Definition Table Form



8. [Optional] click the **Save** button to save the definition to a file for future use.

Feedback

9. Click the **OK** button to close the window and load the memory definition in the Verdi platform.

   You can use the Verdi memory debug features on this one-port static RAM.

Trace the Memory Contents

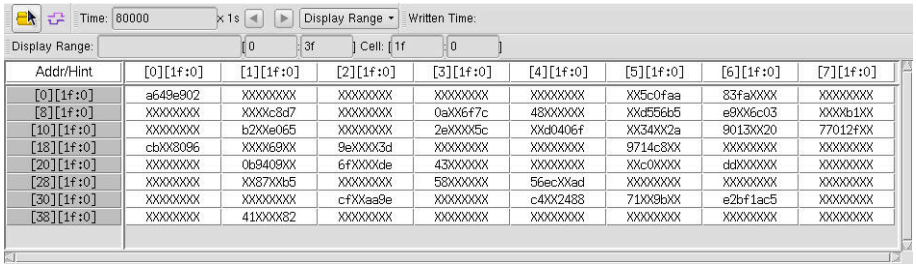
1. In the source code frame, select the 2D-array signal *mem[addr]*.
2. Right-click *mem[addr]* and choose the **Debug Memory -> Show Memory Contents** command and click **Yes** the on the *Question* dialog window to perform Behavior Analysis.
3. In the *Behavior Analysis* form, click **OK**.
4. The *Get Memory Variable* form opens.

   Suppose you want to see the memory content for element 0 to 63 at time 80000.

5. On the **Calculated by Verdi** tab of the *Get Memory Variable* form, enter 80000 in the **Time** text field, and click the **OK** button.

   An *nMemory* frame displays the specified memory, as shown in the following figure:

Figure 311 nMemory Frame Displaying Memory Content



6. Click the value for any address element.

   The Verdi platform reports when the memory element was written on the toolbar.

Display the Memory Contents in *nWave*

1. In the source code frame, select the 2D-array signal *mem[addr]*.
2. Right-click *mem[addr],* and choose the **Debug Memory -> Dump Memory Waveform to FSDB** command.
3. Set the following in the table:
   - Start Display Range: *0*
   - End Display Range: *63*

Feedback

- Start Time: *0*
- End Time: *80000*

4. Click the **Add** button and then the **Start Dumping** button.

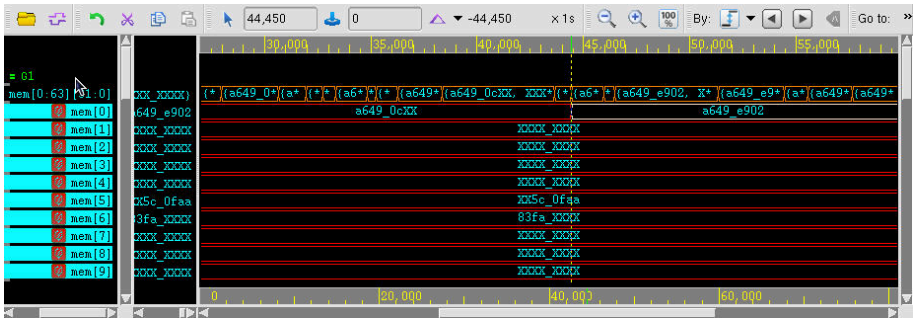
   The Verdi platform displays a new *nWave* window loaded with *memory.fsdb*, which contains the calculated memory contents.

5. In the new *nWave* window, choose the **Signal -> Get All Signals** command to display the memory, click **OK** on the *Confirmation* question box.

6. In *nWave*, double-click the memory signal to expand and show every address of the memory.

7. Zoom out until you can see values on *mem[0].* Note that *mem[0]* changes to "*a649_e902*" at time 44450, as shown in the following figure:

Figure 312 nWave Frame Displaying Memory Contents



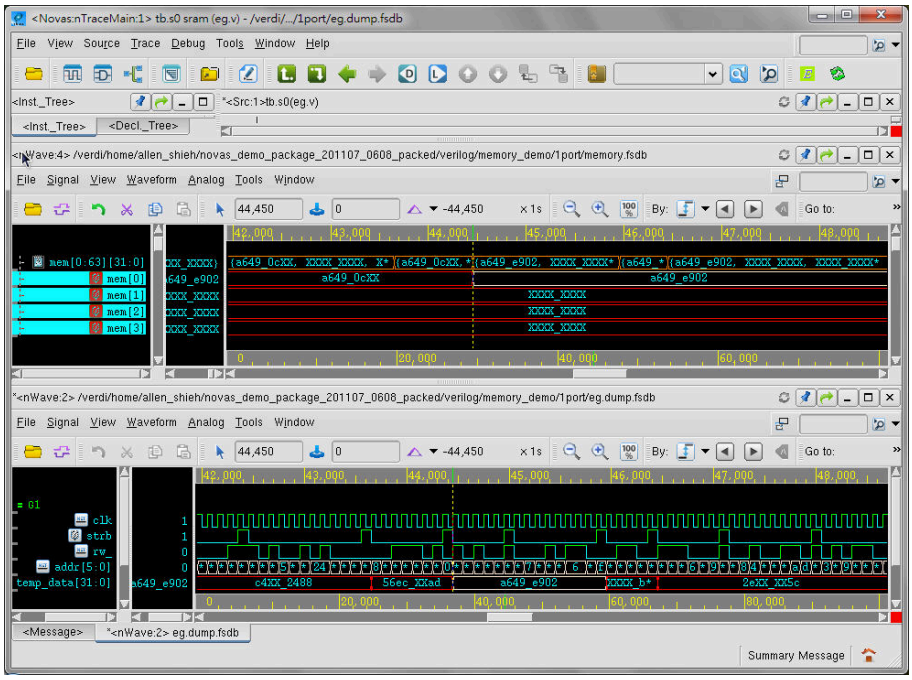
   To verify that the memory location *0* has a write operation, check the memory control logic.

8. Move the new *nWave* frame to a docking position above the original *nWave* frame and arrange the two waveform frames so that you can see both. You can hide the *nMemory* frame or other frames if necessary.

   For example rearrange the frame as shown in the following figure.

Feedback

Figure 313 Rearrange to See Both nWave Frames



9. In the *nWave* frame that is loaded with *memory.fsdb*, choose the **Window -> Sync Waveform View** command to synchronize both waveform frames.
10. Click *mem[0]* at the transition "*a649_e902*" at time 44450. This also changes the cursor time in the other frame to 44450.
11. Locate the control signals *strb* and *rw_* in the waveform. You can see that they are performing a "write" operation at this time and the address is *0*.

    [Optional] You can also load the new FSDB file into the original *nWave* window. Choose the **File -> Open** command, and select the appropriate FSDB file.
12. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

Multiple-port Static RAM

Enter the following commands to start the tutorial:

1. Change to the demo directory:

   ```
   % cd <working_dir>/demo/verilog/memory_demo/mport
   ```
2. Start the Verdi platform:

   ```
   % verdi -play demo.cmd
   ```

Feedback

Create a Memory Model Definition for the Multiple-port Static RAM

1. In the **Instance** tab of the design browser frame, select hierarchy *tb.s0*.

   The following figure shows the concurrent block that describes the write operation to the 2-D array.

Figure 314 nTrace Code Excerpt

```
62
63  reg [7:0] mem[63:0];
64
65  assign #5 dout = (~ce_ & ~pe3_)? mem[addr_o]: 8'bzzzzzzzz;
66
67  always @(posedge clk)
68  begin
69    if(~ce_ && ~pe1_) begin
70       #3 mem[addr1] = din1;
71    end
72    else if (~ce_ && ~pe2_) begin
73       #3 mem[addr2] = din2;
74    end
75  end
76
```

2. Select *mem[addr1],* and then choose the **Tools -> Memory Definition Table** command to create the model.
3. In the *Memory Definition Table* form, enter *sram* in the **Module** text box, *mem* in the **Array** text box.
4. Click the **Add** button to add the memory to the list.
5. In the **Operation List** section, toggle to the **Write** condition, and click the **Add** button.
6. In the *Memory Definition Table Editing Window* form, enter the following, or drag and drop from *nTrace* to this table:
   - Clock/Event Expression: *@(posedge clk)*
   - Write Condition: *~ce_&&~pe1*
   - Write Address: select the **By Expression** option and enter *addr1*
   - Data In: select the **By Expression** option and enter *din1*
7. Click the **OK** button.
8. In the **Operation List** section, click the **Add** button again to create a second write operation.

Feedback

9. In the *Memory Definition Table Editing* form, enter the following:
   - Clock/Event Expression: *@(posedge clk)*
   - Write Condition: *~ce_ && ~pe2_*
   - Write Address: select the **By Expression** option and enter *addr2*
   - Data In: select the **By Expression** option and enter *din2*
10. Click the **OK** button in the *Memory Definition Editing* form.
11. Click the **OK** button on the *Memory Definition Table* form.

    The template is created, and you can calculate and display the memory contents as previously described.
12. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

Debug PLI Memory Models

The Verdi platform can help to trace the content of PLI memory models. The following shows an example of how it works in the Verdi platform.

Create a PLI Memory Definition File

Enter the following commands to start the tutorial.

1. Change to the demo directory:

   ```
   % cd <working_dir>/demo/verilog/memory_demo/PLI_memory
   ```
2. Invoke the Verdi platform:

   ```
   % verdi -play demo.cmd
   ```

   This loads the design, and the FSDB file.
3. Double-click the instance *tb.s0* in the design hierarchy frame to display the associated source code.

   The module defines the PLI tasks for defining memory and the timing for read/write into the memory.

Figure 315 Code Excerpt for Defining a PLI Memory

```
73 initial
74 begin
75     $damem_declare("mem",0,63,0,31);
76 end
```

Feedback

Figure 316 Code Excerpt for Reading a PLI Memory

```
124 always @( strb or rw_ or addr )
125       if( strb && rw_ )
126         $damem_read("mem", addr, dout);
127
```

Figure 317 Code Excerpt for Writing a PLI Memory

```
108 always @( posedge clk )
109   if ( strb ) begin
110     if (~rw_) begin
111       // temp_data = mem[addr];
112       $damem_read("mem", addr, temp_data);
113       if ( !be_[3] )  temp_data[ 31: 24] = din[ 31: 24];
114       if ( !be_[2] )  temp_data[ 23: 16] = din[ 23: 16];
115       if ( !be_[1] )  temp_data[ 15:  8] = din[ 15:  8];
116       if ( !be_[0] )  temp_data[  7:  0] = din[  7:  0];
117       // mem[addr] = temp_data;
118       $damem_write("mem", addr, temp_data);
119     end
120   end
```

The first step is to prepare a PLI memory function definition file. This file tells the Verdi platform what the function is for the PLI declaration and what the function is for memory write.

4. Create a new file that contains the following three lines:

```
API_MEM_DECL $damem_declare(MNAME, ADDR_LEFT, ADDR_RIGHT, RANGE_LEFT,
 RANGE_RIGHT);
API_MEM_WRITE $damem_write(MNAME, ADDR, DATAIN);
API_MEM_READ $damem_read(MNAME, ADDR, DATAOUT);
```

5. Save the file as *pli_memory.def*.

   The first line indicates the PLI memory is defined using the *$damem_declare* function. MNAME is a keyword. It means the first parameter of the function is the memory name in the HDL design.

   The second line indicates data is written into the PLI memory using the *$damem_write* function. ADDR is a keyword, which means the second field represents the address. *DATAIN* is a keyword, which means the third field represents the data to be written into the PLI memory.

   The third line indicates data is read from PLI memory using the *$damem_read* function. ADDR is a keyword, which means the second field represents the address. *DATAOUT* is a keyword, which means the third field represents the data to be read from the PLI memory.

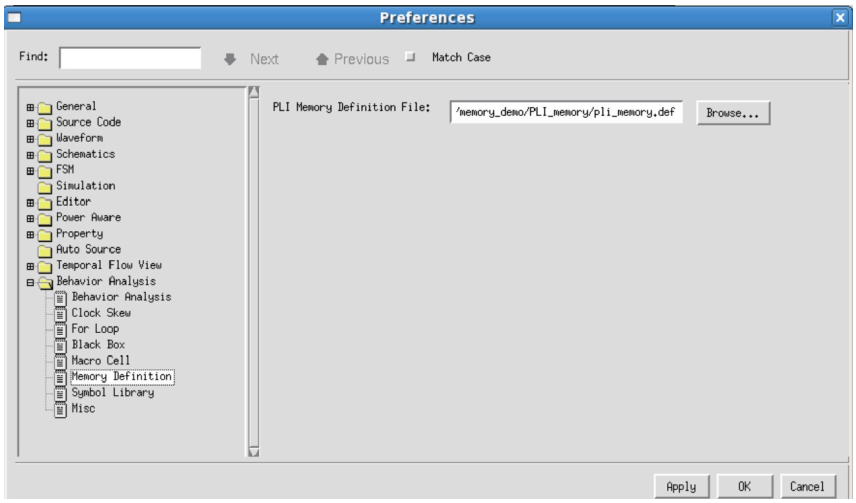
Feedback

Load the PLI Memory Definition File

Perform Behavior Analysis, and load the PLI memory definition file:

1. Choose the **Tools -> Preferences** command, and then select the **Memory Definition** page under the **Behavior Analysis** folder.
2. Enter the PLI memory definition file (pli_memory.def) in the **PLI Memory Definition File** field, as shown in the following figure:

Figure 318 Load PLI Memory Definition File



3. Click the **OK** button to perform Behavior Analysis.

   The Verdi platform performs analysis on the design and extracts the write conditions for the PLI memory that is used to trace memory content.

Trace the Content of the PLI Memory

After loading the PLI definition file, you are ready to trace the content of the PLI memory.

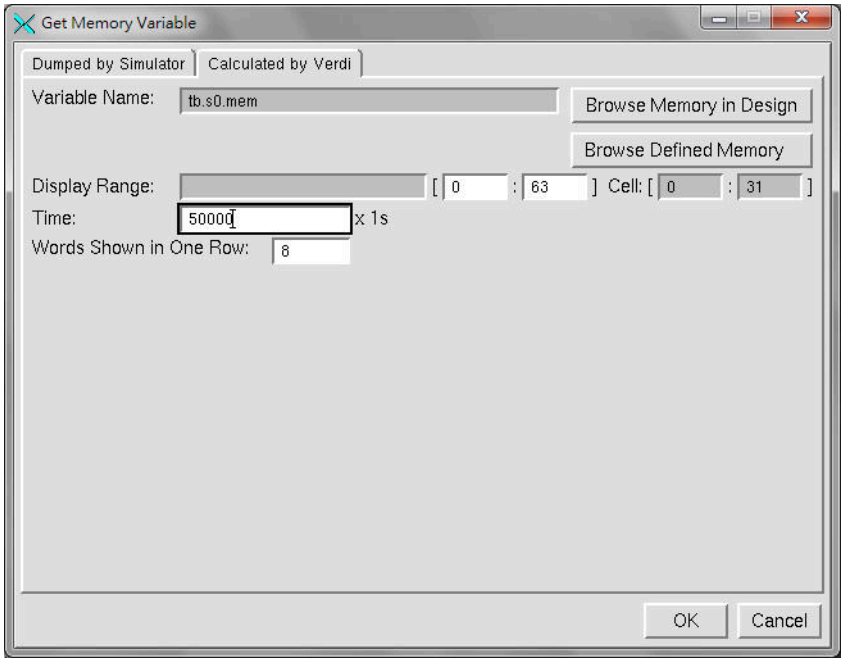
1. In the source code frame, locate *mem* in $*damem_declare("mem",0,63,0,31)*; on line 75.
2. Right-click *mem*, and choose the **Debug Memory -> Show Memory Contents** command.

   Although it may look unselectable because the text is black, double-clicking on *mem* selects it.
3. On the **Calculated by Verdi** tab of the *Get Memory Variable* form, enter the address range and the simulation time.

Feedback

For example, enter 0 and 63 in the **Display Range** text fields and 50000 in the **Time** text field, as shown in the following figure.

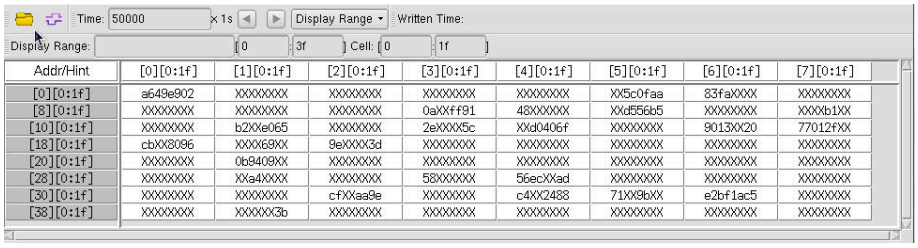
Figure 319 Get Memory Variable - Calculated by Verdi



4. Click the **OK** button.

   The Verdi platform analyzes the content of the PLI memory from 0 to 63 at time 50000 and display the results in the *nMemory* frame:

Figure 320 nMemory Frame for PLI Memory



5. Click any memory element whose value is not X.

   The table reports when the content has been written into the address.

You can also dump PLI memory waveform to FSDB as previously described in 1-port Static RAM.

6. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

Debug Gate vs. RTL Simulation Mismatch

Before you begin this application, follow the instructions in the Before You Begin chapter.

This tutorial guides you through a scenario explaining how to debug a design if discrepancies occur between RTL and gate-level simulations. In this example, you can debug a mismatch on *carry_flag*.

nWave provides a comprehensive comparison capability to compare simulation results from different simulation runs automatically. *nWave* graphically displays the mismatches in the waveform window after comparison. You can step through each mismatch to analyze the differences.

Locate the Signal to Compare

Typically you must build a gate-level symbol library for your design using the following command:

```
% syn2SymDB synopsys.lib
```

Note:
synopsys.lib is not included in the Verdi package. It is available from other Synopsys tool packages.

For this example, the symbol library is already built and is included in the installation.

1. Set the environment variables for the symbol library using the following commands:

```
% setenv NOVAS_LIBPATHS <VERDI_INST>/share/symlib/32 %
setenv NOVAS_LIBS lsi10k_u
```

2. cd to *<working_dir>/demo/verilog/gate*.
3. Compile the gate-level design using the following command:

```
% vericom -f run.f
```

4. Load the compiled design using the following command:

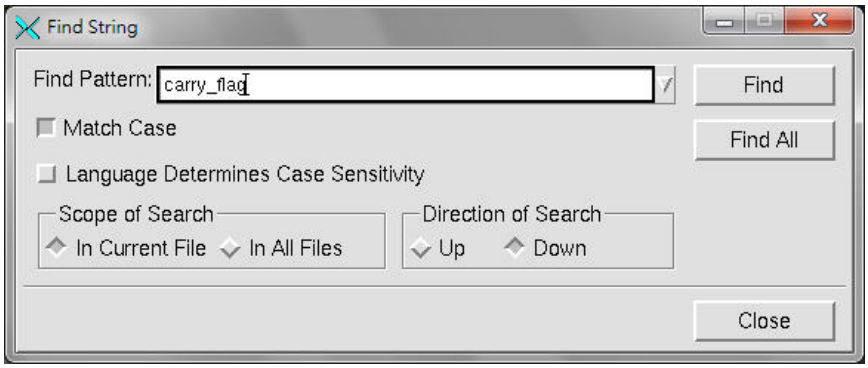
```
% verdi -lib work -top system -workMode hardwareDebug &
```

Feedback

5. In the main window, choose the **Source -> Find String** command to find *carry_flag* through a string search.

   A *Find String* form displays, as shown in the following figure.

Figure 321 Find String Form



6. Enter *carry_flag* in the **Find Pattern** text box.
7. Select **In All Files**.
8. Click the **Find All** button.

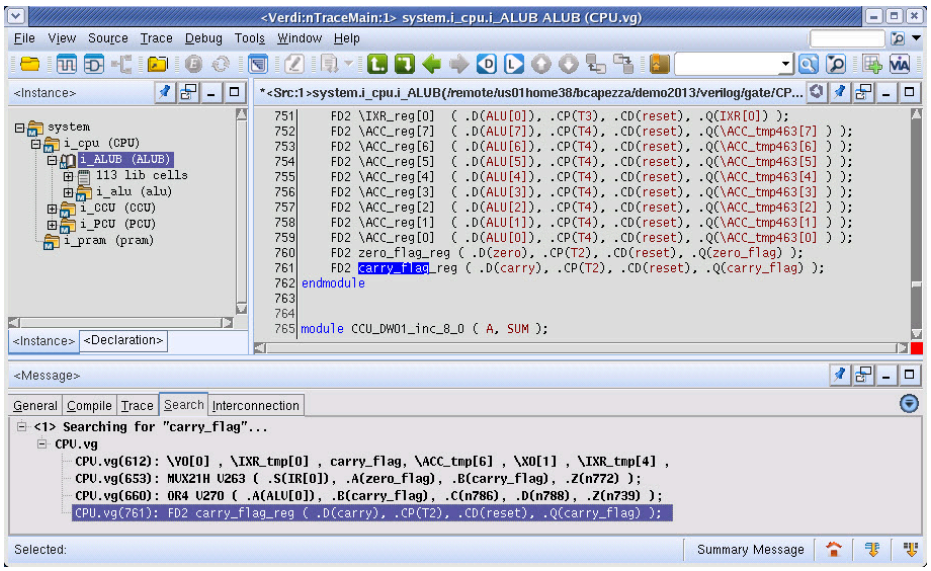
   In the **Search** tab of the message frame, you can see that *carry_flag* is the output of *carry_flag_reg*.

9. Click the **Close** button on the *Find String* form.

Feedback

10. In the **Search** tab of the message frame, double-click the driver, *FD2,* which takes you to the corresponding line in the source code.

Figure 322 Find String Results

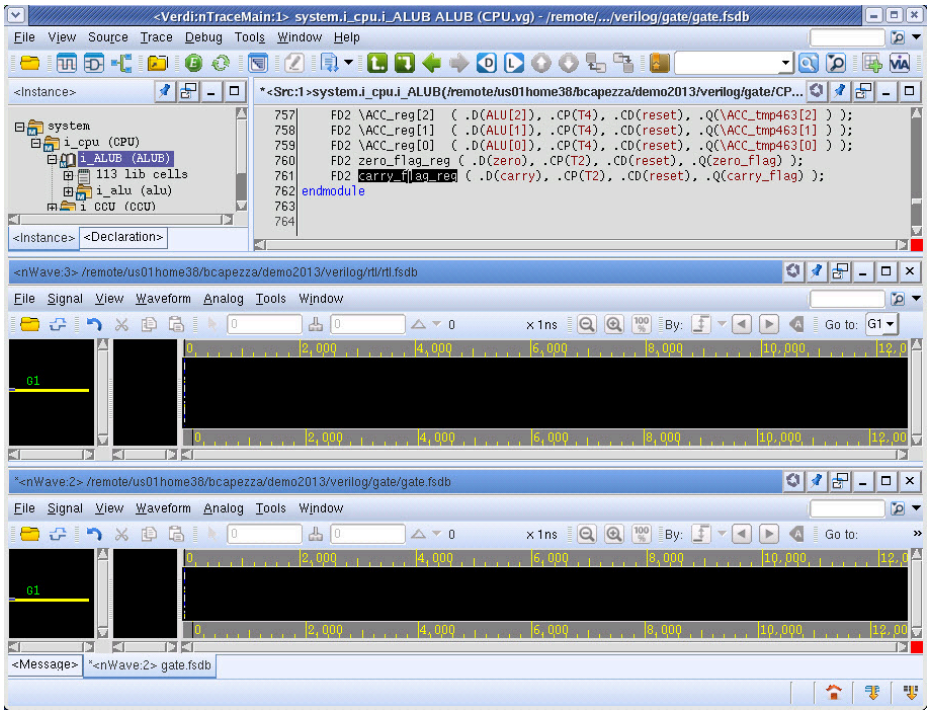


Load Simulation Results and Display Waveforms

1. From the main window, choose the **Tools -> New Waveform** command or click the **New Waveform** icon on the toolbar open the *nWave* frame.
2. In the *nWave* frame, choose the **File -> Open** command to open the *Open Dump File* form and load the gate-level simulation results.
3. Select *gate.fsdb.*
4. Click the **Add** button and then the **OK** button.
5. In the opened *nWave* frame, choose the **Tools -> New Waveform** command to open another *nWave* frame.
6. From this new *nWave* frame, choose the **File -> Open** command to load the RTL simulation result.
7. In the *Open Dump File* form, select *../rtl/rtl.fsdb*.
8. Click the **Add** button and then the **OK** button.
9. Move the new *nWave* frame to a docking position above the original *nWave* frame and arrange the two waveform frames so that you can see both. For example rearrange the frames as shown in the following figure.

Feedback

Figure 323 Compare Two nWave Frames



10. Drag and drop the instance *carry_flag_reg* to both *nWave* frames.

    Note:
    You can determine the gate waveform by looking at *nWave*'s title bar for *gate.fsdb*. In the gate-level *nWave* frame, choose the **Window -> Sync Waveform View** command. However, you can tile windows in any *nWave* window.

11. In the both *nWave* frames, choose the **Window -> Sync Waveform View** command to synchronize the viewing based on the simulation time.

12. Use the Pan Left and Pan Right keys (arrow keys on keyboard) to see the effect with synchronized viewing.

    All the viewing operations and cursor and marker positions under one frame are reflected to the other frame except for the Pan Up and Pan Down scrolling.

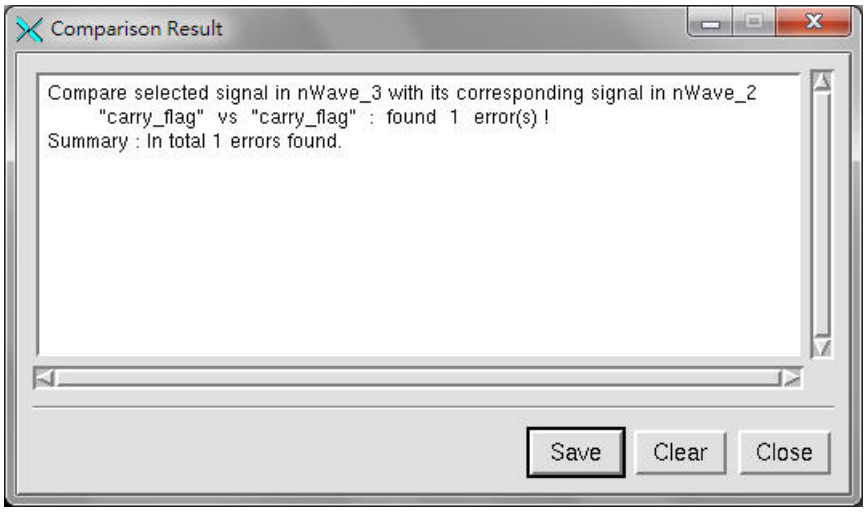
Feedback

Compare the Simulation Results

1. Select *carry_flag* in both *nWave* frames.
2. In the gate-level *nWave* frame, choose the **Tools -> Waveform Compare -> Compare Selected Signals** command to compare the simulation results.

   After the comparison is complete, *nWave* displays a dialog window that shows that there is 1 mismatch and the **Search By** toolbar would be changed to **Search By Mismatches**.

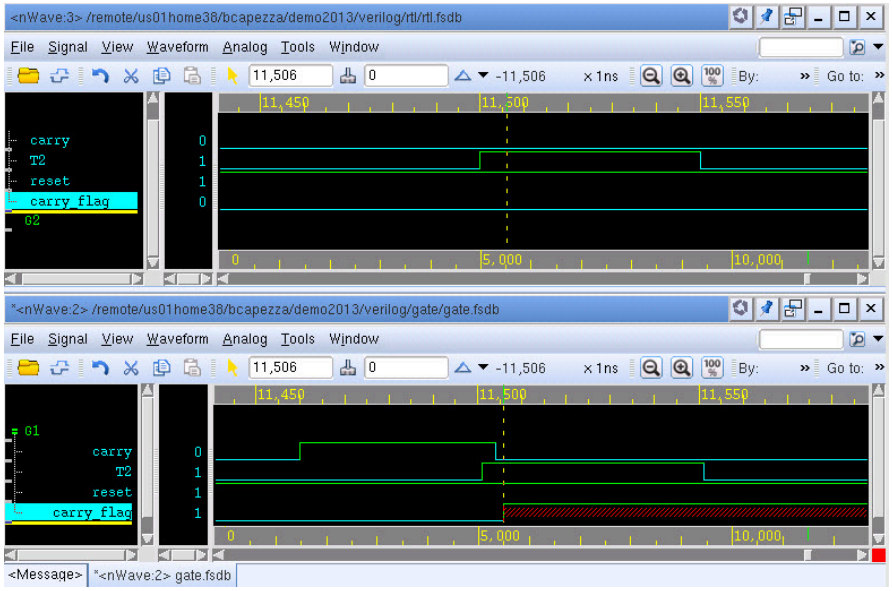
Figure 324 Comparison Result Message Window



3. On the *Comparison Results* window, click the **Close** button.
4. In the gate-level *nWave* frame, locate the mismatch (indicated by red hatch marks) by clicking the right arrow (**Search Forward**) icon on the toolbar.

Feedback

Figure 325 Mismatch Results in nWave frames



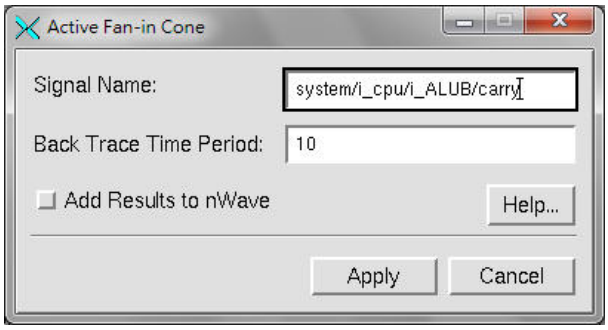
The input (*carry*) to the register in the gate-level design changes too close to the clock edge, thereby causing the mismatch.

Isolate the Problem

1. To find the active driver in the source code frame, double-click the rising edge of *carry* in the gate *nWave* frame.
2. In the *nTrace* main window, choose the **Tools -> New Schematic from Source -> Fan-In** command to generate the fan-in cone for *carry*.

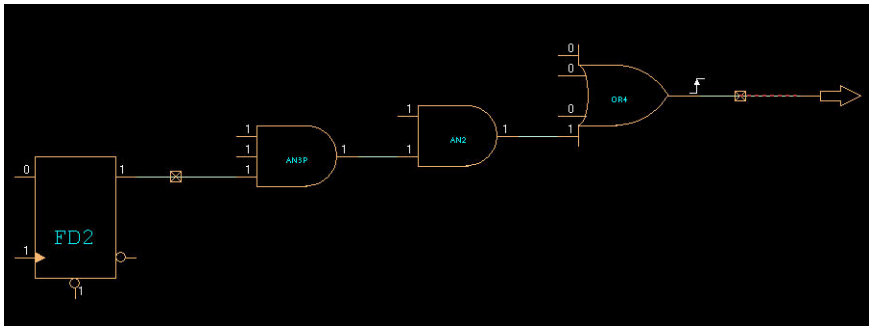
   Note:
   It takes a couple of seconds for this to occur due to the fact that the fan-in cone is quite large which makes it difficult to debug.
3. In the gate-level *nWave* frame, select *carry* and put the cursor on the rising edge of *carry* at time 11460.
4. Choose the **Tools -> Active Fan-In** command and specify *10* in the **Back Trace Time Period** field.

Figure 326 Active Fan-in Cone Window



5. Click the **Apply** button. The **Fan-in** logic has been reduced to four gates and is now ready for further analysis.

Figure 327 Active Fan-in Results



6. Exit the Verdi session.

Behavior Trace for Root Cause of Simulation Mismatches

Simulation mismatches between two simulation runs for the same design occur due to any of the following reasons:

- Different optimization levels applied in a simulator.
- Different versions of a simulator or libraries.
- Different compile-time options.
- Different simulators.

Using the Verdi platform, you can locate the cause of this simulation mismatch by tracing back the behavior and comparing the results of two simulation files to identify the root cause of the mismatches.

Feedback

This feature is different from the *nCompare* module or waveform compare in the Verdi platform in which the goal is to find all the mismatches between two FSDB files.

Before you begin this application, follow the instructions in the Before You Begin chapter.

Locate the Simulation Mismatch

Enter the following commands to start the tutorial and then use waveform compare to locate the signal and time that mismatches.

1. Change to the demo directory:

   ```
   % cd <working_dir>/demo/verilog/cpu/src
   ```

2. Start the Verdi platform and reference the file *run.f* in the current directory and the FSDB file *CPUsystem.fsdb* in the parent directory:

   ```
   % verdi -f run.f -ssf ../CPUsystem.fsdb -workMode -hardwareDebug&
   ```

3. Display the waveform for *AluBuf* signal from the *ALUB* module in the *nWave* frame by dragging from the source code frame or using **Get Signals**.

4. Open another *nWave* frame by clicking the **New Waveform** icon on the toolbar of the main window.

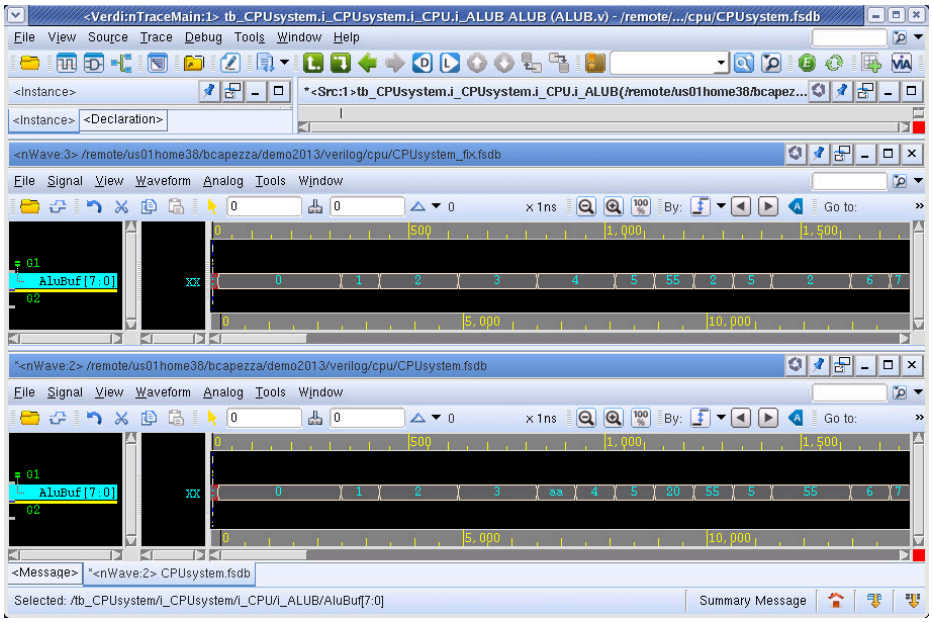
5. In the new *nWave* frame, choose the **File -> Open** command to load the *../CPUsystem_fix.fsdb* file.

6. In the *Open Dump File* form, click **Add** and then **OK** to complete the load.

7. Move the new *nWave* frame to a docking position above the original *nWave* frame and arrange the two waveform frames so that you can see both. For example, rearrange the frames as shown in the following figure.

Feedback

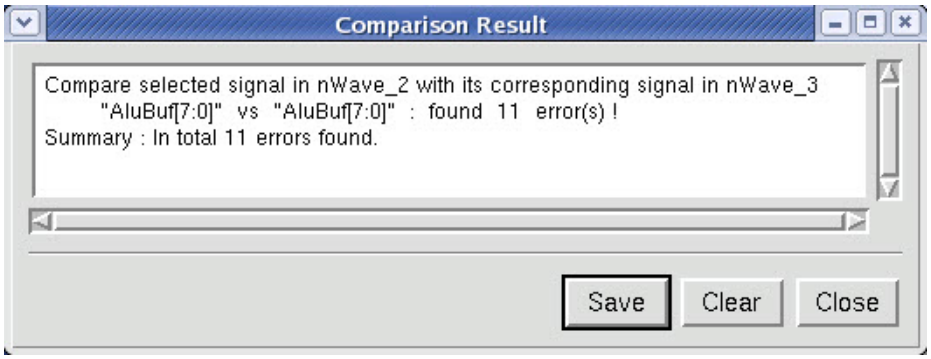
Figure 328 Compare Two Waveforms



8. Drag and drop the *AluBuf* signal from the first *nWave* frame to the second *nWave* frame.
9. Choose the **Window -> Sync Waveform View** command in both *nWave* frames to synchronize the two *nWave* frames.
10. Select the *AluBuf* signal on both *nWave* frames.
11. In the original *nWave* frame (the one that displays *CPUsystem.fsdb* in the toolbar), choose the **Tools -> Waveform Compare -> Compare 2 signals** command.

    The *Comparison Results* dialog window opens, indicating 11 mismatches, as shown in the following figure:

Figure 329 Comparison Result Window



Feedback

12. Click the **Close** button on the *Comparison Result* dialog window.
13. Use the **Search Forward** icon in the original *nWave* frame to locate the first mismatch at time 826ns. One has value *aa* and the other is *4*.

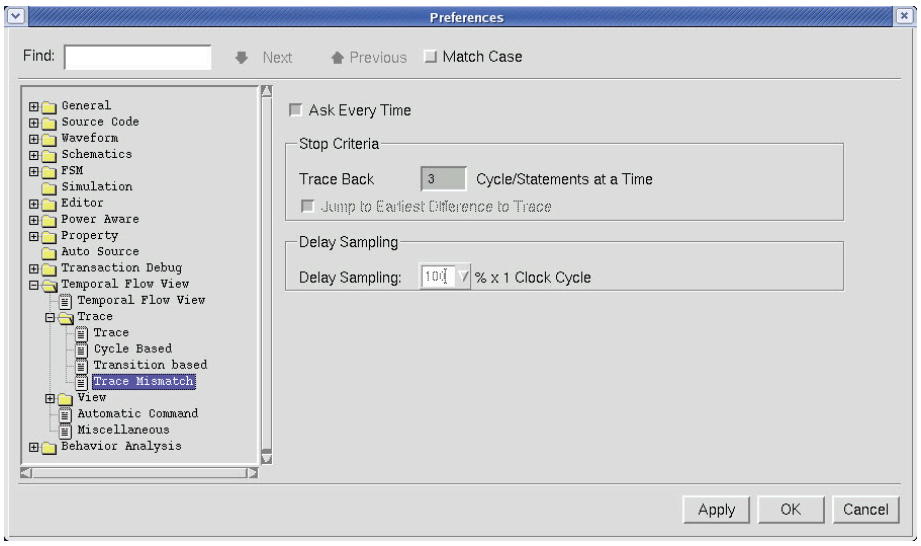
    Now that a mismatch point is located, let's find out the cause.
14. Continue with the next section, Behavior Trace for the Root Cause of Mismatch.

Behavior Trace for the Root Cause of Mismatch

1. Click the *AluBuf* signal in the first *nWave* frame at time 826.
2. Right-click, and choose the **Temporal Flow View -> New Temporal Flow View** command to create the *Temporal Flow View* frame.
3. In the *Temporal Flow View* frame, select the **File -> Load 2nd Waveform for Trace Mismatch** command to load the *CPUsystem_fix.fsdb* file.
4. Before tracing the mismatch, choose the **Tools -> Preferences** command to open the *Preferences* form and customize the options.

   In the *Preferences* form, select the **Temporal Flow View** folder **-> Trace** folder **-> Trace Mismatch** page, similar to the following form:

Figure 330 Preferences Form - Trace Mismatch Options



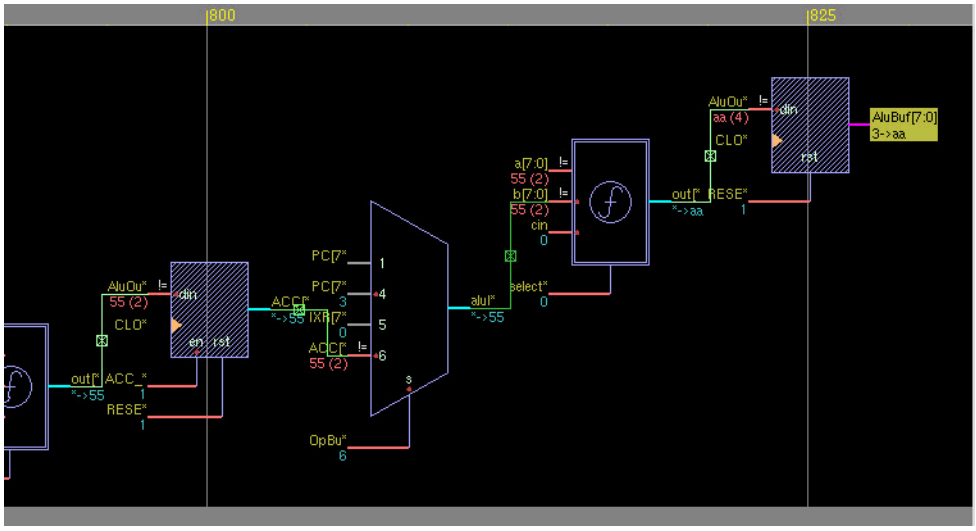
- **Ask Every Time**: the option form opens every time you start the behavior trace.
- **Trace Back N Cycles/Statements at a Time**: specify how many cycles or statements to trace back and compare.

Feedback

- **Jump to Earliest Difference to Trace:** for the fan-in signal that has a different value, instead of continuing the same Behavior Analysis process at the time the value is different, jump to the earliest time there is a difference in the FSDB and continue the trace process.

5. In the *Preferences* form, turn off the **Ask Every Time** and **Jump to Earliest Difference to Trace** options and change the **Trace Back** value to *20*.
6. Click the **OK** button to close the form.
7. Right-click *AluBuf* in the *Temporal Flow View* frame, and choose the **Behavior Trace for FSDB Mismatch** command.
8. After the results are displayed, zoom in around time 800-825 by dragging-left on the time display.

Figure 331 Behavior Trace Mismatch Results



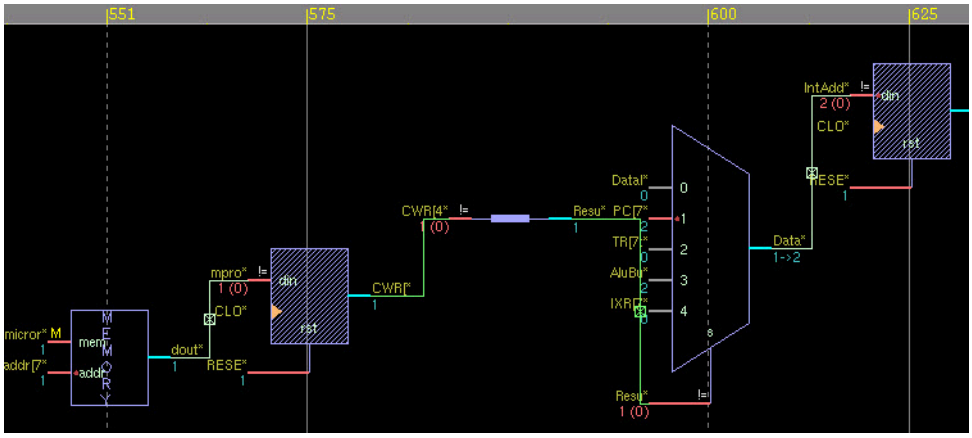
The value of *AluBuf* in the two FSDB files is different at time 800 as shown in the right-most function symbol. The *b* path is automatically traced. The **Behavior Trace** command works in the following way:

- First the fan-in cone of the selected signal is expanded.
- Then, the Behavior Analysis engine is used to determine what the value is and when it happens for the fan-in signals to produce the value of the selected signal.
- The value and time of these fan-in signals is then used to compare them with those in the second FSDB. A mismatch mark is set on the fan-in signals that have different values.

Feedback

9. Scroll to the left end and note that there is still a mismatch on the *b* path at time 575. The next element to the left is a memory - it does not have a mismatch.

Figure 332 Memory without Mismatch



10. Double-click the memory input node to find out when it was written.

    It was initialized to the value.

    In this example, a different microrom initialization file was used for each simulation run. This is what produced the different values for *AluBuf* at time 826.

11. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

Debug Unknown (X) Values

If a signal has an unknown value X, the root cause of the X needs to be found. Using the Verdi platform, you can locate the first X with one command. The Verdi platform analyzes the design and report the possible causes of the X or the path causing the X can be visualized over multiple cycles.

Assume that the signal *ZFout* has an unknown value X at time 2777ns, and you need to find out the root cause of the X. A typical way to debug this problem is to display the signal in the waveform, refer to the source code to find the driving signals, display those signals in the waveform to identify those that are X, and so on. Eventually, and if you are persistent, you can locate the first occurrence of an X.

Before you begin this application, follow the instructions in the Before You Begin chapter.

Feedback

This section includes the following topics:

- Locate the Root Cause of the “X” Value on ZFout
- Visualize the Active Paths in the Temporal Flow View

Locate the Root Cause of the “X” Value on ZFout

Enter the following commands to start the tutorial and then use the Behavior Analysis engine to automatically locate the cause of an unknown value with one command.

1. Change to the demo directory:

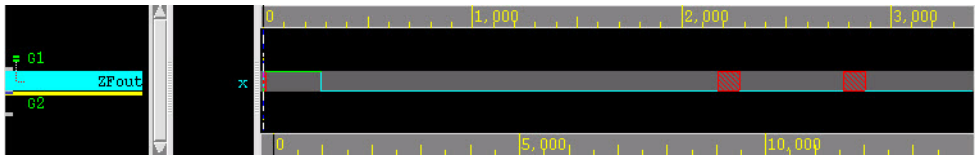
   ```
   % cd <working_dir>/demo/verilog/cpu/src
   ```

2. Start the Verdi platform, and reference the file *run.f* in the current directory and the FSDB file *CPUsystem.fsdb* in the parent directory:

   ```
   % verdi -f run.f -ssf ../CPUsystem.fsdb -workMode hardwareDebug &
   ```

3. In the main window, turn on the **Source -> Active Annotation** toggle command.
4. Display the waveform for *ZFout* from the *ALUB* module in the *nWave* frame.
5. Zoom out in the *nWave* frame until you see “*X*” at time 2777ns. Your task is to locate the cause of this unknown value.

Figure 333 ZFout with Unknown Value



6. Click the *ZFout* signal in *nWave* somewhere close to the transition from the *0* value to the *X* value, and note the following:
   - A vertical cursor appears in the waveform pane.
   - The simulation time of 2777 associated with the cursor's current location is displayed in *nWave* frame toolbar.

   Note:
   By default, the cursor snaps to the closest transition on the selected signal, the transition from *0*t o *X* in this case.

7. Right-click the *ZFout* in the *nWave* frame at the transition from *0* to *X,* and select **Trace X**.

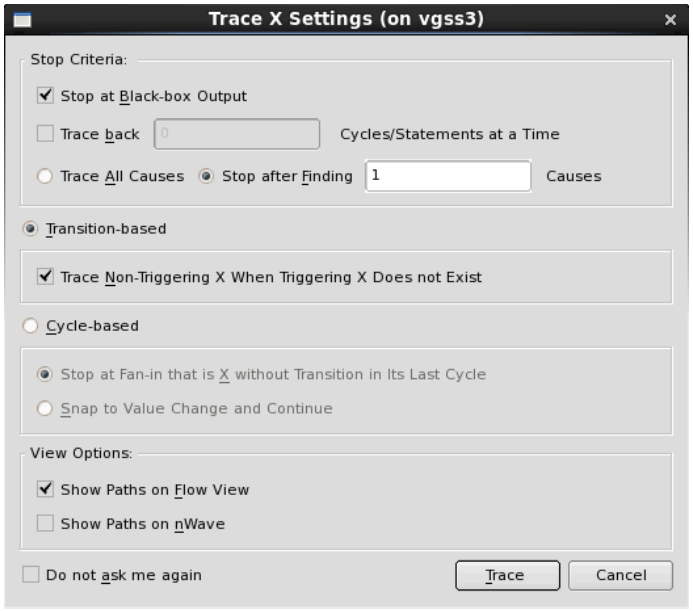
Feedback

A *Question* dialog window displays, indicating that you need to perform Behavior Analysis.

8. Click the **Yes** button on the *Question* dialog window.
9. In the *Behavior Analysis* form, click the **OK** button.

   The *Trace X Settings* form displays, as shown in the following figure:

Figure 334 Trace X Setting Form



The *Trace X Setting* form has several options, including:

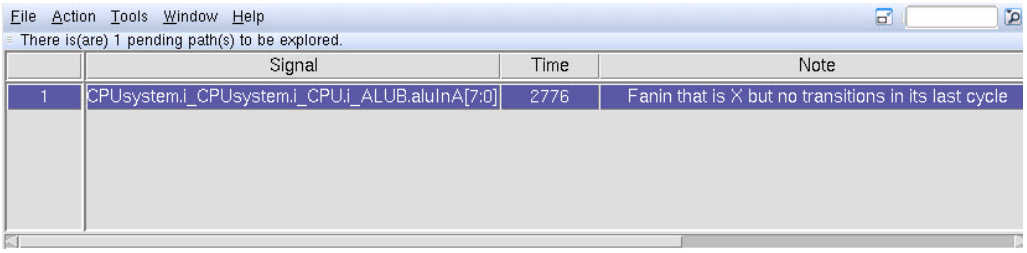
- If you select **Stop at Black Box Output**, tracing stops at these outputs instead of finding the inputs to the black box that are X and continuing the trace from those points.
- The **Stop at Fan-in that is X but No Transition in Its Last Cycle** option instructs the tool to trace back only those fan-in signals that make signal transition.
- The **Snap to Value Change and Continue** option tells the tool to snap to the closest value change point and continue the tracing process.

10. Click the **Trace** button.

    The results displays as a new tab in the same position as the source code frame, as shown in following figure.

Feedback

Figure 335 Trace Triggering X Results Frame



The Verdi platform stops at the signal *aluInA* at time 2776 with the reason that the **Fanin that is X but has no transitions in its last cycle**. The fan-in signal that is X in this case is *DataIn*. This means there is an assignment of X to *DataIn* but this assignment is not the one that causes the value transition in the waveform. This is investigated later.

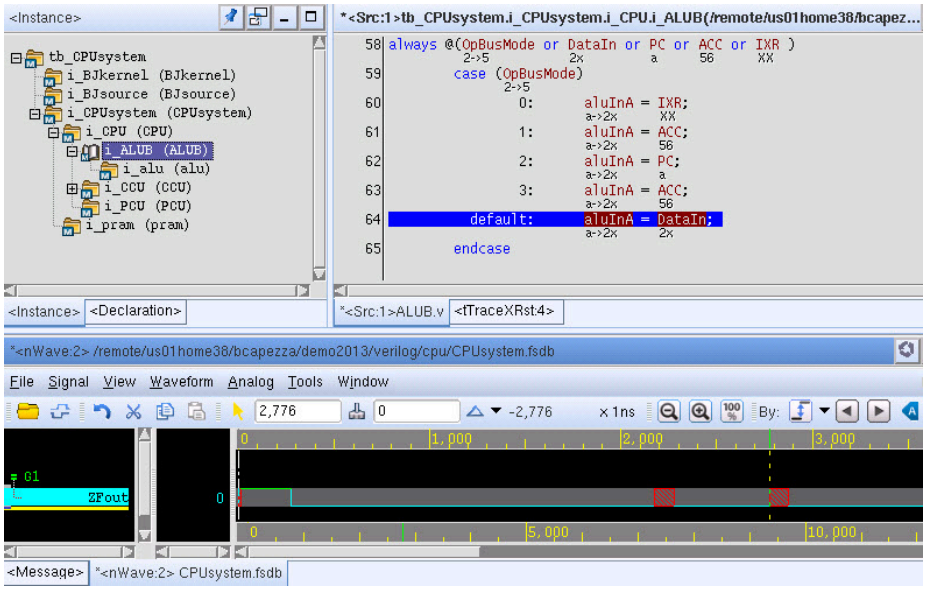
The following table describes the possible causes of an unknown value:

Possible X cause	**Comment**
Fan-in that is X but no transition in its last cycle	Trace-X algorithm stops because all the fan-ins that are X do not change value in the cycle being evaluated. The algorithm stops because the bug may come from the fan-in signals that are not X but have value changes. The cause only appears when user turns on the option “Stop at fan-in that is X but no transition in its last cycle.”
No fan-in that is X	None of the fan-in signals is X, or the X signal has no fan-in.
X from primary input of the work scope	The trace-X algorithm stops at the input signal of the work scope
Load constant ‘X’	Loads a ‘X’ value into a signal, for example, s1 = 1’bx
Memory net (assigned value ‘x’ during initialization)	The memory element has been assigned a ‘X’ value during initialization
Memory net (not initialized)	The memory element has not been initialized
Black box output	Trace-X algorithm stops at the output of a black box. This message appears only when the option “Stop at black box output” is turned on.

Feedback

11. To see the source code for the cause, right-click the *tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.aluInA* signal in the *Trace Active X Results* frame, and select the **Show Source Code on nTrace** command. The RTL statement that drives the signal is highlighted on the source code frame, as shown in the following figure:

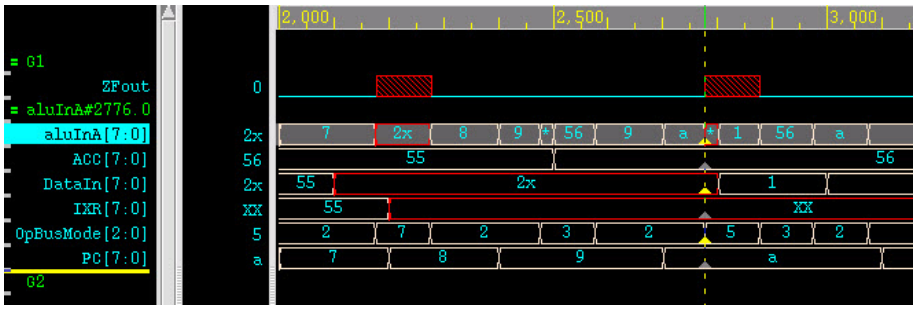
Figure 336 Source Code for Unknown



12. Right-click the signal in the *Trace Active X Results* frame again, and select the option **Add Active Fan-in Signals to nWave**.

    The active fan-in signals of *aluInA* displays on the *nWave* frame, similar to the following figure:

Figure 337 Active Fan-ins for aluInA

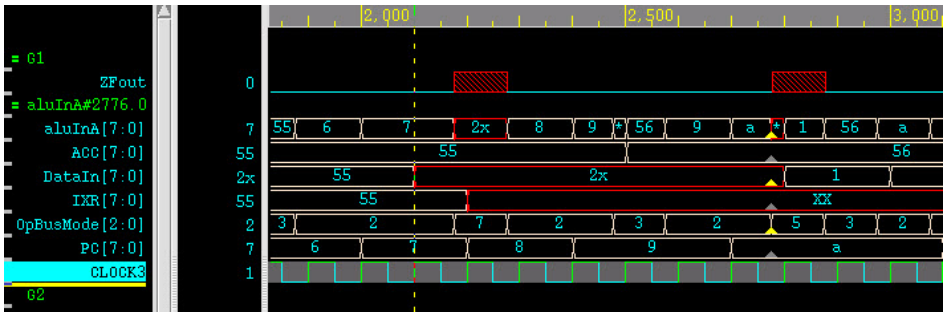


13. To determine if there has been a value change in the last cycle or not, you need to know the clock cycle. In the waveform pane, right-click *DataIn* near the *2x* value, and select **Show Clock (Domain)**.

Feedback

The waveform frame displays similar to the following figure:

Figure 338 Clock for DataIn

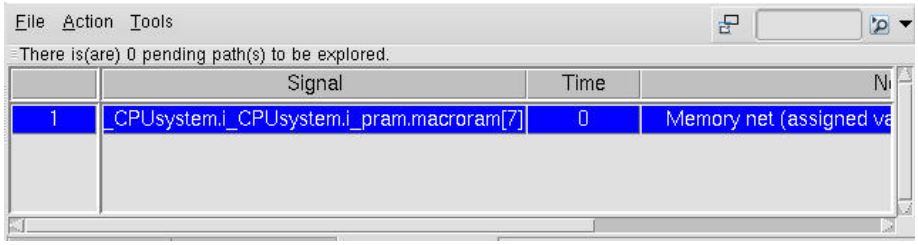


You can see the clock for *DataIn* is *CLOCK3*. The value change for the unknown of *DataIn* comes from several cycles before the *ZFout* signal goes unknown. If this is not the cause of unknown, you may continue the trace back process.

14. In the *Trace Active X Results* frame, select the **Action** -> **Continue to Trace Selected Signal** command.
15. Click **Trace** on the *Trace X Setting* form that displays.

    Trace active X continues on the selected net and stops at a memory output. The cause for this X is that the memory net has been assigned an unknown value during initialization.

Figure 339 Trace Active X Results



16. Close the *Trace Active X Results* frame.

Visualize the Active Paths in the Temporal Flow View

Another way of tracing active X is to visualize the propagation path on the flow view.

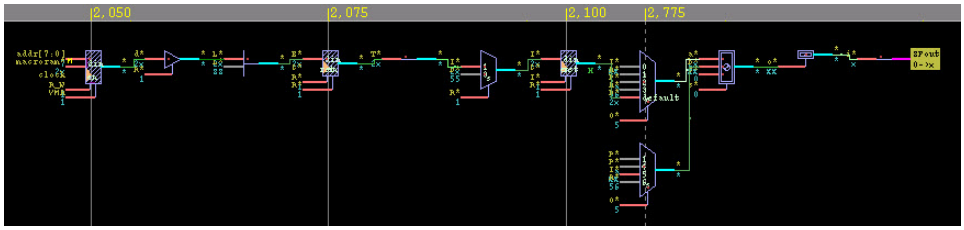
Feedback

1. Click the *ZFout* signal in the waveform pane of the *nWave* frame somewhere close to the transition to the *X* value at time 2777.
2. Right-click the *ZFout* signal on the transition from *0* to *X*, and choose the **Temporal Flow View** -> ***New*** **Temporal Flow View** command. A *Temporal Flow View* frame opens.
3. In the *Temporal Flow View* frame, right-click the *ZFout* signal at time 2777, and select the **Trace X** command. The *Trace Triggering X Setting* form appears.
4. Turn off the **Stop at Fan-in that is X but No Transition in Its Last Cycle** option.
5. Turn on the **Show Paths on Flow View** option.
6. Change the value for **Trace Back N Cycles/Statements at a Time** to 20.
7. Click the **Trace** button.

   An *Information* dialog window opens indicating the memory was assigned during initialization.
8. Click the **OK** button on this dialog window.
9. The *Trace Triggering X Results* frame opens, and the path is traced in the *Temporal Flow View* frame.
10. Click the **Fit Time** icon in the *Temporal Flow View* frame to see the entire path.

    The *Temporal Flow View* frame display updates, as shown in the following figure:

Figure 340 Trace X Results in Temporal Flow View Window

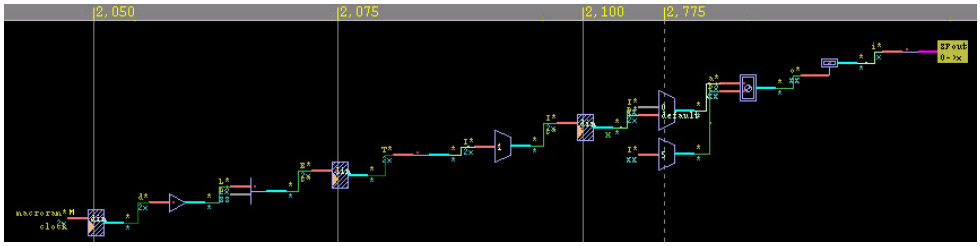


By default all nodes are displayed. The tracing stops at the memory output *macroram[7]*.

11. Choose the **View -> Signal -> Nodes with Value ‘X’ Only** toggle command to remove the known nodes from the display.

Feedback

Figure 341 Unknown Nodes in Temporal Flow View Frame



12. Double-click any node to see its driver in the source code frame.
13. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

Debug Forced Signals

In Verilog, a "force" statement applied to a variable overrides a procedural assignment, a continuous assignment, or an assign procedural continuous assignment to the variable. The value of the variable is 'forced' until a release procedural statement is executed on the same variable. In addition, the **force** and **release** PLI or UCLI commands can be used to impact variables in interactive simulation debug mode. This makes it difficult for you to know the real driver of the variable just by looking at the source code or waveform.

The Force Debug features provide visibility into the force and release actions. When an FSDB file containing dumped forced information is loaded, you are able to view and debug the forced signal through the annotated source code, schematic and waveform. The force/release statement is also be considered as the root cause during tracing. In addition, the forced, deposited, and released events of the specified signals can be extracted to a report file.

This section includes the following topics:

- Enable Force Debug
- Debug in Source Code
- Debug in the Waveform View
- Debug in the Schematic View
- Debug in the Temporal Flow View
- Practice

Feedback

Enable Force Debug

Perform the following steps to enable the Force Debug features.

1. Specify these environment variables to setup your environment and simplify scripting:

   ```
   > setenv VERDI_HOME <VERDI_INST_DIR>
   ```

2. To enable debugging capabilities, the `-debug_access` and `-kdb` compile options are required during compile time for the VCS compiler. For example:

   ```
   vcs -full64 -sverilog -debug_access+all -kdb ./design.sv
   ```

   Note:

   - The tool uses 32-bit platform automatically when 64-bit is not available due to any reason or crash. It is recommended to use only 64-bit platform and not use 32-bit platform.
   - The current available simulators are VCS version I-2014.03 and later.
   - The `-debug_access` compile options are required to enable force features with VCS for force/release statements coming from an external source (for example, UCLI commands), not in the source code.

     The `-debug_access` option is an LCA option in VCS, refer to the VCS documentation for details.

3. There are two methods to enable dumping force/release information into the FSDB file. You can set the environment variable:

   ```
   > setenv FSDB_FORCE 1
   ```

   Alternatively, you can use the **+fsdb+force** runtime option during simulation. For example:

   ```
   > ./simv +fsdb+force
   ```

   The dumped force/release information includes the following:

   - Signal name
   - Time point to be forced
   - Force/Release type: deposit or freeze type

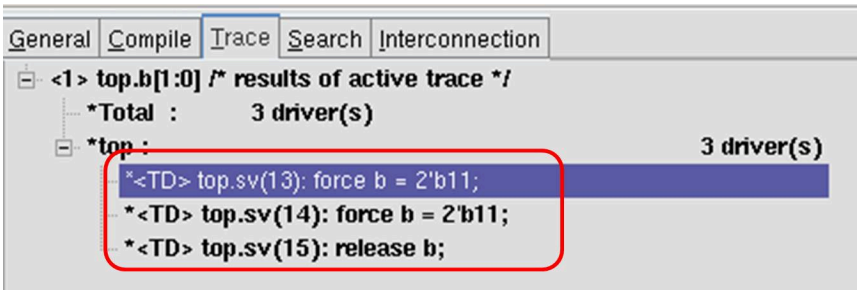
Feedback

Debug in Source Code

Trace Signal

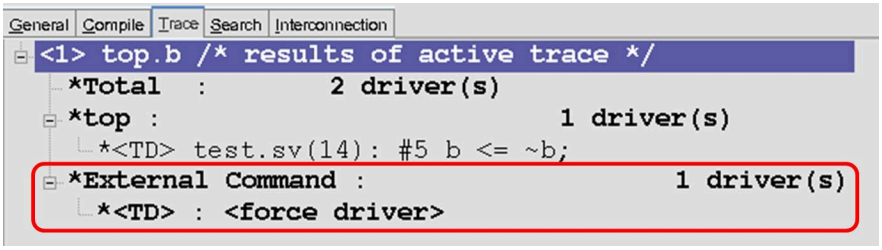
In the source code frame, the results of trace driver (invoked by the **Driver** command in the right-click command menu) for a selected signal include force/release statements. After the FSDB file containing dumped force information is loaded into the Verdi platform, you can use the **Active Trace** command to only show the tracing results with those forced/released events that impact the range of the variable at a specified time. As shown in the following figure, a forced event is treated as a real driver even when the force event does not cause a value change.

Figure 342 Force/Release as the Active Trace Result



When the force or release event is caused by UCLI (for example, use the **force** command in UCLI mode) or PLI commands, the external command is listed at the end of the original result as another category of the active tracing result.

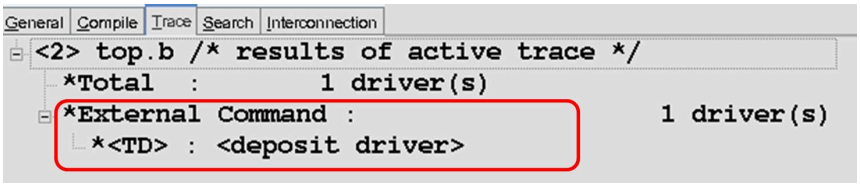
Figure 343 External Force Command as the Active Trace Result



If the signal is frozen forced, *<force driver>* is shown as the result for the selected signal (refer to the figure above). If the signal is deposit forced, *<deposit driver>* is shown (refer to the following figure). If all bits of the selected signal are released, *<release driver>* is shown. If partial bits are still forced and partial bits are released for the selected signal, *<force driver>* is shown.

Feedback

Figure 344 External Deposit Command as the Active Trace Result



Active Annotation

After the FSDB file containing dumped force information is loaded into the Verdi platform, the annotation value is displayed with a force mark. In the source code and *Signal List* frames, use the **Source** -> **Active Annotation** command to see the current forced value annotated below the signal with the caret "^" character. As shown in following figures, the caret "^" character is annotated to highlight the force/release event.

Figure 345 Active Annotation for Forced Value

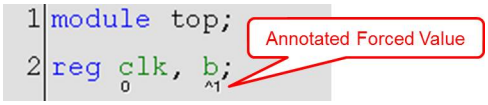
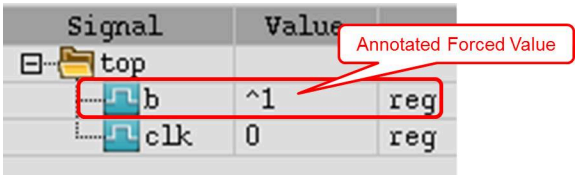
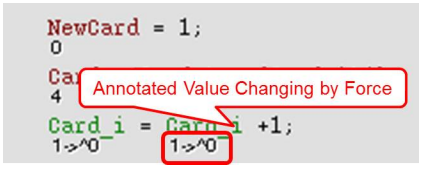


Figure 346 Forced Value in Signal List



If a value change is caused by a force/release command, the transition is annotated with values. For example, if the value change from '1' to '0' is caused by a force command, the annotate value is '*1->^0*' to indicate the value change is caused by a force command, as shown in the following figure.

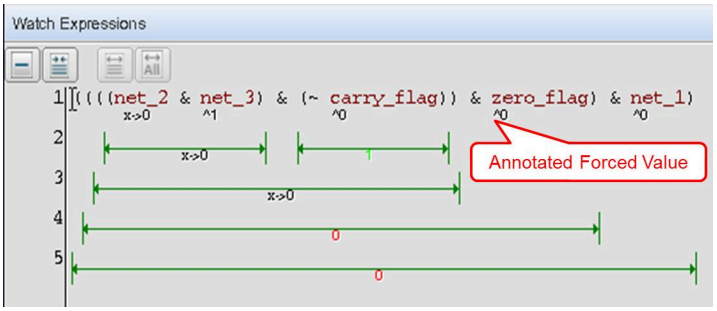
Figure 347 Value Change with Forced Value



In the *Watch Expressions* frame (invoked by the **Tools** -> **Browse/Watch/List** -> **Watch Expressions** command), the forced values are also annotated with the caret "^" character for the variables, as shown in the following figure.

Feedback

Figure 348 Forced Value in Watch Expression



Note:

The expression value in the *Watch Expressions* frame does not show the force information.

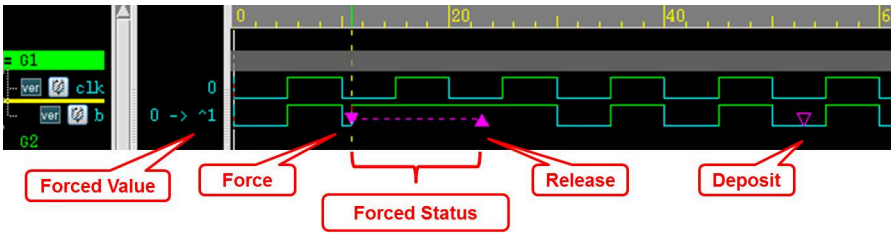
Debug in the Waveform View

When an FSDB file containing dumped forced information is loaded into the Verdi platform, the force/release information is also automatically shown in the *nWave* frame with special icons to highlight the force/release events.

Icons in the Waveform Pane

As shown in the following figure, the force, release, and deposit values, and the forced status are shown with corresponding icons in the waveform pane. The value is annotated with the caret "^" character in the value pane if the current value is forced.

Figure 349 Forced Value in Watch Expression



Icons	Attributions
	The value is forced.
	The value is released.
	The value is deposited.
	The status of the value is forced.

Feedback

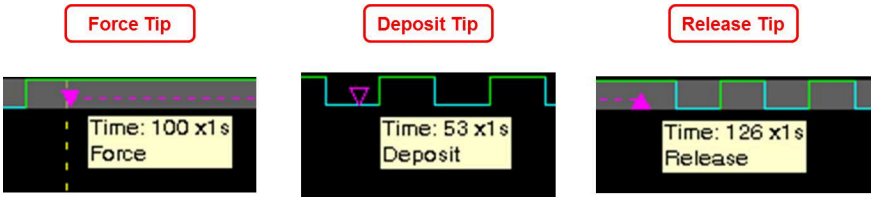
The following table shows the attributes of the forced icons in the nWave frame.

If there is a value change with an icon in the *nWave* frame, you can double-click the icon to trace the force, deposit, or release statement in the main window.

Show Tips

In the waveform pane, you can turn on the **Enable Tip** option in the **Waveform -> View Options -> Waveform Pane -> General** page of the *Preferences* form (invoked with the **Tools -> Preferences** command) to show a tip window when the cursor is placed over force/release event. The following figure shows the examples for the tips.

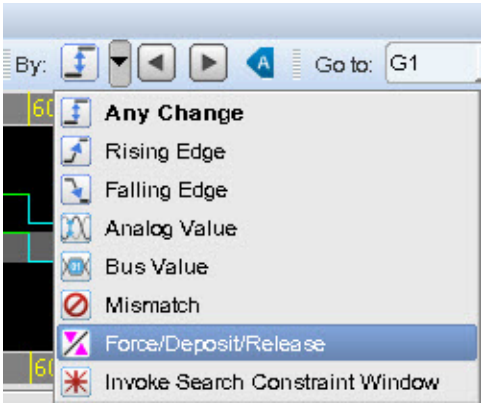
Figure 350 Tip in Waveform Pane



Search by Force/Deposit/Release

As shown in the following figures, you can select the **Force/Deposit/Release** option in the **Search By** list on the toolbar and click the or buttons to search backward or forward for values of the selected signal that are the result of a force, deposit, or release event.

Figure 351 Force/Deposit/Release Option in the Search By List



Feedback

Figure 352 Tip in Waveform Pane

Debug in the Schematic View

After an FSDB file containing dumped force information is loaded into the Verdi platform, you can also see the force values in the *nSchema* frame. In the *nSchema* frame, use the **Schematic -> Active Annotation** command to annotate the current forced value below the signal with the caret "^" character, as shown in the following figure.

Figure 353 Forced Value in nSchema

When the **Trace -> Active Fan-in** command is invoked for the selected signal, and if a force, deposit, or release event causes a value change of the signal, the event is referred as the fan-in tracing result. Active fan-in tracing stops at the cell if its output signal is in one of the following time duration or time points:

- Forced state
- The time point of the forced state changed to release
- The time point of a deposit event

Debug in the Temporal Flow View

In the *Temporal Flow View* frame, the force, deposit, or release events is considered as the cause when tracing the selected signal. As shown in Figure 13, you can see the forced value is shown with the caret "^" character and the *(forced)*, *(deposited)* or *(released)* strings suffix. If a value is forced with unknown value, the "^x" is added as the root cause for the forced value, as shown in the following figure.

Feedback

Figure 354 Forced Value in Temporal Flow View

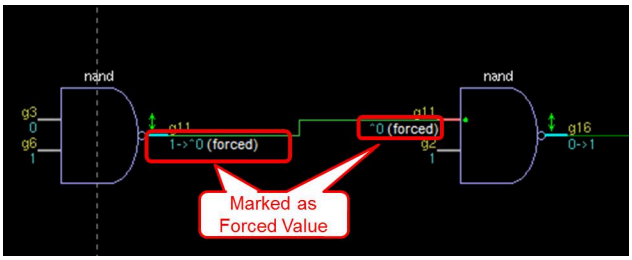
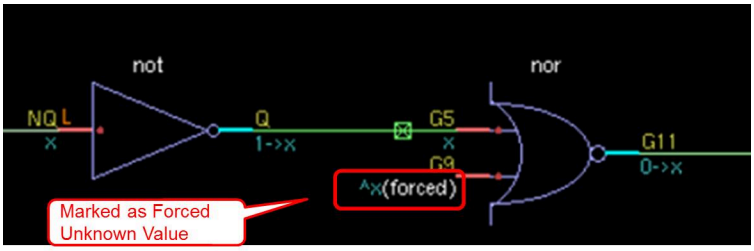


Figure 355 Forced with Unknown Value in Temporal Flow View

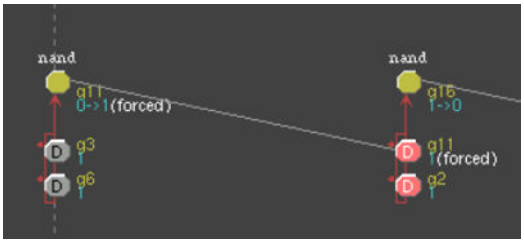


Note:

If the force or release command is in the design, the command and the selected tracing signal should be in the same scope.

In the *Compact Temporal Flow View frame*, the forced value is also shown with the *(forced)*, *(deposited)* or *(released)* strings suffix.

Figure 356 Forced Value in Compact Temporal Flow View



Practice

Complete the following steps to dump the force/release information into the FSDB file and then debug the design with forced values in the Verdi platform.

Feedback

1. Set following variable for facilitating the script:

```
> setenv VERDI_HOME <VERDI_INST_DIR>
```

2. Change to your local directory and copy the demo case to your local directory:

```
> cd
> cp $VERDI_HOME/demo/verilog/cpu/src .
```

3. To illustrate the Verdi Force Debug features with this demo case, modify the design to use a force statement to postpone the appearance of the 'ACE' signal. Add the following code in the *BJsource.v* file:

```
//to force Card_i at 600ns and release it after 100ns
initial
begin
        $fsdbDumpvars("+all");
        #600 force Card_i = 32'h0;
        #100 release Card_i;
end
```

4. Set the environment variable:

```
> setenv FSDB_FORCE 1
```

5. Compile the design, enable force debug dumping features, and run the simulation:

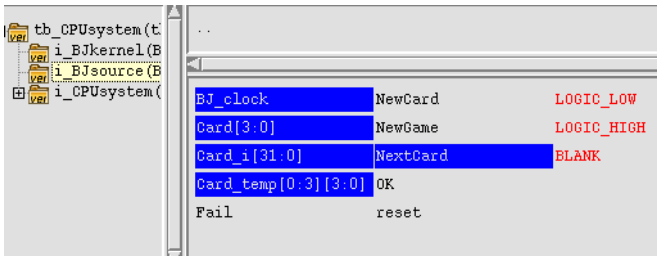
```
> vcs -full64 -sverilog -debug_access+all -kdb ./design.sv
> ./simv
```

6. Import the design and load the generated FSDB file into the Verdi platform:

```
> verdi -ssf novas.fsdb &
```

7. In the nWave frame, click the icon to add signals. Double-click the *ACE* signal of the t*b_CPUsystem* scope to add it to *nWave*. Refer to the following figure, select the *i_BJsource* scope and the following highlighted signals, and click the **OK** button in the *Get Signals* form.

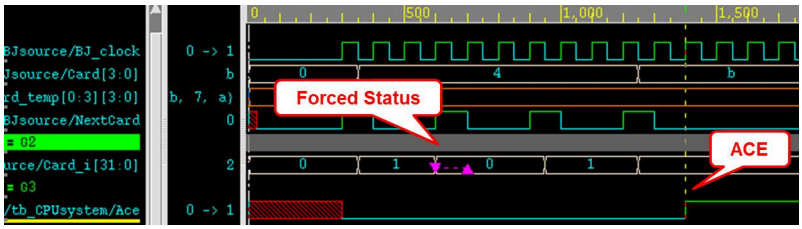
Figure 357 Select Signals in the Get Signals Form



Feedback

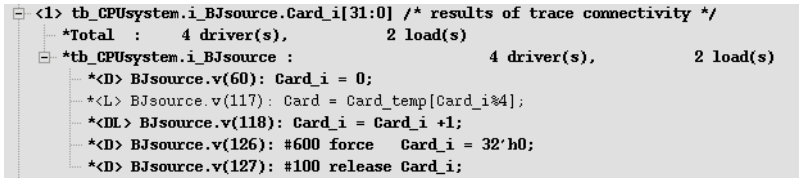
As shown in the following figure, you can see the waveform of the raising edge of the *ACE* signal and the related signals. Also, the forced event and duration of the *Card_i* signal has been marked.

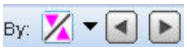
Figure 358 ACE and related signals waveform



8. Drag and drop the *Card_i* signal from the *nWave* frame to the source code frame. The results of trace connectivity are shown in the **OnesTrace** tab of the Message frame. The force/release statements are also regarded as results of drivers.

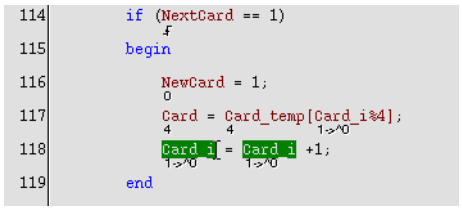
Figure 359 Tracing Result



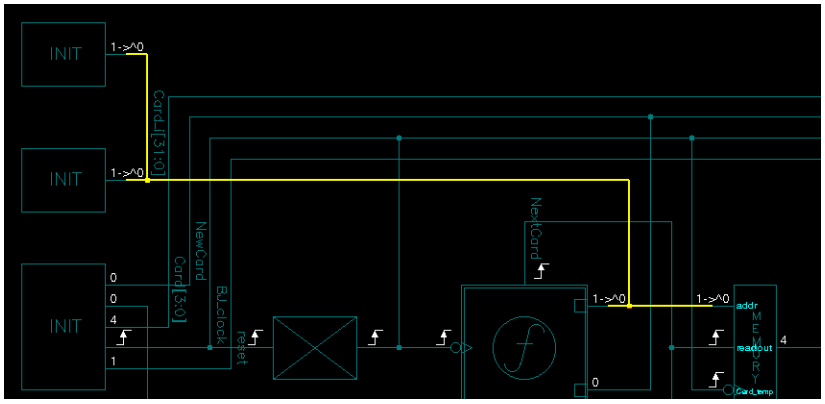
9. In the *nWave* frame, select the *Card_i* signal and use the 

icons to find the first force event. The cursor is located at 600ns.

10. In the main *nTrace* window, invoke the **Source -> Active Annotation** command. The value change with the current force values of the *Card_i* signal are annotated in the source code and *Signal List* frames.

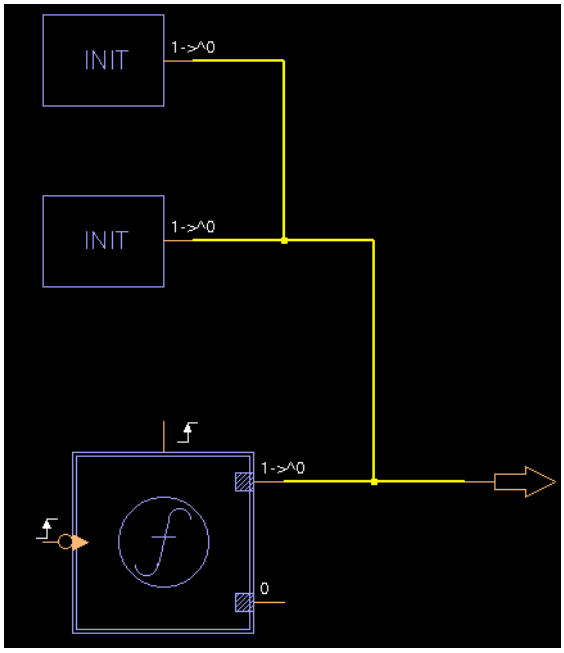


11. In the *nSchema* frame, invoke the **Schematic -> Active Annotation** command. The current force values are annotated.



12. Select the *Card_i* signal and invoke the **Trace -> Active Fan-in** command. As shown in the following figure, the forced event causes a value change of the *Card_i* signal and the active fan-in tracing stops at the cell whose output signal is in the force state.

Figure 360 Fan-in Cone Result of Card_i



Debug With SystemVerilog

Before you begin this application, follow the instructions in the Before You Begin chapter.

The Verdi platform provides a set of features that allow you to debug SystemVerilog designs.

The design is based on the standard Verdi CPU case; however, it has been rewritten in SystemVerilog. There are also some assertions coded in SVA.

This section includes the following topics:

- Import the Design
- Visualize SystemVerilog Source Code
- View SystemVerilog Simulation Results

Import the Design

There are two methods for importing the design: load the files directly or create a compiled library.

1. Change your context to the *systemverilog* sub-directory, which is where all of the demo source code files are located:

   ```
   % cd <working_dir>/demo/systemverilog
   ```

2. Modify the SETUP file to point to the correct Verdi and simulator installation paths in your environment and source the file.

   ```
   % source ./SETUP
   ```

   Refer to the *Language Support* chapter of the *Verdi and Siloti Command Reference Manual* for complete details on compiling and importing different languages.

Load Files Directly

Since the code is all SystemVerilog, you can compile and load the source files directly without pre-compiling.

1. Start the Verdi platform by referencing the design files. If you do not use a common file extension, you need specify the -sv command line option.

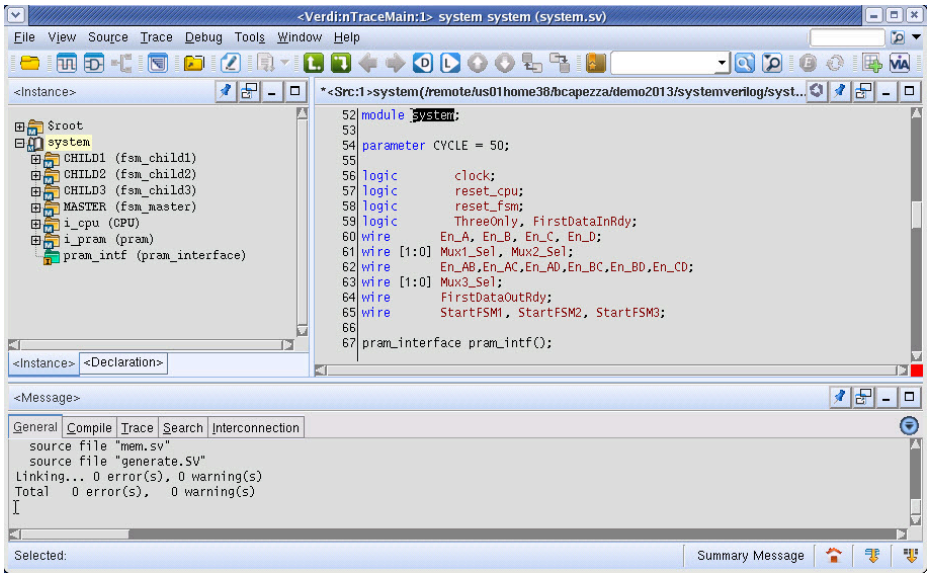
   ```
   % verdi -f run.f -sv -workMode hardwareDebug &
   ```

   Alternatively, if all files have the same file extension (for example, .sv, .SV) you can specify the *+systemverilogext+* option instead. Refer to the RUN script in the demo directory for an example.

Feedback

The Verdi platform opens to display the SystemVerilog design source code.

Figure 361 nTrace with SV Code Loaded



Use Compiled Library - Optional

You can compile the SystemVerilog design into a work.lib++ compiled library and load from there. For designs that are mixed-language, it is recommended to compile the design first and then load. This includes designs that are mixed Verilog/SystemVerilog as you may have a Verilog design (that is not SV-compliant) and add code (including SVA) that is in SystemVerilog.

1. Compile the library. By default work.lib++ is created. If all files have the same file extension (for example, .sv, .SV) you can specify the *+systemverilogext+* option.

   ```
   % vericom -f run.f +systemverilogext+.sv+.SV+
   ```

   Alternatively, if you do not use a common file extension, you need specify the -sv command line option. Refer to the COMPILE script in the demo directory for an example.

2. Load the compiled library.

   ```
   % verdi -lib work &
   ```

Visualize SystemVerilog Source Code

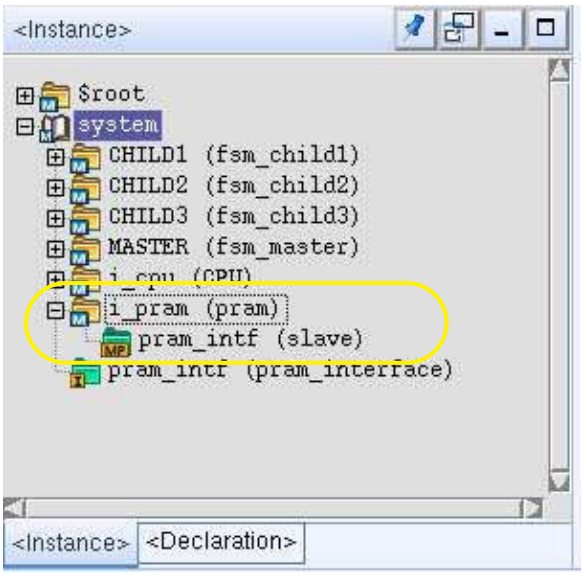
The Verdi platform provides advanced visualization capabilities that allow you to quickly understand SystemVerilog code.

Feedback

Design Browser Frame

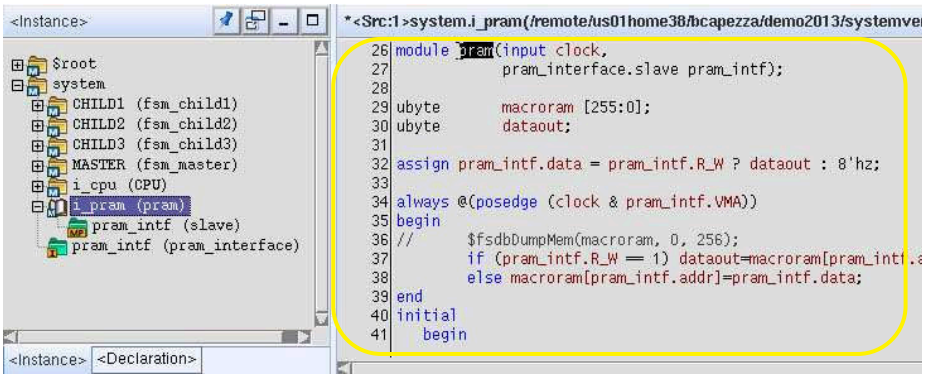
1. In the **Instance** tab of the design browser frame, click the plus symbol to the left *i_pram* block instance name to expand its subblocks. You can see an interface, *pram_intf*.

Figure 362 SV in nTrace



2. Double-click *i_pram* to display its associated source code.

Figure 363 SVA Properties and Sequences in nTrace

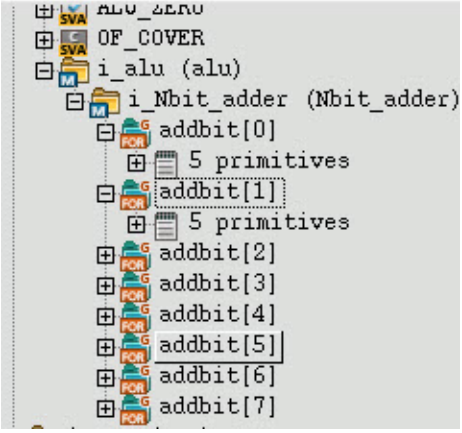


3. In the design browser frame, click the plus symbol to the left *i_cpu*, *i_ALUB*, *i_alu* and *i_Nbit_adder* block instance names to expand their subblocks.

4. Click the plus symbol to the left *addbit[0]* and *addbit[1]* under *i_Nbit_adder* to expand the generate instances. They each contain 5 primitives.

Feedback

Figure 364 Generate Instances



Each generate instance is a scope in the design browser frame with its elements within.

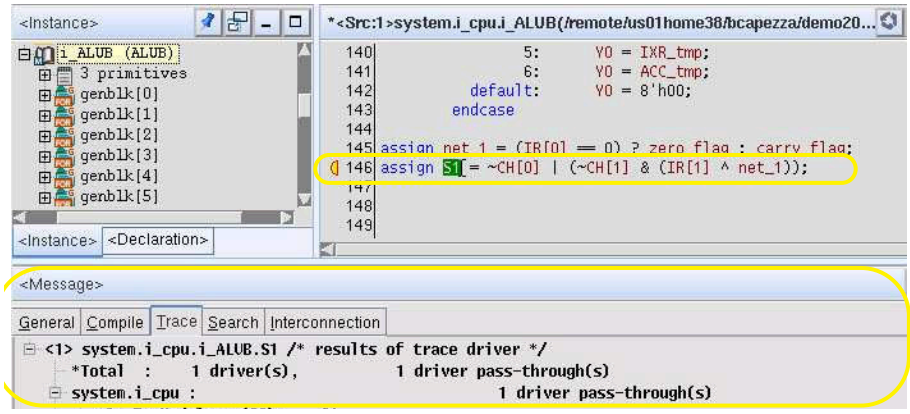
5. Double-click *addbit[0]* and then *addbit[1]* to display the associated source code.

   You can go to the same place in the source code but the corresponding simulation data is annotated based on your current active scope.

Source Code

1. In the design browser frame, double-click *i_ALUB* to display its source code.
2. In the source code frame, double-click *S1* to trace drivers.

Figure 365 Trace Driver Results for S1



The source code changes to the driver location for *S1* and the message frame displays the results summary.

3. In the design browser frame, double-click *pram_intf(pram_interface)* to display the source code.
4. In the source code frame, left-click to select *data*.
5. Right-click to open the right mouse button menu and select the **Connectivity** command to trace connectivity for *data*.

   The **One Trace** tab of the message frame indicates there are 2 drivers, 2 loads and several pass-throughs. You can double-click any line in the message frame to go to the equivalent code.
6. In the *nTrace* main window, click the **Show Previous/Show Next** icons to step through driver/load results in the current scope (*i_pram*).
7. Click the **Show Previous in Hierarchy** icon to locate results in a different scope (*i_PCU*).

   After tracing drivers or loads or both, you can easily traverse the hierarchy to locate the results.

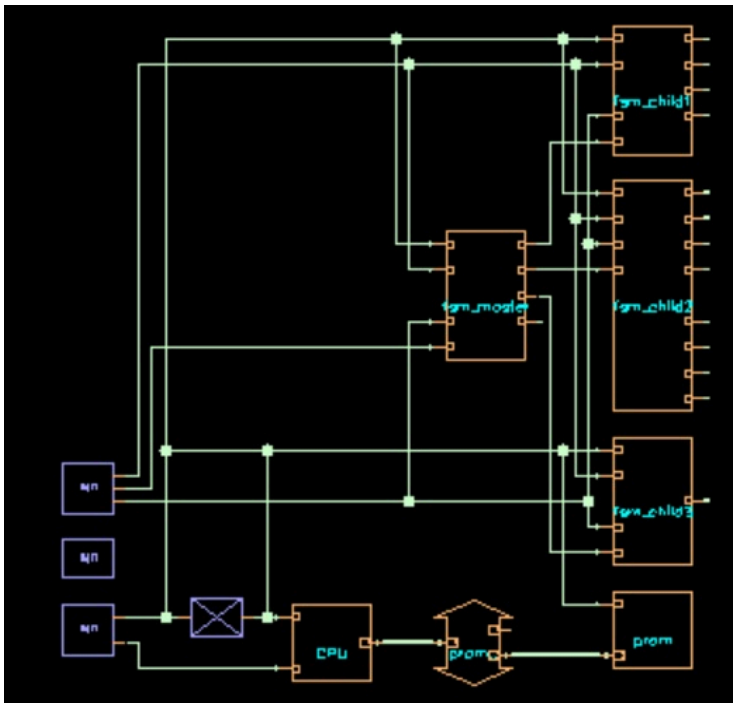
Schematic

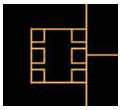
1. In the design browser frame, double-click *system* to display the associated source code.

   On the toolbar, click the **New Schematic** icon to open an *nSchema* frame of the top-level design.

Feedback

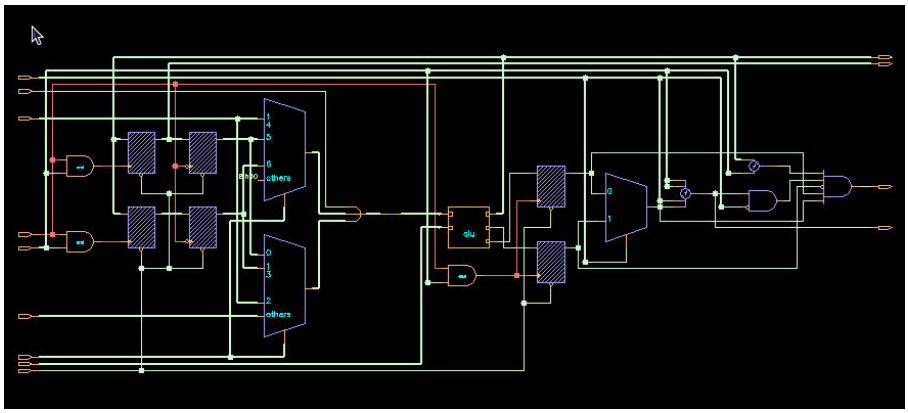
Figure 366 Schematic for system



2. Drag with left mouse button to zoom in on the lower-right corner. Note the interface port symbols  on *pram* and *pram_interface*.
3. Click the **Zoom All** icon.
4. Double-click the *CPU* block in the lower-left to descend to the next level.
5. Double-click the ALUB block on the left to descend to the next level.

Feedback

Figure 367 Schematic for ALUB



6. In the *nSchema* frame, choose the **File -> Close Window** command.

View SystemVerilog Simulation Results

You have loaded the source code and used the Verdi platform to understand SystemVerilog designs. Now you can load simulation results into the system to use the full power of the Verdi platform. A simulation results file (sv.fsdb) already exists; however, you can dump the simulation results again (including assertion results) from your simulator (VCS was used in this example). See the SETUP and SIMULATE scripts in the demo directory for details on appropriate simulator commands.

In this example, system.sv calls *$fsdbDumpfile* to specify the output FSDB file (sv.fsdb), *$fsdbDumpSVA* to dump SVA data, and *$fsdbDumpvars* to dump standard RTL design data.

After simulation, the FSDB data can be loaded into the Verdi platform so you can view waveforms, annotate on source code, and use the automatic tracing capabilities.

Note:

To dump the successful assertion, you must perform any one of the following:

Set the environment variable as follows:

```
% setenv FSDB_SVA_SUCCESS 1
```

Or

Add the `+fsdb+sva_success` option at runtime as follows:

```
% simv +fsdb+sva_success
```

Feedback

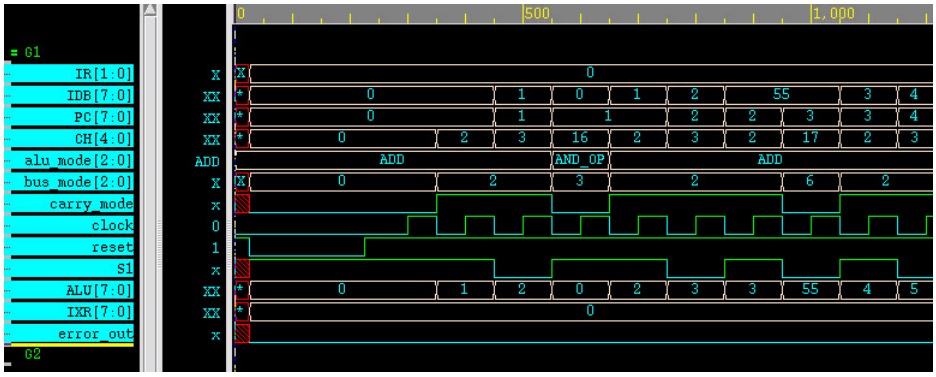
Waveform

1. In the *nTrace* main window, click the **New Waveform** icon to open the *nWave* frame. Alternatively, you can choose the **Tools -> New Waveform** command.
2. In the *nWave* frame, choose the **File -> Open** command to open the FSDB file or click the **Open File** icon on the toolbar to open the *Open Dump File* form.
3. In the *Open Dump File* form, left-click to select *sv.fsdb* and click **Add** and then **OK** to load the file.

   Note:
   You can also load the FSDB file on the command line when you first bring up the Verdi platform.

   For example: `% verdi -sv -f run.f -ssf sv.fsdb ...`
4. In the design browser frame, select *i_ALUB* and use the middle mouse button to drag and drop the scope to the *nWave* frame.

Figure 368 SV in nWave



5. Zoom in around time 800-1500.

   You can search for any changes in the waveform.
6. In the *nWave* frame, left click to select *ALU[7:0]*.
7. Choose the **Waveform -> Set Search Value** command to open the *Search Value* form.
8. In the *Search Value* form, enter 55 in the **Signal Value** field and click the **OK** button.

   The search **By:** field on the *nWave* toolbar changes to **Search by Bus Value**.
9. On the *nWave* toolbar, click the **Search Forward** icon to locate the value of 55 on ALU at time 950.
10. Double-click the 3->55 transition on *ALU[7:0]* to locate the active driver in the source code frame.

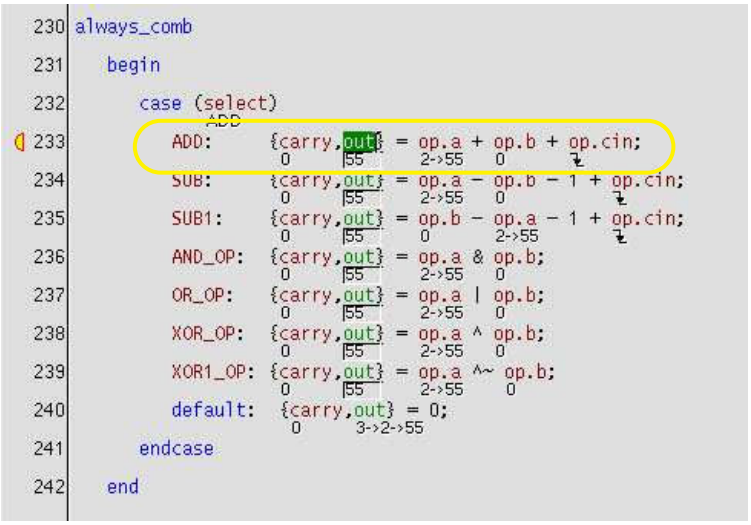
Feedback

Source Code

Another very convenient method to visualize simulation data is through active annotation which allows you to see simulation results under the corresponding variable in the source code frame.

1. In the main window, choose the **Source -> Active Annotation** command (or press the bindkey **X** on the keyboard) to enable active annotation.

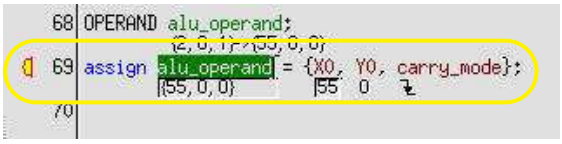
Figure 369 Active Annotation in the Source Code Frame



You can see the simulation values under the signal names. The time is synchronized with the cursor time in the *nWave* frame. If you change the cursor in *nWave*, the values annotated changes accordingly.

2. Left-click to select op.a on line 233.
3. Right-click to open the right mouse button menu and select **Active Trace** to locate the active driver for op.a.

Figure 370 Active Trace Results for op.a



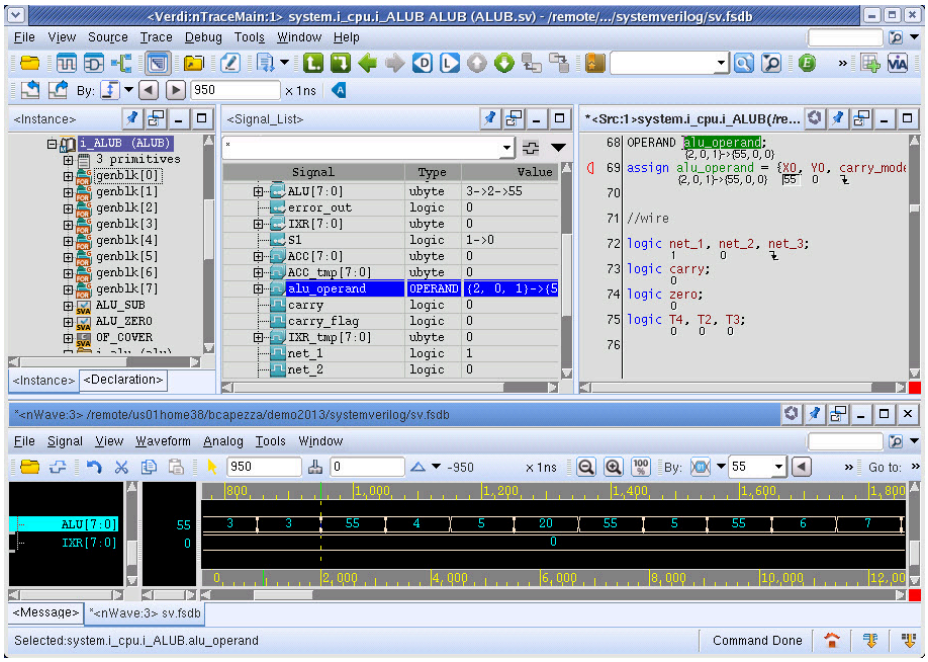
The driver is a complex signal structure in the *ALUB*.

4. In the *nTrace* main window, choose the **View -> Signal List** command to open the *Signal List* frame.

Feedback

5. In the *Signal List* frame, select *alu_operand*. The frame issimilar to the following.

Figure 371 Signal List Frame



The *Signal List* frame makes it easier to view and understand complex signals. You can drag and drop between this frame and source code or *nWave* as needed.

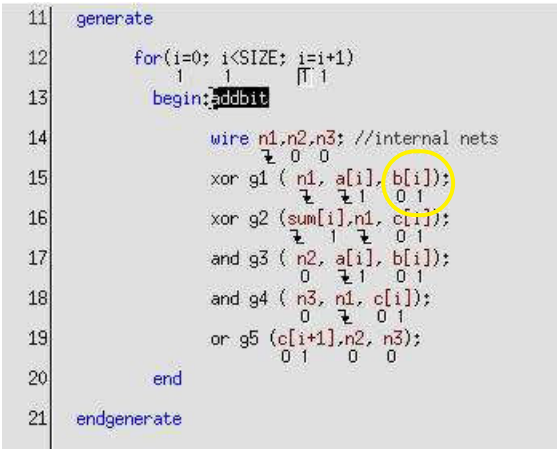
Generate Constructs

For generate constructs, genvars are not dumped to FSDB during simulation. However, the Verdi platform can elaborate the values of genvars and also build the correct hierarchy for the generated instances in the design browser frame. These elaborated values can also be annotated.

Feedback

1. In the design browser frame, double-click *addbit[1]* to change the scope. You can see the annotation under the design signals, and you can also see the value for *i* which is a genvar.

Figure 372 Parameter Annotation



2. Double-click *addbit[3]* under the ALUB hierarchy to change the scope again and you can see 3 annotated for *i*.

Debug With SystemVerilog Assertions

Before you begin this application, follow the instructions in the Before You Begin chapter.

The Verdi platform provides a set of features that allow you to debug assertions.

The design is based on the standard Verdi CPU case; however, it has been rewritten in SystemVerilog. There are also some assertions coded in SVA. This application assumes and uses simulation-based assertion checking. In particular, VCS (2006.06) was used to generate the SV(A) data.

This section includes the following topics:

- Import the Design
- Visualize SVA Source Code
- View SVA Simulation Results
- Analyze SVA Assertions
- Evaluate SVA Assertions

Feedback

Import the Design

There are two methods for importing the design: load the files directly or create a compiled library.

1. Change your context to the *systemverilog* subdirectory, which is where all of the demo source code files are located:

   ```
   % cd <working_dir>/demo/systemverilog
   ```

2. Modify the SETUP file to point to the correct Verdi and simulator installation paths in your environment and source the file.

   ```
   % source ./SETUP
   ```

   Refer to the *Language Support* chapter of the *Verdi and Siloti Command Reference Manual* for complete details on compiling and importing different languages.

Load Files Directly

Since the code is all SystemVerilog, you can compile and load the source files directly without pre-compiling.

1. Start the Verdi platform by referencing the design files. If you do not use a common file extension, you need to specify the `-sv` command line option.

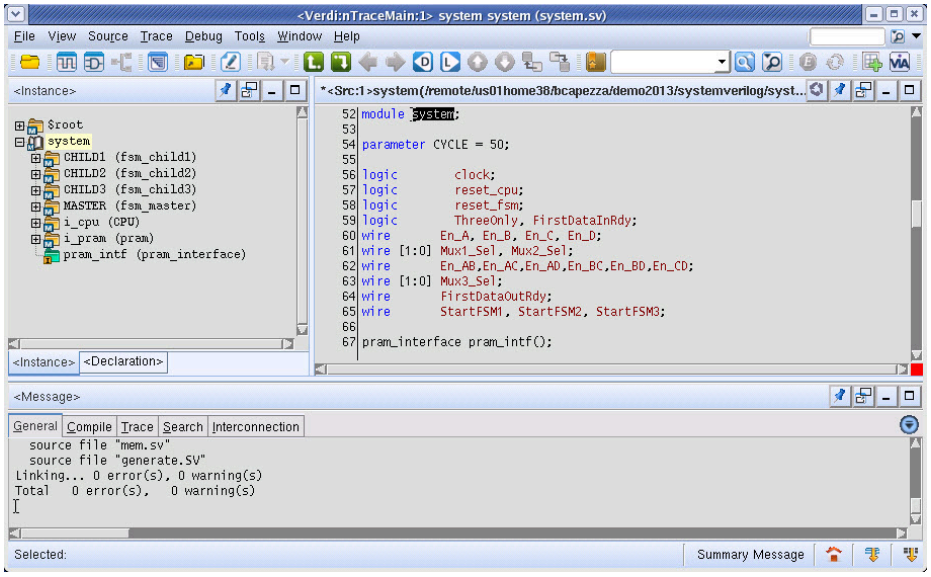
   ```
   % verdi -f run.f -sv -workMode hardwareDebug &
   ```

   Alternatively, if all files have the same file extension (for example, .sv, .SV) you can specify the *+systemverilogext+* option instead. Refer to the RUN script in the demo directory for an example.

Feedback

The Verdi platform opens to display the SystemVerilog source code. The *nTrace* main window serves as the main view from which the other views can be started.

Figure 373 nTrace with SV and SVA Code Loaded



Use Compiled Library - Optional

You can compile the SystemVerilog design into a work.lib++ compiled library and load from there. For designs that are mixed-language, it is recommended to compile the design first and then load. This includes designs that are mixed Verilog/SystemVerilog as you may have a Verilog design (that is not SV-compliant) and add code (including SVA) that is in SystemVerilog.

1. Compile the library. By default work.lib++ is created. If all files have the same file extension (for example, .sv, .SV) you can specify the *+systemverilogext+* option.

   ```
   % vericom -f run.f +systemverilogext+.sv+.SV+
   ```

   Alternatively, if you do not use a common file extension, you need to specify the `-sv` command line option. Refer to the COMPILE script in the demo directory for an example.

2. Load the compiled library.

   ```
   % verdi -lib work -workMode hardwareDebug &
   ```

Feedback

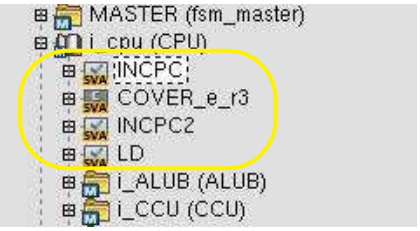
Visualize SVA Source Code

The Verdi platform provides advanced visualization capabilities that allow you to quickly understand SystemVerilog Assertions. These capabilities are even more critical to design teams when new methodologies such as assertions are introduced into the flow.

Design Browser and Source Code

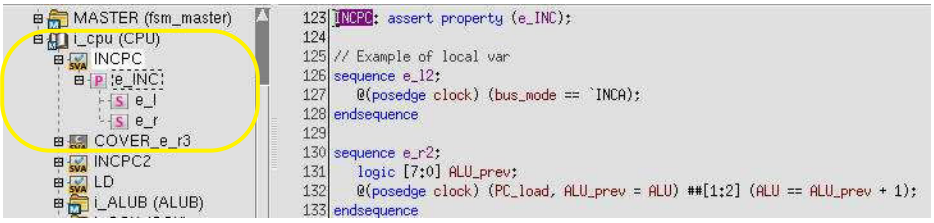
1. In the design browser frame, click the plus symbol to the left *i_cpu* block instance name to expand its subblocks. You can see four assertions: 3 asserts (INCPC, INCPC2, LD) and a cover (COVER_e_r3).

Figure 374 SVA in nTrace



2. Click the plus symbol to the left of the *i_ALUB* block instance name to display several more assertions, including one cover directive.
3. Double-click *INCPC* to display its underlying property (*e_INC*) and associated source code. You can expand any assertion to see the underlying properties the assertion is built upon.
4. Click the plus symbol to the left *e_INC* to display its underlying sequences (*e_l* and *e_r*). You can expand any property to see the sequences it is built upon.

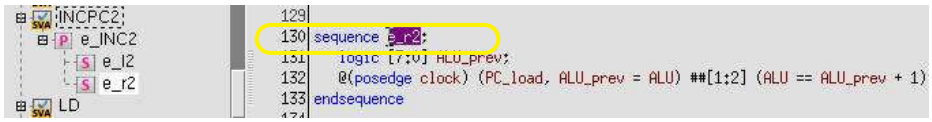
Figure 375 SVA Properties and Sequences in nTrace



5. Display the source code for *INCPC2* by double-clicking on *INCPC2* in the design browser frame.
6. In the source code frame, double-click the *e_INC2* property to trace to its description.
7. Left click to select *e_r2*, right-click to view the right mouse button context menu and select **Show Definition** to trace back to its description (it is a sequence).

Feedback

Note:
Both double-click and the **Show Definition** command displays the description for properties or sequences.



Note that *e_r2* has a local variable, *ALU_prev*. SVA allows local variables which in turn permits users to write powerful assertions. However, local variable and assertions containing them are difficult to debug since assertions by their nature can each have multiple attempts with several sequence threads within. Synopsys provides for the capture and advanced visualization of local variables.

8. In the design browser frame, click the plus symbol to the left *genblk[0]* and *genblk[1]* under *i_ALUB* to expand the generate instances. They each contain a cover directive, *cv*.

Figure 376 Generate Instances

Each generate instance is a scope in the design browser with its elements (assertion / property / sequence) within.

9. Double-click *genblk[0]* and then *genblk[1]* to display the associated source code.

   You can go to the same place in the source code but the corresponding simulation data is annotated based on your current active scope.

View SVA Simulation Results

You have loaded the source code and used the Verdi platform to understand designs containing assertions. Now you can load simulation results into the system to use the full power of the Verdi platform. A simulation results file (sv.fsdb) already exists; however, you can dump the simulation results again (including assertion results) from your simulator (VCS was used in this example). See the SETUP and SIM_SVA scripts in the demo directory for details on appropriate simulator commands.

Feedback

In this example, system.sv calls *$fsdbDumpfile* to specify the output FSDB file (sv.fsdb), *$fsdbDumpSVA* to dump SVA data, and *$fsdbDumpvars* to dump standard RTL design data.

After simulation, the FSDB data can be loaded into the Verdi platform so you can view waveforms, annotate on source code, and use the automatic tracing capabilities.

1. In the *nTrace* main window, select the **Window -> Assertion Debug Mode** command to enable more frames to assist with assertion debug.
2. In the *nWave* frame, choose the **File -> Open** command to open the FSDB file or click the **Open File** icon on the toolbar to open the *Open Dump File* form.
3. In the *Open Dump File* form, left-click to select *sv.fsdb* and click **Add** and then **OK** to load the file.

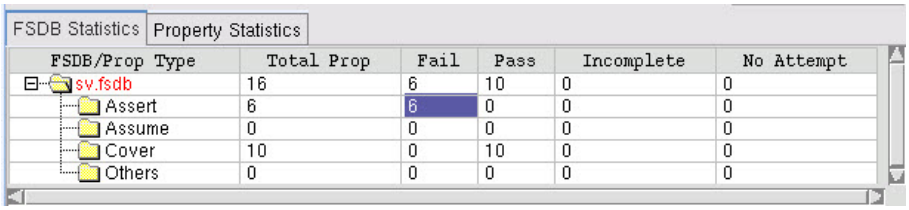
   Note:
   You can also load the FSDB file and enable the assertion debug work mode on the command line when you first bring up the Verdi platform. For example: `% verdi -sv -f run.f -ssf sv.fsdb -workMode assertionDebug`

Statistics Frame

On the *Statistics* frame, the assertion results can be shown in a tabular spreadsheet-like format. The **FSDB Statistics** tab summarizes the results for one or more FSDB files and the **Property Statistics** tab displays the results for individual assertions from all FSDB files. The **Property Details** table displays results for individual assertions.

1. In the *Statistics* frame, click the **FSDB Statistics** tab, select the cell with value 6 in the **Assert** row under the **Fail** column.

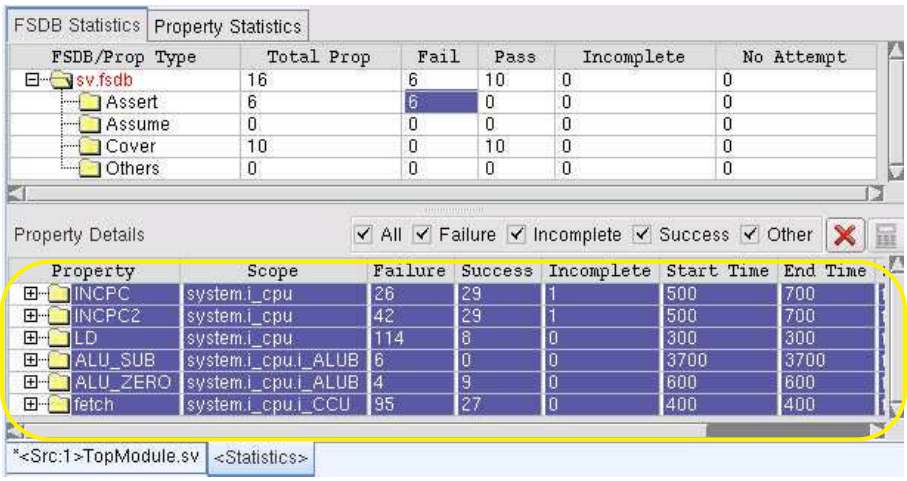
Figure 377 FSDB Statistics



2. Double-click the cell to add all failing assertions to the **Property Details** table.

Feedback

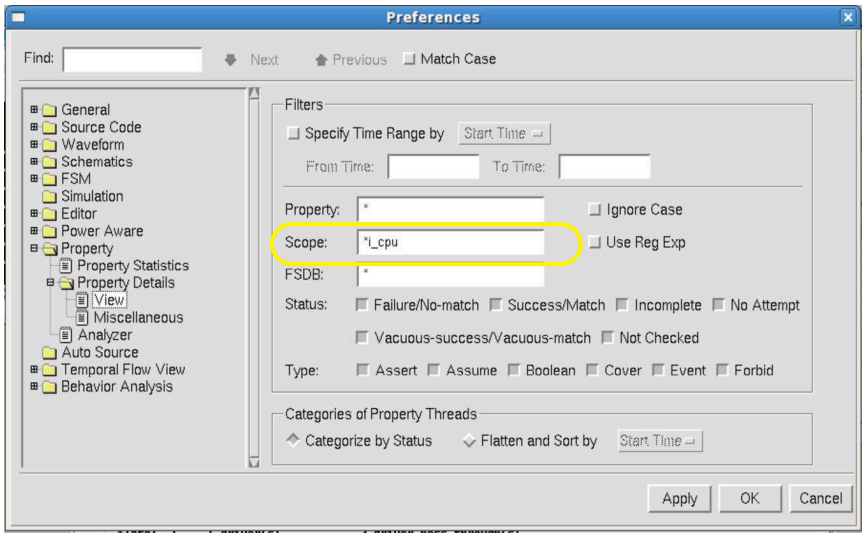
Figure 378 Property Details



You can select the vertical bar between column headers and drag-left to change the column width.

3. In the *Statistics* frame, click the **Options** icon to open the *Preferences* form. The options allow you to control the time range, the property status/type and the fields for display.
4. In the *Preferences* form, select the **Property** folder **-> Property Details** folder **-> View** page, and then specify *i_cpu in the **Scope** field.

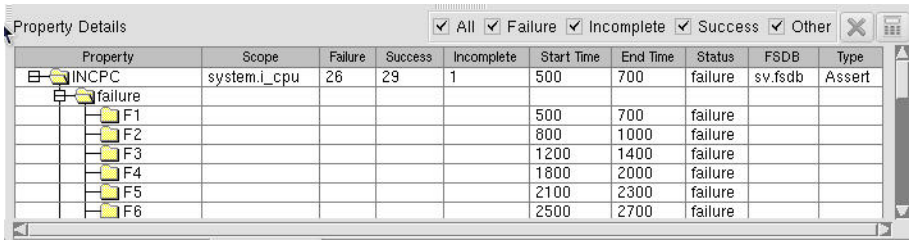
Figure 379 Preferences Form - Property Details Page



Feedback

5. Click the **OK** button. Only the properties for scope *i_cpu* is displayed.
6. Click the + symbol associated with *INCPC* to display failure and success groups. Click the + symbol on failure/success group to display the individual failures/successes.

Figure 380 Assertion Failures



Property	Scope	Failure	Success	Incomplete	Start Time	End Time	Status	FSDB	Type
INCPC	system.i_cpu	26	29	1	500	700	failure	sv.fsdb	Assert
failure									
F1					500	700	failure		
F2					800	1000	failure		
F3					1200	1400	failure		
F4					1800	2000	failure		
F5					2100	2300	failure		
F6					2500	2700	failure		

The failures are named F1 to Fn starting from the first failure in time. Successes are named S1 to Sn starting from the first success in time.

Waveform

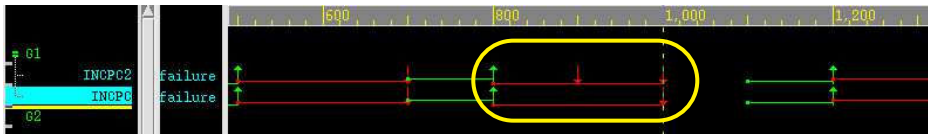
There are a variety of ways to add signals to the waveform. You can drag and drop assertions, properties, sequences, or instances from the design browser frame, the source code frame, or the *Statistics* frame. You can also use the **Get Signals** command. The best method is to select properties and failures of interest from the **Property Details** section of the *Statistics* frame and have them automatically added to the *nWave* frame.

1. In the *Statistics* frame, click the **Options** icon to open the *Preferences* form.
2. In the *Preferences* form, select the **Property** folder **-> Property Details** folder **-> Miscellaneous** page and turn on the **Sync Cursor Time with Selected Property** and **Add Selected Property to nWave When Not Found** options.
3. Select the **Property** folder **-> Analyzer** page and turn on the **Add Evaluated Signals to nWave Automatically** option.
4. In the **Property Details** section of the *Statistics* frame, scroll down to the row containing *INCPC2* and select it. The property is added to the *nWave* frame and the time changes to its first failure.
5. In the **Property Details** section of the *Statistics* frame, scroll back up and select the row containing F2 under *INCPC*. *INCPC* is automatically added to the waveform and

Feedback

the cursor is located at the failure endpoint for F2. The assertions waveforms are similar to the following:

Figure 381 SVA in nWave



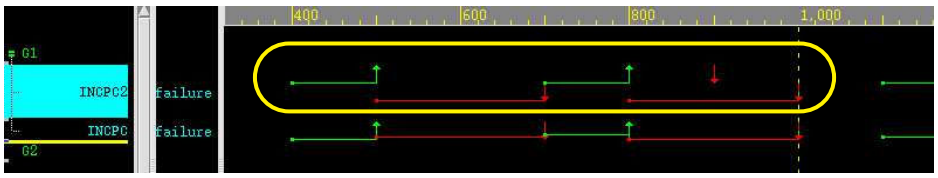
Both *INCPC2* and *INCPC* are temporal assertions. The *nWave* frame shows the time the assertion started evaluating to when it passed or failed using a horizontal line.

6. Zoom in around time 300-1100.

   It is quite difficult to make out the different assertion start times and pass/fail times because of overlapping. In such cases, you can expand the number of rows that are used to display the assertion waveform.

7. Select *INCPC2* (or *INCPC*) and choose the **Waveform -> Property -> Expand Overlapping** command.

Figure 382 INCPC2 Expanded in nWave



The number of rows used to display the waveform for *INCPC2* expands to 2 so that there is no overlap.

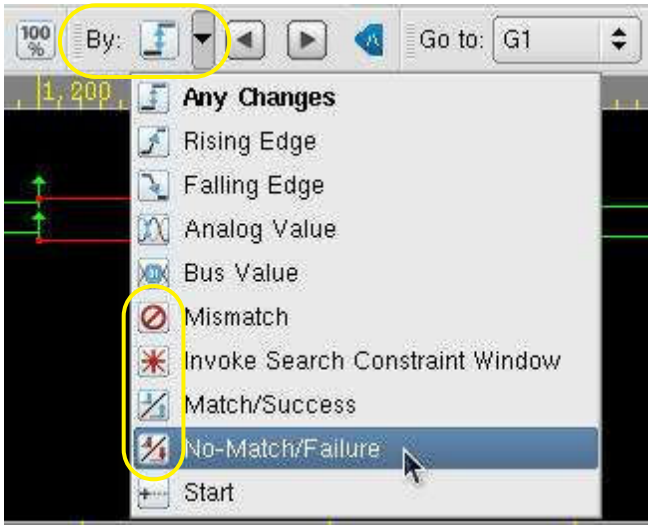
8. Choose the **Waveform -> Property -> Shrink Overlapping** command to go back to a single row.

   You can search for assertion passes, fails, and evaluation begins in the waveform.

9. In the *nWave* frame, click the **By:** menu on the toolbar and select the **No-Match/ Failure** option (red vertical line/down arrow).

Feedback

Figure 383 Search By Options



10. With *INCPC2* selected, click the **Search Forward** or **Search Backward** icons (right or left arrows), to move the cursor to the next or previous failure.

    Let's debug the first failure of the *INCPC2* assert.

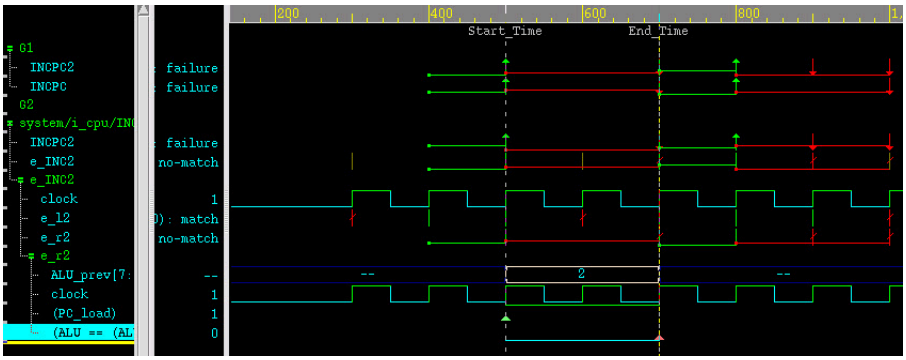
11. In the waveform pane of the *nWave* frame, double-click the first failure of *INCPC2* (at 700 ns) to expand the underlying signals, properties, and sequences from which the assertion is created.

    Note:
    If associated signals are not added into the *nWave* frame, click the **Options** icon in the *Statistics* frame, go to the **Property -> Analyzer** page and turn on the **Add Evaluated Signals to nWave Automatically** option.

    All associated properties, signals, and local variables are expanded, and subgroups are created automatically. All properties which associated with the assert are calculated by Verdi dynamically.

Feedback

Figure 384 Expand Assertions, Properties and Sequences



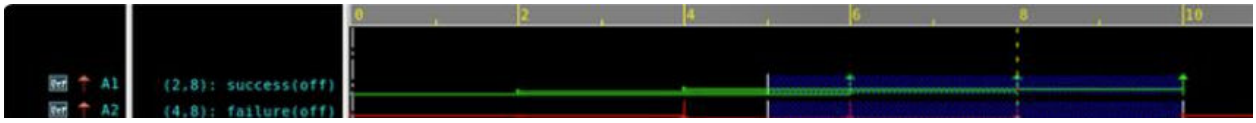
In addition, the failure is evaluated and displayed in the *Analyzer* frame. Refer to the Analyze SVA Assertions section for details.

Assertion Status

Verdi supports the dumping assertion status on/off/kill that corresponds to `$asserton()`/`$assertoff()`/`$assertkill()` in waveform.

The following are the usage of the assertion status:

- **Assertion On**: Reenables the execution of the specified assertion.
- **Assertion Off**: Stops checking of the specified assertion until a subsequent `$asserton`. No new attempts are started. Attempts that are already executing for the assertions, and their pass or fail statements are not affected. For example, 5-10 is the off region (slashed blue line highlight), as shown in the following figure:



- **Assertion Kill**: Aborts the execution of any currently executing attempts for the specified assertion, and then stops the checking of the specified assertion until a subsequent `$asserton`. For example, 15-20 is the kill region (solid blue line highlight), as shown in the following figure:

Feedback

Source Code

Another convenient method to visualize simulation data is through active annotation that allows you to see simulation results under the corresponding variable in the source code frame. This capability has been extended to assertion elements as well.

1. In the design browser frame, double-click *e_INC2* to display the associated source code.
2. In the main window, choose the **Source -> Active Annotation** command (or press **X** on the keyboard) to enable active annotation.

Figure 385 Active Annotation in Source Code Frame

You can see the simulation values under the variable names. For assertions, the following notations are used:

- Green up arrow / Green vertical line: Success / Match
- Red down arrow / Red vertical line: Fail / No-match
- SE: Start Evaluation
- UE: Under Evaluation
- NF: Not Found in FSDB file
- NV: No Value at current time

Active annotation displays NF for assertion-related variables (assertions, properties, sequences, local variables) if you are not in the scope where the variables are referenced. To see the values (instead of NF), change scope to the appropriate assertion, property, or sequence.

The time is synchronized with the cursor time in the *nWave* frame. If you change the cursor in the *nWave* frame, the values annotated change accordingly.

Generate Constructs

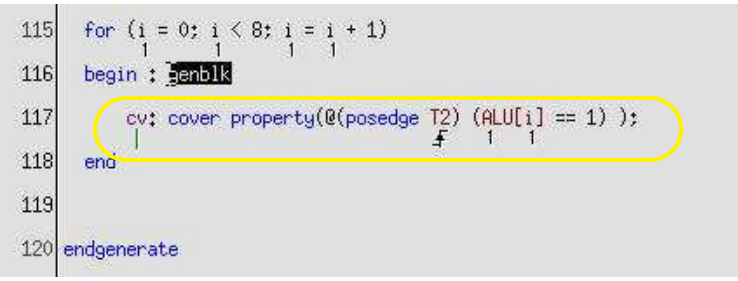
For generate constructs, genvars are not dumped to FSDB during simulation. However, the Verdi platform can elaborate the values of genvars and also build the correct hierarchy

Feedback

for the generated instances in the design browser frame. These elaborated values can also be annotated.

1. In the design browser frame, double-click *genblk[1]* to change the scope. You can see the annotation for cv (cover), ALU (a design signal), and *i* which is a genvar.

Figure 386 Parameter Annotation



You can see the value of *i* annotated as 1.

2. Double-click *genblk[2]* under the ALUB hierarchy to change the scope again and you can see 2 annotated for *i*.

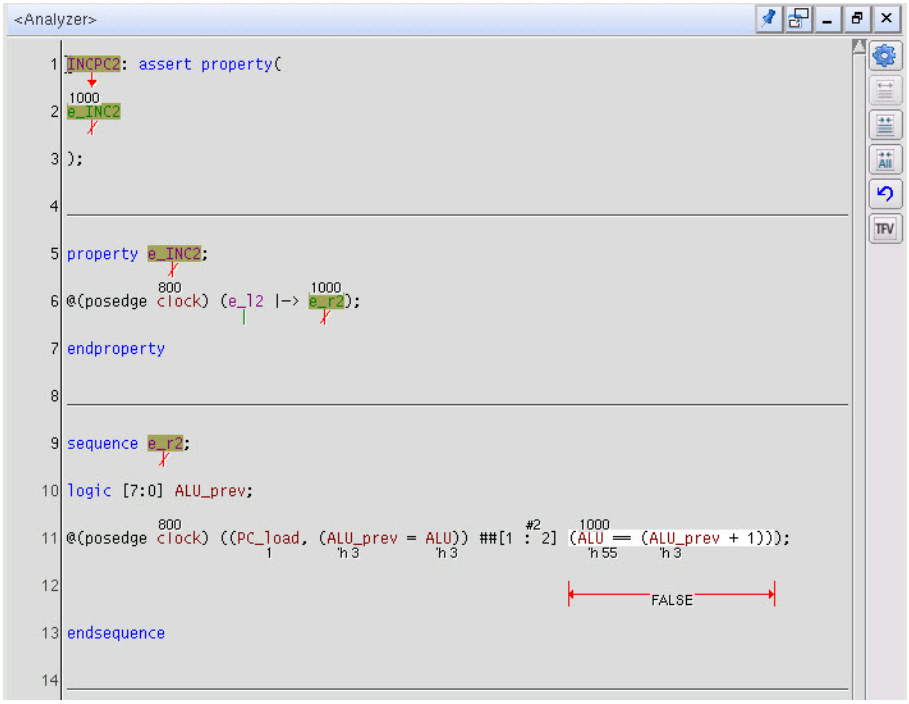
Analyze SVA Assertions

In addition to manually debugging the assertions using the waveform and source code, you can use the Assertion Analyzer to automatically debug the assertion and quickly locate the failing expression and signals.

Feedback

1. In the **Property Details** section of the *Statistics* frame, select the row containing F2 under *INCPC2*.
2. Click the **Analyze Property** button. The *Analyzer* frame updates to display the results. The frame is similar to the following:

Figure 387 Property Tools Window - Analyzer Tab



The *Analyzer* frame is specifically designed for debugging assertions. The Verdi platform adds the related properties so you can easily see the relationship between assertion and property. The extracted source code and the annotated results are displayed the values are annotated according to the time it was evaluated.

Assertion failures always come from the violation of an expression. You can see that ALU does not have the correct value so you can use standard Verdi debug techniques to locate the root cause.

Note:

In addition to analyzing an assertion from the *Statistics* frame, you can start the analysis from a failing signal in the *nWave* frame (just double-click) or in the source code frame (select **Assertion Analyzer** from the right mouse button menu).

Feedback

Note:

If you are using a FSDB file that was generated from the Assertion Evaluator, the local variables is not saved in the file itself; however, when you analyze the assertion, the local variable value is calculated on the fly.

Evaluate SVA Assertions

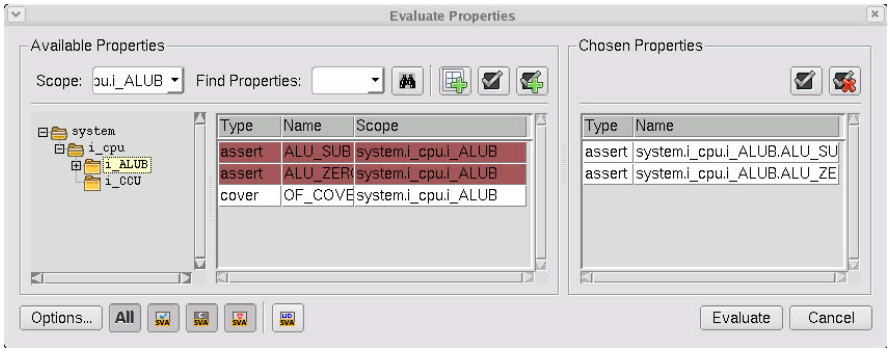
Assume you either have a simulation results file that does not contain SVA results or you have a FSDB file with SVA that you want to recheck. Rather than run the simulation again and dump the SVA results as well, let's use the Assertion Evaluator engine to check SVA against an existing FSDB file.

See the SETUP, SIM_RTL and SIM_SVA scripts and sim_sva_fsdb.do file in the demo directory for details on the appropriate simulator commands if you want to resimulate the design file or use the simulator to check and dump SVA.

In this example, check the currently loaded FSDB file (sv.fsdb) again.

1. In the main window, choose the **Tools -> Property Tools -> Evaluator** command to open the *Evaluate Properties* form.
2. In the left pane click the *i_ALUB* scope (under *system -> i_cpu*), all assertions under the scope is listed in the middle pane.
3. In the middle pane of the *Evaluate Properties* form, drag-left on assertions *ALUB_SUB* and *ALU_ZERO* to select, and then click the **Add Selected Properties** icon to add them to the right pane. These two assertions are flagged to be recalculated by Verdi. The form is be similar to the following:

Figure 388 Evaluate Properties Form



4. In the *Evaluate Properties* form, click the **Evaluate** button.
5. Click **Yes** on the first *Question* dialog window and **OK** on the second.

   The Assertion Evaluator generates an FSDB file called *sva_checker_results.fsdb.vf* containing the assertions and design signals. The **General** tab of the *Message* frame

is brought forward with a summary of the evaluation results. Select the Statistics frame to see the new FSDB file loaded in the **FSDB Statistics** tab. Now you can debug assertion failures as described previously.

SVA Evaluation of Runtime Assertions

The Assertion Evaluation Engine of the Verdi Automated Debug Platform provides a runtime assertion mechanism to add assertions during runtime without compiling and loading the design again.

In general, assertions are added in the design, compiled with the design, and then the results are checked by the Assertion Debug functions after they are loaded and undergo simulation. The Assertion Evaluation Engine also checks assertions against a signal level FSDB trace file for the design.

If there is change in assertion demands, the design, including the newly added assertion code, must be compiled, loaded, and simulated again. The Verdi platform now provides an interface to insert assertions without modifying the original design and check the failure or success of the assertion. There is no need to recompile, reload, and resimulate.

Import Designs and Assertions

The design and SystemVerilog Assertions (SVA) can be imported into the Verdi platform.

See the *Language Support and Compile/Import Methods* chapter in the *Verdi and Siloti Command Reference Manual* for details.

Evaluate With Temporary Assertions

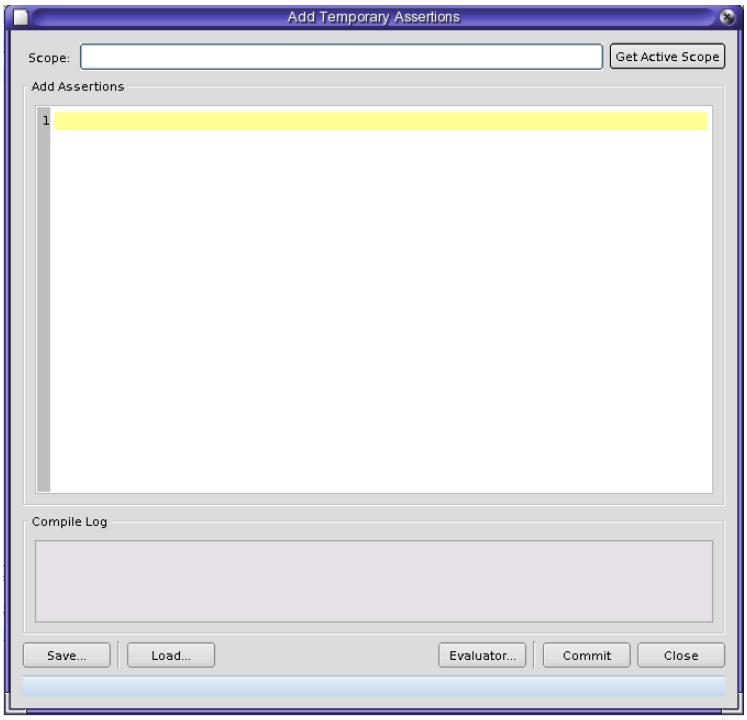
Use the following steps to evaluate assertions:

1. From the **Tools** -> **Property Tools** -> **Evaluator** pane, select the *Evaluator* pane to open it. Or click the **Evaluator** button in the **Add Temporary Assertions** form.

   The assertions that are temporarily added are displayed in blue color to distinguish it from the original assertions.

Feedback

Figure 389 Add Temporary Assertions Form

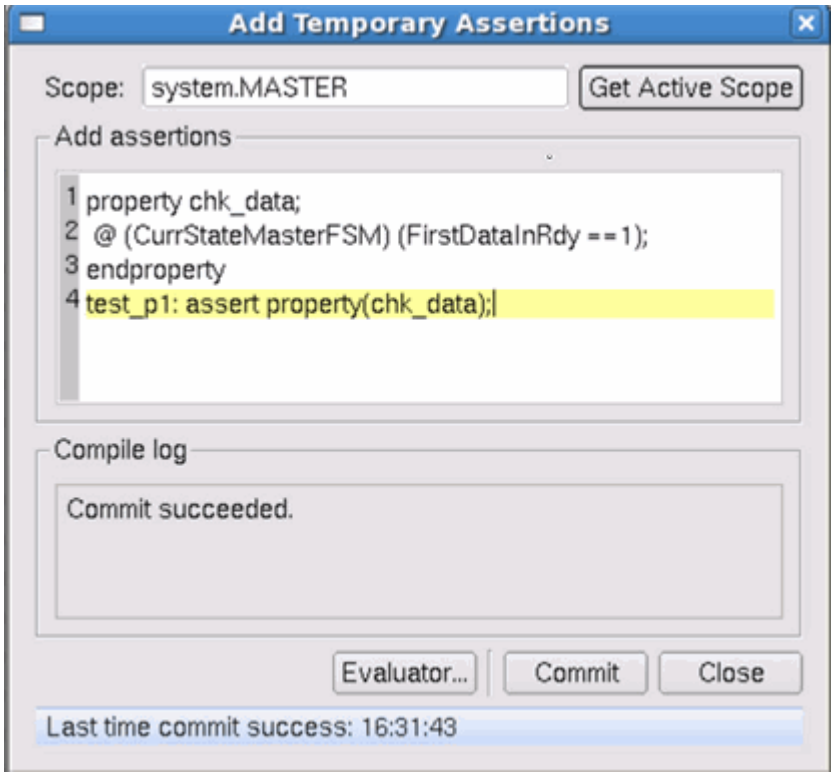


2. Enter the **Scope** or click **Get Active Scope** to automatically enter the active scope.
3. Edit the assertion statement in the **Add Assertions** field as shown in the following figure.
4. Click the **Commit** command.

   The **Compile Log** field displays the compilation results of the new assertion code.

Feedback

Figure 390 Add Temporary Assertion Example



Verdi does not support Macro definitions in **Add Temporary Assertions**.

Consider the following example

```
property P1;

@(posedge clock) R_W == `High && VMA == `Low |-> ## [1:2] addr ==
 8'h55;

endproperty
```

Where,

`High` and `Low` are defined as Macros in RTL as follows:

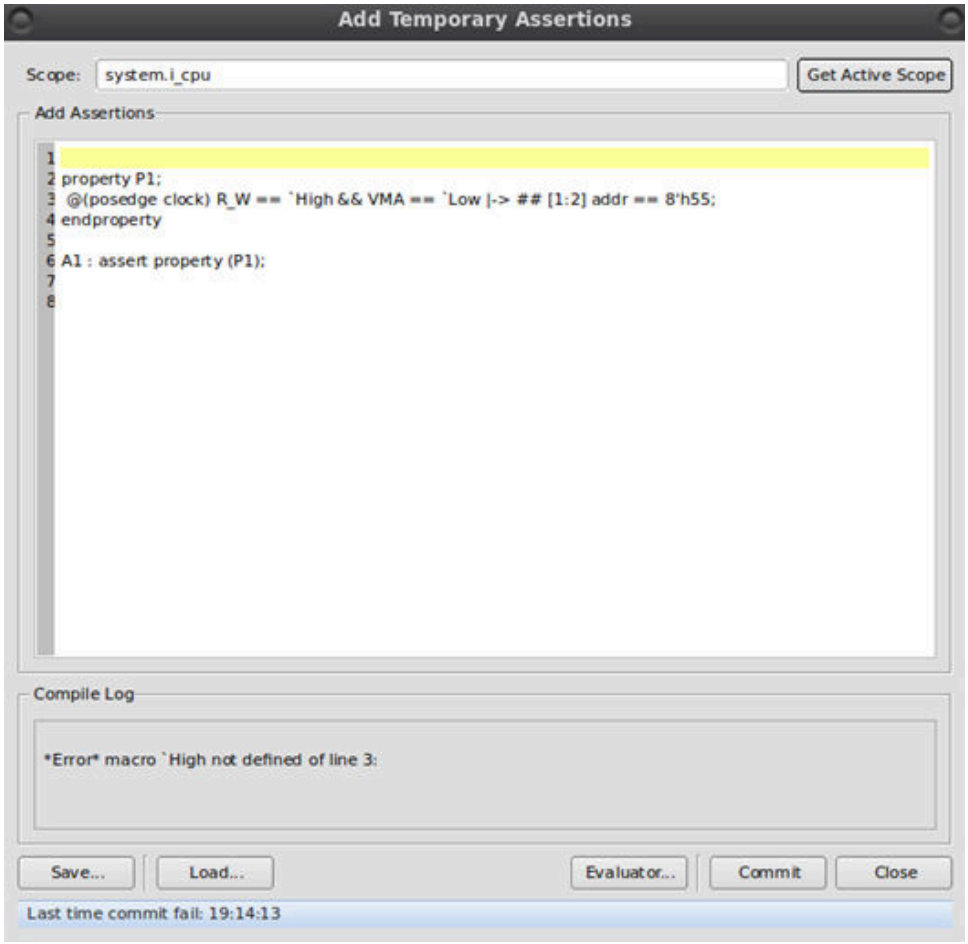
```
-----------------------------------------------------------------
`timescale 1 ns / 1 ns
`define High 1
`define Low 0
```

Feedback

```
module CPU(clock, reset, VMA, R_W, data, addr);
-----------------------------------------------------------------
```

When executing **Add Temporary Assertions**, it displays the error `Error macro `High not defined of line 3`, as shown in the following figure:

Figure 391 No support for Macros in Assertions



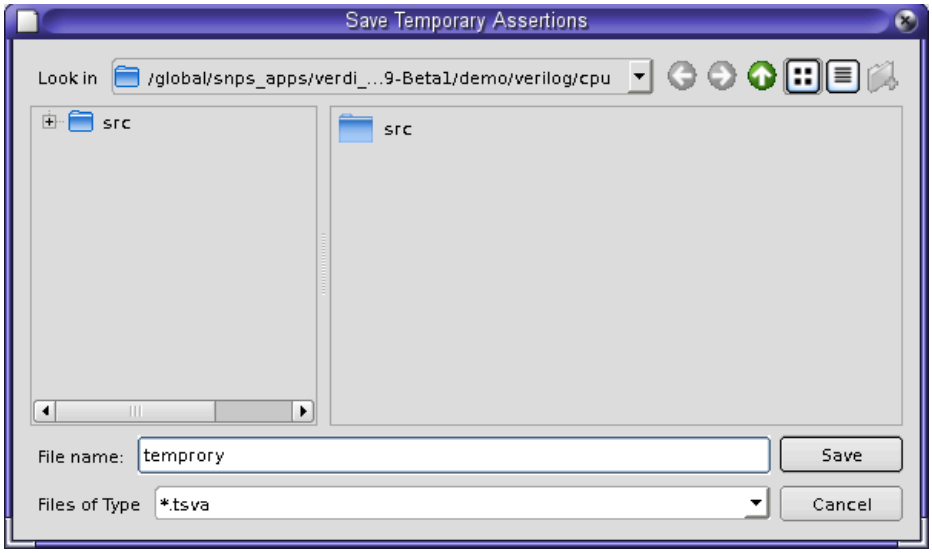
Feedback

Save Temporary Assertions

Perform the following steps to save temporary assertions:

1. Select **Tools** -> **Property Tools** -> **Add Temporary Assertion** command. The **Add Temporary Assertion** form opens.
2. After editing the assertion statements, click **Save** to save the assertions and scope to a file with `.tsva` extension as shown in the following figure.

Figure 392 Save Temporary Assertion



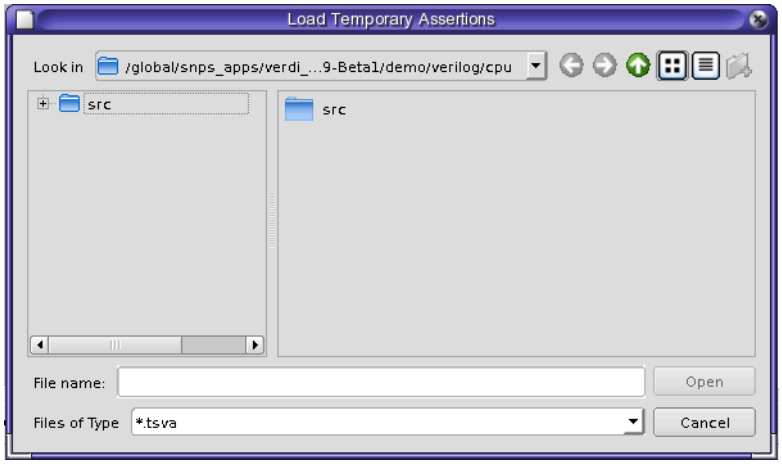
Feedback

Load Temporary Assertions

Perform the following steps to save temporary assertions:

1. Select **Tools** -> **Property Tools** -> **Add Temporary Assertion** command. The **Add Temporary Assertion** form opens.
2. Click **Load** to load the assertions and scope from a file with a.*tsva* extension as shown in the following figure.

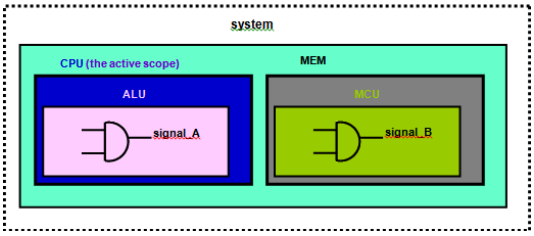
Figure 393 Load Temporary Assertion Form



Note the following:

The SVA evaluation of runtime assertions mechanism only supports the hierarchy name for those signals that belong to or are under the active scope. For example, as shown in the following figure, the active scope is CPU, ALU.signal_A is the allowed signal and system.MEM.MCU.signal_B is not.

Figure 394 A Design Sample



- The SVA evaluation of runtime assertions mechanism only supports the assertion in a scope. Only one scope in the **Scope** text field is allowed.
- The SVA evaluation of runtime assertion mechanism does not support the following:
  - Immediate assertions
  - Property and sequence built-in functions
  - PSL related nodes
  - Using macros in the newly added SVA syntax
- The mechanism used for SVA evaluation of runtime assertions cannot commit an assertion with the same name in the same scope in the design.
- Previously committed assertions are removed after committing a new assertion.

Debug With Transactors

Before you begin this application, follow the instructions in the Before You Begin chapter.

Transactions are an important piece of abstraction in system design and debug. System design is in an early stage of the whole design process. Therefore, a powerful viewing mechanism for transactions is mandatory for system designers.

For testbench verification, if the entire system is to be verified, transaction level checking is efficient and easy to focus comparisons of system behavior against system specification. When an error is found in the transaction level, the signal level is then investigated.

This section covers the following topics:

- What is a Transaction?
- Generating Transaction Data
- View Transactions in nWave
- View Transactions in Transaction Table View Window

What is a Transaction?

Transactions are higher level abstractions of signal-level detailed activity and are organized into streams. Transaction streams can be dumped into FSDB format using dumping libraries provided by Synopsys and its partners or using the Open Transaction Interface (OTI) extension of the FSDB Writer API.

Streams hold transactions. Each transaction consists of a set of attributes and is independent of one another. That is, there is no such concept as "transaction type", as in SCV, even if the sets of attributes that constitute two transactions are the same.

When you create a transaction, you must follow these steps :

1. Create a stream.
2. Create attributes.
3. Create a transaction.
4. Create relationships between existing transaction.

Generating Transaction Data

The transaction data can be obtained from the following sources.

Provided FSDB Dumpers

Dump transaction data from languages directly with native FSDB dumpers.

- SystemC/SCV -- supported OSCI and NCSC simulators.
- Specman/e
- SystemVerilog testbench in conjunction with simulator support (VCS, ModelSim)
- Vera

Refer to the *SystemC Linking* chapter in the *Linking Novas Files with Simulators and Enabling FSDB Dumping* manual for details on linking native FSDB dumpers, SystemC SCV, and HVL simulators.

Transaction IP Partners

Contact Denali (PCI-Express) or Spiratech (AMBA AXI, AHB) directly for details on dumping FSDB format from their available intellectual property (IP).

SVA Extraction

You can add SystemVerilog Assertions (SVA) constructs to your design code to represent transactions. The transactions can then be extracted from a signal level FSDB. Refer to the Extracting Transactions Using SVA section in Appendix D: Additional Transaction Example for more details.

Feedback

FSDB Writer API and the Open Transaction Interface (OTI)

If you are unable to generate transaction data in FSDB format from any of the previously mentioned methods, you can use the Open Transaction Interface (OTI) extension of the FSDB writer API to dump transaction data.

SVTB Automatic Logging of OVM/UVM Component and Port Transactions

SVTB-based testbench environments are typically built on top of an SVTB verification library/methodology like OVM/UVM. Synopsys has leveraged the infrastructure provided in OVM/UVM to record component and port transactions flowing up and down the testbench into the FSDB file. The Synopsys mechanism uses the OVM/UVM's transaction recording capability to record OVM/UVM testbench transactions into the FSDB file for debug and analysis in the Verdi platform.

View Transactions in nWave

This tutorial familiarizes you with transaction viewing and search operations. All *nWave* manipulation functions (zoom, cursor, marker, resize, and so on) are available with FSDB files containing transactions.

1. Change the directory to *<working_dir>/demo/transaction.*

   ```
   % cd <working_dir>/demo/transaction
   ```

2. Execute Verdi to import the FSDB file:

   ```
   > verdi -ssf ahb32.bus.fsdb -workMode hardwareDebug &
   ```

   The Verdi platform opens and the FSDB file is loaded.

   Note:
   This tutorial only has an FSDB file that contains transactions and there is not a related design.

3. In the *nWave* frame, choose the **Signal -> Get Signals** command to open the *Get Signals* form.

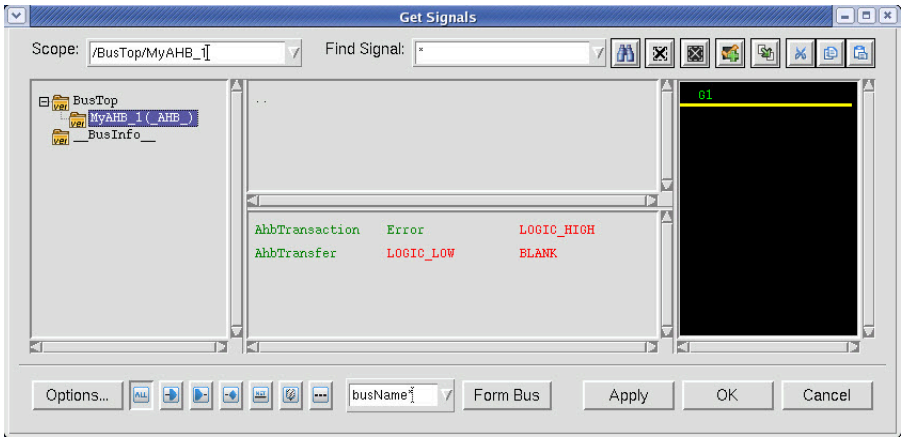
   The transaction FSDB is loaded displaying *BusTop* as the top hierarchy and *MyAHB_1(_AHB_)* as the first hierarchical level which is for different protocols.

4. Click *MyAHB_1* to show the streams under this hierarchy.

Feedback

The results are similar to the following example:

Figure 395 Get Signals - Displaying Streams



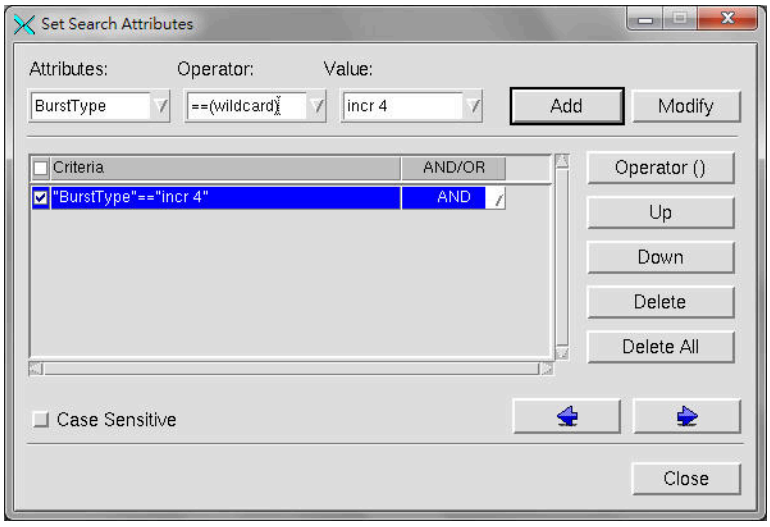
5. Select *AhbTransfer* and *AhbTransaction*, then click **OK**.
6. In the *nWave* frame, re-size the signal and value panes to more readily display the text and zoom in on the waveform pane to see the transaction details.
7. In the signal pan, click the *AhbTransaction* stream to select it.
8. Click the **Search Forward** icon (right arrow) in the toolbar to step through the transactions.

   The cursor moves to the begin time of each transaction.
9. Since there are more attributes than the default signal height can display, you can adjust the height by dragging the small grey line in the lower-left corner of the stream name in the signal pane.
10. In the value pan, move the mouse cursor over the attributes to show the details in a tip.
11. With the *AhbTransaction* stream selected, choose the **Waveform -> Classic Transaction -> Expand Overlapping** command to make it easier to see the transaction overlap.
12. Choose the **Waveform -> Classic Transaction -> Shrink Overlapping** command to return to the overlapped view.
13. Click the **By:** icon in the *nWave* toolbar and choose the last option, **Transaction Attribute Values**.
14. In the *Set Search Attributes* form, enter “*BurstType*” for **Attributes** and “*incr 4*” for **Value**.

Feedback

The form is similar to the following:

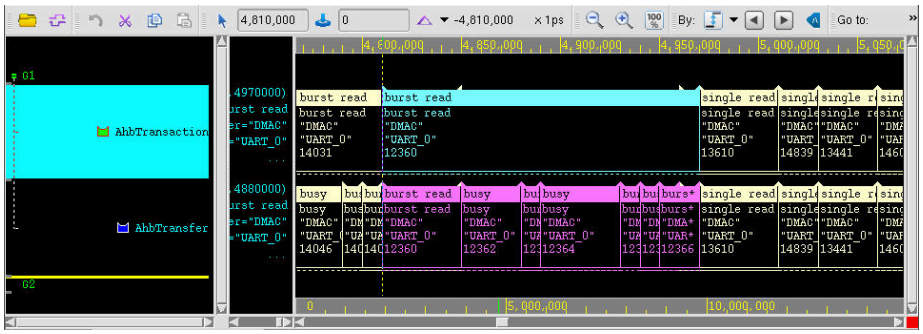
Figure 396 Set Search Attributes



15. Click the **Add** button.
16. Click the Search Forward/Search Backward icons on the *Set Search Attributes* form to locate a matching transaction at 4810000ps.
17. Click the transaction in the waveform pane at time 4,810,000ps, which is burst read of “incr 4” type.

    There is 4 *AhbTransfer* burst read command transactions and 3 busy ones as the children of the selected transaction. The child transactions are highlighted in pink.

Figure 397 Search Results with Related Transactions



18. With the same transaction selected, right-click to open the right mouse button context menu and choose **Properties** to open the *Transaction Property* form which shows all the attributes and relationships for the selected transaction.

Feedback

View Transactions in Transaction Table View Window

This tutorial familiarizes you with transaction viewing and search operations in a spreadsheet-like view.

1. Change the directory to *<working_dir>/demo/transaction.*

   ```
   % cd <working_dir>/demo/transaction
   ```

2. Execute Verdi to import the FSDB file:

   ```
   > verdi -ssf ahb32.bus.fsdb -workMode hardwareDebug &
   ```

   The *nTrace* main window and *nWave* frame open and the FSDB file is loaded.

   Note:
   This tutorial only has an FSDB file that contains transactions and there is not a related design.

3. In the *nWave* frame, choose the **Signal -> Get Signals** command to open the *Get Signals* form.

   The transaction FSDB is loaded displaying *BusTop* as the top hierarchy and *MyAHB_1(_AHB_)* as the first hierarchical level which is for different protocols.

4. Click *MyAHB_1* to show the streams under this hierarchy.

5. Select *AhbTransfer* and *AhbTransaction* and then click **OK**.

Add/Remove Transaction Streams

After loading a FSDB file with transaction data, you can view and manipulate the results in the *Transaction Table View* frame.

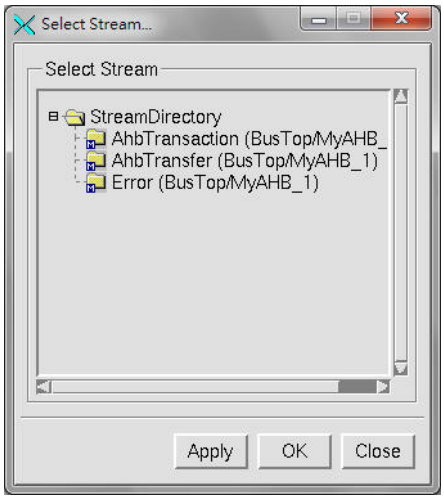
1. In the *nWave* frame, choose the **Tools -> Classic Transaction -> Analysis Window** command. The *Transaction Table View* frame is opened as a new frame in the right half of the *nWave* frame, the FSDB file currently loaded in *nWave* is the default in the *Transaction Table View* frame.

2. In the *Transaction Table View* frame, choose the **Stream -> Get Stream** command to open the *Select Stream* form.

Feedback

The form is similar to the following:

Figure 398 Select Stream Form

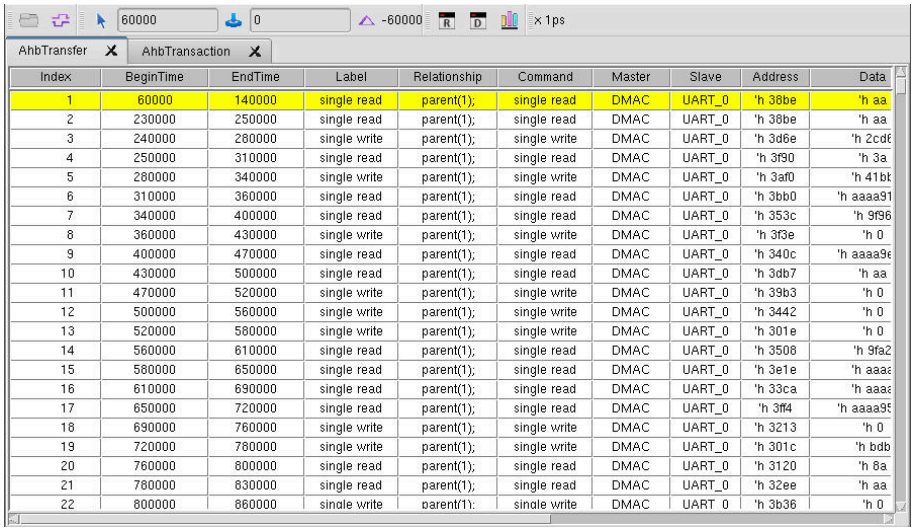


All of the transaction streams available in the FSDB file is listed in a tree-like format.

3. Double-click *AhbTransfer* to automatically add the stream to the *Transaction Table View* frame. The stream name changes to gray and is appended with a red dot.
4. Left-click to select *AhbTransaction* and click **OK** to add the stream and close the form.

   The *Transaction Table View* frame is similar to the following:

Figure 399 Transaction Table View Frame with Streams Loaded



Index	BeginTime	EndTime	Label	Relationship	Command	Master	Slave	Address	Data
1	60000	140000	single read	parent(1);	single read	DMAC	UART_0	'h 38be	'h aa
2	230000	250000	single read	parent(1);	single read	DMAC	UART_0	'h 38be	'h aa
3	240000	280000	single write	parent(1);	single write	DMAC	UART_0	'h 3d6e	'h 2cdf
4	250000	310000	single read	parent(1);	single read	DMAC	UART_0	'h 3f90	'h 3a
5	280000	340000	single write	parent(1);	single write	DMAC	UART_0	'h 3af0	'h 41bb
6	310000	360000	single read	parent(1);	single read	DMAC	UART_0	'h 3bb0	'h aaaa91
7	340000	400000	single read	parent(1);	single read	DMAC	UART_0	'h 353c	'h 9f96
8	360000	430000	single write	parent(1);	single write	DMAC	UART_0	'h 3f3e	'h 0
9	400000	470000	single read	parent(1);	single read	DMAC	UART_0	'h 340c	'h aaaa9e
10	430000	500000	single read	parent(1);	single read	DMAC	UART_0	'h 3db7	'h aa
11	470000	520000	single write	parent(1);	single write	DMAC	UART_0	'h 39b3	'h 0
12	500000	560000	single write	parent(1);	single write	DMAC	UART_0	'h 3442	'h 0
13	520000	580000	single write	parent(1);	single write	DMAC	UART_0	'h 301e	'h 0
14	560000	610000	single read	parent(1);	single read	DMAC	UART_0	'h 3508	'h 9fa2
15	580000	650000	single read	parent(1);	single read	DMAC	UART_0	'h 3e1e	'h aaa
16	610000	690000	single read	parent(1);	single read	DMAC	UART_0	'h 33ca	'h aaa
17	650000	720000	single read	parent(1);	single read	DMAC	UART_0	'h 3ff4	'h aaaa95
18	690000	760000	single write	parent(1);	single write	DMAC	UART_0	'h 3213	'h 0
19	720000	780000	single write	parent(1);	single write	DMAC	UART_0	'h 301c	'h bdb
20	760000	800000	single read	parent(1);	single read	DMAC	UART_0	'h 3120	'h 8a
21	780000	830000	single read	parent(1);	single read	DMAC	UART_0	'h 32ee	'h aa
22	800000	860000	single write	parent(1);	single write	DMAC	UART_0	'h 3b36	'h 0

Feedback

There are two streams, *AhbTransfer* and *AhbTransaction*, in the *Transaction Table View* frame. Each stream has a tab of its own. You can select the stream name to see the details of the stream. The currently selected stream name is in blue. You can change the width of the columns by selecting the vertical line in the column header and dragging-left.

5. Left-click to select the *AhbTransaction*.
6. Choose the **Stream -> Close Stream** command. Note the stream has been removed from the *Transaction Table View* frame.

Merge Transaction Streams

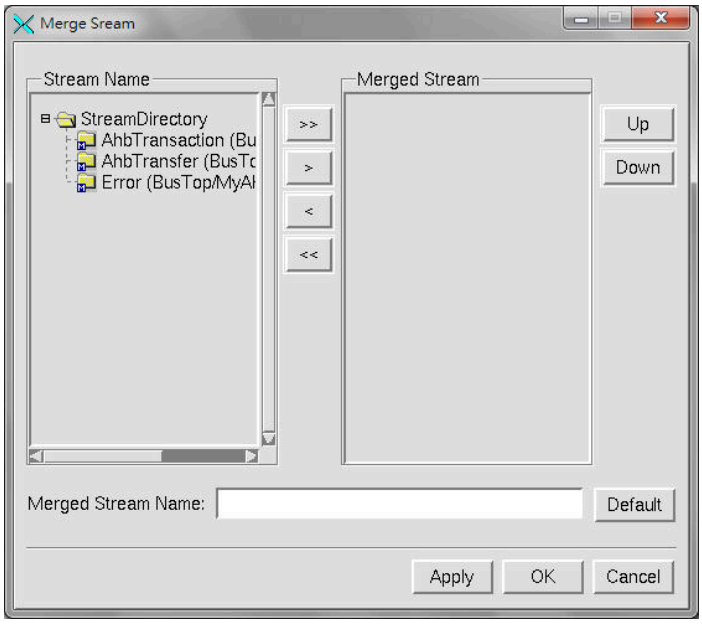
You can also merge two or more streams in the *Transaction Table View* frame. When streams are merged you can search and filter all the transaction attributes simultaneously.

Note:
The **Merge Stream** command only merges the transaction streams for viewing purposes; it does not effect the FSDB file.

1. In the *Transaction Table View* frame, choose the **Stream -> Merge Stream** command to open the *Merge Stream* form.

   The form is similar to the following:

Figure 400 Merge Stream Form



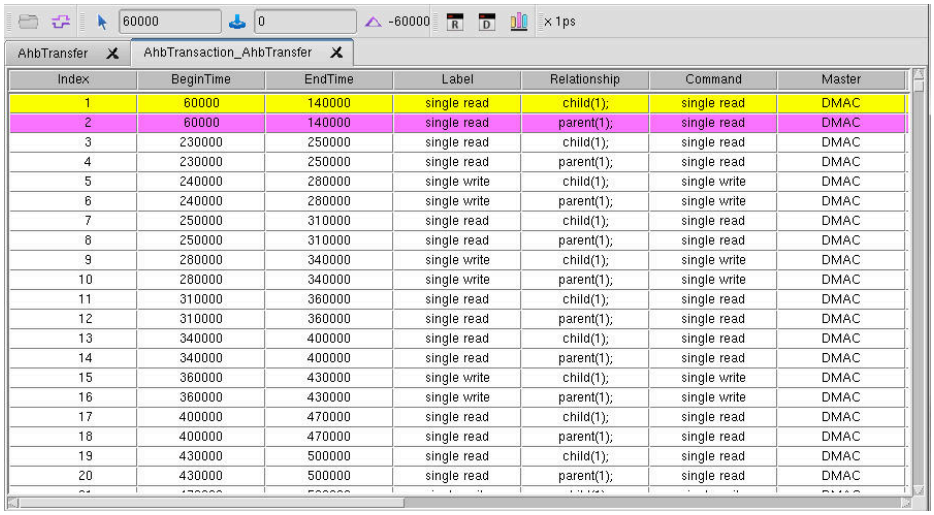
Feedback

All of the transaction streams available in the FSDB file is listed in a tree-like format in the **Stream Name** column.

2. Click the >> button to move all streams to the **Merged Stream** column. After the stream is added, its name becomes gray with a red dot in the **Stream Name** column and cannot be selected again.
3. Left-click to select the *Error* stream in the **Merged Stream** column.
4. Click the < button to move the selection back to the **Stream Name** column. The stream name is changed to black and is selectable again.
5. Left-click to select the *AhbTransaction* stream in the **Merged Stream** column.
6. Click the **Default** button to automatically generate the merged stream name which consists of each stream name linked with an underscore.
7. Click the **OK** button.

   The *Transaction Table View* frame is similar to the following:

Figure 401 Merged Stream in the Transaction Table View Frame



Index	BeginTime	EndTime	Label	Relationship	Command	Master
1	60000	140000	single read	child(1);	single read	DMAC
2	60000	140000	single read	parent(1);	single read	DMAC
3	230000	250000	single read	child(1);	single read	DMAC
4	230000	250000	single read	parent(1);	single read	DMAC
5	240000	280000	single write	child(1);	single write	DMAC
6	240000	280000	single write	parent(1);	single write	DMAC
7	250000	310000	single read	child(1);	single read	DMAC
8	250000	310000	single read	parent(1);	single read	DMAC
9	280000	340000	single write	child(1);	single write	DMAC
10	280000	340000	single write	parent(1);	single write	DMAC
11	310000	360000	single read	child(1);	single read	DMAC
12	310000	360000	single read	parent(1);	single read	DMAC
13	340000	400000	single read	child(1);	single read	DMAC
14	340000	400000	single read	parent(1);	single read	DMAC
15	360000	430000	single write	child(1);	single write	DMAC
16	360000	430000	single write	parent(1);	single write	DMAC
17	400000	470000	single read	child(1);	single read	DMAC
18	400000	470000	single read	parent(1);	single read	DMAC
19	430000	500000	single read	child(1);	single read	DMAC
20	430000	500000	single read	parent(1);	single read	DMAC

If the different streams have transactions at the same time, both are displayed.

Manipulate the Stream View

There are several ways to manipulate the streams in the *Transaction Table View* frame. You can change which columns (attributes) are displayed and in what order. You can also filter the transactions based on one or two attribute conditions.

Feedback

Set the Cursor/Marker

In this example, you'll set the cursor/marker position in the *Transaction Table View* frame and learn how to synchronize it with the other Verdi frames.

1. In the *Transaction Table View* frame, select the *AhbTransfer* stream.
2. Left-click anywhere on the row for **Index 13** to set the cursor time. The selected row is highlighted in yellow.
3. Scroll until you can see **Index 25**.
4. Middle-click anywhere on the row for **Index 25** to set the marker time. The selected row is highlighted in red.
5. Choose the **View -> Sync Cursor Time**command to synchronize the cursor globally.
6. Left-click anywhere on the row for **Index 18** to set the cursor time. Note the cursor time changes in the *nWave* frame as well. If you had a design loaded and active annotation enabled, the cursor time would change in the source code frame and *nSchema* frame as well.

Change the Column (Attribute) Display

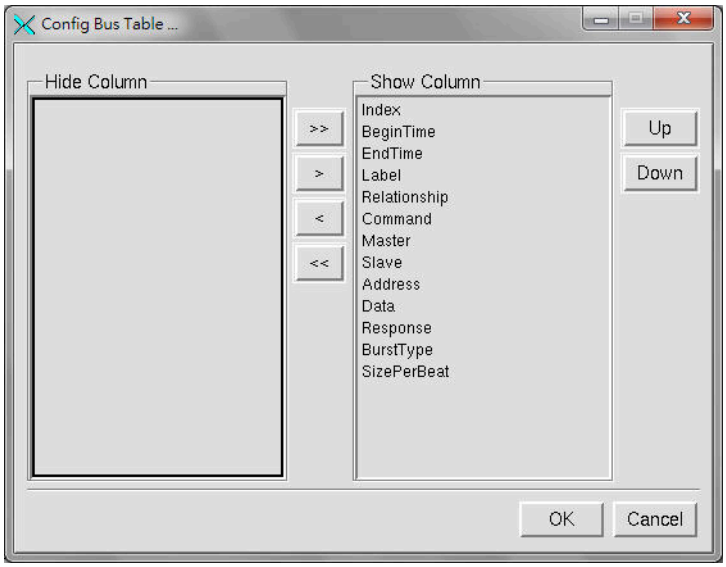
In this example, you'll select some columns (attributes) to remove from the display and reorder the remaining columns.

1. In the *Transaction Table View* frame, select the *AhbTransfer* stream.
2. Choose the **View -> Column Configuration** command to open the *Config Bus Table* form.

Feedback

The form is similar to the following:

Figure 402 Config Bus Table Form



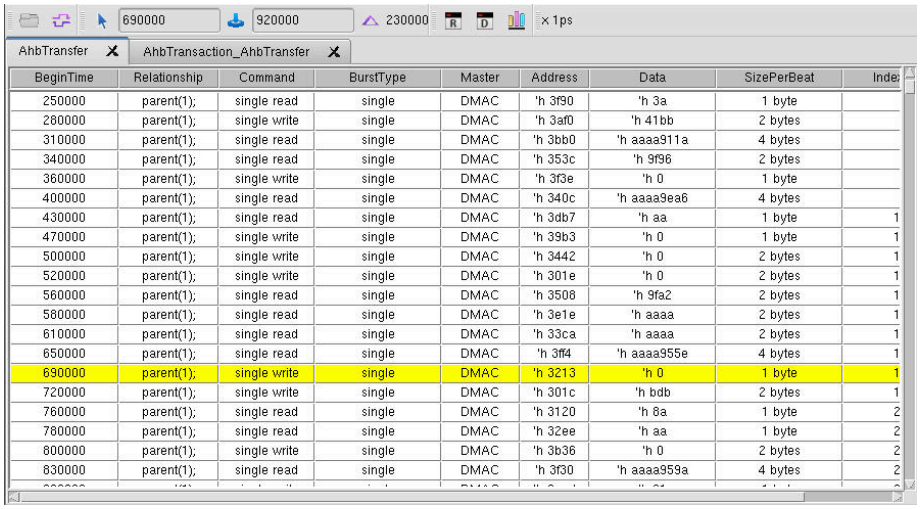
By default, all the columns (attributes) are listed in the **Show Column** section.

3. Select **Label** in the **Show Column** section.
4. Click the button to move it to the **Hide Column** section.
5. Repeat the previous steps for **Response**, **Slave**, and **EndTime** individually. Only one attribute can be selected at a time.
6. In the **Show Column** section, select **Index**.
7. Click the **Down** button multiple times until **Index** is at the bottom of the list.
8. In the **Show Column** section, select **BurstType**.
9. Click the **Up** button multiple times until **BurstType** is located below **Command**.
10. Click **OK**.

Feedback

The *Transaction Table View* frame is updated as follows:

Figure 403 Modified Attribute Display for AhbTransfer Stream



BeginTime	Relationship	Command	BurstType	Master	Address	Data	SizePerBeat	Inde
250000	parent(1);	single read	single	DMAC	'h 3f90	'h 3a	1 byte	
280000	parent(1);	single write	single	DMAC	'h 3af0	'h 41bb	2 bytes	
310000	parent(1);	single read	single	DMAC	'h 3bb0	'h aaaa911a	4 bytes	
340000	parent(1);	single read	single	DMAC	'h 353c	'h 9f96	2 bytes	
360000	parent(1);	single write	single	DMAC	'h 3f3e	'h 0	1 byte	
400000	parent(1);	single read	single	DMAC	'h 340c	'h aaaa9ea6	4 bytes	
430000	parent(1);	single read	single	DMAC	'h 3db7	'h aa	1 byte	1
470000	parent(1);	single write	single	DMAC	'h 39b3	'h 0	1 byte	1
500000	parent(1);	single write	single	DMAC	'h 3442	'h 0	2 bytes	1
520000	parent(1);	single write	single	DMAC	'h 301e	'h 0	2 bytes	1
560000	parent(1);	single read	single	DMAC	'h 3508	'h 9fa2	2 bytes	1
580000	parent(1);	single read	single	DMAC	'h 3e1e	'h aaaa	2 bytes	1
610000	parent(1);	single read	single	DMAC	'h 33ca	'h aaaa	2 bytes	1
650000	parent(1);	single read	single	DMAC	'h 3ff4	'h aaaa955e	4 bytes	1
690000	parent(1);	single write	single	DMAC	'h 3213	'h 0	1 byte	1
720000	parent(1);	single write	single	DMAC	'h 301c	'h bdb	2 bytes	1
760000	parent(1);	single read	single	DMAC	'h 3120	'h 8a	1 byte	2
780000	parent(1);	single read	single	DMAC	'h 32ee	'h aa	1 byte	2
800000	parent(1);	single write	single	DMAC	'h 3b36	'h 0	2 bytes	2
830000	parent(1);	single read	single	DMAC	'h 3f30	'h aaaa959a	4 bytes	2

Note four columns have been removed from the display and the remaining columns have been reordered. The **Index** column is now the right-most column and **BurstType** is next to **Command**.

11. Left-click the **Command** column to sort by the command attribute types.

Filter the Transactions

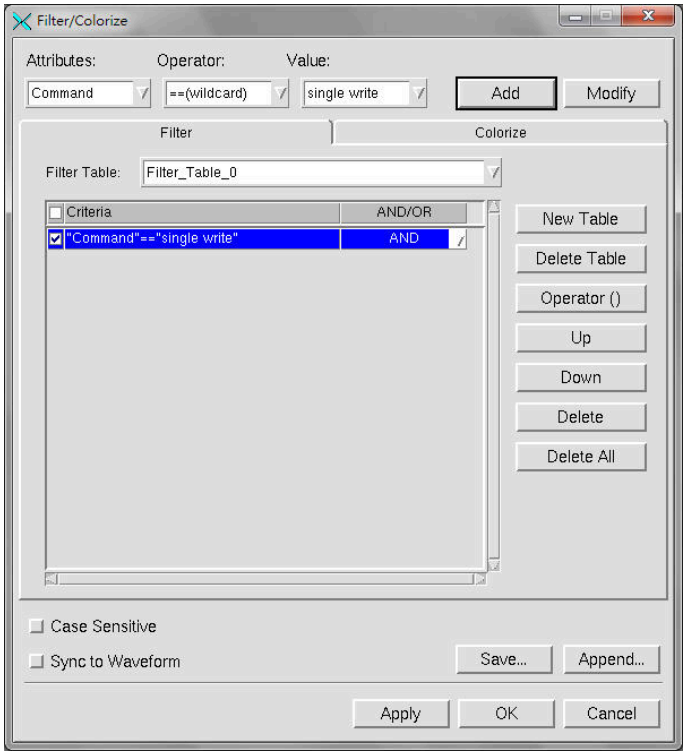
In this example, you can filter the transactions based on certain attributes.

1. In the *Transaction Table View* frame, select the *AhbTransfer* stream.
2. Choose the **View -> Filter/Colorize** command to open the *Filter/Colorize* form.
3. Toggle the **Attributes:** field and select **Command**.

Feedback

Toggle the **Operator** to **==(wildcard)** and enter *single write* in the **Value:** field, then click **Add** button. The form is similar to the following:

Figure 404 Filter/Colorize Form - Command = single write

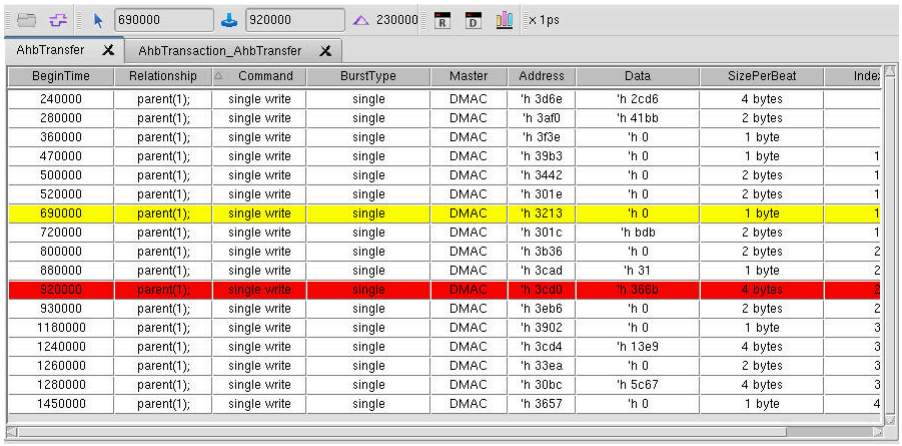


4. Click the **Apply** button.

Feedback

The *Transaction Table View* frame is updated to display transactions whose command attribute is of type single write, similar to the following:

Figure 405 Filter Results for Command single write



BeginTime	Relationship	Command	BurstType	Master	Address	Data	SizePerBeat	Index
240000	parent(1);	single write	single	DMAC	'h 3d6e	'h 2cd6	4 bytes	
280000	parent(1);	single write	single	DMAC	'h 3af0	'h 41bb	2 bytes	
360000	parent(1);	single write	single	DMAC	'h 3f3e	'h 0	1 byte	
470000	parent(1);	single write	single	DMAC	'h 39b3	'h 0	1 byte	1
500000	parent(1);	single write	single	DMAC	'h 3442	'h 0	2 bytes	1
520000	parent(1);	single write	single	DMAC	'h 301e	'h 0	2 bytes	1
690000	parent(1);	single write	single	DMAC	'h 3213	'h 0	1 byte	1
720000	parent(1);	single write	single	DMAC	'h 301c	'h bdb	2 bytes	1
800000	parent(1);	single write	single	DMAC	'h 3b36	'h 0	2 bytes	2
860000	parent(1);	single write	single	DMAC	'h 3cad	'h 31	1 byte	2
920000	parent(1);	single write	single	DMAC	'h 3cd0	'h 366b	4 bytes	2
930000	parent(1);	single write	single	DMAC	'h 3eb6	'h 0	2 bytes	2
1180000	parent(1);	single write	single	DMAC	'h 3902	'h 0	1 byte	3
1240000	parent(1);	single write	single	DMAC	'h 3cd4	'h 13e9	4 bytes	3
1260000	parent(1);	single write	single	DMAC	'h 33ea	'h 0	2 bytes	3
1280000	parent(1);	single write	single	DMAC	'h 30bc	'h 5c67	4 bytes	3
1450000	parent(1);	single write	single	DMAC	'h 3657	'h 0	1 byte	4

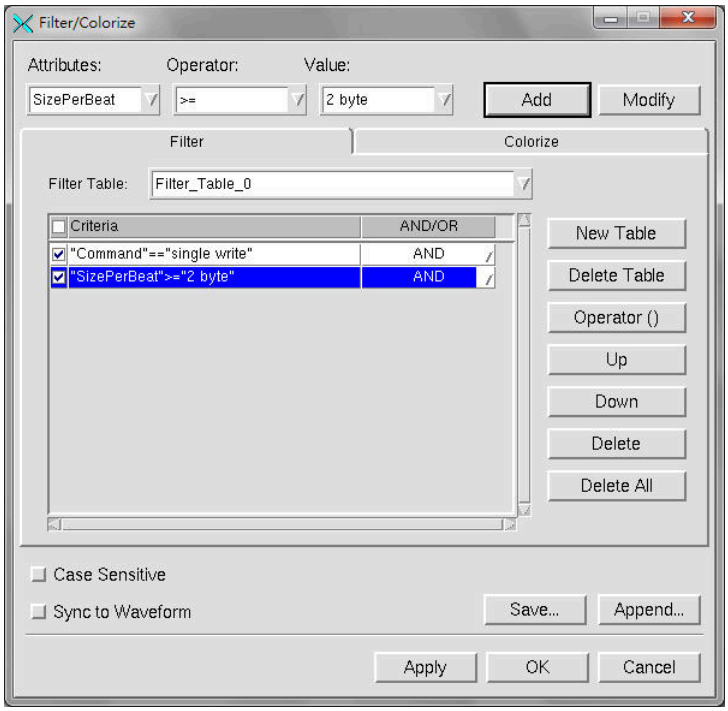
At this point you have several options. You can sort the current results by clicking another column header or you can further reduce the display by specifying another filter or you can restore the stream and start over. Let's specify another filter.

5. In the *Filter/Colorize* form (which should still be open unless you closed it), toggle the **Attributes:** field and select **SizePerBeat**.
6. Toggle the **Operator:** to **>=** and enter *2 byte* in the **Value:** field, and click **Add** button.

Feedback

The form is similar to the following:

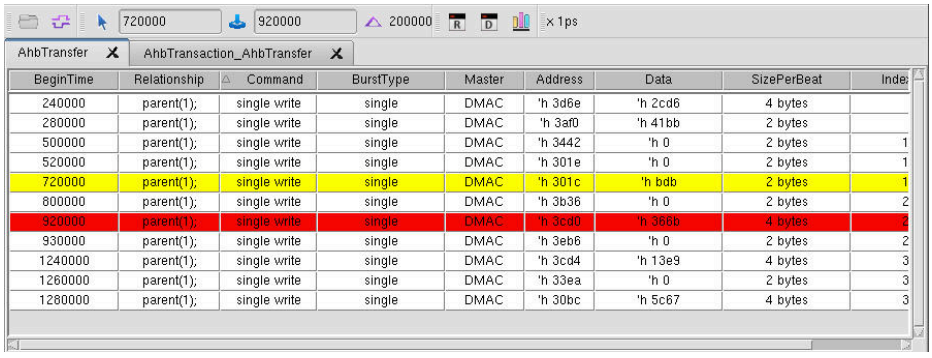
Figure 406 Filter/Colorize Form - SizePerBeat >= 2byte



7. Click the **Apply** button.

   The *Transaction Table View* frame is updated to display transactions whose command attribute is of type single write and whose **SizePerBeat** attribute value is greater than or equal to 2 bytes, similar to the following:

Figure 407 Filter Results for Command single write with SizePerBeat >= 2 bytes



BeginTime	Relationship	Command	BurstType	Master	Address	Data	SizePerBeat	Inde
240000	parent(1);	single write	single	DMAC	'h 3d6e	'h 2cd6	4 bytes	
280000	parent(1);	single write	single	DMAC	'h 3af0	'h 41bb	2 bytes	
500000	parent(1);	single write	single	DMAC	'h 3442	'h 0	2 bytes	1
520000	parent(1);	single write	single	DMAC	'h 301e	'h 0	2 bytes	1
720000	parent(1);	single write	single	DMAC	'h 301c	'h bdb	2 bytes	1
800000	parent(1);	single write	single	DMAC	'h 3b36	'h 0	2 bytes	2
920000	parent(1);	single write	single	DMAC	'h 3cd0	'h 386b	4 bytes	2
930000	parent(1);	single write	single	DMAC	'h 3eb6	'h 0	2 bytes	2
1240000	parent(1);	single write	single	DMAC	'h 3cd4	'h 13e9	4 bytes	3
1260000	parent(1);	single write	single	DMAC	'h 33ea	'h 0	2 bytes	3
1280000	parent(1);	single write	single	DMAC	'h 30bc	'h 5c67	4 bytes	3

Enable the **Sync to Waveform** option in the *Filter/Colorize* form and then the transaction stream in the waveform is also filtered.

Feedback

8. Click the **Sync. Signal Selection Enabled** icon (see left) on both the *Transaction Table View* frame and the *nWave* frame to synchronize the views.

9. In the *Transaction Table View* frame, select the row containing Index 37. The waveform automatically updates and select the related transaction. You can also select a transaction in the waveform and the appropriate row is highlighted.

   You can continue sorting the current results by clicking another column header or you can further reduce the display by specifying another filter or you can restore the stream and start over. You can restore the stream and start over.

Generate Statistics

In addition to viewing and manipulating the transactions in a spreadsheet-like view, you can generate a variety of statistics for the stream.

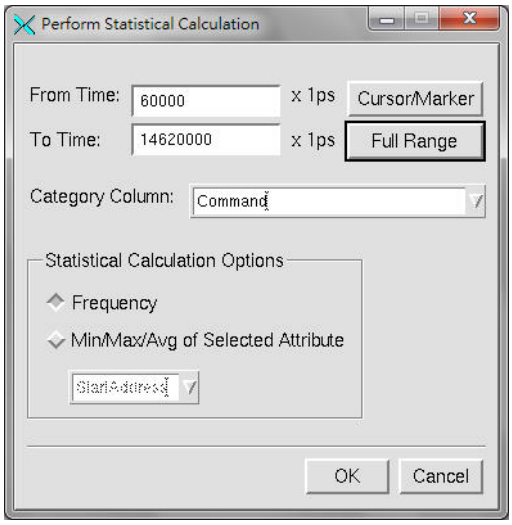
1. In the *Transaction Table View* frame, select the *AhbTransaction_AhbTransfer* merged stream.

   Note:
   Although this example uses the entire merged stream, you can filter the stream first and then generate statistics based on the reduced display.

2. Choose the **Tools -> Statistics Window** command to open the *Perform Statistical Calculation* form.

   The form is similar to the following:

Figure 408 Perform Statistical Calculation Form



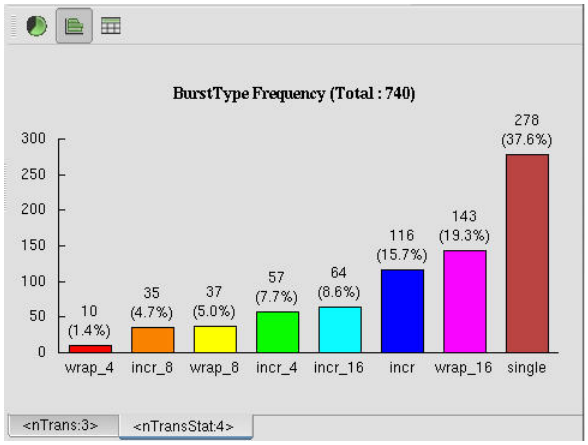
Feedback

You have several options for setting up the form. In this example you want to view the frequency of **BurstType** for the entire simulation range.

3. Click the **Full Range** button to automatically enter the from and to times.
4. Toggle the **Category Column** field and select **BurstType**.
5. Click **OK**.

   A *Statistics* frame similar to the following opens as a new tab in the same location as the *Transaction Table View* frame.

Figure 409 Bar Chart for BurstType



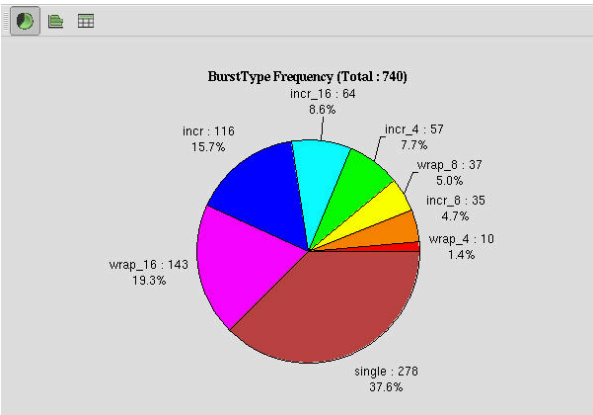
For the stream combination, you can easily see the frequency of the burst types. At this point, you can capture the results in PNG format. You can also change the view to a pie chart or table, or duplicate the window.

6. In the *Statistics* frame, choose the **View -> Pie Chart** command.

   The frame is updated similar to the following.

Feedback

Figure 410 Bar Chart for BurstType



7. Choose the **File -> Close** command to close the *Statistics* frame.

   You can generate more statistics for different attribute types.

8. In the *nTrace* main window, choose the **File -> Exit** command to close the Verdi session.

16

PPA Information in Verdi

This chapter describes the following topics:

- Loading Information in Verdi
- Instance View
- Canvas View

Loading Information in Verdi

To invoke PPA (Power, Performance, and Area) through Verdi, use the following command:

```
% verdi -apex -ppa
```

You can invoke PPA and load design using the following command where, `simv.daidir` is not a part of PPA:

```
%verdi -apex -ppa -dbdir <simv.daidir>
```

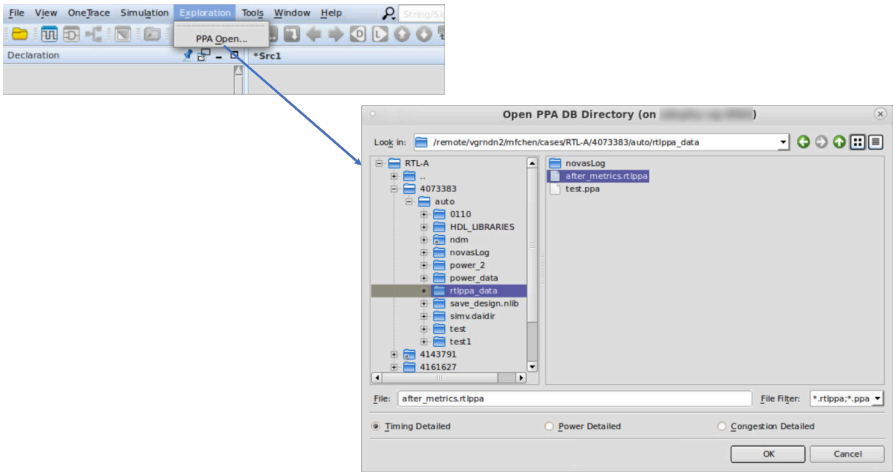
Alternatively, you can invoke Verdi and PPA Database (DB) from the command line. For example,

```
%verdi -apex -ppa <ppa>
```

Feedback

When the PPA is invoked, the **Exploration** menu gets added in the menu bar, as illustrated in the following figure:

Figure 411 Opening PPA From Verdi



1. In the **Exploration** menu, click **PPA Open**.

   This opens the **Open PPA DB Directory** dialog for selecting the PPA database and the data to display.

2. You can select the type of data you want to display. The supported types of data include *Timing Detailed*, *Power Detailed*, and *Congestion Detailed*. The selected data is displayed by default. When none of the data is available in the DB, it displays the following message:

When the PPA DB consists of the Power data and you select the **Timing Detailed** option, then the following message is displayed:

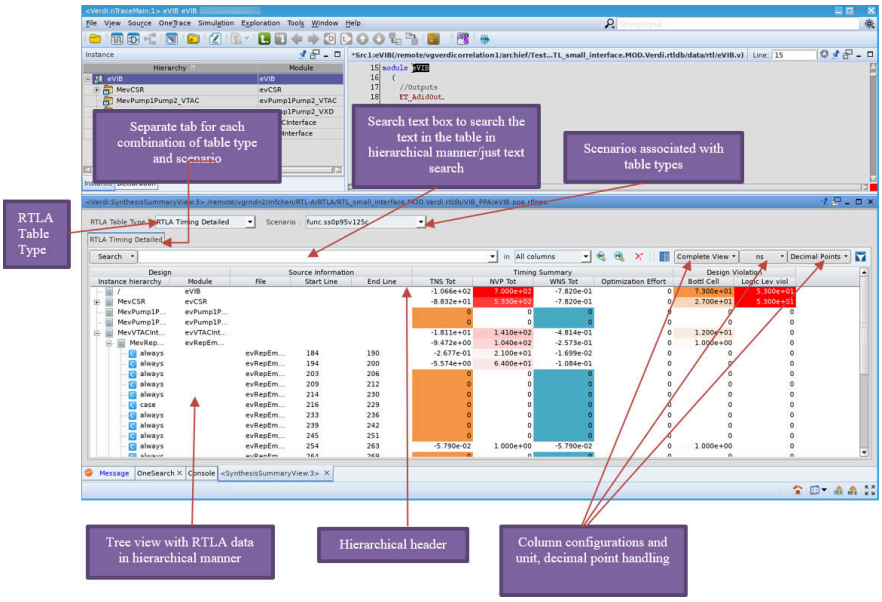
Feedback

Instance View

Instance View displays a high-level table based on sorting and filtering mechanism. When you invoke PPA from the command line or through Verdi, the **Synthesis Summary View** window is displayed by default.

The following figure provides an overview of the **Synthesis Summary** *View*:

Figure 412 Instance View



The Synthesis Summary View consists of the following major sections:

- Table Types
- Scenarios
- Search
- Column Configuration

Feedback

- Units
- Decimal Points
- Hierarchical Header
- Visualizations

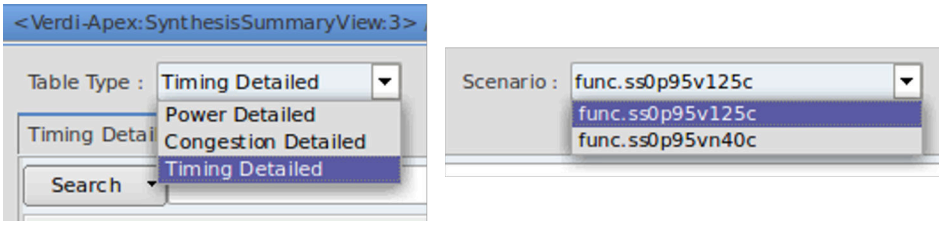
Selecting Table Types And Scenarios

In the **Table Type** field, you can select one of the following types of tables:

- Timing Detailed
- Power Detailed
- Congestion Detailed

After you select the Table Type, the corresponding scenarios are loaded in the **Scenario** box, as shown in the following figure:

Figure 413 Selecting Table Type And Scenarios



Based on your selection, the relevant data is fetched from the PPA database and a Tree view is generated under the selected table type tab.

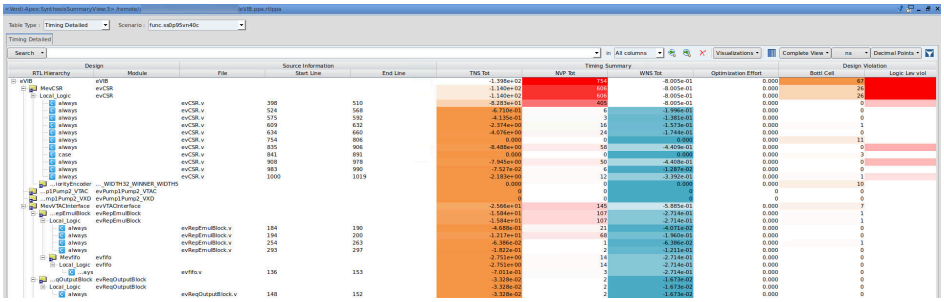
When you select a new combination of the **Table Type** and **Scenario**, it closes the current tab and creates a new tab. The new tab is created with data corresponding to the newly selected combination of the table type and scenario.

Tree View And Functions

Based on the selected **Table Type** and **Scenario**, Tree view is generated by fetching data from the PPA database.

Feedback

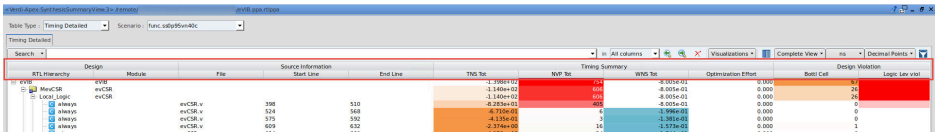
Figure 414 Generated Hierarchical Tree View for Selected Table Type And Scenario



Hierarchical Header

Tree view supports hierarchical header to provide visibility of different columns and display instances in the hierarchical order. You can expand and collapse the nodes.

Figure 415 Hierarchical Header



Columns

In the Tree view, under **Design** tab, **RTL Hierarchy** and **Module** columns are available by default, and power data can also be displayed.

Note:

The power data is displayed only when the **Power Detailed** table type is selected.

You can perform the following actions on each column:

- Configure color for each column. By default, random colors are displayed.
- Select header of the sub-column for sorting values.

Feedback

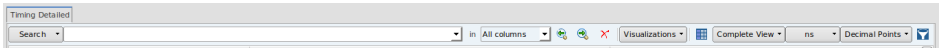
Figure 416 Color Shading And Sorting For a Column

Timing Summary		
TNS Tot	NVP Tot ^	WNS Tot
-1.066e+02	7.000e+02	-7.820e-01
-8.832e+01	5.330e+02	-7.820e-01
-1.811e+01	1.410e+02	-4.814e-01
-9.472e+00	1.040e+02	-2.573e-01
-5.574e+00	6.400e+01	-1.084e-01
-2.677e-01	2.100e+01	-1.699e-02
-3.195e+00	1.400e+01	-2.573e-01
-1.632e-01	2.000e+00	-1.147e-01
-5.790e-02	1.000e+00	-5.790e-02

Toolbar

You can perform various actions on the displayed data using multiple options provided in the toolbar.

Figure 417 View Toolbar



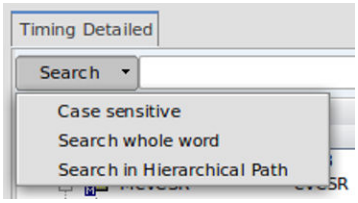
The functions that you can perform on the data displayed in the Tree view are provided in the subsequent sections.

Search

The **Search** text box can be used to search a particular word hierarchy in the Tree view. In the **Search** text box, type or select any one of the following options:

- Case-sensitive
- Search whole word
- Search in Hierarchical Path

Figure 418 Search Box Options

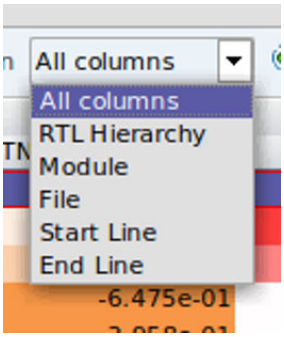


In the **Columns** box, you can select the column in which you want to search the word.

Feedback

Note:
Currently, only the following columns are supported: RTL Hierarchy, Module, and Both (RTL Hierarchy and Module).

Figure 419 Column Options



Next to **Columns**, the following buttons are available:

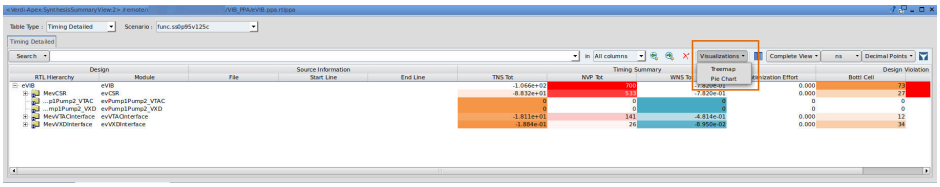
- Click to go to the previous match.
- Click to go to the next match.
- Click to clear the search word from the Search text box.

To start search, type the word and select an option for searching, press **Enter** or click to start the search.

Visualizations

The **Visualizations** button is available on the toolbar for each Table Type with the **Treemap** and **Pie Chart** options, as shown in the following figure:

Figure 420 Visualizations Button

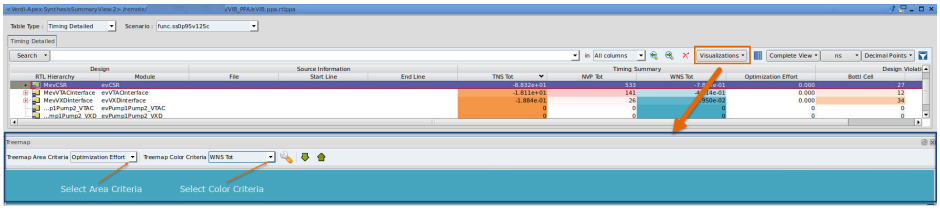


Feedback

Treemap

Choose **Visualizations > Treemap** to create a treemap for the total hierarchy using the immediate children and their corresponding values based on the selected table type. You can select the column and the color criteria associated with the column to be used to draw the treemap.

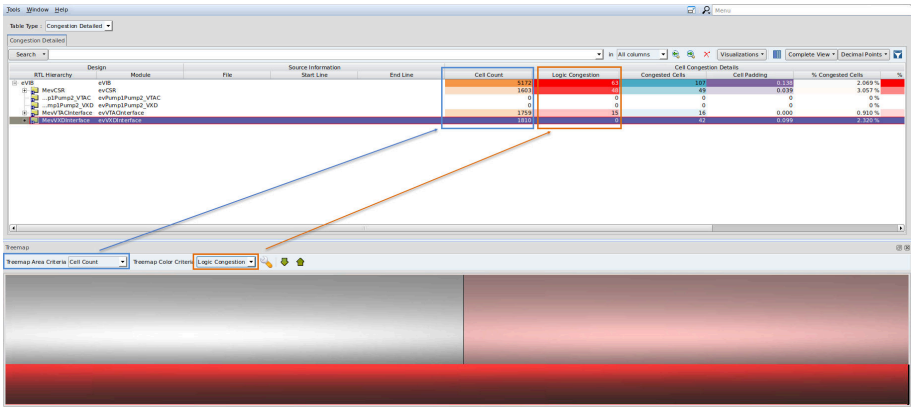
Figure 421 Invoking Treemap Section



Treemap Area And Color

Columns are used for area criteria and color criteria. You can either predefine the color criteria as shown in the following figure or can configure it by clicking the **Configuration** button :

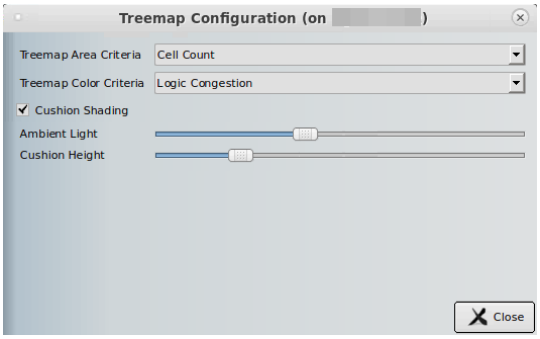
Figure 422 Treemap Area and Color Details



Feedback

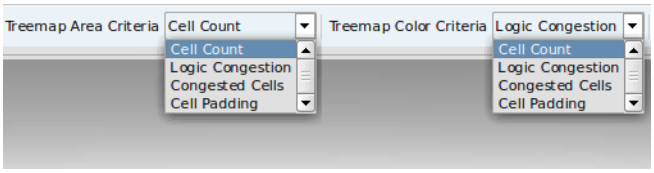
When you click , the following **Treemap Configuration** dialog box appears where you can configure the treemap area and color criteria and color shading:

Figure 423 Treemap Configuration



For example, for the Congestion Detailed Table Type, the following options for the area and color criteria appear:

Figure 424 Example for Congestion Detailed Table Type



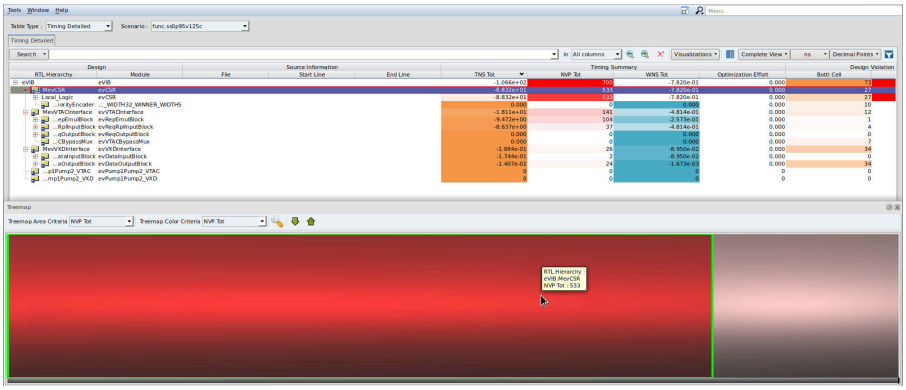
Treemap Tiles

The **Treemap** tiles are created using the values of the selected Treemap Area Criteria. The tile colors are aligned with the different shades of one color. Higher the value, brighter the color of the tile.

Feedback

The initial treemap tile contains the immediate children only. The selected children tile is highlighted in the treemap and values are also displayed as tooltips, as shown in the following figure:

Figure 425 Treemap Tiles



You can get more details about a particular children hierarchy in the following ways:

- Double-click the tile corresponding to the children hierarchy.
- Right-click the children hierarchy on the table, and click **Set as root**.
- Select the tile and click the **Dive in** button .

You can dive in until you reach the leaf hierarchy. When you want to dive out or go back to the previous hierarchy or to the default root hierarchy, you can use the **Restore previous root** and **Restore default root** options, respectively.

You can also use the **Escape** key or the **Dive out** button to navigate back to the previous level of hierarchy.

Pie Chart

Pie Chart compares values of the multiple columns. The columns you want to compare can be preconfigured using Pie Chart criteria.

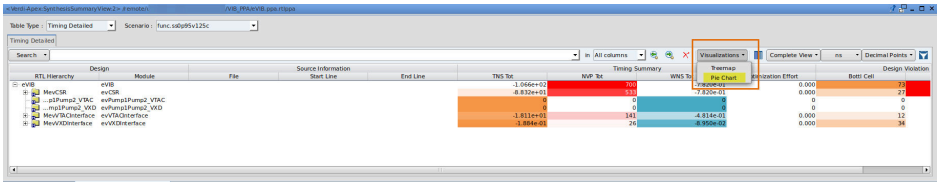
Based on the criteria, a set of combination boxes are provided and the pie chart is drawn for the columns which are configured for the selected criteria. You can select **Scope** in the table.

Invoking Pie Chart

Choose **Visualizations > Pie Chart** to invoke Pie Chart. Treemap and Pie Chart are the dockable widgets and can exist simultaneously.

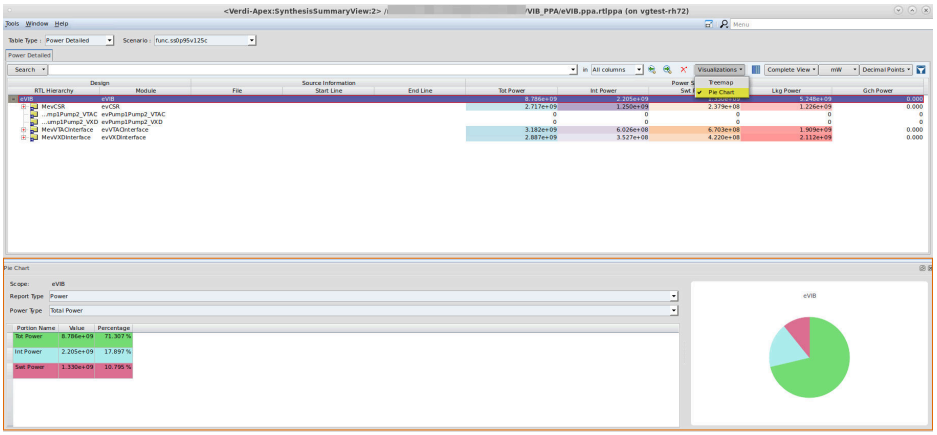
Feedback

Figure 426 Invoking Pie Chart



The following figure displays the Pie Chart:

Figure 427 Pie Chart



Pie Chart Criteria

Pie Chart consists of the following fields as criteria:

- **Report Type**: Report type displays the **Power** option.
- **Power Type**: Click the **Power type** box to see more options. It lists all the column names containing data values.

The following table lists a few examples of configurations:

Configuration	Columns to compare
Report Type: PowerPower Type: Total Power	Tot Power / Int Power / Swt Power
Report Type – PowerPower Type – Int/Swt Power	Int Power / Swt Power

Note:

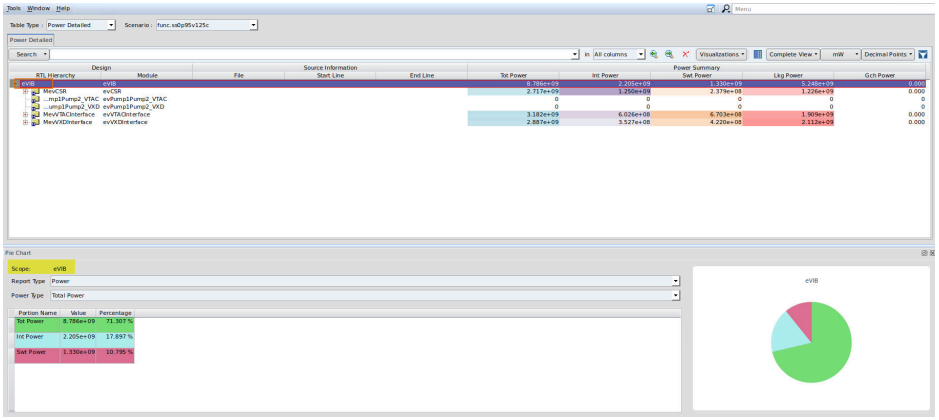
You can have any number of configurations. All the names, number of combination boxes, and columns are configurable.

Feedback

Pie Chart Display

Based on the selected configuration, the column values are displayed in the table and in the pie chart:

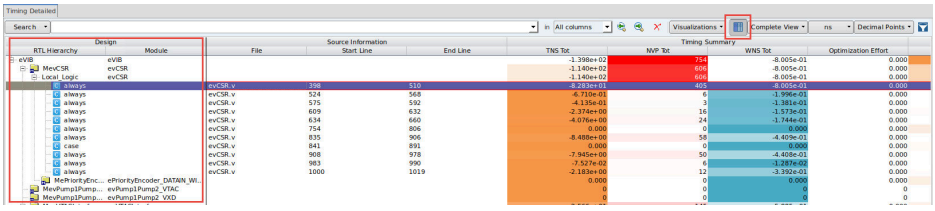
The following figure displays the **Scope** that you can change by selecting a hierarchy in the table view:



Freeze First Column

You can freeze or unfreeze your first column of the Tree view by clicking the **Freeze First Column** button . When you select freeze, the first column gets frozen as shown in the following figure:

Figure 428 Freeze Operation: The First Column is Frozen

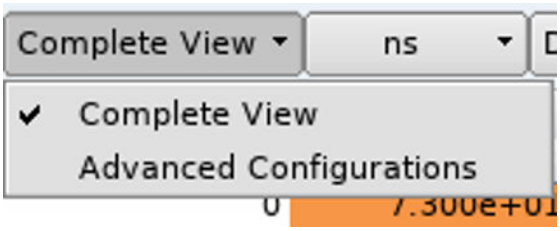


Column Configuration

You can change the configuration of the displayed columns using the **Column Configuration** option. By default, it is set to **Complete View** which displays all the columns.

Feedback

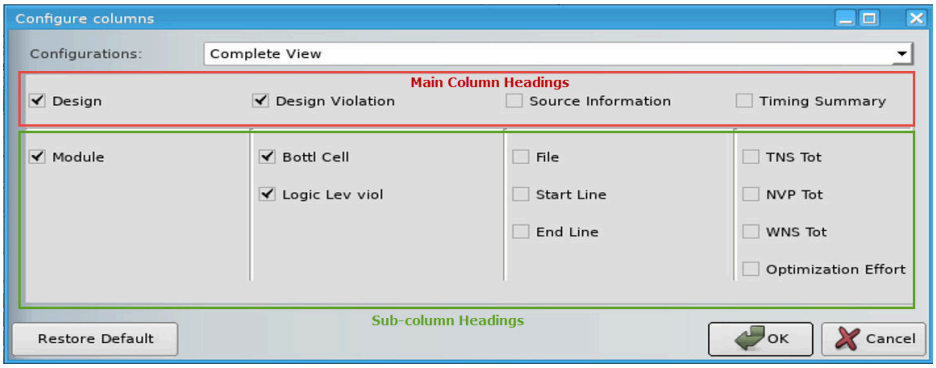
Figure 429 Column Configuration



You can click the **Complete View** arrow to see more options. When you click **Advanced Configurations**, the **Configure Columns** dialog box appears.

The **Configure Columns** dialog box is shown in the following figure:

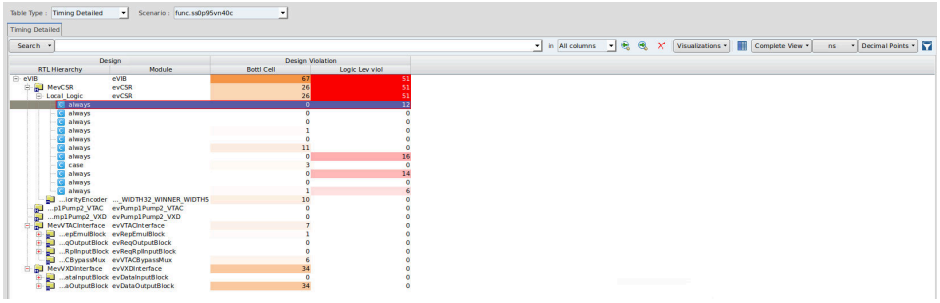
Figure 430 Configure Columns



You can select or clear the column check boxes that you want to add or remove from the Tree view, respectively.

For example, in the following figure, only **Design** and **Design Validation** columns are selected:

Figure 431 Configured Columns Example



Feedback

Note:

The column configuration is saved for each tab, that is, for the selected **Table Type** and **Scenario**.

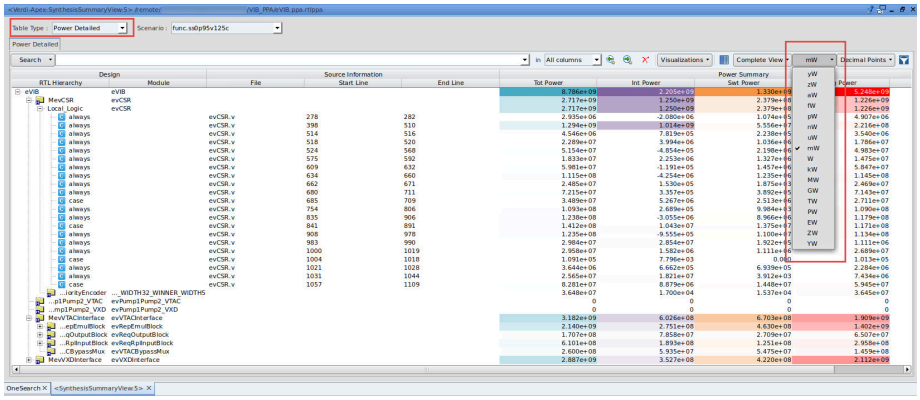
Feedback

Units

You can find different units for different table type. The units that are displayed for the respective table types are as follows:

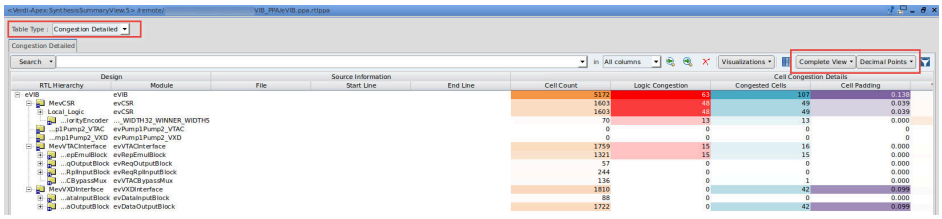
- **Power Detailed**: When you select Power Detailed table type, **Power** units are displayed. The default unit for Power is **mW**. You can select a unit by clicking the **Unit** list. The following figure displays the power units:

Figure 432 Units for Power Detailed Table



- **Congestion Detailed**: For Congestion Detailed table type, units are not displayed as shown in the following figure:

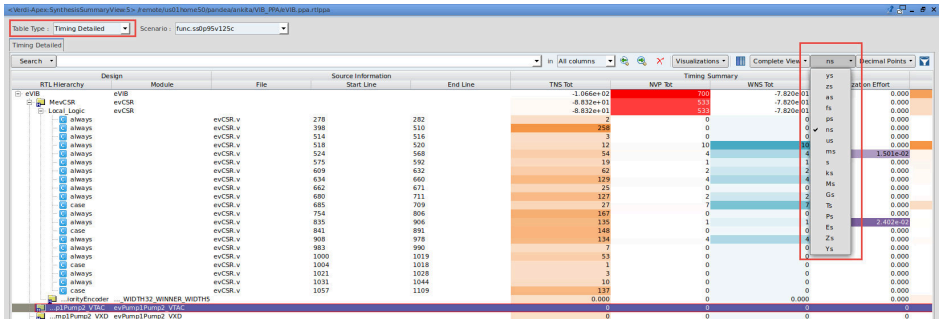
Figure 433 No Units for the Congestion Table



Feedback

- **Timing Detailed**: When you select Timing Detailed table type, **Timing** units are displayed. The default unit for Time is **ns**. You can change the unit by clicking the **Unit** list. The following figure displays Timing units:

Figure 434 Units for Timing Detailed Table

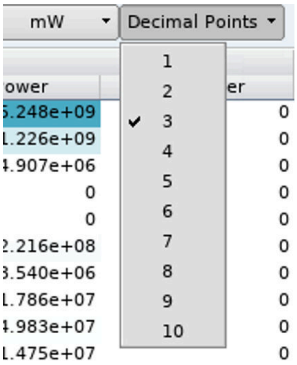


Decimal Points

You can select the number of decimal places you want to display for your values in the Tree view. In the **Decimal Points** list, select the decimal point.

Based on the selected decimal point, the Tree view gets updated. By default, it is set to 3 decimal points. The following figure displays the **Decimal Points** list:

Figure 435 Selecting Decimal Points

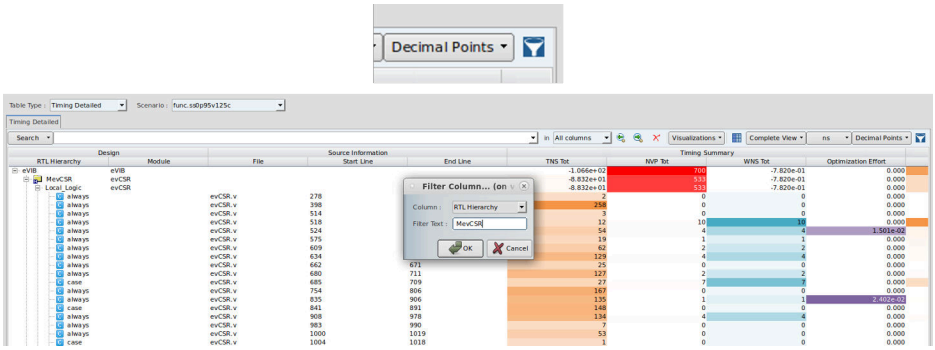


Filter Columns

The **Filter Columns** option is available next to **Decimal Points**. You can click the **Filter Columns** button to open the **Filter Column..** dialog box. Select the **Column**, and in the **Filter Text** field, type the text you want to filter in the selected column.

Feedback

Figure 436 Filtering Rows by Case Keyword

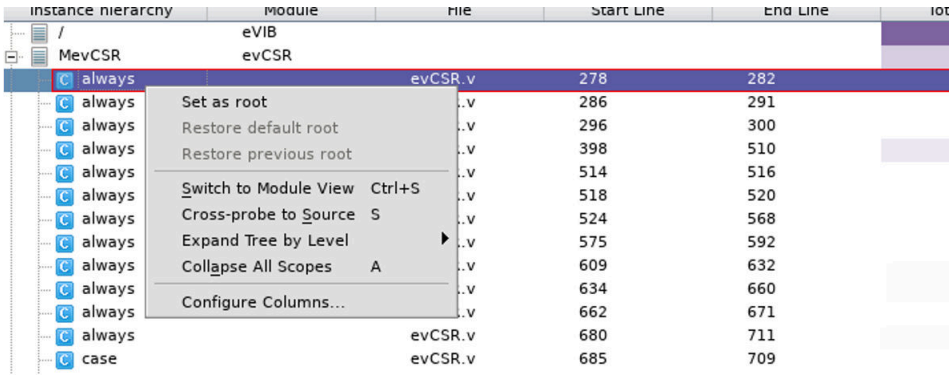


Note:

Currently only **RTL Hierarchy** column is supported.

Tree view supports right-click menu as shown the following figure:

Figure 437 Tree View Right Click Menu



The following main options are available in the right-click menu:

- Switching to Module/Instance View
- Cross-Probe to Source
- Expanding/Collapsing Tree by Level
- Configuring Root
- Configuring Columns
- Configuring Color Scheme

Feedback

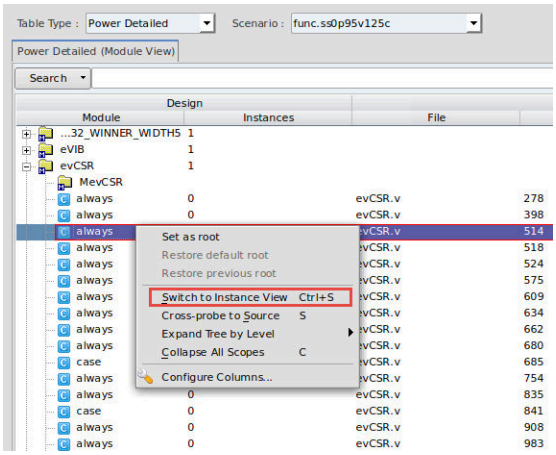
Switching to Module/Instance View

Right-click an instance and click **Switch to Module View** to navigate to the module view of the selected table type and scenario.

The Module view displays only module information. If the module information is not available, “No data” message appears and takes you to the previous view.

The following figure displays the Module view with **Switch to Instance View** option:

Figure 438 Module View With Switch to Instance View Menu Option



When you are in Module view, the **Switch to Module View** option gets changed to **Switch to Instance View**. The keyboard shortcut key, **Ctrl+S**, can also be used to switch between the module view and instance view.

Note:

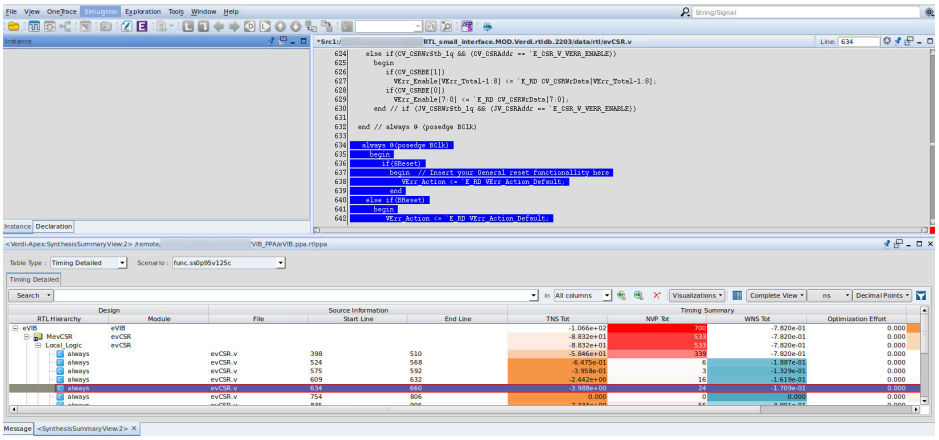
Module view is a separate view, and any setting applied to the module is kept separately. Module view supports same functions as Instance view.

Feedback

Cross-Probe to Source

You can navigate to the source of the selected instance or construct listed in the Tree view. When you select **Cross-probe to Source**, you are navigated to the corresponding place in the Verdi RTL view as shown in the following figure:

Figure 439 Navigating to the Corresponding Place in the Verdi RTL View



You can also use the keyboard shortcut key, **S**, to select **Cross-probe to Source** option.

Tree view supports **Drag and Drop** feature that allows you to drag and drop instance items between Tree view and Verdi windows. For example,

- Drag from the Tree view and drop to Verdi RTL navigation or Instance view.
- Drag from the Verdi Instance view and drop to the Tree view.

In both the scenarios, you can navigate to the corresponding place for the dropped item. This is similar to Cross-probe to Source feature.

Note:
Both, instance and construct, are supported for the **Cross-probe to Source** feature.

Expanding/Collapsing Tree by Level

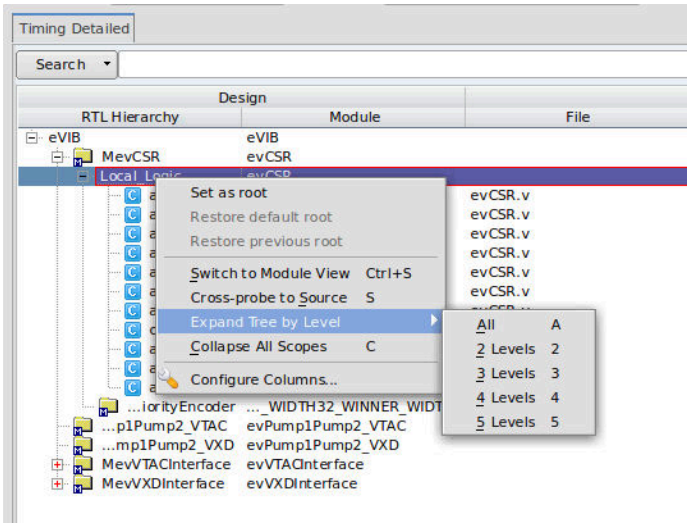
You can expand or collapse all tree levels by right clicking an item listed in the Tree view and clicking **Expand Tree by Level** or **Collapse All Scopes**.

The **Expand Tree by Level** option supports levels. You can select the level for expanding the tree.

Feedback

The following figures displays both the options:

Figure 440 Expand Tree by Level and Collapse All



Configuring Columns

When you select Configure Columns, it opens the **Configure Columns** dialog box. For details on Configure Columns, refer to the Column Configuration section.

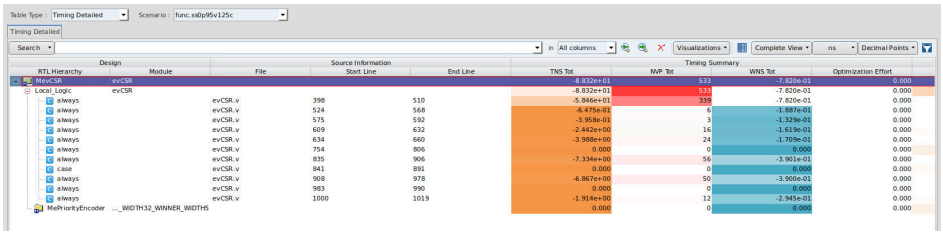
Configuring Root

You can set any instance or construct in the RTL hierarchy as root by selecting the **Set Root** option. This helps in analyzing a complex hierarchy.

The **Restore Default Root** option restores the Tree view to the original state.

The **Restore Previous Root** option sets the previous root as root.

Figure 441 Setting a Sub-Item in the Hierarchy as Root

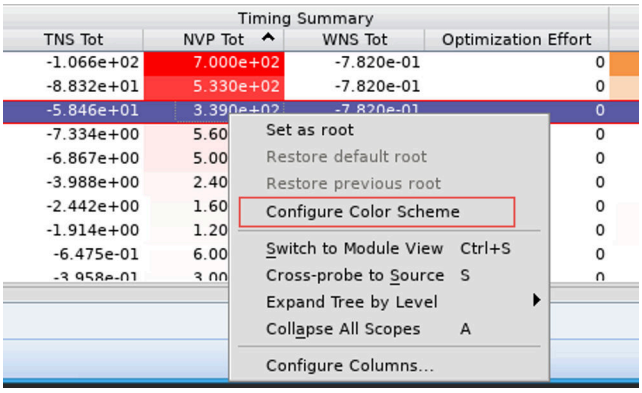


Feedback

Configuring Color Scheme

You can configure different color schemes by right clicking any instance in the Tree view and selecting **Configure Color Scheme**, as shown in the following figure:

Figure 442 Configure Color Scheme Only Available for Columns With Data

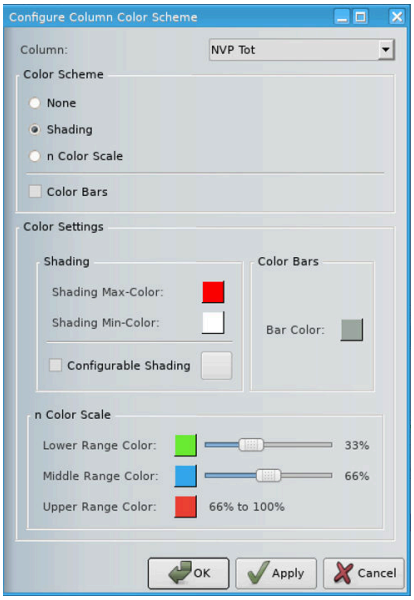


Note:

The **Configure Color Scheme** option is available only for columns that have data, and is not available for the RTL Hierarchy and Module columns.

When you click **Configure Color Scheme**, the following **Configure Column Color Scheme** dialog box appears:

Figure 443 Color Configuration Dialog

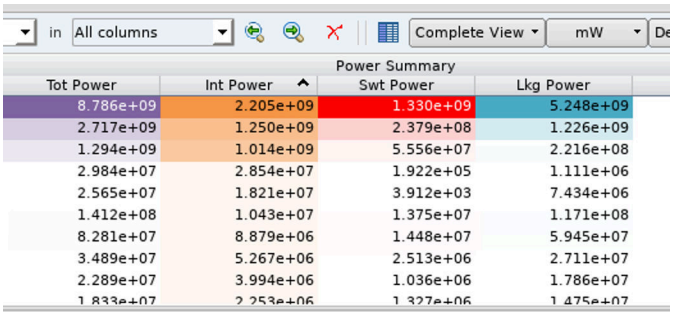


Feedback

In the **Configure Column Color Scheme** dialog box, the following options are available:

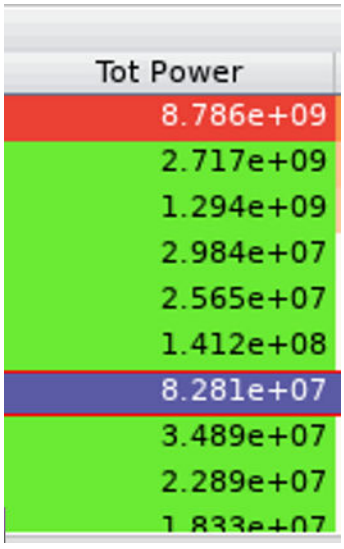
- **Column**: Select the column for which you want to configure color.
- **Color Scheme**: You can select one of the following options:
  - **None**: Color is not applied.
  - **Shading**: Apply shading of the selected color. You can select the color in the **Color Settings > Shading** section.

Figure 444 Shading Applied as Color Scheme



- **nColor Scale**: You can choose two or three color scales that can be applied based on the values in the column. The colors can be configured for the *Lower Range*, *Middle Range*, and *Upper Range* values. Two or three colors can be applied for each row in the column.

Figure 445 Two Color Scale Applied for a Colum

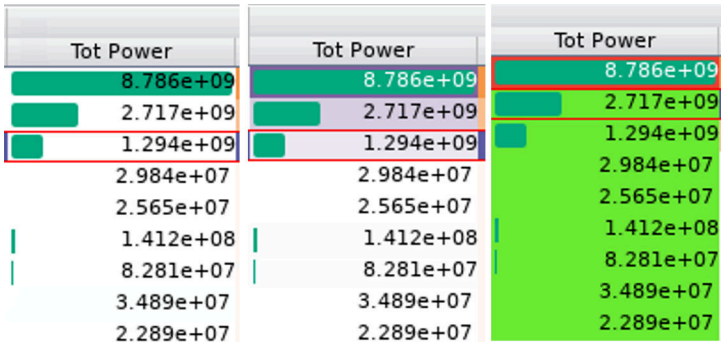


Feedback

- **Color Bars**: You can select the color for the color bars which can be merged with one of the three **Color Scheme** options. Additional color bars are drawn in each cell with respect to the value in the cell.

  When you specify **None** as the color scheme, only the color bars are drawn, else color bars are drawn as per **Shading** or **nColor scale**. This is shown in the following figure:

Figure 446 Only Color Bars/Color Bars With Shading/Color Bars With n Color Scale



Canvas View

The canvas view support enables visualizing, analyzing, and querying RTL objects in the canvas. This section describes how to use the canvas view and the user interface of the canvas view.

The canvas view support section consists of the following topics:

- User Interface
- Canvas View Window

User Interface

You can open the canvas view in the following three ways:

Tcl Command

You can enable the canvas view and open its user interface using the following Tcl command:

```
schCreateCanvasWindow
```

You must specify the `canvasType` and `scope` arguments in the following command:

```
schCreateCanvasWindow -canvasType $type -valueType $valueType -scenario
$scenarioName -scope $scopeName
```

Feedback

The command consists of the following arguments:

- `-canvasType`: Specify it as congestion or power.
- `-valueType`: The `valueType` argument depends on the `canvasType`. If the `valueType` argument is not specified, by default `metrics_cong_num_in_hot_spot` is set for congestion and `total_power` is set for power.

  For congestion, you can specify the following arguments:

  - `metrics_cong_num_in_hot_spot(default)`,
  - `metrics_cong_cell_padding`,
  - `metrics_cong_logic_cong`,
  - `metrics_cong_overflow`

  For power, you can specify the following arguments:

  - `total_power(default)`,
  - `internal_power`,
  - `switching_power`,
  - `leakage_power`,
  - `glitch_power`
- `-scenario`: You can use this argument only when the `canvasType` is Power. If it is not specified, it uses the first scenario as the default value.
- `-scope`: Specify the scope module that you want to open.

For example,

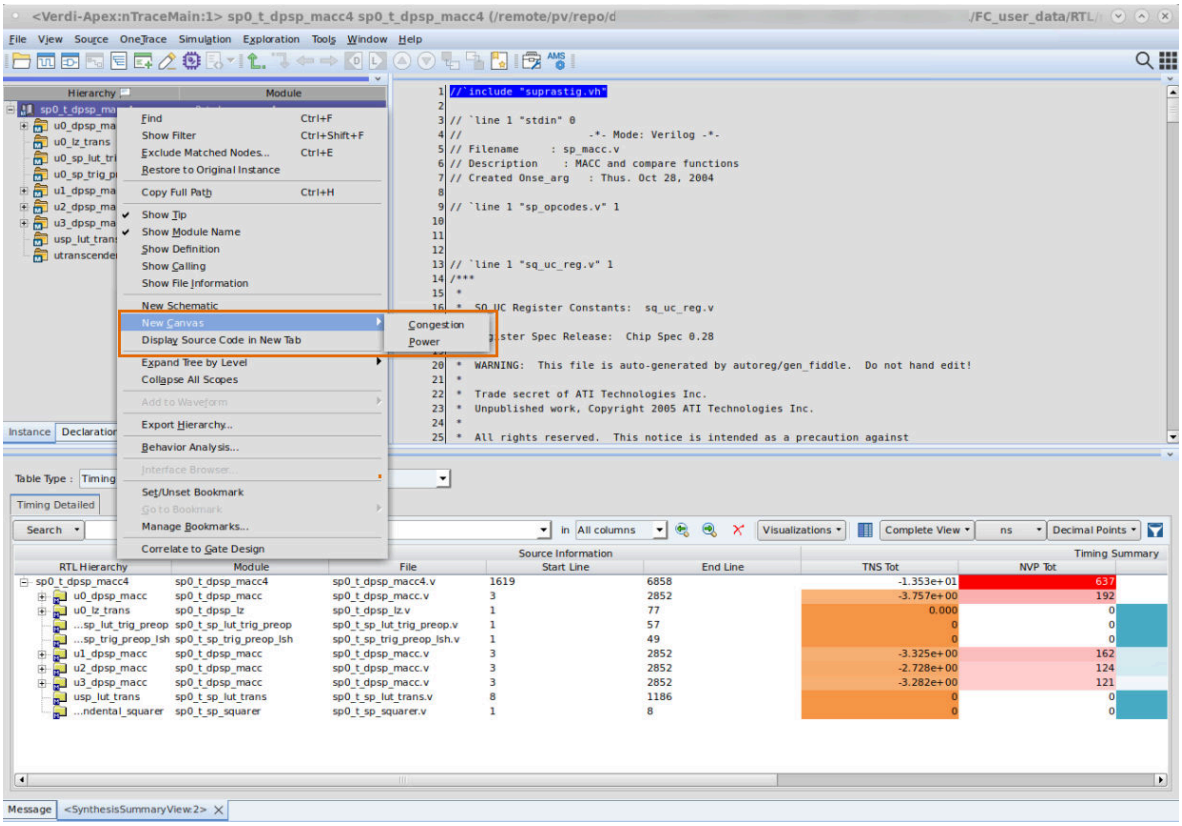
```
schCreateCanvasWindow -canvasType power -scope top.cpu -valueType
 total_power -scenario func.ss0p95v125c
schCreateCanvasWindow -canvasType congestion -scope top.cpu -valueType
```

Feedback

nTrace Hierarchy

The **New Canvas** feature is available only when the PPA Database is imported. In the *Instance* tab, right-click an instance, click **New Canvas**, and select the canvas type, as shown in the following figure:

Figure 447 nTrace Hierarchy Entry

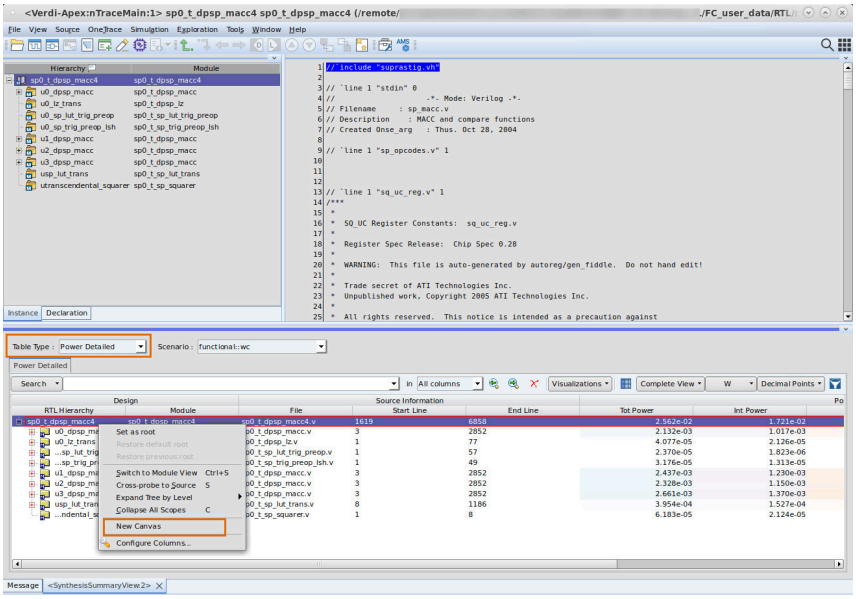


Feedback

Synthesis Summary View

In the Synthesis Summary View window, right-click a hierarchy scope and click **New Canvas**, as shown in the following figure:

Figure 448 Synthesis Summary View



The same canvas type is selected as the table type.

The **New Canvas** option is enabled only when either the **Power Detailed** or **Congestion Detailed** table type is selected.

Feedback

Canvas View Window

The following are the Canvas View Windows with and without hard macros:

Figure 449 Canvas View Without Hard Macros

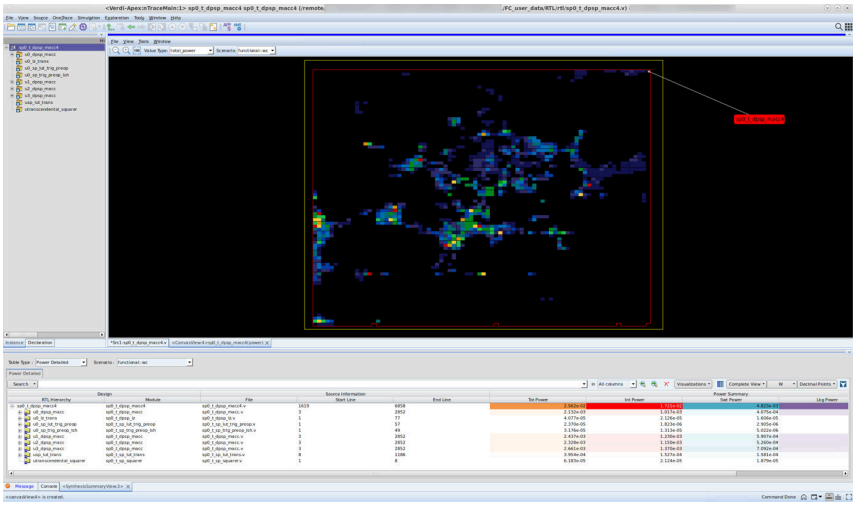
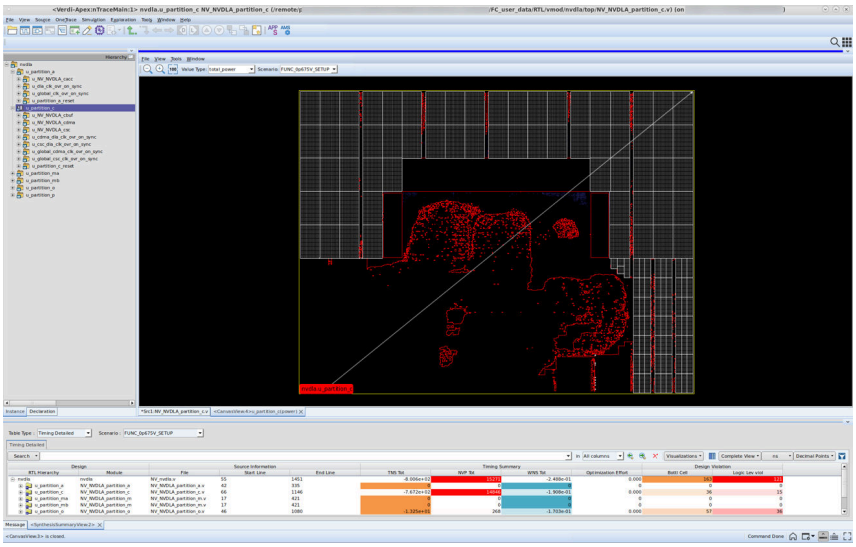


Figure 450 Canvas View With Hard Macros

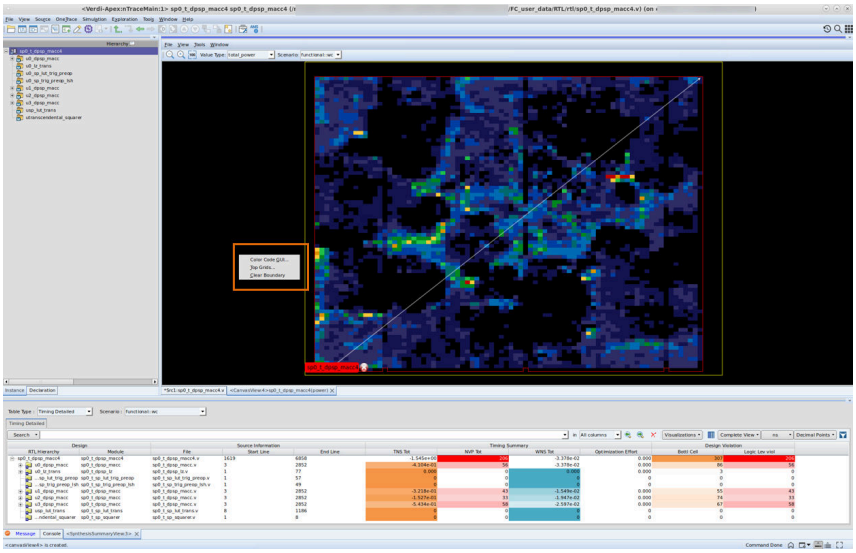


Feedback

Color Code GUI

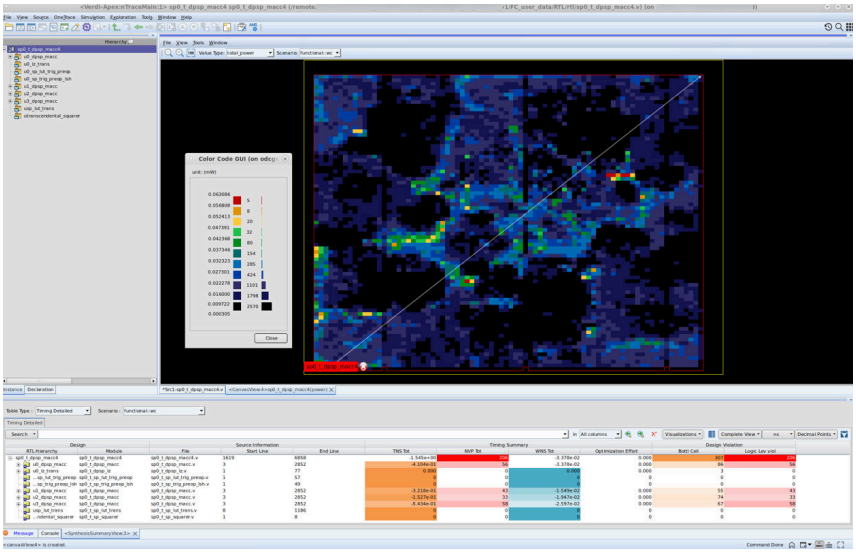
When you right-click anywhere on the **Canvas View** window, the **Color Code GUI** option is displayed:

Figure 451 Right-Click Options for Canvas View



When you click the **Color Code GUI** option, the **Color Code GUI** dialog box appears, as shown in the following figure:

Figure 452 Color Code GUI Option



Feedback

The **Color Code GUI** dialog box consists of the following four columns:

1. The first column displays the divide value.
2. The second column displays the default color.
3. The third column displays the counts per divide.
4. The forth column displays the status per divide.

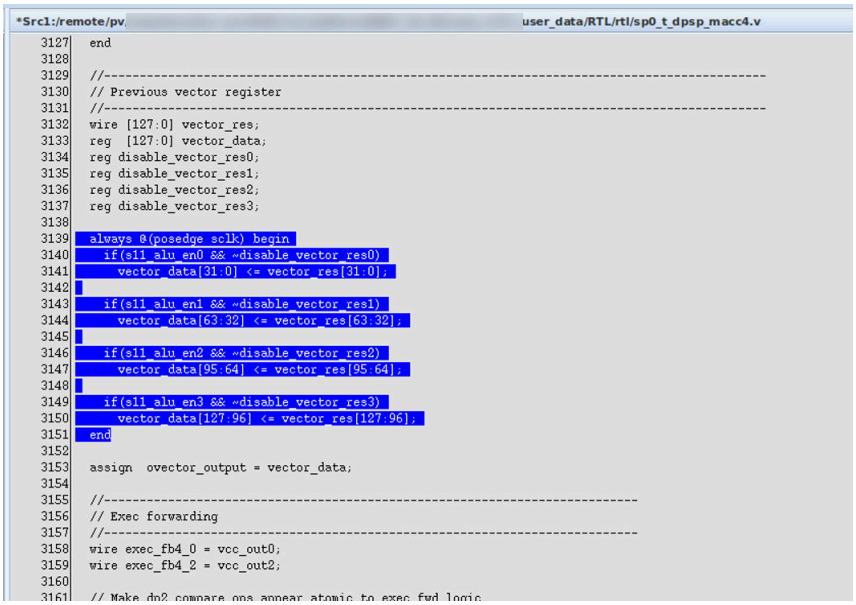
Note:

More the count per divide, longer the status bar.

Double-Click

The grid is only a source of information. When you double-click a grid, its source code is highlighted in the *Source Code* frame, as shown in the following figure:

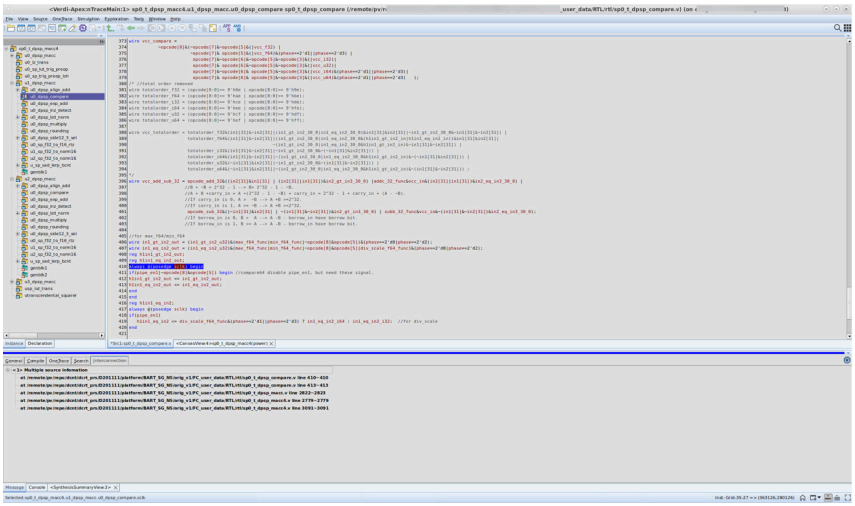
Figure 453 Highlighted Source Code



Feedback

If there are multiple source codes, then the file name and the line number are displayed in the *Interconnection* tab of the *Message* frame, as shown in the following figure:

Figure 454 Multiple Source Code in Interconnection



When you double-click the listed file name or the line number, the respective source code is highlighted in the *Source Code* frame.

Drag And Drop

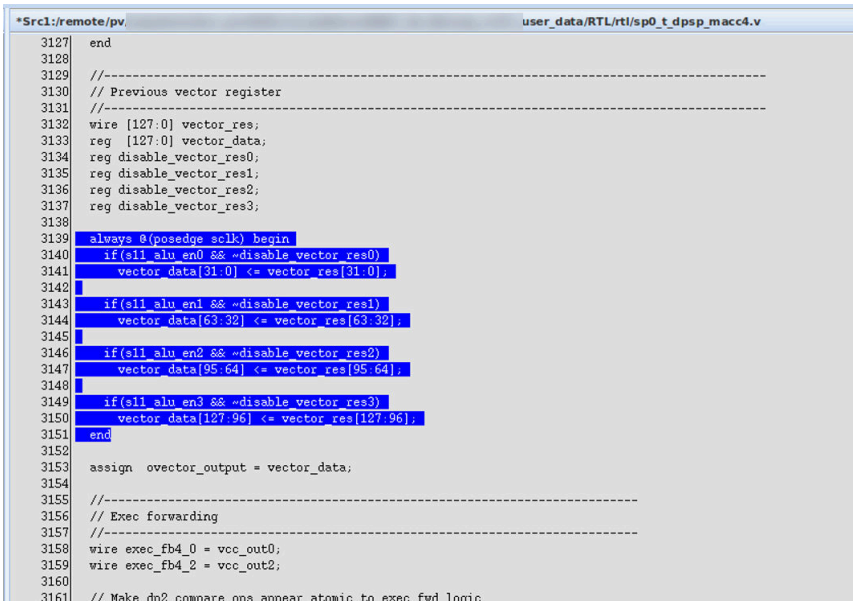
You can drag and drop your data into different windows and view them in different ways.

Feedback

Drag and Drop Canvas View to Source Code Frame

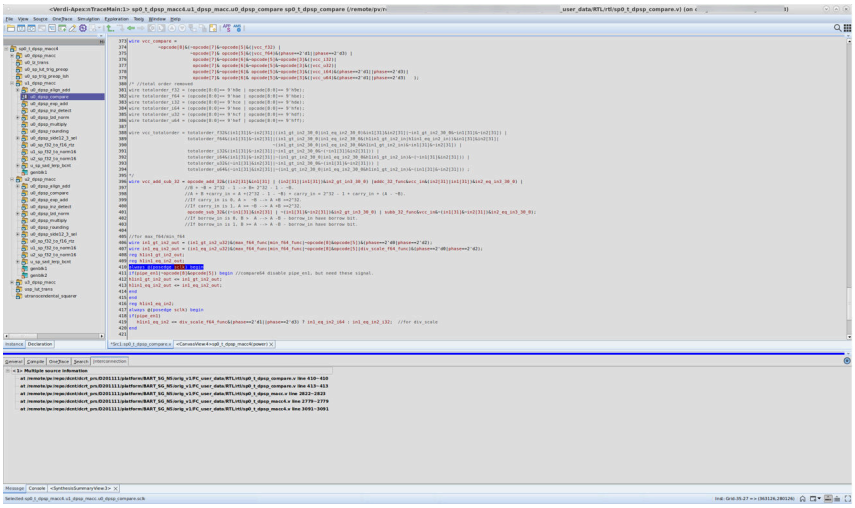
You can drag a grid in the **Canvas View** window and drop it to the *Source Code* frame. The source code gets highlighted as shown in the following figure:

Figure 455 Highlighted Source Code (Drag and Drop)



If there are multiple source codes, the first source code is highlighted and displayed and the file name and line number are displayed in the *Interconnection* tab of the *Message* frame, as shown in the following figure:

Figure 456 Multiple Source Code in Interconnection (Drag and Drop)



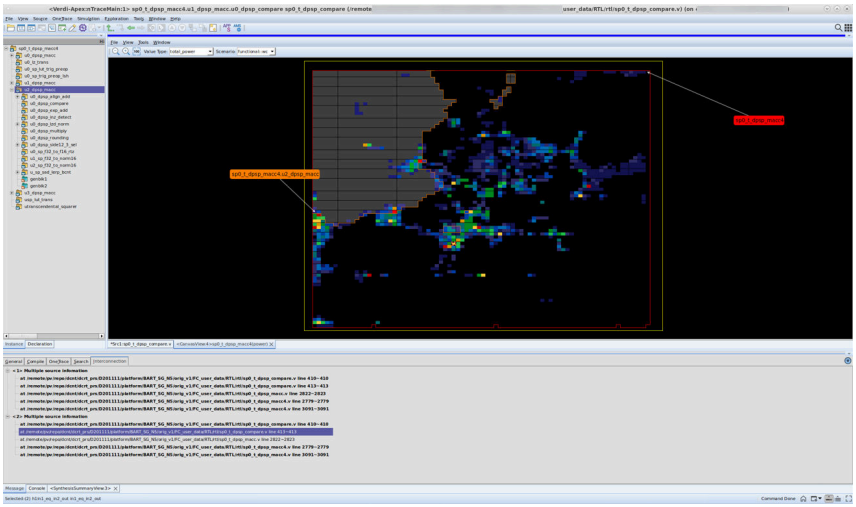
Feedback

When you double-click the items listed in the *Interconnection* tab, the corresponding source code range is highlighted in the *Source Code* frame.

Drag And Drop From nTrace Hierarchy to Canvas View

When you select a parent hierarchy, drag and drop it to the **Canvas View** window, the following view with the scope boundary and locator is displayed:

Figure 457 Scope Boundary And Locator



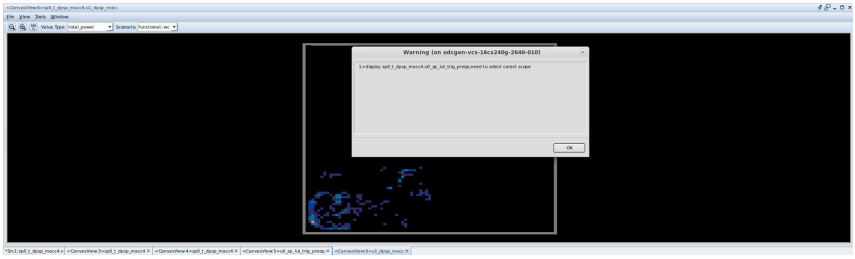
If the selected hierarchy falls under the canvas scope, the grids inside the top hierarchy include its children. When you drag and drop a child hierarchy from the *Instance* tab to the **Canvas View**,

- the grids are highlighted in a specific color.
- a scope boundary is drawn around the highlighted grids.
- a location pointer to the boundary with the same boundary color is available. You can scroll seven colors for different scopes.

Feedback

However, if you select a hierarchy outside the canvas scope, the warning message "`display $scope, need to select correct scope`" appears as shown in the following figure:

Figure 458 Hierarchy Outside the Canvas Scope

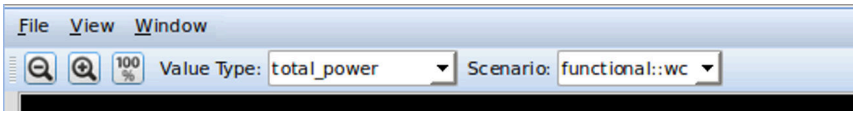


Switch Value Type and Scenario

When the `canvasType` is set as power, the following two fields are available on the **Canvas View** window:

1. Value Type
2. Scenario

Figure 459 Switch Value Type And Scenario



Note:

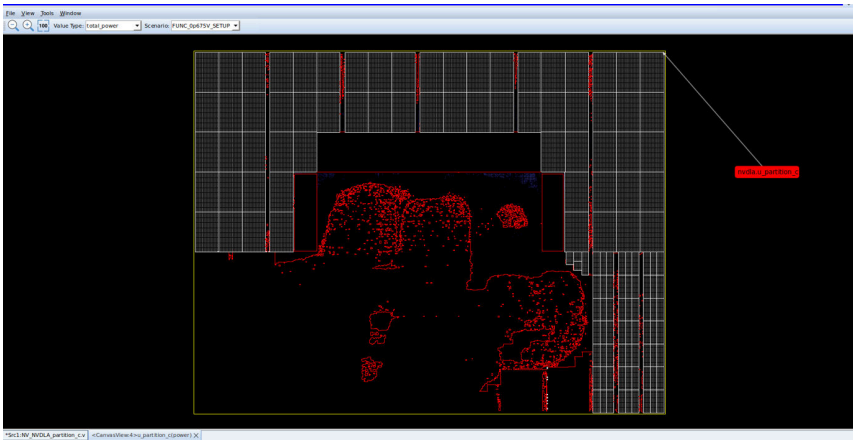
When the `canvasType` is set as congestion, only **Value Type** field is available on the **Canvas View** window.

Feedback

Display Hard Macro

When the `gridElmt` is set as Hard Macro, the Canvas View is filled with dots and the boundary is highlighted with a color as shown in the following figure:

Figure 460 Hard Macro Display



The highlighted boundary and the name of the Hard Macro are displayed on the Canvas View.

When you click a grid inside a hard macro, the name and the tips of the hard macro name are displayed.

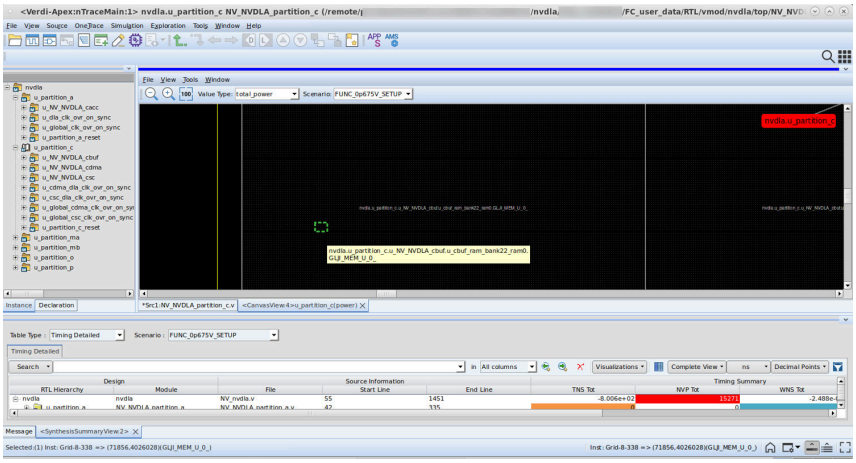
When you select a grid, the left or right corner displays the information about the grid position in the following format:

`Grid-(x,y)=>(ux,uy)(hard macro short name)` where,

- `x` and `y` are the canvas view coordinates.
- `ux` and `uy` are the design coordinates.

Feedback

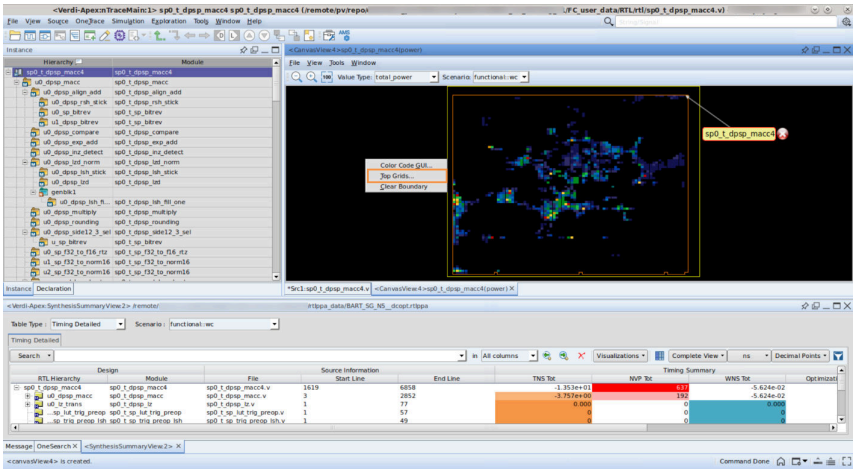
Figure 461 Hard Macro Name And Tips



Top Grids

The **Top Grids** option allows you to display the list of top grids. Right-click anywhere on the Canvas View window, and click **Top Grids** option, as shown in the following figure:

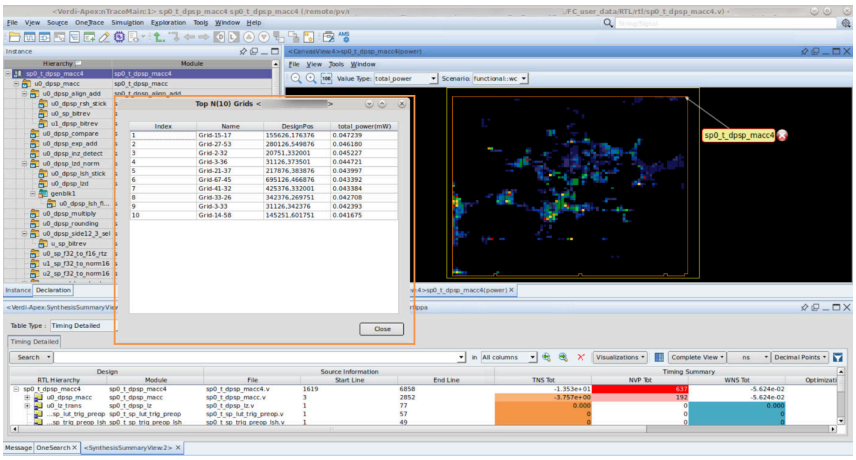
Figure 462 Top Grids Entry



Feedback

The following **Top N Grids** dialog box appears:

Figure 463 Top N Grid Cells

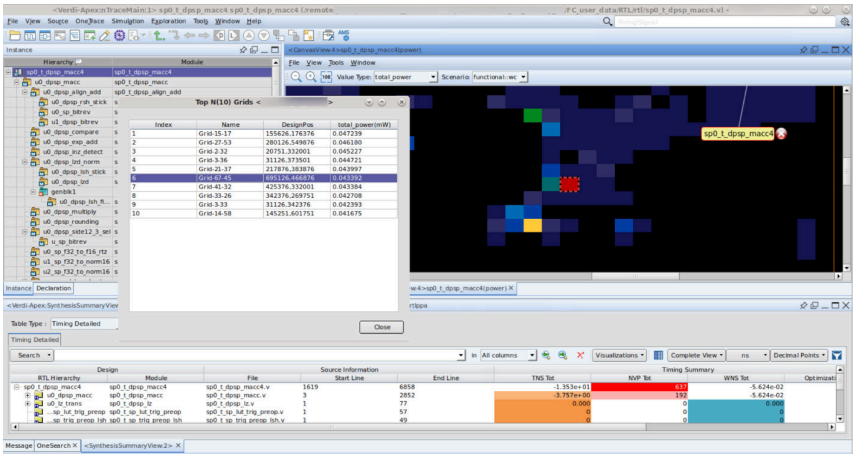


The **Top N Grids** dialog box consists of the following four columns:

1. Index
2. Name
3. DesignPos
4. total_power(mW)

When you select an item in the dialog box, the Canvas View window zooms in to the selected grid, as shown in the following figure:

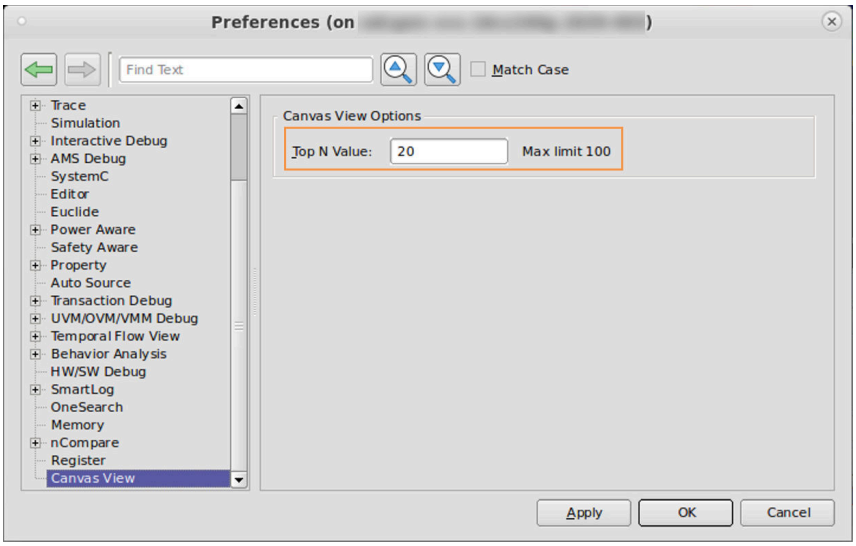
Figure 464 Select Top N Item



Feedback

You can use the **Tool > Preferences** command to modify the list of the top N value. Set the number of Top N values you want to display in the **Top N Grids** dialog box in the following **Preferences** dialog box:

Figure 465 Preferences Dialog Box



Note:

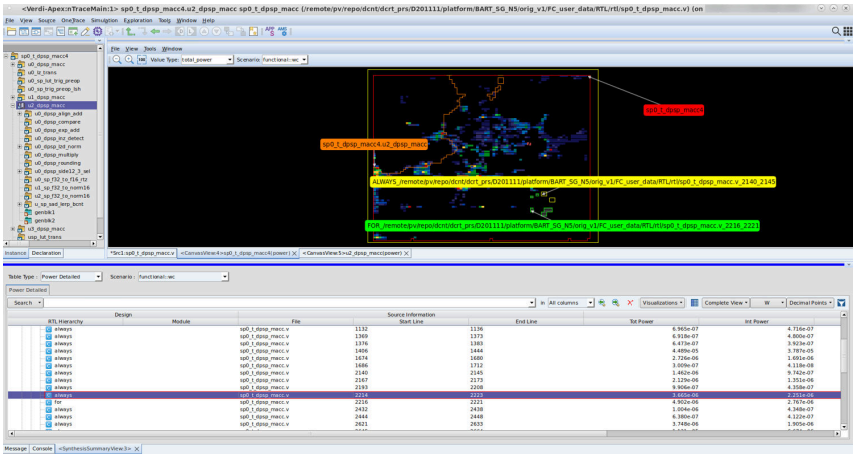
- The maximum limit for the **Top N Value** field is 100.
- The Canvas View is displayed only when the PPA database is loaded and the modified value affects the Canvas View opened after it.

Feedback

Construct Boundary

When you drag and drop the construction from the **Synthesis Summary View** table to the **Canvas View** window, the construction is highlighted, and the boundary and locator points to the boundary are displayed, as shown in the following figure:

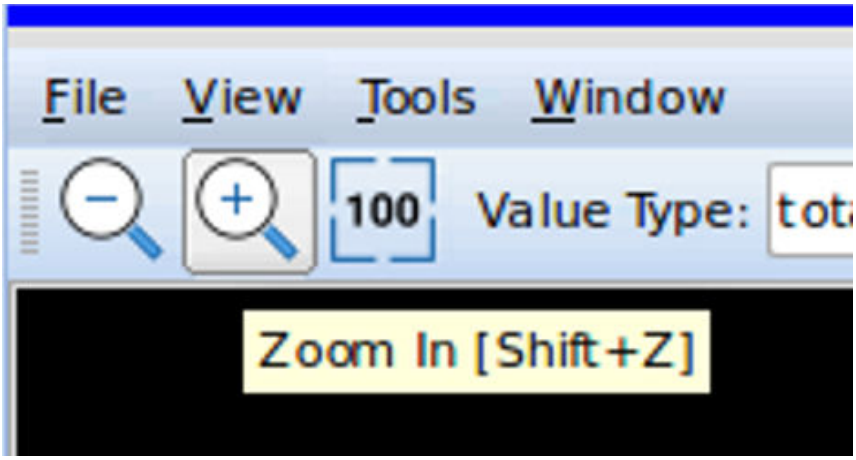
Figure 466 Construct Boundary



The **Canvas View** window consists of the following features:

- **Zoom In**: The keyboard shortcut for zoom-in is **Shift + Z**.

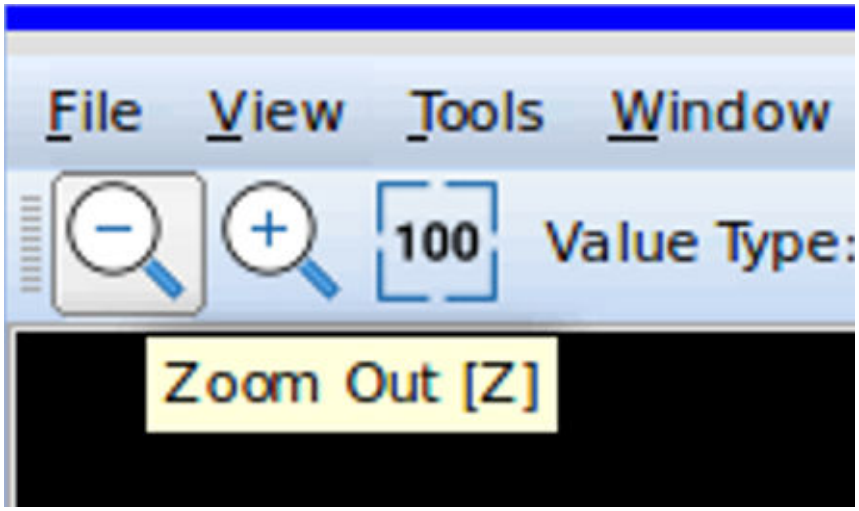
Figure 467 Zoom-in



- **Zoom-out**: The keyboard shortcut for zoom-out is **Z**.

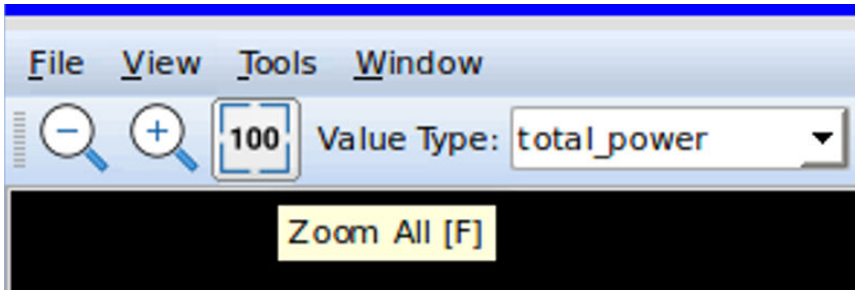
Feedback

Figure 468 Zoom-out



- **Zoom-100**: The keyboard shortcut for zoom 100 is **F**.

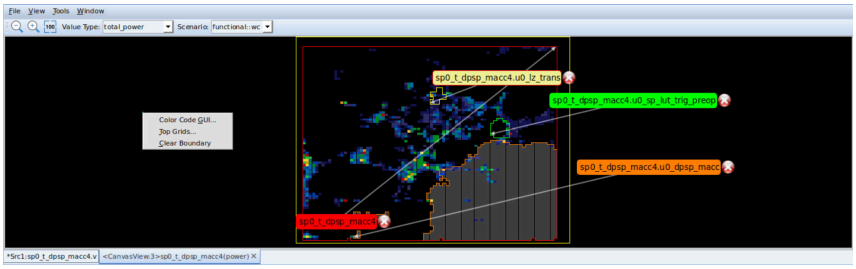
Figure 469 Zoom-100



Clear Boundary

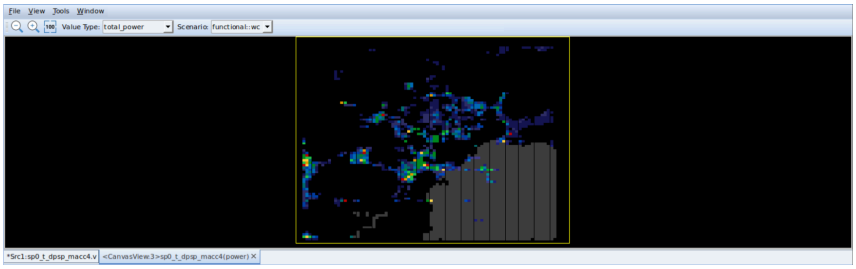
To clear all the boundaries and locators, right-click anywhere on the Canvas View window, and click **Clear Boundary**. The following figures display the Canvas View window before and after clicking **Clear Boundary**:

Figure 470 Before Clear Boundary



Feedback

Figure 471 After Clear Boundary



17

Simprofile

The Simprofile feature in Verdi is used for profiling simulation performance. It allows you to measure where the program spent its runtime and provides insights into the functions called during execution. This profiling data is generated by VCS commands and stored in a `simprofile` database. The **profileView** window in Verdi shows the CPU time of the module, instance, and assertion.

This chapter contains the following sections:

- Loading the Simprofile Database (W-2024.09 and Earlier Releases)
- Loading the Simprofile Database (X-2025.06 Release Onward)
- Opening profileView
- profileView Window Panes
- Menus and Commands of the profileView Window
- Source View Indicator
- SVA Distribution Mode
- Merged Simprofile Report
- DUT/TB Proportion Display
- Limitations

Feedback

Loading the Simprofile Database (W-2024.09 and Earlier Releases)

The **profileView** window uses the `simprofile` database generated by the VCS command line. To load the database from the VCS command line, perform the following steps:

1. Run the following compile-time option in VCS:

   ```
   %vcs -simprofile
   ```

2. Run the following runtime option in VCS:

   ```
   %simv -simprofile time+json_dbg_field
   ```

3. Set the following environment variable:

   ```
   setenv VERDI_SIMPROFILE_VIEW 1
   ```

4. Launch Verdi using the following command:

   ```
   %verdi -apex -lca -profilePath <profile_database_path> -profileFilter
   <percentage>
   ```

   Where,

   - `-profilePath <profile_database_path>`: Specifies the path of the profile database.
   - `-profileFilter <percentage>`: (Optional) Specifies the threshold percentage of CPU time. The default value is 0.5 percent.
   - `-lca`: Enables the LCA feature.
   - `-apex`: Opens Verdi in Apex mode.

Loading the Simprofile Database (X-2025.06 Release Onward)

The **profileView** window uses the `simprofile` database generated by the VCS command line. To load the database from the VCS command line, perform the following steps:

1. Run the following compile-time option in VCS:

   ```
   %vcs -simprofile
   ```

2. Run the following runtime option in VCS:

   ```
   %simv -simprofile time+json_dbg_field
   ```

Feedback

3. Launch Verdi using the following command:

   ```
   %verdi -apex -profilePath <profile_database_path> -profileFilter
   <percentage>
   ```

   Where,

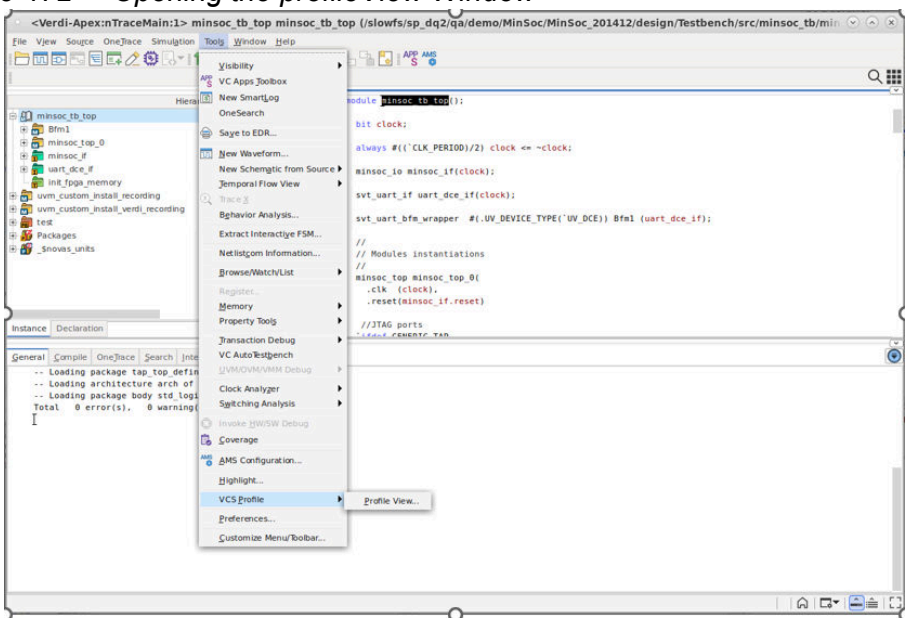
   - `-profilePath <profile_database_path>`: Specifies the path of the profile database.
   - `-profileFilter <percentage>`: (Optional) Specifies the threshold percentage of CPU time. The default value is 0.5 percent.
   - `-apex`: Opens Verdi in Apex mode.

Opening profileView

To open the **profileView** window in Verdi, do the following:

1. Choose **Tools > VCS Profile > Profile View**.

Figure 472 Opening the profileView Window

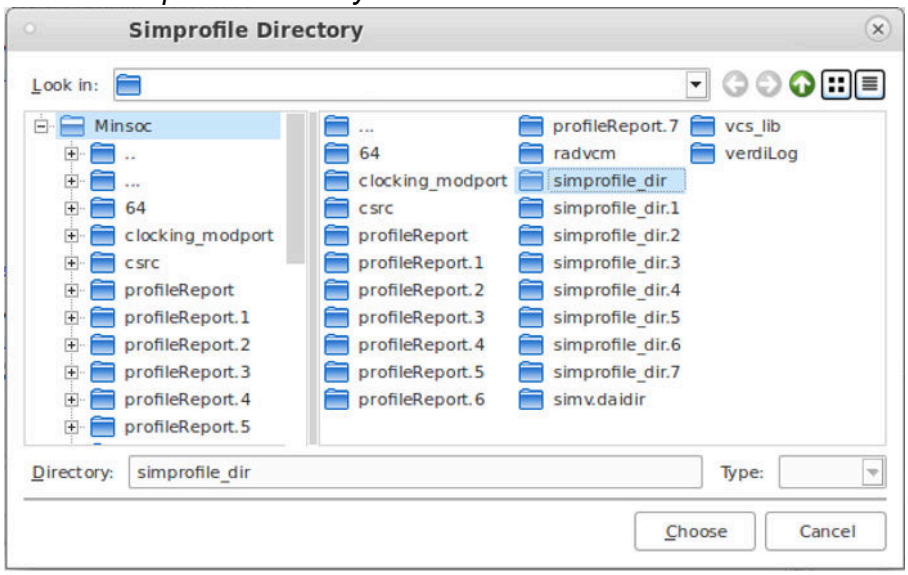


Feedback

The **Simprofile Directory** window opens.

2. In the **Simprofile Directory** window, specify the path of the `simprofile` database, as shown in the following figure:

Figure 473 Simprofile Directory



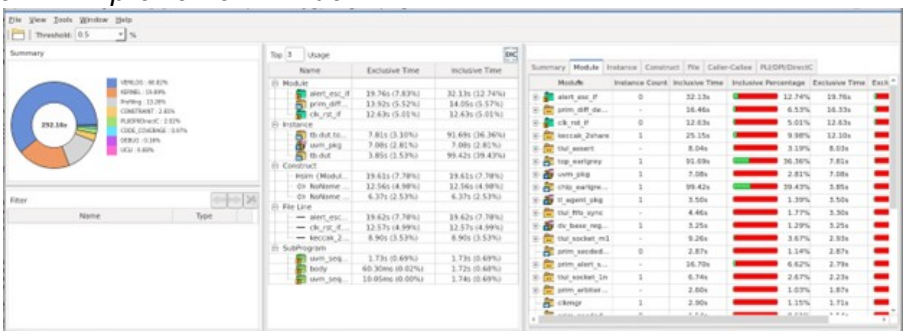
profileView Window Panes

The **profileView** window consists of the following four panes:

- Summary Pane
- Filter Pane
- Top N Usage Pane
- Detail Pane

Feedback

Figure 474 profileView Window



Summary Pane

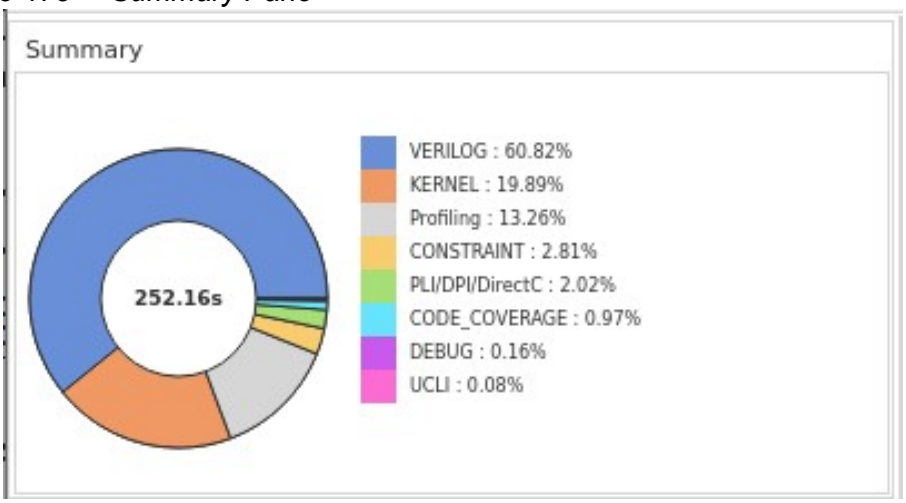
The **Summary** pane provides a visual representation of resource utilization using a pie chart. Each slice in the chart corresponds to a major simulation component and displays its relative CPU time and memory usage.

When you hover the cursor over a specific pie slice, a tooltip displays the following information:

- A brief description of the selected component
- A list of its subcomponents, if applicable

The following figure displays the **Summary** pane:

Figure 475 Summary Pane



Feedback

Filter Pane

The **Filter** pane allows you to drag an object into the table. This pane is non-editable.

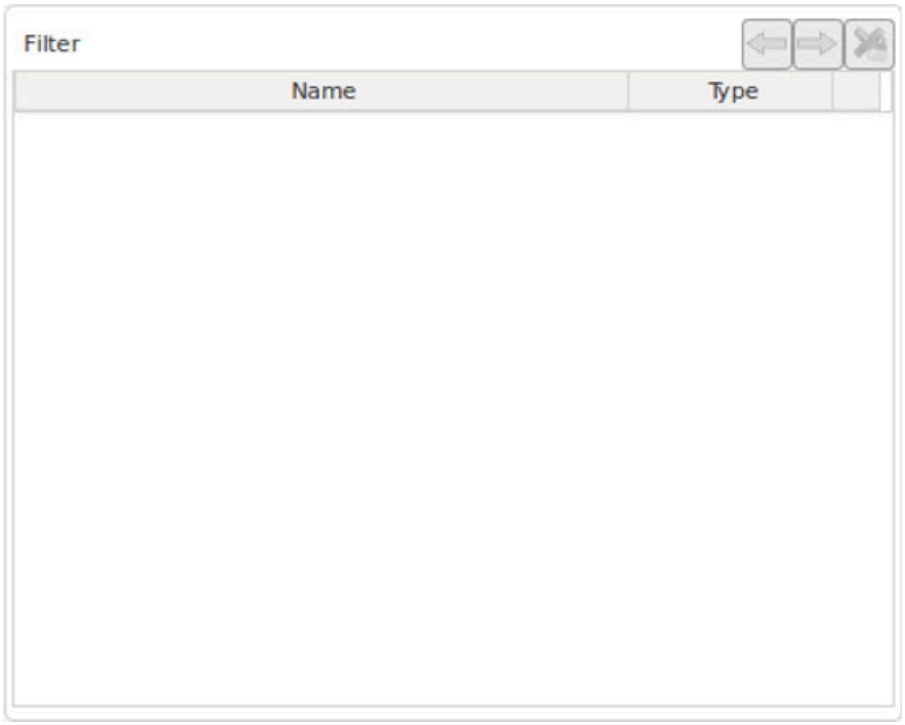
The following table lists the components displayed in the **Filter** pane:

Table 2 Components of Filter Pane

Component	Description
Name	Displays the name of the object.
Type	Displays the object category. The valid types include the following: • Component • Module • Instance • Construct • SubProgram • File Line

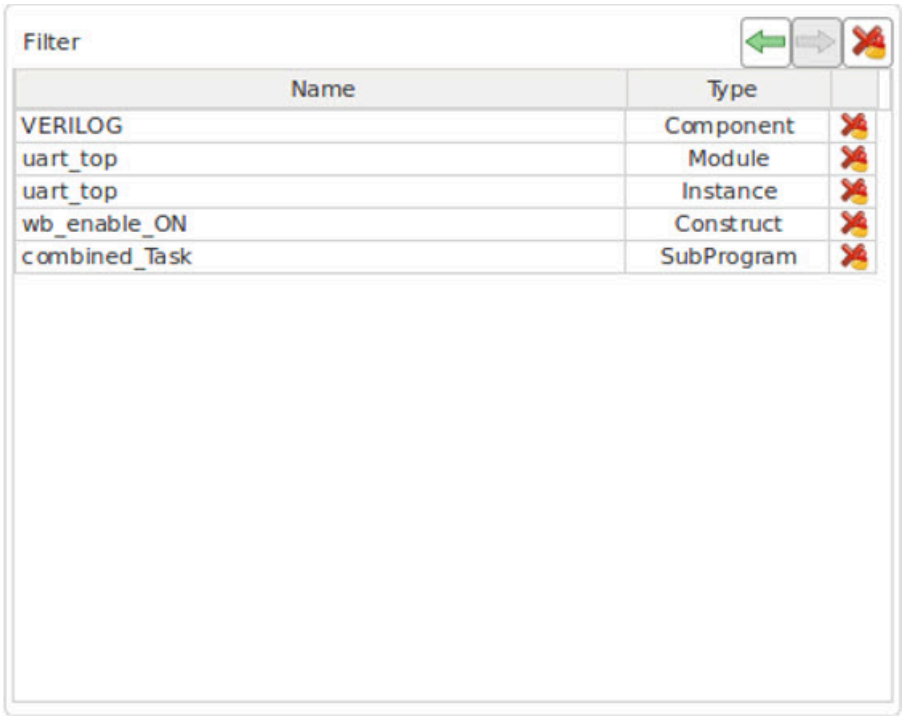
The following figures display the **Filter** pane without data and with data:

Figure 476 Filter Pane Without Data



Feedback

Figure 477 Filter Pane With Data



Top N Usage Pane

The **Top N Usage** pane presents performance data based on objects with the highest usage time, filtered according to the current filter settings. By default, the data is chosen by exclusive time. You can change to choose the data by inclusive time by clicking the (Inc Exc) icon.

The default value of N is 3. To modify this value, click the number and edit the text box. You can drag individual objects from this pane into the **Filter** pane for further analysis.

Note:
Multi-row selection is not supported in this pane.

The following table lists the components displayed in the **Top N Usage** pane:

Table 3 Components of Top N Usage Pane

Component	Description
Name	Displays the name of the object.
Exclusive Time	Displays the time exclusively spent in that object.

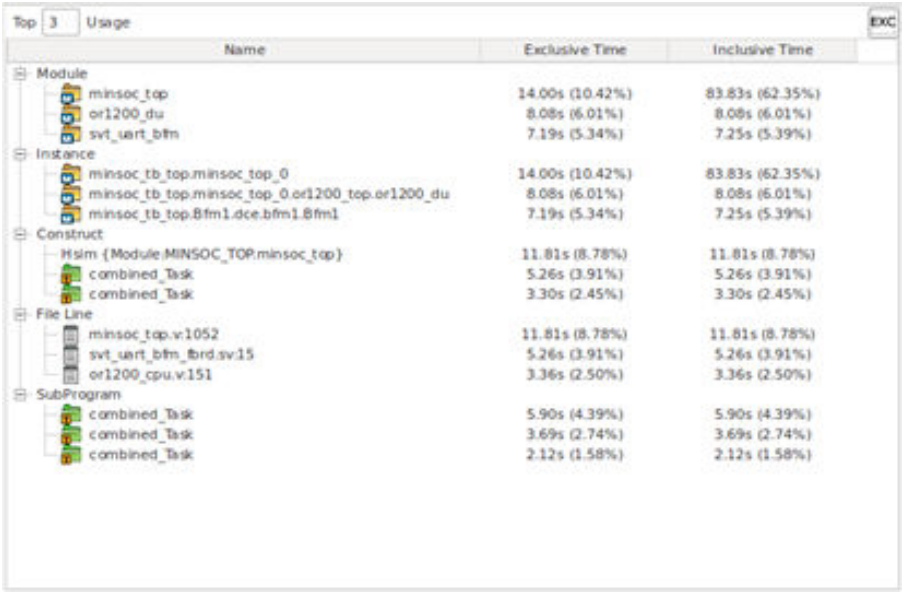
Feedback

Table 3 Components of Top N Usage Pane (Continued)

Component	Description
Inclusive Time	Displays the total time including the child function calls.
Top-Level Object Types	The pane uses a tree view with the following five top-level object types: • Module • Instance • Construct • File Line • Function

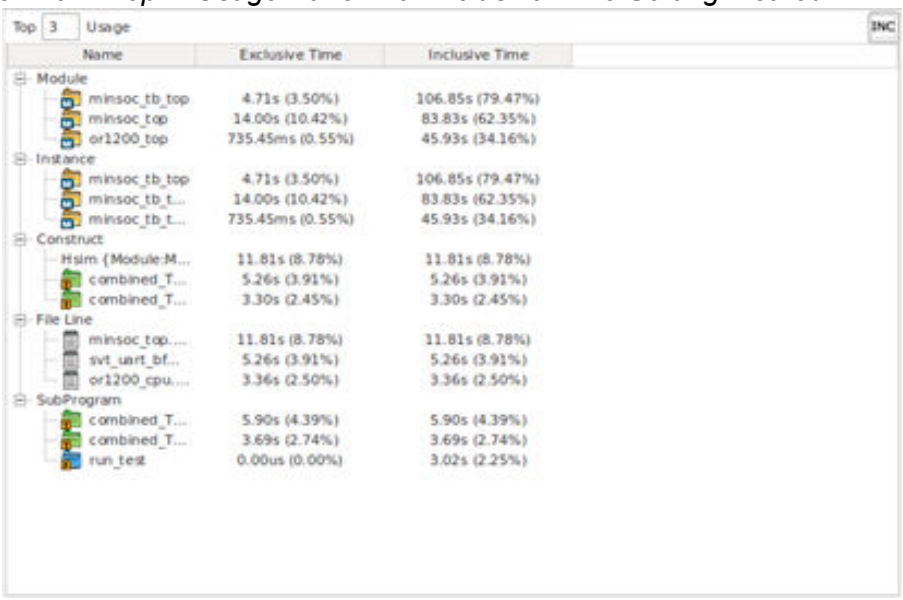
The following figures display the **Top N Usage** pane with the exclusive time and inclusive time sorting methods:

Figure 478 Top N Usage Pane With Exclusive Time Sorting Method



Feedback

Figure 479 Top N Usage Pane With Inclusive Time Sorting Method



Detail Pane

The **Detail** pane provides in-depth information across seven tabs:

- Module Tab
- Instance Tab
- Construct Tab
- Caller Callee Tab
- File Tab
- Summary Tab
- PLI/DPI/DirectC Tab

You can switch between these tabs and drag objects from the hierarchy tree in each tab into the **Filter** pane to narrow down the results.

Note:
Multi-row selection is not supported in this pane.

Each tab dynamically updates its data based on the **Filter** pane content, with the exception of the **Summary** tab, which remains static.

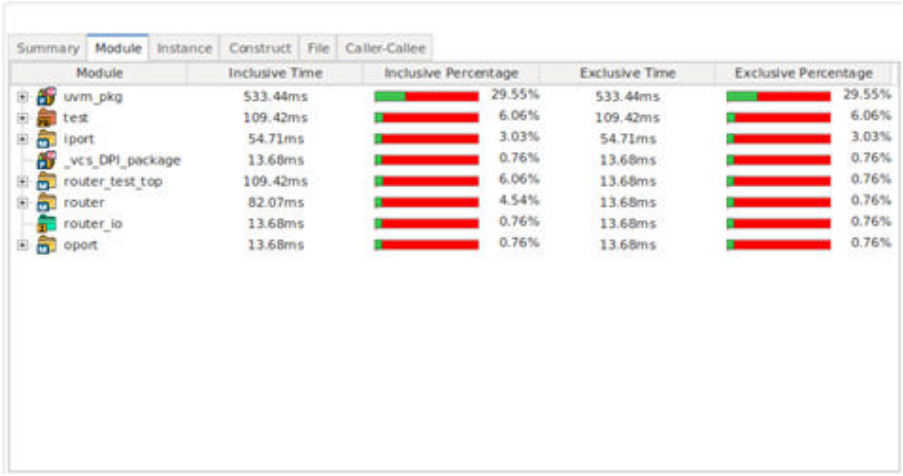
Feedback

Module Tab

The **Module** tab displays module-level information with the module names as top-level nodes. It reflects the same structure and detail as the `simprofile` web report.

The following figure displays the **Module** tab of the **Detail** pane:

Figure 480 Module Tab

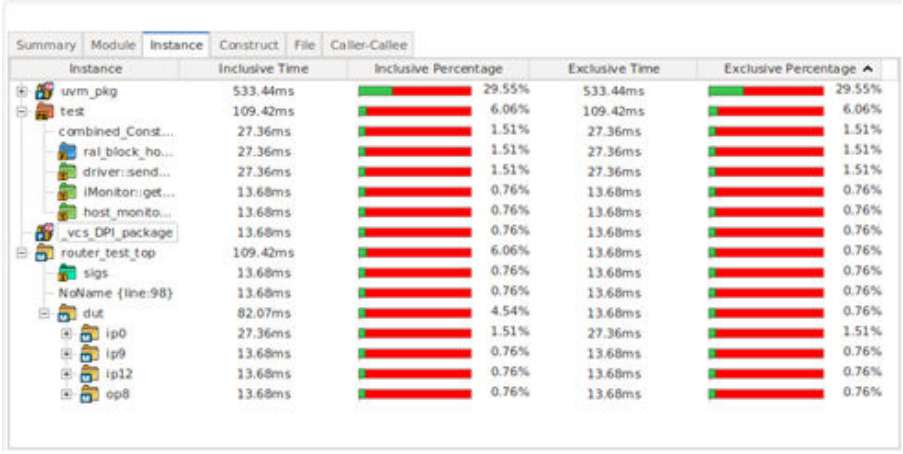


Instance Tab

The **Instance** tab displays a hierarchy of instances and the constructs associated with their respective modules.

The following figure displays the **Instance** tab of the **Detail** pane:

Figure 481 Instance Tab



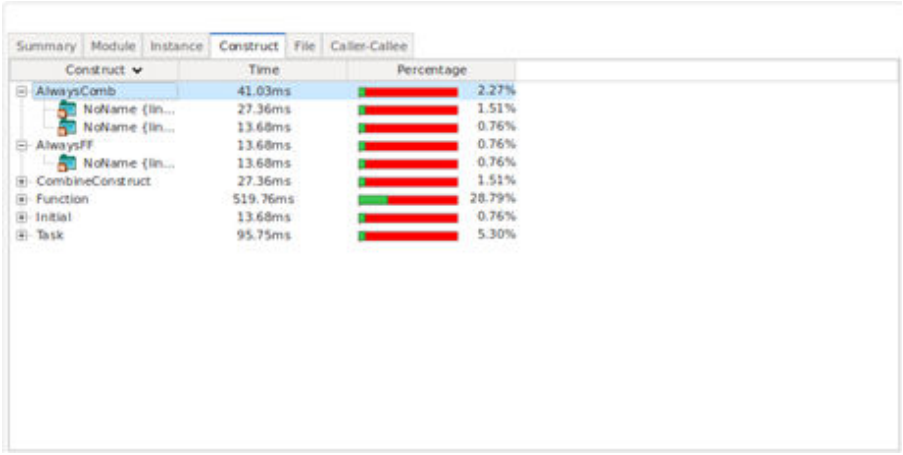
Feedback

Construct Tab

The **Construct** tab displays constructs grouped by construct types at the top-level. Child nodes represent individual constructs.

The following figure displays the **Construct** tab of the **Detail** pane:

Figure 482 Construct Tab



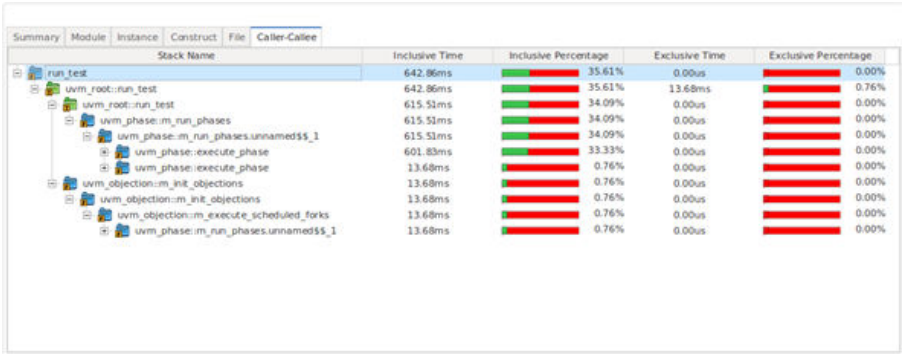
Caller Callee Tab

The **Caller Callee** tab displays the function call relationships using a tree structure, where the top-level nodes are calling functions and child nodes are called functions.

Note:
This tab reflects only the functions or tasks that are listed in the **Construct** tab.

The following figure displays the **Caller Callee** tab of the **Detail** pane:

Figure 483 Caller Callee Tab



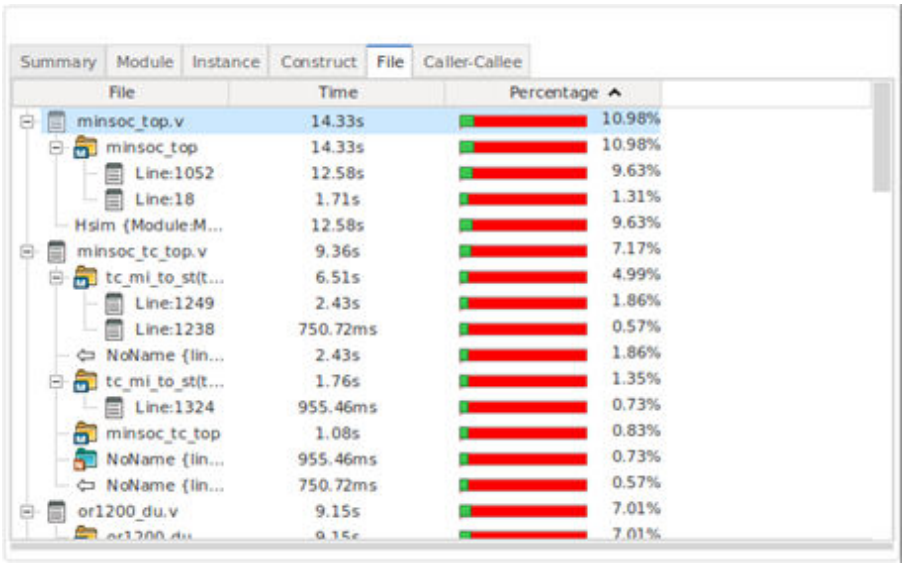
Feedback

File Tab

The **File** tab displays the file hierarchy with filenames as top-level nodes and declarations (modules, lines, constructs) as children. Instances are not included.

The following figure displays the **File** tab of the **Detail** pane:

Figure 484 File Tab



Summary Tab

The **Summary** tab displays the global profile information. Tooltips with brief descriptions appear when you hover over the component columns.

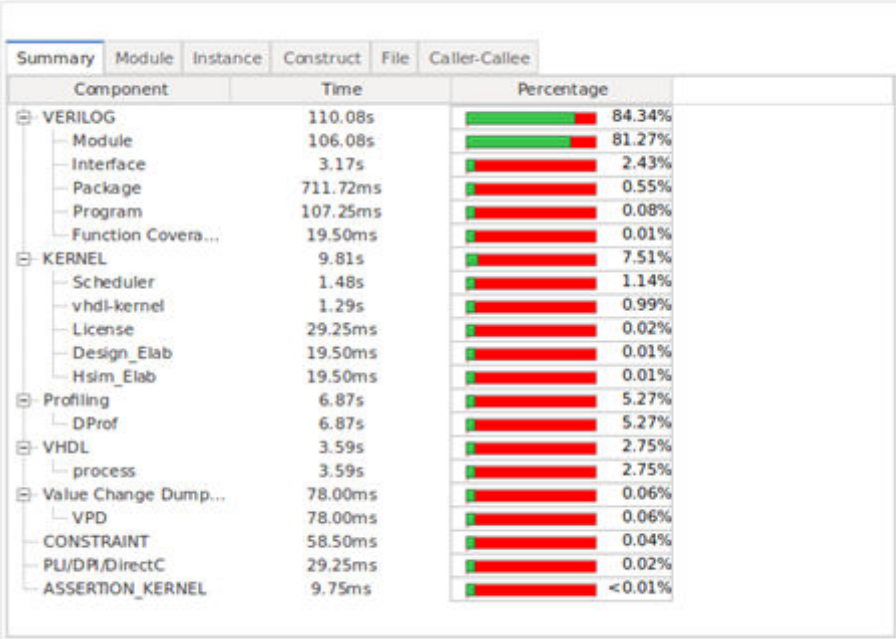
Note:

The data in the **Summary** tab does not change with the **Filter** pane content.

Feedback

The following figure displays the **Summary** tab of the **Detail** pane:

Figure 485 Summary Tab

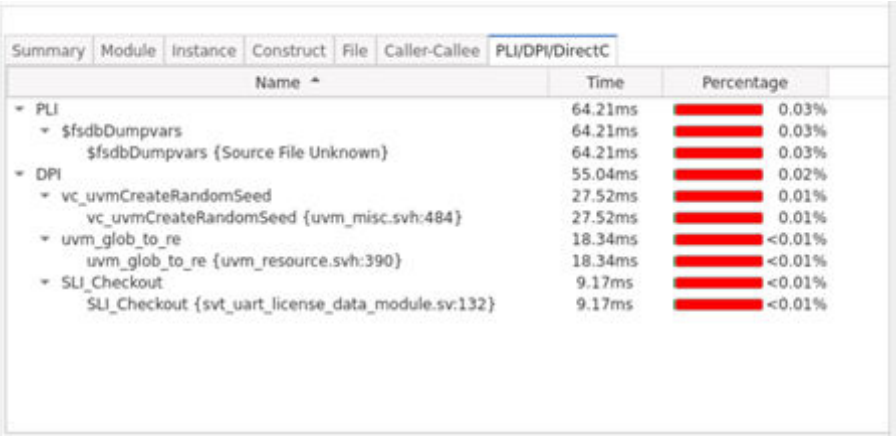


PLI/DPI/DirectC Tab

The **PLI/DPI/DirectC** tab displays a list of functions categorized by PLI, DPI, and DirectC. It has three columns: **Name**, **Time**, and **Percentage**.

The object in the **PLI/DPI/DirectC** tab cannot be added to the filter, and the content in the **PLI/DPI/DirectC** tab does not change with the changes of the filter.

Figure 486 PLI/DPI/DirectC Tab



Feedback

Menus and Commands of the profileView Window

The **profileView** window provides a toolbar, a menu bar, and Right Mouse Button (RMB) context menus across various panes. These components are described as follows:

- Toolbar Commands
- Menu Bar Commands
- RMB Context Menus

Toolbar Commands

The following table lists the commands available in the toolbar of the **profileView** window:

Table 4 Toolbar Commands of the profileView Window

Command	Description
Open	Opens a dialog box to specify the database path. On setting the path, the **profileView** window reloads and displays the updated data
Filter	Allows you to specify a percentage range between 0.01 and 100 for filtering the data based on inclusive time. The default value is 0.5 percent, unless overridden through the command line. When you click **Enter**, it refreshes the data in all tabs of the **Detail** pane. **Note:** The filter toolbar is hidden when the SVA distribution mode is active because filtering is solely based on inclusive time in this mode.

Feedback

Menu Bar Commands

The following table lists the commands available in the menu bar of the **profileView** window:

Table 5 Menu Bar Commands of the profileView Window

Menu	Description
File	• **Open**: Allows you to select a database path. • **Close the Window**: Closes the current **profileView** window.
View	• **SVA Distribution Mode**: Enables the special assertion-focused mode. In this mode, the **Module**, **Instance**, **Construct**, and **File** tabs show only assertion-related information. • **Instance List**: Shows the self-instance for the selected module, the top-N instance, or the top-N assertions.
Tools	**Show Top-N Instances**: Displays the top instances by exclusive time in the instance list. This command has the following five options: **10**, **20**, **50**, **100**, and **All**. The double-click and sorting operations are available along with the percentage menu.
Window	Docks the **profileView** window into a separate window.
Help	Opens an information window that displays details about the current database.

RMB Context Menus

The following tables list the RMB context menus available in the **Top N Usage** and **Detail** panes of the **profileView** window:

Table 6 RMB Context Menus of the Top N Usage Pane

Menu	Description
Add to Filter	Adds the selected object to the filter, if permitted.
Display by Exclusive	Switches between the inclusive time and exclusive time sorting.

Feedback

Table 7 RMB Context Menus of the Detail Pane

Menu	Description
Expand by Level	Expands the selected tree node by one level.
Collapse All	Collapses the selected tree node.
Add to Filter	Adds the selected object to the filter, if permitted.
Show Instance	Activates the **Instance** tab and highlights the first self-instance. It displays all instances of the module in the instance list. This menu is available in the **Module** and **Construct** tabs.
Show Module	Activates the **Module** tab and highlights the module corresponding to the selected instance. This menu is available in the **Instance** and **Construct** tabs.

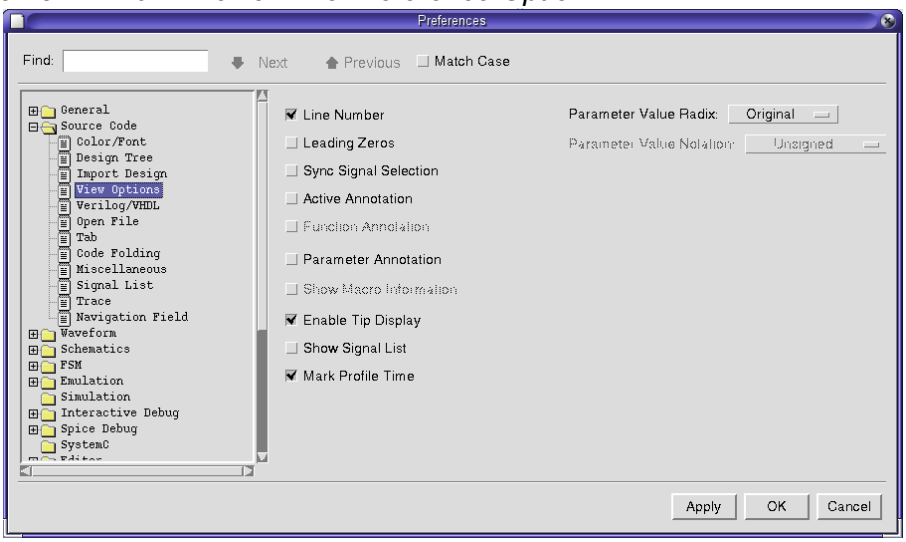
Source View Indicator

When you enable the **Mark Profile Time** option in the **Preferences** window of Verdi, **nTrace** automatically marks the exclusive percentage before the line number in the **Source** window for an active module, when **profileView** is opened. When you hover the pointer over the percentage, **nTrace** displays the exclusive time and percentage, as shown in Figure 488. The time on the line of module is equal to the exclusive module time minus the total time of construct in the current module. The information disappears if the window is closed.

nTrace follows the HTML specification to display the module-based time information. If you use the **Marco** folder, the entire Marco is displayed in the **Source Code** window.

Feedback

Figure 487 Mark Profile Time Preference Option

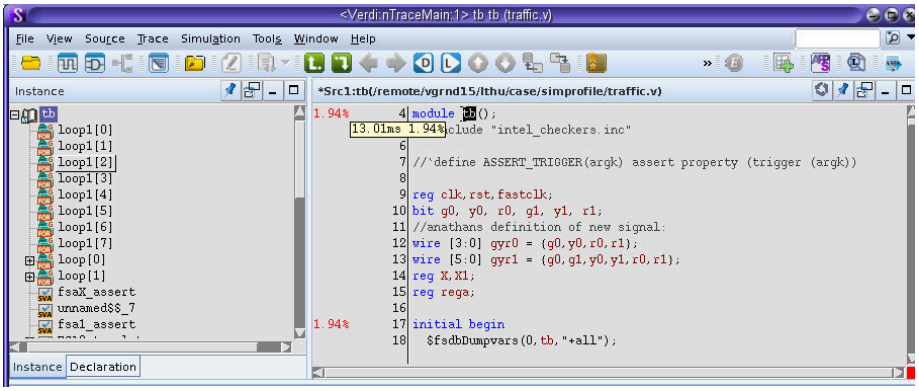


To enable the **Mark Profile Time** option, perform one of the following steps:

- From the menu bar of Verdi, choose **Tools > Preferences > Source Code > View Options > Mark Profile Time**.
- From the menu bar of Verdi, choose **Source > Mark Profile Time**.

The following figure displays the source view indicator:

Figure 488 Source View Indicator



Feedback

Limitations of the Source View Indicator

The source view indicator has the following limitations:

- If the current instance used to navigate to the source code is filtered out in the `simprofile` database, **nTrace** cannot display the indicator for the profile time.
- The source view indicator does not support VHDL modules.
- **nTrace** does not show the source code and the indicator if the current scope is encrypted.
- The text indicator generated by the simulator is not supported in **nTrace**.

SVA Distribution Mode

The SVA distribution mode in **profileView** shows the location of the assertion and the assertion density. It also provides the SVA performance analysis.

The following tasks can be performed in the **profileView** window to invoke the SVA distribution mode:

- Enabling the SVA Distribution Mode
- Switching Between SVA and Non-SVA Distribution Mode

Enabling the SVA Distribution Mode

To enable the SVA distribution mode from the **profileView** window, choose **View > SVA Distribution Mode**.

Switching Between SVA and Non-SVA Distribution Mode

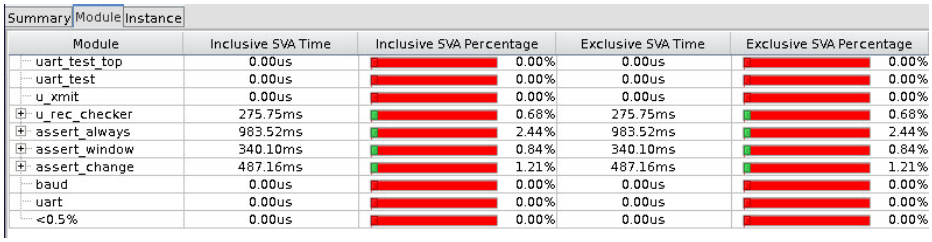
You can switch between the SVA distribution mode and non-SVA distribution mode. The following changes occur on enabling the SVA distribution mode:

- The **Module** tab displays the following column name changes:
  - **Module**: Displays the name of the module.
  - **Inclusive SVA Time**: Displays the sum of SVA time in the current scope and the sub-scopes.
  - **Inclusive SVA Percentage**: Displays the ratio of inclusive SVA time and CPU time.

Feedback

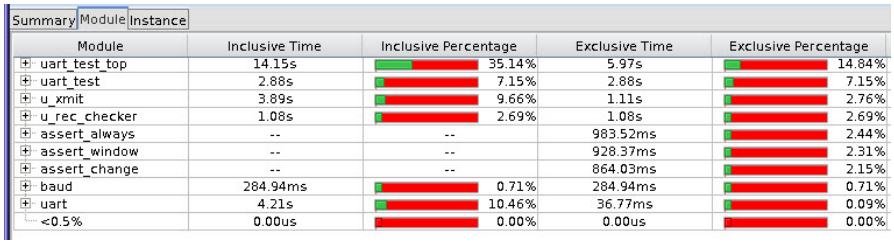
- **Exclusive SVA Time**: Displays the SVA time of the current scope.
- **Exclusive SVA Percentage**: Displays the ratio of exclusive SVA time and the CPU time.

Figure 489 Module Tab in SVA Distribution Mode



Module	Inclusive SVA Time	Inclusive SVA Percentage	Exclusive SVA Time	Exclusive SVA Percentage
uart_test_top	0.00us	0.00%	0.00us	0.00%
uart_test	0.00us	0.00%	0.00us	0.00%
u_xmit	0.00us	0.00%	0.00us	0.00%
u_rec_checker	275.75ms	0.68%	275.75ms	0.68%
assert_always	983.52ms	2.44%	983.52ms	2.44%
assert_window	340.10ms	0.84%	340.10ms	0.84%
assert_change	487.16ms	1.21%	487.16ms	1.21%
baud	0.00us	0.00%	0.00us	0.00%
uart	0.00us	0.00%	0.00us	0.00%
<0.5%	0.00us	0.00%	0.00us	0.00%

Figure 490 Module Tab in Non-SVA Distribution Mode



Module	Inclusive Time	Inclusive Percentage	Exclusive Time	Exclusive Percentage
uart_test_top	14.15s	35.14%	5.97s	14.84%
uart_test	2.88s	7.15%	2.88s	7.15%
u_xmit	3.89s	9.66%	1.11s	2.76%
u_rec_checker	1.08s	2.69%	1.08s	2.69%
assert_always	--	--	983.52ms	2.44%
assert_window	--	--	928.37ms	2.31%
assert_change	--	--	864.03ms	2.15%
baud	284.94ms	0.71%	284.94ms	0.71%
uart	4.21s	10.46%	36.77ms	0.09%
<0.5%	0.00us	0.00%	0.00us	0.00%

- The **Instance** tab displays the following column name changes:
  - **Instance**: Displays the name of an instance.
  - **Inclusive SVA Number**: Displays the sum of SVA number in the current scope and the sub-scopes.
  - **Inclusive SVA Time**: Displays the sum of SVA time in the current scope and the sub-scopes.
  - **Inclusive SVA Percentage**: Displays the ratio of inclusive SVA time and the CPU time.
  - **Exclusive SVA Number**: Displays the SVA number in the current scope.
  - **Exclusive SVA Time**: Displays the SVA time of the current scope.
  - **Exclusive SVA Percentage**: Displays the ratio of the exclusive SVA time and the CPU time.

Feedback

Figure 491 Instance Tab in SVA Distribution Mode

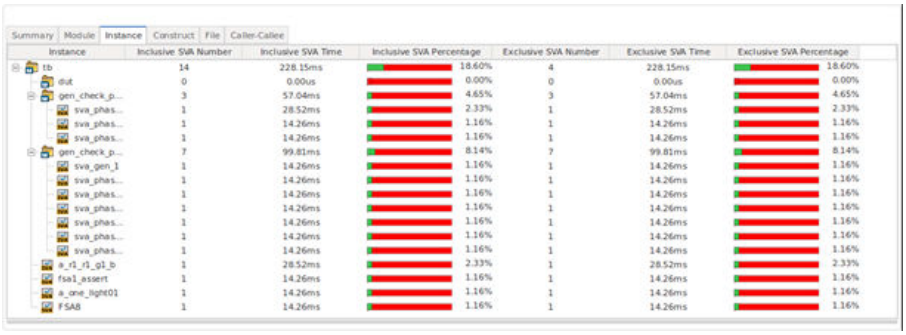


Figure 492 Instance Tab in Non-SVA Distribution Mode

Merged Simprofile Report

To merge multiple Simprofile reports into a single report, use the following command:

```
profrpt -f <report_dirs_file>-output <merge_report_dir> -format json
```

Here,

`-f <report_dirs_file>`: Specifies a text file that lists the paths to all the Simprofile reports that you want to merge.

Example:

```
simprofile_dir
simprofile_dir.1
simprofile_dir.2
```

Feedback

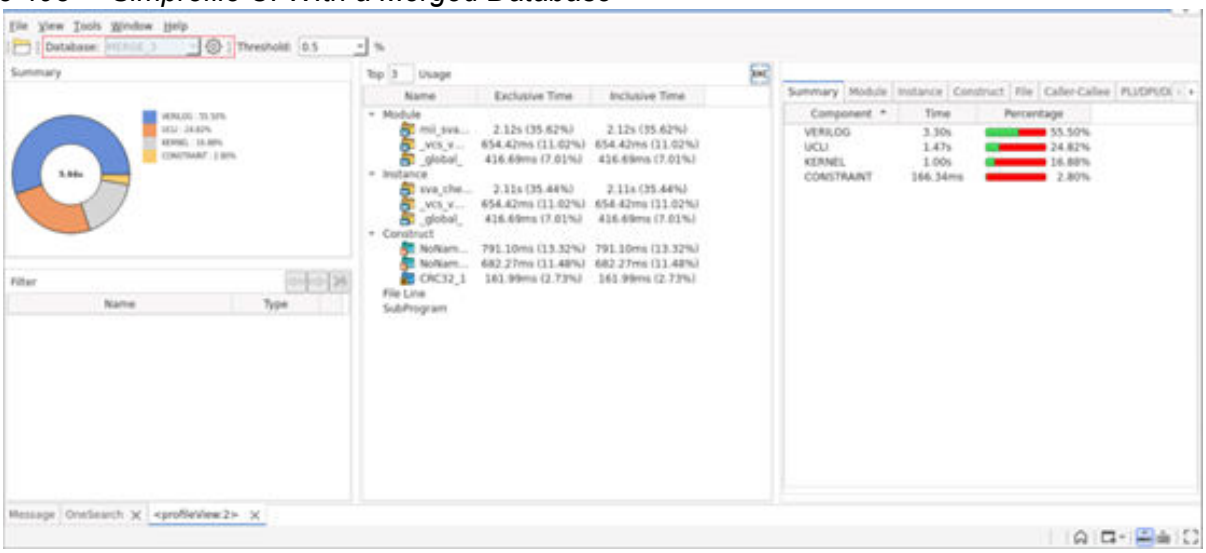
`-output <merge_report_dir>`: Specifies the directory where the merged report is saved.

`-format json`: The merged report is generated in JSON format.

Merged Simprofile Report Interface in Verdi

When Verdi opens a merged Simprofile database, the interface display the merged database name and a configuration icon next to the **Open File** button.

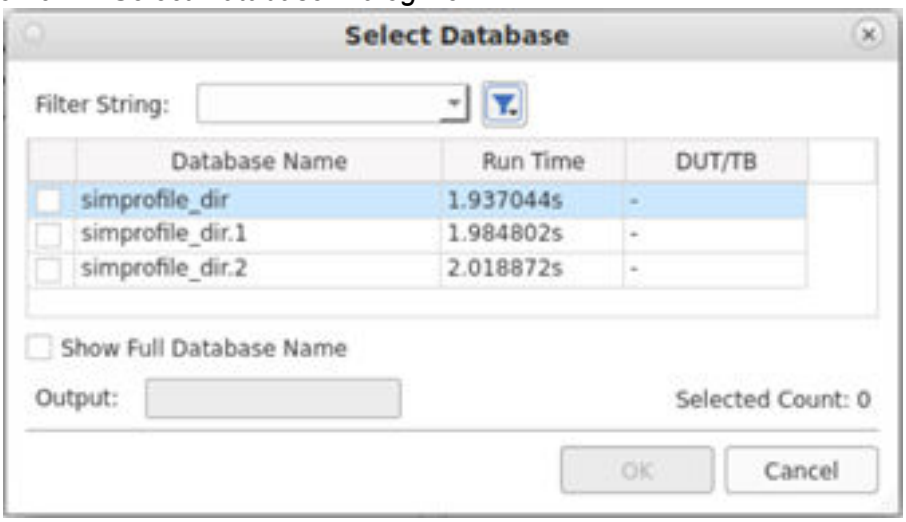
Figure 493 Simprofile UI WIth a Merged Database



Clicking the configuration icon opens the **Select Database** dialog box, which shows a list of databases included in the merged file.

Feedback

Figure 494 Select Database Dialog Box

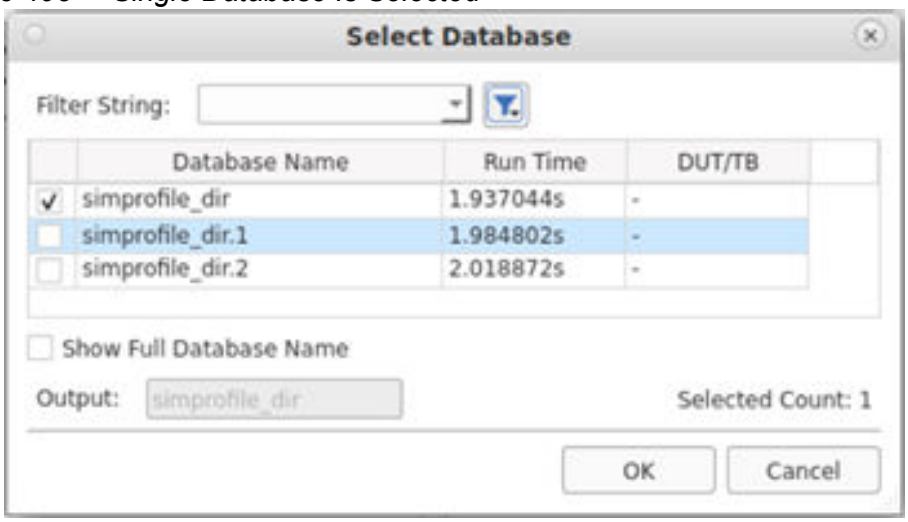


The **Select Database** dialog box displays a table with three columns: **Database Name**, **Run Time**, and **DUT/TB**. Each row includes a check box for selection.

Behavior Based on Selection:

- **Single Selection:** The output field shows the name of the selected database (read-only). Clicking **OK** loads the selected database.

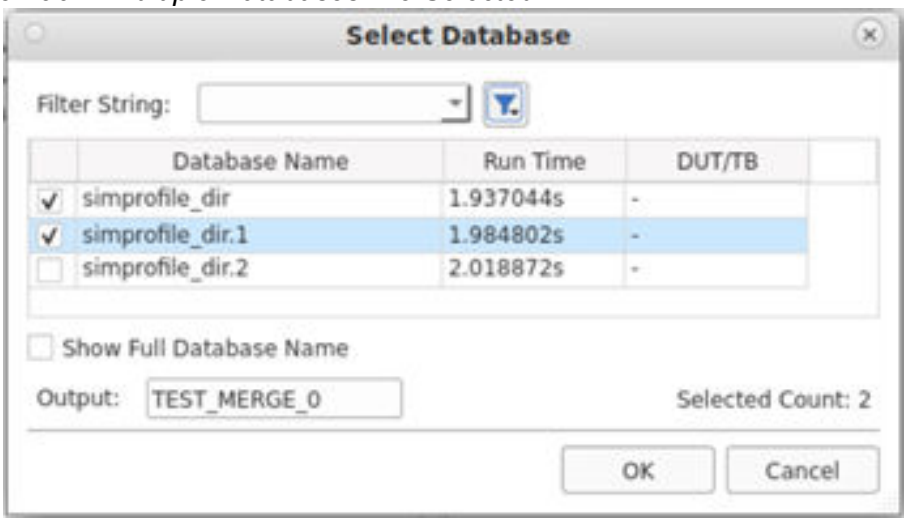
Figure 495 Single Database Is Selected



Feedback

- **Multiple Selections:** The output field displays a generated name for the new merged database (editable). Clicking **OK** merges and loads the selected databases.

Figure 496 Multiple Databases Are Selected



Additional Options in the UI:

- **Filter String:** Enter a keyword to filter the list by database name.
- **Show Full Database Name:** Displays the complete path of each database in the list.

DUT/TB Proportion Display

To enable Design Under Test/Testbench (DUT/TB) proportion display in the **Summary** pane, include the following parameter in your simulation command:

```
simv -simprofile time+json_dbg_field+dut_config=<config_file_name>
```

Here,

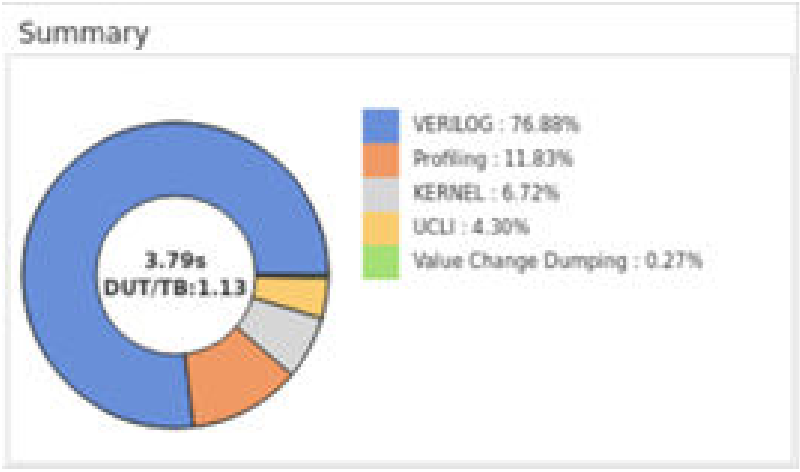
`+dut_config=<config_file_name>`: Specifies the configuration file that contains DUT and TB information.

UI Behavior in Verdi

When the `+dut_config` option is used during simulation, Verdi Simprofile enhances the **Summary** pane by showing the DUT/TB proportion at the center of the pie chart.

Feedback

Figure 497 DUT/TB Proportion Display



Limitations

The Simprofile feature in Verdi has the following limitations:

- Does not support save session and restore session.
- Does not support cross-platforms. For example, `Verdi_linux_64bit` can correctly load the profile database only from `VCS_linux_64bit`.
- Supports only time profiling. Memory profiling and performance analysis are not supported.
- Supports only concurrent assertions; other assertion types are not supported.
- Supports only the post-simulation mode.
- Supports loading the profile database only from the consistent version and sub-version of VCS. For example, `Verdi_2016.06-1` can correctly load the profile database only from `VCS_2016.06-1`.
- The functions or tasks in the **Caller Callee** tab are related only to the constructs in the **Construct** tab.
- Supports only Verilog as the component filter.
- Supports showing only the top-N instance list.
- Does not support RMB context menus in the **Summary** and **Filter** panes of the **profileView** window.
- Cannot identify if an object is addable in the RMB context menu.

Feedback

- Does not support the **Show in Detail** RMB context menu in the **Top N Usage** pane. You can double-click the pane instead to view the details.
- Does not support the **Show in Source Code** RMB context menu in the **Detail** pane. You can double-click the pane instead to view the source code.

A

Appendix A: Supported Waveform Formats

This appendix covers the following topics:

- Overview
- Fast Fourier Transformers (FFT)
- EVCD
- Analog Waveform Example

Overview

In addition to the FSDB waveform format, the following formats are supported:

- VCD (Value Change Dump)
- EVCD (Extended Value Change Dump)
- Analog - Powermill, Spice

Fast Fourier Transformers (FFT)

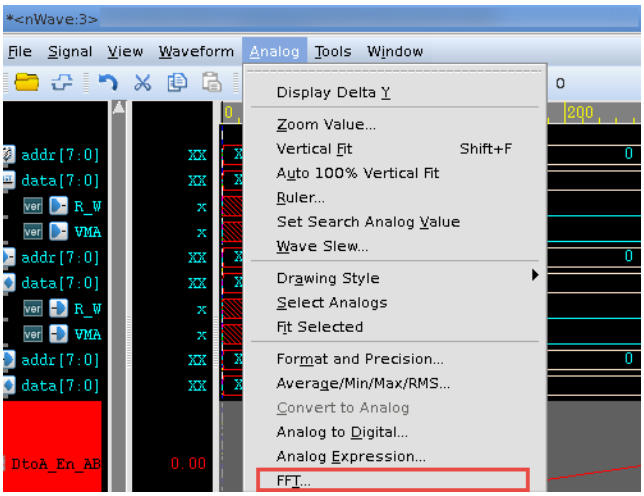
nWave provides the capability of viewing and analyzing analog signals in the frequency domain. An FFT window in *nWave* is used to display and process frequency waveforms. *nWave* can process analog signals through FFT to get the frequency results.

Getting Data From Analog Signal

After analog signals are imported into *nWave*, choose the **Analog -> FFT** command to access the FFT window.

Feedback

Figure 498 Open FFT from nWave

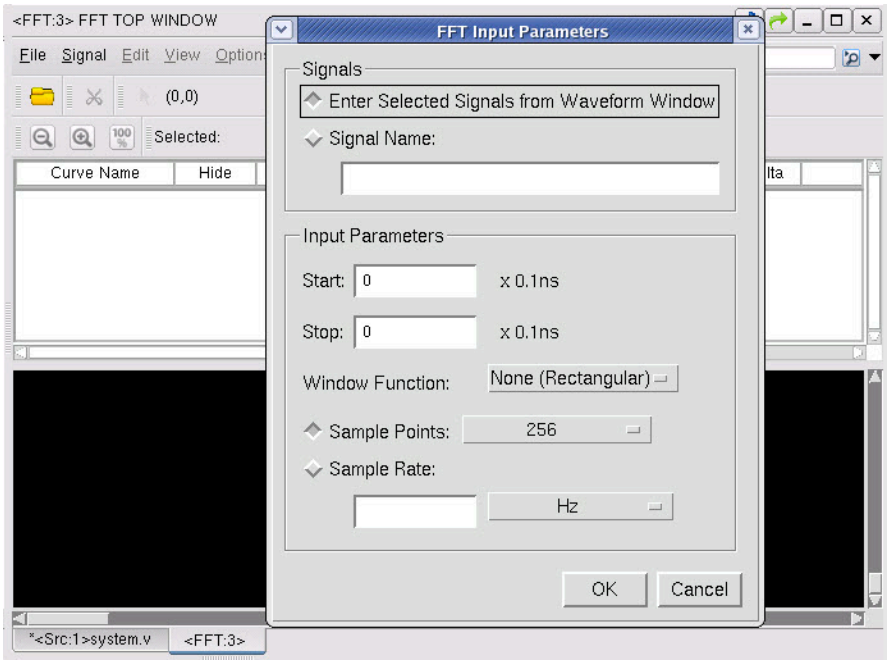


The FFT window is created without any waveforms displayed.

Choose the **Signal -> Add FFT Signal** command in the FFT window.

The *FFT Input Parameters* form displays, as shown in the following figure. You can specify parameters for FFT in this form.

Figure 499 Add FFT Signal Form



Feedback

Target signal can be specified through *nWave* window selection or drag from *nWave* into the **Signal Name** text box. The **Start** and **Stop** times have to be specified to define the range of the FFT process.

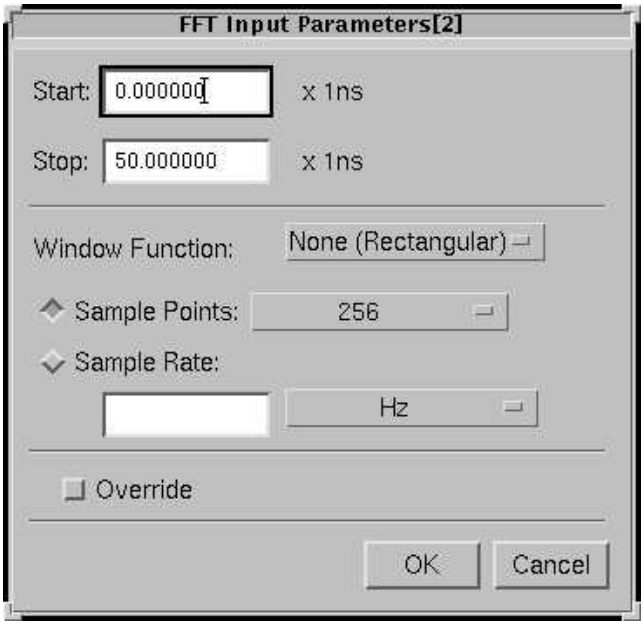
Seven types of window functions (rectangular, Blackman, Hamming, Hanning, Parzen, Triangular and Welch) can be used in *nWave*. The resolution of the FFT result can be specified with sample point number or sample rate.

Different FFT methods on the same target signal can be compared in the FFT window.

Select the signal in the FFT window, then use the **Edit -> Calculate Selected** command. The different parameters are specified in the *FFT Input Parameters [2]* form (as shown in the following figure), and the recalculated waveform are displayed for comparison.

Enable the **Override** option to overwrite the original result.

Figure 500 FFT Input Parameters [2] Form



The FFT results of analog waveforms can be saved in the FSDB, and restored for use later. You can also export the FFT result to an ASCII format text file.

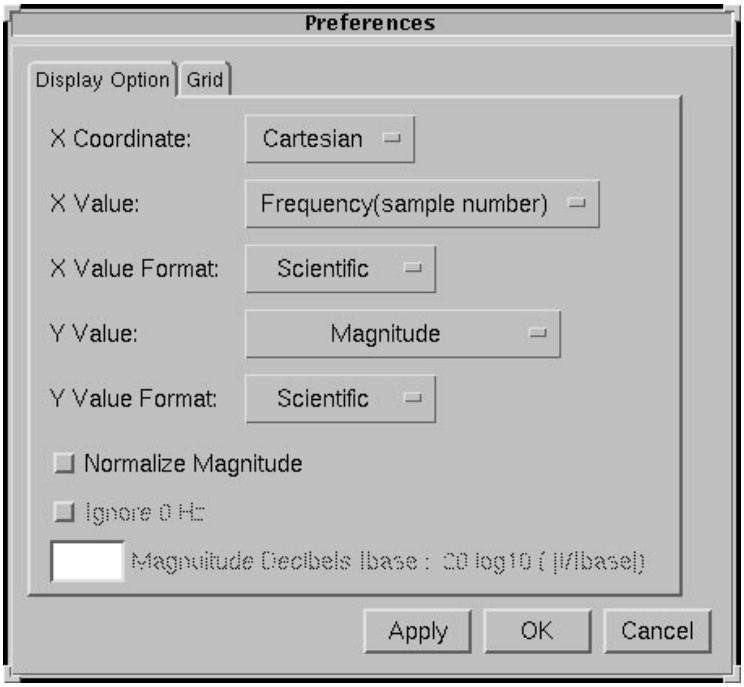
Data Manipulation in FFT Window

The FFT window provides several methods to change data display formats for analysis under the **Options -> Preferences** command.

Feedback

In the *Preferences* form, the *Display Option* tab contains several setting for data manipulation, as shown in the following figure:

Figure 501 Preferences Form



The y-axis can be changed to **Magnitude decibels (dB)** as conventional notation. Changing the Ibase value can modify the offset value of y-axis in dB.

For example, changing Ibase value from 1 to 10 means the dB offset value changed from 0 to -20. If Ibase is specified as 0.1, the offset value is changed to +20.

The Ibase value only affects analog waveform FFT results.

Feedback

The following figures are example FFT waveforms fixed are -40 dB:

Figure 502 FFT Waveform

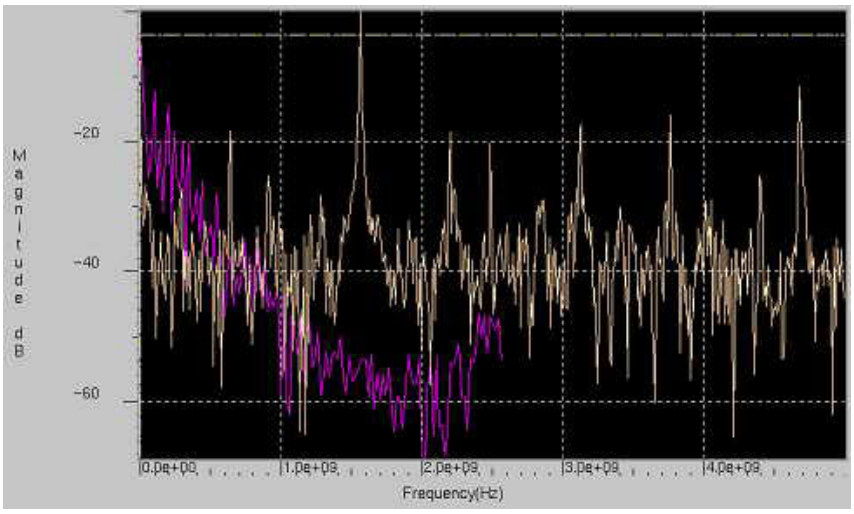
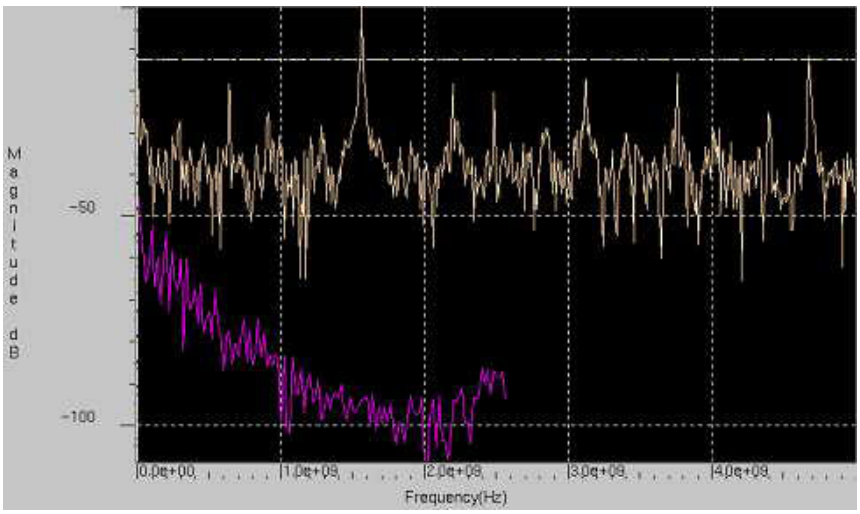


Figure 503 FFT Waveform



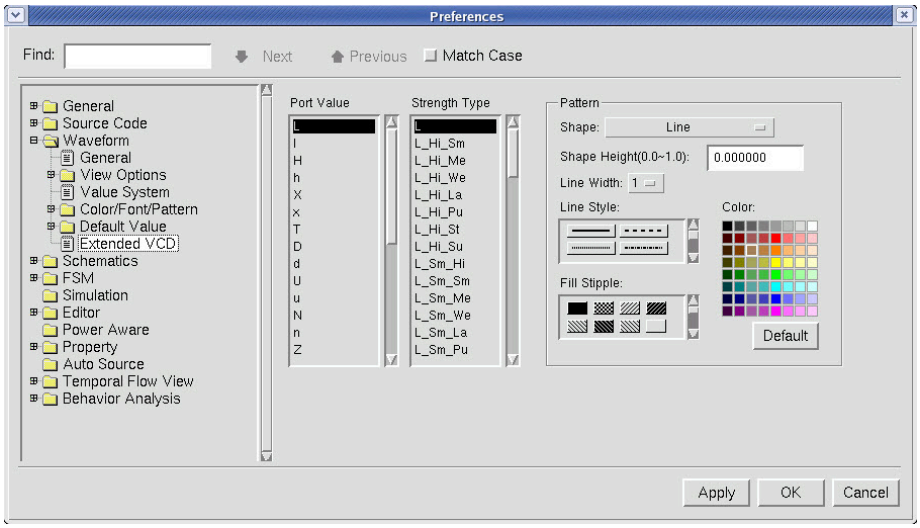
EVCD

A EVCD (Extended VCD) file saves the instance ports' logic and driver information. It is valuable if it can be read as a waveform and back-annotated to the design source code and schematics. The capabilities for supporting EVCD are listed as follows.

Feedback

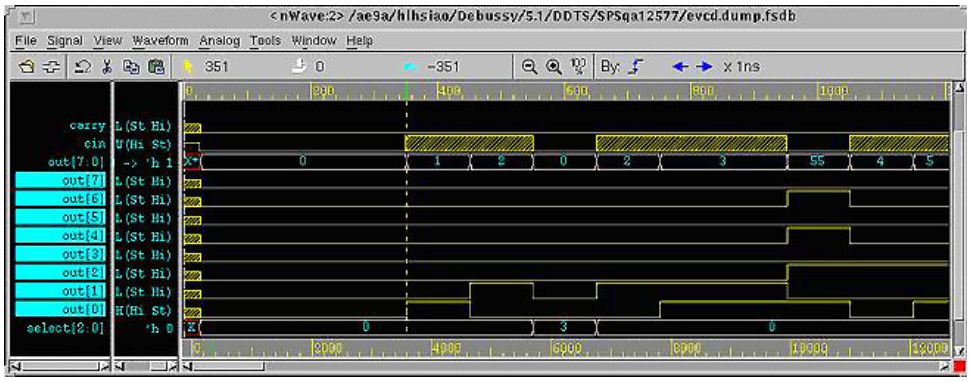
- Convert EVCD to FSDB -- You can convert your dumped EVCD to FSDB by *vfast*.
- The converted FSDB retains full range of values to represent logic and driver information. The values include 25 port values, 8 strengths for 0 and 8 strengths for 1.
- *nWave* displays full range of values to represent logic and driver information -- *nWave* can display 25 port values, 8 strengths for 0 and 8 strengths for 1. In total, *nWave* can display 1600 patterns (25*8*8) for EVCD. Through the *Preferences* form (**Waveform** folder, **Extended VCD** page), you can configure all of the patterns as you like.

Figure 504 EVCD Preferences Form



- Map the logic and driver information to standard VCD value -- For bus ports, values are mapped to IEEE standard Verilog value space (0,1,z.x). To see each individual port's value, you can expand the bus port to single bit ports as shown in the following figure:

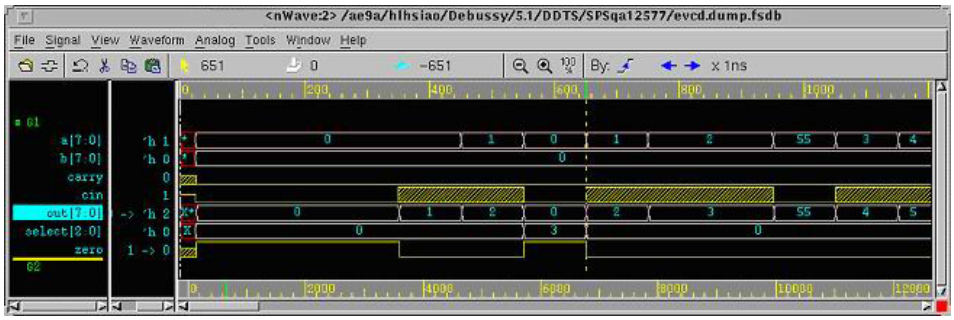
Figure 505 Example Expanded Bus Ports



Feedback

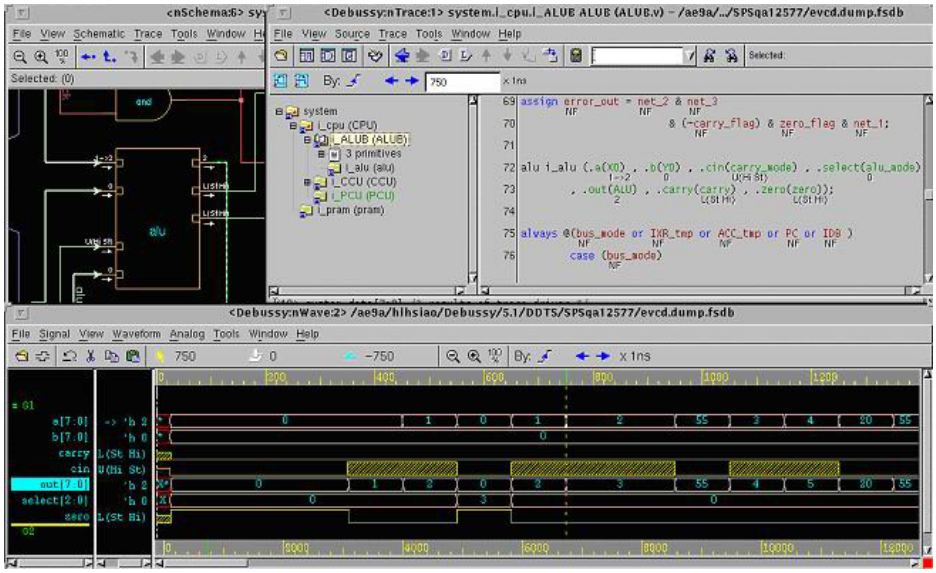
- Map values of all single bit ports to IEEE standard Verilog value space by turning on the **Normalize EVCD Display Value** option under the **Waveform** folder -> **General** page of the *Preferences* form (invoked with the **Tools -> Preferences** command).

Figure 506 Example Normalized EVCD Display Value



- Support search value for any changes, rising and falling -- The search value for bus ports is the same as VCD since their values have been mapped to IEEE standard Verilog value space. When you search value for single bit ports, the Verdi platform maps them to IEEE standard Verilog value space internally. For example, the Verdi platform maps L(str0,str1)->H(str0,str1) to 0->1 and recognize it as a rising change.
- Annotate EVCD value or the mapped VCD value to *nTrace* and *nSchema* -- In *nTrace*, port value is annotated. In *nSchema*, port value and direction are annotated.

Figure 507 Annotate EVCD/Mapped VCD Value to nTrace and nSchema



The following features are not supported:

- Utilities (*fsdb2vcd*, *fsdbextract*, *fsdbmerge*, *fsdbReport*) for converted FSDB.
- *Trace-X* and *List-X* for the converted FSDB.

Analog Waveform Example

This section covers the following topics:

- View the Analog Waveform
- Manipulate the Analog Waveform
- View Different Simulation Results in the Same Window
- Overlap Analog Signals From Different Simulation Results

View the Analog Waveform

1. Change your context to the analog subdirectory, which is where all of the demo source code files are located:

   ```
   % cd <working_dir>/demo/analog
   ```

2. Start *nWave*.

   ```
   % nWave &
   ```

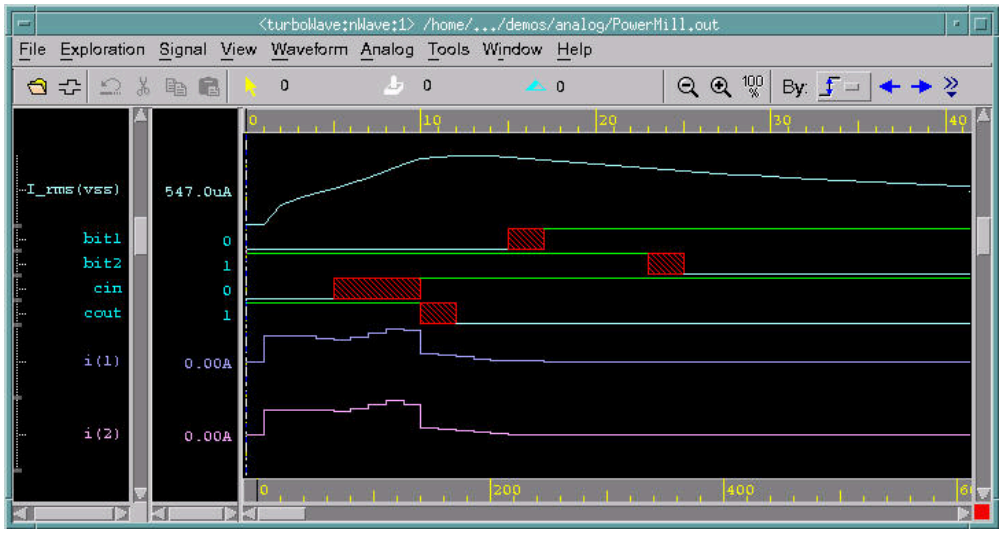
3. Choose the **File -> Open** command, and type *.* in the **Filter** text box of the *Open Dump File* form.
4. Select the file *PowerMill.out* and click the **Add** and then the **OK** button.
5. Click the **Yes** button on the *Question* dialog window for direct read.
6. Choose the **Signal -> Get All Signals** command.

Feedback

7. Click the **Yes** button on the *Confirmation* dialog window.

   An analog waveform is displayed in *nWave*, as shown in the following fugure:

Figure 508 Analog/Digital Waveforms in nWave



The *nWave* frame displays analog waveforms differently from digital waveforms in two ways:

- *nWave* makes analog signals taller than digital signals.
- *nWave* uses different colors to display each newly added analog signal.

Manipulate the Analog Waveform

Similar to the way you change display formats for digital signals, you can change the format for analog signals by clicking-right on the value pane to change to the required format. The supported analog display formats are: V, mV, A, mA, and uA.

Note:
Choose the **Analog -> Format & Precision** command, to open the *Format* form and set the **Analog Format** to **Scientific** or **Engineering** first.

Change the Signal Height

1. Select the signals *i(1)* and *i(2)*.
2. Choose the **Waveform -> Height** command, and enter *200* pixels for the new signal height.

   The signal height increases, and only one signal can be seen in the *nWave* window.

3. Maximize the window to see both signals.

   Note:
   nWave limits the minimum signal height to the signal name height and the maximum signal height to the height of the waveform window. After you resize an *nWave* window, *nWave* changes the signal height automatically if the signal height is taller than the waveform pane.

Display the Analog Ruler

1. Choose the **Analog -> Ruler** command to display the *Analog Ruler* form.
2. Enter *10000* in the **Grid Step** field to display the vertical grid step at every 10000 value unit.
3. Click **Apply** to display the ruler on the selected signals.
4. Click the **Cancel** button to close the *Analog Ruler* form.

View Different Simulation Results in the Same Window

You can open multiple simulation result files in the same *nWave* window. This capability is especially useful for analyzing analog waveforms from different simulation runs or different simulators. For example, you can mix your Verilog VCD waveforms with waveforms from PowerMill.

1. Highlight the existing signals in *nWave* (use the **Signal -> Select All** command), and use the **Cut** icon to remove them from the display.
2. Choose the **Signal -> Get Signals** command.
3. Select the signals *i(cin)* and *i(node1)*, and click the **OK** button.
4. Select group *G1*.
5. Right-click and choose **Rename** to change the group name *G1* to *PwrMill*.
6. Set the signal cursor under group *G2*.
7. Choose the **File -> Open** command to open the *Open Dump File* form.
8. Select the file *SmartSpice.out,* and click the **Add** and then the **OK** button.
9. Click the **OK** button on *Information* dialog window.
10. Choose the **Signal -> Get Signals** command.
11. Select the signals *i(cin)* and *i(node1)*, and click the **OK** button.
12. Select group *G2*.

Feedback

13. Right-click and choose **Rename** to change the group name *G2* to *SmtSpice*.

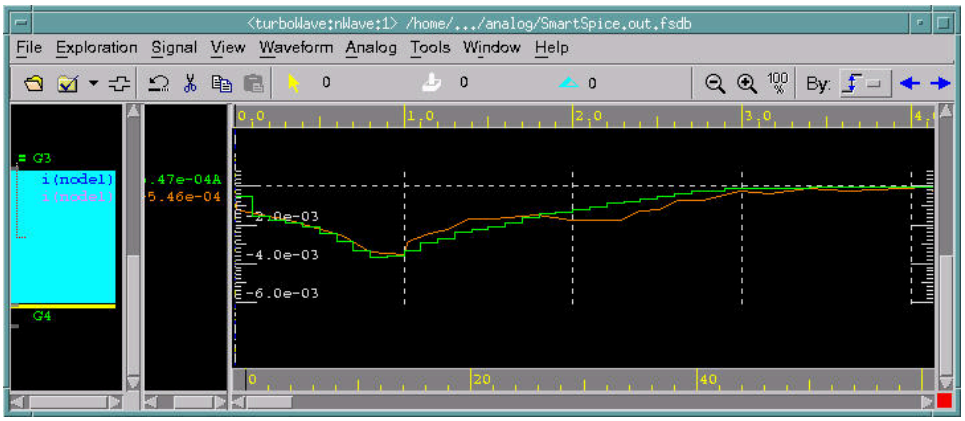
    You should now see two simulation results in the same window.

14. If you want to add more signals from the first open file, choose the **File -> Set Active** command to switch the current active file. *nWave* places no logical limit on the number of files you can open.

Overlap Analog Signals From Different Simulation Results

1. Set the signal cursor under group *G3*.
2. Select the signal *i (node1)* from group *PwrMill* and the signal *i (node1)* from group *SmtSpice* (hold the <Ctrl> key to select multiple non-contiguous signals).
3. Choose the **Signal -> Overlay** command to overlap the two signals.
4. Choose the **Analog -> Ruler** command to turn on the ruler.

Figure 509 Overlapped Analog Waveforms



Now you can analyze the differences between these two signals.

B

Appendix B: Supported FSM Coding Styles

This appendix covers the overview and FSM coding styles supported by Verdi platform:

:

- Overview
- One-Process Always
- Two-Process Always
- One-Hot Encoding
- Shift Arithmetic Operation
- Case-Statement vs. If-Statement
- Gate-Like FSM
- Next_State = signal
- Next_State = Current_State + N
- VHDL Record Type

Overview

Finite State Machine (FSM) Coding is very common in RTL design. The Verdi platform extracts FSM from the source code automatically and provides a visual state diagram and state animation to trace whole FSM actions. It is very helpful for an IC designer to analyze and debug an RTL design, especially for large FSMs. There are various kinds of FSM coding styles.

Feedback

One-Process Always

The definition of a one-process FSM is that all of its functions are specified in one VHDL process or one Verilog `always` statement. The following sections contain examples of one-process FSMs:

- Example 1 - Verilog (one_process.v)
- Example 2 - VHDL (one_process.vhd)

The functions of these two FSMs are equal. In *nState*, the state diagrams of these two FSMs are identical.

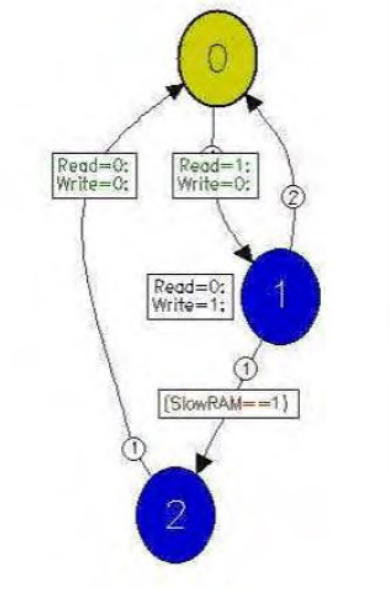
Example 1 - Verilog (one_process.v)

```
module FSM1_BAD (Clock, SlowRAM, Read, Write);
      input  Clock, SlowRAM;
      output Read, Write;
      reg Read, Write;
      integer State;
   always @(posedge Clock)
     begin: SEQ_AND_COMB
       case (State)
         0 :
           begin
             Read  = 1;
             Write = 0;
             State = 1;
           end
         1 :
           begin
             Read  = 0;
             Write = 1;
             if (SlowRAM == 1)
               State = 2;
             else
               State = 0;
           end
         2 :
           begin
             Read  = 0;
             Write = 0;
             State = 0;
           end
       endcase
     end
 endmodule
```

Refer to Verilog (one_process.v) for the *nState* diagram.

Feedback

Figure 510 Verilog (one_process.v)



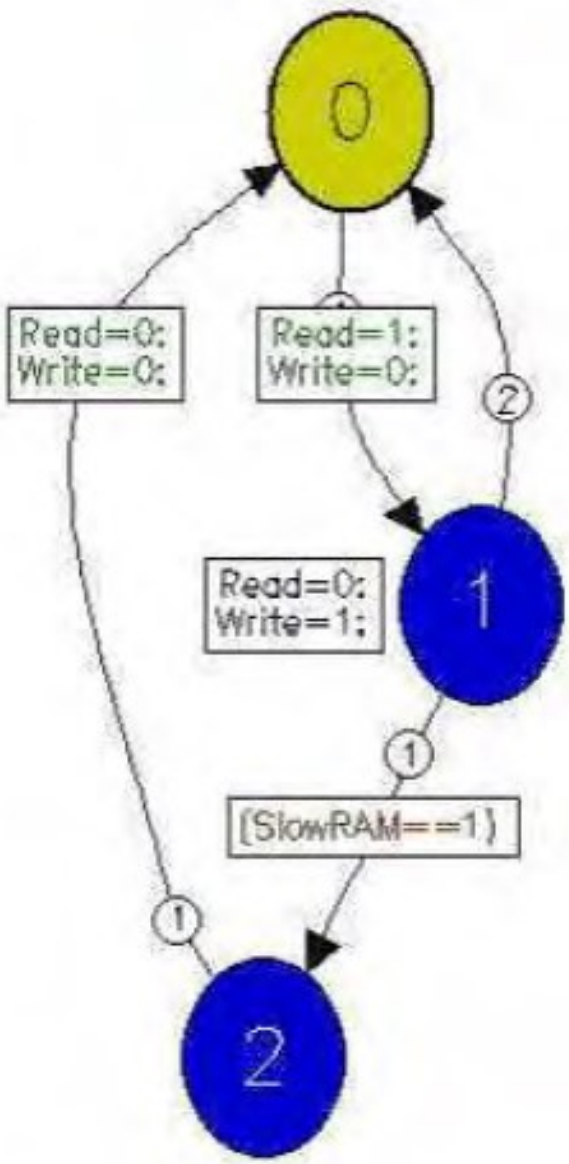
Example 2 - VHDL (one_process.vhd)

```
Library IEEE;
Use IEEE.STD_Logic_1164.all;
Entity FSM1_BAD is
   port (Clock:      in  std_logic;
         SlowRAM:    in  std_logic;
         Read,Write: out std_logic);
End entity FSM1_BAD;
Architecture RTL of FSM1_BAD is
Begin
    SEQ_AND_COMB: process
            variable State: integer;
    begin
      wait until rising_edge(Clock);
      case State is
         when 0=>
            Read  <= '1';
            Write <= '0';
            State := 1;
         When 1=>
            Read  <= '0';
            Write <= '1';
            if (SlowRAM = '1') then
               State := 2;
            Else
               State := 0;
            end if;
```

```
            when 2=>
               Read  <= '0';
               Write <= '0';
               State := 0;
            When others=> null;
         end case;
      end process SEQ_AND_COMB;
   end architecture RTL;
```

Refer to VHDL (one_process.vhd) for the *nState* diagram.

Figure 511 VHDL (one_process.vhd)



Two-Process Always

In two-process FSM, the FSM is split into a combinational circuit and a sequential circuit. The combinational circuit of the FSM is written in one process statement and the sequential circuit is written in the other process statement. Synopsys strongly

Feedback

recommends using this type of FSM. The following sections contain examples of two-process FSMs:

- Example 1 - Verilog (two_process.v)
- Example 2 - VHDL (two_process.vhd)

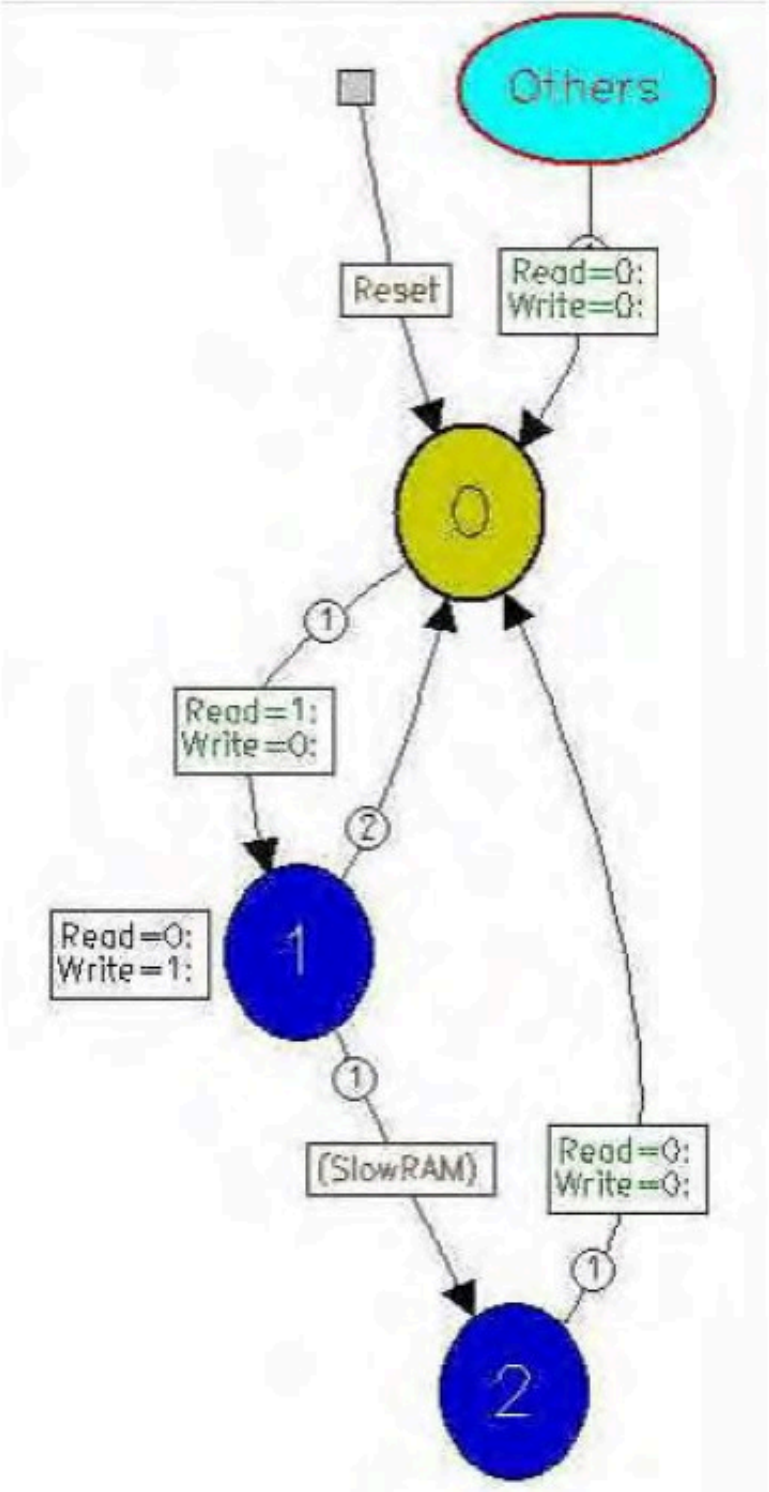
Example 1 - Verilog (two_process.v)

```
module FSM1_GOOD (Clock, Reset, SlowRAM, Read, Write);
      input  Clock, Reset, SlowRAM;
      output Read, Write;
      reg Read,Write;
      reg  [1:0] CurrentState, NextState;
 always @(posedge Clock)
     begin: SEQ
       if (Reset)
         CurrentState = 0;
       else
         CurrentState = NextState;
     end
 always @(CurrentState or SlowRAM)
     begin: COMB
       case (CurrentState)
         0 :
           begin
             Read  = 1;
             Write = 0;
             NextState = 1;
           end
         1 :
           begin
             Read  = 0;
             Write = 1;
             if (SlowRAM)
               NextState = 2;
             else
               NextState = 0;
           end
         2 :
           begin
             Read  = 0;
             Write = 0;
             NextState = 1;
           end
         default :
           begin
             Read  = 0;
             Write = 0;
             NextState = 0;
           end
```

```
        endcase
      end
endmodule
```

Refer to Verilog (two_process.v) for the *nState* diagram.

Figure 512 Verilog (two_process.v)



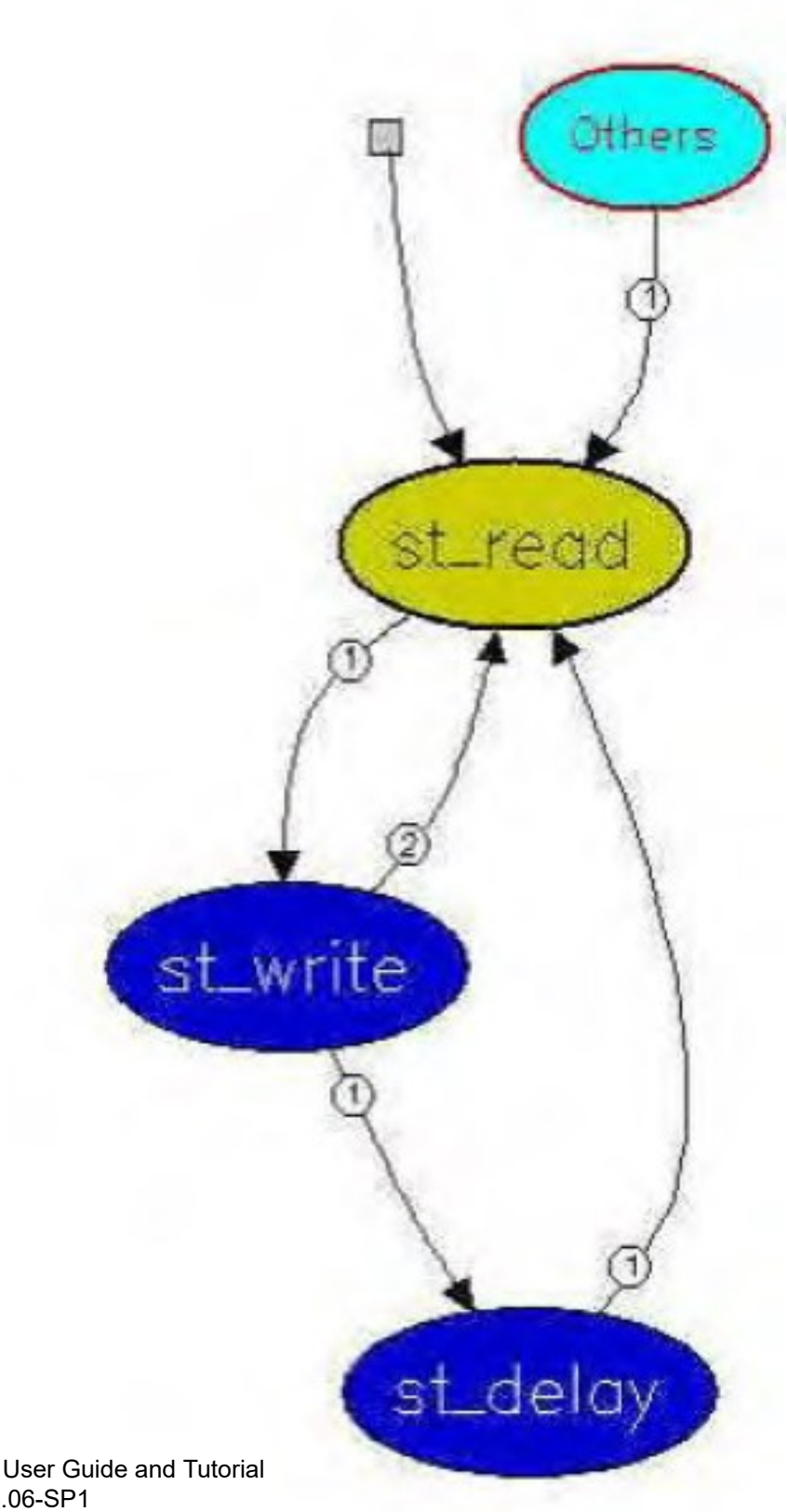
Feedback

Example 2 - VHDL (two_process.vhd)

```
library IEEE;
use IEEE.STD_Logic_1164.all;
entity FSM1_GOOD is
   port (Clock, Reset: in  std_logic;
         SlowRAM:      in  std_logic;
         Read, Write:  out std_logic);
end entity FSM1_GOOD;
architecture RTL of FSM1_GOOD is
   type StateType is (ST_Read, ST_Write, ST_Delay);
   signal CurrentState,NextState: StateType;
begin
    SEQ: process
   Begin
      wait until rising_edge(Clock);
         if (Reset = '1') then
            CurrentState <= ST_Read;
         Else
            CurrentState <= NextState;
         end if;
   end process SEQ;
   COMB: process (CurrentState)
   Begin
      case CurrentState is
         when ST_Read =>
            Read  <= '1';
            Write <= '0';
            NextState <= ST_Write;
         when ST_Write =>
            Read  <= '0';
            Write <= '1';
            if (SlowRAM = '1') then
               NextState <= ST_Delay;
            Else
               NextState <= ST_Read;
            end if;
         when ST_Delay =>
            Read  <= '0';
            Write <= '0';
            NextState <= ST_Read;
         when others =>
            Read  <= '0';
            Write <= '0';
            NextState <= ST_Read;
      end case;
    end process COMB;
end architecture RTL;
```

Refer to VHDL (two_process.vhd) for the *nState* diagram.

Figure 513 VHDL (two_process.vhd)



Feedback

One-Hot Encoding

State-encoding is the way in which binary numbers are assigned to states. The different state encoding formats commonly used are sequential, gray, johnson, one-hot, and define-your-own. The Verdi platform supports sequential, gray, johnson, and one-hot. However, one-hot is written in a different manner than the other formats. An example of Verilog one-hot format is as follows:

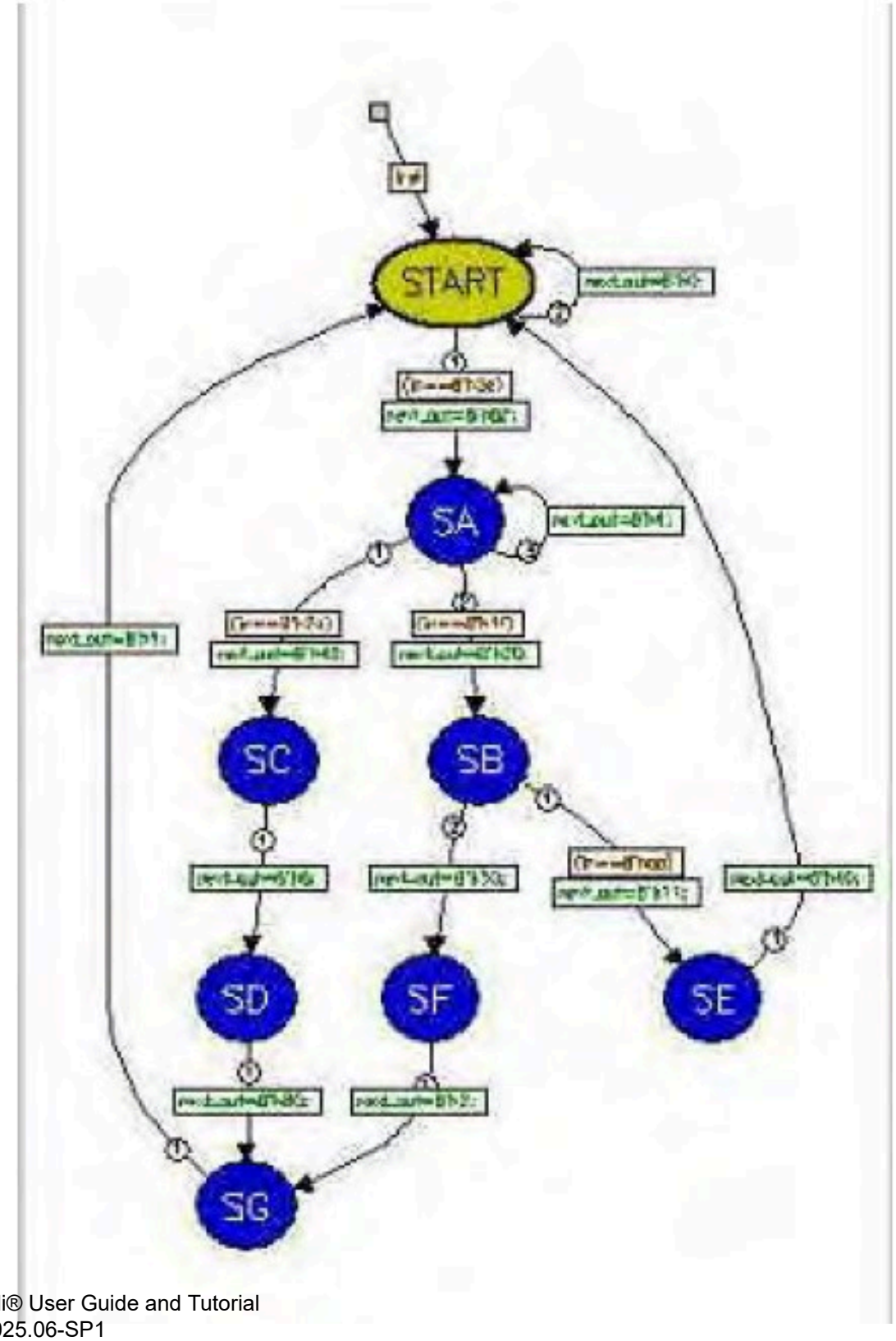
```
module prep3 (clk, rst, in, out) ;
input clk, rst ;
input [7:0] in ;
output [7:0] out ;
parameter [2:0]
    START = 0 ,   SA    = 1 ,
    SB    = 2 ,   SC    = 3 ,
    SD    = 4 ,   SE    = 5 ,
    SF    = 6 ,   SG    = 7 ;
reg [7:0] state, next_state ;
reg [7:0] out, next_out ;
always @ (in or state) begin
    // default values
    next_state = 8'b0 ;
    next_out = 8'bx ;
    case (1'b1) // synopsys parallel_case full_case
    state[START]:
        if (in == 8'h3c) begin
            next_state[SA] = 1'b1 ;
            next_out = 8'h82 ;
            end
        else begin
            next_state[START] = 1'b1 ;
            next_out = 8'h00 ;
            end
    state[SA]:
        case (in) // synopsys parallel_case full_case
            8'h2a:
                begin
                next_state[SC] = 1'b1 ;
                next_out = 8'h40 ;
                end
            8'h1f:
                begin
                next_state[SB] = 1'b1 ;
                next_out = 8'h20 ;
                end
            default:
                begin
                next_state[SA] = 1'b1 ;
                next_out = 8'h04 ;
                end
     endcase
```

Feedback

```
        state[SB]:
            if (in == 8'haa) begin
                next_state[SE] = 1'b1 ;
                next_out = 8'h11 ;
                end
            else begin
                next_state[SF] = 1'b1 ;
                next_out = 8'h30 ;
                end
        state[SC]:
            begin
            next_state[SD] = 1'b1 ;
            next_out = 8'h08 ;
            end
        state[SD]:
            begin
            next_state[SG] = 1'b1 ;
            next_out = 8'h80 ;
            end
        state[SE]:
            begin
            next_state[START] = 1'b1 ;
            next_out = 8'h40 ;
            end
        state[SF]:
            begin
            next_state[SG] = 1'b1 ;
            next_out = 8'h02 ;
            end
        state[SG]:
            begin
            next_state[START] = 1'b1 ;
            next_out = 8'h01 ;
            end
        endcase
        end
    // build the state flip-flopsalways
      always @(posedge clk or negedge rst)
        begin
        if (!rst) begin
            state <= #1 8'b0 ;
            state[START] <= #2 1'b1 ;
            end
        else
            state <= #1 next_state ;
        end
        endmodule
```

Refer to Figure 514 for the *nState* diagram.

Figure 514 One-Hot State-Encoding FSM



Feedback

Shift Arithmetic Operation

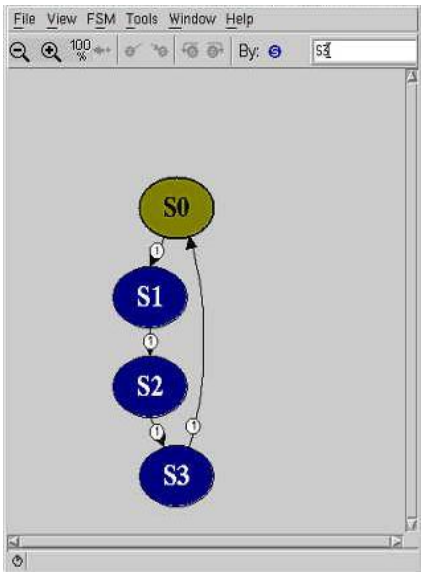
FSMs with the next state using simple shift-left arithmetic operations are supported, which is another way to specify one-hot FSM transitions.

An example of Verilog shift arithmetic format is as follows:

```
module sig_control(clock);
parameter
    S0 = 0,
    S1 = 1,
    S2 = 2,
    S3 = 3;
input clock;
reg [3:0] state;
always @(posedge clock)
begin
      case (1'b1)
         state[S0]:
          begin
             state = 1<<S1;
          end
         state[S1]:
          begin
             state = 1<<S2;
          end
         state[S2]:
          begin
             state = 1<<S3;
          end
         state[S3]:
          begin
             state = 1<<S0;
          end
      endcase
end
endmodule
```

Refer to Figure 515 for the *nState* diagram.

Figure 515 FSM Using Shift Arithmetic Operation



Case-Statement vs. If-Statement

Designers often use a `case` statement to specify the relationship between the current state and next state. The Verdi platform also allows designers to use `if` statements to do this, even for the conditional operator.

Two Verilog examples are as follows:

Example 1

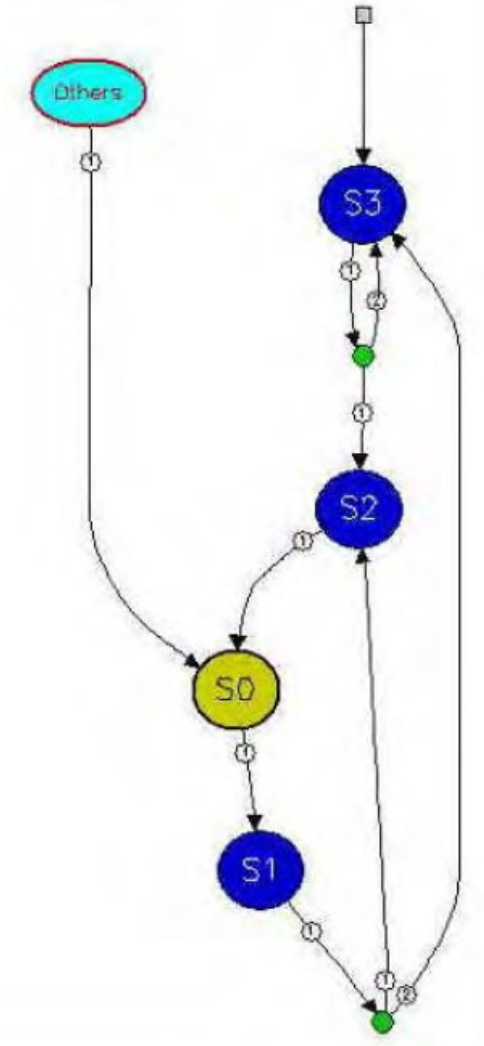
```
module FSM_1ProcIf;
wire clk, rst;
wire a, b;
reg [1:0] cs, ns;
parameter [1:0] S0=2'b00, S1=2'b01,
                S2=2'b10, S3=2'b11;
always @(posedge clk or posedge rst or a or cs)
begin
    if (rst)
        cs=S3;
else
    if (cs==S0 && a)
            cs=S1;
        else
            if (cs==S1)
                if (b)
                    cs=S2;
```

Feedback

```
                else
                    cs=S3;
            else
                if (cs==S2)
                    cs=S0;
                else
                   if (cs==S3)
                        if (a & b)
                            cs=S2;
                        else
                            cs=cs;
                    else
                        cs=S0;
end
endmodule
```

Refer to FSM Using if Statement - 1 for the *nState* diagram.

Figure 516 FSM Using if Statement - 1



Example 2

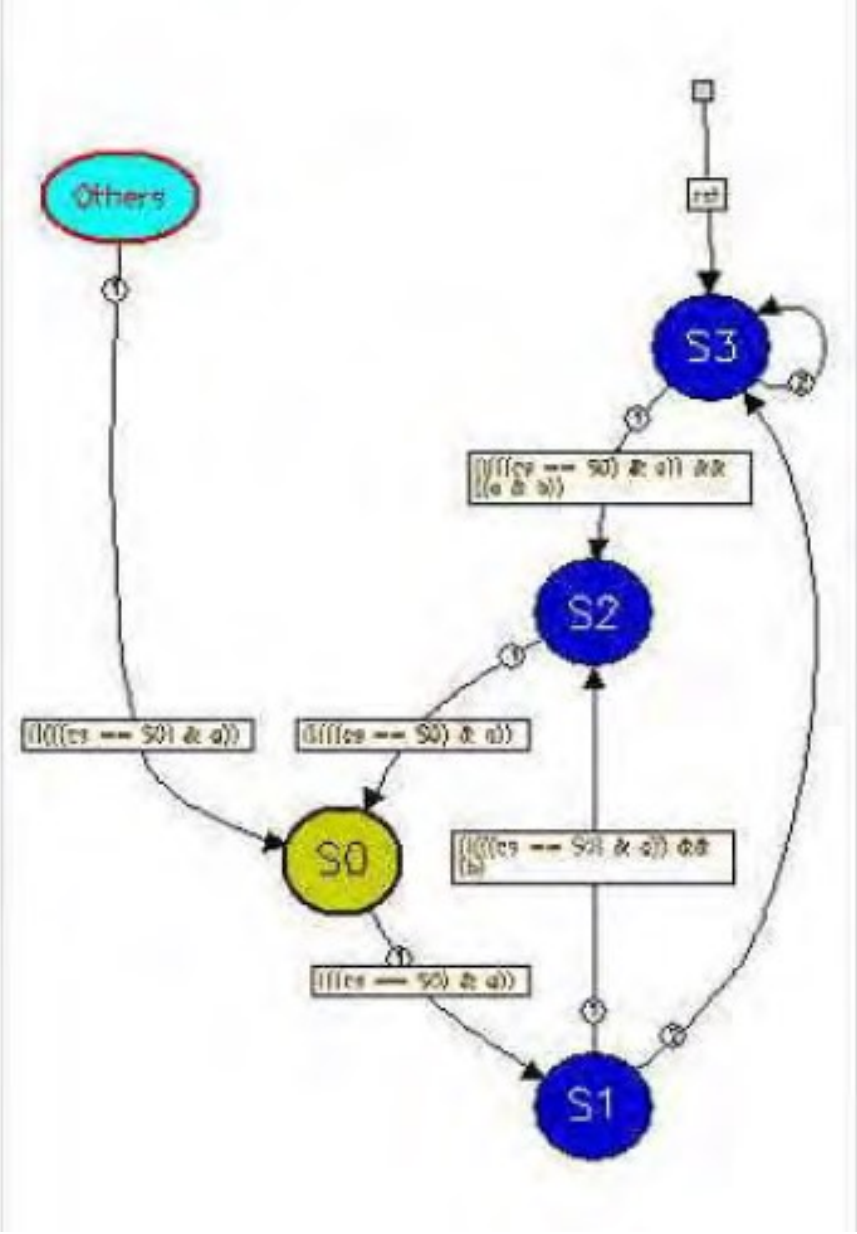
```
module FSM_2Passign;
wire clk, rst;
wire a, b;
reg [1:0] cs;
wire [1:0] ns;
parameter [1:0] S0=2'b00, S1=2'b01,
        S2=2'b10, S3=2'b11;
always @(posedge clk or rst)
if (rst)
    cs=S3;
else
```

Feedback

```
    cs = ns;
assign ns = ((cs == S0) & a) ? S1 :
    (cs == S1) ? ((b) ? S2 : S3) :
    (cs == S2) ? S0 :
    (cs == S3) ? ((a & b) ? S2 : S3) : S0;
endmodule
```

Refer to FSM Using if Statement - 2 for the *nState* diagram.

Figure 517 FSM Using if Statement - 2



Feedback

Gate-Like FSM

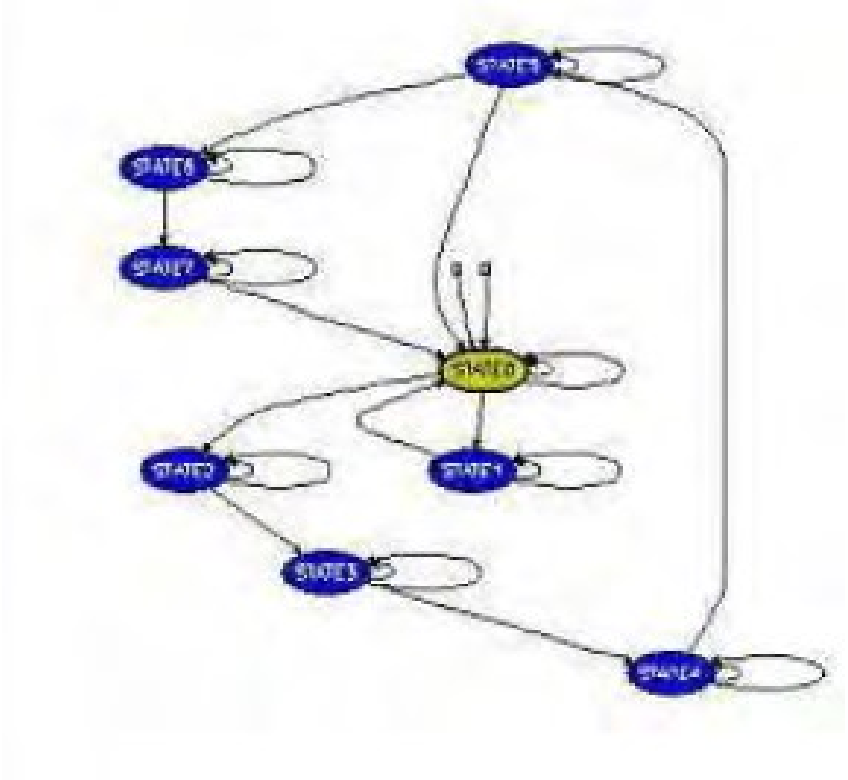
Gate-like FSM is a special type of FSM. Some high performance ASIC vendors prefer this type of FSM because designers perform the optimization themselves, writing it as an RTL statement, as shown in the following example:

```
module state_machine (clk,a,b,c,d,e,f,g,h,i,j,k,l);
input clk,a,b,c,d,e,f,g,h,i,j,k,l;
output STATE0,STATE1,STATE2,STATE3,STATE4, STATE5,STATE6,STATE7;
wire STATE0,STATE1,STATE2,STATE3, STATE4,STATE5,STATE6,STATE7;
wire NSTATE0,NSTATE1,NSTATE2,NSTATE3, NSTATE4,NSTATE5,NSTATE6,NSTATE7;
assign NSTATE0 = (STATE5 & !a & b & !c) |
                 (STATE0 & !d & !e) |
                 (STATE7 & f  & !c) |
                 (STATE1 & !g & !c) |
                 (STATE0 & c) |
                          (h) |
                          (i);
assign NSTATE1 = (STATE0 & !i & !h & d & !c) |
                          (STATE1 & !i & !h & g) |
                          (STATE1 & !i & !h & c);
assign NSTATE2 = (STATE0 & !i & !h & !d &
                         e & !c) |(STATE2 & !i & !h & c) |
                          (STATE2 & !i & !h& !j);
assign NSTATE3 = (STATE2 & !i & !h & !c & j) |
                          (STATE3 & !i & !h & !k) |
                          (STATE3 & !i & !h & c);
assign NSTATE4 = (STATE3 & !i & !h & k & !c) |
                          (STATE4 & !i & !h & !f) |
                          (STATE4 & !i & !h & c);
assign NSTATE5 = (STATE4 & !i & !h & f & !c) |
                          (STATE5 & !i & !h & !b) |
                          (STATE5 & !i & !h & c);
assign NSTATE6 = (STATE5 & !i & !h & a & b
                          & !c) | (STATE6 & !i & !h & !l) |
                          (STATE6 & !i & !h & c);
assign NSTATE7 = (STATE6 & !i & !h & l & !c) |
                          (STATE7 & !i & !h & !f) |
                          (STATE7 & !i & !h & c);
        sffp #(1)
STATE0l(.ck(clk), .d(NSTATE0), .q(STATE0));
        sffp #(1) STATE1l(.ck(clk), .d(NSTATE1), .q(STATE1));
        sffp #(1) STATE2l(.ck(clk), .d(NSTATE2), .q(STATE2));
        sffp #(1) STATE3l(.ck(clk), .d(NSTATE3), .q(STATE3));
        sffp #(1) STATE4l(.ck(clk), .d(NSTATE4), .q(STATE4));
        sffp #(1) STATE5l(.ck(clk), .d(NSTATE5), .q(STATE5));
        sffp #(1) STATE6l(.ck(clk), .d(NSTATE6), .q(STATE6));
        sffp #(1) STATE7l(.ck(clk), .d(NSTATE7), .q(STATE7));
endmodule
module sffp(ck, q, d);
        parameter width = 1;
```

```
          parameter init = {width {1'b0}};
          output [width-1:0] q;
          input ck;
          input [width-1:0] d;
          reg [width-1:0] q;
       reg [width-1:0] m;
          always @(posedge ck) q <= m;
          always @(negedge ck) m <= d;
endmodule
```

Refer to Figure 518 for the *nState* diagram.

Figure 518 Gate-like FSM



Next_State = signal

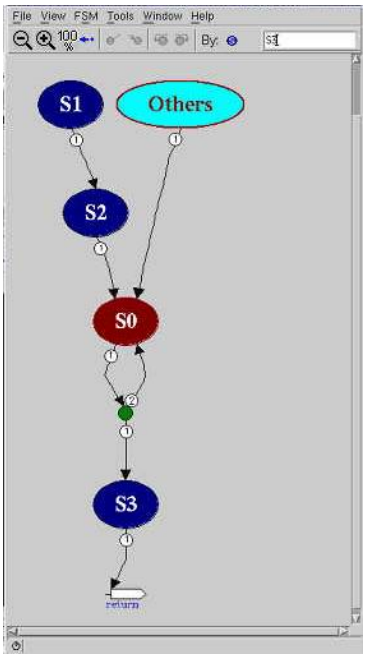
Usually, FSMs are written as `next_state = constant_value`. If `next_state = signal`, the signal's value cannot be determined. This kind of statement is created as a transition to a special bundle node, which means that it may have many indeterminable transitions. The following example shows this type of FSM:

```
module sig_control(clock);
parameter
```

```
    S0 = 2'h1,
    S1 = 2'h2,
    S2 = 2'h3,
    S3 = 2'h0;
input clock;
reg [3:0] current_state;
reg reset,enable;
reg [3:0] return;
always @(posedge clock)
begin
   if (reset)
      return <= S0;
   else
   begin
      case (current_state)
     S0: begin
          return <= S1;
          if (enable)
           current_state <= S3;
          else
           current_state <= S0;
         end
     S1: current_state <= S2;
     S2: current_state <= S0;
     S3: current_state <= return;
     default: current_state<= S0;
      endcase
   end
end
endmodule
```

Refer to Figure 519 for the *nState* diagram.

Figure 519 next_state = signal FSM



Next_State = Current_State + N

For a one process state machine, you may write `current_state = next_state` within the sequential circuit part, and `next_state = current_state + N` (N: positive integer) within the combinational circuit part. The value of `next_state` value is computed automatically.

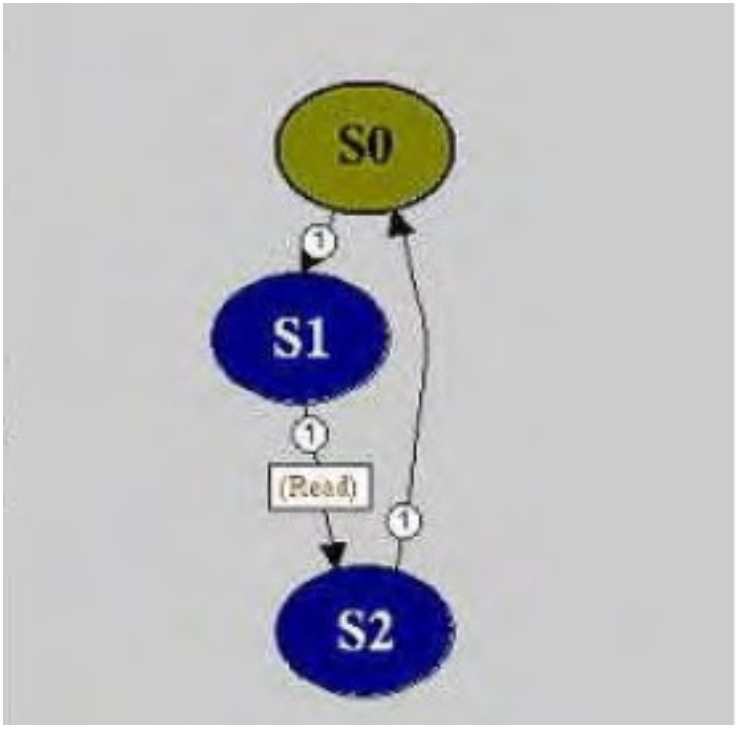
```
module FSM(Clock,Read,Write);
   input Clock;
   output Read,Write;
   reg Read,Write;
   reg [1:0] State;
   wire [1:0] next_state;
   assign next_state = State + 1;
   always @(posedge Clock)
     begin: SEQ_AND_COMB
     parameter S0=0,S1= 1,S2=2,S3=3;
     case (State)
     S0:
        State = next_state;
     S1:
      begin
        if (Read)
             State = next_state;
        end
     S2:
```

Feedback

```
          State = S0;
       endcase
     end
endmodule
```

Refer to Figure 520 for the *nState* diagram.

Figure 520 next_state = current_state + N FSM



VHDL Record Type

The FSM extractor in the Verdi platform supports the VHDL `record` type. Usually, the register is the whole `record`, for example:

```
process
begin
   wait until rising_edge(Clock);
   r <= v;
end process SEQ;
process
begin
   case r.state is
        IDLE=> v.state <= WAIT; -- state transitions
         ....
   endcase
end
```

Feedback

Note:
One-hot coding style using `record` is not supported.

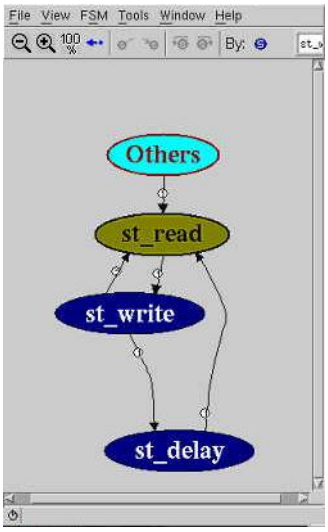
```
library IEEE;
use IEEE.STD_Logic_1164.all;
package RecordTypes is
   type StateType is (ST_Read, ST_Write, ST_Delay);
   type R1_Type is record
      State: StateType;
      Output:std_logic;
   end record;
end package RecordTypes;
library IEEE;
use IEEE.STD_Logic_1164.all,
IEEE.Numeric_STD.all;
use work.RecordTypes.all;
entity FSM2_GOOD is
   port (Clock: in  std_logic;
         SlowRAM:      in  std_logic;
         Read, Write:  out std_logic);
end entity FSM2_GOOD;
architecture RTL of FSM2_GOOD is
signal r,v: R1_Type;
begin
   SEQ: process
   begin
      wait until rising_edge(Clock);
            r <= v;
   end process SEQ;
   COMB: process (r)
   begin
      case r.State is
         when ST_Read =>
            Read  <= '1';
            Write <= '0';
         v.Output <= '1';
            v.State <= ST_Write;
         when ST_Write =>
            Read  <= '0';
            Write <= '1';
         v.Output <= '1';
            if (SlowRAM = '1') then
               v.State <= ST_Delay;
            else
               v.State <= ST_Read;
            end if;
         when ST_Delay =>
            Read  <= '0';
            Write <= '0';
         v.Output <= '1';
            v.State <= ST_Read;
         when others =>
            Read  <= '0';
```

Feedback

```
                Write <= '0';
              v.Output <= '0';
                 v.State <= ST_Read;
          end case;
       end process COMB;
    end architecture RTL;
```

Refer to Figure 521 for the *nState* diagram.

Figure 521 VHDL Record Type



C

Appendix C: Enhanced RTL Extraction

This appendix covers the overview and examples of the enhanced detail RTL extractions:

- Overview
- Instance Array
- For Loop
- Aggregate
- Partial Bits Assignment
- Displaying Pure Memory Blocks

Overview

nSchema can display instance array, for loop statements and create detailed extracted schematic view. Typically, these complex functions are displayed with a function symbol. The RTL can be extracted to show more detailed view.

To turn on detail RTL extraction feature, choose the **Tools -> Preferences** option to open the *Preferences* form and then turn *on* the **Enable Detail RTL** option on the **RTL** page under the **Schematics** folder. With this option turned *on*, the Verdi platform extracts more RTL as follows.

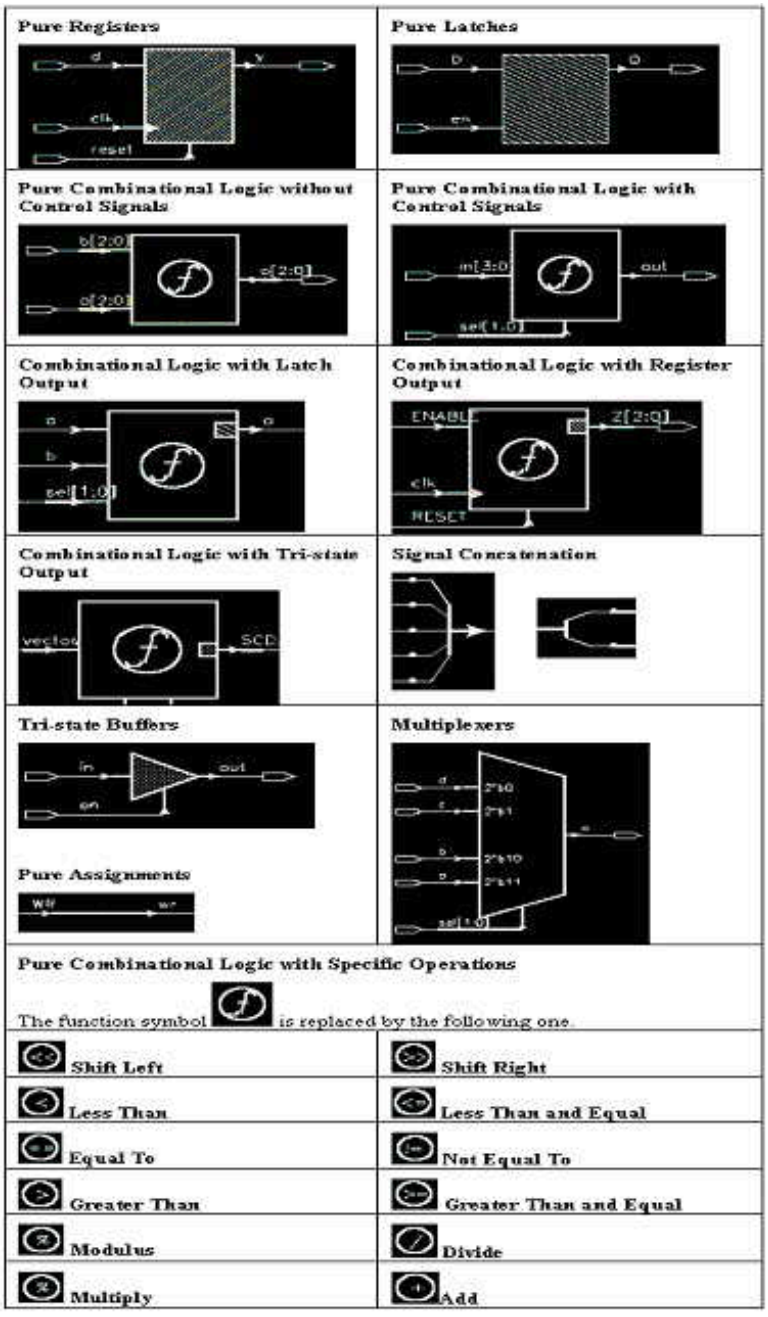
- Expand *instance array* to individual instance bit.
- Expand contents of *for loop* statement.
- Handle *aggregate* with positional notation, named notation and others.

Feedback

- For partial bit assignment, split constant, concatenation and aggregate correctly according to their specific bit range.

  The following figure shows a list of recognized RTL blocks:

Figure 522 List of Recognized RTL Blocks

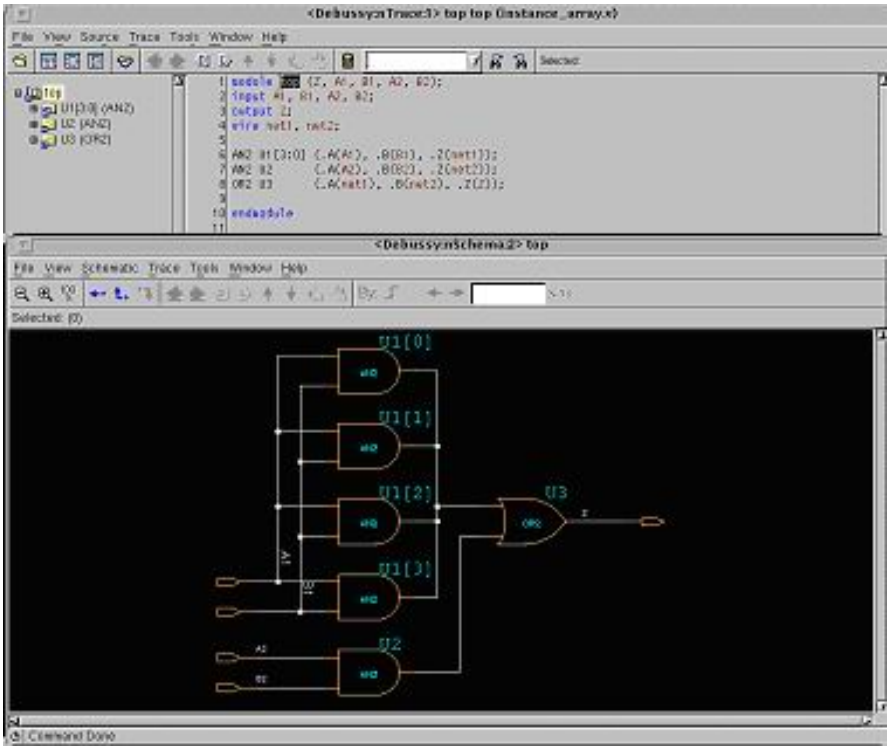


Feedback

Instance Array

The *instance array* can be expanded to individual bits. In the following example, *U1[3:0]* can be expended to *U1[0]*, *U1[1]*, *U1[2]* and *U1[3]* with correct port connections.

Figure 523 Instance Array



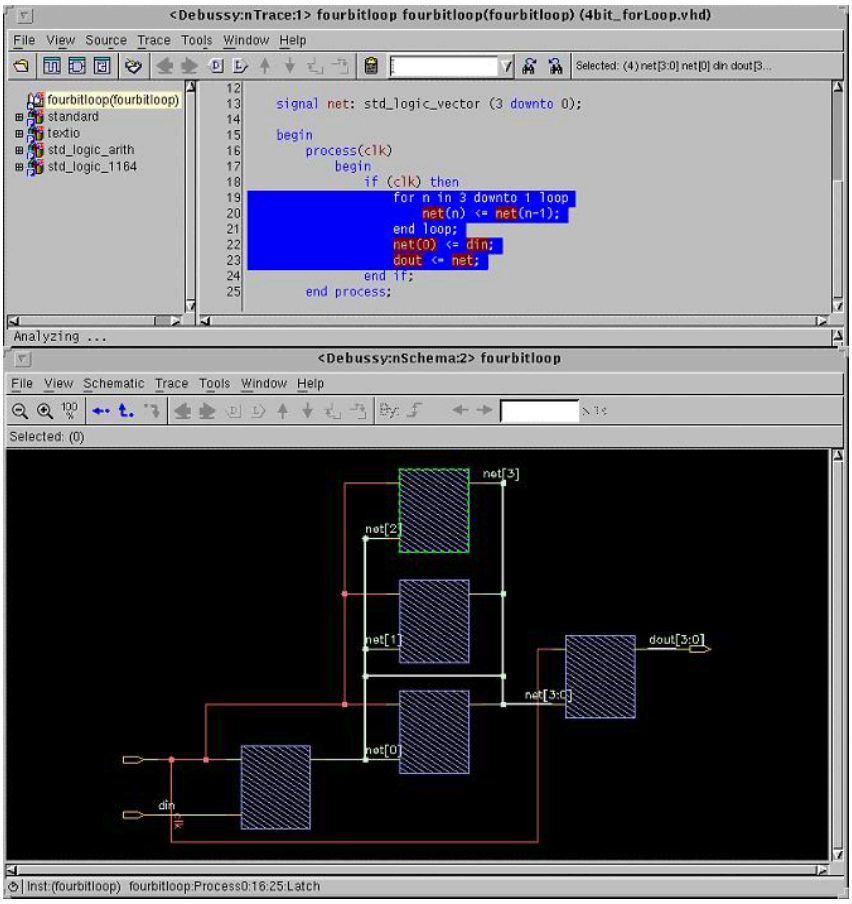
For Loop

The for loop can be expanded. In the following *for loop* example, net is expended to *net(1)*, *net(2)* and *net(3)*.

Note:
Nested for loops are not supported.

Feedback

Figure 524 For Loop



Aggregate

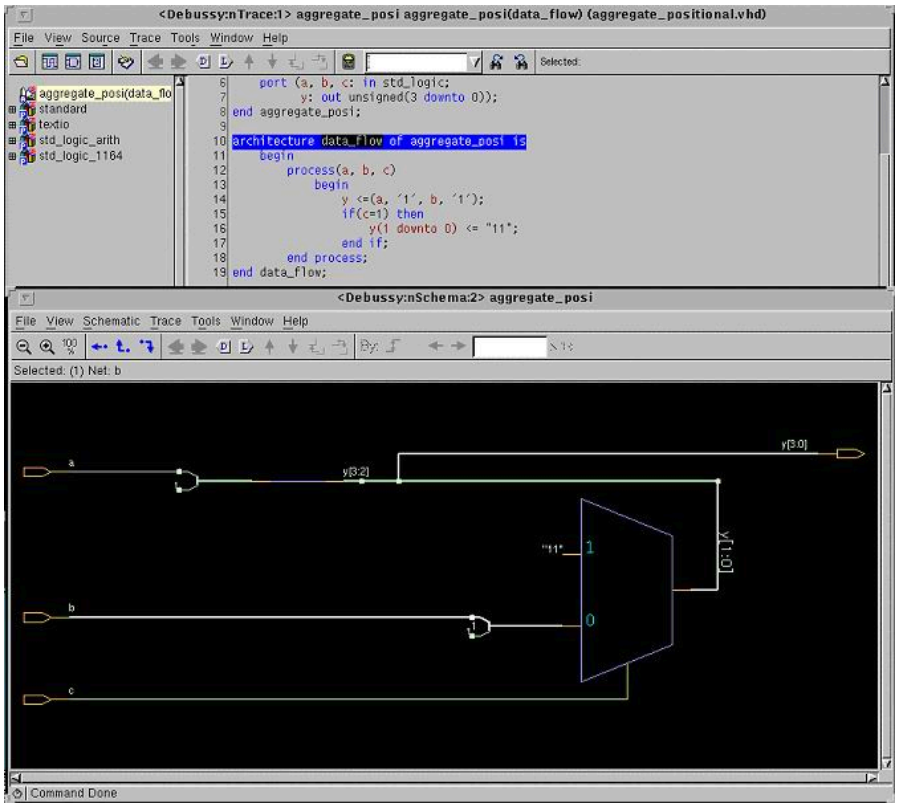
1. Positional notation: ('1', 'a', 'b')

   In the following example, the extracted RTL shows the following:

```
Y(3) <= a;
Y(2) <= '1';
If (c = '1') Y(1) <= '1' else Y(1) <= b;
If (c = '1') Y(0) <= '1' else Y(0) <= '1';
```

Feedback

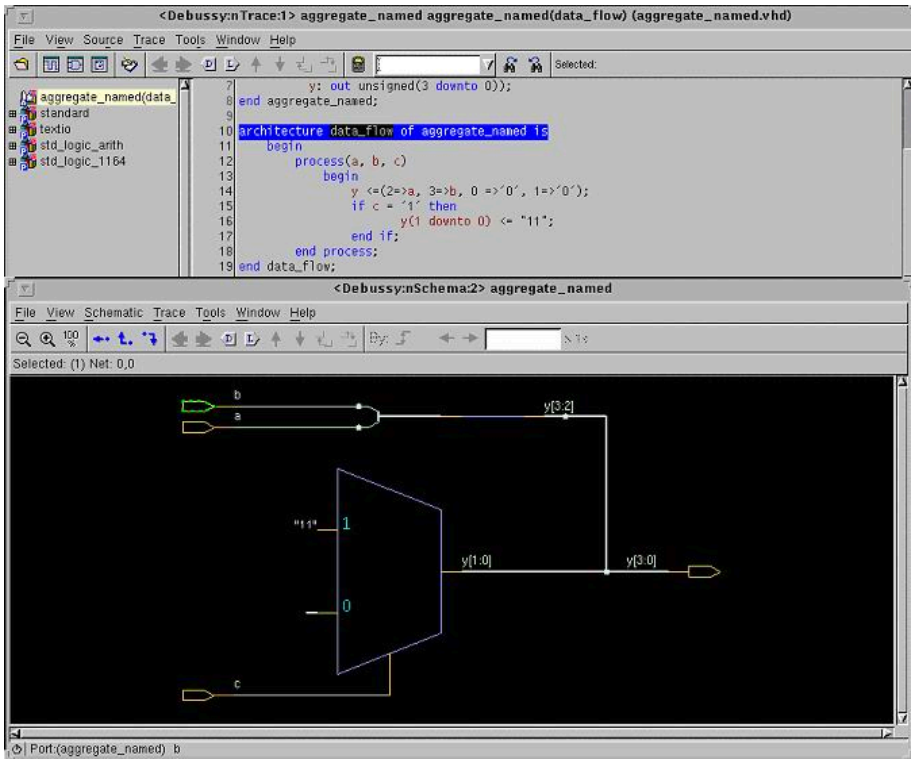
Figure 525 Aggregate: Positional Notation



2. Named notation: (2=>'a', 3=>'b', 0=>'0', 1 => '0')

Feedback

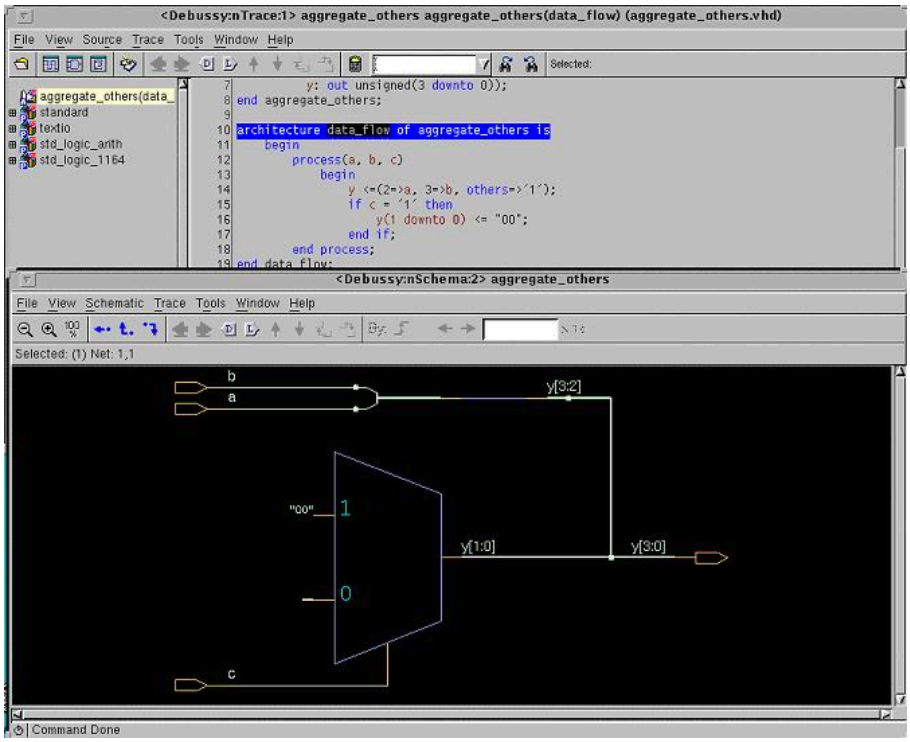
Figure 526 Aggregate: Named Notation



Feedback

3. Others: (2=>'a', 3=>'b', others=>'1')

Figure 527 Aggregate: Others

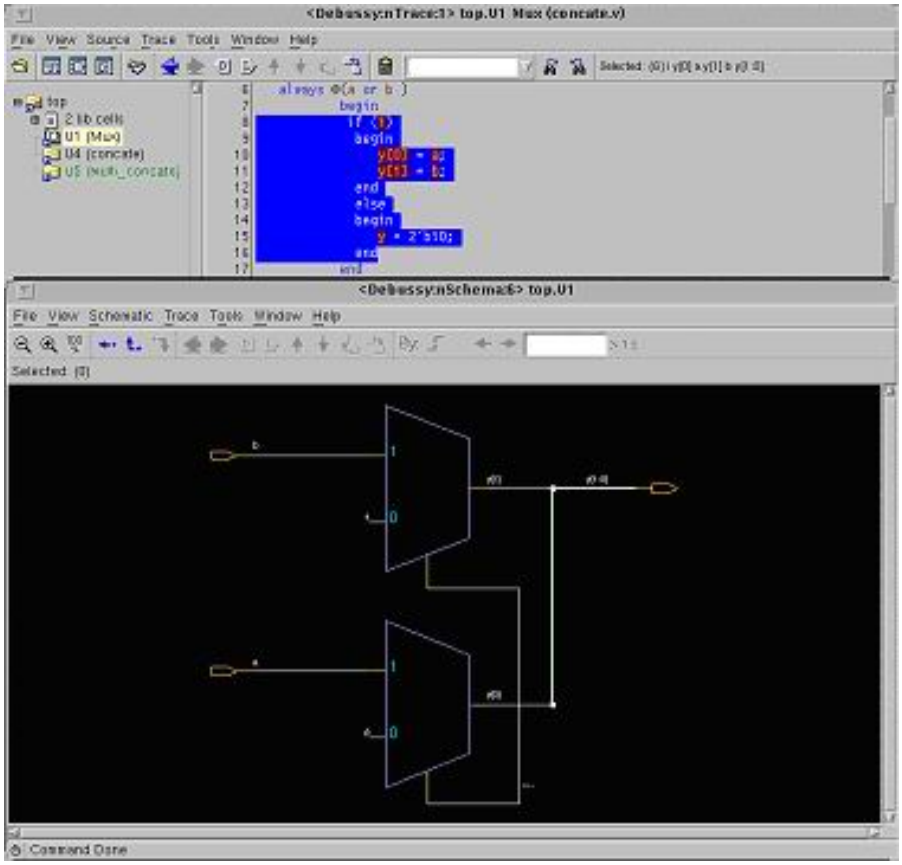


Feedback

Partial Bits Assignment

1. Constant

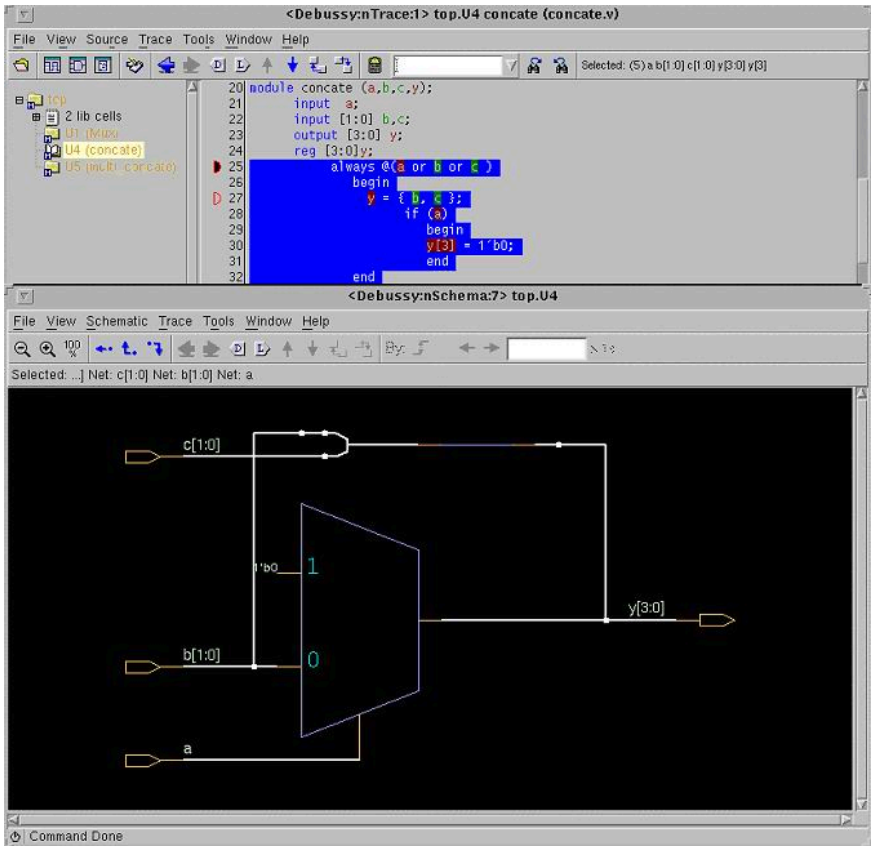
Figure 528 Partial Bits Assignment: Constant



2. Concatenation

Feedback

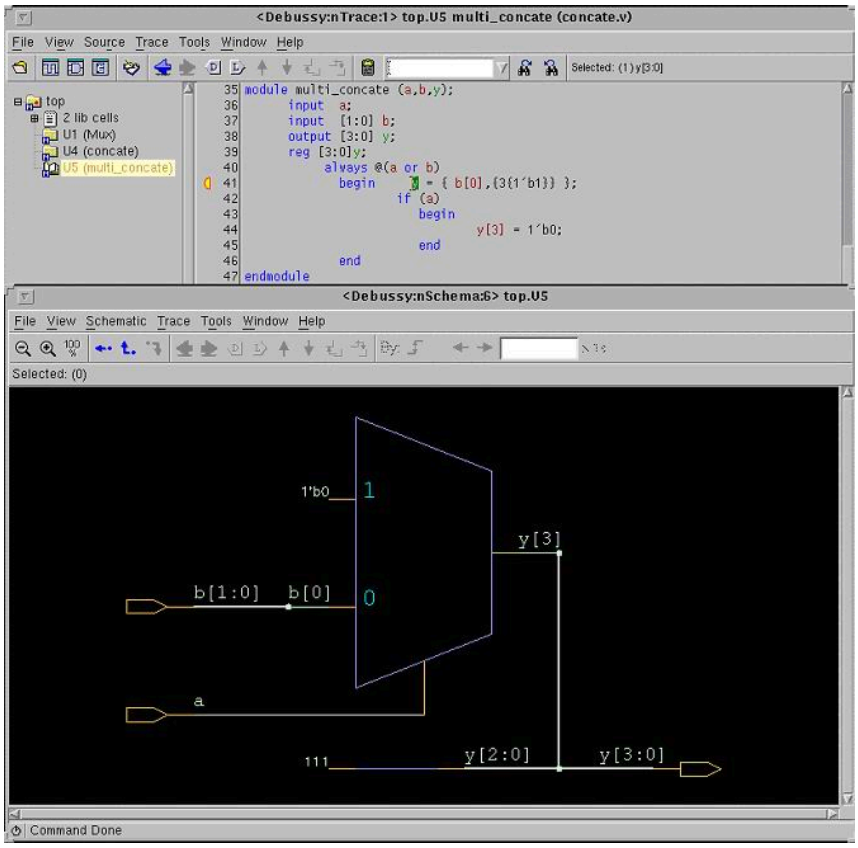
Figure 529 Partial Bits Assignment - Concatenation



Feedback

3. Multi-Concatenation

Figure 530 Partial Bits Assignment -- Multi-Concatenation



Displaying Pure Memory Blocks

To show the pure memory block in *nSchema*, the memory block is separated from other circuits.

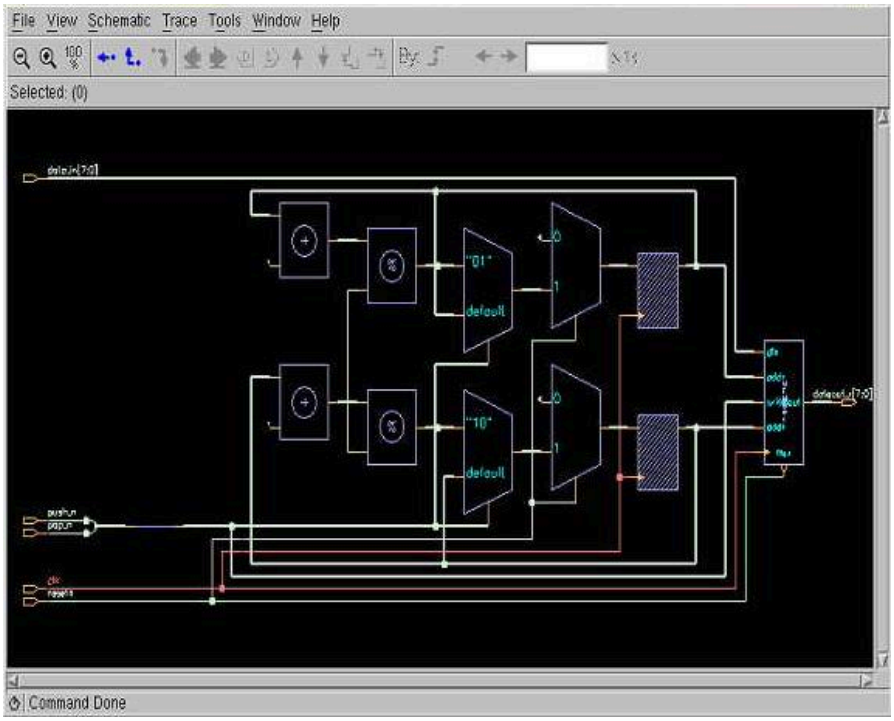
For example, in the case of the FIFO, `FIFO_r` is used to store information. `WrPntr_r/RdPntr_r` is a pointer that increases or decreases as the FIFO is pushed or popped:

```
Pointers_Proc : process is
  begin  -- process Pointers_Proc
    wait until Clk = '1';
    case PushnPopn is
      when "00" =>                      -- Push and pop at same clock
        -- no change to pointers or status
        FIFO_r(WrPntr_r) <= data_in;    -- store data
      when "01" =>                      -- Push, no pop
          FIFO_r(WrPntr_r) <= data_in;  -- store data
```

```
          WrPntr_r        <= (WrPntr_r + 1) mod Depth_g;
          -- right argument must evaluate to a constant integer power of
2
      when "10" =>                        -- no push, pop
          RdPntr_r <= (RdPntr_r + 1) mod Depth_g;
      when "11" =>                        -- no push, no pop
        null;
      when others => null;
    end case;
    if Resetn = '0' then
      WrPntr_r <= 0;
      RdPntr_r <= 0;
      FIFO_r   <= (others => (others => '0'));
    end if;
  end process Pointers_Proc;
  DataOut_r <= FIFO_r(RdPntr_r);
```

The detail RTL view of this example is shown in the following figure:

Figure 531 Memory Block



D

Appendix D: Additional Transaction Example

This appendix introduces how to extract transactions using SystemVerilog Assertions (SVA).

This appendix includes the following topic:

- Extracting Transactions Using SVA

Extracting Transactions Using SVA

SystemVerilog Assertions (SVA) are added to your design and then extracted to display as transactions.

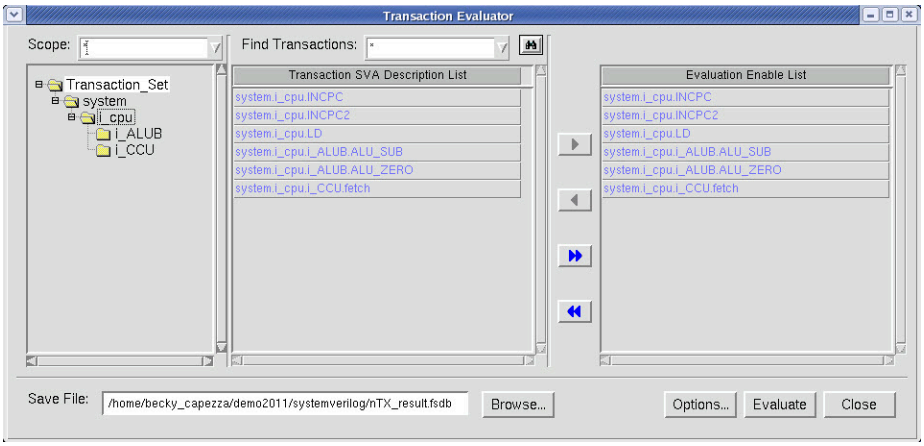
Before you can extract transactions from SVA, you must do the following:

1. Add the SVA code to your design either inlined or as a separate file.
2. Generate an FSDB file containing design data with your preferred simulator.
3. Load the design and FSDB files into the Verdi GUI.

After the design and FSDB files are loaded into the Verdi GUI, you can extract the transactions by invoking the **Tools -> Transaction -> Evaluator** command in the *nTrace* main window. This opens the *Transaction Evaluator* form where all SVA assert signals are listed.

Feedback

Figure 532 Transaction Evaluator Form



In the *Transaction Evaluator* form, the design hierarchy is displayed in the left pane. After you have traversed to the scope of interest, the transactions are listed in the middle pane. You can select the assertions to be extracted and click either the **Add Selected Transaction** button or **Add All Transactions** button to move the selection to the **Evaluation Enable List** pane. You can also drag any of the assertions to the source code frame to see the related code.

After you click **Evaluate**, the transactions are extracted from the assertion code and are saved to the specified file. This FSDB file is automatically loaded into the Verdi GUI and you can start using all transaction viewing and analysis commands for debug in addition to the standard Verdi capability.

Note:

You need to add the transaction waveforms using the **Get Signals** command of *nWave*. Transaction signals have an *_nTX* suffix appended to the assertion name.

SVA Code

The following sections contain a summary of recommended and unsupported coding styles:

- Recommended Coding Style
- Unsupported Coding Style
- Code Example

Feedback

Recommended Coding Style

The following coding styles are recommended for optimum transaction extraction results:

- Only “assert” directive is supported.
- Most SVA constructs are supported. For details, see Unsupported Coding Style.
- Using constructs, the sequence layer is recommended for modeling transactions. Deep nesting range repetition and unbound range delay, for example `##[0:$]`, are not recommended as it impacts performance.
- SVA local variables, including those declared in the sub-sequence of a specific assertion, are recorded as attributes of transactions. Therefore, it is not recommended to declare local variables with the same name across different sequences or properties.

Example 1:

```
sequence single_read;
    logic [31:0] addr;
    logic [31-1:0] data;
    int ws;

    @(posedge hclk)
        (`true,ws = 0) ## 0
        (hready) ##1
        (!hready && hsel) [*0:$] ##1
        ((hready && hsel && `SR_CTRL), addr = haddr) ##1
        ((!hready && hsel), ws = ws + 1) [*0:$] ##1
        (hready, data = hrdata);
endsequence
SINGLE_READ: assert property(single_read);
```

The `addr`, `data`, and `ws` local variables of the `single_read` sequence are recognized as the attributes of the assertion statement, `SINGLE_READ`.

Example 2:

```
sequence s1;
    int localvar;
    ...
endsequence
sequence s2;
    int localvar;
    ...
endsequence
a_trans1: assert property(@(posedge clk) s1 and s2);
```

You can modify the sequence as follows:

```
sequence s1;
    int localvar1;
    ...
```

Feedback

```
endsequence
sequence s2;
    int localvar2;
    ...
endsequence
a_trans1: assert property(@(posedge clk) s1 and s2);
```

- You can specify the transaction label name of a specific sequence by declaring a string type local variable named `label_nTX`, and assigning a label name to it. For example, if you specify the following for a sequence/property, the transaction label name is `my_single_read`:

```
sequence single_read;
string label_nTX;
(..., label_nTX = "my_single_read",...) ...;
endsequence
```

If you specify the following for an assertion statement, the `label_nTX` variable (if exists) of the sub-sequence/property is used as its transaction label:

```
sequence s1;
    string label_nTX;
    (..., label_nTX = "my_s1",...) ...;
endsequence
sequence s2;
    string label_nTX;
    (..., label_nTX = "my_s2",...) ...;
endsequence
a_s1 : assert property((@posedge clk) s1 ##1 s2);
```

In this case, the label is either `my_s1` or `my_s2`.

Unsupported Coding Style

The following coding styles are not supported for transaction extraction:

- Multiple clocking is not supported.
- Immediate assertion is not supported.
- `cover` and `assume` directives of SVA are not supported. Only the `assert` directive is supported.
- Three types of assertion successes are not recognized as a transaction:
  - The vacuous success of the implication is not recognized as a transaction.
  - The abort success of `disable iff` is not recognized as a transaction.
  - Empty matches, for example `seq1[*0];`, are not recognized as a transaction.

Feedback

Code Example

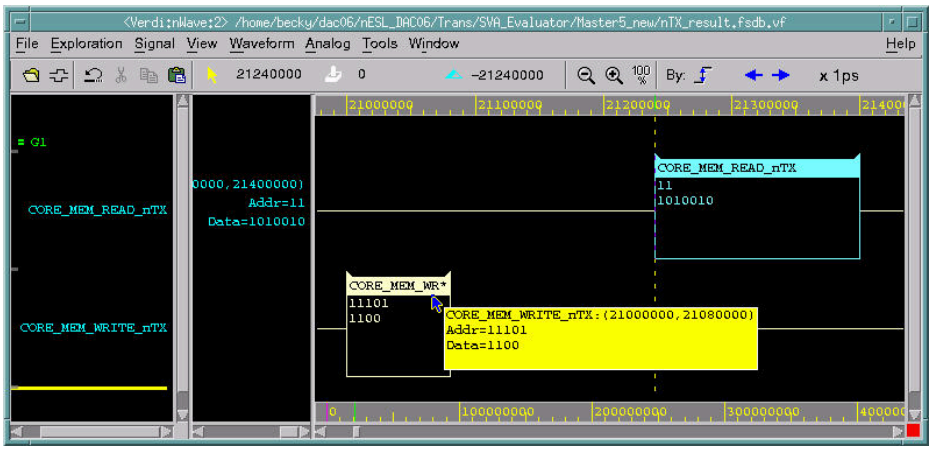
Consider the following SVA code example:

```
bind test assert_checker bind_transaction_evaluator(
    .EN (test.uFL_AMBA_SRAM.ram_2kx32.mem.EN),
    .WE (test.uFL_AMBA_SRAM.ram_2kx32.mem.WE),
    .ADDR (test.uFL_AMBA_SRAM.ram_2kx32.mem.ADDR),
    .DI (test.uFL_AMBA_SRAM.ram_2kx32.mem.DI),
    .DO (test.uFL_AMBA_SRAM.ram_2kx32.mem.DO),
    .CLK (test.uFL_AMBA_SRAM.ram_2kx32.mem.CLK),
    .RST (test.uFL_AMBA_SRAM.ram_2kx32.mem.RST),
    .RDInvalid (test.uFL_AMBA_SRAM.uSMI.iXOEN_d)
);
module assert_checker (
    input EN,
    input WE,
    input [10:0] ADDR,
    input [31:0] DI,
    output [31:0] DO,
    input CLK,
    input RST,
    input RDInvalid
);
sequence core_memory_write;
    logic [10:0] Addr;
    logic [31:0] Data;
    (1) ## 0
    (EN == 1'b1 && WE == 1'b1, Addr = ADDR, Data = DI) ##1
    (!(EN == 1'b1 && WE == 1'b1));
endsequence
sequence core_memory_read;
    logic [10:0] Addr;
    logic [31:0] Data;
    (1) ## 0
    (WE==1'b0 && RST==1'b0 && RDInvalid==1'b0, Addr = ADDR) ##1
    (RDInvalid == 1'b0) ##1
    (1, Data = DO);
endsequence
CORE_MEM_WRITE : assert property(@(posedge CLK) core_memory_write);
CORE_MEM_READ : assert property(@(posedge CLK) core_memory_read);
endmodule
```

Feedback

The example is extracted and displayed as transaction waveforms similar to the following:

Figure 533 Extracted Transaction Waveform



E

Appendix E: Integration Features

This appendix provides the information about the following native integrations that are available as part of the Verdi platform:

- Native Integration of Verdi and VCS
- Unified Transaction Debug - Verdi and Protocol Analyzer Integration
- Unified UVM Library
- Scope-Based Peak Analysis

Native Integration of Verdi and VCS

This section consists of the following native integrations that are available as part of Verdi:

- UCLI Dump Command for FSDB Dumping
- Optimized Performance of Gate-Level Designs Using Native FSDB Gate

UCLI Dump Command for FSDB Dumping

This section consists of the following subsections:

- Introduction
- Use Model
- Enhanced and New UCLI Dump Options
- Limitation

Introduction

The UCLI `dump` command is enhanced to dump the Fast Signal Database (FSDB) file in addition to the VPD and EVCD file dumping.

Now, you can use the UCLI `dump` command to dump the FSDB file by default, instead of calling the FSDB system tasks or using FSDB commands on the UCLI command prompt.

Feedback

You can also perform the following operations using the `dump` command:

- Simultaneously open single VPD, EVCD, and FSDB dump files and manage them individually.
- Simultaneously open multiple FSDB dump files and manage them individually.

Use Model

The following steps describe the use model:

1. The default dump type of VCS is VPD. You can use the following environment variable to set the default GUI as Verdi and the default dump type as FSDB

   ```
   % setenv SNPS_SIM_DEFAULT_GUI verdi
   ```

2. Set `VERDI_HOME` as provided in the following command line:

   ```
   % setenv VERDI_HOME <novas_path>
   ```

3. Compile your designs with the `-debug_access+cbk` option, as provided in the following command line:

   ```
   % vcs -debug_access+cbk <file_name>
   ```

Key Points to Note

- If a single dump file is open, it is not required to specify the `-fid` argument with the dump commands that follow the `dump -file` command. If multiple dump files are open, you must specify the `-fid` argument with the dump commands that follow the second dump `-file` command.
- During simulation, if the number of open dump files return to one, you can exclude the `-fid` argument. VCS issues an error message, if a dump command is specified without the `-fid` argument when multiple dump files are open

Enhanced and New UCLI Dump Options

Several UCLI `dump` options are enhanced and new added for dumping FSDB file. For details, see the VCS documentation.

Limitation

For details, see the VCS documentation.

Feedback

Optimized Performance of Gate-Level Designs Using Native FSDB Gate

This section consists of the following subsections:

- Introduction
- Prerequisites
- Using the FSDB Gate Feature
- Limitations

Introduction

Verdi provides the Fast Signal Database (FSDB)-Gate feature for gate-level designs without Standard Delay Format (SDF) information. You can invoke the FSDB-Gate feature using VCS, which supports optimized FSDB gate-level dumping.

To enable this feature, use the VCS `+fsdb+gate` runtime option. It directs VCS to analyze essential signals and the netlist information including the signature, function table, and partition mapping, and uses the FSDB Dumper to record this information in an FSDB file. Applications, such as Waveform Viewer and FSDB Reader, retrieve the mapping data stored in the FSDB file. The retrieved data is further used by the VCS computation engine to generate the complete signal data during debugging.

Additionally, if you enable the VCS force capability along with the `-debug_access+f +fwn` compilation option, the VCS dynamic de-aliasing capability is also enabled while dumping forced signals into the FSDB file. The FSDB Reader then interprets the event and generates the related waveform in Verdi.

The FSDB Gate and dynamic de-aliasing acceleration features reduce the FSDB file dumping size and optimize the VCS simulation time for specific coding styles and forced signal flows.

Prerequisites

These features are available starting with the following versions:

- VCS simulator 2014.12
- Verdi 2014.12
- FSDB Reader 5.2 (for user of FSDB reader API)
- If the API libraries of the FSDB Reader are used to read the FSDB file with new format, a Verdi license is required.

Feedback

Using the FSDB Gate Feature

Use the `+fsdb+gate` runtime option in the VCS simulation command line to enable these features.

For example,

```
%> ./simv +fsdb+gate
```

Alternatively, set the following environment variable before starting the simulation:

```
%> setenv FSDB_GATE 1
```

Key Points to Note

- After simulation, a new format of the FSDB file is generated.
- FSDB cannot be read by previous Verdi version, for example 2014.03.
- Expect to see a higher simulation speed in the SystemVerilog Gate-Level design without SDF.
- FSDB reading performance (CPU or memory) when using Verdi debug might be impacted.

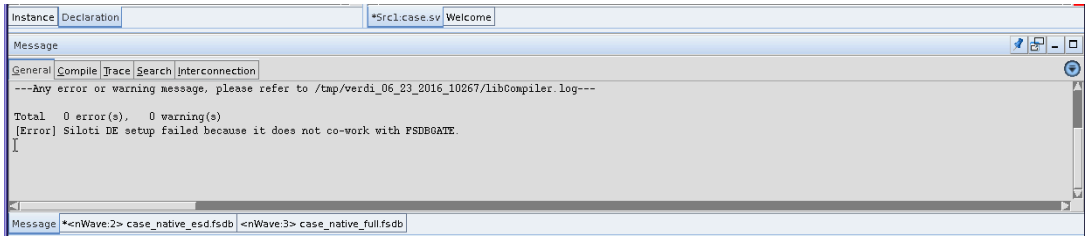
Limitations

The following are the limitations for the FSDB-Gate feature:

- The `+fsdb+gate` option is disabled with a warning message, if you add any of the following FSDB Dumper options in the simulation command line or if you specify them using the `setenv` command:
  - `+fsdb+glitch=<num>` (its corresponding environment variable is `NOVAS_FSDB_ENV_MAX_GLITCH_NUM` or `FSDB_GLITCH`): If the `<num>` argument is not equal to 1, the `+fsdb+gate` option is disabled.
  - `+fsdb+dumpon_glitch+time` and `+fsdb+dumpoff_glitch+time`
  - `+fsdb+region` (its corresponding environment variable is `FSDB_REGION`)
  - `+fsdb+sequential` (its corresponding environment variable is `NOVAS_FSDB_ENV_DUMP_SEQ_NUM`)
  - `+fsdb+strength=on` (its corresponding environment variable is `NOVAS_FSDB_STRENGTH`)
  - `+fsdb+esdb` (its corresponding environment variable is `FSDB_ESDB`)
- If the `+fsdb+gate` option is enabled, the `+strength` option in dumping tasks is ignored with a warning message.

- The FSDB Gate acceleration does not support VCS MVSIM Native flow to have the optimized performance.
- The FSDB utilities require many computations. A performance slowdown is expected when using the FSDB utilities.
- Siloti Data Expansion does not work with FSDB-GATE. The following message is displayed when a Data Expansion setup is applied on the FSDB-GATE FSDB:

Figure 534 Warning Message Displayed -Siloti Data Expansion and FSDBGATE



Unified Transaction Debug - Verdi and Protocol Analyzer Integration

This section consists of the following subsections:

- Introduction
- Use Model

Introduction

Protocol Analyzer and Verdi are now integrated for VIP to improve the productivity in protocol, transaction, and signal-level debugging. With this integration, you can directly invoke Protocol Analyzer from Verdi and the protocol-related information is automatically loaded into Protocol Analyzer. After Protocol Analyzer is launched, it gets synchronized with Verdi automatically. You can also directly invoke Verdi with the loaded FSDB file from Protocol Analyzer. This invoking mechanism reduces the time consumed in setting different configurations for Verdi and Protocol Analyzer, and in comparing the corresponding objects at different levels. This also enables more efficient protocol-level analysis and signal-level debugging of issues, and increases the productivity of the debug process.

Additionally, transaction-based FSDB can be directly generated or converted from the result of VIP simulation and the FSDB files can be used in both Protocol Analyzer and Verdi.

Feedback

The transaction debug capability offers the following features:

- Integrating Verdi and the Protocol Analyzer GUI
- Generating Transaction-Level FSDB File for VIP
- Reading Transaction-Based FSDB in Protocol Analyzer
- Loading Protocol Extension Files into Verdi

Use Model

For the details about the features, see the *Verdi Transaction and Protocol Debug User Guide*.

Unified UVM Library

The unified UVM library is provided to integrate the instrumented UVM libraries of VCS and Verdi. VCS and Verdi transaction recorder and message catcher coexist and are compiled together.

You can directly use the unified UVM library with the Verdi-provided recording mechanism during simulation, and for debugging with Verdi. Thus, accelerating the overall verification cycle.

The unified UVM library also improves the debug productivity while debugging UVM-based environments with VCS and Verdi, as both the tools use the same UVM library to eliminate the disparity between simulation and debug libraries.

Single compilation, UUM, and UVM-VMM interoperability flows are supported in the unified UVM library. The unified UVM library can also be qualified and validated using Synopsys VIPs.

Use Model

For the details about its usage, see the *Verdi UVM Debug User Guide*.

Scope-Based Peak Analysis

The scope-based peak analysis is now provided to analyze which time has the most transitions (peak analysis) based on the scope for the design and FSDB file. The report of the analysis is displayed in the scope-based table view. You can also check the result in the waveform and in a Comma Separated Values (CSV) format. After checking the report, you can further generate the What-if configuration files to perform advanced power estimation based on the checking result.

The scope-based peak analysis capability offers the following features:

- Viewing reports in the scope-based table view
- Generating waveform of certain scopes
- Saving and restoring report as an XML file
- Dumping report in the Comma Separated Values (CSV) format
- Exporting the What-if configuration file

  Note:

  - Only a trigger at the top scope of the same net is counted.
  - The following two triggered types are not counted as a trigger:
    - A value changes to **X**
    - **X** changes to a value

This section consists of the following subsections:

- Use Model
- Generating Waveform With Report Entry
- Exporting the What-if Configuration File

Use Model

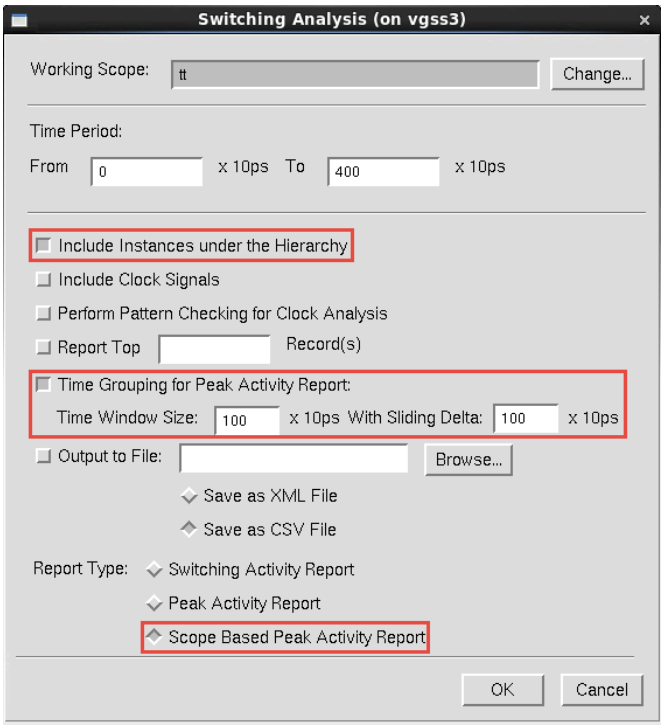
To enable this feature, specify the following environment variable before starting the Verdi platform to enable this feature:

```
%> setenv TFV_SCOPE_PEAK_ANALYSIS 1
```

After loading your design and the FSDB file, use the **Tools -> Switching Analysis -> New Query** command to invoke the *Switching Analysis* form.

Feedback

Figure 535 Switching Analysis Form



Enable the following options to configure the scope-based peak analysis for power estimation:

- **Include Instances under the Hierarchy**

  Includes sub-scopes of the specified scope to the analysis. If this option is not enabled, the analysis only includes the selected scope.

- **Time Grouping for the Peak Activity Report**

  Groups time in the report based on the time window size and sliding delta specified respectively in the **Time Window Size** and **With Sliding Delta** fields. It is strongly recommended to enable this option and specify the time window size and sliding delta.

- **Report Type: Scope Based Peak Activity Report**

  Enables to generate the scope-based peak activity report.

  You can also generate the report with the following options:

- **Output to File**

  Generates the output file with the specified file type (based on the enabled **Save as XML File** or **Save as CSV File** toggle options) for the report. The output file is generated while the analysis is completed and the report is ready.

Feedback

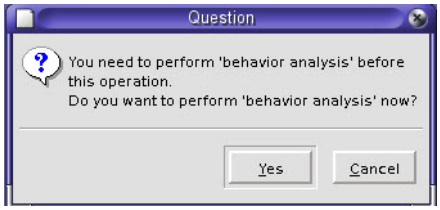
- **Save as XML File**

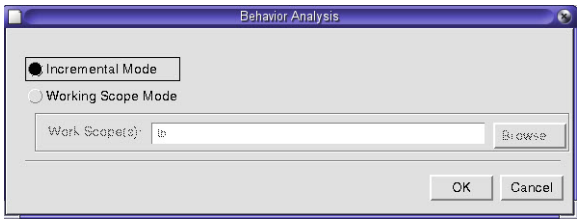
  When the file is saved as an XML format, the XML file can be loaded in the *Switching Analysis Report* form using the **File -> Load from XML file** command.

- **Save as CSV File**

  When the file is saved as a CSV format, the CSV file is readable and is used in the power estimation flow.

  After completing the configurations, click the **OK** button in the *Switching Analysis* form and click the **Yes** button in the *Question* dialog to open the *Behavior Analysis* form.

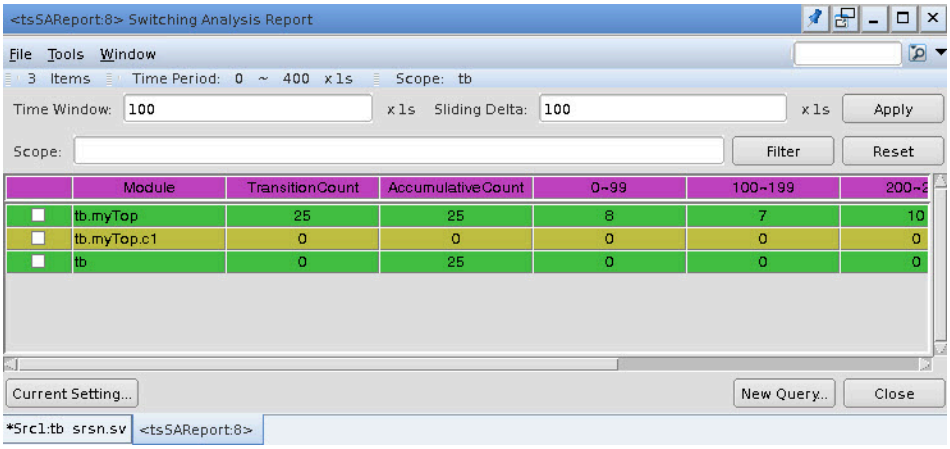




  Click the **OK** button to perform the behavior analysis and the scope-based peak analysis.

Note:
If the behavior analysis is performed in the current working directory, the dialog box is not displayed.

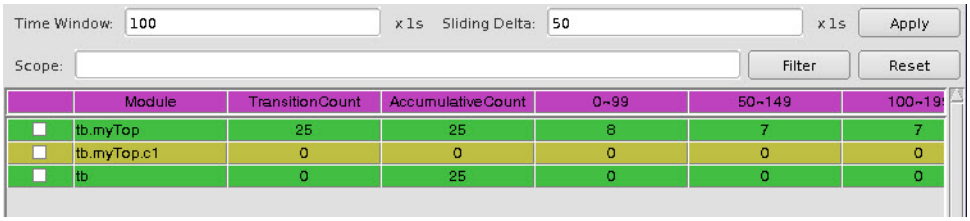
The analysis report is shown in the invoked *Switching Analysis Report* frame.



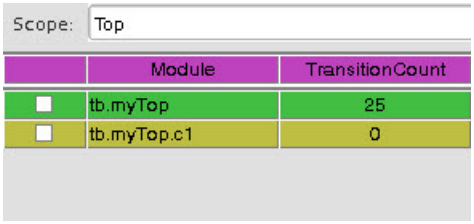
Feedback

You can see the transition counts and accumulative counts for the module in the **Module**, **Transition Count**, and **Accumulative Count** columns accordingly.

The time windows, for example, **0~99**, **100~199**, **200~299**, are displayed based on the values specified in the **Time Window Size** and **With Sliding Delta** fields of the *Switching Analysis* form. You can also change them in the *Switching Analysis Report* frame.

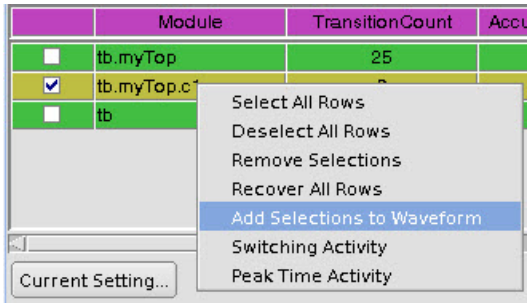


You can enter a scope name in the **Scope** filter and click the **Filter** button to select the scope..



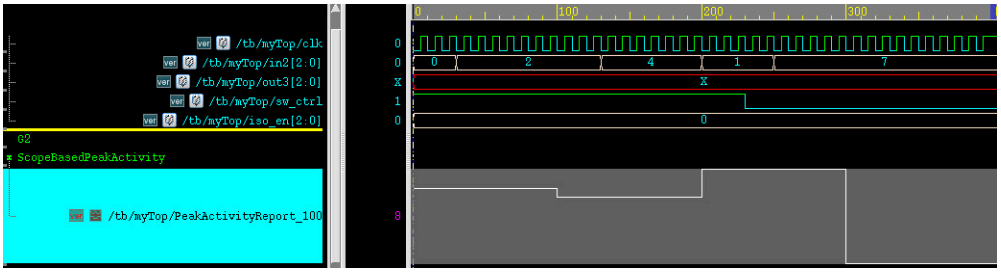
Generating Waveform With Report Entry

You can select a report entry and use the **Add Selections to Waveform** right-click command or the **Tools -> Add Selections to Waveform** menu command to add the selected entry into the *nWave* frame.



The selected entry is added in the *nWave* frame as a computed signal.

Feedback

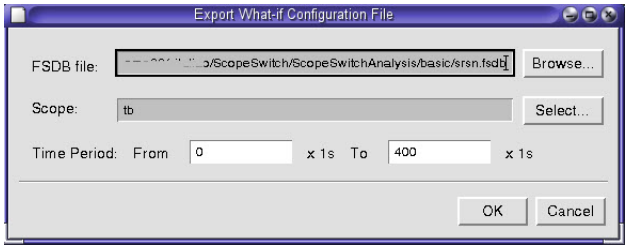


Exporting the What-if Configuration File

After checking the analysis report, you may want to perform the What-if correlation flow for power estimation with the specified scope and time duration. You can use the **File -> Export What-if Configuration File** command to generate the What-if configuration file and its related scripts that are needed in the What-if flow. The generated configuration file and script files make it easy for you to complete the configurations during the What-if flow.

To generate the configuration file and script files, perform the following steps:

1. In the *Switching Analysis Report* frame, use the **File -> Export What-if Configuration File** command to invoke the *Export What-if Configuration File* form.
2. Specify the FSDB file, scope, and time period in the corresponding fields.
3. Click the **OK** button.



The following configuration file and script files are generated accordingly:

- `wi_config_file`

  The configuration file for the What-if flow.

- `vcs_wi_compile.rc`

  The VCS compilation script used in the What-if flow.

- `vcs_wi_run.rc`

  The VCS simulation script used in the What-if flow.

- `ius_wi_compile.rc`

Feedback

The IUS compilation script used in the What-if flow

- `ius_wi_run.rc`

  The IUS simulation script used in the What-if flow

The following is the example of the configuration file and VCS script files:

```
//wi_config_file
      set FSDB = ./srsn.fsdb
      set Scope = tb
      set Map = <Required>
      set Begin_Time = 200
      set End_Time = 249
      set Time_Unit = 1s
      set Simulation_Compile_Script= ./vcs_wi_compile.rc
      set Simulation_Run_Script= ./vcs_wi_run.rc
      #For IUS user=====================================
      #set Simulation_Compile_Script= ./ius_wi_compile.rc
      #set Simulation_Run_Script= ./ius_wi_run.rc
//vcs_wi_compile.rc
      setenv VCS_HOME <cad tool path>/Synopsys/VCS_vE-2011.03-3
      setenv VERDI_HOME <Verdi_Install_path>
      setenv PATH ${VCS_HOME}/tools/bin:${VERDI_HOME}/bin:${PATH}
      alias vcs "\vcs -full64 -debug_access+all -debug_region=cell"
      vcs -sverilog -f wi_run.f
//vcs_wi_run.rc
      setenv VCS_HOME <cad tool path>/Synopsys/VCS_vE-2011.03-3
      setenv VCSI_HOME $VCS_HOME
      setenv LM_LICENSE_FILE 27005@sps403:$LM_LICENSE_FILE
      setenv VERDI_HOME <Verdi_install_path>
      alias vcs "\vcs -full64 -debug_access+all -debug_region=cell"
      ./simv
```